



Knowledge-Based Artificial Intelligence

A Cognitive Approach to Artificial Intelligence

Ashok Goel, David Joyner, and Steven Bryant

Knowledge-Based Artificial Intelligence

A Cognitive Approach to Artificial Intelligence

Knowledge-Based Artificial Intelligence

A Cognitive Approach to Artificial Intelligence

Ashok Goel, David Joyner, and Steven Bryant

Knowledge-Based Artificial Intelligence: A Cognitive Approach to Artificial Intelligence

By Ashok Goel, David Joyner, and Steven Bryant

© 2025 Georgia Tech Design Intelligence Laboratory

Independently Published on the Kindle Publishing Platform by:

Design Intelligence Laboratory – The KBAI Book

Georgia Institute of Technology

College of Computing

North Avenue

Atlanta, GA 30332

www.TheKBAIBook.com

This book may be purchased for educational, business, or sales promotional use. A PDF edition is distributed freely by the authors and is available at <https://TheKBAIBook.com>.

July 2025

First Edition

ISBN

xxx--xxxxxxx-x-x (Paperback Edition)

Revision History

06022025

Pre-Release DRAFT

Georgia Tech, Design Intelligence Laboratory, and Jill Watson are all trademark of the Georgia Institute of Technology. All other trademarks used in this book are trademarks of their respective organizations. All companies named in this book are fictitious unless otherwise stated.

The views expressed in this work are those of the authors, who performed a good faith effort to ensure that the information, instructions, and content created in this work are accurate. The authors, publisher, and the Georgia Institute of Technology disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use or reliance on this work. Use of the information and instructions contained in this work is at your own risk.

Contents

<u>CONTENTS</u>	<u>7</u>
------------------------	-----------------

<u>PREFACE</u>	<u>XVII</u>
-----------------------	--------------------

Bridging AI and Human Cognition: The Role of Computational Psychometrics	xix
How This Book Is Organized	xix
Who Should Read This Book	xx
About the Authors	xxi

<u>INTRODUCTION TO KNOWLEDGE-BASED ARTIFICIAL INTELLIGENCE</u>	<u>1</u>
---	-----------------

The Quest for Thinking Machines	1
Core Conundrums in Artificial Intelligence	2
Characteristics of AI Problems	3
Characteristics of AI Agents	4
AI Agents in Problem Solving: Illustrative Examples	6
What is Knowledge-Based AI?	8
The Four Schools of AI (Russell & Norvig Framework)	10
What are Cognitive Systems?	14
Overview of Topics in KBAI (The Eight Key Areas)	16

Book Structure and Personalization	23
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	23
Chapter Summary	25
Additional Readings & References	25

SEMANTIC NETWORKS **27**

Introduction to Semantic Networks	27
The Essence of Knowledge Representation	28
Semantic Networks in Action: Solving Raven's Progressive Matrices	29
Structure of Semantic Networks	33
Characteristics of Good Representations	34
Semantic Networks for Dynamic Problems: The Guards and Prisoners Problem	36
Revisiting Raven's Analogy Problem with Semantic Networks	45
Semantic Networks and Cognition	52
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	52
Chapter Summary	53
Additional Readings & References	54

GENERATE AND TEST **55**

Introduction to Generate and Test	55
the Generate and Test Method in Problem-Solving	56
Enhancing Generate and Test	63
Generate and Test for Raven's Problems	67
Cognitive Connection	69
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	69
Chapter Summary	70
Additional Readings & References	71

MEANS-ENDS ANALYSIS AND PROBLEM REDUCTION **73**

Introduction	73
The Blocks World Problem: A Classic AI Challenge	74
State Spaces	76
Means–Ends Analysis (MEA)	80
Limitations of MEA: Local Optima and Getting Stuck	82
Problem Reduction	88
MEA and Problem Reduction as “Weak Methods”	97
Cognitive Connection	98

Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	98
Chapter Summary	99
Additional Readings & References	100

PRODUCTION SYSTEMS **101**

Introduction: The Pitcher's Dilemma	101
Cognitive Architectures	103
Soar: An Exemplar Cognitive Architecture for Production Systems	108
Production Systems Explained	109
Learning in Production Systems: The Chunking Process	116
Production Systems and Cognition	120
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	120
Summary	122
Additional Readings & References	122

FRAMES **125**

Introduction to Frames	125
The Function of Frames	127
Characteristics of Frames	131
Frames and Other KBAI Topics	134
Frames and Cognition	142
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	142
Chapter Summary	143
Additional Readings & References	144

LEARNING BY RECORDING CASES **147**

Introduction to Learning by Recording Cases	147
Applying Learning by Recording Cases	149
Retrieval using the Nearest Neighbor Method	151
K-Nearest Neighbors (KNN)	156
Nearest Neighbors and Cognition	158
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	158
Chapter Summary	160
Additional Readings & References	160

CASE BASED REASONING **163**

Introduction to Case-Based Reasoning (CBR)	163
Case-Based Reasoning Core Processes	165
Key Assumptions of CBR	167
Adaptation Strategies in CBR	168
Evaluation and Storage in CBR	174
Advanced Case Retrieval	185
Advanced Case-Based Reasoning Considerations	187
Case Based Reasoning and Cognition	188
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	189
Chapter Summary	190
Additional Readings & References	191

INCREMENTAL CONCEPT LEARNING **193**

Introduction to Incremental Concept Learning	193
Core Concepts of Incremental Concept Learning: The “Foo” Example	194
The Role of Background Knowledge	196
The Incremental Concept Learning Algorithm	199
Key Processes: Variabilization, Generalization, and Specialization	201
An Example Alternative Representation (Iterative Refinement)	214
Heuristics for Concept Learning (Summary of Heuristics)	216
Incremental Concept Learning and Cognition	225
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	226
Chapter Summary	228
Additional Readings & References	229

CLASSIFICATION **231**

Introduction to Classification	231
Revisiting Concept Learning: The Foundation for Classification	232
Defining Classification: The Core Task	233
Equivalence Classes: Grouping Percepts	236
Structuring Knowledge: Conceptual Hierarchies	238
A Spectrum of Concepts: From Axiomatic to Exemplar	242
Classification Strategies: From Periodic Tables of the Mind to Algorithmic Approaches	248
The Cognitive Connection: Classification in Human Thought	250
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	251
Chapter Summary	252
Additional Readings & References	253

LOGIC **255**

Introduction to Logic	255
Building Blocks of Propositional Logic: The Role of Logic in AI Agents	256
Truth Tables: Evaluating Logical Expressions	265
Properties of Logical Operations	269
Rules of Implication and Inference	276
Rules of Inference: Modus Ponens and Modus Tollens	279
Challenges of Traditional Logical Inference	281
Predicate Calculus: Extending Logic with Variables and Quantifiers	283
Resolution Theorem Proving: An Efficient Method for Automated Reasoning	284
Logic and Cognition	292
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	293
Chapter Summary	294
Additional Readings & References	295

PLANNING **297**

Introduction to Planning	297
The Nature of Planning and Goal Prioritization	298
Representing Planning Problems in Logic	300
Planning Approaches and Challenges	307
Partial-Order Planning (POP)	311
Hierarchical Task Network (HTN) Planning	325
PLanning and Cognition	329
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	330
Chapter Summary	331
Additional Readings & References	332

UNDERSTANDING **335**

Introduction to Understanding	335
The Challenge of Story Understanding in AI	336
The General Process of Understanding	337
Frames and Semantic Roles in Understanding	338
Understanding and Cognition	354
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	355
Chapter Summary	356
Additional Readings	357
References	357

COMMONSENSE REASONING **359**

Introduction to Commonsense Reasoning	359
Understanding Stories: The Foundation of Commonsense	360
Building AI Agents with Primitive Actions	362
Thematic Roles and Action Frames	366
Implied Actions and Commonsense Reasoning	371
Sub-Actions and Commonsense Reasoning	373
State Changes in Commonsense Reasoning	376
Commonsense Reasoning and Cognition	383
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	384
Chapter Summary	385
Additional Readings & References	386

SCRIPTS **389**

Introduction to Scripts	389
Understanding Stories and the Need for Scripts	390
Scripts: Structured Knowledge for Sequences of Events	391
Defining Script Components	392
Expectation Generation and Violation	397
Tracks: Variations within Scripts	398
Script Selection and Invocation in AI Agents	398
Scripts and Cognition	402
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	403
Chapter Summary	404
Additional Readings & References	405

EXPLANATION-BASED LEARNING **407**

Introduction to Explanation-Based Learning	407
Understanding EBL Through a Practical Scenario	408
The Nature of Incremental Learning in EBL	410
The Role of Prior Knowledge in EBL	412
Abstraction: Generalizing from Specifics	417
Knowledge Transfer and Causal Explanations	418
Advancing with Explanation-Based Learning	421
Practical Exercises in EBL	422
The Power of Exploration-Based Learning	425
Explanation-Based Learning and Cognition	426
Integrating KBAI: Enhancing Modern AI with Knowledge-Based Approaches	427

Chapter Summary	429
Additional Readings & References	430

ANALOGICAL REASONING **431**

Introduction to Analogical Reasoning	431
Understanding Similarity in Analogical Reasoning	432
Indexing Schemes for Case Retrieval	434
Cross-Domain Analogies	436
A Hierarchical Process for Analogical Reasoning	442
Degrees of Similarity: Superficial vs. Deep	443
Types of Similarities: Semantic, Pragmatic, and Structural	445
Understanding Analogical Mapping: The Correspondence Problem	449
Exercise: Mapping Deep Relationships - Solar System and Atomic Structure	451
The Role of Structural Similarity	455
Analogical Design in Robotics: The Challenge of Walking on Water	458
Integrative Design using Compound Analogies	460
Advanced Open Issues in Analogy Research	462
Analogical Reasoning and Human Cognition	463
Integrating KBAI: Analogical Reasoning in the Landscape of Modern AI	464
Chapter Summary	466
Additional Readings & References	466

GENERALIZATION AND LEARNING WITH VERSION SPACES **469**

Introduction to Generalization and Learning with Version Spaces	469
Version Spaces and Incremental Concept Learning	470
Visualizing the Version Space Learning Process	473
Comparing Incremental Concept Learning and Version Spaces: A Visual Explanation	478
Illustrative Example: Learning Allergic Reaction Conditions with Version Spaces	484
The Principle of Convergence in Version Spaces	489
Interactive Example: Learning Allergic Reactions with Version Spaces (Extended)	493
Alternative Approach: Decision Tree Learning for Classification	499
Version Spaces and Cognition	504
Integrating KBAI: Generalization and Version Spaces in Modern AI	506
Chapter Summary	507
Additional Readings & References	508

CONSTRAINT PROPAGATION **509**

Introduction to Constraint Propagation	509
Understanding 3D Objects through Constraint Propagation	510
Constraint Propagation: Definition and Characteristics	514
Understanding Constraint Propagation in Computer Vision: Marr's Framework	517
Line Labeling and Junction Constraints in Visual Perception	520
Exercise: Identifying Junction Types in a Cube	524
Exploring Grammar Through Constraint Propagation	529
Constraint Propagation and Cognition	530
Integrating KBAI: Constraint Propagation in the Landscape of Modern AI	531
Chapter Summary	533
Additional Readings & References	534

CONFIGURATION **535**

Introduction to Configuration	535
Understanding Design	536
Exploring Configuration Through Practical Application: Basement Design	536
Exploring Configuration as a Design Process	539
The Configuration Process: From Specifications to Arrangement Model	540
Detailed Exploration of Chair Configuration	543
Integration with Configuration and other KBAI Techniques	550
Configuration and Cognition	555
Integrating KBAI: Configuration in the Modern AI Landscape	556
Chapter Summary	558
Additional Readings & References	559

DIAGNOSIS **561**

Introduction to Diagnosis	561
Introduction to the Concept of Diagnosis Through a Medical Exercise	562
The Process of Diagnosis Across Domains	564
Mapping Between Data Space and Hypothesis Space in Diagnosis	565
Effective Data Abstraction in Diagnosis	567
The Intricacies of Diagnostic Classification in AI	568
Exploring Abduction within Deductive and Inductive Reasoning Frameworks	569
Applying Abduction in Diagnostic Hypothesis Selection	571
Mapping Diagnosis to Treatment: Completing the Diagnostic Journey	577
Diagnosis and Cognition	577
Integrating KBAI: Diagnosis in the Modern AI Ecosystem	578

Chapter Summary	580
Additional Readings & References	580

LEARNING BY CORRECTING MISTAKES **583**

Introduction to Learning by Correcting Mistakes	583
Learning by Correcting Mistakes through Practical Application: The Robot and the Cup	584
Addressing Learning from Mistakes Through Critical Questions	587
Visualizing Error Detection: Using Venn Diagrams	590
Algorithm for Isolating Mistakes: Identifying True and False Suspicious Features	592
Refinement Through Rule Revision: Evolving the Concept of a “Cup”	594
Integrating Knowledge with Action: Cognitive Architecture in Learning from Mistakes	600
Learning By Correcting Mistakes and Cognition	602
Integrating KBAI: Learning by Correcting Mistakes in Modern AI	602
Chapter Summary	604
Additional Readings & References	605

META-REASONING **607**

Introduction to Meta-Reasoning	607
Addressing Knowledge Errors in Meta-Reasoning	608
Addressing Knowledge Gaps in Meta-Reasoning	611
Blurring the Lines Between Metacognition and Deliberation	614
Strategy Selection and Metacognition	616
Strategy Selection in Learning	618
Strategy Integration in MetaCognition	619
The Inner Workings of Meta-Reasoning: Applying Strategies to Itself	621
An Example of Meta-Reasoning in Action: The Robot and the Camera	623
Connections: Metacognition Across KBAI Topics	624
The Metacognition Experience in This Book	628
Metacognition and Human Cognition	629
Integrating KBAI: Meta-Reasoning in the Modern AI Landscape	630
Chapter Summary	631
Additional Readings & References	632

GLOSSARY **635**

RECOMMENDED READINGS **669**

Classical Artificial Intelligence	669
-----------------------------------	-----

Cognitive Science	671
Neuroscience	672

APPENDIX I – RAVEN’S PROGRESSIVE MATRICES **675**

Introduction to Raven’s Progressive Matrices (RPM)	675
Milestone 1: Defining the Problem Representation	678
Milestone 2: Implementing Reasoning Strategies	679
Milestone 3: Meta-Reasoning and Error Analysis	680
Milestone 4: Evaluation and Final Reflection	681
Conclusion	682
Recommended Reading	683
Additional Readings & References	683

INDEX **685**

Preface

Welcome to *Knowledge-Based Artificial Intelligence (KBAI): A Cognitive Approach to Artificial Intelligence*. This book invites you to explore the fascinating intersection where the foundational principles of Artificial Intelligence converge with the intricate workings of human cognition. Our examination is firmly grounded in the classical tradition of AI – a world where the structured representation of knowledge and the power of logical reasoning form the bedrock of how machines can learn to understand and interact with the world in ways that resonate with human thought.

At the heart of KBAI lies a compelling aspiration: to align artificial cognition more closely with human thinking processes. This ambition is not merely an academic pursuit; it is a quest to craft AI systems that are more interpretable, relatable, and ultimately, more effective in their collaboration with humans. By examining the mechanisms through which humans reason, learn, solve problems, and make decisions, we aim to illuminate pathways for endowing AI systems with similar capabilities. The goal is to enhance their utility and foster a more seamless integration into the fabric of our daily lives.

While KBAI shares common ground with the broader field of AI, including influential areas covered by modern Machine Learning (ML), this volume focuses specifically on the principles, architectures, and techniques emblematic of the classical, knowledge-based approach. These include the explicit definition and organization of knowledge, the systematic use of logical inference mechanisms, and the representation of knowledge in symbolic forms. Such elements are essential for creating AI systems capable of reasoning about complex problems, articulating

their decision-making processes, and dynamically adapting their knowledge to new and evolving situations.

It is important to note that while this book centers on KBAI, the principles and concepts discussed are increasingly relevant and synergistic with today's evolving AI landscape. The lines between classical AI and ML are becoming more fluid, and the integration of structured, symbolic reasoning with data-driven statistical learning represents a promising and vibrant frontier in AI research. This synergy holds the potential to yield AI systems that are not only more powerful and versatile but also more akin to the nuanced and adaptable nature of human thought.

As we navigate through the chapters of this book, we will journey through diverse knowledge representation techniques, explore powerful reasoning strategies, and examine cognitive architectures that underpin the KBAI paradigm. From the structured elegance of frames and semantic networks to the dynamic rule-based operations of production systems and beyond, we will uncover the intricate structures and mechanisms that empower AI systems to store knowledge, reason about it, and apply it adeptly to solve a myriad of problems.

This book's origins trace back to the Knowledge-Based Artificial Intelligence course at the Georgia Institute of Technology, a cornerstone of both undergraduate and graduate computer science curricula. As a pioneer in educating at scale, Georgia Tech delivered much of this course content to its "online campus" Master of Science students via video lectures. The rich repository of these video lessons, along with comprehensive transcripts, formed the initial academic content for this volume. In crafting this book, we harnessed the innovative power of generative AI, which played a crucial role as an assistant transcriber, writer, and editor, transforming the dynamic and interactive classroom material into the polished narrative you find here. This endeavor was closely supervised by the authors, ensuring that the AI's output remained steadfastly true to the original educational content and pedagogical intent. Strict boundaries were established for the generative AI, limiting its role to enhancing the clarity and flow of the given material without deviating from established concepts or introducing new ideas not already present in the transcripts or explicitly provided by the authors. This methodical and human-supervised approach guaranteed that the essence and intellectual rigor of the original course were preserved, while leveraging AI's capabilities to improve engagement and accessibility for a diverse readership.

Our use of AI, however, did not conclude with the book's completion. The content of this book has been integrated into Jill Watson, an AI-based teaching assistant also pioneered at Georgia Tech. Today, students can engage with the material in multiple ways: they can read the book in the traditional manner, or they can interact with Jill Watson, allowing them to explore concepts, ask questions, and consume knowledge in novel, conversational ways. We invite you to experience this innovative learning approach firsthand by visiting the book's website at TheKBAIBook.com. Through these efforts, we aim to extend the reach of this valuable learning resource, making it accessible and engaging for a broader audience keen to explore the intricacies and profound potential of Knowledge-Based Artificial Intelligence.

BRIDGING AI AND HUMAN COGNITION: THE ROLE OF COMPUTATIONAL PSYCHOMETRICS

Before we outline the organization of this book, it is important to introduce a concept that underpins much of our exploration: Computational Psychometrics. At its core, psychometrics is the field dedicated to the theory and technique of psychological measurement, involving the study of human intelligence, aptitudes, and knowledge, often employing various standardized tests. Computational Psychometrics innovatively extends this field by tasking computational agents – AI systems – with undertaking the same kinds of tests designed to assess human intelligence and cognitive abilities.

Throughout this book, we will touch upon the creation of AI agents that can engage with established intelligence tests, such as Raven's Progressive Matrices, in a manner comparable to humans. This endeavor is not merely about benchmarking AI performance against human standards; it is a deeper scientific inquiry into understanding the parallels and divergences in error patterns, reasoning processes, and problem-solving strategies between humans and AI. By meticulously analyzing how these AI agents perform – observing their successes, their failures, and the nature of their errors in comparison to human participants – we gain invaluable insights into the potential similarities in their reasoning pathways. Such comparative analysis not only deepens our comprehension of AI's capabilities and limitations but also offers a unique and reflective lens through which we can study and understand human cognitive processes.

As you journey through the chapters, we encourage you to keep in mind this dual focus: designing AI agents capable of intelligent behavior, and concurrently, using their performance and processes as a means to reflect on the complexities of human intelligence. This approach underscores the profound and interconnected relationship between Knowledge-Based AI and cognitive science, enriching our exploration and appreciation of both fields.

HOW THIS BOOK IS ORGANIZED

This book is structured to guide you through the landscape of Knowledge-Based AI, focusing on four key learning goals:

- **Introduce the Core Methods of Knowledge-Based AI:** Readers will gain a foundational understanding of the principal techniques, methodologies, and architectural approaches that define KBAI, setting the stage for deeper exploration and application.
- **Understand Common Tasks Addressed by Knowledge-Based AI:** The book elucidates a variety of tasks and complex challenges that KBAI is particularly well-suited to tackle, thereby illustrating the versatility, applicability, and power of these methods.

- **Learn Ways AI Agents Can Use KBAI Methods to Solve Common Tasks:** Through practical examples, illustrative case studies, and thoughtful exercises, readers will learn how AI agents can effectively employ KBAI strategies to navigate and resolve common, real-world problems.
- **Understand the Relationship Between KBAI and Cognitive Science:** The text endeavors to draw explicit connections between the mechanisms of KBAI and the established principles of human cognition, offering insights into how AI can both mimic human thought and inform our understanding of cognitive processes.

To achieve these goals, this book covers Knowledge-Based AI through eight key thematic areas. A detailed mapping of how each chapter aligns with these eight key areas – **Fundamentals, Planning, Common Sense Reasoning, Learning, Analogical Reasoning, Visuospatial Reasoning, Design and Creativity, and Metacognition** – is provided in the introductory chapter (Chapter 1) to help you navigate the content. This structure is designed to organize the material logically and make it easier to see how each part contributes to a comprehensive understanding of the broader field of Knowledge-Based AI.

This work, while drawing partial inspiration from foundational texts such as Patrick Winston's "Artificial Intelligence," charts its own distinct course by placing a unique and consistent emphasis on the intersection of AI and human cognition. While readers familiar with classical AI literature may find echoes of Winston's seminal concepts within these pages, our approach diverges significantly in its dedicated pursuit of aligning artificial intelligence more closely with the rich and often nuanced mechanisms and processes of human thought. This distinctive focus on the cognitive underpinnings of AI not only differentiates this work from others but also aims to deepen our collective understanding of artificial intelligence through the illuminating lens of human cognitive processes.

WHO SHOULD READ THIS BOOK

This book is designed for a diverse audience, including students, researchers, and practitioners who are intrigued by the potential of AI systems that can mirror and even extend the complexity and subtlety of human intelligence. It is suitable for newcomers to the field of Artificial Intelligence seeking a solid grounding in its classical underpinnings, as well as for experienced individuals aiming to enrich their knowledge of knowledge-based approaches and their cognitive relevance. We believe it offers a thorough, engaging, and accessible exploration of Knowledge-Based Artificial Intelligence.

By the conclusion of this book, readers should be equipped with the understanding necessary to design and conceptualize Knowledge-Based AI agents, apply strategic KBAI thinking to tackle practical issues, and reflect critically on the interplay between AI solutions and human cognitive processes. We hope this journey will enrich your comprehension of both artificial intelligence and cognitive science, fostering a deeper appreciation for the quest to create truly intelligent systems.

ABOUT THE AUTHORS

Ashok Goel

Ashok Goel is a professor of Computer Science and Human-Centered Computing in the School of Interactive Computing at Georgia Institute of Technology, and a Fellow of the Cognitive Science Society. He is also the Chief Scientist and a founding member of the Center for Biologically Inspired Design at Georgia Tech, which is dedicated to innovation at the intersection of biology and engineering. Dr. Goel is the editor of *AI Magazine* and an associate editor of the *IEEE Intelligent Systems* journal. His research focuses on artificial intelligence, cognitive science, and human-centered computing, with a particular emphasis on biologically inspired and cognitive systems. He is a recipient of AAAI's Outstanding AI Educator Award.

David Joyner

David Joyner is the Executive Director of Online Education & OMSCS in Georgia Tech's College of Computing. He is a leading voice in the development and delivery of at-scale online learning, particularly within the context of computer science education. Dr. Joyner has extensive experience in creating engaging and effective learning environments for tens of thousands of students globally. His work often explores the intersection of technology, pedagogy, and accessibility in education. He is a co-author of multiple books and a frequent speaker on the future of online learning and MOOCs.

Steven Bryant

Steven Bryant explores the strategic implementation and cognitive dimensions of Artificial Intelligence in complex systems, drawing on both academic research and extensive practical application. A faculty member in Georgia Tech's College of Computing, where he also earned his MS in Computer Science (AI), his work is informed by three decades spearheading transformative AI initiatives in global organizations. As a founder of Bryant Research, he continues to conduct research into human-AI collaboration, especially at its intersection with business, and investigates the evolving AI landscape. Steven also holds an MBA from the University of San Diego and is co-author of *The Agentic Enterprise: A Leader's Guide to Navigating the AI Revolution*, contributing to AI strategy and its real-world impact.

Chapter 1

Introduction to Knowledge-Based Artificial Intelligence

THE QUEST FOR THINKING MACHINES

What if we could build machines that not only compute but *think*? Machines that learn, reason, and even create, perhaps mirroring the complex cognitive tapestry of the human mind? This isn't just the stuff of science fiction; it's the driving ambition behind Artificial Intelligence (AI). Within the vast landscape of AI, one particular approach seeks to build intelligent systems by explicitly representing and manipulating knowledge, much like we humans do. Welcome to the world of Knowledge-Based Artificial Intelligence (KBAI).

This book invites you on an exploration into KBAI, drawing upon decades of research and teaching experience from the Georgia Institute of Technology. Guiding you will be Professor Ashok Goel, whose 30 years in AI research, particularly focusing on computational creativity and cognitive systems, bring a deep, nuanced perspective. Joining him is Dr. David Joyner, an expert in online education and harnessing technology to make complex topics accessible, ensuring this journey is as engaging as it is informative. Together, through a blend of explanation and conversation, we'll navigate the core principles of KBAI, placing it within the broader context of AI's different schools of thought and exploring the fascinating concept of “Cognitive Systems” – AI designed with insights from human cognition.¹⁻⁴ This isn't just a theoretical

exercise; it's about understanding how we can build AI that reasons, learns, and solves problems in ways that are both powerful and interpretable.

CORE CONUNDRUMS IN ARTIFICIAL INTELLIGENCE

Before diving into the “how” of KBAI, let's ponder some of the “whys” and “what ifs” – the fundamental challenges, or conundrums, that anyone attempting to build truly intelligent systems must confront. These aren't simple roadblocks; they are deep puzzles about the nature of intelligence itself. Professor Goel often refers to these as the “Grand Challenges” of AI.⁵⁻⁷

- **Conundrum 1: The Resource Riddle (Limited Resources vs. Computational Intractability):** Intelligent agents, whether digital or biological, operate with finite resources – limited processing power, memory, and time. Yet, the problems we want them to solve are often computationally immense. How can AI deliver near real-time insights on complex challenges with these inherent limitations?
- **Conundrum 2: Local Actions, Global Problems (Local Computation vs. Global Constraints):** Computation is inherently local, step-by-step. But many problems demand globally optimal solutions or adherence to global constraints. How can systems working with local information effectively tackle problems with global scope?
- **Conundrum 3: The Logic Loophole (Deductive Logic vs. Abductive/Inductive Problems):** Formal logic, the bedrock of computation, is primarily deductive (if A, then B). But much of real-world reasoning is not deductive. We often reason inductively (finding patterns from examples) or abductively (finding the best explanation for observations). How can AI systems designed around deductive logic handle problems demanding these other forms of reasoning? (We'll unpack terms like abduction later!)
- **Conundrum 4: The Dynamic World Dilemma (Dynamic World vs. Limited/Static Knowledge):** The world is constantly changing, yet the knowledge an AI starts with is necessarily finite and potentially static. How can an agent use its limited knowledge to effectively address novel situations and problems it hasn't encountered before?
- **Conundrum 5: The Explanation Enigma (Complexity of Problem Solving vs. Even Greater Complexity of Explanation/Justification):** Solving a problem is complex, but explaining how you solved it, justifying your reasoning, and learning from the process adds layers of difficulty. How can we build AI that not only finds answers but can also explain its reasoning, fostering trust and understanding?

Fundamental Conundrums of Artificial Intelligence

- *Intelligent Agents have limited resources*
- *Computation is local, but problems have global constraints*
- *Logic is deductive, but many problems are not*
- *The world is dynamic, but knowledge is limited*
- *Problem solving, reasoning, and learning are complex, but explanation and justification are even more complex*

Table 1-1 Fundamental Conundrums of AI

These conundrums frame our exploration. KBAI offers specific strategies and perspectives for tackling these grand challenges, often by drawing inspiration from the one intelligent system we know intimately: the human mind. The discussion of these core conundrums offers an in-depth look into these key challenges, making it essential for those who want to understand the detailed complexities and finer points of AI problem-solving and reasoning.⁸⁻¹¹ It helps build a clearer understanding of the limitations and challenges in the field, while also considering possible strategies to tackle these issues. This discussion is vital for anyone looking to gain a well-rounded understanding of the practical aspects of artificial intelligence.

CHARACTERISTICS OF AI PROBLEMS

The conundrums highlight theoretical limits, but what about the practical nature of the problems AI agents face? Understanding the typical characteristics of these problems shapes how we design AI solutions. We now aim to demystify the nature of AI problems and bring the discussion to a practical level. This exploration is crucial for understanding how AI agents are designed and how they operate in real-world scenarios.

- **Knowledge Arrives Incrementally:** Data rarely comes all at once; it often streams in over time. AI systems must learn and adapt continuously as new information becomes available.^{12,13}
- **Patterns Repeat:** Many problems aren't entirely unique; they exhibit recurring patterns or structures. Recognizing and leveraging these patterns is key to efficient problem-solving.
- **Multiple Levels of Detail/Abstraction:** Problems can often be viewed at different levels of granularity or abstraction. AI needs the flexibility to reason across these levels.
- **Computational Hurdles/Intractability:** Many interesting AI problems are computationally intractable, meaning finding a perfect solution might take an

impractical amount of time or resources. We often need strategies that find “good enough” solutions efficiently.¹⁴

- **Dynamic World vs. Static Knowledge:** The world changes, but an AI's knowledge base might lag behind. Systems need mechanisms to update their understanding to remain relevant.
- **Open-Ended World vs. Limited Knowledge:** The real world is open-ended, full of possibilities beyond an AI's programmed knowledge. Agents must function effectively despite incomplete information.

Characteristics of AI Problems

- *Knowledge often arrives incrementally*
- *Problems exhibit recurring patterns*
- *Problems have multiple levels of granularity*
- *Many problems are computationally intractable*
- *The world is dynamic, but knowledge of the world is static*
- *The world is open-ended, but knowledge is limited*

Table 1-2 Characteristics of AI Problems

To address these problems, the question then becomes: How can we design AI agents capable of addressing problems with these characteristics? These issues form the core challenges that will be discussed in this book. Understanding these characteristics is not just an academic exercise but a practical necessity, as it directly influences the development and effectiveness of AI agents in real-world applications. This section sets the stage for a deeper exploration of how AI can be tailored to meet these diverse and complex challenges.

CHARACTERISTICS OF AI AGENTS

Just as AI problems have specific traits, so too do the AI agents designed to solve them. Recognizing their inherent properties and limitations is crucial. In artificial intelligence, the conversation often revolves around the complexities of AI problems. However, an equally important aspect to consider is the properties of AI agents themselves. AI agents are not just passive recipients of programming; they possess distinct characteristics that define their capabilities and limitations in interacting with the world.¹⁵ Let's examine some key properties of AI agents that shape their functionality and effectiveness:

- **Bounded Rationality:** Agents operate with limited computing power, memory, sensory input, and attention. They cannot process everything simultaneously or perfectly.
 - **Limited Computing Power:** AI agents are equipped with limited computing resources. This includes constraints on processing speed, computing power, and memory size, which inherently restrict their operational capacity.
 - **Limited Sensors:** The sensory perception of AI agents is limited. They do not have the capability to perceive every aspect of the world, leading to a partial and sometimes skewed understanding of their environment.
 - **Limited Attention:** AI agents possess limited attention. They cannot simultaneously focus on every element in their environment, necessitating a selective approach to processing information.
- **Logic's Foundation:** Their underlying computational logic is fundamentally deductive.¹⁶
- **Incomplete Knowledge:** An AI's knowledge is always incomplete relative to the vastness and complexity of the real world.

Characteristics of AI Agents

- *Agents have limited computing power*
- *Agents have limited sensors*
- *Agents have limited attention*
- *Computational logic is fundamentally deductive*
- *AI agents' knowledge is incomplete relative to the world*

Table 1-3 Characteristics of AI Agents

These characteristics collectively raise a critical question: how can AI agents, with such bounded rationality and constraints, effectively address open-ended problems in the real world? KBAI seeks answers through clever knowledge representation and reasoning strategies. Understanding these properties is crucial for advancing the field of AI, as it allows us to develop more sophisticated and capable AI systems that can navigate the complexities of their environments more effectively. This discussion lays the groundwork for exploring the potential and the limitations of AI agents in a comprehensive manner.

AI AGENTS IN PROBLEM SOLVING: ILLUSTRATIVE EXAMPLES

Given these challenges and agent limitations, what kinds of tasks are suitable for AI, particularly KBAI? Let's consider a few diverse examples:

- Answering trivia questions on Jeopardy!? – Requires vast knowledge and language processing.
- Configuring basement dimensions for a house? – Needs spatial reasoning and constraint satisfaction.
- Tying shoelaces? – Demands fine motor control and adaptability, a challenge for current robotics.
- Planning a route to a new place? – Relies on data analysis and optimization.
- Making sense of a news broadcast? – Involves advanced language understanding and context.
- Designing a robot that walks on water? – A complex engineering problem integrating physics and design principles.
- Deciding if a flowerpot can be a drinking cup? – Requires understanding material properties and function – a commonsense reasoning task.
- Classifying a new animal as a bird? – A classic pattern recognition and classification problem.

Ashok David, which of these strike you as AI problems?

David Honestly, Ashok, I think all of them are AI problems. These are all tasks humans perform. If AI aims to replicate or augment human intelligence, it should ideally be capable of tackling any of these.

Ashok I agree completely. And indeed, throughout this book, we'll touch upon AI approaches relevant to each of these tasks. For now, let's zoom in on that first one: answering Jeopardy! questions.

Let's start by looking at an example of an AI agent in action. Many of you are familiar with Watson, the IBM program that plays Jeopardy. Some of you may not be, and that's fine; we'll show an example in a minute. When you watch Watson in action, try to think about the following: What are some of the things Watson must know? What are some of the things that Watson must be able to reason about to play Jeopardy? Write them down.

When you watch Watson in action, try to think about the following: What are some of the things Watson must know? What are some of the things that Watson must be able to reason about to play Jeopardy? Write them down.



A Jeopardy video clip featuring Watson can be seen here
(<http://tinyurl.com/kbai-ch1-IBMWatson>).

A transcript of the conversation follows:

Watson Olympic Oddities, for 200.

Alex *Milorad Cavic almost upset this man's perfect 2008 Olympics, losing to him by one-hundredth of a second.* Watson.

Watson Who is Michael Phelps.

Alex Yes, go.

Watson Name the decade for 200.

Alex *Disneyland opens, and the peace symbol is created.* Ken.

Ken What are the 50s?

Alex Yes.

Ken Final Frontiers for 1,000, Alex.

Alex *Tickets aren't needed for this event, a black hole's boundary from which matter cannot escape.*
Watson.

Watson What is Event Horizon?

AI in Practice: Watson

Ashok What are some things Watson must be able to do to participate in Jeopardy? David, what did you write down?

David So, I said that the four fundamental things that Watson must be able to do to play Jeopardy are: first, read the clue; then, search through its knowledge base; next, decide on its answer; and finally, phrase its answer in the form of a question.

Ashok That's right. And throughout this book, we'll discuss each part of David's answer.

WHAT IS KNOWLEDGE-BASED AI?

In this insightful section, we look into the core processes that define knowledge-based AI agents. By examining these processes, we gain a clearer understanding of how these agents function and interact with their environment. These processes are fundamental elements, each playing a critical role in the overall architecture of AI agents.

The Core Triad: Reasoning, Learning, and Memory

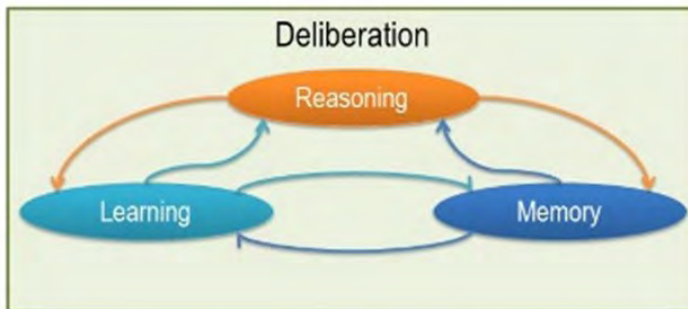


Figure 1-1 Three Fundamentals of Knowledge-Based AI

- **Reasoning: The ability of AI agents to process and manipulate information to form judgements or decisions.** Reasoning stands as a cornerstone in knowledge-based AI. It encompasses a range of activities, including understanding natural language, decision-making, and even generating responses. This process is broadly defined as the ability of AI agents to interpret and manipulate information to reach conclusions or make decisions.
- **Learning: The capacity of AI agents to adapt and evolve based on experiences.** Learning is another key process in AI. It involves the agent's ability to adapt based on experience. As exemplified by Watson, an AI agent learns from both successes and failures, storing information to refine its future responses and decisions.¹⁷

- **Memory: The mechanism through which AI agents store and retrieve learned information.** Memory is critical for learning and reasoning. It serves as the repository where knowledge is stored and accessed. Memory processes not only store what an AI agent learns but also provide the necessary data for reasoning tasks.

These three foundational processes – learning, memory, and reasoning – are intricately interlinked, forming the bedrock of knowledge-based AI. The process of learning is instrumental for effective reasoning; as AI agents reason, they often encounter new insights, which in turn fosters further learning. This acquired knowledge is then stored in memory, forming a continuous cycle of cognitive growth and development. However, it's important to recognize that the initiation of this cycle relies on existing knowledge, highlighting the interdependence of these fundamental processes in AI.

Deliberation: The Underlying Concept

Deliberation is a key element in the study of knowledge-based AI.¹⁸ The more we know, the more we can learn. Reasoning requires knowledge that memory can provide access to. The results of reasoning can also go into memory. Deliberation is a unifying concept that intertwines the three fundamental processes of reasoning, learning, and memory. This integration is pivotal to the functioning of AI agents, moving beyond the traditional approach of examining these elements in isolation.

The deliberation process begins with the foundational premise that the expansion of knowledge enhances the capacity for further learning. Reasoning, a critical component of this process, relies heavily on the knowledge stored in memory. As reasoning occurs, it not only utilizes this stored knowledge but also contributes new insights back into the memory, creating a dynamic, continuous cycle of cognitive growth. This deliberation process is one part of the overall architecture of a knowledge-based AI agent.

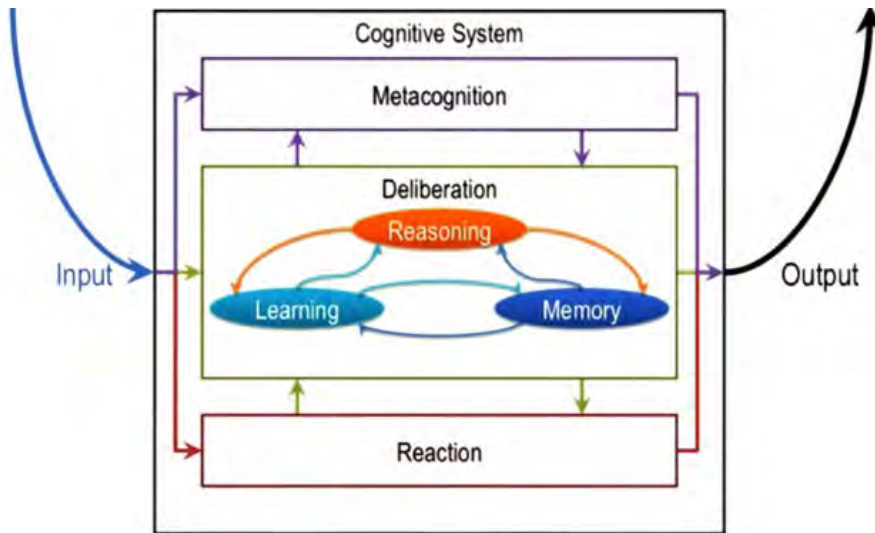


Figure 1-2 Overall architecture of a Knowledge-Based AI agent showing Deliberation, Metacognition, Reaction

Throughout this book, a focus is placed on theories of knowledge-based AI that do not isolate reasoning, learning, and memory as separate entities. Instead, these processes are viewed as interconnected aspects of a singular, cohesive process – deliberation. This approach contrasts with certain schools of AI that tend to compartmentalize these functions. Deliberation is a crucial part of the overall architecture of a knowledge-based AI agent. This architecture is not just about processing inputs (in the form of world perceptions) and generating outputs (as actions in the world). Instead, it involves a complex array of processes that map these perceptions to actions. While the immediate focus is on deliberation, the architecture also encompasses other critical aspects like metacognition and reaction, which will be explored later in the book.

THE FOUR SCHOOLS OF AI (RUSSELL & NORVIG FRAMEWORK)

This section introduces a framework for understanding the diverse schools of thought in AI, often based on the work of Russell & Norvig. This framework is presented as a spectrum, helping to contextualize knowledge-based AI within the broader landscape of AI disciplines. The spectrum is defined by two dimensions: one ranging from 'thinking' to 'acting', and the other from 'optimal' functioning (or 'rationally') to functioning 'like humans'. The spectrum can be thought of as consisting of four quadrants:

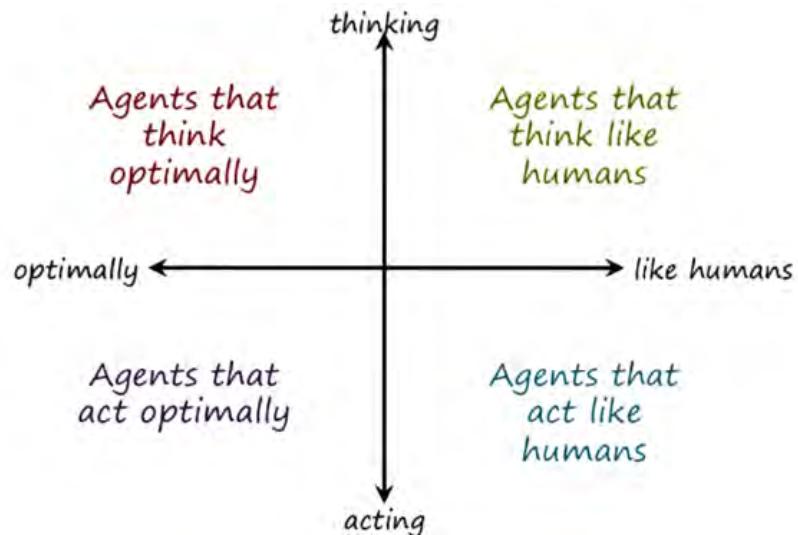


Figure 1-3 The AI Foundational Spectrum
 Imaged based on ideas from Artificial Intelligence, A Modern Approach 3rd Edition

- **Thinking Optimally (Thinking Rationally):** Agents that think optimally.¹⁹ This quadrant includes AI systems that focus on thinking processes and strive for optimal performance according to some rational standard. Many machine learning algorithms fall into this category. They analyze vast amounts of data to identify patterns and regularities, embodying optimal thinking but not necessarily mimicking human-like thought processes.
- **Acting Optimally (Acting Rationally):** Agents that act optimally.²⁰ Here, AI agents are designed for optimal performance in action-oriented tasks, aiming to achieve the best possible outcome. An example is airplane autopilot systems, which act in the world with a high degree of precision and efficiency.
- **Acting Human-like:** Agents that act like humans.²¹ This category encompasses AI agents that perform actions in a manner similar to humans. Examples include improvisational robots that can dance to music, exhibiting human-like movements and reactions.
- **Thinking Human-like:** Agents that think like humans.²² The focus here is on AI agents that think in ways akin to humans. Technologies like the Semantic Web, which understands and interprets web pages and information, are representative of this approach. KBAI often aligns with this quadrant. They engage in 'thinking' processes in a style that more closely resembles human cognition.

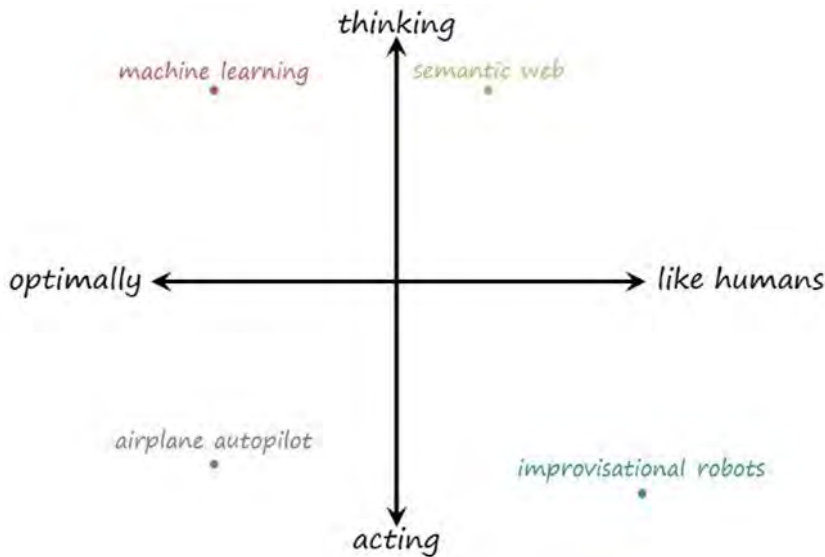


Figure 1-4 Example of well-known computational techniques in the AI Foundational Spectrum

These examples illustrate the diverse applications and approaches within the field of AI, each fitting into a distinct quadrant of the AI spectrum. The field of AI is constantly evolving, and there is an ongoing demand for new ideas and contributions.

Discussion: Where do autonomous vehicles fit?

Ashok David, where do you think an autonomous vehicle would fall on our AI spectrum?

David Well, an autonomous vehicle certainly navigates and moves in the world, so it's actively engaging in its environment. Given that driving is inherently a human-like activity, I'd argue that it falls into the category of 'acting like a human.'

Ashok David, do we really need to be concerned about whether an autonomous vehicle thinks and acts like us?

David You know, when I think about it, as long as the vehicle safely gets me to my destination without any incidents, I'm not particularly concerned if it thinks like I do. And frankly, considering my own driving habits, I'd prefer it if it didn't act like me either.

Ashok Right, so perhaps we should consider the autonomous vehicle as leaning more towards the 'acting rationally' side of the spectrum. Simultaneously, observing human activities like writing might inform our approach to designing a robot. Conversely, studying robot designs could offer insights into human cognition. This interplay is a key aspect of knowledge-based AI.

Interactive Exercise: Classifying Roomba, C-3PO, Siri, Google Maps

Let's do an interactive exercise. We're presented with the four quadrants of AI again, and this time, we'll examine four distinct artifacts:

1. Roomba (automated vacuum cleaner)
2. C-3PO (a character from Star Wars)
3. Siri (Apple's virtual assistant)
4. Google Maps (route navigation tool)

You're likely familiar with all four. It's important to note that C-3PO, while a fictional character, still represents a significant concept in AI. Our task is to categorize each of these artifacts into the most appropriate quadrant.

Ashok: What are your thoughts on this, David?

David: Let's start with Roomba. I would place Roomba in the bottom left quadrant, 'acting optimally'. It acts in the world, but not necessarily in the way humans do. It moves in a crisscross pattern across the floor, vacuuming everything up, aiming for optimal coverage.

C-3PO, fluent in over 6 million forms of communication, often interacts with humans and other species. To do this, it must understand natural sentences and respond in kind, which means it needs to 'act like humans'.

As for Apple's virtual assistant, Siri, it doesn't physically act in the world. It falls more on the thinking side of the spectrum. However, like C-3PO, Siri interacts with humans, interpreting and responding in human language, which places it in the 'thinking like humans' quadrant.

Google Maps, on the other hand, is involved in route planning from an origin to a destination. It's engaged in thinking, not in physical actions. It doesn't need to plan routes as humans would; instead, it considers factors like traffic and construction. This approach is more about 'thinking optimally', as it seeks the most efficient route, which may differ from human intuition.

Ashok: That's a well-thought-out categorization, David. I concur with your assessment. However, it's worth noting that some aspects of Siri might fit into other quadrants. While we've placed it under 'thinking like humans', Siri could also be seen as 'acting' when it provides a response. Moreover, some elements of Siri's functionality might be 'optimal', diverging from human-like processing.

WHAT ARE COGNITIVE SYSTEMS?

We examine the nuances of the term “Cognitive Systems” by dissecting its components.

- **Cognitive:** Dealing with human-like intelligence, with the ultimate goal being the development of human-level intelligence.
- **Systems:** Multiple interacting components, such as learning, reasoning, and memory.
- **Cognitive Systems:** Systems that exhibit human-like intelligence through processes like learning, reasoning, and memory.

Cognitive Systems Architecture

In the spectrum of our discussion, KBAI and Cognitive Systems lean towards the human aspect, focusing on both thinking and acting while always considering human cognition.

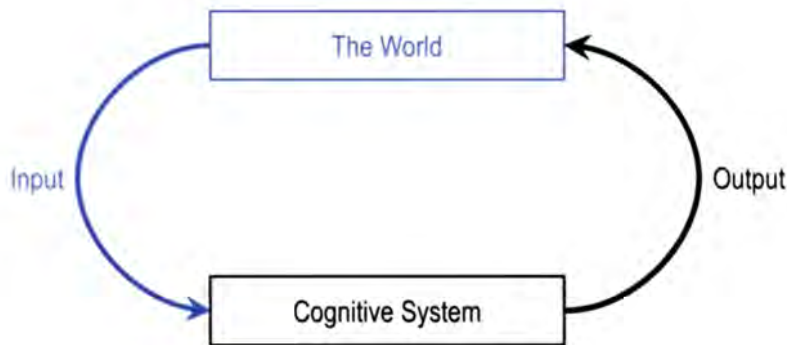


Figure 1-5 Cognitive Systems and the World

Cognitive Systems Architecture The term 'cognitive system' is often used interchangeably with 'knowledge-based AI agent.' However, these terms can be distinguished based on their design perspectives. A knowledge-based AI system could be designed without human-like qualities, or it could be intended to emulate human-like characteristics. A cognitive system is

essentially situated in the physical world, interacting with its immediate environment, which includes tangible elements like screens and microphones

In the world of cognitive systems, perception plays a critical role. Percepts are the pieces of information a system gathers from its environment through its sensors. For instance, perceiving a straight line, the color of an object, or the texture of a surface. These systems utilize sensors to perceive such elements, forming the input for the cognitive system. Additionally, cognitive systems employ actuators, akin to human fingers, to interact with and act upon the world. These percepts form the basis of the system's understanding and interaction with the world.

While the focus so far has been on single cognitive systems, it's important to note that multiple cognitive systems can coexist and interact, both within the physical and social worlds.

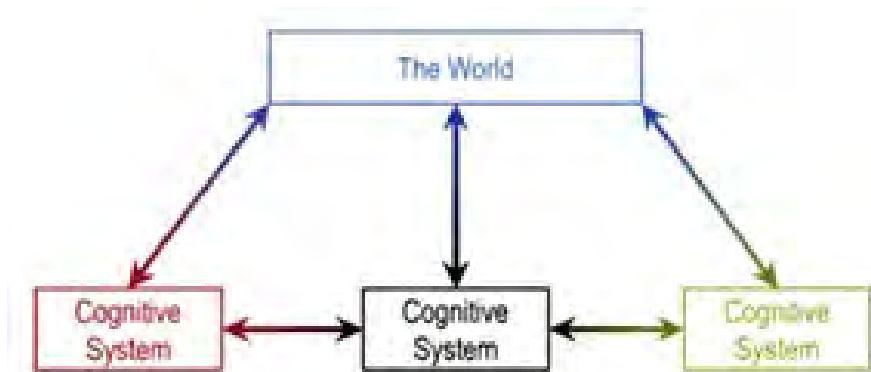


Figure 1-6 Multiple Agent Cognitive System

In cognitive systems, mapping percepts to actions can vary in complexity. Examining the architecture of a cognitive system reveals its core functionality: converting percepts about the world into actionable outputs. This conversion can occur through different layers:

- **Reactive Systems:** This refers to a simpler form of mapping percepts to actions. In reactive systems, the response to a stimulus is immediate and automatic, without the need for deliberative processing. For example, when driving, seeing the brake lights of the car ahead immediately leads to pressing the brakes. This is a direct, reactive response without elaborate planning.
- **Deliberative Systems:** A more complex task like changing lanes while driving involves deliberation. Deliberation is about reasoning based on external environmental percepts. This process includes considering goals and environmental percepts to formulate a plan, incorporating key components such as learning, reasoning, and memory, which interconnect in sophisticated ways.
- **Metacognitive Systems:** Metacognition adds another layer, focusing on the introspection of internal mental processes. Metacognition involves evaluating and

potentially adjusting the internal processes based on external feedback, like modifying lane-changing strategies following an incident.

The architecture of cognitive systems comprises these three layers: reaction, deliberation, and metacognition.²³ Intelligence in this context is the ability to map percepts to the most appropriate actions, which can be reactive, deliberative, or include metacognitive analysis of both.

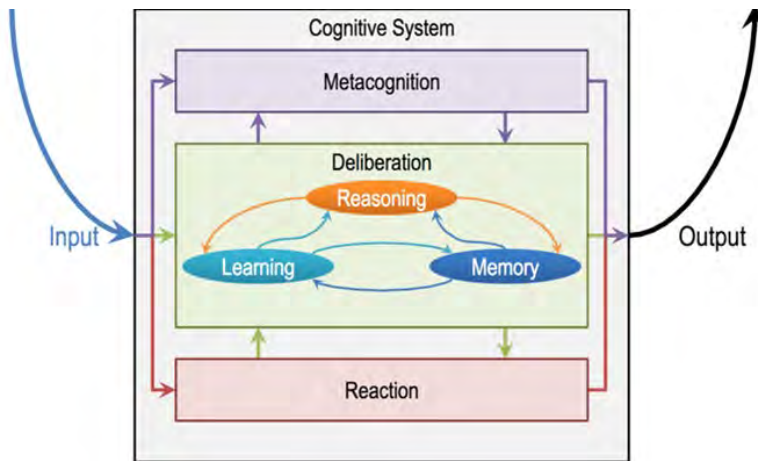


Figure 1-7 Cognitive Systems Architecture

This three-layered framework offers a comprehensive understanding of cognitive AI systems, serving as a foundational concept in the study of cognitive systems in AI.

OVERVIEW OF TOPICS IN KBAI (THE EIGHT KEY AREAS)

This Knowledge-Based Artificial Intelligence (KBAI) book is structured into eight comprehensive units. Each unit focuses on a critical aspect of KBAI, offering a deep dive into various methodologies, theories, and applications. The accompanying chart provides a visual representation of these units, guiding learners through the progressive stages of the book.

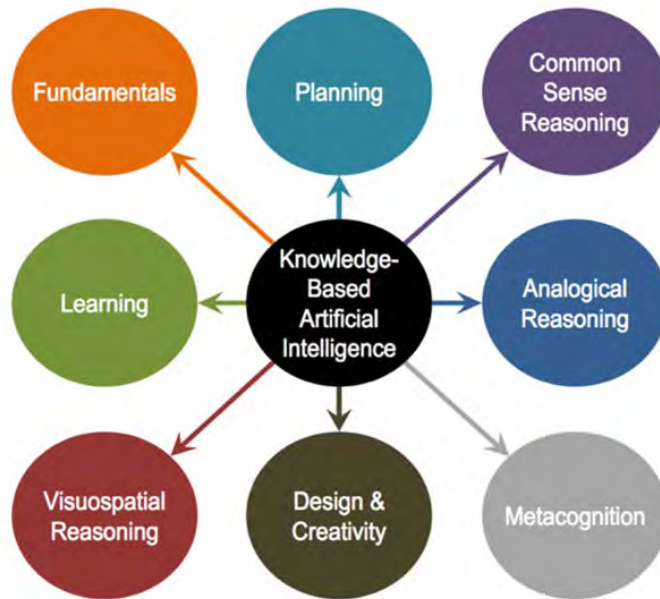


Figure 1-8 KBAI's eight major areas

Fundamentals of Representation and Reasoning

The first unit covers the basics of knowledge representations and reasoning strategies. Key topics include Semantic Networks and Production Systems, as well as reasoning methods like Generate and Test, Means-End Analysis, and Problem Reduction. While these concepts are independent, there's an intentional order in their presentation. For instance, discussions on Production Systems may reference concepts from Semantic Networks. Similarly, Means-End Analysis might draw on aspects covered in Generate and Test. This ordering, however, doesn't imply a dependency between these methods or knowledge representations; it's simply the chosen sequence for the course.

Fundamentals

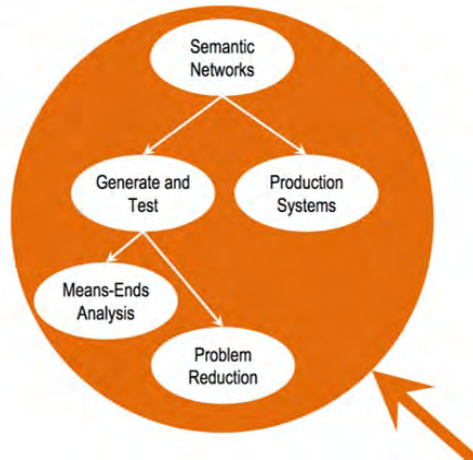


Figure 1-9 Fundamental KBAI Components

Planning

The second unit explores Planning, a critical problem-solving activity in AI, aimed at developing effective plans to achieve set goals. This unit begins with a discussion on Logic as a knowledge representation, laying the groundwork for a systematic exploration of Planning. Logic, in the context of AI, provides a structured way to represent facts, rules, and relationships within a given problem domain. By employing logical frameworks, AI systems can process complex information and perform reasoning tasks more effectively. With a solid grounding in Logic as a knowledge representation, the book then continues to examine Planning (as specific area within the broader Planning domain). Planning, a key problem-solving activity in AI, involves creating strategies or sequences of actions to achieve specific goals. The logical representation of knowledge enables AI systems to evaluate possible actions, predict outcomes, and make informed decisions, making the Planning process more systematic and effective.

Planning

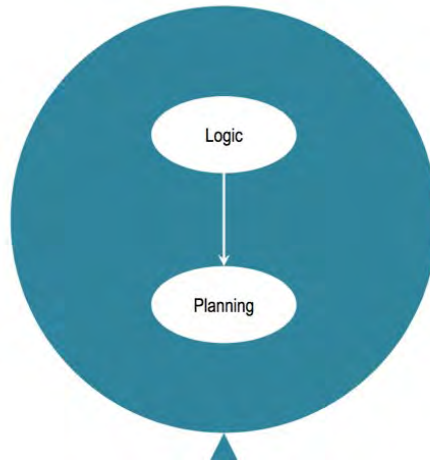


Figure 1-10 Planning KBAI Components

Common Sense Reasoning

The third unit examines Common Sense Reasoning, which is crucial for understanding and interpreting everyday situations. Common Sense Reasoning involves making inferences, which are logical conclusions drawn from available data or facts. This unit also introduces frames, a concept in AI that refers to structured representations of stereotypical situations, helping AI systems to organize and interpret information like human understanding. Alongside frames, various methods (e.g., scripts) employed for Common Sense Reasoning are discussed, providing insights into how AI systems can process and react to everyday scenarios in a manner that aligns with human logic and expectations.

Common Sense Reasoning

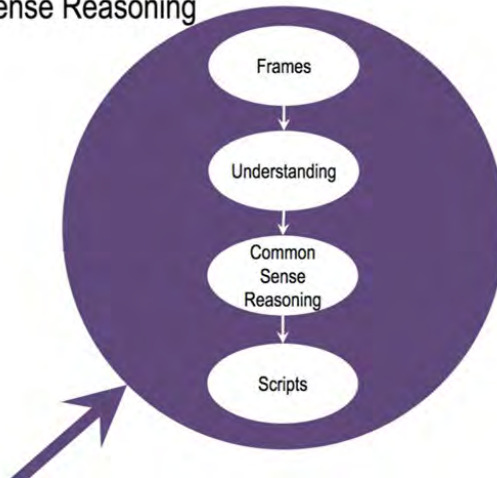


Figure 1-51 Common Sense Reasoning KBAI Components

Learning

In the Learning unit, while we focus on the fundamental process of learning within deliberation, it's important to note that the topic is broad and multifaceted. The unit explores several key components of learning in AI, including Learning by Recording Cases, Incremental Concept Learning, Classification, and Version Spaces. It's important to remember that these components, while crucial, do not encompass all aspects of learning in AI.

Learning

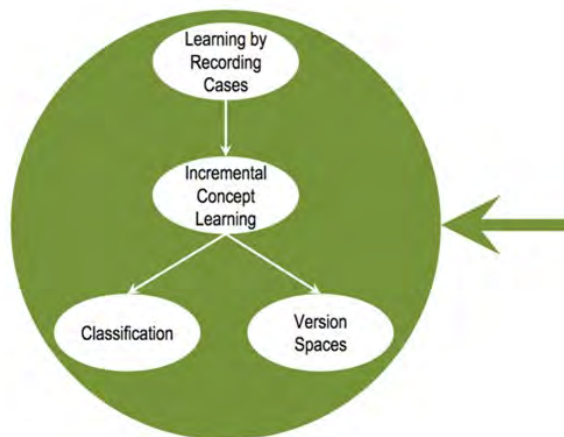


Figure 1-12 Learning KBAI Components

Analogue Reasoning

The unit on Analogue Reasoning explores how AI systems solve new problems by drawing parallels with known situations. This approach is integral to cognitive AI, enabling the application of learned solutions to novel scenarios. Crucial to this unit is the concept of Learning by Recording Cases, which forms the basis for Case-Based Reasoning. Additionally, Explanation-Based Learning is discussed, highlighting how AI systems understand and apply underlying principles from examples to new situations.

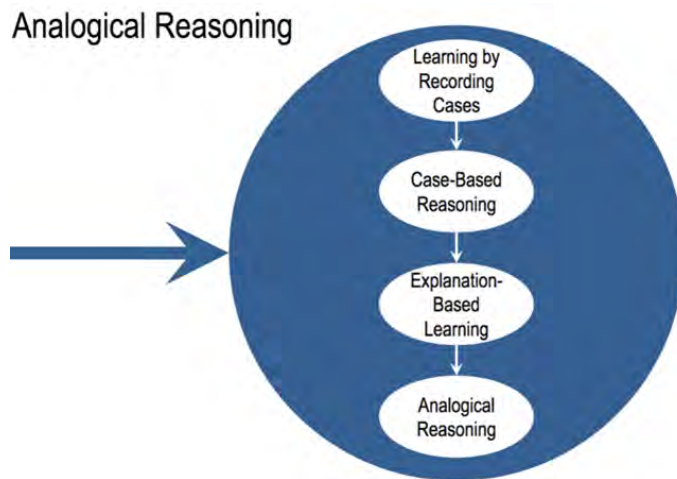


Figure 1-13 Analogue Reasoning KBAI Components

Visuospatial Reasoning

The unit on Visuospatial Reasoning addresses the process of reasoning with visual and spatial information. This unit focuses on two key concepts: Constraint Propagation and Visuospatial Reasoning itself. Constraint Propagation is a method used in AI to solve problems where a set of constraints limits possible solutions. Visuospatial Reasoning refers to the ability of AI systems to interpret and process visual and spatial data.

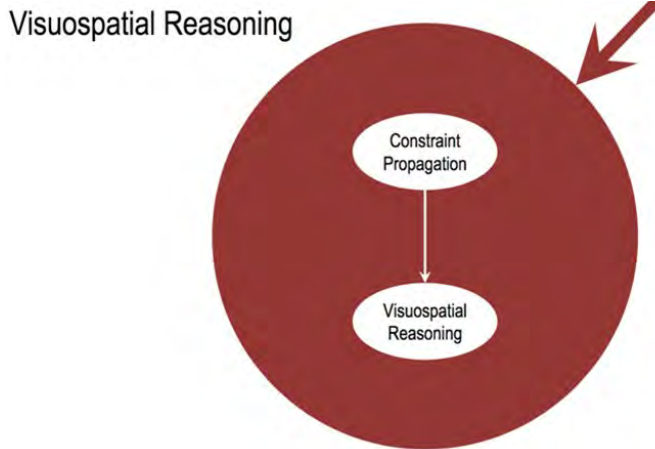


Figure 1-14 Visuospatial Reasoning KBAI Components

Design & Creativity

The Design & Creativity unit explores the capacity of AI systems to innovate and create in novel situations. This encompasses key concepts like Configuration (arranging components to meet objectives), Diagnosis (identifying and resolving system problems), Design (creation of new systems or processes), and Creativity (generating novel and valuable ideas).

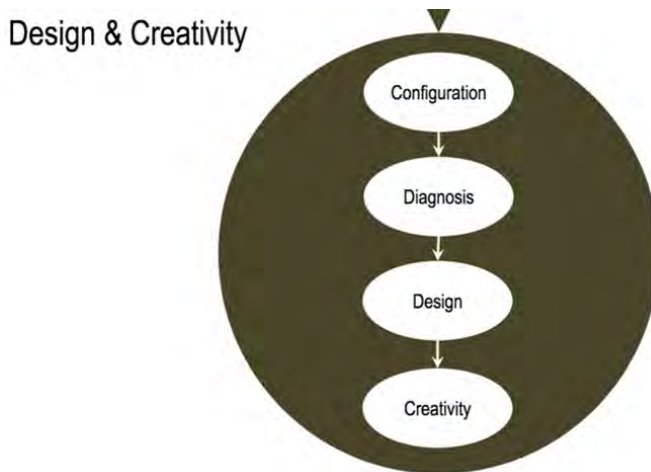


Figure 1-15 Design and Creativity KBAI Components

Metacognition

The Metacognition unit focuses on the higher-level thinking processes of AI systems. Key concepts include Learning by Correcting Mistakes, Meta-Reasoning (reflecting on own reasoning processes), and Ethics in Artificial Intelligence.

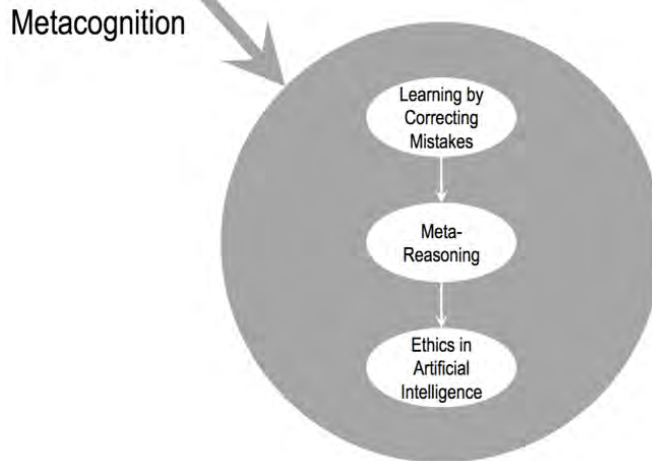


Figure 1-16 Metacognition KBAI Components

BOOK STRUCTURE AND PERSONALIZATION

The book is structured to interleave various topics, allowing for a more integrated understanding of KBAI. While a specific order is suggested, students are encouraged to explore the topics in their own sequence, personalizing their learning experience. This book presents a unique opportunity to look into advanced topics like Analogical Reasoning, Creativity, and Metacognition, offering a comprehensive overview of Knowledge-Based Artificial Intelligence.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

While KBAI focuses on explicit knowledge representation and symbolic reasoning, its principles and techniques can be powerfully integrated with other major AI paradigms to create more robust, interpretable, and capable systems. Understanding how KBAI can enhance fields like Probabilistic Machine Learning, Deep Learning, Reinforcement Learning (RL), Natural Language Processing (NLP), Generative AI (GenAI), and Agentic AI is crucial for a holistic view of artificial intelligence.

- **Probabilistic Machine Learning:** KBAI's symbolic knowledge can be integrated with probabilistic models to create hybrid systems that handle uncertainty while maintaining interpretability. For instance, KBAI can define the structure or constraints for a Bayesian network, guiding its learning and inference, or use symbolic rules to interpret probabilistic outputs.
- **Deep Learning (Neuro-Symbolic AI):** KBAI's explicit knowledge representations (like ontologies or rules) and reasoning mechanisms can be fused with deep learning architectures. This synergy, often termed Neuro-Symbolic AI, can involve using symbolic knowledge to guide neural network training, improve data efficiency, verify outputs, provide interpretable explanations for “black box” models, or enable more complex, multi-step reasoning.
- **Reinforcement Learning (RL):** RL agents learn optimal policies by interacting with an environment and receiving rewards or penalties. While RL focuses on learning *what* to do, KBAI can provide the knowledge about *why* certain actions are taken or *how* the environment works. Integrating KBAI with RL can lead to more sample-efficient learning and agents that can explain their policies.
- **Natural Language Processing (NLP) & Generative AI (GenAI):** KBAI has historical ties to NLP, particularly in semantic interpretation using frames and scripts. For modern NLP and GenAI models like Large Language Models (LLMs), KBAI's structured knowledge (e.g., knowledge graphs) and reasoning frameworks can be applied to ground LLM outputs in factual knowledge, verify factual accuracy, ensure logical consistency, perform more complex reasoning over generated text, or steer generation towards more coherent and contextually appropriate responses.
- **Agentic AI:** KBAI provides core architectural patterns (like the deliberation and metacognition layers discussed earlier), planning algorithms (e.g., means-ends analysis, problem reduction, partial-order planning), and explicit goal management techniques. These are fundamental to developing sophisticated AI agents capable of autonomous decision-making, planning, and action in complex, dynamic environments. Modern agentic systems often benefit from combining learned components with these explicit KBAI reasoning and planning mechanisms.

The future of AI likely lies in such hybrid systems – sometimes referred to as Hybrid AI, Integrated AI, or Composite AI – that leverage the strengths of different approaches. KBAI's emphasis on explicit knowledge, interpretability, structured reasoning, and commonsense can complement the data-driven strengths of modern machine learning techniques, leading to AI systems that are not only powerful but also more trustworthy, explainable, and aligned with human understanding.

CHAPTER SUMMARY

In each chapter, a recap will be provided, summarizing the key points discussed and connecting them to future topics. This first chapter started with an exploration of the central conundrums and characteristics of AI. This discussion may align with experiences from other AI topics, such as machine learning and AI for robotics. The lesson then moved on to examine the four schools of AI, with a particular focus on knowledge-based AI, its essence, and how it integrates with other branches of AI. The concept of cognitive systems was also discussed, highlighting their emphasis on human-like intelligence. The book's structure was outlined, revealing that it is segmented into eight major categories, including learning, planning, and analogical reasoning. In the following lessons, more specific details about the book itself will be discussed, covering goals, expected outcomes, and learning strategies.

ADDITIONAL READINGS & REFERENCES

- 1 Hayes-Roth, F. & Jacobstein, N. The state of knowledge-based systems. *Communications of the ACM* **37**, 26-39 (1994).
- 2 Davis, R. Knowledge-based systems. *Science* **231**, 957-963 (1986).
- 3 Bade, D. Cognitive systems. *Journal of Documentation* **64** (2008).
- 4 Vernon, D. *Artificial cognitive systems: A primer*. (MIT Press, 2014).
- 5 Gwynne, P. The AI Conundrum. *Research-Technology Management* **67**, 7-8 (2024).
- 6 Joshi, R. Ethical conundrums and privacy concerns in new innovations: A systematic review. *Aspects of Quality Management in Value Creating in the Industry 5.0 Way*, 202-223
- 7 Nweke, O. C. & Nweke, G. I. Legal and Ethical Conundrums in the AI Era: A multidisciplinary analysis. *International Law Research* **13**, 1-1 (2024).
- 8 Wooldridge, M. Intelligent agents. *Multiagent systems: A modern approach to distributed artificial intelligence* **1**, 27-73 (1999).
- 9 Schonlau, M., Welch, W. J. & Jones, D. R. Global versus local search in constrained optimization of computer models. *Lecture notes-monograph series*, 11-25 (1998).
- 10 Caseau, Y., Laburthe, F., Le Pape, C. & Rottembourg, B. Combining local and global search in a constraint programming environment. *The Knowledge Engineering Review* **16**, 41-68 (2001).
- 11 Focacci, F., Laburthe, F. & Lodi, A. in *Handbook of metaheuristics* 369-403 (Springer, 2003).
- 12 Beydoun, G. & Hoffmann, A. Incremental acquisition of search knowledge. *International Journal of Human-Computer Studies* **52**, 493-530 (2000).
- 13 Spokoyny, A. & Shahar, Y. Incremental application of knowledge to continuously arriving time-oriented data. *Journal of Intelligent Information Systems* **31**, 1-33 (2008).
- 14 Papadimitriou, C. H. & Tsitsiklis, J. Intractable problems in control theory. *SIAM journal on control and optimization* **24**, 639-654 (1986).
- 15 Rabuzin, K., Maleković, M. & Bača, M. A survey of the properties of agents. *Journal of Information and Organizational Sciences* **30**, 155-170 (2006).
- 16 McDermott, D. V. Logic, problem solving, and deduction. *Annual Review of Computer Science* **2**, 187-229 (1987).

- 17 High, R. The era of cognitive systems: An inside look at IBM Watson and how it works. *IBM Corporation, Redbooks* **1**, 16 (2012).
- 18 Loui, R. P. & Moore, D. M. Dialogue and deliberation. (1997).
- 19 Russell, S. J. & Subramanian, D. Provably bounded-optimal agents. *Journal of Artificial Intelligence Research* **2**, 575-609 (1994).
- 20 Cassandra, A. R., Kaelbling, L. P. & Littman, M. L. in *Aaai*. 1023-1028.
- 21 Arthur, W. B. On designing economic agents that behave like human agents. *Journal of Evolutionary Economics* **3**, 1-22 (1993).
- 22 Lake, B. M., Ullman, T. D., Tenenbaum, J. B. & Gershman, S. J. Building machines that learn and think like people. *Behavioral and brain sciences* **40**, e253 (2017).
- 23 Langley, P. Cognitive architectures and general intelligent systems. *AI magazine* **27**, 33-33 (2006).

Chapter 2

Semantic Networks

INTRODUCTION TO SEMANTIC NETWORKS

In the expansive field of Knowledge-Based Artificial Intelligence (KBAI), semantic networks are recognized as a fundamental and highly effective knowledge representation scheme. Essentially, semantic networks are structures that encapsulate 'semantics' – the meanings and concepts of various entities, and 'networks' – the intricate web of relationships connecting these entities or objects. This chapter is dedicated to unraveling the complexities and functionalities of semantic networks, showcasing their crucial role in the 'represent-and-reason' paradigm within KBAI.

The essence of semantic networks lies in their ability to do more than just catalog information about a problem; they organize and structure knowledge in such a way that it becomes a potent tool for AI systems to employ in solving complex, multifaceted problems. By establishing a clear definition of semantic networks as systems that represent entities or objects and illustrate the connections between them, we set a solid foundation for a deeper exploration of how knowledge can be interlinked and harnessed for effective problem-solving.

Throughout this chapter, we will look into the defining characteristics of semantic networks and their pivotal role in KBAI. We'll explore a variety of example problems that demonstrate the practical application of semantic networks, providing a comprehensive view of how they function and why they are so integral to knowledge representation in AI. By examining the intricate workings of semantic networks, we aim to provide not only a thorough understanding of their structure and purpose but also insight into their operational mechanisms and the

reasoning processes they support. This exploration is geared towards revealing the multifaceted nature of semantic networks, illuminating their significance in representing knowledge and facilitating intelligent, problem-solving capabilities in AI systems.

THE ESSENCE OF KNOWLEDGE REPRESENTATION

What exactly constitutes a knowledge representation in artificial intelligence? To understand this, imagine a collage showcasing the diverse range of knowledge representations developed in AI.²⁴ Each of these representations is characterized by two fundamental components: a language and its content.

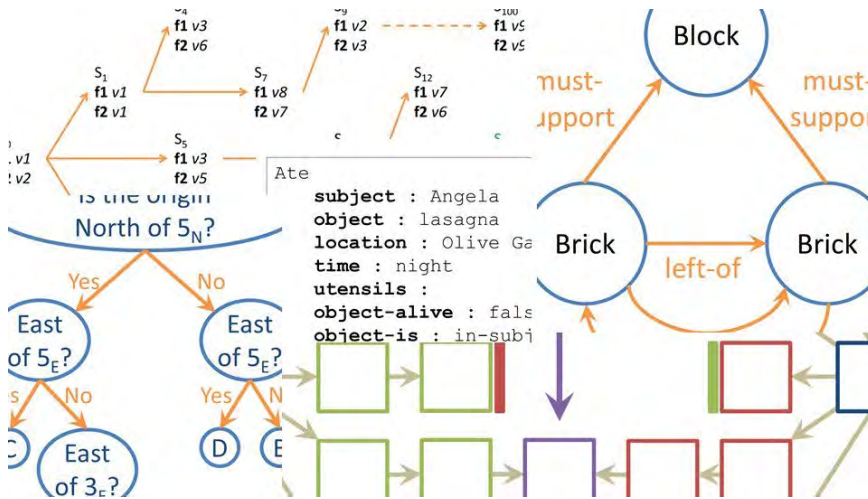


Figure 2-1 Collage of Knowledge Representations

Each knowledge representation has a language. The language component of a knowledge representation includes a specific vocabulary, a set of symbols or terms used to convey ideas within that framework. It's the medium through which knowledge is expressed. In addition to this language, there's the content - the actual substance of the knowledge being represented.

To illustrate, consider Newton's second law of motion, a concept familiar to many. In this case, the knowledge representation is the formula $F = ma$, where F denotes force, m is mass, and a represents acceleration. Here, the language is that of algebraic equations (e.g., $y = mx + b$), and the content is the physical law itself (i.e., Newton's second law of motion – “*force equals mass multiplied by acceleration*”). This simple yet powerful representation encapsulates a fundamental principle of physics using a specific vocabulary (symbols like F , m , and a , and the equality sign) and the conceptual understanding of the law. This duality of language and content is a recurring

theme in various knowledge representations in AI. AI has developed not one, but many knowledge representations. Each representation has its own affordances and its own constraints.

SEMANTIC NETWORKS IN ACTION: SOLVING RAVEN'S PROGRESSIVE MATRICES

To grasp the concept of semantic networks as a knowledge representation tool, let's consider a practical example. Imagine a Raven's Progressive Matrices problem where we must determine the relationship between two sets of images, labeled as A is to B as C is to D.²⁵ The challenge is to identify the correct image for D from six given choices. We begin by representing our knowledge of A and B. We will then use this knowledge to select the best alternative of the six options presented in the bottom of the image.

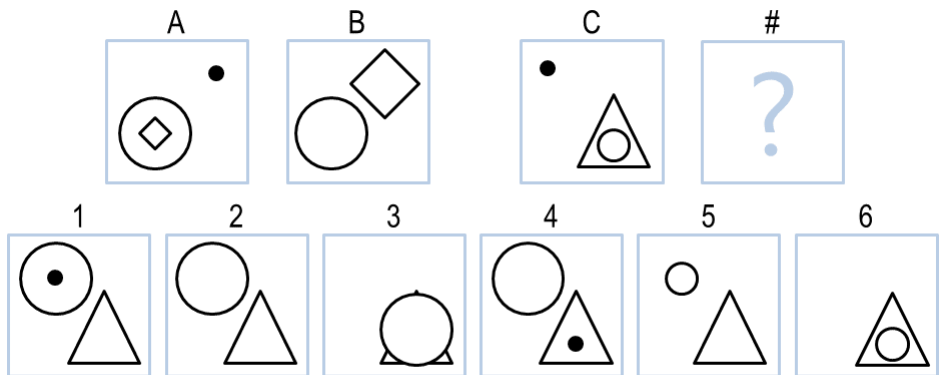


Figure 2-2 Example of Raven's Test

In image A, imagine there are three objects: a circle (labeled x), a diamond inside the circle (labeled y), and a black dot (labeled z). The task is to construct a semantic network that represents our understanding of these objects and their relationships.²⁶

We represent each object as a node: x for the circle, y for the diamond, and z for the black dot. To represent the relationships between these objects, we create links between the nodes. These links can be labeled. For instance, we can label a link from y to x indicating that y (the diamond) is inside x (the circle). Similarly, links from z to y and z to x can signify that z (the black dot) is above y and x. By following this approach, the semantic network for image A encapsulates both the objects and their interrelationships.

We apply the same methodology to image B, identifying its objects and their relationships. The next step is to capture the transformation from image A to image B. This involves constructing links between the objects in A and B, illustrating the changes or consistencies between them. For instance, if object *x* remains unchanged between A and B, we label the corresponding link as “unchanged.” If *y* expands in size from A to B, we label this transformation as “expanded.” If *z* in image A is absent in image B, we introduce a dummy node in B and label the link from *z* as “deleted.” This deletion captures the idea that *z* does not occur in B.

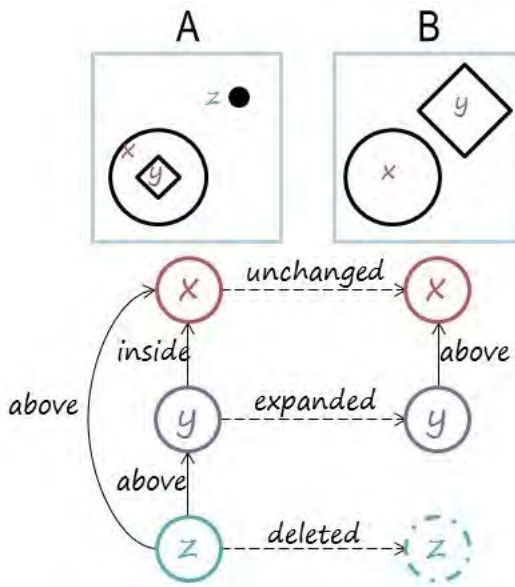


Figure 2-3 Semantic Network for shapes A and B

The semantic network effectively breaks down the problem into three components:

- Representation of individual objects in each image.
- The relationships among these objects within each image.
- The transformations or consistencies between the corresponding objects across the two images.

This detailed semantic network effectively captures the nuanced dynamics between A and B. It not only represents static objects and their spatial relationships but also the dynamic process of change, illustrating the powerful representational capabilities of semantic networks. This ability to encode transitions is particularly significant because it aligns closely with human cognitive processes – our understanding of the world is not just about recognizing objects but also about grasping the nature of their changes and interactions over time.

By applying this methodological approach to semantic networks, we gain a robust framework for reasoning about and solving problems that are presented in visual or abstract forms. It's a method that allows for the dissection of complex problems into comprehensible relationships and transformations, enabling clearer understanding and more effective problem-solving strategies. Whether dealing with simple pairs of images or more intricate scenarios, the principles of semantic networks remain the same, providing a versatile tool in the arsenal of knowledge-based artificial intelligence.

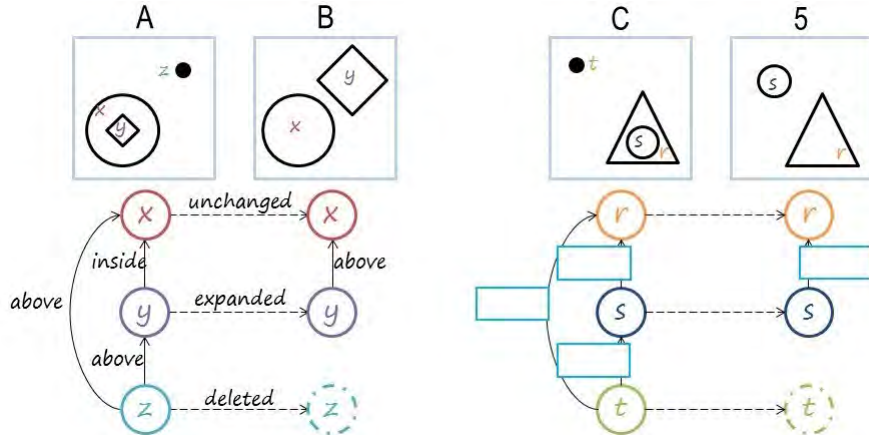


Figure 2-4 Semantic Network for C & Option 5/2 (Ashok & David discussion)

Ashok: Let's now shift our focus to the set labeled C, and I've selected option five from the six available choices for our exercise. We will construct a semantic network for C and five, paralleling the one we crafted for A and B. All objects in C and five are displayed; your challenge is to label the links connecting these objects, as well as the links between the objects in five. David, what labels did you assign in your semantic network?

David: I mirrored the approach we took for A and B. Observing the relationships in A, I applied the same terminology to C and 5. In C, just as z was positioned above x and y in A, I found that t is positioned above both r and s. Consequently, I labeled the links to indicate that t is above s and also above r. Similarly, reflecting the relationship in A where y was inside x, in C, s is enclosed within r. Moving to image 5, I observed and labeled that s is above r, maintaining the parallelism with image B where y was above x.

Ashok: David's observation here is crucial. He's utilized the vocabulary of 'inside' and 'above' consistently, as was done with A and B. It's essential for us to apply a consistent vocabulary across the representations of this problem class within

semantic networks. By adopting a uniform set of terms like 'inside' and 'above', we establish a standard method to represent and analyze problems of this nature. This consistency is key in ensuring clarity and reliability in our semantic representations.

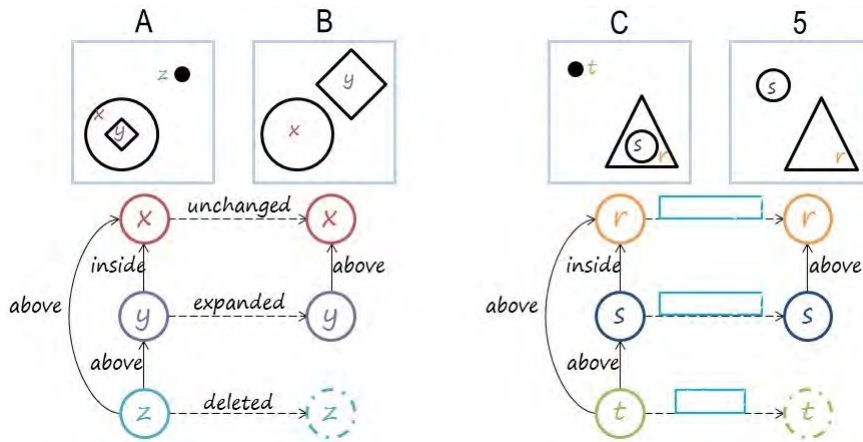


Figure 2-5 Semantic Network for C & Option 5/2 (Ashok & David discussion continued)

Ashok Let's continue. David, can you share the labels you've come up with for the transformations in the semantic network based on C and 5?

David I've taken the approach we used earlier where I used the same vocabulary to describe the relationships and transformations within the figures. For instance, *r* remains consistent between images C and 5, similar to how *x* was consistent between A and B. Therefore, I've labeled *r* as "unchanged." Similarly, *s*, although it has moved, retains its shape and size, so it too is marked "unchanged." Its movement was previously captured in the relationship between *s* and *r*. Then, considering the absence of *t* in image 5, akin to the absence of *z* in B, I've labeled it "deleted," as we've established in our previous network.

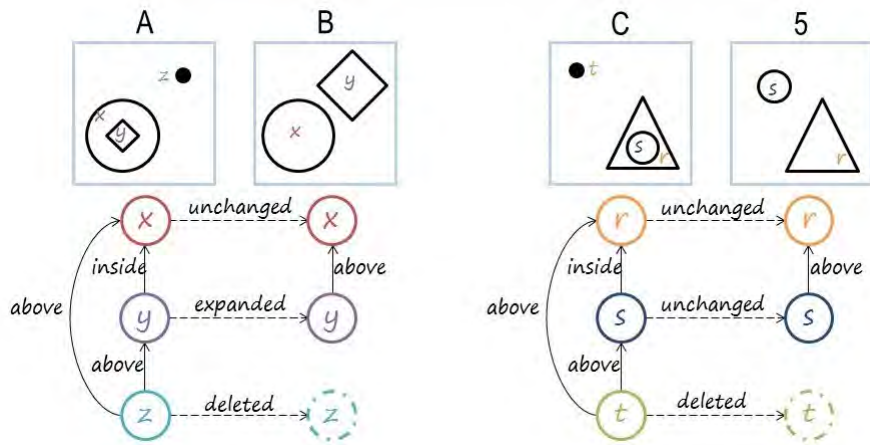


Figure 2-6 Semantic Network for C & Option 5/2 (Ashok & David discussion continued)

Ashok That's a well-reasoned response, David. It's important to recognize the consistency of shapes across the figures,

STRUCTURE OF SEMANTIC NETWORKS

Having explored semantic networks through practical examples, let's now distill semantic networks into their core components as a form of knowledge representation.

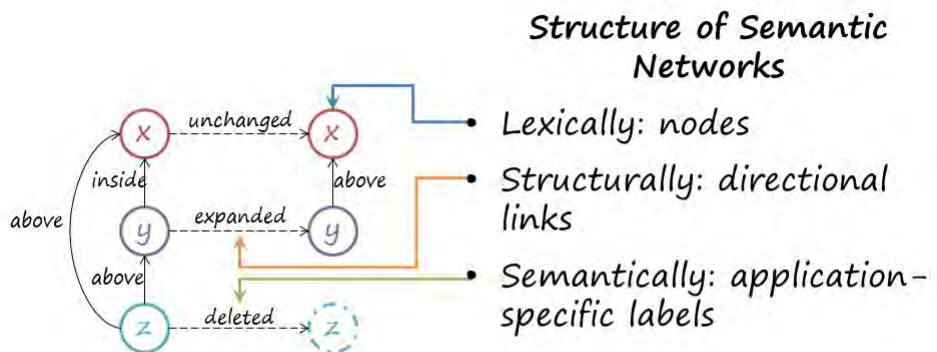


Figure 2-7 Structure of Semantic Networks

A robust knowledge representation has several key elements:

- **Lexicon:** The lexicon is the foundational vocabulary of the representation language. It defines the set of symbols or terms that can be used to construct the knowledge representation.²⁷ In a semantic network, the lexicon consists of nodes representing objects or concepts, such as *x*, *y*, and *z*.
- **Structural Specification:** This aspect details how the elements of the lexicon can be organized into a complex representation structure.²⁸ In semantic networks, this structure is articulated through directed links. These directed links represent the relationships between nodes, enabling the combination of individual nodes into complex representations that capture the interdependencies and hierarchies present in the knowledge being modeled.
- **Semantics:** Semantics provide the meaning to the structure, outlining how the elements and their compositions can be interpreted and used for reasoning.²⁹ By application-specific labels on the links in a semantic network, we assign specific meanings to the relationships between nodes. These labels are essential for inference, allowing us to deduce new information from the existing structure.

In essence, semantic networks as a knowledge representation provide a structured, interpretable way of representing knowledge that supports reasoning and inference-making.²⁸ Through their nodes and labeled links, they allow for the representation of complex interrelations and serve as a foundation for AI systems to process and deduce new information. This systematic approach to organizing knowledge is what makes semantic networks a powerful tool in the field of Knowledge-Based Artificial Intelligence.

CHARACTERISTICS OF GOOD REPRESENTATIONS

Having observed semantic networks in practical scenarios, we encounter an essential inquiry: What distinguishes an effective knowledge representation? The answer lies in the ability of a good representation to make relationships between entities explicit. It brings objects and relationships together. For instance, in a problem involving a two-by-one matrix with various objects like circles, triangles, and dots, the relationships among these objects – whether one is left of another, inside, or above – are explicitly defined by the semantic network.

An optimal representation operates at an appropriate level of abstraction. It encapsulates all necessary information, exposing their natural constraints while eschewing superfluous details. The hallmark of a quality representation is its transparency and conciseness – it is comprehensive yet not cluttered with irrelevant data. This streamlined approach not only aids in the clarity of the representation but also enhances its computational efficiency.

A well-crafted representation is swift and computationally viable, enabling rapid processing and facilitating the extraction of inferences critical for problem-solving. It articulates the natural constraints of the problem at hand, ensuring that every piece of essential information is represented and can be utilized effectively. In essence, a good knowledge representation is

characterized by its clarity, precision, and the ability to support the inferential processes required to navigate and solve complex problems.

Characteristics of Good Representations

- *Makes relationships explicit*
- *Exposes natural constraints*
- *Brings objects and relations together*
- *Excludes extraneous details*
- *Transparent, concise, complete, fast, and computable.*

Table 2-1 Characteristics of Good Representations

Ashok	Let's consider what constitutes a good representation in the context of our daily experiences. David, can you provide an example from everyday life?
David	One practical example is the nutritional label found on food and beverage containers. It breaks down the content into categories for each nutrient and specifies the quantities present in the product.
Ashok	That's an interesting choice. It does appear to effectively communicate the essential information about the contents. It's structured and clear, providing a snapshot of the nutritional value, which is what we need from such a representation. David, your example seems quite appropriate. It's concise yet comprehensive, offering a snapshot of vital information that consumers can use to make informed dietary choices. Do you believe this enables the drawing of accurate inferences?
David	Absolutely. If I'm monitoring my caloric intake, the label provides the necessary calorie information, guiding my consumption choices. Similarly, it offers the data needed for other dietary considerations.
Ashok	What about information that might be superfluous to individual needs?
David	That's a valid consideration. The label offers information that could be pertinent to various users, which means it inevitably includes data that some may find unnecessary. For instance, I might be indifferent to carbohydrate content, which for me would be extraneous, but for others, it might be crucial.

- Ashok** Does the label clarify the relationships among the nutritional elements?
- David** That's where it falls short. Take calories from fat as an example – it lists the amount of fat and the calories from fat, but it doesn't explicitly link the two. Without recognizing this relationship, one might not be able to make informed inferences about one's diet based on that information alone.
- Ashok** This highlights a critical aspect of inference-making. Nutritional labels provide a snapshot of information that facilitates some inferences but may not encompass the entirety of knowledge needed for more complex dietary decisions.

SEMANTIC NETWORKS FOR DYNAMIC PROBLEMS: THE GUARDS AND PRISONERS PROBLEM

We're now going to shift our attention from the structured domain of semantic networks for static problems to a different kind of challenge, one that's more narrative and dynamic in nature – the Guards and Prisoners problem. This classic conundrum is also known by a host of other names, such as the Cannibals and Missionaries problem or the Jealous Husbands problem. It has a storied history, first appearing in a mathematical textbook around the year 880 and has since been utilized extensively within AI to discuss problem representation.

Problem Definition and Constraints

Picture this scenario: Three guards and three prisoners find themselves on the bank of a river, and they need to get to the opposite shore. There is only one boat available, and it has a limited capacity, able to carry either one or two individuals at a time. The boat cannot cross the river by itself; it requires at least one person aboard to row it. A critical rule governs their crossing: at no point can prisoners outnumber guards on either bank, for if they do, the guards will be overpowered. Consequently, the number of guards present must always be equal to or greater than the number of prisoners at any given time on both banks.

These prisoners, we'll consider them to be cooperative for the sake of this problem – they won't attempt to escape when left unguarded. However, should they find themselves in greater numbers than the guards, they may seize the opportunity to overpower them. This is where the puzzle's complexity unfolds, presenting a real challenge that requires careful strategic planning to ensure the safety and successful crossing for all involved.

The guards and prisoners problem is a quintessential example of the types of scenarios we use to test the boundaries of problem-solving strategies. It's an exercise that pushes the limits of practical reasoning, forcing us to consider the implications of each move we make. This problem, while theoretical, underscores the importance of constraints in problem-solving and the need

for careful representation of situations to navigate through potential solutions effectively. Engaging with problems like this one not only sharpens analytical skills but also provides entertainment and a sense of accomplishment upon devising a successful strategy.

To effectively apply semantic network representation to the guards and prisoners problem, we envision each distinct scenario or 'state' of the problem as a node within the network. For instance, one node might represent the initial state: one guard and one prisoner are positioned on the left bank, while the boat and the remaining individuals are on the right bank.

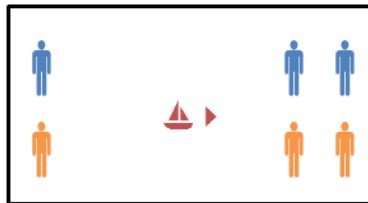


Figure 2-8 A State in Guards & Prisoners

This node not only captures the current state of affairs but also incorporates the lexicon of the semantic network. The next step involves detailing the structural component – the transformations. These are actions that transition one state to another, represented by links connecting the nodes. By labeling these links, we encode the semantics of the network, allowing us to derive meaningful inferences for problem-solving.

Consider a second node, depicting an alternate state where two guards and two prisoners have shifted to the left bank, with the boat also on the left, and the remaining guard and prisoner on the right bank. This forms another node in our semantic network.

To connect this node with the previous one, we create a link – using colored chevrons – that represents the transition or action taken, such as “two prisoners and one guard rowed to the left bank.” The labels on these links are critical as they articulate the specific actions and their effects on the problem's state, providing a narrative of the progression from one state to another.

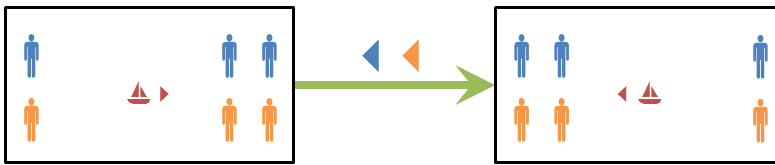


Figure 2-9 A Semantic Network for the Guards and Prisoners project consisting of two nodes, a link relating the nodes, and colored chevrons denoting their link labels

This semantic network grows as we add more nodes, each symbolizing a potential state of the problem. The links between these nodes illustrate the permissible moves within the constraints of the problem, such as never allowing the number of prisoners to exceed the number of guards on either bank. In our representation, we utilize icons to signify objects and actions, which is an effective and visually intuitive way to represent the states and transitions. Icons can often convey complex information succinctly and are especially useful when dealing with a problem that has a spatial or visual component, such as the guards and prisoners problem. By employing icons as labels rather than words in our semantic network, we achieve a balance between visual representation and information. This enhances the clarity of the network, making it a powerful tool for visualizing the problem space and facilitating strategic planning to reach the solution.

Using the Semantic Network to Prune Illegal/Unproductive Moves

Adhering to an old adage in AI, we find that with the right knowledge representation, problem-solving can indeed become significantly more straightforward. Applying this principle to our guards and prisoners problem, we've constructed a semantic network representation that captures the various states and potential actions within the problem. Let's examine how this representation aids in identifying effective problem-solving strategies.

We start at the first node, our initial state, where all three guards and three prisoners are together on the left bank with the boat. From here, the semantic network helps us discern five potential moves:

1. Moving only one guard to the right.
2. Moving a guard and a prisoner to the right.
3. Moving two guards to the right.
4. Moving two prisoners to the right.
5. Moving just one prisoner to the right.

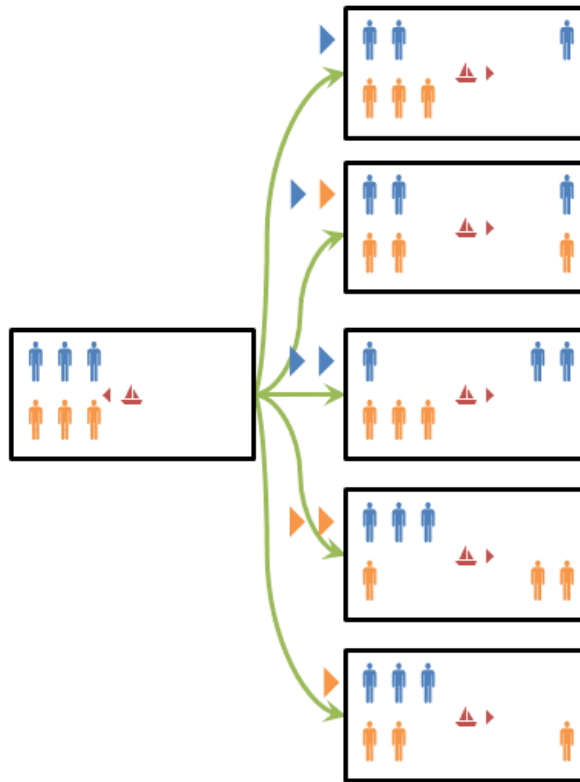


Figure 2-10 Potential next states for the Guard and Prisoners problem.

However, our network also allows us to evaluate the legality and productivity of these moves. For instance, the first and third moves violate the rule that prisoners must not outnumber guards on either bank. Therefore, these options can be immediately discarded as illegal.

Similarly, the fifth move, moving one prisoner to the right, is also unproductive. If executed, it would necessitate bringing the same prisoner back to the initial side to avoid isolation, effectively returning us to our starting point and making no progress.

This process of elimination leaves us with two viable moves that are both legal and productive, aligning with the constraints of our problem. This example illustrates how a well-constructed semantic network not only represents the problem but also guides us towards feasible solutions by allowing us to analyze and rule out unproductive or illegal options.³⁰

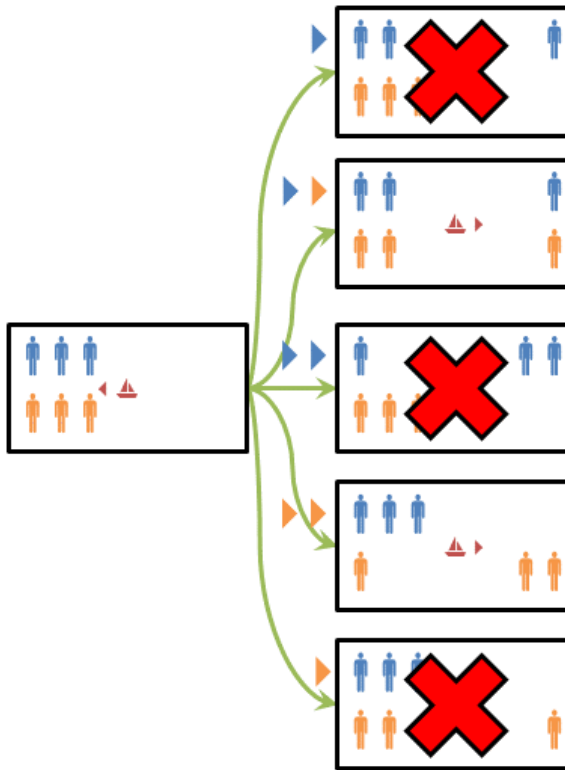


Figure 2-11 Legal and productive next states for the Guards and Prisoners problem.

In subsequent discussions, we'll explore how AI programs can leverage various methodologies to differentiate between productive and unproductive moves. For now, our focus remains on navigating through the problem space using the insights provided by our semantic network. This approach demonstrates the power of effective knowledge representation in simplifying complex problem-solving tasks.

Collaborative Problem Solving with Semantic Networks (Ashok & David build the network)

Ashok David, let's collaborate on building a semantic network for our problem. Can you identify the legal and productive states following our current ones?

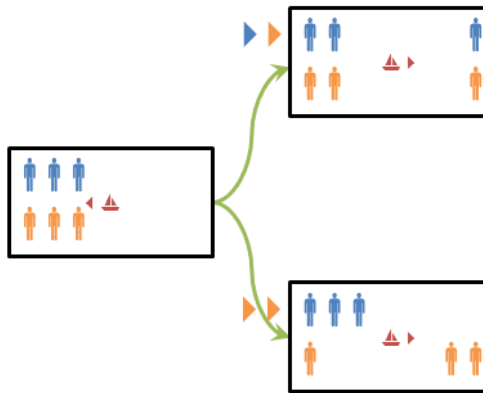


Figure 2-12 Guards and Prisoners problem after one state transition.

David Sure, Ashok. Since the boat moves from right to left, it will be on the left in the subsequent states. In the top state, our options are to move both individuals back, just the guard, or just the prisoner. Returning both would lead us to our starting point – legal but unproductive. Sending back only the prisoners would create an imbalance, making it illegal. Therefore, the only viable move is to send the guard back.

Similarly, in the bottom state, we can either move both prisoners or just one. Moving both again leads to the original state, which isn't productive. So, the only legal next state is moving one prisoner back to the left.

Ashok Excellent analysis, David. It's important to note that the two states you've identified are, in fact, identical. This means we can merge them into a single state in our network, simplifying our representation as shown in this figure.

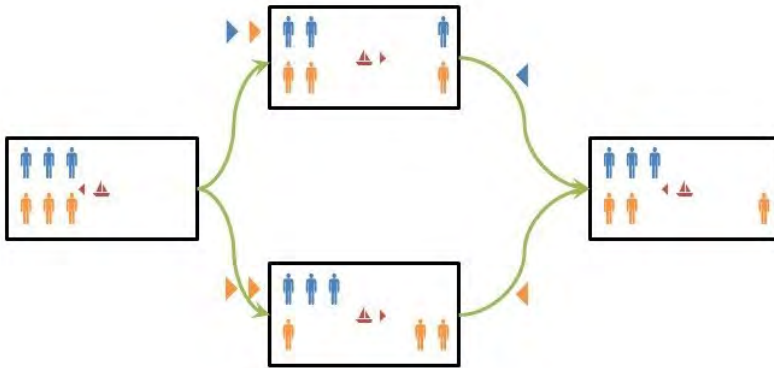


Figure 2-13 Guards and prisoners problem after two state transitions.

David Right, the key in this semantic network is not how we arrived at a state, but rather recognizing the state we're in. It simplifies the problem-solving process because we focus on the current state of the problem without worrying about the paths taken to reach it.

Ashok Let's extend our problem-solving approach. Now in this unified state, let's enumerate all potential legal moves that follow. Some may turn out to be unproductive, but for now, let's list them all. David, have you worked out these moves?

David Yes, similar to the initial state, there could be five potential moves. However, two are immediately illegal: moving two guards leaves more prisoners than guards on the left; and moving a guard and a prisoner results in too many prisoners on the right. So, we disregard those options. The remaining legal moves are moving one prisoner, two prisoners, or one guard.

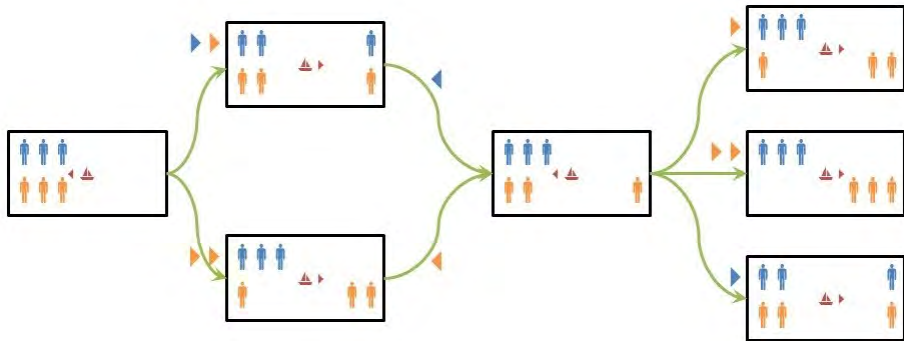


Figure 2-14 Legal fourth states for the Guards and Prisoners problem.

David

But upon closer examination, I realized that this state essentially mirrors a previous one. As we established, revisiting a previous state doesn't help us progress, making such moves unproductive. Thus, we can exclude them, leaving us with only one legal and productive move to follow from this state.

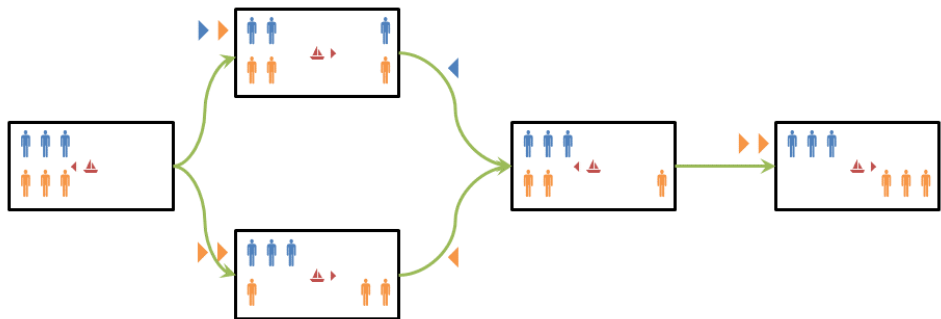


Figure 2-15 Legal and productive fourth state for the Guards and Prisoners problem.

Ashok

That's a crucial observation, David. This example illustrates how a well-structured semantic network aids in clarifying the constraints of the problem. It allows us to efficiently compare states and discern which moves are productive and which are not. This clarity is invaluable in problem-solving as it allows us to quickly determine which moves are productive and legal and which are not.

David Absolutely, Ashok. When I first attempted to solve this problem independently, I found myself going in circles, not realizing I was returning to the same state repeatedly.

Ashok That's a common challenge, David. The strength of using a semantic network in this context is its systematic approach. It lays out all the elements – the constraints, objects, relationships, and possible actions - in a clear, explicit manner, allowing us to systematically solve the problem. This structure doesn't just represent the problem; it actively guides us through the problem-solving process, helping us avoid unproductive paths and focus on viable solutions.

Ashok Now that we have a deeper understanding of this problem, let's consider extending our problem-solving approach to fully resolve the guards and prisoners scenario. While I won't go through the entire solution here due to its complexity and the space constraints, I encourage you to tackle it independently. Determine how many moves it takes to safely transport all guards and prisoners to the other side of the river. Once you've figured it out, note the number of moves it took you to achieve this.

David, what was your solution? How many moves did it take you?

David After working through it, I managed to solve the problem in 11 moves. While I won't detail every step right now, it's important to note that while 11 moves is the most efficient solution I found, there might be other combinations of 11 moves that could also solve the problem.

Ashok That's an excellent accomplishment, David. You've correctly identified the optimal solution. This exercise is not only a great way to challenge one's problem-solving skills but also a fun and engaging puzzle to share with others. Imagine demonstrating your strategy at a social gathering; it's a unique way to engage and entertain. However, we should recognize that identifying the most productive states and avoiding unproductive ones is a key challenge in AI problem-solving.

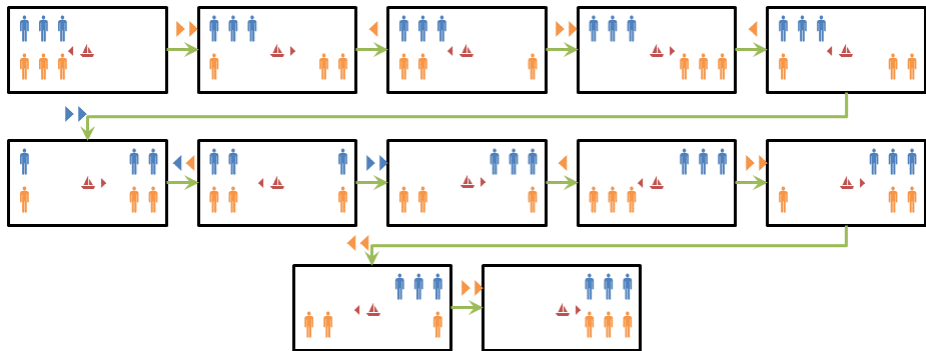


Figure 2-16 All legal and productive state transitions to solve the Guards and Prisoners problem.

This guards and prisoners problem is a classic example of how AI methods need to discern productive states from unproductive ones efficiently. In our upcoming chapters, we'll look deeper into how AI approaches these kinds of challenges, determining the most effective paths in complex scenarios.

REVISITING RAVEN'S ANALOGY PROBLEM WITH SEMANTIC NETWORKS

As we have observed the efficacy of semantic network knowledge representation in solving complex problems, let's revisit the earlier challenge involving the relationship of A to B and C to potential answer choices for D (Recall Figure 2-2). We've already established representations for A is to B. Now, the pivotal question is whether we can leverage this representation to determine the correct completion for C.

Comparing Transformations

Upon examining the detailed representations of both pair A to B, and C to a potential answer (e.g., choice 5), we notice similarities and differences. For example, if in the transformation from A to B, an object 'y' expanded, but in the transition from C to choice 5, a corresponding object 's' remains unchanged, this discrepancy suggests that choice 5 might not be the ideal match. The goal is to find an answer choice where the semantic network representation of C to that choice mirrors the representation of A to B, ensuring consistency in the pattern of transformation across both pairs. This approach demonstrates the nuanced application of semantic networks in reasoning and problem-solving, especially in scenarios requiring pattern recognition and matching.

Ashok

Let's embark on another exercise, David. This time, I've selected a different option, choice 2, for our Raven's problem. As we did before, we will build a representation for A is to B, and now for C is to 2. Could you label the links in these representations and decide if choice 2 is the correct answer for this problem? What do you think, David?

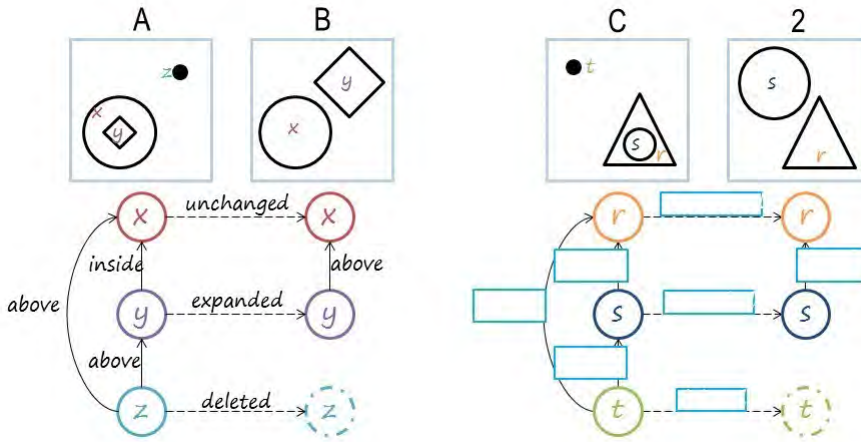


Figure 2-17 Semantic Network for C and Option 2.

David

After analyzing the representations, I believe choice 2 is indeed the correct answer. The relationships in frame A and frame C appear consistent, as do those in frame B and frame 2. In both A and C, x remains unchanged, which is mirrored by r in the latter frames. The expansion of y in A to B is reflected in the expansion of s in C to 2. Similarly, the deletion of z in the transition from A to B corresponds to the deletion of t in C to 2. Also, the spatial relationship of y being above x in B is analogous to s being above r in 2. Hence, I conclude that choice 2 is the correct answer.

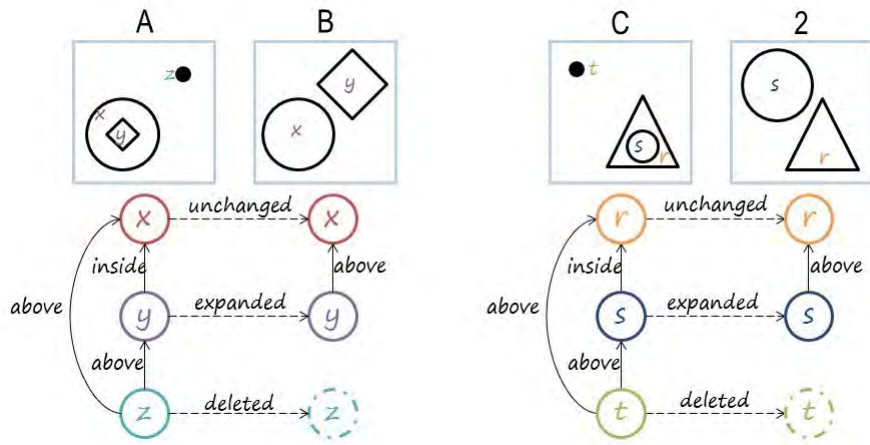


Figure 2-18 Completed semantic network for option 2.

Ashok Excellent reasoning, David. You correctly identified that choice 2 aligns best with the transformations seen in A and B. But let's consider a scenario where the match isn't as clear-cut, like when we considered choice 5. If the match between the representations were only partial, how would you approach choosing between two partially matching options?

David In cases where we don't have a perfect match, we'd need to quantify the similarity between transformations. For instance, we might evaluate the transformations and conclude that one pair has a 95% similarity while the other has only 70%. This kind of comparative analysis would help us decide which option is more closely aligned with the pattern we observe in A and B.

Ashok That's a reasonable approach, and it's something we'll explore in more depth soon. This kind of nuanced decision-making is crucial in problem-solving, especially when dealing with partial matches or incomplete information.

Quantifying Similarity (Weighted Transformations)

Ashok In this exercise, we're dealing with a situation where there isn't a single definitive answer. This reflects real-world scenarios where ambiguity often leads to multiple plausible solutions. Here we have the familiar A is to B, C is to D setup with six choices for D. David, based on this setup, which option do you think is correct?

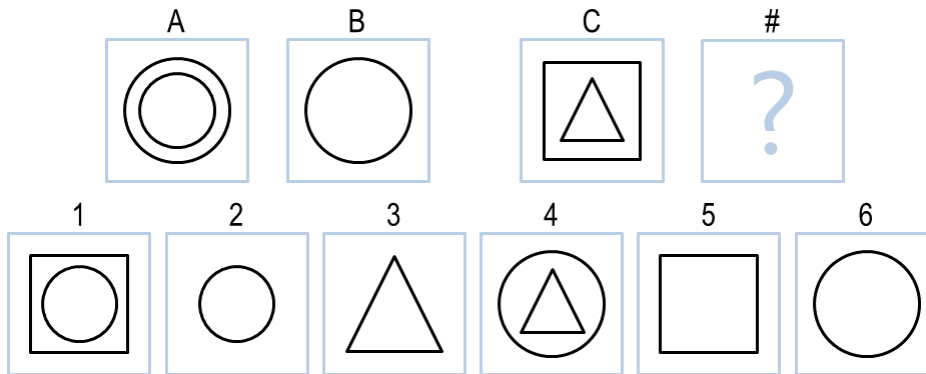


Figure 2-19 Ravens Analogy Problem 2. Options 3 and 5 are possible answers.

David This version of the problem is slightly different from the previous one we encountered. Previously, the square wasn't an option, and we chose the triangle by default. But now, with the square included as a choice, it seems the simpler transformation would be the disappearance of the middle shape. Therefore, the triangle vanishes, leaving us with just the square. I'd say that choice five is the correct answer.

Ashok That's a well-thought-out answer. However, it's important to note that choice three could also be viable. If we interpret the relationship between A and B differently, such as the outer circle disappearing and the inner circle expanding, we might end up with a different result. In that scenario, the outer square disappears and the inner shape expands, leading us to choice three as another legitimate answer.

David I see, so the ambiguity lies in how we interpret the transformations. While I leaned towards choice five, it's interesting that choice three could also be considered correct. But it does make me wonder, why would most people, including myself, be inclined to choose five over three?

Ashok That's an insightful observation, David. The preference for one answer over another in ambiguous situations often depends on how we perceive and process the information.

In addressing the question of why most people might choose to answer five over answer three in the Raven's problem, it's important to examine the mechanics of decision-making,

particularly in the context of ambiguous situations. The semantic network representation for the relationship between A and B can be viewed from different perspectives. For instance, one perspective might see q, the outer circle, as remaining unchanged, and p, the inner circle, as being deleted. Another perspective might interpret p as expanding and q as being deleted. Both interpretations are valid, which adds a layer of complexity to the decision-making process. To systematically evaluate these options, an AI agent could employ a metric that assigns weights to different kinds of transformations based on ease and similarity. Transformations such as scaling, rotation, and reflection could be rated on a scale where a higher value indicates easier transformation and greater similarity. Conversely, a lower value would suggest a more difficult transformation and less similarity.

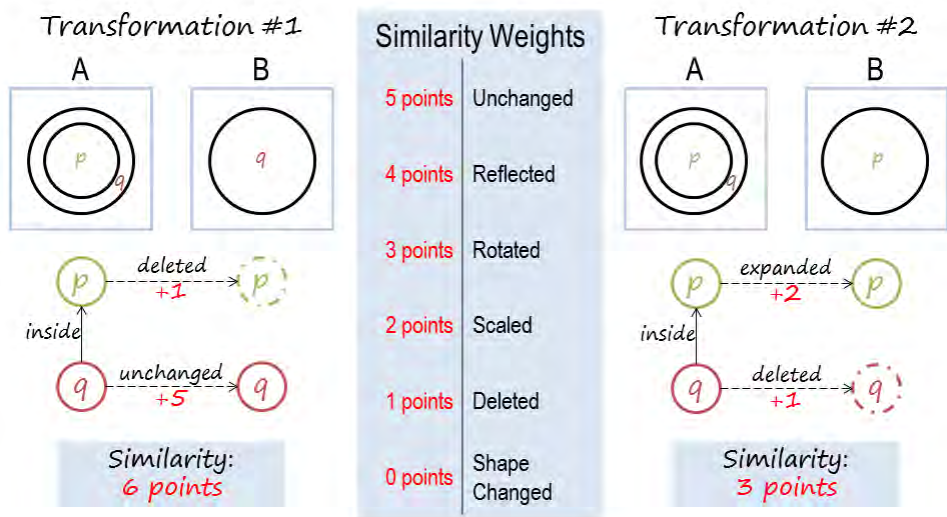


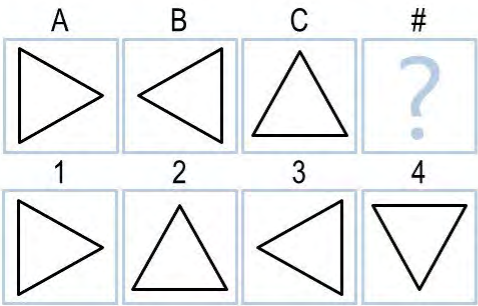
Figure 2-20 Semantic network transformation with imperfect matches.

For example, in transformation #1, where p is deleted (given a weight of 1) and q remains unchanged (given a weight of 5), the total weight would be 6. In transformation #2, with p expanding (weight of 2) and q being deleted (weight of 1), the total weight comes to 3. This weighted approach helps to quantify the ease and similarity of transformations, providing a more objective basis for comparison. In this framework, if one prioritizes the first transformation scenario over the second, it becomes clear why the square (choice five) might be preferred over the triangle. This methodology not only explains why choices 3 and 5 are both legitimate but also illustrates why an AI agent, using a similarity metric, might lean towards choice 5. Such a system of weighted evaluations is invaluable in AI problem-solving, particularly when navigating through problems where multiple interpretations or solutions are possible. By quantifying the

similarities and differences between various options, AI agents can make more informed and rational choices, even in the face of ambiguity.

This approach to problem-solving is reflective of a broader strategy in AI, where complex decisions are broken down into quantifiable components. By assigning weights to these components based on predefined criteria such as ease of transformation or similarity, AI systems can analyze and compare potential solutions in a structured, logical manner. This not only enhances the accuracy of the decision-making process but also provides a clear rationale for why one option may be favored over another. Understanding and applying these concepts of weighted transformations and similarity metrics are crucial for advancing AI's capability to deal with complex, ambiguous scenarios. It allows AI systems to move beyond binary, right-or-wrong answers and navigate the nuanced spectrum of possibilities that characterizes real-world problem-solving. In this case, the choice of five might seem more straightforward or intuitive based on the most apparent transformation pattern. However, when we look deeper and consider alternative interpretations, as with choice three, we realize the complexity and ambiguity inherent in the problem. This exercise highlights the importance of examining problems from multiple angles and considering various possibilities before settling on a solution, especially in scenarios where ambiguity is a significant factor.

Ashok To further cement our understanding of selecting the most fitting answer in ambiguous situations, let's re-examine a previous exercise. This version is a bit simpler, presenting only four choices instead of six. The similarity weights for each transformation are provided. David, considering these weights and choices, which do you believe is the right answer for this problem?



Similarity Weights	
5 points	Unchanged
4 points	Reflected
3 points	Rotated
2 points	Scaled
1 points	Deleted
0 points	Shape Changed

Figure 2-21 Ravens Analogy Problem 3.
Draft -Version:6/2/2025 3:06 PM

- David** Given the choices and the weights, I think the correct answer is number 2. It appears to align best with the transformation pattern we're observing.
- Ashok** Let's scrutinize this decision. Both choices 2 and 4 could be seen as valid. Choice 2 is plausible if we interpret the transformation from A to B as a reflection around the vertical axis. Applying the same logic to C and D would lead us to choice 2. On the other hand, choice 4 could be correct if we consider the transformation from A to B as a 180-degree rotation. Rotating C similarly would result in choice 4. However, our assigned weights play a crucial role here. We've rated reflection higher than rotation, suggesting that reflection is a more likely or easier transformation in this context. Therefore, given these considerations, David's choice of 2 is indeed the correct one according to our weighted criteria.

In revisiting the exercises, particularly the nuances of the Ravens Analogy and the Guards and Prisoners problems, we gain insight into the varied approaches to object representation, relationships, and transformations. This diversity is further complemented by the implementation of weighted systems, which play a crucial role in enhancing decision-making precision in AI. In the Ravens Analogy, we observed a unique approach to object representation. Objects were depicted as circles, each with internal labels to identify different elements. Relationships between these objects were shown using vertically oriented arrows, each labeled to describe the specific nature of the connection. Transformations, or changes from one state to another, were represented through horizontally aligned arrows, each carefully labeled to clarify the nature of change or action. Conversely, the Guards and Prisoners problem used a more literal form of representation. Here, objects (guards and prisoners) were symbolized by distinct icons or figures. Their relationships were portrayed through their spatial placement within a rectangular shape, symbolizing the riverbank or boat. The transformations, such as the movement of guards and/or prisoners, were represented by right-pointing horizontal arrows with directional chevrons, visually conveying movement and direction. The importance of weighted systems in these exercises cannot be overstated. They provide a structured way to evaluate and prioritize different transformations, enhancing the AI's ability to navigate through scenarios with multiple valid interpretations. By assigning weights to various transformation options based on criteria like ease and similarity, AI systems can make more informed and rational decisions, even in the face of ambiguity or complex visual patterns. This adaptability, combined with the precision afforded by weighted systems, underscores the sophistication and effectiveness of AI approaches in problem-solving. These strategies not only tackle immediate challenges but also significantly contribute to understanding how AI can mimic, interpret, and interact with human cognition and problem-solving techniques.

SEMANTIC NETWORKS AND COGNITION

Semantic networks offer valuable insights into human cognition:

- **Memory Organization:** Semantic networks can model how knowledge might be organized and accessed in human memory.
- **Correspondence Problem:** They highlight the correspondence problem – determining corresponding elements between different situations – which is central to analogical reasoning.
- **Focus on Relationships:** KBAI emphasizes relationships between objects, not just their individual properties, mirroring how humans understand context.
- **Spreading Activation Networks:** The way nodes activate and spread activation to connected nodes in a semantic network is similar to the concept of spreading activation in human memory models, where recalling one concept can trigger related concepts.³¹ For instance, hearing a story about a particular topic might activate related concepts in one's mind, even if those concepts weren't explicitly mentioned. This mechanism is akin to how semantic networks function, where activating one node can lead to the activation of connected nodes, forming a pathway of related information.

The study of semantic networks in AI advances our understanding of AI and sheds light on the intricacies of the human mind.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

Semantic networks, while a classical KBAI representation, have important connections and relevance to modern AI paradigms:

- **Knowledge Graphs:** Large-scale semantic networks are often referred to as **knowledge graphs**. These are used extensively in search engines (like Google's Knowledge Graph), recommendation systems, and by LLMs to ground their responses in factual data.
- **Graph Neural Networks (GNNs):** Deep learning has embraced graph-structured data. GNNs are neural networks designed specifically to learn from data represented as graphs, such as semantic networks or knowledge graphs. They can learn node embeddings, predict links, and perform graph-level classification, enabling ML on KBAI-style representations.
- **Embeddings for Nodes and Relationships:** Techniques from NLP and deep learning, like Word2Vec or more specialized graph embedding methods, are used to create dense vector representations (embeddings) of nodes and relationships in semantic networks. These embeddings capture semantic similarities and can be used for tasks like link prediction or analogy detection within the network.

- **LLMs for Populating or Querying Knowledge Graphs:** Large Language Models (LLMs) can be used to automatically extract entities and relationships from text to populate knowledge graphs. Conversely, knowledge graphs can provide structured, factual information to LLMs, helping to ground their outputs and reduce hallucinations. LLMs can also be used to translate natural language queries into formal queries over a knowledge graph.
- **Semantic Web and Linked Data:** Semantic networks are the foundational data model for the Semantic Web, where data is published on the web in a machine-readable format using technologies like RDF (Resource Description Framework) and OWL (Web Ontology Language), which are formalisms for defining semantic networks.
- **Agentic AI:** AI agents often build internal models of their world and the relationships between entities within it. Semantic networks provide a natural way to represent this knowledge, supporting the agent's reasoning, planning, and decision-making processes.

These connections show that the principles of semantic networks continue to be relevant, often forming the backbone or a complementary component in more complex, modern AI systems.

CHAPTER SUMMARY

This chapter introduced semantic networks as a cornerstone of knowledge representation in KBAI. We began by defining knowledge representation itself, emphasizing its core components: a language for expression and the content being expressed. We then focus on semantic networks, illustrating their structure through nodes (representing concepts or objects) and labeled, directed links (representing relationships).

The power of semantic networks was demonstrated through practical examples. In solving Raven's Progressive Matrices, we saw how semantic networks can represent objects, their internal relationships, and the transformations between different problem states. This highlighted their utility in visual reasoning and pattern recognition. Furthermore, the Guards and Prisoners problem showcased how semantic networks can model dynamic scenarios, representing states and permissible transitions, and aiding in pruning the search space by identifying illegal or unproductive moves. The discussion also touched upon using weighted transformations to handle ambiguity and quantify similarity in analogical tasks.

Throughout these examples, the characteristics of good knowledge representations were emphasized: making relationships explicit, exposing natural constraints, bringing objects and relations together, excluding extraneous details, and being transparent, concise, complete, fast, and computable.

Finally, we explored the cognitive relevance of semantic networks, noting parallels with human memory organization, the challenges of the correspondence problem in analogical reasoning, the importance of focusing on relationships, and the concept of spreading activation. The chapter also briefly outlined the connections between semantic networks and modern AI

paradigms like knowledge graphs and graph neural networks, underscoring their enduring significance. Ultimately, semantic networks provide a robust framework for AI systems to organize knowledge, reason about it, and interact with the world in an intelligent manner.

ADDITIONAL READINGS & REFERENCES

- 24 Ji, S., Pan, S., Cambria, E., Marttinen, P. & Yu, P. S. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE transactions on neural networks and learning systems* **33**, 494-514 (2021).
- 25 Kunda, M. AI, visual imagery, and a case study on the challenges posed by human intelligence tests. *Proceedings of the National Academy of Sciences* **117**, 29390-29397 (2020).
- 26 Hogan, A. *et al.* Knowledge graphs. *ACM Computing Surveys (Csur)* **54**, 1-37 (2021).
- 27 Aronoff, M. & Rees-Miller, J. *The handbook of linguistics*. (John Wiley & Sons, 2020).
- 28 Pustejovsky, J. The semantics of lexical underspecification. *Folia linguistica* **51**, 1-25 (2017).
- 29 Geeraerts, D. Lexical semantics. *Dalam Oxford Research Encyclopedia of Linguistics, oleh Dirk Geeraerts*. Oxford: Oxford University Press. <https://doi.org/10.1093/acrefore/9780199384655.013.29> (2017).
- 30 Lee, G. *et al.* 406-422 (Springer Nature Switzerland).
- 31 Herrmann, M., Ruppin, E. & Usher, M. A neural model of the dynamic activation of memory. *Biological cybernetics* **68**, 455-463 (1993).

Chapter 3

Generate and Test

INTRODUCTION TO GENERATE AND TEST

In this chapter, we transition from the study of knowledge representations, such as semantic networks, to focus on problem-solving methods in Knowledge-Based Artificial Intelligence (KBAI). Central to our discussion is a fundamental and versatile method known as **Generate and Test**. This approach is straightforward yet powerful in addressing a wide array of problems.

The essence of the generate and test method lies in its two-step process. First, given a problem, it involves generating a range of potential solutions. Then, these solutions are rigorously tested to evaluate their efficacy in resolving the problem at hand. This method assumes a scenario where complete and accurate knowledge of the world is not always available, computational resources are finite, and not all reasoning methods are infallible. Under such realistic conditions, generating plausible solutions and then testing them becomes a practical and often necessary approach.

The generate and test method, in its simplicity, holds significant importance in KBAI. It represents a strategy that acknowledges and adapts to the limitations in knowledge, computational capacity, and the accuracy of reasoning methods. In this chapter, we will explore how this method can be effectively employed in conjunction with semantic networks, building on the example of the guards and prisoners problem discussed in the previous chapter. This exploration will provide a deeper understanding of how generate and test functions as a critical

tool in the problem-solving arsenal of AI, offering insights into its applicability, strengths, and limitations in various scenarios.

THE GENERATE AND TEST METHOD IN PROBLEM-SOLVING

Knowledge-based Artificial Intelligence (KBAI) encompasses three primary components: knowledge representations, problem-solving techniques, and architectures. While we have previously explored knowledge representation, particularly through semantic networks, our focus has not yet extended to problem-solving methods or architectural considerations in this specific context. In this section, we aim to bridge this gap by introducing and illustrating the problem-solving method known as “generate and test,” revisiting the Guards and Prisoners problem.

Simple Generators and Testers

To contextualize this method, let's revisit the examples previously examined under semantic networks. In those instances, we identified various states within a given problem and selectively pruned some. However, the process by which an AI agent determines which states to eliminate wasn't explicitly addressed. Imagine a generator tasked with creating all possible successive states from an initial or current state. For illustrative purposes, consider this generator to be rudimentary in its functionality — it indiscriminately generates a comprehensive array of potential states.

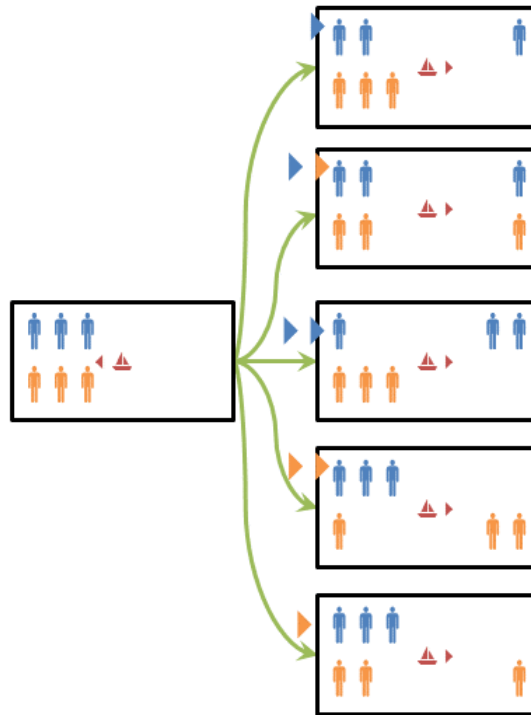


Figure 3-1 Generation of all possible next states.

The essence of the “generate and test” method extends beyond mere generation. It incorporates a critical testing phase. This tester scrutinizes all states produced by the generator, identifying, and discarding those that don't align with the problem's constraints. In our simplified scenario, the tester operates at a basic level, eliminating only those states that blatantly contravene the problem's rules. For example, in scenarios where prisoners outnumber guards on either riverbank — a clear violation of the problem's stipulations — the corresponding states are promptly dismissed by the tester.

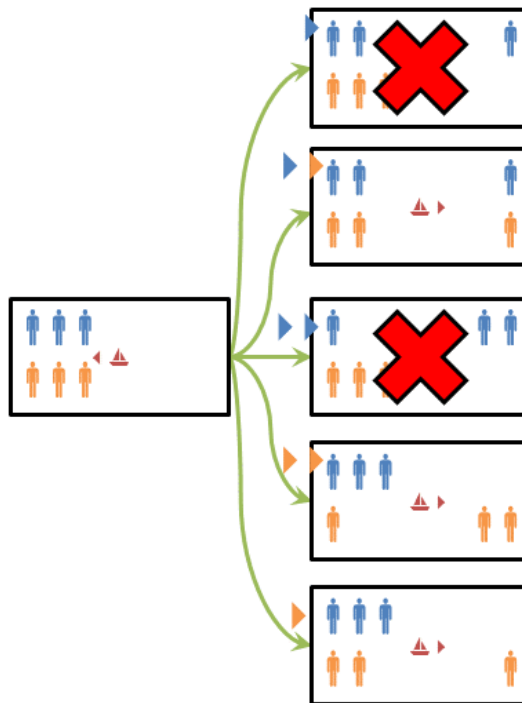


Figure 3-2 Possible next states with illegal states identified.

This Generate and Test approach provides a structured yet adaptable framework for problem-solving in KBAI. It highlights the dynamic interplay between the generation of possibilities and their subsequent evaluation, a process central to navigating complex problem landscapes.

Iterative Application and Combinatorial Explosion

Let's join Ashok and David as they discuss the intricacies of the generate and test method through a practical exercise. The exchange explores how a simple generator and tester model can evolve and improve problem-solving efficiency.

Ashok Let's advance our exploration of generate and test with another step. We now have three successor states. David, can you identify what next states a basic generator might produce from these?

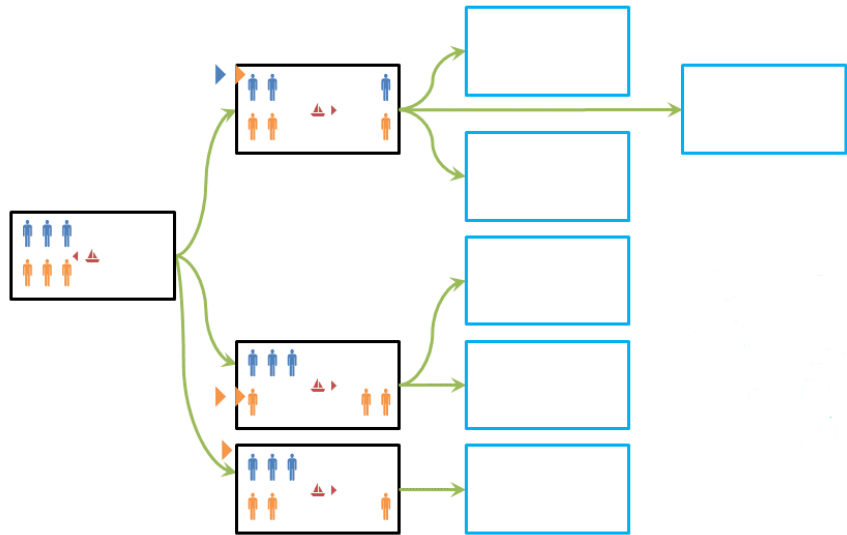


Figure 3-3 Second-level legal states suggestive of third-level state possibilities.

David From the top state, we could have three new states: moving both the guard and the prisoner, moving just the prisoner, or just the guard. From the middle state, the options are moving one or two prisoners. And from the last state, the only action seems to be moving the prisoner back to the left.

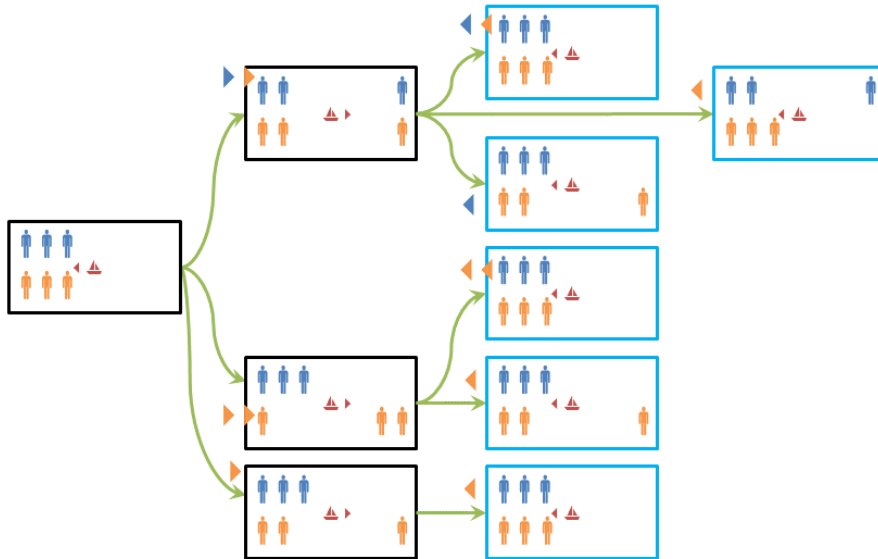


Figure 3-4 Generated legal third-level states.

Ashok Remember, David isn't creating these states himself; he's predicting what a simple generator might produce. So, with these newly generated states, what would a simple tester eliminate? David, what are your thoughts?

David The only state that violates our rule – more prisoners than guards on a shore – is this the one of the far right. So, it's the only state the simple tester will discard. The remaining five states are legal as they abide by the rules as understood by our simple tester. But, after elimination, we notice that we are left with just two unique states – everyone is on the left coast, or we have almost everyone on the left coast except for one prisoner located on the right coast.

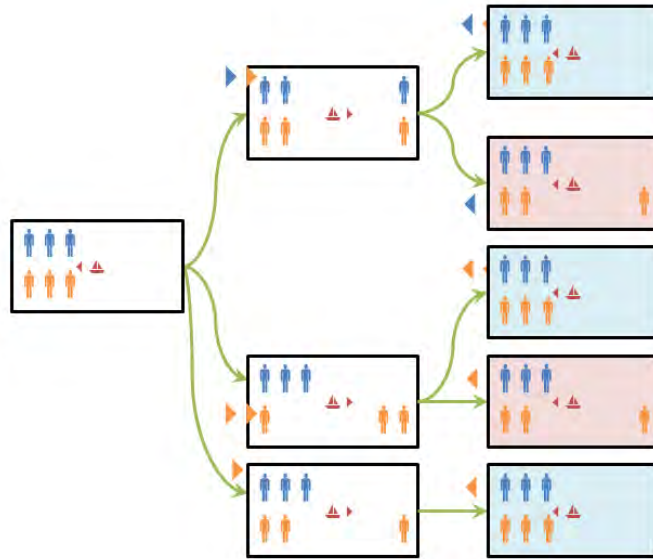


Figure 3-5 Generated legal next state with repeated states identified by color.

David

With this knowledge, we can collapse and simplify the state space to those two states alone.

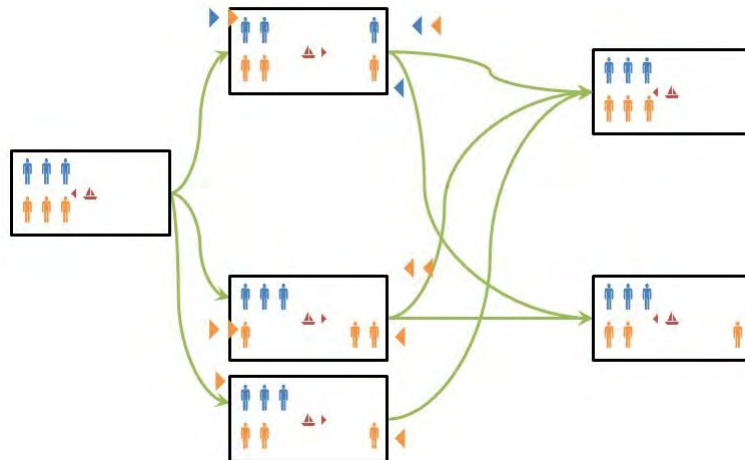


Figure 3-6 Generated legal next state with repeated states consolidated.

Ashok This process illustrates how we can iteratively apply the generate and test method. Applying it to these two states yields many successor states, a classic example of **combinatorial explosion**, where one starts with a small number of states and the number of successor states increases rapidly.

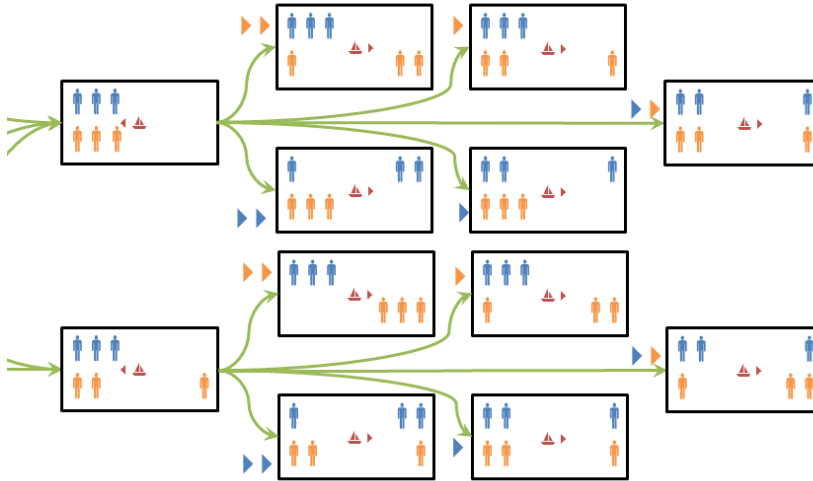


Figure 3-7 Combinatorial Explosion from 2 third-level states to many fourth-level states.

Ashok This explosion didn't occur with semantic networks because here we're dealing with repeated states, like our initial state, due to our unsophisticated generator and tester.

The root of this combinatorial explosion lies in our basic setup: we are using a simplistic generator and tester. This is exemplified by states that replicate the initial state, such as having all three guards and three prisoners on one side of the bank, yet these states are not pruned away due to the lack of sophistication in our tools. Despite this, if we were to apply the generate and test method iteratively with these basic tools, we could eventually reach the goal state. However, finding the path to this goal would be marked by computational inefficiency, a direct consequence of the simplistic nature of our generator and tester.

This observation brings us to a pivotal question: is it possible to enhance the efficiency of this problem-solving method by developing a smarter generator and tester? Such an advancement could significantly streamline the process, reducing the computational load and potentially lead to quicker, more efficient solutions.

It's important to recognize, however, that even in its basic form, the generate and test method is an incredibly potent tool in problem-solving. Its ability to methodically explore a wide

array of possible states and systematically test them against defined criteria makes it a cornerstone technique in Knowledge-Based Artificial Intelligence. The prospect of refining this method by introducing more intelligent generators and testers opens exciting possibilities for more effective problem-solving strategies in AI.

ENHANCING GENERATE AND TEST

Smart Testers

Introducing a more sophisticated tester into the generate and test method can significantly enhance our problem-solving approach. This advanced tester can identify states that replicate those previously encountered. Consequently, it can efficiently dismiss redundant states, such as those identical to the initial setup, as well as states that violate the constraints of our problem – for instance, scenarios where the number of prisoners exceeds that of guards on any riverbank.

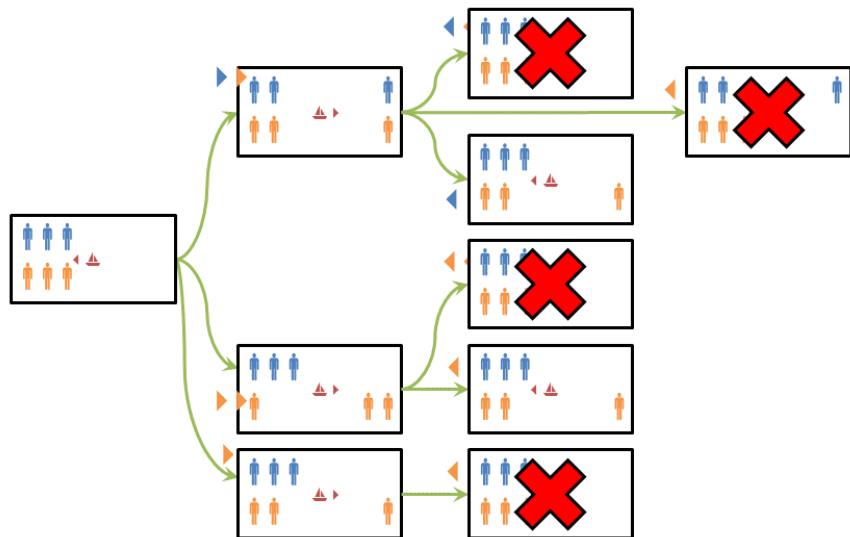


Figure 3-8 Redundant and illegal states identified during smart testing.

This strategic dismissal leads us to a more streamlined and focused exploration of potential solutions. However, we observe that certain states, despite being legal, lead to dead ends with no productive successor states. By identifying and merging identical states and removing illegal states, we arrive at a configuration remarkably like what we encountered while using semantic networks. This parallel draws attention to an important aspect of problem-solving in

Knowledge-Based Artificial Intelligence: the interplay between knowledge representation and problem-solving methods.

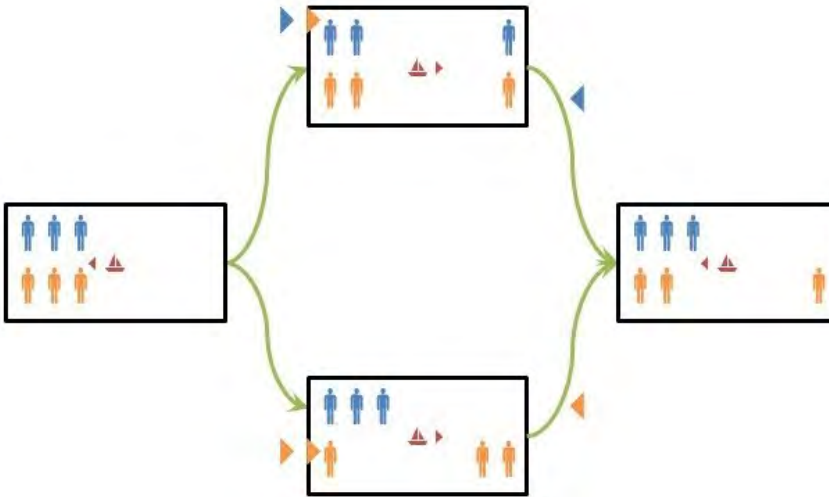


Figure 3-9 Simplified state diagram using a Smart Tester.

A striking revelation emerges from this approach: the configuration of states we end up with closely mirrors what we encountered with semantic networks. This similarity underscores a key insight – while knowledge representations like semantic networks are highly valuable, they are not standalone solutions. Semantic networks alone do not solve problems. Their true power is unlocked when coupled with an effective problem-solving method, such as generate and test.

In KBAI, the synergy, or coupling, between knowledge representation and problem-solving techniques is fundamental. Semantic networks, for instance, provide a structured form of knowledge that, when paired with the generate and test method, leads to efficient and effective problem-solving. Initially, our approach utilized a basic generator complemented by a more discerning tester, capable of identifying and eliminating repetitive states. Moving forward, the potential of enhancing the generator's capabilities opens up new avenues for even more efficient problem-solving, marking a significant step in the evolution of AI problem-solving strategies.

Smart Generators

Let's refine the generate and test methodology, focusing on enhancing the generator's capabilities. A sophisticated generator preemptively filters out states that either replicate the initial state or have been previously explored, streamlining the problem-solving process.³² This strategic approach minimizes the workload for the tester, who then only needs to evaluate a more targeted set of potential states.

The intelligence of the generator plays a pivotal role, particularly in complex scenarios. For example, in our current problem, an advanced generator not only avoids creating redundant states but also proactively identifies and excludes states that contravene the problem's rules, such as those violating the constraint of maintaining a balance between guards and prisoners.

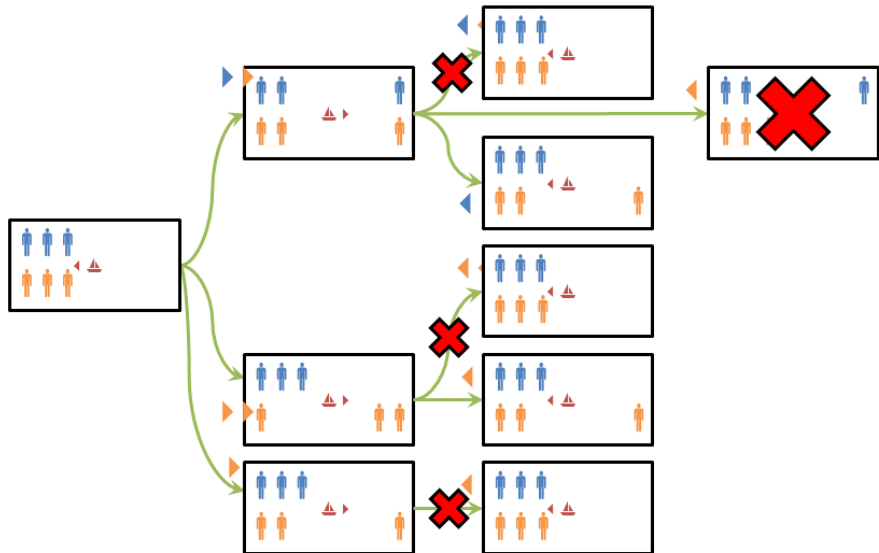


Figure 3-10 Third layer of states using a Smart Generator, recognizing duplicate and illegal states and self-pruning.

The relationship between the generator and tester is akin to a dynamic tree, where the branches represent the potential states generated. The complexity and spread of these branches hinge on the sophistication and knowledge embedded within the generator. This is especially crucial in complex problems involving a vast array of potential states, where the balance between an astute generator and an efficient tester is key to effective problem-solving.

Connection to Genetic Algorithms

This refined approach is not unique to our discussion but resonates with methodologies like genetic algorithms in broader AI applications. These algorithms function as a sophisticated generator, creating a multitude of potential states based on simple yet powerful recombination rules. Here, the generate part of the methodology is embodied in the algorithm's ability to produce a diverse range of states, each representing a unique solution or approach to the problem at hand. The test aspect is ingeniously handled by what is known as a fitness function. This function serves as a smart tester, evaluating each generated state for its effectiveness or

suitability in solving the given problem. It measures how 'fit' or appropriate a solution is, based on predefined criteria, and filters out less suitable options.

Genetic algorithms are particularly effective in tackling large-scale and complex problems, as they can navigate through a vast solution space to identify optimal or near-optimal solutions.^{33,34} However, while they are adept at generating a wide array of solutions, the fitness function, acting as a tester, might not always capture the nuanced requirements of more intricate problems.³⁵ This highlights a persistent challenge in AI: developing more nuanced and intelligent tools for problem-solving.

Balancing Responsibility between Generator and Tester

Let's rejoin David and Ashok as they examine the assignment of responsibility between a smart generator and a smart tester.

Ashok Recall that we had this state as a kind of hanging state (see Figure 3-10, far right illegal state). It had no successor state. And we said that, well this state can be dismissed, but here is a question. Who will dismiss it, the generator or the tester? What do you think, David?

David So I think the tester would do it. We could have the tester basically say any state where there is only one person on the side of the river that has the boat can be pruned off because the only way to get into that state is to send that person over. And the only way to get out of that state is to undo the previous move. So my rule would be for the tester, any time only one person is on the side of the river with the boat, throw out that state.

Ashok David is right. The tester could, in fact, test and dismiss the state using the kind of rule David was talking about. But, David, do you think there is another way that this could be done?

David So, I guess, actually, the generator could do it as well. So, in this case, we could also have the generator say, I don't need to ever generate a state that involves sending one person over to an empty coast. That's always going to result in a state that the tester would have thrown out. And it's not going to result in any states that the tester wouldn't have thrown out. So, the generator could do it, too.

Ashok That sounds like a good answer to me. So once again, we are back to the issue of where do we draw the balance of responsibility between the generator and the tester? The important thing to note from here, however, is that generate and test, when endowed with the right kind of knowledge, can be a very powerful method.

GENERATE AND TEST FOR RAVEN'S PROBLEMS

Let's revisit our problem from the intelligence test to understand how the generate and test method applies as a problem-solving strategy. Recall the problem we previously encountered.

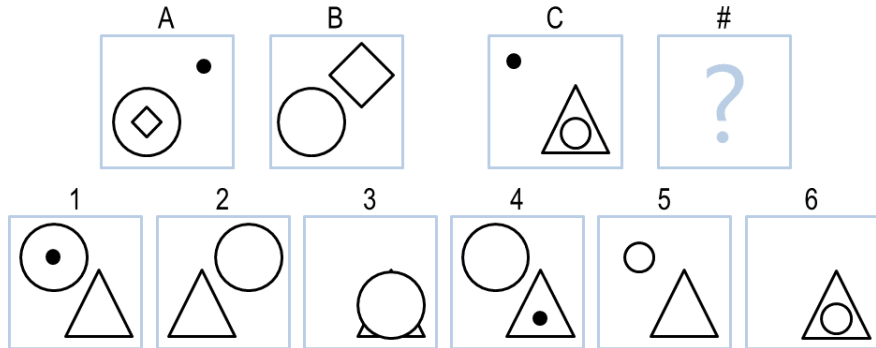


Figure 3-11 Raven's Intelligence Test Example.

This problem is more complex than the guards and prisoners scenario due to the nature of its transformations. In the guards and prisoners problem, each transition from one state to another was discrete and clearly defined, involving specific number of guards and prisoners and their associated movements. However, this new problem poses a different challenge.

In this problem, if we observe the transformation between A and B, we notice that the diamond inside the circle has moved outside and increased in size. Now, if we attempt a similar transformation from C to D, the situation becomes more intricate. The circle inside the triangle, when moved outside, can be placed at various distances from the triangle and altered in size in multiple ways. This variability introduces a range of possibilities that must be considered.^{36,37} This establishes the need for a smarter generator and tester, otherwise the space of potential states can become very large, very quickly.

This is where the knowledge representation of semantic networks becomes invaluable. It provides an abstraction level that simplifies the problem's representation and analysis. Although the specific placement and size of the diamond 'y' could vary, the semantic network focuses on the critical aspect: 'y' gets expanded, and that is what matters. This abstraction is powerful because it allows the knowledge representation to ignore minute details, simplifying the problem-solving process.

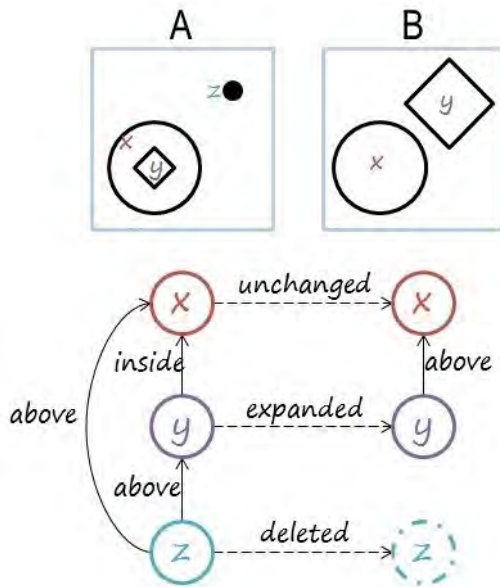


Figure 3-12 Semantic Network for the Ravens Intelligence Example Problem.

The key takeaway here is that a knowledge representation does not solve a problem in isolation, nor does a problem-solving method independently resolve issues. It's the combination of a suitable knowledge representation and an effective problem-solving method, like generate and test, that leads to successful reasoning and solutions. This coupling is what enables the AI to process and analyze problems at a higher, more abstract level, focusing on what's essential and disregarding extraneous details.

In this next exchange, Ashok and David explore the application of the generate and test method in conjunction with semantic networks for solving a specific class of problems. Let's join their conversation:

Ashok So, let's say we're utilizing a semantic network for representing this type of problem. David, how would you apply the generate and test method here?

David If I understand correctly, I would use the transformation observed between A and B, apply that same transformation from C to generate a possible solution for D. Then, I would test my generated solution for D against the options 1 through 6 to see which one aligns most closely. To enhance the accuracy, I could implement a more intelligent tester and generator. This would involve setting a confidence threshold for the answer to match the generated solution. If it doesn't meet this threshold, the system would reevaluate to determine if an alternative

transformation could be more appropriate. This approach would help address situations like the earlier problem, where either the middle shape or the outer shape could be the key to the transformation.

Ashok That's an insightful approach, David. It represents an alternative method for tackling this problem using generate and test along with semantic networks. One could systematically test each option, say option 1, by placing it under D, generating the transformation from C to D, and then testing this against the transformation observed from A to B. The same process could be repeated with option 2 and so forth for all six choices. This would help in determining which transformation most closely matches the one from A to B. Therefore, in this scenario, the generate and test method can be employed in various ways, all while utilizing the same knowledge representation framework. It's important to remember that a knowledge representation captures details about the world at a certain level of abstraction and can be paired with different problem-solving methods or variations of a single method.

COGNITIVE CONNECTION

Generate and test is a fundamental method in problem-solving that is not only prevalent in AI but also mirrors human cognitive processes. Humans frequently rely on this method due to our limited knowledge of the world, finite computational resources, and the absence of infallible reasoning methods. We naturally generate potential solutions and test them against real-world scenarios. This method's relevance extends beyond human cognition, as exemplified in Genetic Algorithms. Inspired by biological evolution, Genetic Algorithms use processes like crossover and mutation to create and evaluate solutions against a fitness function. This demonstrates how generate and test is not only intertwined with human thought processes but also with the principles of biological evolution.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

The Generate and Test paradigm, while a foundational KBAI method, finds echoes and applications in several modern AI fields:

- **Reinforcement Learning (RL):** An RL agent often generates actions (or sequences of actions) based on its current policy. The environment then tests these actions by providing a reward signal. The agent learns by iteratively generating actions and observing their tested outcomes, refining its policy to maximize rewards. This is a continuous cycle of generation (action selection) and testing (environmental feedback).

- **Generative Models (e.g., GANs):** In Generative Adversarial Networks (GANs), a generator network creates synthetic data samples (e.g., images). A discriminator network then tests these samples, trying to distinguish them from real data. The generator learns to produce more realistic samples based on the feedback (test results) from the discriminator.
- **Beam Search in NLP:** When generating sequences in tasks like machine translation or text summarization, models often use beam search. At each step, the model generates a set of most probable next words (the beam). These partial sequences are then tested (e.g., scored by their probability or other heuristics), and only the top-k candidates are kept for further generation. This iterative generate-and-test (or generate-and-prune) process helps find high-quality output sequences without exploring the entire exponential search space.
- **Monte Carlo Tree Search (MCTS):** Widely used in game playing (e.g., AlphaGo), MCTS involves generating possible game move sequences through simulations (rollouts). These simulated games are then tested by evaluating their outcomes (win/loss). The results of these tests are used to update the values of game states and guide future generation (exploration vs. exploitation) of moves.
- **Hypothesis Generation and Testing in Scientific Discovery:** AI systems for scientific discovery might generate hypotheses based on existing data and theories. Modern approaches increasingly explore the use of Large Language Models (LLMs) to generate a broad range of initial hypotheses by synthesizing information from vast textual corpora. These AI-generated hypotheses, whether from traditional systems or LLMs, are then tested through simulated experiments, real-world experiments, or by checking consistency with further data.

While the specific mechanisms differ, the core idea of proposing candidates (generation) and evaluating their fitness or correctness (testing) is a powerful and recurring pattern across many advanced AI systems. KBAl's explicit formulation of Generate and Test provides a conceptual basis for understanding these more complex, often data-driven, instantiations.

CHAPTER SUMMARY

This chapter introduced Generate and Test as a fundamental problem-solving method in KBAl. We defined its core two-step process: first, the generation of potential solutions or states, and second, the testing of these candidates against problem constraints or goals. Its versatility allows it to be applied across various problems, especially when knowledge is incomplete or computational resources are limited.

We explored the practical application of Generate and Test using the Guards and Prisoners problem, illustrating how even simple generators and testers can find solutions, albeit sometimes inefficiently due to combinatorial explosion. The discussion then progressed to enhancing Generate and Test by introducing “smart” generators (which preemptively filter candidates) and

Draft -Version:6/2/2025 3:06 PM

“smart” testers (which apply more sophisticated evaluation criteria, such as checking for redundant states). This highlighted the crucial balance and interplay between the generator and tester in optimizing the search process. The connection to Genetic Algorithms was made, showing how principles of generation (recombination) and testing (fitness function) are mirrored in evolutionary computation.

The chapter also demonstrated how Generate and Test can be coupled with knowledge representations like semantic networks to solve problems such as Raven's Progressive Matrices. Here, transformations are generated based on observed patterns and then tested against answer choices, or vice-versa. This showcased the method's adaptability and its reliance on good representations for abstract problems.

A key theme was the cognitive connection: Generate and Test mirrors human problem-solving strategies, where we often propose ideas and evaluate them. This fundamental nature makes it a recurring concept, implicitly or explicitly, in more advanced AI techniques, including those in modern machine learning like RL and generative models.

Ultimately, Generate and Test stands as a foundational KBAI technique, illustrating a core loop of proposing and evaluating that is essential for intelligent behavior and forms a building block for more complex reasoning and learning systems discussed later in this book.

ADDITIONAL READINGS & REFERENCES

- 32 Methni, A., Hedia, B. B., Lemerre, M., Haddad, S. & Barkaoui, K. in *9th Workshop on Verification and Evaluation of Computer and Communication Systems, VECoS 2015*. 65-75 (CEUR-WS).
- 33 Katoch, S., Chauhan, S. S. & Kumar, V. A review on genetic algorithm: past, present, and future. *Multimedia tools and applications* **80**, 8091-8126 (2021).
- 34 Shukla, A., Pandey, H. M. & Mehrotra, D. in *2015 international conference on futuristic trends on computational analysis and knowledge management (ABLAZE)*. 515-519 (IEEE).
- 35 Liashchynskiy, P. & Liashchynskiy, P. Grid search, random search, genetic algorithm: a big comparison for NAS. *arXiv preprint arXiv:1912.06059* (2019).
- 36 Wang, K. & Su, Z. in *Proceedings of the 24th International Conference on Artificial Intelligence*. 903-909.
- 37 Kunda, M., McGregor, K. & Goel, A. K. in *AAAI Fall Symposium: Multi-Representational Architectures for Human-Level Intelligence*.

Chapter 4

Means-Ends Analysis and Problem Reduction

INTRODUCTION

In this chapter, we explore two pivotal problem-solving methods in Knowledge-Based Artificial Intelligence (KBAI): Means-Ends Analysis and Problem Reduction. These methodologies are particularly effective for tackling well-defined problems, offering structured and strategic approaches to finding solutions. Alongside Generate-and-Test, these methods, in conjunction with Semantic Networks as a fundamental knowledge representation framework, form the backbone of core topics in KBAI.

We start our exploration with an understanding of state spaces. State spaces represent all possible states or configurations a problem can assume. Grasping the concept of state spaces is vital for the effective application of any problem-solving method.

Next, we examine Means-Ends Analysis.³⁸ This strategy focuses on identifying and reducing the differences between the current state and the goal state. It's a process akin to navigating a path, where each step is a deliberate move towards the desired outcome. We'll examine how this method discerns the most significant differences between states and finds ways to diminish these disparities. Following this, we'll illustrate how Means-Ends Analysis is applied in real-world

scenarios. This section aims to provide practical insights into navigating through state spaces and achieving problem resolution using this method.

Finally, we examine Problem Reduction. This approach breaks down a complex problem into smaller, more manageable sub-problems. By addressing these sub-problems, the larger issue is gradually resolved. This method is especially beneficial in scenarios where addressing the entire problem in one go can be daunting.

Together, Means-Ends Analysis and Problem Reduction demonstrate powerful strategies for solving well-defined problems in the field of KBAI. They highlight the importance of segmenting problems and strategically targeting specific components to reach efficient solutions. This chapter aims to impart a thorough understanding of these methods and their practical application using KBAI, providing readers with essential tools for effective problem-solving.

THE BLOCKS WORLD PROBLEM: A CLASSIC AI CHALLENGE

We begin by examining the Means-Ends Analysis method through a classic problem in Artificial Intelligence: the “Blocks World” problem.³⁹⁻⁴¹ This problem is a staple in AI study, frequently featured in textbooks due to its simplicity in concept yet complexity in execution.

The “Blocks World” scenario presents us with a straightforward setup: a table with three blocks – A, B, and C. Initially, Block A and Block B are on the table, while Block C is on top of Block A. The goal is to rearrange these blocks so that Block C is on the table, Block B is on Block C, and Block A is on Block B.

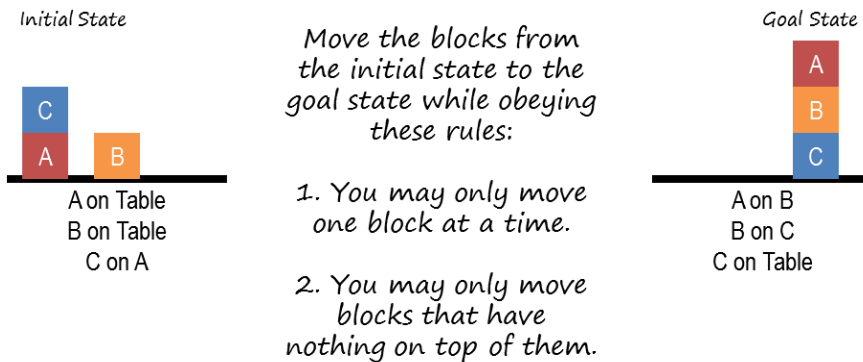


Figure 4-1 Blocks World Initial/Goal State.

However, there are constraints that add complexity to this seemingly simple task. First, only one block can be moved at a time, preventing the simultaneous movement of Blocks A and B. Secondly, a block can only be moved if there is nothing resting on top of it. For instance, in the initial setup, Block A cannot be moved because Block C is on top of it.

To navigate this problem, we're provided with a set of operators. These operators define potential actions, like moving Block C to the table, or on top of Block B. While not all operators are applicable at every stage due to the blocks' positions, they offer a range of possibilities to alter the blocks' arrangement.

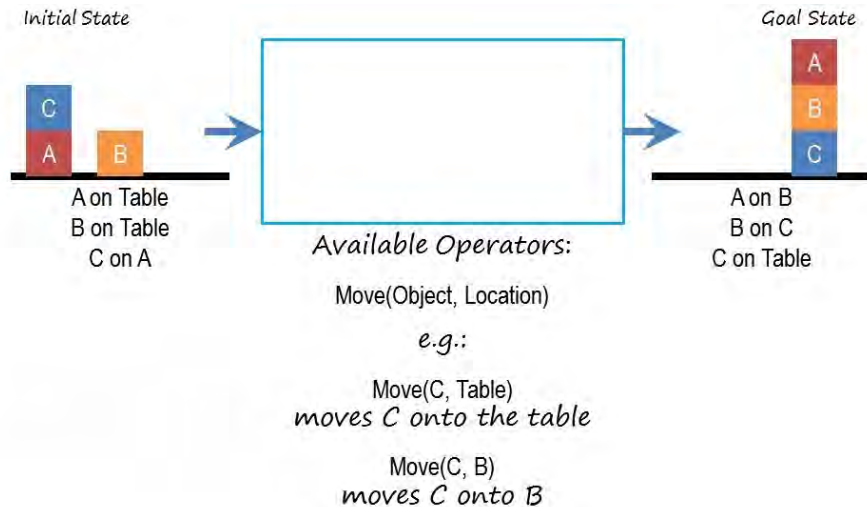


Figure 4-2 Available Operators.

The task, then, is to devise a sequence of operations using these operators that will transition the blocks from their initial state to the desired goal state. This exercise not only demonstrates the application of Means-Ends Analysis but also encapsulates the essence of problem-solving in AI – working within constraints and utilizing available tools to achieve a specific objective.

Let's rejoin a conversation between Ashok and David, we'll see how they approach this problem, dissecting their thought process and strategy. Their dialogue will illuminate how Means-Ends Analysis is applied in practice, offering valuable insights into effective problem-solving techniques in AI.

Ashok Alright David, how did you approach this problem?

David I started by moving Block C to the table. Next, I placed Block B on top of Block C. Finally, I moved Block A on top of Block B. It seems impossible to achieve the goal with fewer than three moves.

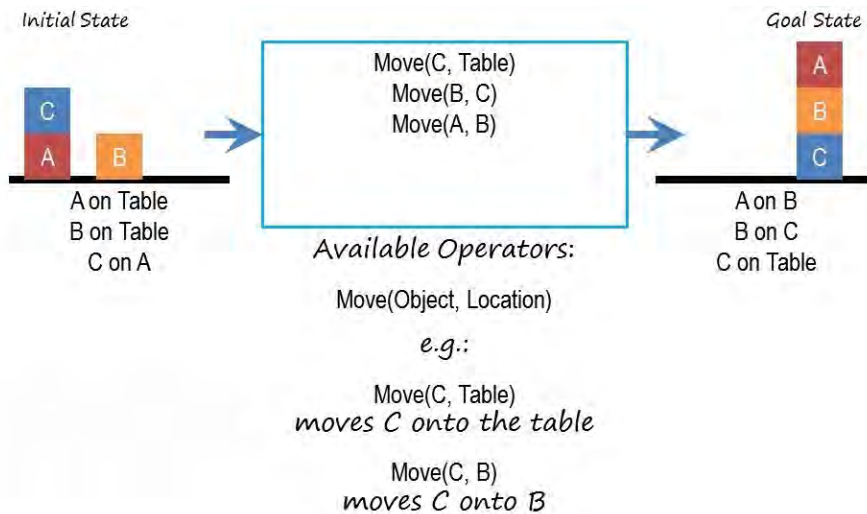


Figure 4-3 Solution to the Blocks World problem.

Ashok That's a solid approach, David, and indeed the correct sequence. But now, let's pivot to how we can program an AI agent to deduce a similar sequence. Specifically, how does the Means-Ends Analysis method tackle this problem to determine the right sequence of operations?

STATE SPACES

In problem-solving, such as when identifying the right sequence of operations, we can conceptualize the process as navigating through a state space. Envision an initial state and a goal state as two distinct points. The state space encompasses all possible states that could arise from the initial state through the iterative application of various operators within this defined environment. The objective is to identify a path within this state space that leads from the initial state to the goal state. While there might be multiple paths available, identifying even one viable path is crucial. The intriguing question is, how might an AI agent deduce such a path that efficiently transitions it from the initial state to the desired goal state?

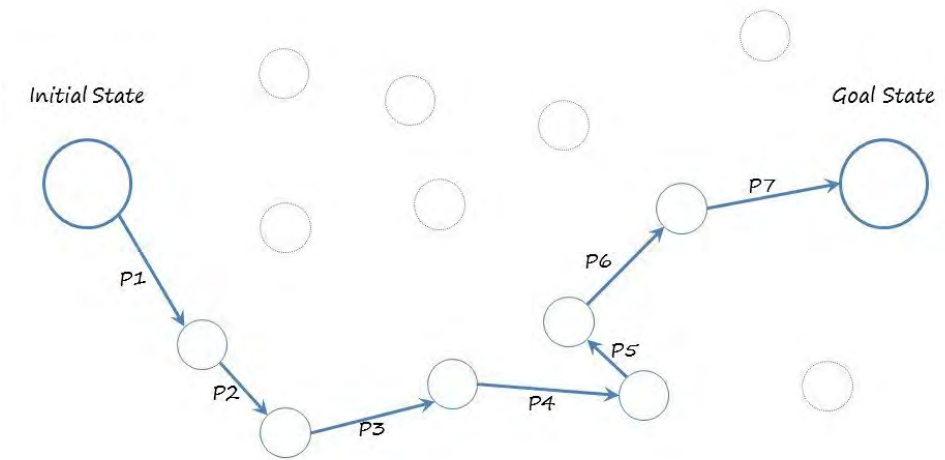


Figure 4-4 Illustrative State Space.

Returning to our block's world problem, let's explore the concept of pathfinding within a state space. From the initial state, one possible path to reach the goal state involves a sequence of specific actions. First, we move block C onto the table. Next, block B is placed on top of C. Finally, block A is positioned on top of B. This sequence aligns with the solution provided by David. It represents one of potentially many paths that transition from the initial state to the goal state.

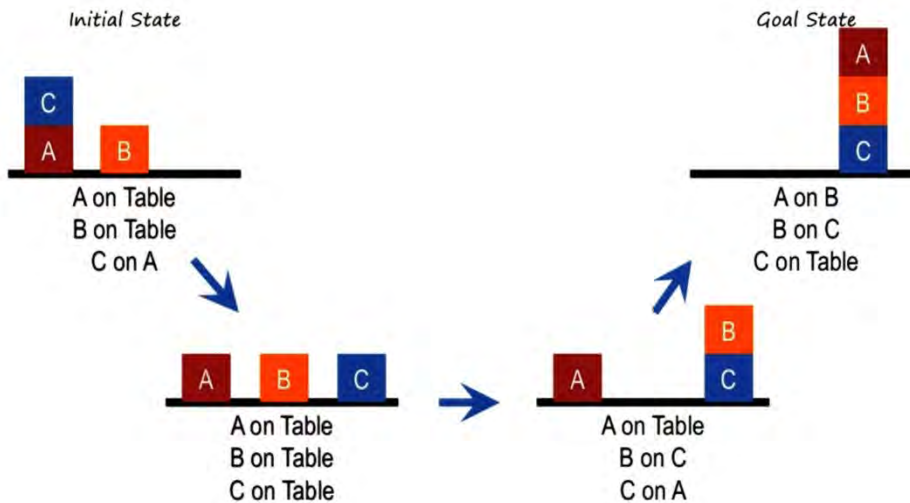


Figure 4-5 Potential path in Blocks World.

The crucial question, however, is how an AI method determines the appropriate operation to execute at any given state. For instance, consider a particular state (depicted in the lower left of the figure) where multiple operations are feasible. The AI could opt to place C on top of B or move B on top of A. The decision-making process of the AI agent at this juncture is key. How does it discern which operation is the most strategic or beneficial to execute in this specific state to progress towards the goal state effectively? This decision-making process lies at the heart of the AI's problem-solving capability.

One way of thinking about this is as the difference between states. Consider the illustrative chart below, which delineates the disparities between different states and the desired goal state. For instance, if we focus on a particular current state, the red line in the chart symbolizes the deviation from the goal state.

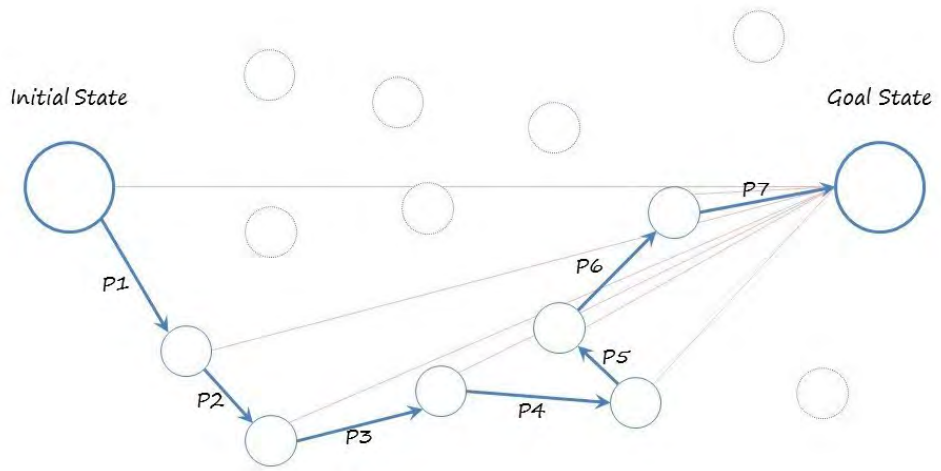


Figure 4-6 Differences between states and goal.

Incorporating the concept of “difference” as an ordinal function into the decision-making process of AI, particularly in the context of Means-Ends Analysis, provides a nuanced way of evaluating the proximity of various states to the goal state. This function, while ordinal in nature, allows for a relative comparison between states, offering a structured approach to determine the most strategic actions to progress towards the goal.

For example, in a situation where an AI agent is faced with multiple feasible operations from a given state, the “difference” function can be applied to each potential resulting state to quantify its divergence from the goal state. This divergence could be measured in objective terms, such as physical distance in a spatial navigation task, where the difference might be the actual metric distance from the target location. Alternatively, in scenarios where the evaluation is more subjective, such as assessing customer satisfaction levels in a business context, the “difference” could be represented through Harvey ball indicators, offering a visual gauge from 'very good' to 'very bad.'

In practice, this means that for each possible operation the AI could execute from a given state, it would assess the resulting state's “difference” from the goal state using this ordinal function. The operation leading to the state with the least “difference” would be considered the most beneficial or strategic to execute at that moment. This approach allows the AI to navigate through the state space effectively, making informed decisions that incrementally reduce the “difference” to the goal state, thereby optimizing its problem-solving strategy.

MEANS-ENDS ANALYSIS (MEA)

In applying Means-Ends Analysis (MEA), the core idea is to reduce the difference between the current state and the goal state. The strategy involves greedily selecting an operator that minimizes this disparity. Essentially, the aim is to reduce the gap between where we are (the current state) and where we need to be (the goal state). This is the essence of the term “Means-Ends”: the 'means' refers to the application of an appropriate operator, while the 'ends' pertains to the reduction of the difference between the current and goal states.

Interestingly, this concept bears a resemblance to pathfinding in robotics, such as designing a robot's route from one office to another within a building. In robotics, the navigation relies on physical distances. However, in our scenario, the term 'distance' is more metaphorical or abstract. We often use 'difference' as a substitute for 'distance' to better suit the abstract nature of the space we're dealing with. This approach is about navigating not through physical space, but through a conceptual landscape of problem states, striving to methodically reduce the variance between our current position and our intended destination.

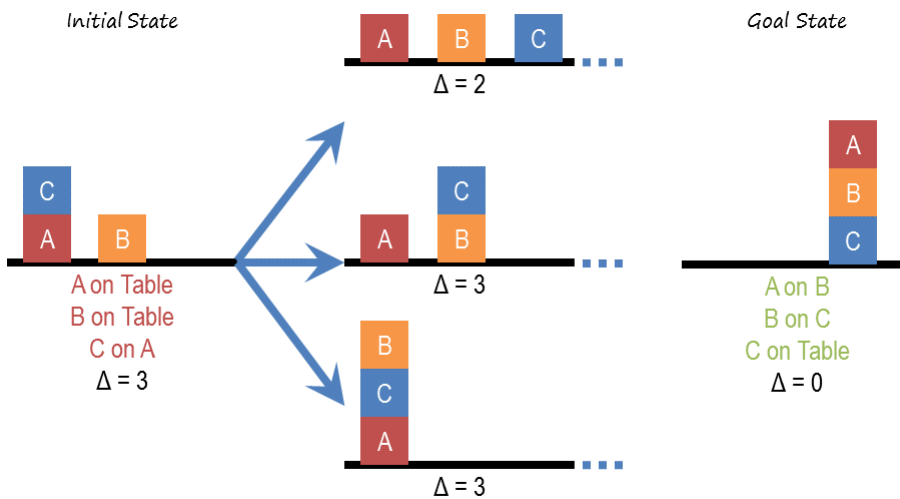


Figure 4-7 Possible next states in Blocks World with differences.

Continuing our exploration of the Means-Ends Analysis with the Blocks World example, we return to the transition from the initial state to the goal state. On examining the initial state (right side of figure above), we identify three key differences when compared to the goal state (left side of figure above): firstly, A is on the table, but it should be on B; secondly, B is also on the table, whereas it should be on C; and thirdly, C is atop A, whereas it should be on the table.

Given these variances, we have a range of nine potential operations at our disposal to address these discrepancies.

In conducting a Means-Ends Analysis, we have several options. One approach is to apply an operator that moves C to the table, thereby reducing the difference count to two (top of figure above). Alternatively, we might consider an operator that places C on B, but this would maintain the difference count at three (center of figure above). Another feasible operation is placing B on C, which also results in a difference count of two (bottom of figure above). Here, we encounter a critical juncture: the Means-Ends Analysis presents us with two viable paths, each leading to a state with an equal difference count of two. This point in the analysis doesn't inherently guide the AI agent in choosing between these two possible courses of action.

This dilemma underscores a limitation of the Means-Ends Analysis: while it helps in narrowing down choices by focusing on reducing differences, it doesn't always provide a clear directive on the optimal sequence of operations. This is an aspect we'll revisit later in this chapter and explore further when we consider planning later in the book. For our current discussion, let's proceed with the choice that places C on the table, as illustrated in the top course of action in our figure. This decision aligns with the path already taken in our example, advancing us towards the desired goal state with strategic application of operations.

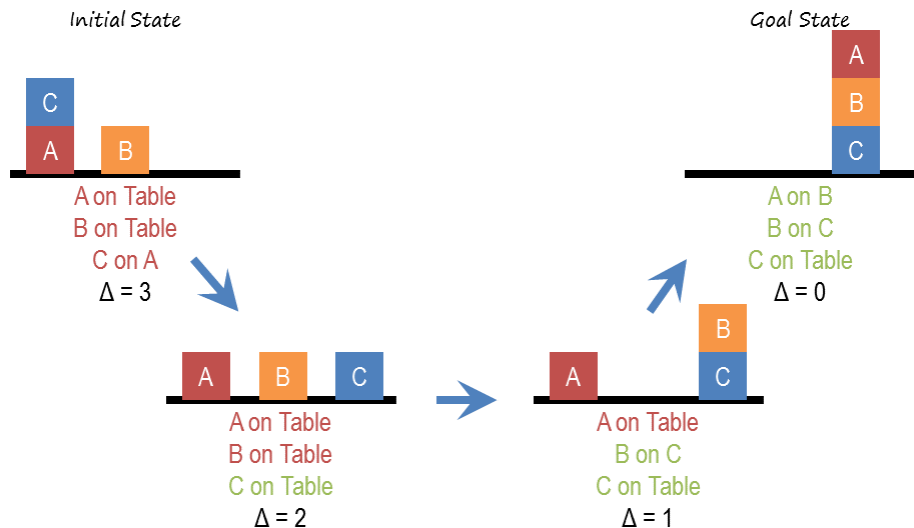


Figure 4-8 Example of difference reduction through the Blocks World state space.

As illustrated above, each move strategically lessens the gap between the current configuration of blocks and the desired end configuration. This progressive reduction is the crux of the Means-Ends Analysis. At each step, the AI agent chooses an operation that diminishes the number of differences. Initially, there are three key discrepancies between the current and

goal states. The first operation, moving C to the table, reduces these differences to two. The subsequent move, placing B on C, further decreases the gap to just one difference. Finally, positioning A on B aligns the current state perfectly with the goal state, eliminating all differences and achieving the desired outcome.

The beauty of the Means-Ends Analysis lies in its systematic approach to problem-solving. It guides the AI agent to consistently make choices that bring it closer to the goal state. By focusing on reducing the 'distance' or differences at each step, the AI agent efficiently navigates through the state space, avoiding unnecessary or counterproductive moves. This methodology ensures a logical and streamlined path to the solution, highlighting the power of this problem-solving technique in tackling well-defined problems.

Means-Ends Analysis is a methodical approach to problem-solving that unfolds in several distinct steps. At its core, this method involves a thorough comparison between the current state and the desired goal state. This comparative analysis is vital as it highlights the specific discrepancies or differences that need to be addressed to achieve the goal. The next crucial step in this process is the identification of potential operators or actions that can be applied in the current situation. This step is pivotal for determining the most effective path forward. Each operator is assessed based on its ability to bridge the gap between the current state and the goal state, with the ultimate aim of minimizing these differences.

Once the relevant differences are identified, and potential operators are evaluated, the process then involves selecting the operator that most effectively reduces the discrepancy between the current and goal states. This selection is strategic, focusing on the operator that promises to bring the system closer to the desired outcome in the most efficient way.

This approach was demonstrated effectively in our analysis of the blocks world problem. In this scenario, each move was strategically chosen to systematically reduce the differences between the current arrangement of blocks and the target configuration. Similarly, in the guards and prisoners problem, we applied Means-Ends Analysis by eliminating states that did not contribute to progressing towards the goal state. In essence, Means-Ends Analysis is a structured and deliberate problem-solving strategy. It revolves around identifying key differences, evaluating applicable actions, and strategically selecting the most effective operator to reduce the gap between the current state and the goal.

LIMITATIONS OF MEA: LOCAL OPTIMA AND GETTING STUCK

To examine the nuances of Means-Ends Analysis, let's consider a more complex scenario involving four blocks – A, B, C, and D. In the initial state, these blocks are arranged as shown in the accompanying figure on the left. Conversely, the goal state, depicted in the figure on the right, presents a different arrangement of these blocks.



Figure 4-9 Initial and goal states of the Blocks World problem with four blocks.

Upon comparing the initial and goal states, we observe three key differences. These differences form the basis of our heuristic measure, guiding the AI agent's decision-making process. Firstly, block A, which rests on the table in the initial state, is supposed to be on top of block B in the goal state. Secondly, while block B's position atop C aligns with the goal state, block C's placement varies – it's on the table in the initial state but should be on D in the goal state. Finally, block D, positioned on the table in the initial state, needs to be beneath block B in the goal state.

Given these discrepancies and the stipulation that only one block can be moved at a time, the AI agent must carefully consider the potential states that can be derived from the initial arrangement. Each move must be evaluated not just for its feasibility but also for its effectiveness in reducing the difference between the current and goal states. The task for the AI agent, therefore, is to generate a series of moves that progressively align the current state with the goal state. This requires a strategic approach where each move is selected based on its potential to decrease the heuristic measure of difference. The agent must prioritize moves that result in an arrangement increasingly resembling the goal state while adhering to the constraints of moving only one block at a time and not disturbing blocks with others on top of them.

In this next discussion, we witness an insightful exchange between Ashok and David as they apply the Means-Ends Analysis to a more complex blocks problem.

Ashok So, David, how did you analyze this scenario? What operations did you identify as possible and legal?

David Well, considering the constraints, there are three legal moves we can make here. We can move D onto A, move D onto the table, or move A onto D. For my first choice, I've put A on D, which leaves the other blocks as they are. The second option involves moving D onto A, again keeping the other blocks unchanged. And

for the third option, I've put D directly onto the table, with no changes to the positions of the other blocks.

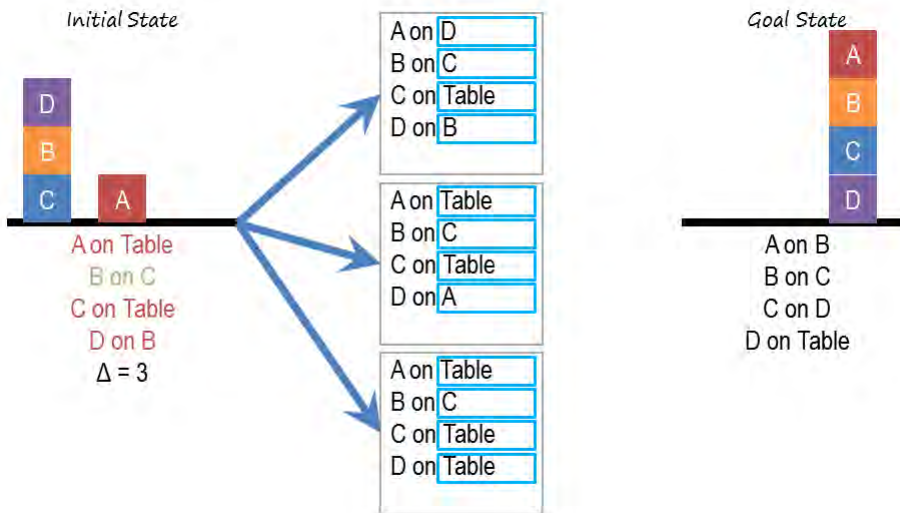


Figure 4-10 Potential next stats from the initial state in the four-block example problem.

Ashok: Excellent analysis, David. Your approach correctly identifies the possible moves within the constraints we have. This is crucial in devising a strategy to move from our current state to the desired goal state. Now, David, for each of these potential states that can arise from the initial state, how do they compare to the goal state? What are the differences you've identified?

David: Well, if we move A onto D, as in the first option, we don't actually achieve any of our goals that weren't already achieved. So, the difference in this scenario remains 3. The same goes for moving D onto A – again, we're left with a difference of 3 since we haven't accomplished anything new towards our goal. However, when we move D onto the table, as in the third option, we address one of the unmet goals. This reduces our difference to 2.

Ashok: Excellent, David. Your analysis of each state against the goal state, and identifying the differences, is spot on. Now, given these choices, which operation would Means-Ends Analysis select?

David Means-Ends Analysis aims to choose the state that most reduces the distance to the goal state. In our case, the third option is the best choice as it decreases our difference to 2, bringing us closer to the goal state.

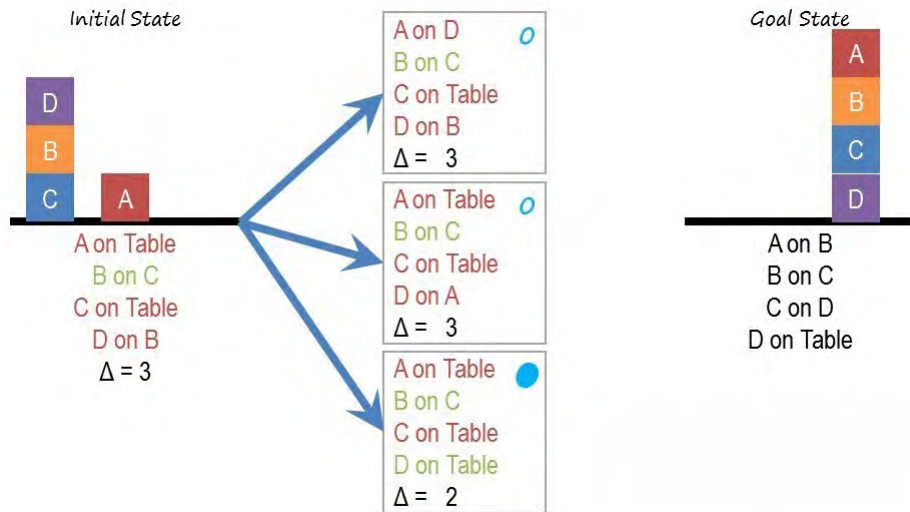


Figure 4-11 Next states with their corresponding differences.

Ashok Precisely, David. You've correctly identified that Means-Ends Analysis focuses on reducing the discrepancy between the current state and the goal state. In this instance, placing D on the table is the most effective move towards achieving our goal. Now that we have this current state, let's apply Means-Ends Analysis iteratively. Considering the vast number of options available from this point, I won't list them all. David, can you tell me the number of possible next states you identified, and how many of these states reduce the difference to our goal?

David Sure, I counted seven potential next moves: placing B on A, B on D, B on the table, A on B, A on D, D on B, or D on A. Of these, only one move, placing A on B, accomplishes a new goal and thus reduces our difference.

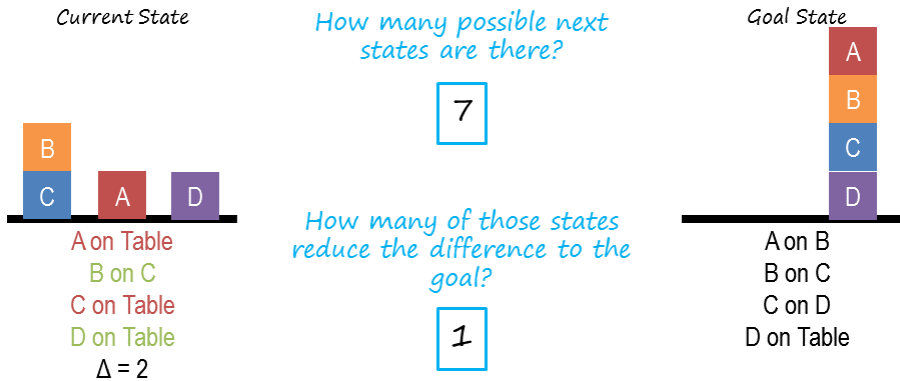


Figure 4-12 Count of next states and number of those that reduce the difference between the initial and goal states.

Ashok Excellent observation, David. Placing A on B does indeed bring us closer to our desired state. Now, considering this new state, let's continue our Means-Ends Analysis. Again, without going into each potential state, can you tell me the possible next states and which ones decrease our distance from the goal?

David In this new scenario, we have three possible next states. We can place A on the table, A on D, or D on A. However, none of these options reduce the difference to the goal. In fact, each of these moves takes us further away. For instance, moving D on A undoes our previous accomplishment of having D on the table. Similarly, moving A either to the table or on D undoes our progress of having A on B. So, with Means-Ends Analysis, it doesn't look like we can get any closer to the goal state.

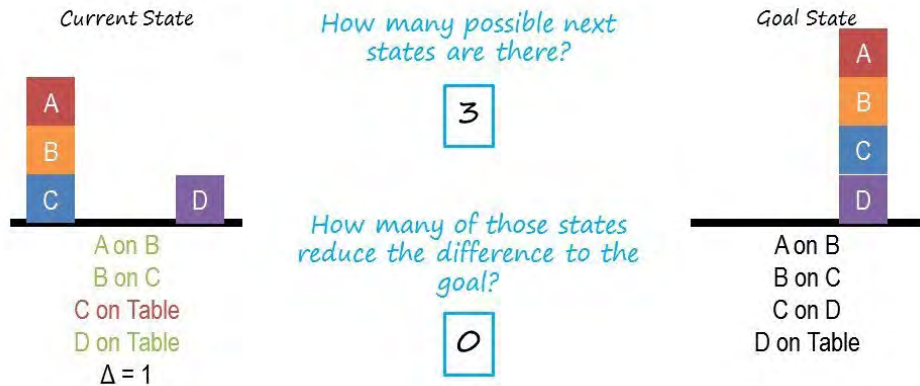


Figure 4-13 Count of next states and number of those states that reduce the difference between the start on the left and the goal state.

Ashok Precisely, David. This illustrates an important aspect of Means-Ends Analysis: it doesn't always lead us directly to the goal. Sometimes, the analysis might move us away from the goal or can get caught in a loop. This can require us to reconsider our approach or backtrack.

Means-Ends Analysis is an important AI method, but it does not always provide a straight path to the solution. This technique, much like Generate-and-Test, is considered a Universal AI Method; one applicable to a broad range of problems. However, it comes with certain limitations:

1. **No Guaranteed Success:** Means-Ends Analysis, while versatile, does not guarantee a successful outcome in every scenario. It may lead one away from the goal, or in some cases, even cause looping, where the process repeats without making progress.
2. **Computational Cost:** These universal methods are often computationally expensive. They do not assure computational efficiency nor guarantee the optimality of the solutions they produce.^{42,43}
3. **Wide Applicability with Limitations:** The strength of Means-Ends Analysis lies in its wide applicability across various problem classes. But this flexibility doesn't always equate to efficiency or the best possible solution.
4. **Comparison with Specialized Methods:** In contrast to these universal methods, there exist specialized problem-solving techniques that are tailored to specific types of problems. These specialized methods, though limited in their applicability, can often offer more efficient solutions, and sometimes guarantee the optimality of these solutions.⁴⁴

While Means-Ends Analysis may not have been the most effective for the Blocks World problem in this instance, its value lies in its general applicability to many AI problems. This is something we will consider in the sections on planning, where we will explore more powerful

and specialized methods. These methods are adept at handling specific classes of problems, providing solutions with greater efficiency and sometimes even with guaranteed optimality.

PROBLEM REDUCTION

The concept of Problem Reduction, a vital problem-solving method in Knowledge-Based Artificial Intelligence, revolves around an intuitive approach: breaking down complex challenges into manageable, simpler sub-problems.⁴⁵ This method is likely familiar to many, especially those involved in computer programming or software design, where complex tasks are routinely dissected into more digestible segments.

Consider the process of tackling a challenging programming task. The method of Problem Reduction would advise splitting this task into smaller, more focused problems, such as:

- **Input Handling:** Determining the most efficient and error-free way to read the required input.
- **Data Processing:** Figuring out the necessary algorithms or logic to transform the input into a desired format or to perform calculations.
- **Output Generation:** Deciding how to present the results or the processed data back to the user or another system component.

This method isn't just about simplifying the problem; it's about understanding the fundamental role of knowledge in problem-solving. Knowledge guides us in decomposing a complex problem into its constituent parts. By solving these smaller, more straightforward problems, we gradually build towards a comprehensive solution for the larger issue. Problem Reduction is particularly effective because it allows us to focus on one aspect of a problem at a time, reducing cognitive overload and increasing the chances of finding effective solutions. Furthermore, once solutions for these sub-problems are found, they can be reassembled to address the original, more complex problem. This approach not only simplifies the problem-solving process but also often leads to more robust and well-structured solutions.⁴⁶

Solving the Block World Problem with Problem Reduction

The task, involving an arrangement of blocks A, B, C, and D, presented a significant challenge when approached through Means-Ends Analysis alone. However, by applying Problem Reduction, we can simplify this task by breaking it down into more manageable sub-goals.



Figure 4-14 Current state example for Problem Reduction.

Let's revisit the scenario: Our current state shows a specific arrangement of blocks, while our goal state requires a different, specific arrangement. The objective seems daunting at first, but it becomes more approachable as we dissect the goal into its constituent parts. In this case, our overall goal comprises four distinct sub-goals:

- Sub-Goal 1: Place block D on the table.
- Sub-Goal 2: Stack block C on top of D.
- Sub-Goal 3: Stack block B on top of C.
- Sub-Goal 4: Finally, place block A on top of B.

By focusing on one sub-goal at a time, we can channel our problem-solving efforts more effectively. In this instance, we opt to address the sub-goal involving blocks C and D first. This decision might raise the question: Why prioritize this particular sub-goal over others? The answer lies partly in the nature of our current state, which shows that addressing C and D first aligns closely with the most immediate differences we need to resolve to reach our goal state. It's important to note that Problem Reduction doesn't inherently dictate the order in which sub-goals should be tackled. This sequencing is a separate challenge we will explore in the context of planning strategies later.

With our focus narrowed down to just two blocks, C and D, our task becomes clearer. We temporarily set aside concerns about the positions of blocks A and B, concentrating solely on achieving the sub-goal of positioning C on D, with D placed firmly on the table. This targeted approach exemplifies how Problem Reduction simplifies complex problems by isolating and addressing one segment of the problem at a time. Next, we'll explore how Means-Ends Analysis, adapted to this more focused scope, can be employed to effectively resolve this specific sub-problem.

In the exploration of the Problem Reduction method, Ashok and David engage in a conversation to navigate the complexities of the “Blocks World” problem, demonstrating the practical application of this strategy.

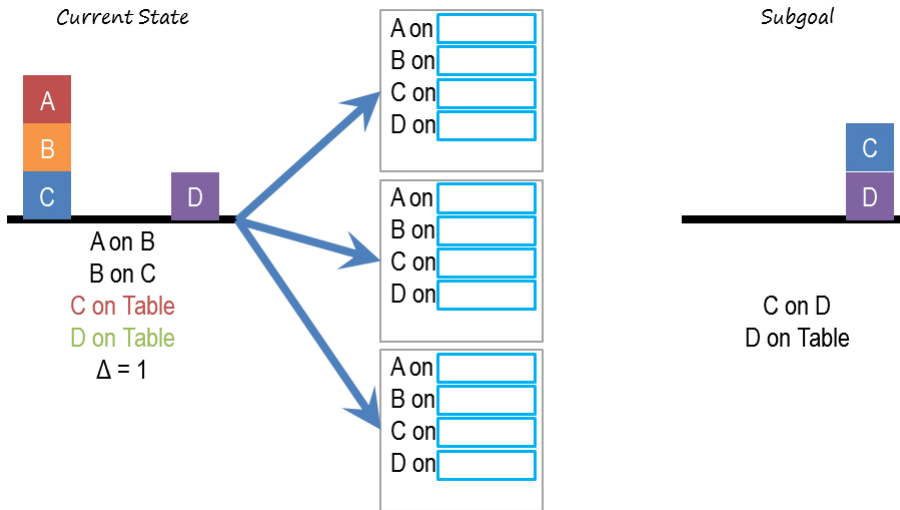


Figure 4-15 Identification of potential successor states from given state toward goal state.

Ashok David, let's evaluate the successor states possible from our current state using Means-Ends Analysis. Could you outline these states?

David Sure, Ashok. We have three moves to consider: up top we can move A onto D, in the center we can move D onto A, and on the bottom we can place A on the table. For each move, I've visualized the resulting state, considering the constraints of our problem.

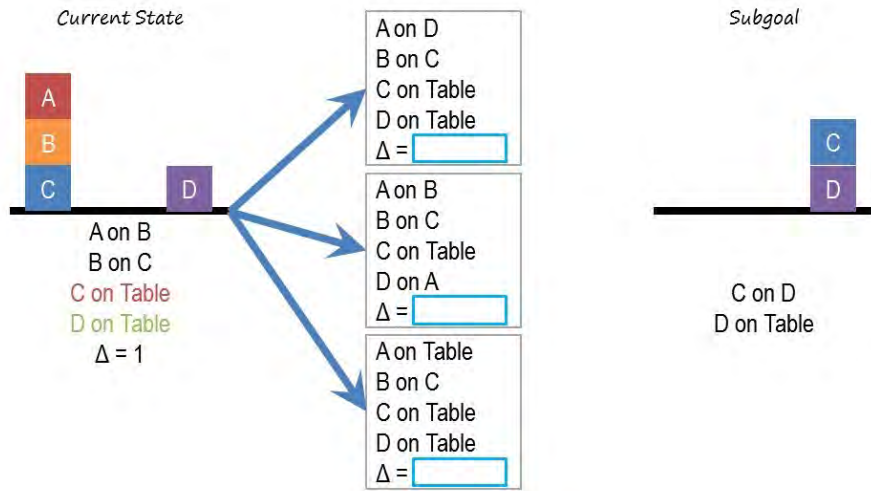


Figure 4-16 Identified potential successor states from given state toward goal state.

Ashok Excellent. Now, let's analyze the differences between these states and our goal state. What are your observations, David?

David In all cases, the positioning of C remains unchanged, so the goal of C on D will not be accomplished. In the center option, however, we moved D onto the table, but that increased the difference to the goal state, suggesting that it moves us away from the goal state. So, means-ends analysis would choose the top or bottom states, even though the difference is unchanged from our current state.

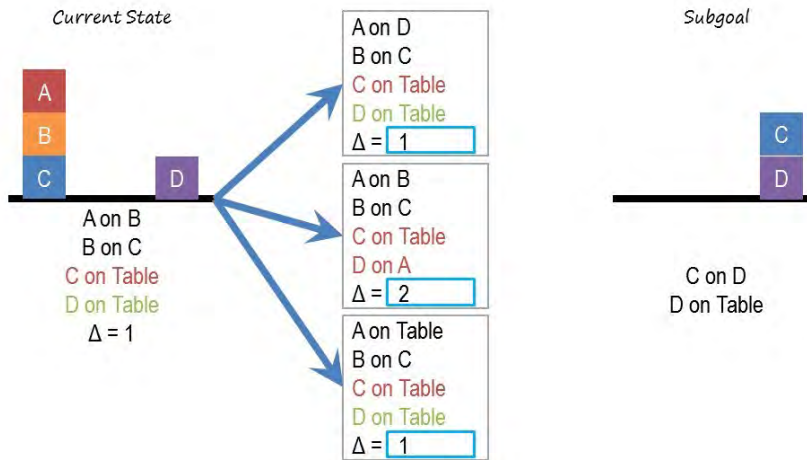


Figure 4-17 Identified potential successor states from given state toward goal state and their corresponding differences.

Ashok

That's right, David, and you've made an interesting observation. The top state and the bottom state are an equal distance from the goal. When this occurs, means-ends analysis can pick either state. For this exercise, let's proceed with the state where A is on the table (as shown in the bottom state in the figure). The reason is because if we place A and D, it will prevent us from solving the rest of the problem. This decision may seem arbitrary, but later we'll discuss how an AI agent might determine the most effective path. David, considering this state, can you suggest a sequence of operators to reach the goal state?

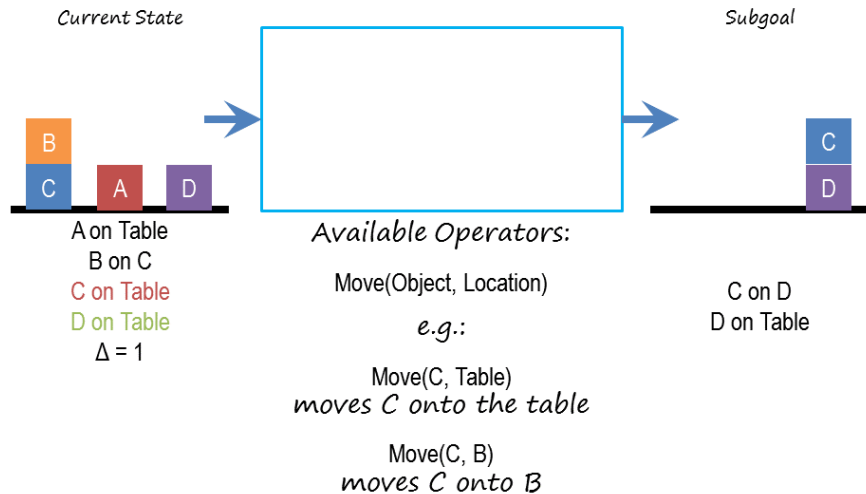


Figure 4-18 Example used to identify operations in moving from the current state (left) to the goal state (right).

David From the state on the left, there are seven possible initial moves. Some moves, like putting D on A or B, would hinder our progress because they move us away from our goal state. In means-ends analysis, the remaining moves are essentially the same because none of the moves reduce the difference indicating progress toward the goal. However, we can see that we need to move B out of the way to move C onto D. Moving B onto the table seems like a strategic choice, opening the way to stack C onto D.

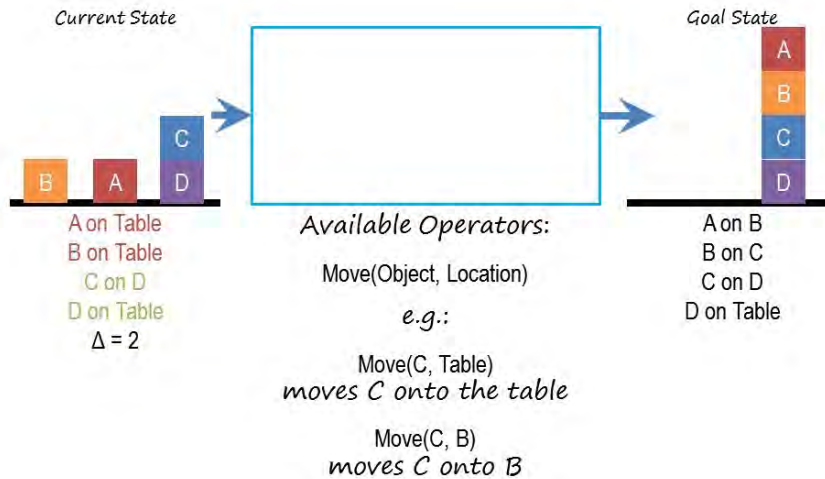


Figure 4-19 Example with C moved onto D used to identify operations in moving from the current state (left) to the goal state (right).

Ashok That was the right answer, David. This move aligns with our first sub-goal. Now, with C on D, we can focus on the remaining sub-goals involving B and A. David, given our current state and the final goal, what sequence would you recommend to complete our task?

David Initially, we have two moves that bring us closer to our goal: placing A on B or B on C. We can recognize moving B on C is the right subgoal because moving A on B or C gets in the way of our moving B. So, we'll choose B on C first because that reduces our difference; and because it's a necessary step before stacking A. Finally, placing A on B will achieve our goal state.

Ashok David, your analysis was spot on. It's intriguing how simple this problem appears to us as humans. The transition from one state to another seems almost effortless. Yet, viewing it through AI's lens reveals numerous underlying questions. It's a reminder that as humans, we subconsciously tackle these complexities. AI's strength lies in making these implicit thought processes explicit. This transparency in problem-solving is one of AI's most significant attributes. However, our discussion also uncovers many unresolved questions, each demanding a solution. This realization points us towards the need for developing methods capable of addressing these intricate questions in AI.

The Problem Reduction method, like Generate-and-Test and Means-Ends Analysis, is universally applicable but does not guarantee success. It underscores the need for AI methods that address the myriad questions arising in complex problem-solving scenarios.

Solving Raven's Problem with MEA and Problem Reduction

We now revisit a challenge encountered during our discussion of the Raven's Intelligence Test. Picture this scenario: A represents our initial state, and B is our goal state. We're aware that A has transitioned into B, and we can conceptualize a 'sequence of operations' facilitating this transformation.



Figure 4-20 Example Raven's problem with initial state A on left and goal state B on right.

Consider this approach: 'First, delete the dot, then extricate the diamond from the circle, and finally, enlarge it.' This process can be viewed through the lens of Means-Ends Analysis, as each step incrementally bridges the gap to our goal state. The merit of this analytical approach is its utility in replicating the transformation sequence from the initial state to the goal state, and then applying similar transformations to image C.

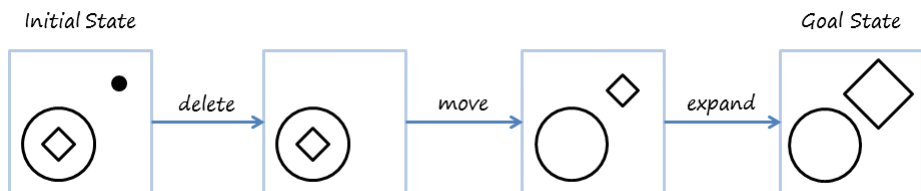


Figure 4-21 Problem reduction state path from the initial state to the goal state.

The advantage of doing this analysis and coming up with a sequence of transformations that will take us from the initial state to the goal state is that we can now think in terms of applying this same sort of transformations to image D.

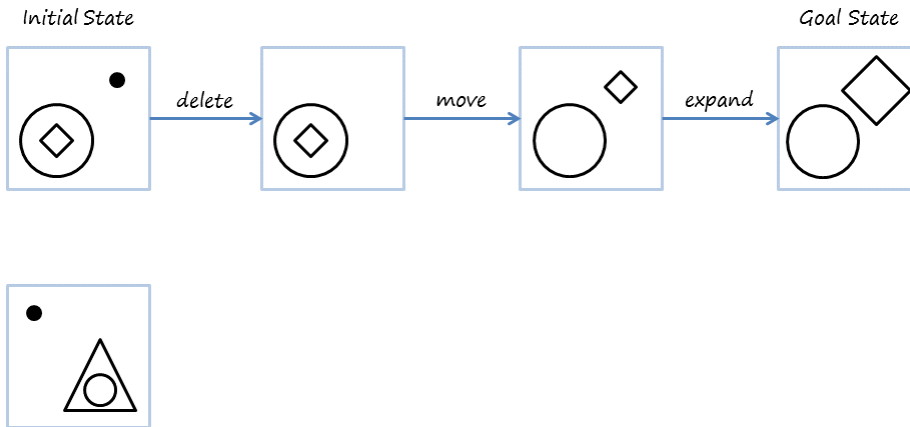


Figure 4-22 Applying Means-Ends Analysis and Problem Reduction to solve the Raven's problem.

Let's execute this step-by-step. First, we apply the 'delete' operation, removing the dot. Next, we use the 'move' operation, which results in multiple potential states – we've illustrated two here, both satisfying the criteria of moving the diamond outside the circle. Following this, each state undergoes the 'expand' operation, enlarging the circle differently.

Here's a nuanced aspect: how do we determine the correspondence between elements in images A and C? Recall our semantic network representation of image A, where we established the diamond as inside the circle. A similar representation for image C places the circle inside the triangle. This 'inside' relationship is a crucial hint, suggesting the circle in C corresponds to the diamond in A.

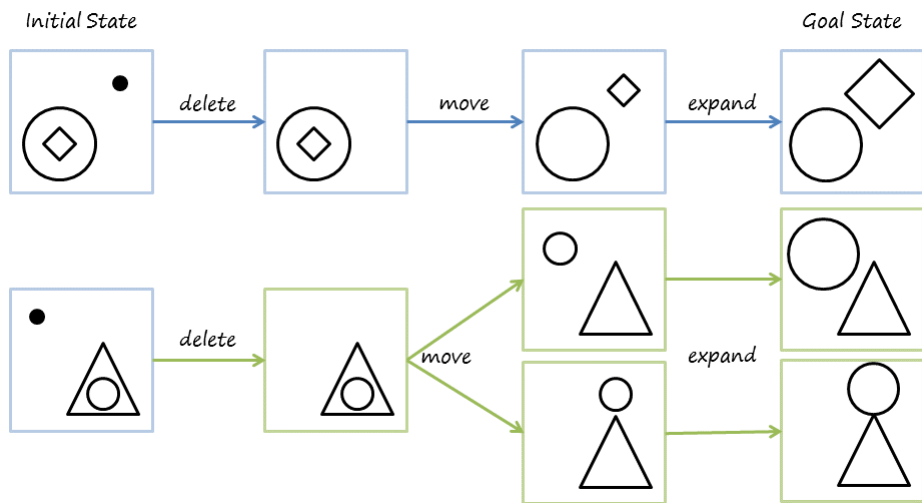


Figure 4-23 Potential goal states after applying Means-Ends Analysis and Problem Reduction to solve the Raven's problem.

This process can also be viewed through the lens of problem reduction. We've dissected a larger problem into manageable sub-goals: identifying the transformation between A and B, transferring this transformation to C, and exploring potential states for D. Moreover, this scenario embodies elements of generate-and-test, as we're actively generating potential solutions to be evaluated against available options.

What's fascinating is how this complex problem-solving exercise integrates multiple AI techniques. Depending on the phase, we might leverage problem reduction, generate-and-test, or means-ends analysis. The semantic network knowledge representation underpins all these strategies, showcasing its versatility. Yet, it's important to note the 'weak' coupling between this universal method and the knowledge representation. In other words, the relationship between the method and the knowledge it uses is not tightly interwoven. This characteristic is what classifies these methods as 'Weak Methods', a concept we will examine further in subsequent discussions, especially when we explore methods where knowledge and problem-solving are more intricately connected.^{47,48}

MEA AND PROBLEM REDUCTION AS “WEAK METHODS”

Means-Ends Analysis and Problem Reduction, much like Generate-and-Test, are often referred to as “weak methods” in AI. This term does not imply they are ineffective; rather, it signifies their general applicability across a wide range of problems and their minimal reliance on domain-specific knowledge. This is in contrast to “strong methods,” which are knowledge-intensive and

typically tailored to specific problem domains, often offering more efficient or optimal solutions within those domains.

COGNITIVE CONNECTION

Humans frequently use Means-Ends Analysis and Problem Reduction in their daily problem-solving. When faced with unfamiliar domains or problems where specific expertise is lacking, we often resort to these general-purpose “weak methods.” We break down complex tasks into simpler steps (Problem Reduction) and focus on reducing the most obvious differences between our current situation and our desired goal (Means-Ends Analysis). This adaptability underscores the versatility of these strategies in human cognition.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

The principles of Means-Ends Analysis (MEA) and Problem Reduction find conceptual parallels and direct applications in various modern AI paradigms:

- **Hierarchical Reinforcement Learning (HRL):** HRL directly embodies Problem Reduction by decomposing complex tasks into a hierarchy of sub-goals or sub-tasks. An agent learns policies not just for the main goal but also for achieving these intermediate sub-goals, making learning more efficient and solutions more interpretable.
- **Planning Algorithms in Robotics:** Many planning algorithms used in robotics employ heuristics that are MEA-like. Robots often need to reduce the “difference” (e.g., spatial distance, configuration mismatch) between their current state and a goal state, selecting operators (actions) that achieve this. Path planning, for instance, can be seen as reducing the difference in coordinates.
- **Automated Program Synthesis:** This field aims to automatically generate computer programs from specifications. Problem Reduction is a key strategy here, breaking down the high-level programming task into smaller, more manageable functions or modules that can be synthesized individually and then combined. Modern AI-powered coding assistants leverage similar principles by breaking down coding requests into manageable steps and generating code snippets to reduce the “difference” between the current code state and the desired functionality.
- **Goal-Conditioned Policies in RL:** In RL, agents can be trained with goal-conditioned policies, where the policy takes the current state and a goal state (or sub-goal) as input to generate an action. This is akin to MEA, where the agent's actions are aimed at reducing the difference to the specified (sub-)goal. For example, DeepMind's work on agents learning to play complex games (like some Atari games or even more complex

simulations) often involves training policies that can generalize to different goals within the game environment, such as reaching a specific location or collecting a particular item. These agents learn to select actions that progressively reduce the “distance” to the current active goal.

- **Large Language Models (LLMs) for Complex Task Solving:** When LLMs are prompted to solve complex, multi-step problems, they often implicitly (or can be explicitly prompted to) break the problem down into smaller steps (Problem Reduction) and generate intermediate solutions aimed at reducing the gap towards the final answer (MEA-like reasoning). Frameworks like Chain-of-Thought (CoT) prompting, or more advanced techniques like Tree-of-Thoughts (ToT), explicitly encourage LLMs to decompose problems and explore intermediate “thinking” steps, which aligns well with the principles of problem reduction and iterative difference reduction inherent in MEA.

These connections demonstrate that the foundational KBAI methods of MEA and Problem Reduction continue to offer valuable frameworks for structuring problem-solving in more complex, data-driven, and learning-based AI systems.

CHAPTER SUMMARY

This chapter examined two fundamental KBAI problem-solving methods: Means-Ends Analysis (MEA) and Problem Reduction. We began by establishing the concept of state spaces as the environment in which these methods operate, representing all possible configurations of a problem.

Means-Ends Analysis was introduced as a strategy focused on iteratively reducing the “difference” between the current state and a desired goal state by selecting appropriate operators. We explored its application through the classic Blocks World problem, illustrating how MEA guides an agent by choosing actions that minimize deviations from the goal. However, we also highlighted MEA's limitations, such as its potential to get caught in local optima or loops, as demonstrated in a more complex four-block scenario, where it couldn't reach the goal state by strictly reducing differences.

To address such limitations, Problem Reduction was presented as a method for decomposing complex problems into smaller, more manageable sub-problems or sub-goals. By solving these simpler parts, often using MEA for each sub-goal, the overall problem can be tackled more effectively. This was demonstrated by revisiting the four-block problem, where breaking it into sub-goals allowed for a successful solution. We also saw how MEA and Problem Reduction could be conceptualized for solving Raven's Progressive Matrices, by identifying transformations as sequences of operations and breaking down the analogy task.

Both MEA and Problem Reduction were characterized as “weak methods” – general-purpose strategies that don't rely heavily on domain-specific knowledge, making them broadly

applicable but not always optimal. This contrasts with “strong methods” which are knowledge-intensive.

The chapter also highlighted the cognitive connection, noting that humans often employ these heuristic strategies, especially in unfamiliar situations. Finally, we explored how the core principles of MEA (difference reduction) and Problem Reduction (task decomposition) are conceptually mirrored or directly applied in modern AI paradigms such as Hierarchical Reinforcement Learning, robotics planning, automated program synthesis (including AI coding assistants), Goal-Conditioned Policies in RL (seen in complex game-playing agents), and even in how Large Language Models (LLMs) using frameworks like Chain-of-Thought approach complex tasks.

This chapter emphasized that while these methods have their constraints, they form crucial building blocks for intelligent reasoning and are foundational to understanding more advanced problem-solving techniques in KBAI and beyond. The discussion sets the stage for exploring other KBAI methods, such as production systems, and more sophisticated planning techniques that build upon these fundamental ideas.

ADDITIONAL READINGS & REFERENCES

- 38 Citroresmi, P. N. *et al.* in *International Workshop on Learning Technology for Education Challenges*. 216-226 (Springer).
- 39 Gupta, N. & Nau, D. S. On the complexity of blocks-world planning. *Artificial intelligence* **56**, 223-254 (1992).
- 40 Slaney, J. & Thiébaux, S. Blocks world revisited. *Artificial Intelligence* **125**, 119-153 (2001).
- 41 Hao, S. *et al.* Reasoning with language model is planning with world model. *arXiv preprint arXiv:2305.14992* (2023).
- 42 Everitt, T. & Hutter, M. Universal artificial intelligence: Practical agents and fundamental challenges. *Foundations of trusted autonomy*, 15-46 (2018).
- 43 Hutter, M. in *European conference on machine learning*. 226-238 (Springer).
- 44 Masoudnia, S. & Ebrahimpour, R. Mixture of experts: a literature survey. *Artificial Intelligence Review* **42**, 275-293 (2014).
- 45 Hélié, S. & Pizlo, Z. When is Psychology Research Useful in Artificial Intelligence? A Case for Reducing Computational Complexity in Problem Solving. *Topics in Cognitive Science* **14**, 687-701 (2022).
- 46 Norman, G. R. Problem-solving skills, solving problems and problem-based learning. *Medical education* **22**, 279-286 (1988).
- 47 Korf, R. E. Macro-operators: A weak method for learning. *Artificial intelligence* **26**, 35-77 (1985).
- 48 Filkorn, V. in *Studies in Logic and the Foundations of Mathematics* Vol. 126 319-331 (Elsevier, 1989).

Chapter 5

Production Systems

INTRODUCTION: THE PITCHER'S DILEMMA

In this chapter, we embark on an exploration of production systems, a captivating subject especially for those intrigued by the intersection of cognitive architecture and learning.⁴⁹ As the final topic under fundamental themes, production systems promise a deep dive into the mechanics of knowledge representation and the processes underlying learning.

The chapter begins with an overview of cognitive architectures. These architectures are the frameworks within which cognitive processes and knowledge representation combine, providing a structural blueprint for understanding how artificial intelligence emulates human cognition. The chapter also explores cognitive architectures to establish a foundation for understanding the dynamics of production systems. Focusing on production systems, we will uncover how knowledge is structured and utilized in the form of rules⁵⁰⁻⁵². This rule-based approach to knowledge representation is not just a cornerstone of KBAI but also a critical component in modeling how our minds process and organize information. Finally, the chapter introduces a specific learning mechanism known as chunking. This fascinating process showcases how complex information can be broken down, organized, and used, enabling both KBAI systems and humans to enhance their cognitive capabilities and knowledge base. By the end of this chapter, you will gain a comprehensive understanding of production systems as a cognitive architecture, the intricacies of rule-based knowledge representation, and the transformative power of learning through chunking. This knowledge will not only enrich your grasp of

fundamental KBAI concepts but also provide valuable insights into the parallels between artificial and human intelligence.

We begin our exploration into the concept of production systems through a relatable and engaging scenario: a baseball game.⁵³ Picture yourself as a pitcher in a riveting match between the Atlanta Braves and the Arizona Diamondbacks. You're standing on the mound, faced with a crucial decision: should you pitch the ball to the batter or opt to walk the batter instead?

Context Driven Decision Making: A Baseball Final Out Example

Imagine that it's the top of the 7th inning. There are runners on 2nd and 3rd base. There are two outs. The Diamondback's batter has an average of .256 and bats fourth in the batting order. The Braves are winning 3–2. The pitcher has struck out this batter at his last at bat. The pitcher's goal is to escape the inning without the Diamondback scoring additional runs. The pitcher can pitch or intentionally walk the batter.



Figure 5-1 Illustrative baseball example.

Ashok David, in your opinion, what action should the pitcher take?

David Well, Ashok, my thought is that the pitcher would intentionally walk the batter. This is due to a tactic in baseball known as a 'force out.' In this situation, the second and third bases are occupied, but the first base is not. With two outs already on the board, the team only needs one more out to conclude the inning. Therefore, the most strategic move is to walk the batter, thereby filling the first base and

setting up a potential force out situation, which is an easier way to secure that final out.

Ashok Your understanding of baseball is clearly more in-depth than mine, David. I'll trust your judgment on this. But let's reflect on what's occurring here. You've applied your extensive baseball knowledge to arrive at a strategic decision. Now, the key question is: how are you leveraging this knowledge? What cognitive architecture or reasoning process is guiding you towards this specific decision? This brings us to a fundamental aspect of our chapter – understanding the mechanisms through which intelligent agents process information and make complex decisions in various contexts.

This setup is a classic example of a production system in action. In the world of cognitive architectures, production systems represent knowledge as a set of rules. These rules guide decision-making processes, akin to a pitcher deciding on their next move in a game. As we progress through this chapter, we'll unravel how production systems function, especially in situations where the knowledge domain isn't fully known or understood. It's an exploration of how rules can guide intelligent behavior, even in scenarios where expertise is limited or absent.

COGNITIVE ARCHITECTURES

Cognitive Agents

In our exploration of cognitive agents within modern AI, it's essential to first define what we mean by a “cognitive agent.”⁵⁴ Various perspectives exist, but one widely accepted definition in contemporary AI views a cognitive agent as a function that translates a history of perceptual experiences into a specific action. In formal terms, this is represented as the function:

$$f: P^* \rightarrow A$$

where P^* symbolizes the cumulative history of percepts, and A represents the action.

This concept underscores the primary role of cognitive agents: to continually decide on the next course of action. Whether it's navigating a vehicle, choosing what to do during a meal, or responding in a conversation; cognitive agents, including humans, are perpetually engaged in action selection.⁵⁵ This decision-making process is heavily influenced by the accumulated sensory information and experiences in each context, or a history of percepts. This succinct definition captures the essence of a cognitive agent, emphasizing its role in processing perceptual history to inform subsequent actions.

Theories of Cognitive Architectures (Marr's and Newell's Levels)

In knowledge-based AI, we can establish theories at multiple levels of abstraction, as depicted in the following figure.^{56,57}

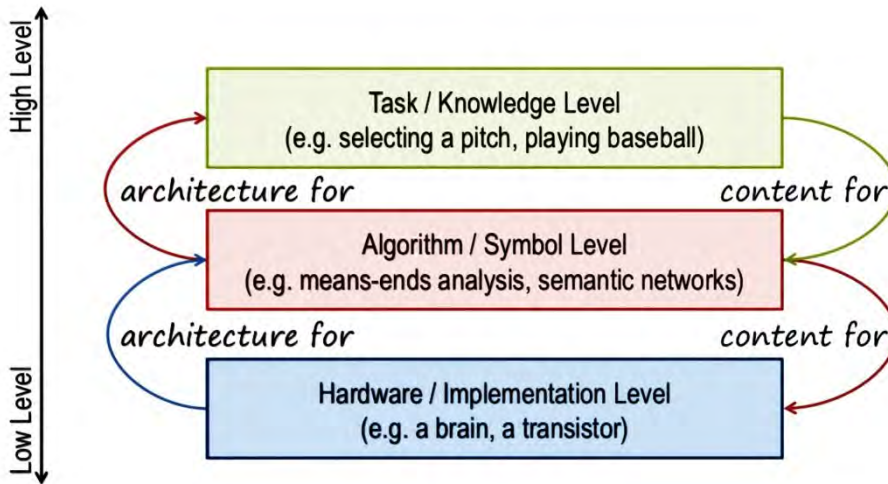


Figure 5-2 Cognitive Architecture Layers.

- **Hardware/Implementation Level (Brain, Transistor):** At the foundational or lowest level, we discuss the intricacies of hardware, like brains or microchip technology.^{58,59}
- **Algorithm/Symbol Level (MEA, Semantic Networks):** Moving up a level, we engage with methods and representations, such as the algorithm behind Means-Ends Analysis or the symbolic framework of Semantic Networks.
- **Task/Knowledge Level (Selecting a pitch, Playing baseball):** Ascending further to the highest level, we encounter the domain of knowledge and tasks, probing into the nature of decision-making and the specific knowledge required for various tasks. This was exemplified in our earlier discussion, where David's response to the baseball scenario demonstrated his application of substantial knowledge to a particular task.

Historically, prominent AI thinkers like David Marr and Allen Newell have emphasized these multi-tiered perspectives. Marr highlighted three levels: computational theory (task level), algorithms, and the implementation level. Newell, meanwhile, spoke of the knowledge level, the symbol level, and foundational levels like hardware. Each level interconnects with others; the hardware layer serves as the implementation ground for algorithmic processes, which in turn provides an architecture for task-level activities. Conversely, the task level informs the content

representation for algorithms, which then shapes what is to be manipulated or processed at the hardware level.

Examples of Cognitive Architectures

Understanding the decision-making process in a given task can lead to the identification of suitable algorithms and methods for that decision. Consider the representation of knowledge in a semantic network. The content of what is represented - say, the knowledge required to play baseball - emanates from the knowledge level. Once you have this content, it can be manifested in various forms, such as a semantic network. Similarly, the decision-making process at a high level dictates the choice of methods for achieving those decisions. This interplay between levels emphasizes the necessity of considering multiple perspectives when analyzing intelligent systems, be it a semantic network's role in knowledge representation or the choice of algorithms based on task requirements.

Imagine encountering a smartphone for the first time, perhaps as a visitor from Mars, unfamiliar with Earth's technology. In seeking to understand this device, you might engage with explanations on several levels. At a basic level, you might learn about the device's hardware – the intricate dance of microchips and sensors that power its operations. Looking deeper, you'd explore the algorithms and symbolic representations that enable its functions, from navigating maps to sending messages. But to truly grasp the smartphone's essence, you'd consider its purpose and the knowledge it embodies. This is where the conversation shifts from circuits and code to the phone's role in human life – it connects us across vast distances, enabling communication and access to information in ways once unimaginable. Each perspective offers a valid lens through which to view the smartphone, yet no single view fully captures its complexity. Understanding the device requires integrating these levels of analysis, from the tangible hardware that cradles its capabilities to the abstract algorithms that drive its functions and the broader tasks and knowledge it serves. This multi-layered approach unveils the richness of cognitive systems, revealing how they blend physical form, computational processes, and the fulfillment of human needs and tasks.

This approach of layered analysis, familiar from our understanding of smartphones, extends intriguingly to other devices. Take, for instance, a calculator. Its functionality, ranging from the hardware buttons to the digital display and the underlying algorithms that compute our inputs, can be dissected in similar layers. But what about intelligent agents, those entities designed to mimic human cognition or perform tasks autonomously? Can we apply this multi-layered framework to understand both natural cognitive systems, like the human mind, and artificial ones, crafted by technology? Are these distinct layers (e.g., hardware, algorithms, and task-knowledge) relevant when we scrutinize cognitive systems, regardless of their origin in nature or a lab? At which of these layers should we anchor our theories? These questions not only challenge us to broaden our analytical scope but also to pinpoint where the essence of cognition truly resides, guiding us in our quest to understand and replicate intelligence.

This layered analytical approach is not just applicable to devices like smartphones or calculators but extends to intelligent agents, both natural and artificial. These layers offer valuable insights into the workings of cognitive systems. It suggests that understanding cognitive systems requires considering multiple levels of abstraction, from the tasks and knowledge they encompass to the algorithms they employ and the hardware they operate on. Knowledge about tasks and the required knowledge informs the necessary algorithms and representations, which in turn are shaped by the available hardware. Conversely, the nature of the hardware provides possibilities and limits for algorithms and knowledge applications, which then influence what tasks can be accomplished and what knowledge is applicable. This reciprocal relationship underscores the need for a holistic view. Knowing the tasks and knowledge at play guides us towards suitable algorithms and representations, which in turn, depend on the hardware's affordances. Conversely, our hardware's characteristics can dictate the feasibility of certain algorithms and the complexity of knowledge we can model, influencing the tasks we can undertake. Our focus in this book will primarily be on the top two layers of abstraction, though we will occasionally touch upon the hardware level, highlighting the rich interplay between tasks, algorithms, and representations. The field of AI predominantly operates within these conceptual and algorithmic areas, crafting and refining the tools and methods that underpin intelligent systems.

IBM Watson Example

Ashok Reflecting on our discussion about Watson as a cognitive system, let's look at the various abstraction layers involved in its analysis.^{17,60} David, what's your take on the layers at play within Watson?

David Considering our previous discussions, the foundational layer for Watson would be its hardware. Although I'm not entirely certain about the specifics of Watson's hardware, it's safe to assume it involves sophisticated computing machinery. In the next layer, the algorithmic layer, Watson employs advanced search and decision-making algorithms that enable it to navigate its vast knowledge base and reach conclusions. These capabilities are distinct from the specific content it's processing at any given moment. At the top layer, we focus on the immediate tasks, such as responding to a given clue. This involves parsing the clue's content, applying relevant knowledge, and formulating an answer tailored to that specific clue.

Ashok Nicely put, David. You've clearly outlined the three tiers. Particularly at the task level, where answering a clue is central, it's imperative to recognize the role of knowledge. Understanding how Watson harnesses this knowledge, the representation forms it employs, and how these are orchestrated at the algorithmic layer is crucial.

Assumptions of a Cognitive Architecture

The cognitive architectures approach within AI rests on several foundational assumptions about cognitive agents:

- **Goal-Oriented Nature:** Cognitive agents are driven by goals, undertaking actions specifically aimed at achieving these objectives.
- **Complex Environments:** These agents operate within environments that are not only rich and multifaceted but also subject to change.
- **Knowledge Utilization:** To navigate and succeed in their dynamic surroundings, cognitive agents leverage their understanding of the world.
- **Knowledge Abstraction and Representation:** This knowledge is abstracted to emphasize essential aspects while omitting extraneous details, often represented symbolically to capture the world's nuances at a manageable level of abstraction.
- **Adaptive Behavior:** Flexibility is key; cognitive agents adapt their actions based on environmental cues and changes, ensuring their behavior is contextually relevant.
- **Learning Capability:** A hallmark of cognitive agents is their ability to learn continually, refining their knowledge and strategies through ongoing interaction with the environment.

These principles underscore the intricate nature of cognitive agents, emphasizing their adaptability, knowledge-driven actions, and the continual learning process that shapes their interaction with the world. The fundamental principle of cognitive architectures can be succinctly expressed through the equation: $\text{architecture} + \text{content} = \text{behavior}$:

$$\text{architecture} + \text{content} = \text{behavior}$$

This equation can be understood from two distinct angles: The Design perspective and the Flexibility perspective.

- **Design Perspective:** When the goal is to create an intelligent system with specific behavioral capabilities, this equation guides the process. It suggests that the desired behavior can be achieved by carefully crafting the system's underlying architecture and then populating this framework with the appropriate knowledge content. This dual focus on structure and information is crucial for engineering intelligent behavior.
- **Flexibility Perspective:** Consider a scenario where the architecture of a system is predetermined or fixed. Under these circumstances, varying the knowledge content within this stable architectural framework can lead to a range of behaviors. This perspective highlights the transformative power of knowledge content in shaping behavior, even when the structural foundation remains constant. This approach offers a streamlined path to modifying behavior, emphasizing the pivotal role of knowledge in driving diverse outcomes within a given architectural context.

This concept mirrors principles in computer architecture, where the hardware remains constant while the software, or stored programs, change to produce different outputs. Just as changing the code alters the behavior of a computer without modifying its physical structure, varying the knowledge within a cognitive architecture can lead to diverse behaviors. This approach underscores the importance of identifying effective architectures, a challenge that is central to the fields of artificial intelligence and cognitive science.

SOAR: AN EXEMPLAR COGNITIVE ARCHITECTURE FOR PRODUCTION SYSTEMS

In our exploration of cognitive architectures, we encounter the high-level framework of deliberation, which encompasses not just the process of thoughtful consideration but also touches on reactive behaviors and aspects of meta-cognition.⁶¹ A quintessential architecture that epitomizes this comprehensive approach is **Soar** (also called SOAR), developed by pioneers Allen Newell, John Laird, and Paul Rosenbloom.^{62,63} Over three decades, Laird and Rosenbloom have continued to evolve Soar, enriching its capabilities and applications.

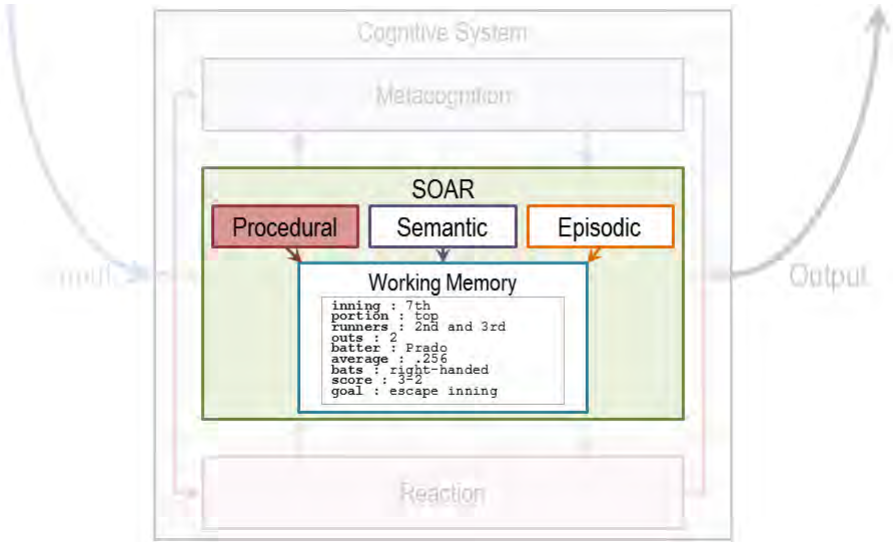


Figure 5-3 Soar High-Level Architecture diagram.

At the core of Soar lies a bifurcation of memory into long-term and working components, each serving distinct functions in the cognitive process. The long-term memory is further subdivided into procedural, semantic, and episodic categories, each storing a unique type of knowledge. Episodic memory catalogs specific events and experiences – recalling, for example,

last night's dinner. Semantic memory, on the other hand, houses generalizations, concepts, and models, like the understanding of what constitutes a human or the principles of aerodynamics that allow a plane to soar. Procedural memory is all about the 'how-tos', encapsulating the steps for tasks such as pouring water from a jug into a glass.

This structured interplay of memory types forms the backbone of Soar's architecture, enabling a rich tapestry of reasoning and learning processes.⁶⁴ It's this orchestrated interaction among Soar's components that facilitates a broad spectrum of cognitive functions, from the deliberative to the reactive and beyond, offering a window into the intricate workings of both artificial and natural intelligence systems.

PRODUCTION SYSTEMS EXPLAINED

Returning to our baseball pitcher scenario, where the pitcher must choose his next move under pressure, we envision him as a cognitive agent within a production system framework. This pitcher, tasked with a critical decision, effectively maps his accumulated perceptions to a decisive action. Drawing from the detailed scenario previously discussed, where David suggested the strategic move of intentionally walking the batter, we look into the cognitive mechanics behind such a decision.

Imagine this pitcher's mind as a complex system, synthesizing both internal knowledge and external perceptions. He is not just reacting; he's deliberating, weighing the intricacies of the game's current state – bases occupied, batter's stance, game score, and player positions – all these external cues enrich his situational awareness. But there's more; internally, he harbors goals and strategies, a deep-seated understanding of the game's nuances, and perhaps even an intuition honed by experience. This rich tapestry of external perceptions and internal knowledge forms the basis for his decision-making. How does he translate this complex web of information into a single, strategic action on the mound? This is where we bridge human cognition and artificial intelligence, crafting theories that elucidate the decision-making processes of both the flesh-and-blood pitcher and a hypothetical AI agent designed to emulate such nuanced reasoning. Through this lens, we explore how a pitcher's amalgam of perceived realities and ingrained knowledge coalesces into the tactical decision to walk the batter intentionally, showcasing the intricate dance of cognition and strategy in the heat of the game.

Picture Kris Medlen of the Atlanta Braves on the pitcher's mound, with Martin Prado of the Arizona Diamondbacks ready at bat. Medlen's overarching goal: to end the inning without conceding any runs. How does he navigate this high-stakes decision? Let's look into Medlen's cognitive process, unfolding his strategic considerations.

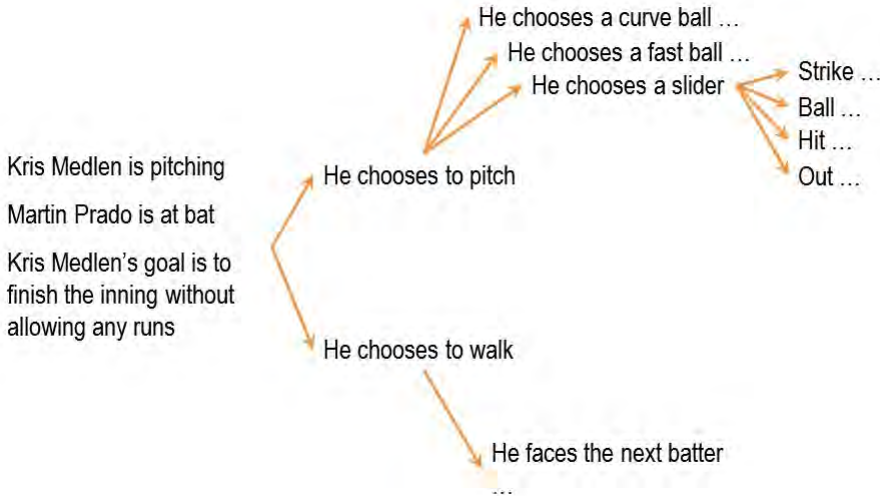


Figure 5-4 Conceptualized action state-space for the pitcher in the Braves vs Diamondbacks game.

Medlen faces a fork in the road: to pitch directly to Prado or to intentionally walk him. Choosing to walk Prado unfurls a new set of possibilities, including confronting the subsequent batter. Opting to pitch, however, demands another layer of decision-making – what type of pitch to deliver: a slider, a fastball, or a curveball? Each choice branches into further outcomes: a potential strike, ball, hit, or the coveted strikeout, expanding the tapestry of possible game states.

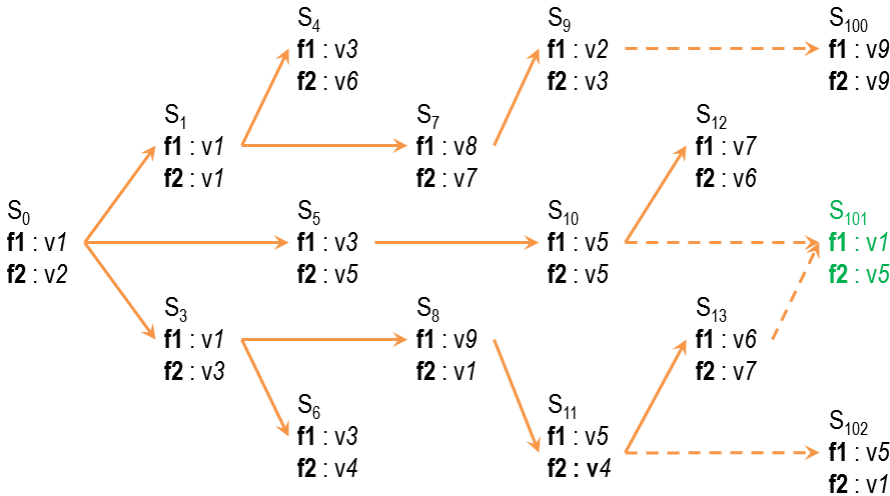


Figure 5-5 Abstract Action state-space diagram.

This mental exercise Medlen undergoes can be formalized into a concept known as a state-space – a comprehensive map of all conceivable outcomes stemming from varying combinations of actions, starting from the current moment. Each point in this space, each potential game state, is defined by specific features (f1, f2, etc.), each with its range of values (v1, etc.). Initially, Medlen is at state s0, aiming for a state S101, where his goal is realized, the inning ends favorably. Before any physical move is made, Medlen mentally constructs and navigates this abstract state-space, contemplating the best path from his current position to the desired outcome. This cognitive journey through potential futures is the essence of strategic decision-making in the intense milieu of a baseball game.

To go further into our analysis, let's translate the situational elements and Kris Medlen's goal into a structured format compatible with the cognitive architecture of SOAR. This translation involves breaking down the scenario into discrete features with corresponding values, creating a framework that SOAR can process. For instance, we denote the inning as the 7th, indicating a crucial juncture in the game. This detail is captured as Inning: 7th. The fact that it's the top of the inning is also crucial, represented as Inning Part: Top. The positioning of runners on 2nd and 3rd base is another critical piece of information, noted as Runners: 2nd and 3rd.

At the heart of this scenario lies Medlen's overarching objective: to conclude the inning without conceding any runs, ensuring the opposing team doesn't add to their score. This goal is succinctly encapsulated as Goal: Escape Inning Without Runs.

*It's the top of the 7th
inning. There are runners
on 2nd and 3rd base.*

*There are two outs. The
batter, Martin Prado, has
an average of .256 and
bats fourth in the batting
order. We are winning 3-
2. I struck this batter out
last time. My goal is to
escape the inning.*



inning : 7th
portion : top
runners : 2nd and 3rd
outs : 2
batter : Prado
average : .287
bats : right-handed
score : 3-2
goal : escape inning

Figure 5-6 Mapping content into the Working Memory of the Soar architecture.

This structured representation of the game's state and Medlen's intentions serves as the foundation for the subsequent steps in the decision-making process. In Soar's framework, the working memory is now populated with a mix of external perceptions and the pitcher's internal goals, reflecting the complex situation on the baseball field. This memory acts as a trigger,

activating various rules stored within Soar's extensive long-term memory, particularly within its procedural knowledge base. These rules are formulated as **Production Rules**, a foundational concept in Soar that lends its name to the broader term Production Systems.

<pre> inning : 7th portion : top runners : 2nd and 3rd outs : 2 batter : Prado average : .256 bats : right-handed score : 3-2 goal : escape inning </pre>	<pre> (r1) If goal is to <i>Escape</i>, I perceive 2 outs, I perceive a runner on 2nd and I perceive no runner on 1st <u>then suggest goal intentionally walk batter</u> (<i>Intentional Walk</i>) (r2) If goal is to <i>Escape</i>, I perceive fewer than 2 outs, or I perceive a runner on 1st, or I perceive no runner on 2nd, or I perceive no runners <u>then suggest goal to get the batter out via pitching</u> (<i>Pitch</i>) (r3) If goal is <i>Intentional Walk</i> <u>then suggest intentional-walk operator</u> (r4) If goal is <i>Pitch</i> and I perceive a new batter who is left/right-handed <u>then add batter not out, balls 0, strikes 0, bats left/right</u> (r5) If the goal is <i>Pitch</i> and batter not out <u>then suggest throw-curve-ball operator</u> (r6) If the goal is <i>Pitch</i> and batter not out and bats left-handed <u>then suggest throw-fast-ball operator</u> (r7) If only one operator has been selected <u>then send operator to the motor system and add pitch thrown to state</u> </pre>
--	---

Figure 5-7 Example Production Roles for the pitcher in the baseball game.

Imagine a set of production rules like those depicted in the accompanying figure. These rules, seven in number for illustrative purposes, though there could be many more, guide the pitcher's decision-making process. They range from determining whether to pitch the ball or opt for an intentional walk, to more nuanced decisions based on the game's current state. For instance, one rule (Rule 1) might state: "If the objective is to conclude the inning without complications, and there are two outs with a runner on second but none on first, then the advisable strategy is to intentionally walk the batter." This is a tactical move aimed at setting up a more favorable situation for the pitcher. Another rule (Rule 2) could suggest: "If the goal is to end the inning and there are two or fewer outs, with the possibility of a runner on first or the bases less occupied, then the recommendation is to strive for an out by pitching to the batter." This reflects a more aggressive approach, relying on the pitcher's skill to resolve the inning. Further exploring these rules, we encounter one (Rule 3) that elaborates on the specifics of intentionally walking a batter, indicating the precise conditions under which this strategy becomes viable and the corresponding action to take. Let's explore an illustrative rule, labeled as Rule 7: "If a single operator has been selected" for action, then communicate this chosen operator to the motor system for execution, and concurrently update the working memory to reflect that the pitch has been thrown. This integration of action selection and execution with memory updates encapsulates the dynamic nature of decision-making in Soar, ensuring that the system's internal state remains in sync with its actions.

To encapsulate what we've covered, Soar's long-term memory is a repository for diverse types of knowledge, with a current focus on procedural knowledge. This subset of knowledge, dedicated to the 'how-to' aspect of tasks, is encapsulated within production rules. Each rule follows a structured "if-then" format, containing conditions (antecedents) that, when met, trigger specified consequences. These conditions can be interconnected with logical operators like "and" and "or," allowing for complex decision-making criteria that can handle a variety of situations. The content within these production rules is intricate, reflecting the goal-oriented nature of cognitive architectures. Goals are prominently featured across various rules, guiding the system's actions in alignment with its objectives. This detailed and specific knowledge embedded within the rules exemplifies the depth and breadth of information that cognitive systems like Soar utilize. It's this richness of knowledge that enables Soar to adeptly respond to a wide array of perceptual inputs from the environment, activating relevant rules that guide the system towards its goals. These production rules, encoded within Soar's procedural knowledge, are instrumental in navigating the pitcher through the intricate decision-making required in the heat of a baseball game. They not only offer a structured approach to evaluating the current state of play but also ensure that the pitcher's actions are aligned with the overarching goal of maintaining a strategic advantage.

The Recognize-Act Cycle

Let's listen as Ashok and David examine how the Soar architecture might be applied.⁶⁵

Ashok Considering the procedural knowledge within Soar's long-term memory and the current state of working memory, which includes both the pitcher's perceptions and objectives, we face a crucial decision point. The options include 'Intentional-walk', 'Throw-curve-ball', 'Throw-fast-ball', or potentially, the system might not be able to make a decision. David, which direction do you reckon Soar will lean towards?

David It seems Soar is in agreement with my earlier prediction. The initial production rule comes into play here, given the scenario of two outs, a runner on second, and first base unoccupied, pointing towards the goal of an intentional walk. Subsequently, this goal triggers the specific operation of 'Intentional-walk', as outlined in another rule, sidelining the other potential actions. Therefore, the system is poised to select 'Intentional-walk'.

Ashok Precisely, David. The dynamics here are fascinating. The working memory's current data interacts with the procedural knowledge embedded in the long-term memory through production rules. This interaction leads to the 'firing' of specific rules based on their alignment with the working memory's state. As you deduced,

the activation of one rule can cascade, activating further related rules, which in this instance leads to choosing the 'Intentional-walk' action. This process continues iteratively until no new rules are activated, culminating in a decisive action based on the consequences of the engaged rules. This sequence, from perception to action, is the essence of how Soar, and by extension, cognitive agents, navigate decision-making processes.

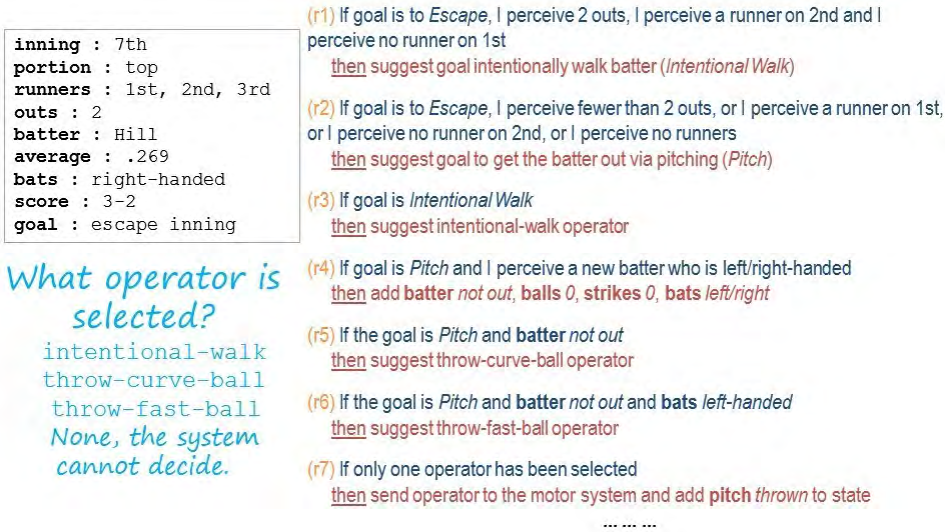


Figure 5-8 Next state with new working member (left) and long-term memory (right).

Ashok With the successful walk of the batter, our scenario evolves. Runners now occupy all bases, altering the pitcher's situation and, consequently, the content of the working memory. Yet, our set of procedural knowledge, or production rules, remains the same. This shift in the game's state poses a new decision-making challenge for our pitcher. David, given this new configuration, what's your take on the pitcher's next move?

David This time, the situation diverges from the previous. The presence of a runner on the first base nullifies the first production rule, steering us away from an intentional walk. However, the second rule now comes into play, prompted by the runner on the first base, directing the pitcher towards engaging the batter with a pitch. This adjustment in strategy leads us to consider the 'throw-curve-ball' operator, given the current setup and the batter's stance. Thus, the system is inclined towards executing a curveball.

Ashok Excellent analysis, David. This exercise showcases the dynamic interplay between the working memory's evolving state and the static procedural knowledge in long-term memory. As the working memory's contents adapt to unfolding events, relevant rules are triggered, orchestrating the system's response. This continuous dialogue between memory components underscores the fluid nature of decision-making within cognitive architectures like Soar, where the constancy of procedural knowledge contrasts with the fluidity of the working memory, facilitating adaptive responses to changing circumstances.

<pre> inning : 7th portion : top runners : 1st, 2nd, 3rd outs : 2 batter : Parra average : .273 bats : left-handed score : 3-2 goal : escape inning </pre>	(r1) If goal is to <i>Escape</i>, I perceive 2 outs, I perceive a runner on 2nd and I perceive no runner on 1st <u>then</u> suggest goal intentionally walk batter (<i>Intentional Walk</i>) (r2) If goal is to <i>Escape</i>, I perceive fewer than 2 outs, or I perceive a runner on 1st, or I perceive no runner on 2nd, or I perceive no runners <u>then</u> suggest goal to get the batter out via pitching (<i>Pitch</i>) (r3) If goal is <i>Intentional Walk</i> <u>then</u> suggest intentional-walk operator (r4) If goal is <i>Pitch</i> and I perceive a new batter who is left/right-handed <u>then</u> add batter not out, balls 0, strikes 0, bats left/right (r5) If the goal is <i>Pitch</i> and batter not out <u>then</u> suggest throw-curve-ball operator (r6) If the goal is <i>Pitch</i> and batter not out and bats left-handed <u>then</u> suggest throw-fast-ball operator (r7) If only one operator has been selected <u>then</u> send operator to the motor system and add pitch thrown to state
--	--

What operator is selected?
o intentional-walk
o throw-curve-ball
o throw-fast-ball
o None, the system cannot decide.

Figure 5-9 Next state with new working member (left) and long-term memory (right) after switching batters.

Ashok This twist adds complexity, David. Suppose the Arizona Diamondbacks' manager outsmarts the pitcher by swapping in a left-handed batter (see above figure). This minor change in the lineup alters the pitcher's perception, tweaking the content of the working memory, yet our procedural knowledge – our set of production rules – remains unchanged. In this altered scenario, what decision might our pitcher lean towards? David, your thoughts?

David Following the previous logic, we'd initially aim for a pitch to get the batter out. The working memory updates accordingly, but we hit a snag with rules four through six. While the intention to pitch propels us towards the 'throw-curve-ball operator', the batter's left-handed stance introduces the 'throw-fast-ball operator' into the mix. This results in a problem when we reach rule seven, which expects a

singular operator choice, thereby stalling the decision-making process. Essentially, we're stuck with multiple operators and no clear directive on how to proceed.

Ashok Precisely, David, and that's the crux of the issue. Up until this point, we operated under the assumption that the interplay between percepts and procedural knowledge would unequivocally guide us to a definitive single action, even if it required navigating through a series of preliminary rules. However, we're now faced with a scenario where the system is paralyzed by ambiguity, lacking a clear rule to dictate the next move. This highlights a potential shortfall in our system's ability to adapt to nuanced changes in the environment.

In the discussion above, the Soar cognitive architecture encountered a dilemma, choosing not two distinct actions but facing a conundrum between them. This scenario in Soar's framework is termed an impasse, a state of uncertainty where a decision cannot be made. This impasse arises either from a lack of sufficient knowledge or from the presence of several viable actions without a clear method for choosing among them. Specifically, in the previous example, the pitcher found himself in a bind, unable to choose between delivering a curve ball or a fastball. At this juncture, Soar engages in a learning process aimed at resolving the impasse. The architecture seeks to derive a new rule that could provide a clear directive in similar future situations, particularly when faced with the choice between a fastball and a curveball. To facilitate this learning, Soar taps into its episodic memory, recalling past instances that might shed light on the current predicament. This process illustrates how Soar leverages its experiences, stored as episodic knowledge, to learn and formulate a rule capable of breaking the deadlock and guiding decision-making in comparable scenarios.

LEARNING IN PRODUCTION SYSTEMS: THE CHUNKING PROCESS

When Soar encounters an impasse (e.g., unable to choose between a curveball and a fastball), it initiates a learning process to derive a new rule. This involves its episodic memory.

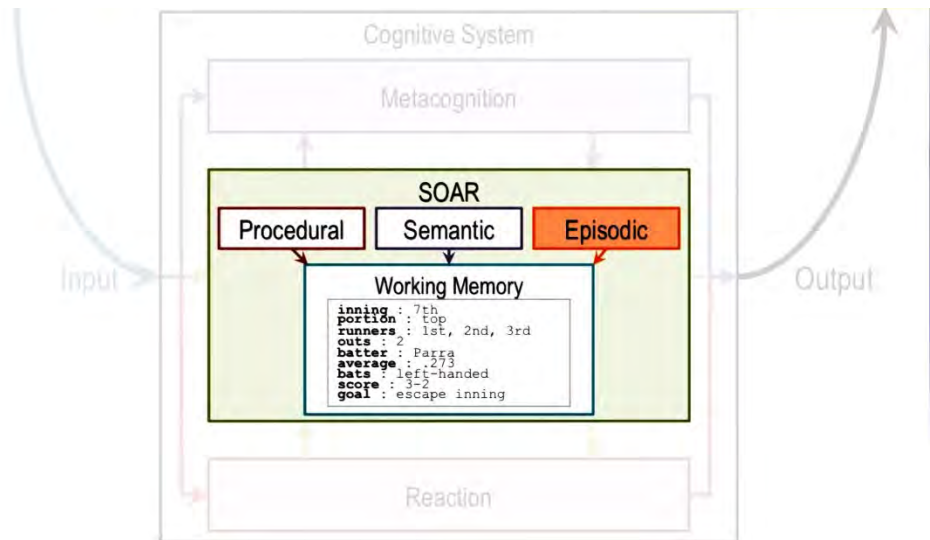


Figure 5-10 Soar High-Level Architecture diagram emphasizing Episodic content.

In envisioning Soar's episodic memory, consider it as a repository of past occurrences, storing detailed accounts of previous game scenarios, including one where, in the fifth inning with gusty winds, the pitcher faced Parra, a left-handed batter. In that instance, opting for a fastball resulted in Parra hitting a home run, a situation the current pitcher aims to sidestep. Drawing from this episodic recollection, Soar activates its learning mechanism, adept at transforming insights from past experiences into actionable rules for future guidance.⁶⁶ This process, known as chunking, synthesizes the essence of the episodic memory into a procedural rule that enriches Soar's decision-making arsenal. The newly minted rule articulates a clear directive: if faced with a choice between two actions, one being a fastball, and the batter is Parra, then the fastball option should be eliminated from consideration. This rule embodies the learning from the previous encounter, advising against a repeat of the strategy that led to an unfavorable outcome. Through chunking, Soar not only learns from its past but also adapts its procedural knowledge to navigate similar challenges more adeptly in the future, showcasing the dynamic interplay between memory and decision-making in cognitive architectures.

Chunking emerges as a pivotal learning strategy within Soar, specifically designed to navigate through and resolve impasses. It's a mechanism that springs into action when Soar encounters a decision deadlock, such as the one where two competing rules are activated without a clear path forward. This deadlock, or impasse, not only signals a challenge but also delineates the objective for chunking: to forge a rule that can clear the impasse. Triggered by the impasse, Soar looks into its episodic memory, seeking parallels between the current predicament and past experiences. It scrutinizes the present situation's perceptual cues against those archived from previous events, looking for insights that could offer a resolution. A key factor in this search is

the relevance of the information to the current quandary, particularly any outcomes tied to similar actions faced in the impasse. Upon identifying a pertinent episodic memory – for instance, the adverse outcome of a fastball to a specific batter – Soar endeavors to distill this experience into a procedural rule. This rule is tailored to avert the undesired outcome, in this case, advising against choosing the fastball option when facing the same batter. This encapsulation of episodic learning into procedural knowledge exemplifies the dynamic interplay between Soar's memory components and its decision-making processes.

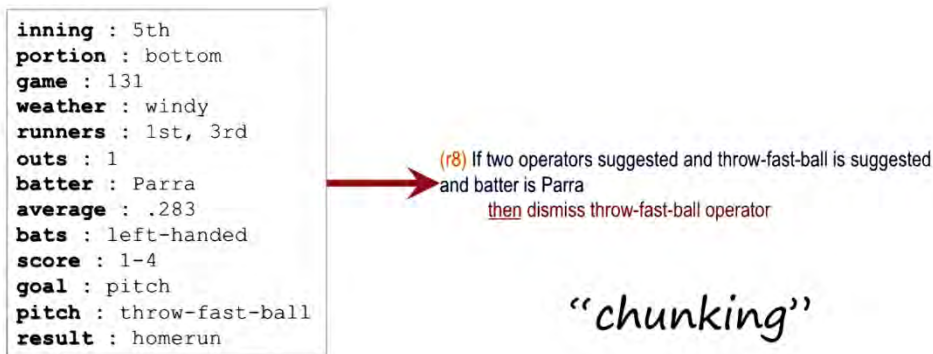


Figure 5-11 Chunking to dismiss a rule.

The chunking process underscores the intricate linkage between reasoning, learning, and memory within cognitive architectures like Soar. It showcases how procedural and episodic memories converge to inform decision-making and how learning mechanisms like chunking adaptively refine Soar's knowledge base. For those intrigued by chunking and its implications in cognitive systems, further exploration is available in the detailed study by Lehman, Laird, and Rosenbloom, which offers a comprehensive insight into this aspect of AI learning.

Let's rejoin Ashok and David as they explore another scenario in their ongoing discussion:

Ashok Notice that we've incorporated an additional rule into the procedural knowledge, which is rule 8. This new rule is the outcome of the chunking process we discussed earlier. It states: “If two operators are suggested, and the throw-fast-ball operator is one of them, and the batter is Parra, then dismiss the throw-fast-ball operator.” Given this updated set of rules and the same game situation, what do you predict will be the selected operator? David, could you share your thoughts?

David It appears that the entire purpose of introducing chunking was to resolve the dilemma we faced between two competing operators. According to the new rule,

if the situation involves a fastball being considered among two possible operators, and the batter at the plate is Parra, then the rule directs us to eliminate the fastball option. This leaves us with just one viable operator, which is to throw a curveball. Therefore, that's the action we'll take.

```

inning : 7th
portion : top
runners : 1st, 2nd, 3rd
outs : 2
batter : Parra
average : .273
bats : left-handed
score : 3-2
goal : escape inning

```

What operator is selected?

- ☐ intentional-walk
- ☒ throw-curve-ball
- ☐ throw-fast-ball
- ☐ None, the system cannot decide.

(r1) If goal is to *Escape*, I perceive 2 outs, I perceive a runner on 2nd and I perceive no runner on 1st
then suggest goal intentionally walk batter (Intentional Walk)

(r2) If goal is to *Escape*, I perceive fewer than 2 outs, or I perceive a runner on 1st, or I perceive no runner on 2nd, or I perceive no runners
then suggest goal to get the batter out via pitching (Pitch)

(r3) If goal is *Intentional Walk*
then suggest intentional-walk operator

(r4) If goal is *Pitch* and I perceive a new batter who is left/right-handed
then add batter not out, balls 0, strikes 0, bats left/right

(r5) If the goal is *Pitch* and **batter not out**
then suggest throw-curve-ball operator

(r6) If the goal is *Pitch* and **batter not out** and **bats left-handed**
then suggest throw-fast-ball operator

(r7) If only one operator has been selected
then send operator to the motor system and add pitch thrown to state

(r8) If two operators suggested and throw-fast-ball is suggested and batter is Parra
then dismiss throw-fast-ball operator

Figure 5-12 Example of Soar selecting an action operator with one rule dismissed due to chunking.

Ashok Exactly, David, and that's a splendid illustration of how chunking can effectively refine decision-making processes within cognitive architectures. It demonstrates the capacity for learning from past experiences and applying that knowledge to similar future situations, thus enhancing the system's problem-solving capabilities.

In our introduction of the chunking process, we've broached the subject of learning, a pivotal aspect of cognitive systems. This leads us to ponder several interrelated questions: *What is it that agents learn? From where does this learning originate? When does this learning occur, and what prompts it?* These questions do not stand alone; they're part of a larger line of questioning into the objectives and purposes that learning serves for an agent. Knowledge-based AI posits a compelling approach to these inquiries. It suggests beginning with a robust theory of reasoning. Such a theory can illuminate crucial aspects like what should be learned, the timing of learning, and the motivations behind it. This reasoned foundation then paves the way to exploring the methodologies of learning. This approach was exemplified in our exploration of production systems. When faced with an impasse — an obstacle in decision-making — the system turned to learning to navigate through the predicament, drawing upon episodic memories. This

technique underscores a broader principle: the quest for a cohesive theory that intertwines reasoning, memory, and learning. In this integrated framework, the imperatives of memory and reasoning define and inform the bounds and pathways for learning, underscoring the interconnectedness of these cognitive facets.⁶⁷

PRODUCTION SYSTEMS AND COGNITION

Production systems, as exemplified by architectures like Soar, offer significant insights into human cognition:

- **Skill Acquisition:** They model how humans acquire and automate skills. Initially, tasks might require conscious rule application (deliberative thought). With practice and chunking (learning from experience), these rules become more compiled and efficient, leading to more automatic, proceduralized behavior. This mirrors how humans learn to drive a car or play a musical instrument.
- **Routine Behavior:** Many everyday human actions can be seen as the execution of learned production rules. When conditions in our environment match the IF part of a mental “rule,” we execute the THEN part, often without extensive conscious thought.
- **Working Memory Parallels:** The concept of a working memory in production systems, which holds the current state of the problem and relevant information, has parallels with human short-term memory and its role in active processing and decision-making.
- **Problem Solving and Impasses:** The way production systems handle impasses (by setting up subgoals or learning new rules) can be a model for human problem-solving when faced with novel or difficult situations. When we don't know what to do next, we might reflect on past experiences (episodic memory) or try to break the problem down.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

While production systems are a cornerstone of classical KBAI, their principles and functionalities connect to and can be enhanced by modern AI paradigms:

- **Rule-Based Expert Systems & Modern Counterparts:** Classical expert systems heavily relied on production rules. Modern machine learning offers techniques like **decision tree learning** that can automatically learn rule sets from data, offering a data-driven way to populate the procedural memory of a production system.
- **Reinforcement Learning (RL):** RL agents learn policies (which can be viewed as a set of implicit or explicit rules mapping states to actions) through trial and error. The KBAI concept of an agent mapping its perceptual history (P*) to an action resonates

with RL, where an agent's current state in a Markov Decision Process (MDP) ideally summarizes all relevant past percepts, or a belief state is formed from this history in Partially Observable MDPs (POMDPs). The “chunking” mechanism in Soar, where new rules are learned to resolve impasses, has conceptual similarities to how RL agents might learn specific sub-policies or heuristics to handle critical states or improve decision-making in certain situations. For example, learning a specific rule after an “impasse” (a state with high uncertainty or negative reward) can be seen as analogous to policy refinement in RL.

- **Neuro-Symbolic AI:** This field aims to combine the strengths of neural networks (learning from raw data) and symbolic reasoning (like production rules). One could imagine a system where a neural network handles perception or complex pattern recognition to populate the working memory of a production system, which then uses its explicit rules for higher-level reasoning and decision-making. Conversely, the output or the reasoning trace of a production system could guide the learning or attention mechanism of a neural network, or provide structured explanations for its behavior.
- **Agent Architectures (e.g., BDI):** Agent architectures like BDI (Beliefs, Desires, Intentions) often use rule-like structures for decision-making and plan execution. The procedural knowledge in production systems aligns well with the plan libraries or rule sets that BDI agents might use to select actions based on their current beliefs and intentions. The recognize-act cycle of production systems is a fundamental processing loop for many agent architectures.
- **Explainable AI (XAI):** Because production systems operate on explicit IF-THEN rules, their reasoning process is inherently more transparent and explainable than “black box” models like many deep neural networks. The sequence of rule firings can serve as a direct explanation for an agent's decision. Techniques from production systems could inspire methods for extracting or representing explanations in other AI systems, aiming to make their operations more understandable to humans.
- **Natural Language Processing (NLP) & Generative AI (GenAI):** While not a direct replacement, rule-based systems can complement statistical NLP and GenAI. For instance, production rules could be used to enforce constraints on language generation, ensure grammatical correctness in specific contexts, or implement dialogue management strategies based on recognized user intents and dialogue states. They can also be used to parse specific types of structured information from text.

Integrating the explicit, rule-based reasoning of production systems with the learning capabilities of modern ML can lead to AI systems that are both powerful in their learning and transparent in their reasoning.

SUMMARY

This chapter offered an in-depth look at Production Systems, a foundational KBAI topic that illustrates how rule-based systems can model complex decision-making and learning. We started by placing production systems within the larger framework of cognitive architectures, discussing various levels of abstraction (hardware, algorithm/symbol, knowledge/task) and introducing Soar as a key example, highlighting its memory components (working, and long-term procedural, semantic, and episodic).

The core mechanism of production systems, the recognize-act cycle, was detailed using an illustrative baseball scenario. This showed how IF-THEN rules stored in procedural memory are matched against the current state of working memory (containing perceptions and goals) to trigger actions. We saw how this cycle allows an agent to make decisions by firing a sequence of rules.

A critical aspect discussed was how production systems learn, particularly through the chunking process in Soar. When the system reaches an impasse – a situation where no rule applies or multiple conflicting rules apply – it initiates a learning sub-process. By accessing relevant episodic memories, the system “chunks” this experience into a new production rule that can resolve similar impasses in the future, thereby improving its performance over time.

The chapter also underscored the strong cognitive connection of production systems, explaining how they model human skill acquisition (from deliberate rule-following to automatic execution), routine behaviors, and problem-solving strategies, with parallels drawn between working memory and human short-term memory.

Finally, we connected production systems to modern AI paradigms, illustrating their continued relevance. Their explicit rule-based nature makes them inherently explainable (XAI). Principles from production systems find echoes in rule induction algorithms, policy learning in Reinforcement Learning (where KBAI's P* or perceptual history relates to MDP states), the symbolic component of Neuro-Symbolic AI, the decision-making logic in agent architectures, and even in providing constraints or structure for NLP and Generative AI.

In essence, Production Systems provide a robust model for how knowledge, represented as rules, can drive sophisticated reasoning and adaptive learning, offering valuable insights into both artificial intelligence and human cognition, and demonstrating clear pathways for integration with contemporary AI approaches.

ADDITIONAL READINGS & REFERENCES

- 17 High, R. The era of cognitive systems: An inside look at IBM Watson and how it works. *IBM Corporation, Redbooks* 1, 16 (2012).
- 49 Davis, R. & King, J. *An overview of production systems*. (Stanford University, 1975).
- 50 Grosan, C., Abraham, A., Grosan, C. & Abraham, A. Rule-based expert systems. *Intelligent systems: A modern approach*, 149-185 (2011).

- 51 Masri, N. *et al.* Survey of rule-based systems. *International Journal of Academic Information Systems Research (IJAIRS)* **3**, 1-23 (2019).
- 52 Davis, R. & King, J. J. The origin of rule-based systems in AI. *Rule-based expert systems: The MYCIN experiments of the Stanford Heuristic Programming Project* (1984).
- 53 Soloway, E. in *Psychology of Learning and Motivation* Vol. 20 193-236 (Elsevier, 1986).
- 54 Huhns, M. N. & Singh, M. P. Cognitive agents. *IEEE Internet computing* **2**, 87-89 (2002).
- 55 de Visser, E. J. *et al.* in *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*. 263-267 (Sage Publications Sage CA: Los Angeles, CA).
- 56 Langley, P., Laird, J. E. & Rogers, S. Cognitive architectures: Research issues and challenges. *Cognitive Systems Research* **10**, 141-160 (2009).
- 57 Thagard, P. Cognitive architectures. *The Cambridge handbook of cognitive science* **3**, 50-70 (2012).
- 58 Goertzel, B., Lian, R., Arel, I., De Garis, H. & Chen, S. A world survey of artificial brain projects, Part II: Biologically inspired cognitive architectures. *Neurocomputing* **74**, 30-49 (2010).
- 59 Samsonovich, A. V. Emotional biologically inspired cognitive architecture. *Biologically inspired cognitive architectures* **6**, 109-125 (2013).
- 60 Gliozzo, A., Biran, O., Patwardhan, S. & McKeown, K. in *Proceedings of the Fourth Workshop on Teaching NLP and CL*. 85-92.
- 61 Huyck, C. R. A neural cognitive architecture. *Cognitive Systems Research* **59**, 171-178 (2020).
- 62 Laird, J. E., Newell, A. & Rosenbloom, P. S. Soar: An architecture for general intelligence. *Artificial intelligence* **33**, 1-64 (1987).
- 63 Wray, R. E. & Jones, R. M. An introduction to Soar as an agent architecture. *Cognition and multi-agent interaction: From cognitive modeling to social simulation*, 53-78 (2005).
- 64 Young, R. M. & Lewis, R. L. The Soar cognitive architecture and human working memory. *Models of working memory: Mechanisms of active maintenance and executive control*, 224-256 (1999).
- 65 Bridges, R. A. *et al.* Testing SOAR tools in use. *Computers & Security* **129**, 103201 (2023).
- 66 Howes, A. & Young, R. M. The role of cognitive architecture in modeling the user: Soar's learning mechanism. *Human-Computer Interaction* **12**, 311-343 (1997).
- 67 Chong, H.-Q., Tan, A.-H. & Ng, G.-W. Integrated cognitive architectures: a survey. *Artificial Intelligence Review* **28**, 103-130 (2007).

Chapter 6

Frames

INTRODUCTION TO FRAMES

In this chapter, we examine the intricate world of frames, uncovering their essential role as a fundamental component in the architecture of Knowledge-Based Artificial Intelligence (KBAI). As a versatile tool for knowledge representation, frames embody a cornerstone in our journey towards understanding and implementing common sense reasoning – a quest that underpins the aspirations of artificial intelligence to mimic the human ability to understand and interact with the world.

The discussion begins by exploring the function of frames. This section aims to shed light on why frames are integral to knowledge representation and what attributes render them exceptionally beneficial in various AI applications. By breaking down the intricacies of how frames operate, we aim to provide insight into their capability to organize and structure knowledge in a manner that is both accessible and meaningful for AI systems.

Following this, we will investigate several key properties of frames. These characteristics not only differentiate frames from other forms of knowledge representation but also highlight their unique capacity for enhancing the sophistication and depth of AI reasoning processes. Moreover, we will articulate the relationship between frames and other topics previously introduced in this book. By drawing connections to semantic networks and production systems, this discussion aims to contextualize frames within the broader landscape of knowledge representation techniques.^{68,69} Through this comparative analysis, we aim to elucidate how

frames complement and intersect with other models, thereby contributing to a more holistic understanding of AI knowledge structures.

Finally, the chapter will culminate in an examination of how frames facilitate advanced sense-making, particularly in story understanding. Through practical examples and theoretical insights, we will demonstrate how frames empower AI systems to parse and interpret complex narratives, thereby unlocking new dimensions of understanding that bridge the gap between mere data processing and genuine comprehension.

In summary, this chapter endeavors to provide a comprehensive overview of frames, from their foundational function in AI systems to their role in enabling sophisticated cognitive abilities like sense-making and story understanding. By the end of this exploration, readers should be well-equipped with a deep understanding of the significance of frames in the development of intelligent, perceptive AI agents.

We identified frames as a useful knowledge representation tool for making machines think like us, especially when it comes to common sense reasoning. But you might ask, “What exactly is common sense reasoning?” Well, it's something both you and I do without even thinking about it. The real puzzle is, how can we get a machine to do the same?

Common Sense Reasoning Example

Let's take a simple sentence:

Ashok ate a frog.

You read it, and right away, you get what it means. But let's dig a bit deeper. What did you really understand from those few words? Consider these questions: Is the frog dead or alive? What kind of thing is the frog? Is Ashok happy or sad?

Ashok David, did you figure it out?

David I think so. For starters, when we're talking about the frog, we know we mean the real, living creature, not a toy or a drawing of a frog. Is the first frog dead or alive? It's hard to imagine you eating a frog and it still being alive, so I'll go with dead. And if you ate it, it's probably in your stomach, right? Now, as for whether you're happy or sad, that's a bit tricky. I don't know your feelings about eating frogs, but since it was your suggestion, let's assume you're into it. So, I guess you're happy.

Ashok Thanks, David. What David just showcased is common sense reasoning. The sentence didn't specify if the frog was dead or alive, nor did it mention Ashok's emotions. Yet, David made smart guesses. We'll dive deeper into common sense

reasoning later. For now, let's focus on a concept called Frames, which helps us make these kinds of inferences.

This quick trip through your thoughts on a simple sentence showcases how effortlessly and quickly human brains put together pieces of a puzzle by using common sense. It's about spotting the connections between what's said and a huge pile of unspoken, yet understood, bits of knowledge.

THE FUNCTION OF FRAMES

So, the big question is how to get a machine to navigate through this complex terrain of understanding. That's where frames come facilitating the use of inferences. Returning to the sentence, "Ashok ate a frog." When uttered, its meaning seems immediate to you. Yet, let's ponder more deeply. What exactly did you grasp upon hearing it? Essentially, what is the "meaning of the meaning" of such a sentence? And critically, how can this understanding be encapsulated? More importantly, how is this comprehension captured in a machine?

Initially, let's direct our attention to the verb in our sentence, "ate." We will associate this verb, or action, with a frame. Although frames can link with both verbs and nouns in sentences, our current focus will be with the verb. Considering the conventional action of eating, several elements become apparent. Commonly, there's an agent engaging in the act, aligning with the sentence's subject. An object is consumed, represented as the sentence's object. The eating action may occur at a specific location or time, and tools like utensils may be employed. Particularly in eating, what's consumed is generally not alive, especially from a human perspective. An interesting aspect to examine is the "object-is" slot, reflecting on the consumed item's fate. Post-consumption, it logically resides within the subject's body. The subject's mood might also be influenced, with happiness often post-meal.

Here's a glimpse at the "slots" we associate with the action of eating, which isn't a comprehensive list but provides a foundation (refer to figure). These slots may be filled with various values, known as "fillers." Some fillers are default assumptions, while others stem from the context provided. In our case, "Ashok" fills the slot for the subject, and "a frog" fills the object slot.

Ashok ate a frog.

```
Ate
  subject : Ashok
  object  : a frog
  location :
  time    :
  utensils :
  object-alive : false
  object-is  : in-subject
  subject-mood : happy
```

Figure 6-1 Frames consisting of Slots and Fillers.

Thus, a “frame” emerges as a knowledge structure. The term “structure” underscores the multifaceted nature of this knowledge model, embodying various interconnected elements and insights.^{70,71}

To help deepen our understanding, let’s draw upon an analogy I’m confident you’ll find familiar. Picture the distinction between an atom of knowledge representation and a molecule of knowledge representation. Some representations are like atoms – singular, independent units. A production rule could be likened to an atom in this sense. Frames, however, resemble molecules; they possess a more complex structure and comprise multiple elements. Just as molecules can expand or contract, frames can be adapted and modified extensively. This versatility allows frames to encompass far more than what a simple production rule could. Essentially, a frame is a robust knowledge structure featuring slots and their corresponding fillers. Often, these fillers are preset to default values, addressing typical scenarios in a stereotypical fashion.

Now, let’s apply this concept to a different sentence: “David ate a pizza at home.” Visualize how a frame for this sentence might be outlined (refer to figure on the left).

Ashok ate a frog.

```
Ate
  subject : Ashok
  object  : a frog
  location :
  time    :
  utensils :
  object-alive : false
  object-is  : in-subject
  subject-mood : happy
```

David ate a pizza at home.

```
Ate
  subject : David
  object  : a pizza
  location : at home
  time    :
  utensils :
  object-alive : false
  object-is  : in-subject
  subject-mood : happy
```

Figure 6-2 Comparing the frame for “Ashok at a frog” with “David at a pizza at home”.

Here, both the subject and the object have changed, and notably, this sentence introduces a new element: location. In one case we know the location but in the other we do not. Let's take a moment to compare the frames constructed for both sentences. You'll observe that the slots remain consistent across both frames because they both relate to the act of eating. The fillers, however, vary to match the specifics provided in each sentence. The only fillers that are the same in both frames are those associated with default values for certain slots, underscoring the inherent flexibility and adaptability of frames in handling diverse information.

Example of Using Frames

Ashok Let's try out an exercise. We have the sentence, "*Angela ate lasagna with her dad last night at the Olive Garden.*" Can you develop a frame that captures the relevant slots and fillers for the verb "ate" based on this input? David, did you come up with a solution?

David Yeah, I started with the slots we mentioned before. So for our subject, it's Angela. The object here is lasagna. We've got a location specified this time; it's the Olive Garden. The time is night. There wasn't any mention of utensils or the state of the lasagna being alive or not, or whether Angela was happier after eating. I thought maybe we could include something about her dad too, but I haven't figured out which slot he'd fit into yet.

*Angela ate lasagna
with her dad last night
at Olive Garden.*

Ate

```
subject : Angela
object  : lasagna
location : Olive Garden
time    : night
utensils :
object-alive : false
object-is  : in-subject
subject-mood : happy
```

Figure 6-3 A frame for the action "ate" consisting of slots and fillers for the sentence "Angela ate lasagna with her dad last night at the Olive Garden".

Ashok That's a very thoughtful response, David. You're right, and you also noticed that our current slots don't cover the details about her dad. Before we tackle adding her dad into the mix, let's review some properties of frames.

As you've seen, frames have slots you can fill with specific data, and these fillers can come from a variety of possible values. Some slots will have default fillers. This setup allows for some common sense inferences. For instance, when David deduced the frog was dead in the sentence about Ashok, it was because the default state for an eaten object is not being alive. Similarly, the object's location after being consumed is generally inside the subject's body. We could introduce more slots with their default values to enrich our representation. We'll take a closer look at common sense reasoning later on. For now, our focus is grasping the concept and structure of frames in knowledge representation.

We can now explore the concept of frames, not just for verbs, but also for nouns, broadening the scope of our understanding. Frames can encapsulate entities such as individuals, objects, or locations, enabling a rich, interconnected representation of knowledge. Consider once again the sentence “Angela ate lasagna at Olive Garden with her dad.” Here, frames can be utilized for each noun – Angela, lasagna, and Olive Garden – providing a structured way to organize related information. These noun frames can then be linked to the action described by the verb frame “ate,” creating a network of related frames.

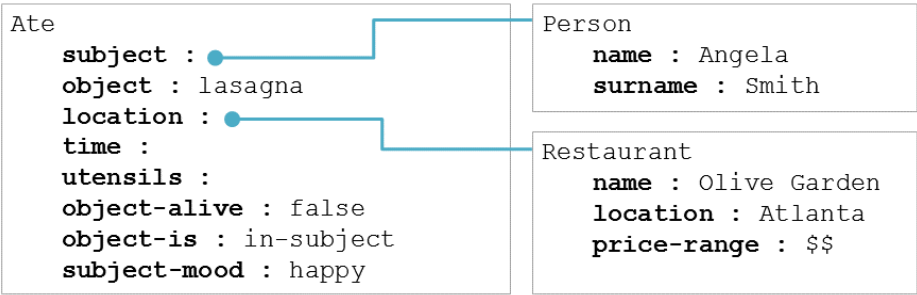


Figure 6-4 A complex collection of connected frames with slots holding points to other frames, here using filter pointers to frames for Angela and Olive Garden.

For example, a frame for “Angela” might include various attributes and experiences beyond the specific dining event, offering a comprehensive view of her character. Similarly, the “Olive Garden” frame could contain details about its location, ambiance, and menu offerings. The power of frames lies in their ability to connect: the filler of a slot in one frame can serve as a pointer to another frame, weaving a web of related information.

This connected structure highlights the potential of frames for making advanced inferences. If Angela's frame includes a slot for “food preferences” with “Italian food” as a filler, we can infer that her dining experience was particularly enjoyable, given her preference aligns with the meal choice.

This concept extends beyond individual sentences to discourse-level understanding, where multiple sentences form a coherent narrative or argument. Initial sentences might establish

frames for key subjects, such as Angela's preferences or the characteristics of Olive Garden. Subsequent sentences can then expand these frames or link them to new ones, building a complex tapestry of information. This interconnected network of frames facilitates understanding larger linguistic units and supports complex inferences, laying the groundwork for common sense reasoning and deeper comprehension in AI systems.⁷²⁻⁷⁴

CHARACTERISTICS OF FRAMES

Frames, as a knowledge representation method in artificial intelligence, embody three core characteristics that enhance their utility in modeling complex systems and concepts.

Stereotypes as the Foundation

The first essential feature of frames is their ability to encapsulate stereotypes, which are generalized representations or assumptions about categories or situations. For instance, when we think of the verb “eat” or “ate,” a frame for this action might include stereotypical slots such as the subject (who is eating), the object (what is being eaten), the location (where the eating is happening), and the time (when the eating occurs). These slots derive from our collective, stereotypical understanding of the eating process. Stereotypes, by their nature, are highly influenced by cultural contexts and personal experiences, leading to variations in how individuals conceptualize the same action or object. While one person's stereotype of eating might include a specific setting, like a family dinner at home, another's might involve a quick meal at a fast-food restaurant. Despite these variations, the fundamental structure of the frame remains consistent, offering a flexible yet organized way to represent knowledge

Properties of Frames

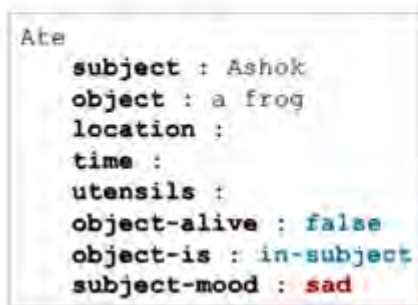
- Represent stereotypes

```
Ate
  subject : Ashok
  object  : a frog
  location :
  time    :
  utensils :
  object-alive :
  object-is :
  subject-mood :
```

Figure 6-5 Properties of Frames - Stereotypes.

Default Values and Exceptions

A second critical aspect of frames is their provision for default values within these slots, which represent common or expected states or attributes within the stereotype. For instance, an “eating” frame might default to assuming that the consumed object no longer exists in its original form and is now within the eater's digestive system, typically resulting in satisfaction or happiness for the eater. This feature allows for efficient knowledge representation by minimizing the need to specify every detail explicitly. However, the real world is replete with exceptions, and frames are adept at accommodating these deviations from the norm. The framework allows for the overriding of default values when the specific circumstances of an event or action deviate from the expected. For instance, if the sentence “Ashok ate a frog” is processed within the “eating” frame, and it is known that Ashok has an aversion to frogs or that consuming frogs makes him ill, this specific instance can override the default assumption of satisfaction, indicating instead that Ashok experienced discomfort or unhappiness as a result. This capability for exception handling within frames is both a strength and a complexity. It's advantageous because it enables the representation of knowledge to be both general and flexible, allowing for the depiction of typical scenarios while also accommodating unique or unusual instances. However, managing exceptions can become increasingly complex as more instances are added, each potentially introducing new exceptions and variations to the default values. This complexity necessitates sophisticated mechanisms within the frame structure to manage and resolve conflicts between default assumptions and specific exceptions effectively.



```
Ate
  subject : Ashok
  object  : a frog
  location :
  time    :
  utensils :
  object-alive : false
  object-is : in-subject
  subject-mood : sad
```

Properties of Frames

- Represent stereotypes
- Provide default values

Figure 6-6 Properties of Frames - Default Values and Exceptions.

Inheritance Hierarchy

The third fundamental property is the inheritance mechanism within frame systems, allowing for the creation of hierarchical relationships among frames. This structure enables more general frames (e.g., “Animal”) to define broad attributes and slots that more specific frames (“Ant,” “Human”) can inherit and refine. This hierarchical organization not only economizes knowledge representation by avoiding redundancy but also mirrors natural classification systems, making frames an intuitive model for representing real-world knowledge. At the base of this hierarchy are general frames, such as “Animal,” which define broad characteristics applicable to all items within this category. These frames contain slots that represent common attributes or actions associated with the category. For instance, an “Animal” frame might include slots for habitat, diet, and common behaviors. From this general level, the hierarchy branches out to more specific frames or subclasses, such as “Ant” and “Human.” These subclasses inherit the slots from their parent class, “Animal,” and can add more specific details or override the default values provided by the parent frame. For example, the “Ant” frame might specify the number of legs as six and adjust the habitat slot to reflect typical ant environments, while the “Human” frame could introduce additional slots for complex behaviors and social structures not applicable to ants. This inheritance mechanism allows for efficient knowledge representation, as it prevents the need to replicate shared information across every frame. Instead, specific frames can focus on what sets them apart from other items in their category, while still retaining the foundational knowledge provided by their parent classes.

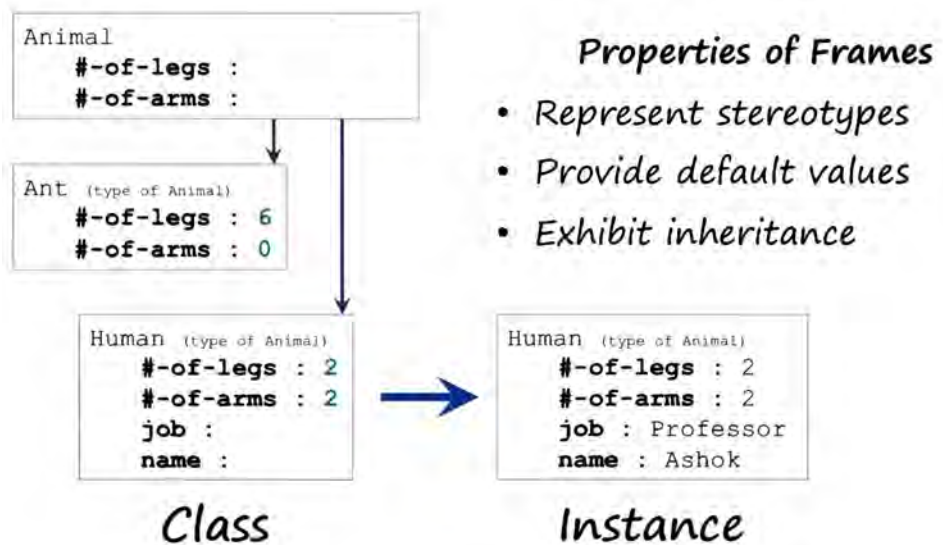


Figure 6-7 Properties of Frames – Inheritance.

Moreover, the parallels between frames in AI and classes in object-oriented programming (OOP) highlight a profound connection between knowledge representation in AI and software engineering principles. Both frames and classes encapsulate data and behaviors, provide mechanisms for default values and inheritance, and facilitate the organization of complex information into manageable, hierarchical structures. This similarity is not coincidental but a reflection of the concurrent evolution of these concepts during the 1960s and 1970s, evidencing a cross-pollination of ideas between the fields of AI and computer science.

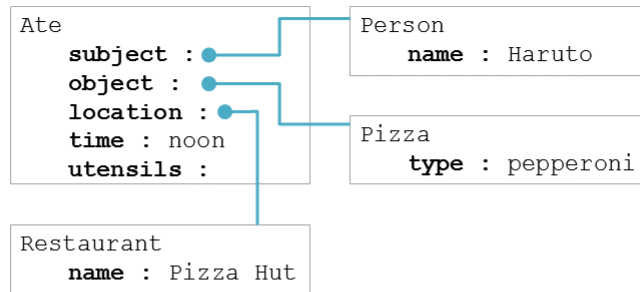
FRAMES AND OTHER KBAI TOPICS

Sentence Generation and Comprehension

Sentence Generation and Sentence Comprehension represent two fundamental concepts in linguistics and artificial intelligence. Sentence Generation involves constructing sentences from structured data or conceptual frameworks, such as frames, making it a process of translating structured representations into natural language. On the other hand, Sentence Comprehension entails the reverse process, where sentences are parsed and understood to build structured representations, like frames or frame networks, effectively decoding natural language into structured, comprehensible units.

Ashok and David engage in an interactive exercise to explore the relationship between frames and sentence construction:

Ashok Let's work through an example together. Suppose we have a collection of frames that represent a certain piece of conceptual knowledge. Based on these frames, what sentence could be formed? David, what's your take on this?



What sentence is expressed by these frames?

Figure 6-8 Using Frames for Sentence Generation.

David Well, there could be multiple sentences that fit these frames. In my interpretation, I started with Haruto, as he's the one performing the action of eating. The object of this action is pepperoni pizza, and the location is Pizza Hut. The event occurred at noon. There was no mention of utensils, so I left that out. My constructed sentence is: 'Haruto ate a pepperoni pizza at Pizza Hut at noon.'

Ashok Nicely done, David.

However, it's worth considering an interesting aspect of David's answer. The original sentence might have been 'Haruto ate a pepperoni pizza at Pizza Hut at noon,' leading to the creation of this frame setup. In this scenario, 'Haruto' is mapped to the subject and 'ate' to the verb, and so forth. Conversely, this frame setup could have been the starting point, with the sentence being generated from it. This suggests that frames could serve as an intermediary representation, facilitating both the comprehension and the generation of sentences. Clearly, sentence generation and comprehension involve complexities beyond what we've covered in this section.

Frames and Semantic Networks

Frames can be effectively employed to tackle Raven's Progressive Matrices, a topic we explored in earlier chapters. This approach underscores the close relationship between frames and semantic networks. To illustrate, let's examine the problem depicted here, where we transition

from a semantic network representation of an image to a framed structure. In this transformation, individual frames are created for each object within the image – namely, objects x, y, and z.

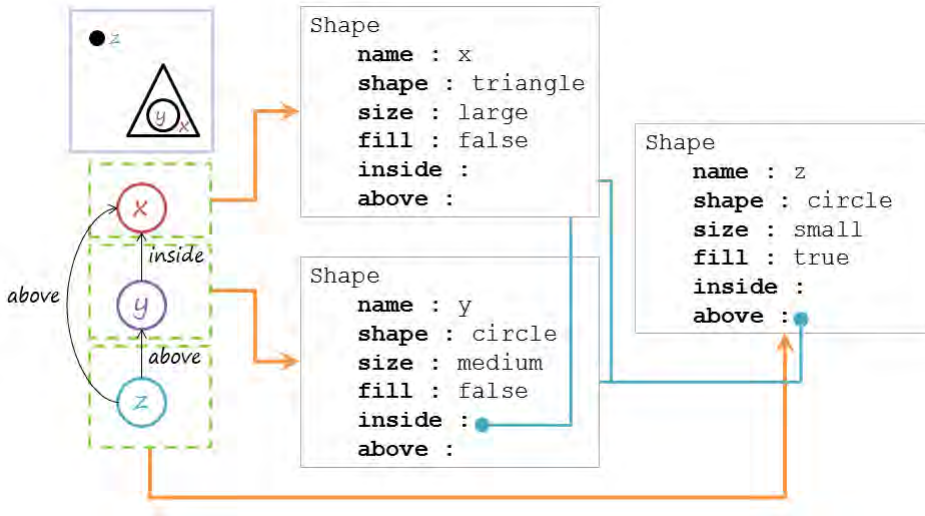


Figure 6-9 Representing Semantic Networks as Frames.

Focusing on the frame for object z, we observe the following slots: the name is z, its shape is a circle, its size is small, and it possesses a filled attribute, as visually represented. Moreover, frames allow us to articulate the relationships between objects. For instance, considering the relationship where object y is nested within x, this is mirrored in the frame for y with a slot indicating 'inside' and pointing towards x, thus establishing y's position relative to x.

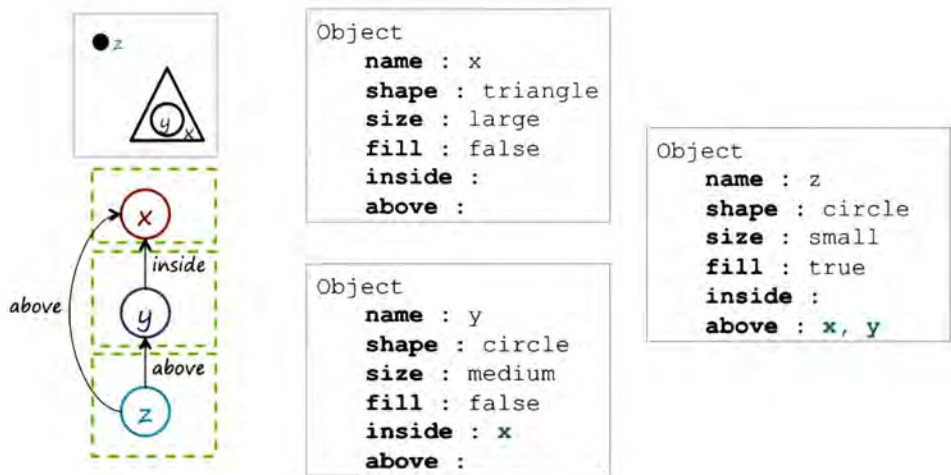


Figure 6-10 Representing Semantic Networks as Frames using Fillers to point to other Frames.

This parallelism⁷⁵ between the semantic network and frame representations is noteworthy, where each object's frame mirrors its semantic network node, and the inter-object relationships are depicted as connections between frames. While such relationships can be visually represented with connecting lines, indicating one frame's reference to another, a more direct method involves explicitly naming other frames within a frame's slots. For example, within the frame for **y**, we could specify 'inside x,' a method we'll adopt for subsequent exercises in this chapter, offering a clear and direct way to represent relationships between frames.

Let's try an exercise to ensure our grasp on framing representations for images such as the one depicted in the top left of the figure provided.

Ashok David, have you managed to jot down the slots and their corresponding fillers for these three frames?

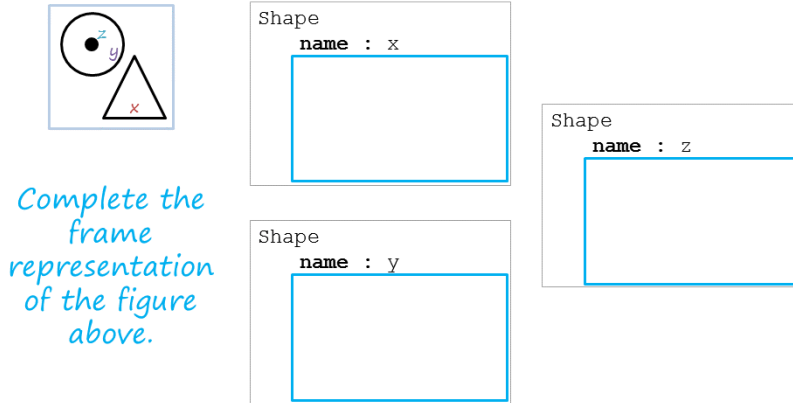


Figure 6-11 Example of translating an image into a Frame representation.

David

Yes, I have. For object x, I noted it as a large, unfilled triangle. Object y is represented as a large, unfilled circle. And for object z, I described it as a small, filled circle. Regarding their relationships, in this visualization, z is positioned inside y, which is mirrored in the frame. Additionally, z is depicted above x, a relationship that's consistent in the frame. Similarly, y is shown above x both in the image and its frame representation.

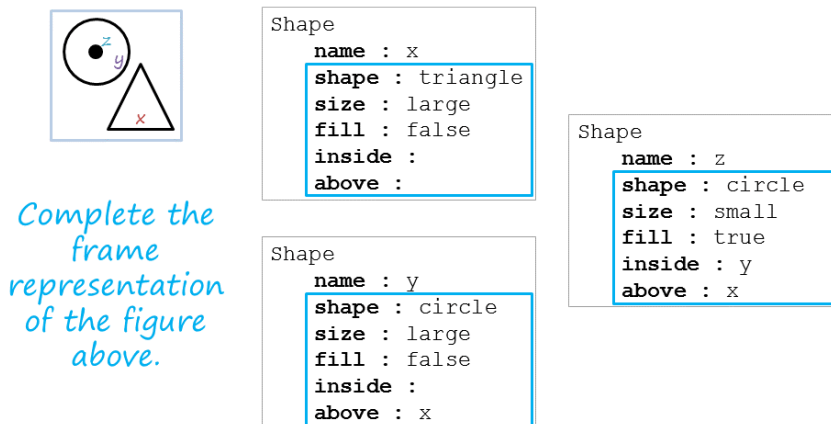


Figure 6-12 Completed example of an image represented by Frames.

Ashok Excellent, David. That aligns with my understanding as well.

Frames and Production Systems

In our earlier discussions on production systems, we encountered the concept of frames. Consider a diagram featuring procedural, semantic, and episodic knowledge alongside a structured working memory, reminiscent of a frame with slots and corresponding values.⁷⁶ These frames can be seen as encapsulating conceptual knowledge stored within the semantic memory.

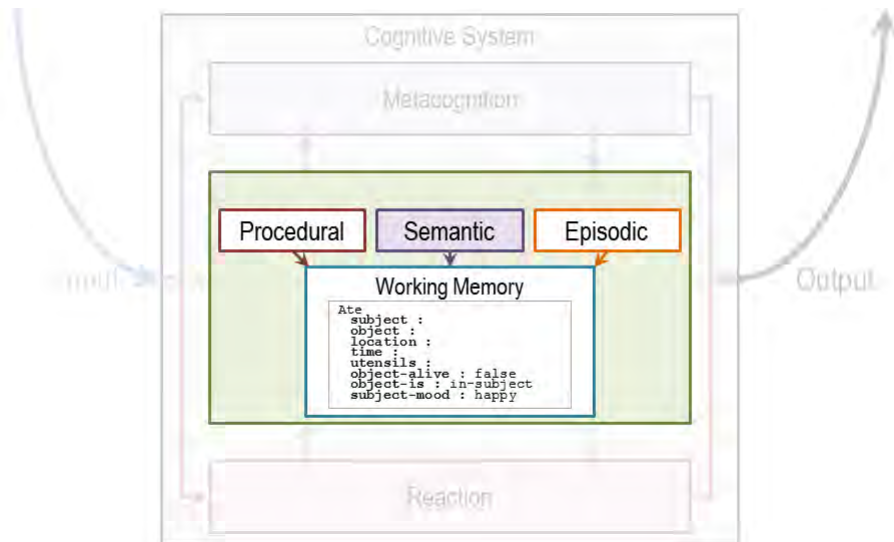


Figure 6-13 Frames and Production Systems.

Take, for instance, the statement “Ashok ate a frog.” The action verb “ate” is key here, as it is retrieved into the working memory, bringing along the comprehensive frame associated with “ate” from the semantic memory. This activation of the frame not only retrieves stored information but also sets up a series of expectations about what typically accompanies the action of eating – like a subject, an object, a location, possibly a time, and maybe even utensils.

This framework prompts us to fill in the blanks: what or who is the subject? What is the object being consumed? In the context of “Ashok ate a frog,” the frame guides us on what to anticipate next. This interaction illustrates that the processing of information isn't solely bottom-up, originating from language or external stimuli and moving inward. Rather, our cognition also operates top-down, utilizing structured knowledge representations like frames to generate expectations and guide our understanding of incoming information.

Let's join Ashok and David as they explore the strengths and challenges of using frames for knowledge representation by working through an exercise.

Ashok

Consider the narrative depicted in the figure, which describes an earthquake event. Based on this narrative, how would you populate the frame specific to an earthquake with relevant slots and fillers? David, could you share how you approached this task?

Today, an extremely serious earthquake of magnitude 8.5 hit Lower Slabovia, killing 25 people and causing \$500 million in damage. The President of Lower Slabovia said that the hard-hit area near the Sadie Hawkins fault has been a danger zone for years.

Write a frame representation of this story on the right.

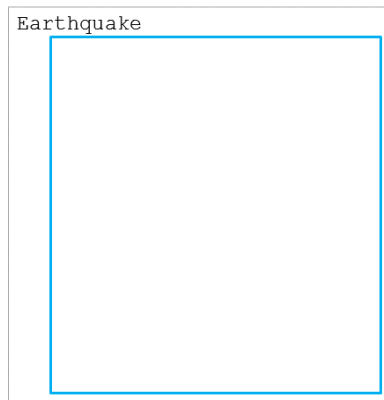


Figure 6-14 Example translating a Frames to represent complex understanding.

David

Sure, Ashok. To tackle this, I first outlined what I typically expect from an earthquake scenario: the date it occurred, its location, the resultant damage and casualties, the specific fault line involved, its magnitude, the exact time it struck, its variety, and duration. With these slots in mind, I then sifted through the story for corresponding fillers. The earthquake happened today, struck in Lower Slobbovia, among other details. Not all slots were filled by the story's information, but I captured what was available.

Today, an extremely serious earthquake of magnitude 8.5 hit Lower Slabovia, killing 25 people and causing \$500 million in damage. The President of Lower Slabovia said that the hard-hit area near the Sadie Hawkins fault has been a danger zone for years.

Write a frame representation of this story on the right.

Earthquake	
day :	Today
location :	lower Slabovia
damage :	\$500 million
fatalities :	25
faultline :	Sadie Hawkins
magnitude :	8.5
time :	
type :	
duration :	

Figure 6-15 Completed example of using frames to represent complex understanding.

Ashok Nicely done, David. Your approach showcases how frames can systematically organize event details, even when some information remains unspecified.

In David's response, we observe two critical aspects: the slots are drawn from his inherent understanding of typical earthquake events, and the fillers are extracted from the narrative provided. This exemplifies the seamless integration of bottom-up and top-down processing, where the mention of an earthquake prompts a retrieval of the relevant frame from semantic memory, guiding the search for slot values within the narrative.

However, this method's limitations become apparent when we consider an alternative narrative. Analyzing a different story, as shown in the subsequent figure, reveals the complexity and subtlety in interpreting such events.

Today, an extremely serious earthquake of magnitude 8.5 hit Lower Slabovia, killing 25 people and causing \$500 million in damage. The President of Lower Slabovia said that the hard-hit area near the Sadie Hawkins fault has been a danger zone for years.

Today, the **President of Lower Slabovia killed 25** proposals totaling **\$500 million** for research in **earthquake** prediction. Our Lower Slabovian correspondent calculates that **8.5** research proposals are rejected for every one approved. There are rumors that the President's science advisor, **Sadie Hawkins**, is at fault.

Figure 6-16 Comparison to two sentences that produce the same fillers.

The term “killed” in one context refers to the rejection of 25 proposals, whereas in another, it pertains to the loss of 25 lives. Similarly, “Sadie Hawkins” could be identified as a scientific advisor or a fault line, depending on the context. The frame representation, in isolation, doesn't provide the means to discern between these nuanced interpretations. This challenge underscores the need for a deeper exploration into understanding and common-sense reasoning, which we will look into later in this book, to navigate such complexities.

FRAMES AND COGNITION

Frames are a fundamental aspect of human cognition, serving as structured knowledge representations that encapsulate complex information akin to molecules, in contrast to the atomic nature of production systems.⁷⁷ This organized packet of knowledge allows for a more nuanced approach to cognitive processing that transcends the simplicity of production rules.

Importantly, frames facilitate a hybrid model of cognitive processing that intertwines bottom-up and top-down mechanisms. While sensory data from the external world initiates bottom-up processing, frames retrieved from memory actively shape and interpret this data, introducing a top-down component. This dynamic interplay allows for the generation of expectations and the contextual understanding of new information.

Moreover, frames embody stereotypes – preconceived notions about events and situations that, while potentially leading to biases, serve a critical function in cognitive efficiency. These stereotypes, ingrained within the frames, come equipped with default values, enabling rapid generation of expectations without the need for exhaustive analysis each time a familiar situation is encountered. This efficiency underscores the cognitive utility of stereotypes, despite their drawbacks.

These three aspects – structured knowledge representation, hybrid top-down and bottoms-up model, and stereotypes – highlight the intricate relationship between frames and human cognition, showcasing how frames not only structure our understanding of the world but also influence the efficiency and direction of our cognitive processes. As we look deeper into the subject, the multifaceted connections between frames and cognition will become increasingly evident, revealing their pivotal role in shaping human thought and perception.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

Frames, as a structured knowledge representation, offer valuable connections and potential integrations with several modern AI paradigms:

- **Knowledge Graphs & Semantic Web Technologies:** Frames are conceptually very similar to the nodes and properties found in knowledge graphs. Modern knowledge graphs, often built using Semantic Web technologies like RDF and OWL, can be seen

as large-scale, interconnected networks of frames. Frames provide the schema or template for the types of entities and relationships stored in these graphs, which are used by search engines, recommendation systems, and increasingly by LLMs for factual grounding.

- **Natural Language Processing (NLP) & Understanding (NLU):** In NLU, frames (often called “semantic frames” or “frame semantics” inspired by Fillmore's work) are used to represent the meaning of sentences by identifying semantic roles related to a central verb or concept. For example, an “ordering_food” frame might have slots for buyer, food_item, quantity, restaurant. This helps systems understand the intent and key information in user utterances, crucial for dialogue systems and information extraction.
- **Generative AI (GenAI) & Large Language Models (LLMs):** While LLMs learn implicit structures from data, frames can provide explicit structure to guide or constrain LLM outputs. For instance, an LLM could be tasked to fill the slots of a predefined frame to generate a structured summary or to ensure its creative writing adheres to certain narrative structures (which can be represented as sequences of frames or scripts). Conversely, LLMs can be used to populate frame slots from unstructured text.
- **Explainable AI (XAI):** The slot-filler structure of frames can contribute to explainability. If an AI system makes a decision based on a frame, it's easier to trace which pieces of information (fillers) in which conceptual roles (slots) led to that decision, compared to opaque models like deep neural networks.

The structured nature of frames, their ability to represent stereotypes with defaults and exceptions, and their hierarchical organization offer complementary strengths to the data-driven, pattern-recognition capabilities of many modern AI techniques. Hybrid systems that combine frame-based reasoning with machine learning are an active area of interest.

CHAPTER SUMMARY

In this chapter, we explored frames as a pivotal knowledge representation mechanism in KBAI. We established that frames are structured like “molecules” of knowledge, organized around slots (variables) and fillers (values), which allows them to represent complex concepts and situations. This structure facilitates common sense reasoning by providing a way to organize stereotypical knowledge.

We identified three core characteristics of frames:

1. They encapsulate stereotypes, providing a generalized understanding of concepts or events.
2. They utilize default values for slots, enabling efficient reasoning by assuming typical information unless specified otherwise, while also allowing for exceptions to handle variability.

3. They support inheritance hierarchies, where general frames pass down attributes to more specific ones, promoting knowledge reuse and organization, similar to classes in object-oriented programming.

The chapter illustrated the function of frames through examples, including sentence comprehension (e.g., “Ashok ate a frog,” “Angela ate lasagna”) where frames help infer unstated information. We saw how frames can be created for both verbs (actions) and nouns (entities), and how these can be interconnected to form richer knowledge networks, enabling more advanced inferences.

Connections were drawn between frames and other KBAI topics. We noted the representational similarity between frames and semantic networks, where nodes and relationships in a semantic network can be mapped to frames and their slot-fillers. Furthermore, we discussed how frames can serve as the structured conceptual knowledge within the semantic memory of production systems like Soar, guiding the interpretation of working memory contents and the generation of expectations in a top-down manner. The earthquake narrative example highlighted both the power of frames in organizing information and their limitations in disambiguating complex linguistic nuances without further reasoning capabilities.

The chapter also touched upon the cognitive relevance of frames, emphasizing how they model human-like understanding through stereotypes, default reasoning, and the integration of bottom-up (data-driven) and top-down (expectation-driven) processing.

Finally, we considered the relationship between frames and modern AI paradigms. Frames align closely with knowledge graphs and Semantic Web technologies, play a key role in Natural Language Understanding (semantic frames), can structure or guide Generative AI and LLM outputs, and contribute to Explainable AI due to their transparent structure.

As we move forward, the understanding of frames as a versatile and powerful knowledge representation tool will be crucial for exploring more advanced topics such as learning from specific experiences and building systems with robust common-sense reasoning.

ADDITIONAL READINGS & REFERENCES

- 68 Davis, R., Shrobe, H. & Szolovits, P. What is a knowledge representation? *AI magazine* **14**, 17-17 (1993).
- 69 Mylopoulos, J. An overview of knowledge representation. *ACM SIGART Bulletin*, 5-12 (1980).
- 70 Petersen, W. Representation of concepts as frames. *Meaning, frames, and conceptual representation* **2**, 43-67 (2015).
- 71 Fikes, R. & Kehler, T. The role of frame-based representation in reasoning. *Communications of the ACM* **28**, 904-920 (1985).
- 72 Chaâri, L., Pesquet, J.-C., Tournet, J.-Y., Ciuciu, P. & Benazza-Benyahia, A. A hierarchical Bayesian model for frame representation. *IEEE Transactions on Signal Processing* **58**, 5560-5571 (2010).
- 73 Faber, P. & Cabezas-García, M. Specialized knowledge representation: From terms to frames. *Research in Language* **17**, 197-211 (2019).

- 74 Minsky, M. Artificial Intelligence Memo No. 306. *Artificial Intelligence* (1974).
- 75 Nazaruks, V. & Osis, J. in *Special Session on Model-Driven Innovations for Software Engineering*. 346-354 (SciTePress).
- 76 Smith, D. E. & Clayton, J. E. in *AAAI*. 154-156.
- 77 Löbner, S. Frames at the interface of language and cognition. *Annual Review of Linguistics* 7, 261-284 (2021).

Chapter 7

Learning by Recording Cases

INTRODUCTION TO LEARNING BY RECORDING CASES

In this chapter, we examine the concept of Learning by Recording Cases, marking our initial foray into *learning* within the context of Knowledge-Based Artificial Intelligence. Learning by recording cases stands as a pivotal aspect of analogical reasoning, widely regarded as a fundamental cognitive process.⁷⁸ Our exploration begins with an overview of case recording in its broadest sense, laying the groundwork for understanding how experiences and instances can inform and enhance artificial intelligence systems.⁷⁹

We then narrow our focus to a specific technique known as the nearest neighbor method, an approach that underscores the significance of proximity and similarity in learning from recorded cases.^{80,81} This method serves as a springboard into the more sophisticated k-nearest neighbor method, which expands on the concept by considering multiple proximate cases to make more informed decisions or predictions.

As we progress, we will explore the application of methods like Nearest Neighbor to various scenarios. The concepts introduced in this chapter also lay foundational groundwork for understanding more advanced topics to be explored later, such as Incremental Concept Learning, Classification (as a broader topic), and Version Spaces. These advanced topics will offer unique perspectives on how AI systems can evolve and adapt through the accumulation and analysis of cases. Through this overall exploration, we aim to illuminate the intricate

mechanisms by which knowledge-based AI systems can learn, reason, and ultimately mirror the nuanced processes of human cognition.

A Simple Illustration: The Colored Blocks Scenario

In our exploration of learning by recording cases, let's consider a simplified scenario involving a collection of colored blocks. These blocks vary in shape and size, providing a diverse set of examples within this hypothetical block world. Imagine a set of six blocks, each distinct in its characteristics, laid out before us.

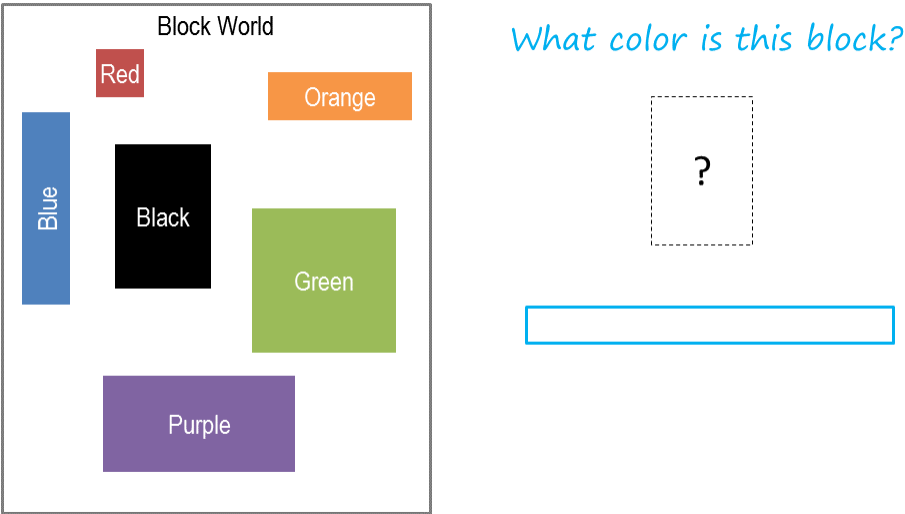


Figure 7-1 Colored Blocks Example - What is the color of the block on the right?

Now, let's engage in a thought experiment. Drawing upon your observations and experiences with these blocks, I pose a question to you: If you were to encounter another block within this world, what color do you anticipate it would be? This question is designed to prompt you to apply the concept of learning from recorded cases, where your past interactions with the blocks inform your prediction about the characteristics of a new, unseen block.

Let’s rejoin Ashok and David as they explore this further.

Ashok What's your take on this, David?

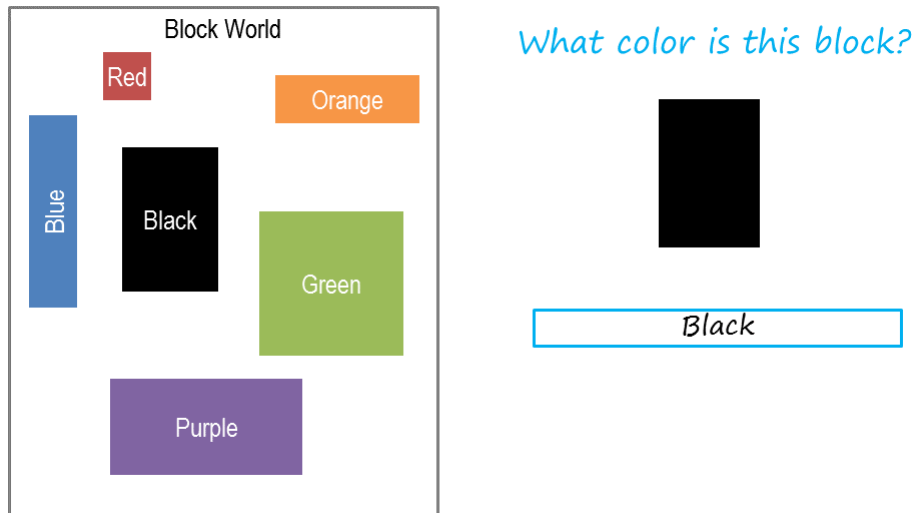


Figure 7-2 Colored Blocks Example with a Recorded Cases based answer.

David Given the shape, size, and orientation of this particular block, it shares more similarities with the black block on the left side of the figure. My guess would be that this block is also black.

Ashok Exactly, David. Among the blocks we have, this one indeed bears the **closest** resemblance to the black block. This suggests that the most plausible inference is that the block in question is black. This scenario serves as a prime example of learning by recording cases. The agent has stored information about six different cases in its memory. When confronted with a new problem, the agent draws upon its repository of past cases, identifying the one that most closely aligns with the current situation, and bases its response on this analogous case.

APPLYING LEARNING BY RECORDING CASES

In learning by recording cases, the process unfolds somewhat like this: when faced with a new challenge, depicted here by a unique color, the agent's memory, which houses a vast array of previous instances distinguished by various colors, springs into action. It identifies the case, let's call it 'b', that most closely mirrors the new challenge 'a'. The selection in this instance is based on color similarity. The solution that was effective for case 'b' is then adapted and applied to the new problem 'a'. Although this visualization serves as a helpful illustration, it remains somewhat abstract.

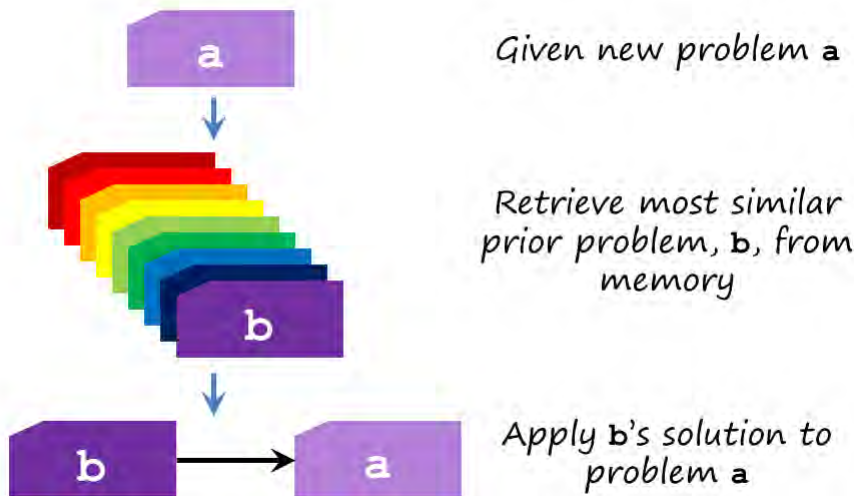


Figure 7-3 Retrieval using Learning By Recording Cases.

To ground this concept in more relatable terms, let's consider a few everyday scenarios. Picture a typical morning when you're preparing for a jog. You slip on your sneakers and face the task of tying the shoelaces. This routine act is, in essence, a 'new' problem each day. Yet, given the countless times you've tied laces on various shoes before, your memory is rich with related experiences – each instance serving as a 'case'. When you begin tying your laces for the day, your mind instinctively retrieves the most relevant past case and applies its solution. The process is so ingrained and automatic that we seldom pause to meticulously consider each step of tying our shoelaces; doing so would be incredibly time-consuming.⁸²

Now, consider the journey of a programmer embarking on the creation of a new Java application. Faced with the task of laying the groundwork for this program, the programmer naturally turns to their repository of past experiences. They recall a previous instance when initiating a Java project, where the foundational step involved crafting the familiar 'public static void main' structure. This recollection serves as a guiding light, enabling the programmer to seamlessly apply this tried-and-true approach to the current project without the need for adjustments. As the development progresses, the programmer may encounter a familiar foe: a null pointer error. This obstacle prompts a dive into the depths of memory, searching for a similar challenge faced in a past project. The recollection of a previous null pointer error surfaces, bringing with it the solution that proved effective: running the program in debug mode to pinpoint the exact variable that's null at the time of execution. This strategy, retrieved from the history of the programmer's experiences, offers a direct path to resolving the current issue.

Extending the concept of learning by recording cases to medical diagnosis offers a compelling illustration of its breadth and efficacy. Envision a scenario where you, as a patient,

present a constellation of signs and symptoms to a physician. The physician is then confronted with a novel challenge: discerning the most accurate diagnosis for your specific condition. Within the physician's mental archive lies a wealth of previously encountered cases, each a snapshot of her clinical experiences. Faced with your unique situation, she sifts through this repository, seeking a case that mirrors yours with the closest fidelity. Upon identifying such a case, perhaps denoted as 'b' in her mental catalog, she draws a parallel between your symptoms and those of case 'b'. With this analogy in hand, she extends the diagnosis rendered in case 'b' to your current condition.⁸³ formula for Euclidean distance This process underscores the essence of a case as a distilled essence of prior experience.⁸⁴ The strategy of learning by recording cases demonstrates its versatility and potency across a spectrum of scenarios, from the simplicity of fastening shoelaces to the complexity of diagnosing medical conditions.

RETRIEVAL USING THE NEAREST NEIGHBOR METHOD

Exploring the nuances of learning by recording cases brings us to the pivotal concept of identifying the 'most similar' or 'most closely resembling' case. To render this concept more tangible and operational, we look into a practical illustration involving a world of colored blocks. Imagine these blocks not just as physical entities but as data points within a knowledge representation framework. We can map our understanding of these blocks onto a two-dimensional grid, where one axis represents the width and the other the height of each block. In this representation, the blue block might be positioned at one coordinate, the red block at another, and so forth.

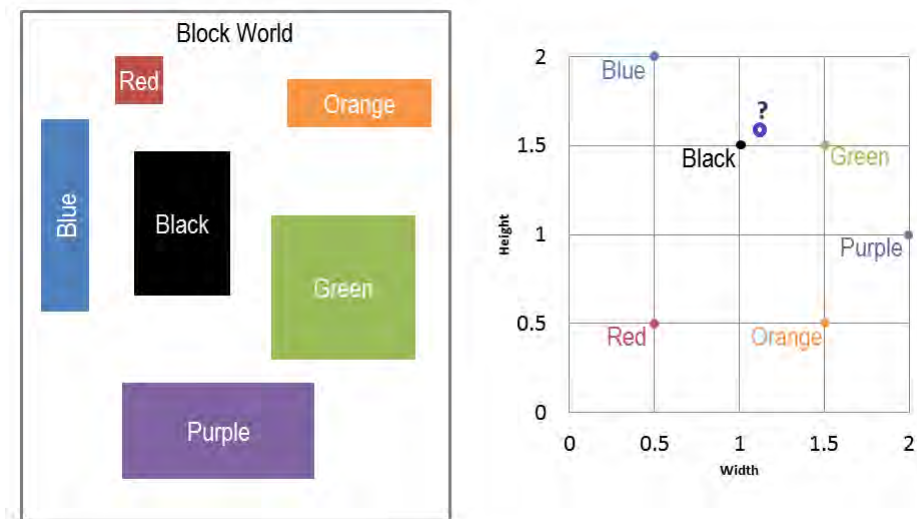


Figure 7-4 Retrieval using Nearest Neighbor Method.

When confronted with a new problem, akin to a new block, we plot it within the same grid based on its dimensions. Suppose the new block corresponds to a specific point within this grid. The crux of the nearest neighbor method lies in quantifying the proximity of this new problem to our existing cases, essentially measuring the distance from the new problem to each known case.^{80,81} To systematically compute these distances, we employ a measure known as the Euclidean distance. The formula for Euclidean distance calculates the spatial separation between two points: one representing a known case x_i , represented as the point (x_c, y_c) , and the other representing the new problem x_j , represented as the point (x_n, y_n) .

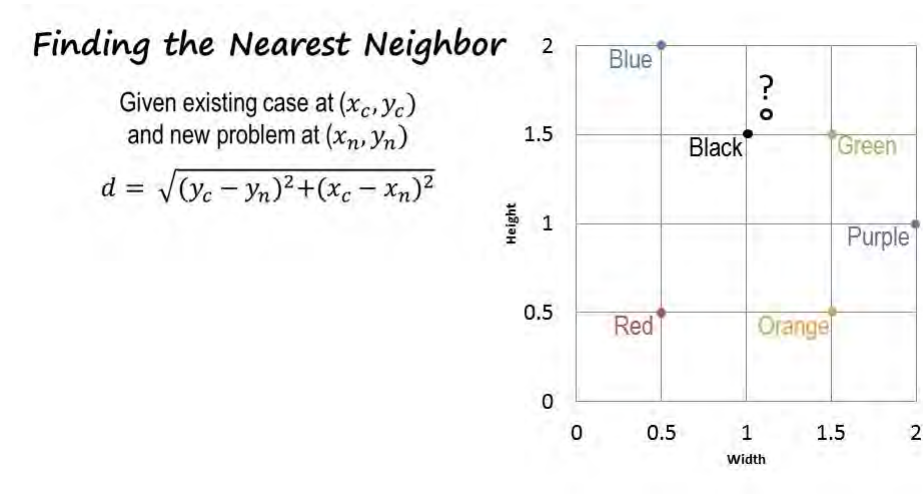


Figure 7-5 Using Euclidean Distance with Nearest Neighbors.

Armed with the Euclidean distance formula, we can ascertain the distances between the new problem and each existing case. A summary table of these distances enables us to rapidly deduce that the black block is the nearest neighbor to the new problem. This proximity suggests that the new block is likely black as well, illustrating the efficacy of the nearest neighbor method in identifying the case that most closely mirrors the new problem.

Nearest Neighbor Euclidean Distance Blocks Example

Let's jump into an exercise that applies the nearest neighbor method in a practical scenario. We are presented with a block characterized by a width and height of 0.8 each. The task at hand is to deduce the color of this block based on its dimensions.

What color is this block?

?

Width = 0.8

Height = 0.8

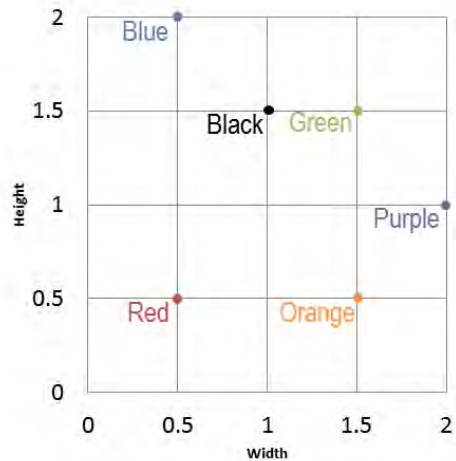


Figure 7-6 Using Euclidean Distance with Nearest Neighbor.

Ashok David, could you share your reasoning and conclusion regarding the color of the block?

David Certainly, Ashok. I positioned this block within the same two-dimensional framework where we plotted all other blocks, landing it at the coordinates (0.8, 0.8), corresponding to the unknown block in our discussion. By calculating the Euclidean distances between this block and the others, I noted that it was marginally closer to the red block compared to the rest, including the orange and black blocks. Therefore, my conclusion was that the block is red.

What color is this block?



Block	x_c	y_c	x_n	y_n	d
Blue	0.5	2.0	0.8	0.8	1.24
Red	0.5	0.5	0.8	0.8	0.42
Black	1.0	1.5	0.8	0.8	0.72
Green	1.5	1.5	0.8	0.8	0.98
Orange	1.5	0.5	0.8	0.8	0.76
Purple	2.0	1.0	0.8	0.8	1.22

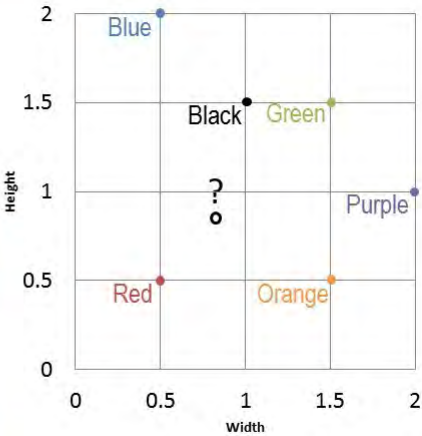


Figure 7-7 Retrieval using Euclidean Distance with Nearest Neighbor

Ashok That's a thoughtful analysis, David.

In this instance, our decision-making was simplified by the two-dimensional grid, which adequately represented each block with just x and y coordinates. However, real-world problems often demand more complex representations, sometimes necessitating a multi-dimensional space to encapsulate all aspects of the cases and the new problem at hand. Let's progress to a more intricate example that illustrates this complexity.

In this scenario, we are examining a map of a section of Long Island, New York, where an automated vehicle is tasked with navigating its streets. This vehicle is initially programmed with six distinct cases – labeled A, B, C, and so forth – that serve as its foundational knowledge for navigation. The vehicle's operational principle in this neighborhood is solely based on the method of learning by recording cases, relying entirely on these pre-programmed scenarios for guidance.



What route is most similar to this new problem?

Figure 7-8 Navigation route selection using Nearest Neighbor

Now, let's introduce a new challenge for this autonomous car. The task at hand is to devise a route from point Q to a specified destination, indicated by an arrow on the map. The critical question we need to address is: which of the pre-existing routes, among the six cases known to the vehicle, bears the closest resemblance to this new routing task? This will determine the most viable path for the vehicle to follow, based on its existing case-based knowledge.

Ashok Given our scenario with the autonomous car in Long Island, New York, we have to decide which of the pre-loaded routes it should follow from point Q to the designated destination. David, based on the cases A to F we have, which one would you say the car should consider most similar to its current task?

David After considering the options, I believe route D is the most suitable choice. While routes A and B are closer to the starting point, their trajectories diverge significantly from our intended direction. Route E, on the other hand, concludes nearer to our target but starts much farther away than desired. Route D strikes a balance by being relatively close to both our starting point and destination, making it the most analogous case to follow.

Ashok You've made a sound choice with route D. Now, the challenge lies in crafting an AI agent that can autonomously make such decisions based on similarity to pre-existing cases. Let's explore how we might achieve that.

As we seek to deduce the most suitable route for our autonomous vehicle navigating a segment of Long Island, New York, we employ a methodical approach to assess similarity based

on two critical factors: the point of origin and the destination. We represent this analysis through two-dimensional grids, where one grid focuses on the origin points of pre-recorded routes compared to the new challenge, and the other emphasizes their respective destinations.

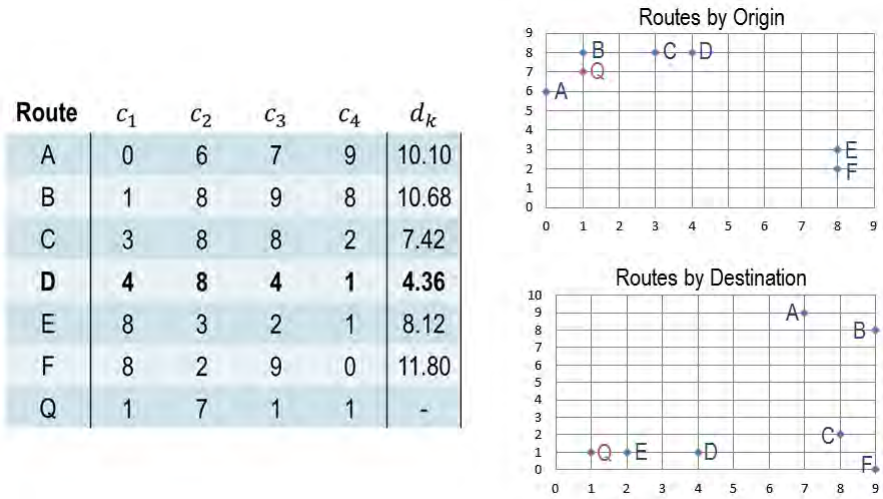


Figure 7-9 Considering multiple features when using Nearest Neighbor retrieval.

By calculating the Euclidean distances between the new problem's origin, denoted as Q, and those of the existing cases, we attempt to gauge which pre-existing route aligns most closely with Q's starting point. Similarly, we assess the proximity of Q's intended destination to those of the known routes. When viewed in isolation, route B emerges as the most proximate to Q's origin, while route E appears closest to Q's intended endpoint. However, this dual perspective reveals a conundrum: route B, while near to the starting point, diverges significantly from the desired endpoint, and conversely, route E, despite its nearness to the destination, starts from a point considerably distant from Q. This poses a complex challenge for an AI agent: synthesizing these disparate factors to identify the most viable path. How, then, can the AI determine that route D, which balances proximity to both the origin and destination, is the optimal choice amidst these alternatives?

K-NEAREST NEIGHBORS (KNN)

Transitioning from our previous discussion on calculating Euclidean distance in a two-dimensional space, we expand our understanding to accommodate multidimensional contexts. This expanded formula allows for the computation of the nearest neighbor in a space defined by *n* dimensions, thereby enabling a comprehensive evaluation of both cases and problems

Draft -Version:6/2/2025 3:06 PM

within this more intricate framework. The formula's generalization facilitates the measurement of Euclidean distances in this n -dimensional area, considering both the origin and destination, each represented by their x and y coordinates.

Finding the Nearest Neighbor

Given existing case at (x_c, y_c)
and new problem at (x_n, y_n)

$$d = \sqrt{(y_c - y_n)^2 + (x_c - x_n)^2}$$

	Origin			Destination		
Route	x_o	y_o	d_o	x_d	y_d	d_d
A	0	6	1.41	7	9	10.00
B	1	8	1.00	9	8	10.63

Given existing case at $(c_1, c_2 \dots c_k)$
and new problem at $(p_1, p_2 \dots p_k)$

$$d = \sqrt{\sum_{i=1}^k (c_i - p_i)^2}$$

C	3	8	2.24	8	2	7.07
D	4	8	3.16	4	1	3.00
E	8	3	8.06	2	1	1.00
F	8	2	8.60	9	0	8.06
Q	1	7	-	1	1	-

Figure 7-10 K Nearest Neighbor based retrieval.

An illustrative table elucidates the Euclidean distances between various cases and the new problem, considering the multifaceted nature of both origin and destination. This comprehensive analysis reveals that, contrary to initial assessments focused solely on singular dimensions, case D emerges as the most analogous to the new problem Q, not cases B or E as previously inferred from isolated considerations.

This refined approach is often extended to the K-Nearest Neighbors (KNN) method, an elegantly simple yet profoundly effective methodology.⁸⁰ Instead of just finding the single nearest neighbor, KNN considers K (a small integer) nearest neighbors and makes a decision based on them (e.g., by majority vote for classification or averaging for regression). However, its applicability in real-world scenarios is not without challenges. One significant hurdle is the potential vastness of dimensions required to accurately represent problems and cases in complex environments, which complicates the process of identifying the most relevant cases (often called the “curse of dimensionality”). Furthermore, the mere proximity of a new problem to an existing case (or cases) does not guarantee the direct applicability of the latter's solution to the former, highlighting a critical need for nuanced retrieval and adaptation strategies.⁸⁵

To address these complexities, we discuss Case-Based Reasoning in the subsequent chapter, exploring sophisticated techniques for retrieving and adapting historical cases to novel problems, thereby extending the utility and applicability of knowledge-based AI in diverse and dynamic settings.

NEAREST NEIGHBORS AND COGNITION

The Nearest Neighbors method, while a fundamental concept within artificial intelligence, also resonates deeply with human cognition. This connection underscores the intrinsic patterns and regularities that define our interaction with the world. Much like cognitive agents, we are repeatedly confronted with familiar problems that our memory has already encountered and resolved. This cyclical nature of challenges, from the mundane task of tying shoelaces to more complex problem-solving, emphasizes the pivotal role of memory in our cognitive processes.

Our cognitive architecture, as previously delineated, comprises three integral components: reasoning, memory, and learning.⁶⁷ Traditional views of intelligence have predominantly celebrated the reasoning aspect, associating it with problem-solving capabilities, decision-making, and logical analysis. However, the Nearest Neighbors approach and the broader concept of learning by recording cases advocate for a more balanced appreciation of cognitive functions. They highlight the indispensable contributions of learning and memory, suggesting that these components are equally vital in shaping our intelligent behavior.

In the context of Nearest Neighbors, this cognitive parallel is particularly evident. The method's reliance on historical cases to inform present decisions mirrors our own tendency to draw upon past experiences to navigate current situations. It suggests that, rather than engaging in exhaustive reasoning for each new problem, both artificial and human agents lean on the repository of past experiences stored in memory. This repository acts as a reservoir of solutions, ready to be accessed and applied, thereby reducing the cognitive load associated with constant problem-solving.⁸⁶⁻⁸⁸

This perspective on cognition proposes a more integrated view of intelligence, where reasoning, memory, and learning interact seamlessly. It posits that our intellectual prowess is not solely the product of our ability to reason through new challenges but also our capacity to learn from past experiences and to efficiently retrieve and apply those lessons when similar situations arise.⁷⁹ In essence, the Nearest Neighbors method and the broader principle of learning by recording cases offer a window into understanding the cognitive strategies that underpin both artificial and human intelligence, emphasizing the symbiotic relationship between reasoning, memory, and learning in the orchestration of intelligent behavior.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

Learning by Recording Cases, particularly through the Nearest Neighbor and K-Nearest Neighbors (KNN) methods, has conceptual links and practical applications in various modern AI paradigms:

- **Instance-Based Learning & Lazy Learning:** KNN is a classic example of instance-based learning (or lazy learning), where the model doesn't learn an explicit function from the training data but instead stores the training instances (recorded cases).

Predictions for new instances are made by comparing them to these stored cases. This contrasts with eager learning methods (like training a neural network) which build a general model beforehand.

- **Recommendation Systems:** Many recommendation systems work on principles similar to nearest neighbors. To recommend items (e.g., movies, products) to a user, the system might find “neighboring” users with similar tastes or items similar to those the user has liked in the past. The “cases” here are user profiles or item profiles.
- **Pattern Recognition & Classification:** KNN is a straightforward and often surprisingly effective algorithm for classification. A new data point is classified based on the majority class among its k nearest neighbors in the feature space. The “recorded cases” are the labeled training examples.
- **Deep Learning for Similarity Learning:** Deep learning models excel at learning powerful feature representations (embeddings) from data, making the concept of “distance” or “similarity” highly meaningful for finding recorded cases. For example, Word2Vec learns vector representations (embeddings) for words where semantic relationships are captured; the classic analogy “Man is to Woman as King is to Queen” can often be solved using vector arithmetic on these embeddings (e.g., $\text{vector}(\text{'King'}) - \text{vector}(\text{'Man'}) + \text{vector}(\text{'Woman'})$ is close to $\text{vector}(\text{'Queen'})$). More advanced techniques like Siamese networks or triplet networks are also explicitly trained to learn embeddings where similar items are close together and dissimilar items are far apart in the embedding space. These learned embeddings can then be used with a nearest neighbor approach for sophisticated retrieval or classification based on recorded cases.
- **Information Retrieval & Semantic Search:** When you search for documents similar to a query document, or images similar to a query image, the underlying systems often use nearest neighbor search in a high-dimensional embedding space. The “cases” are the documents/images in the database, and the system retrieves the ones “nearest” to the query.
- **Anomaly Detection:** An instance that is far away from all (or most) of its k nearest neighbors in a dataset can be flagged as an anomaly or outlier. The recorded cases define what is “normal.”
- **Generative AI & Retrieval Augmented Generation (RAG):** In RAG systems, LLMs are augmented with information retrieved from a knowledge base. This retrieval step often involves finding the most relevant “cases” (chunks of text, documents) from the knowledge base that are similar to the user's query, using embedding-based nearest neighbor search.

While learning by recording cases and nearest neighbor methods are foundational, their core idea of leveraging similarity to past instances for current decision-making remains a powerful concept that permeates many advanced AI systems, often enhanced by modern representation learning techniques.

CHAPTER SUMMARY

In this chapter, we introduced Learning by Recording Cases as a fundamental learning method within knowledge-based AI. This approach emphasizes the storage and reuse of individual experiences or instances (cases) to inform future problem-solving and decision-making. We saw this illustrated through simple examples like predicting a block's color and more complex scenarios like a programmer recalling past solutions or a physician diagnosing a patient based on similar past cases.

A key technique explored for implementing this was the Nearest Neighbor method, which identifies the most similar past case to a new problem by calculating distances (often Euclidean distance) in a feature space. This was demonstrated with block world examples and a navigation task for an autonomous vehicle, showing how features of a problem (like dimensions or start/end points) can be used to find the closest recorded case.

We then briefly touched upon the K-Nearest Neighbors (KNN) method as an extension, where multiple similar cases are considered to make more robust predictions. The challenges of this approach, such as the “curse of dimensionality” and the fact that similar cases don't always guarantee directly applicable solutions, were acknowledged.

The chapter also highlighted the strong cognitive connection of these methods, as humans frequently rely on past experiences (recorded cases) and similarity judgments to solve new problems, often without extensive reasoning. This underscores the importance of memory and learning alongside reasoning in intelligent behavior.

Finally, we connected Learning by Recording Cases and Nearest Neighbors to several modern AI paradigms. These foundational concepts are echoed in instance-based learning, recommendation systems, pattern recognition using KNN, deep learning for similarity learning, semantic search in information retrieval, anomaly detection, and Retrieval Augmented Generation (RAG) for LLMs.

This exploration sets the stage for understanding more sophisticated learning and reasoning techniques, such as Case-Based Reasoning (discussed in the next chapter), which builds upon these ideas of storing, retrieving, and adapting past experiences.

ADDITIONAL READINGS & REFERENCES

- 67 Chong, H.-Q., Tan, A.-H. & Ng, G.-W. Integrated cognitive architectures: a survey. *Artificial Intelligence Review* **28**, 103-130 (2007).
- 78 Mitchell, M. Abstraction and analogy-making in artificial intelligence. *Annals of the New York Academy of Sciences* **1505**, 79-101 (2021).
- 79 Aha, D. W., Kibler, D. & Albert, M. K. Instance-based learning algorithms. *Machine learning* **6**, 37-66 (1991).
- 80 Cunningham, P. & Delany, S. J. K-nearest neighbour classifiers-a tutorial. *ACM computing surveys (CSUR)* **54**, 1-25 (2021).

- 81 Taunk, K., De, S., Verma, S. & Swetapadma, A. in *2019 international conference on intelligent computing and control systems (ICCS)*. 1255-1260 (IEEE).
- 82 Ullman, M. T. in *Neurobiology of language* 953-968 (Elsevier, 2016).
- 83 McLean, S. F. Case-based learning and its application in medical and health-care fields: a review of worldwide literature. *Journal of medical education and curricular development* **3**, JMECD. S20377 (2016).
- 84 Danielsson, P.-E. Euclidean distance mapping. *Computer Graphics and image processing* **14**, 227-248 (1980).
- 85 Guo, G., Wang, H., Bell, D., Bi, Y. & Greer, K. in *On The Move to Meaningful Internet Systems 2003: CoopIS, DOA, and ODBASE: OTM Confederated International Conferences, CoopIS, DOA, and ODBASE 2003, Catania, Sicily, Italy, November 3-7, 2003. Proceedings.* 986-996 (Springer).
- 86 Mahr, J. B. & Csibra, G. Why do we remember? The communicative function of episodic memory. *Behavioral and brain sciences* **41**, e1 (2018).
- 87 Anderson, J. R. A spreading activation theory of memory. *Journal of verbal learning and verbal behavior* **22**, 261-295 (1983).
- 88 Hendler, J. A. *Integrating marker passing and problem solving: A spreading activation approach to improved choice in planning.* (Psychology Press, 2014).

Chapter 8

Case Based Reasoning

INTRODUCTION TO CASE-BASED REASONING (CBR)

This chapter focuses on Case-Based Reasoning (CBR), a powerful problem-solving paradigm where cognitive agents tackle new challenges by recalling, reusing, and adapting solutions from similar past experiences.⁸⁹ Building directly on the concepts from the previous chapter on Learning by Recording Cases (which primarily dealt with finding identical or very close matches), CBR takes a significant step further. It addresses the more common and complex scenario where new problems are *similar* to, but not exact replicas of, past encounters. This requires not just retrieval of past instances, but a more sophisticated process of reasoning to modify and apply those past solutions effectively.

CBR is characterized by a cycle of core processes that we will explore in detail:

- **Case Retrieval:** Finding and selecting the most relevant past case(s) from a memory store.
- **Case Adaptation:** Modifying the solution from the retrieved case(s) to fit the specific nuances of the current problem. This is the “reasoning” heart of CBR.
- **Case Evaluation:** Assessing the effectiveness of the adapted solution, often through simulation or real-world application.

- **Case Storage (Learning):** Integrating the new problem and its validated solution (or the lessons learned from a failure) back into the case memory, thereby enriching the agent's experience base for future problem-solving

Throughout this chapter, we will examine these phases, explore different strategies for case adaptation, and discuss methods for organizing and retrieving cases efficiently. Our goal is to understand how CBR enables AI systems to learn from experience and exhibit flexible, human-like problem-solving in complex situations.

From Identical to Similar: The Colored Blocks Revisited

To demonstrate the distinction between Case-Based Reasoning and Learning by Recording Cases (also known as Instance-Based Learning, which we touched upon with the nearest neighbor method), we return to our familiar micro-world of colored blocks. In this world, the variety of blocks presents itself through distinct colors, shapes, and even tactile experiences, providing a sense of their varying sizes. Imagine being presented with a new block, one that notably differs in size from any existing blocks in this micro-world. Given this context, what might be your prediction about the color of this new, uniquely sized block?

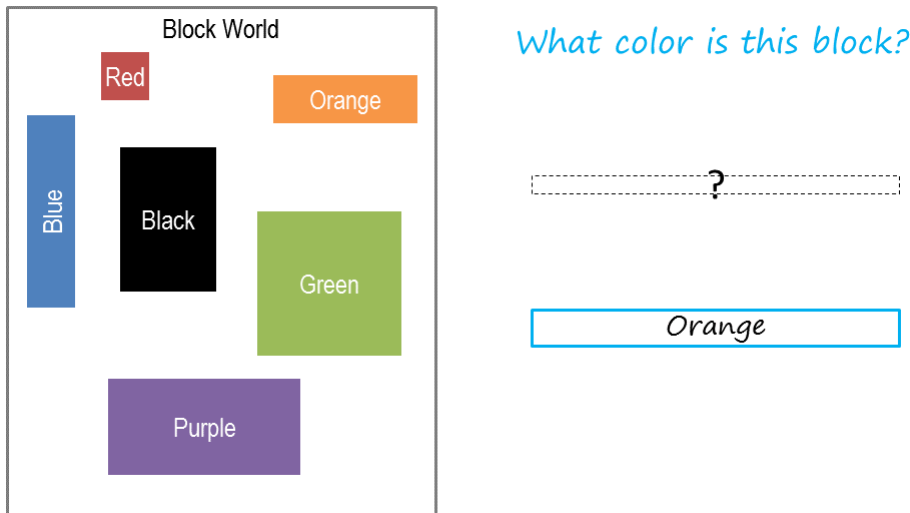
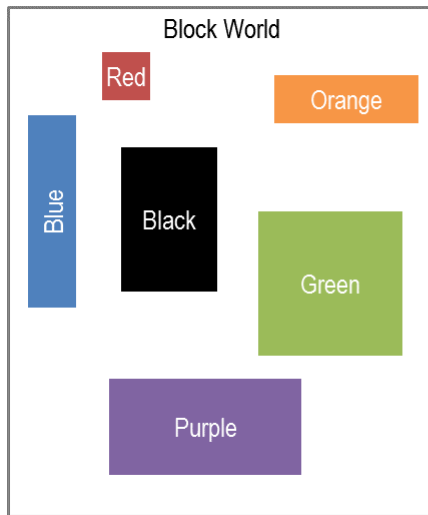


Figure 8-1 Blocks World Example with a similar, but not identical block to a previously learned case.

To illustrate the point further, let's join the conversation between Ashok and David, following the introduction of a new, distinctly sized block in our micro-world of colored blocks.

Ashok David, considering the unique size of this new block, what color do you speculate it to be?

David Given its unusual long, narrow shape, unlike anything we've seen in this collection, I ventured a guess with orange. However, I'm not sure about this choice. And as it turns out, that guess was incorrect; the block is actually pink.



What color is this block?

Figure 8-2 Blocks World Example with the missing block reveals as a pink, long, narrow block.

Ashok Exactly, David, and that's a crucial observation. Frequently, new challenges we encounter don't directly match any previous experiences. When faced with such unique situations, mere retrieval from memory isn't sufficient. We must engage in deeper reasoning. The term “case-based reasoning” encapsulates two concepts: 'case-based', which we've explored through memory retrieval, and 'reasoning', which involves adapting retrieved knowledge to tackle new problems effectively. Let's now shift our focus to this reasoning aspect and explore how we can adapt solutions from past cases to address new challenges.

CASE-BASED REASONING CORE PROCESSES

To examine a more practical example, let's revisit the navigation problem from Long Island we explored in our last chapter, referred to here as “the Q problem.” The task is to navigate from point Q to a specified endpoint. We identified case D as potentially useful for the Q problem,

providing a route from one location to another. However, case D cannot be applied directly due to differences in start and end locations, necessitating adaptation.



Figure 8-3 Map of Long Island where Q is the new starting location and D is a known starting location and path.

This brings us to the core process of Case-Based Reasoning (CBR), which comprises four essential steps⁸⁹:

1. **Retrieval:** This initial step involves fetching relevant cases from memory. We previously discussed using the K-Nearest Neighbor method for this purpose.
2. **Adaptation:** After retrieving a case, the next step is to adjust it to suit the current problem. For instance, altering the route in case D to match the start and end points of the Q problem. In programming, this might involve adapting a previously used program design to address a new task.
3. **Evaluation:** The adapted solution is then assessed to ensure it resolves the new problem effectively. This could involve simulating the proposed navigation route or running the newly adapted program to test its efficacy.
4. **Storage:** Successful solutions are encapsulated as new cases and added to the memory. This enriches the repository of cases, enhancing the system's ability to tackle future problems.

Through these steps, CBR integrates memory, reasoning, and learning. It leverages a rich database of past cases, applies logical adjustments during adaptation, assesses solutions' viability, and learns by storing new experiences. This method exemplifies the synergy of cognitive processes in problem-solving.

KEY ASSUMPTIONS OF CBR

Case-Based Reasoning (CBR) operates under several key assumptions about the nature of problems and solutions encountered by cognitive agents:

- **Recurring Patterns:** The foundation of CBR lies in the observation that the world presents recurring patterns of problems. Much like the laws in physics, such as Newton's second law of motion, identify regularities in the physical world, CBR posits that intelligence must grapple with recurring types of challenges. This assumption underscores the cyclical nature of problems, suggesting that what has been encountered before will likely be encountered again.
- **Similarity in Solutions:** A central tenet of CBR is the belief that similar problems generally have similar solutions.⁹⁰ This is akin to noticing that in different geographic grids, like those of Long Island and Dallas, Texas, proximal problems tend to have proximal solutions.



Assumptions of Case-Based Reasoning

- *Patterns exist in the world*
- *Similar problems have similar solutions*

Figure 8-4 Similarity of problem solving illustrated with a map grid of Long Island and Dallas.

However, this assumption is not without exceptions, as illustrated by two examples:

- **Velcro Straps on Shoes:** Consider the routine task of fastening shoes. Traditionally solved by tying shoelaces, the introduction of Velcro straps presents a similar problem – securing the shoe – but demands a fundamentally different solution.
- **Evolution of Touchscreens:** Early touchscreens, designed to detect a single point of contact, struggled with multiple simultaneous touches. Modern devices, however, adeptly handle multi-touch inputs. Despite the similarity in user interaction (touching the screen), the solutions diverge significantly due to advancements in technology and materials.

These examples highlight instances where the principle of similar problems yielding similar solutions does not hold. Despite these exceptions, the assumption generally stands; solutions to closely related problems often bear resemblance to one another, making CBR a viable approach to problem-solving in many, though not all, circumstances.

ADAPTATION STRATEGIES IN CBR

Having previously established the foundational steps of Case-Based Reasoning, we will now focus on the adaptation phase, assuming that a relevant case has been successfully retrieved, potentially through the K-Nearest Neighbor method discussed in the preceding chapter.

Our exploration of Case-Based Reasoning in the last chapter highlighted two critical challenges faced by AI agents: the inherent computational complexity of the problems they encounter and the constraints imposed by their limited calculative resources. This raises a pivotal question: How can AI agents tackle these complex problems efficiently and seemingly effortlessly, despite these constraints? The process of adaptation in Case-Based Reasoning offers a compelling approach to this conundrum, allowing AI agents to modify and apply solutions from past cases to new, yet similar, problems within the bounds of their computational capabilities.

Continuing from our previous discussion, we introduce a nuanced amendment to the concept of memory's role in problem-solving within AI. Rather than providing a complete solution, memory offers an almost correct answer, necessitating only minor adjustments or tweaks for adaptation. This subtle yet significant shift underscores the importance of the adaptation phase in Case-Based Reasoning. By leveraging nearly suitable solutions from past experiences, AI agents are tasked with making minor, efficient modifications to address the new problem at hand effectively. This approach enables AI systems to navigate the complexities of problem-solving with limited computational resources, emphasizing the critical role of adaptation in the broader context of Case-Based Reasoning.

In Case-Based Reasoning, adaptation often involves modest modifications to existing solutions, allowing for efficient problem-solving in a variety of contexts. One illustrative example is the task of cooking, where a favorite recipe might be slightly altered to accommodate different guests or to pair with a new side dish. The essence of the meal remains the same, but small adjustments are made to enhance the dining experience for the specific occasion. Similarly, in

the domain of software development, the process of importing data from a file is a routine task that seldom requires a complete overhaul for each new project. Instead, developers typically refer to previous implementations, adjusting the code minimally to cater to new file formats or data structures. This approach underscores the principle of leveraging past experiences while making necessary adjustments to address the nuances of the current problem.

This notion of evolutionary design is a common theme across various disciplines, encapsulating the idea that most new solutions are derived from existing ones through incremental improvements. Consequently, the challenge in Case-Based Reasoning lies in identifying effective strategies for adapting past cases to new problems. Three prevalent methods for achieving this are the model-based method, which relies on adaptation prior solutions to the current problem; the recursive case-based method, which involves breaking down the problem into smaller, more manageable cases; and the rule-based method, which employs heuristics to the problem. Each of these approaches offers a unique pathway to tailoring past solutions to new challenges, illustrating the diverse toolkit available within Case-Based Reasoning for problem-solving.

Model-Based Reasoning for Adaptation

Exploring the model-based method for case adaptation reveals a strategic approach to navigating from known solutions to new problems. In our illustrative micro-world scenario, imagine the task of traveling from an office to a restaurant. Upon encountering this challenge, a previously stored case, perhaps detailing a route to a doctor's office near the restaurant, is retrieved from memory. This case, while useful, does not exactly match the current need.

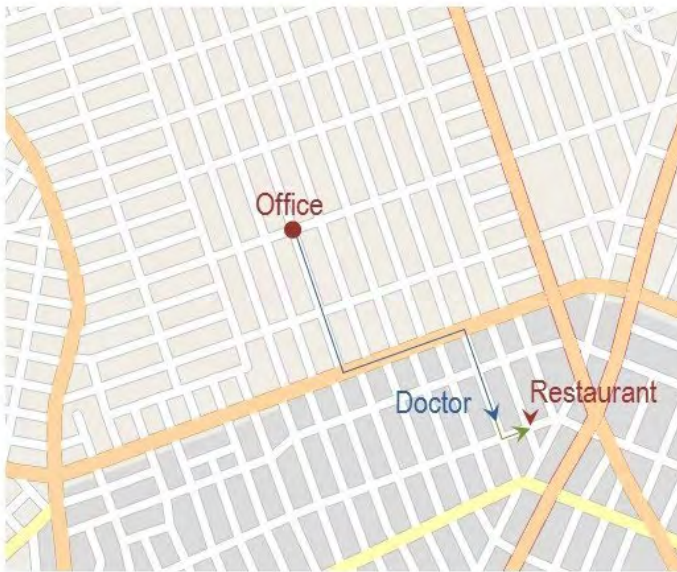


Figure 8-5 Illustrative example showing how a path from the office to the doctor is then adapted to reach the restaurant.

To bridge this gap, the model-based method employs a comprehensive model or map of the environment to guide adaptation. This model provides a structured representation of the world, outlining possible paths and connections between locations. In our scenario, the model would suggest a specific route from the doctor's office to the restaurant, effectively extending the retrieved case to cover the new destination. By leveraging a detailed model of the world, this method allows for the logical extension of existing solutions to encompass new problems. The process involves integrating the retrieved case with insights drawn from the model, culminating in an adapted solution that precisely addresses the new task. This exemplifies the power of model-based adaptation in Case-Based Reasoning, where the depth of world knowledge enables flexible and accurate problem-solving.

In the world of programming, the adaptation of solutions across different languages showcases the flexibility of the model-based method in Case-Based Reasoning. For instance, transitioning from file input operations in Python to Java may initially seem daunting due to language differences. However, the fundamental process remains consistent across both languages. By leveraging a conceptual model of Java's operational framework, one can adapt the familiar Python-based approach to suit Java's syntax and libraries, effectively translating the solution from one programming context to another. Similarly, in the field of design, particularly in the creation of Very-Large-Scale Integration (VLSI) circuits, the adaptation process is

informed by both the configuration of design elements and an underlying model of their intended functionality. This approach ensures that adaptations are not merely structural but also functional, aligning with the operational principles of the design.

This concept of utilizing models for adaptation was a pioneering theme in Ashok's doctoral research during the 1980s, which was among the first to amalgamate model-based reasoning with case-based reasoning.⁹¹ The crux of the dissertation was the strategic use of models to facilitate the adaptation, evaluation, and storage of cases, highlighting the integral role of models in enhancing the applicability and effectiveness of Case-Based Reasoning across diverse problem-solving domains.

Recursive Case-Based Reasoning for Adaptation

The concept of recursive case-based reasoning offers a dynamic approach to adapting solutions by leveraging the methodology of Case-Based Reasoning itself. Consider the scenario where an individual has previously navigated from their office to a restaurant, establishing a known route. Subsequently, the need arises to travel from home to the same restaurant. Although the direct route from home to the restaurant is unfamiliar, the individual recalls the route from home to the office, as well as the previously determined path from the office to the restaurant.



Figure 8-6 Illustrative example showing how a path from home to the restaurant is found by recursively finding solutions to elements of the overall problem.

By sequentially applying these known routes, a comprehensive solution is formulated, connecting home to the restaurant via the office. This process exemplifies recursive case-based reasoning: the initial retrieval of the case from home to the office provides a segment of the solution. The unresolved portion of the journey, from the office to the restaurant, is then treated as a new problem and reintroduced into the case memory system. This prompts the retrieval of another case, detailing the route from the office to the restaurant. The amalgamation of these cases constructs a complete pathway from the starting point at home to the desired destination, the restaurant. This methodology showcases the adaptability of Case-Based Reasoning, where partial solutions from distinct cases are concatenated to address more complex or segmented problems, illustrating the iterative potential of case-based reasoning in constructing comprehensive solutions from modular, previously solved cases.

In programming, the task of persisting data between program instances exemplifies the application of recursive case-based reasoning. This overarching challenge can be decomposed into two distinct subproblems: file input and file output. Addressing these subproblems may involve recalling solutions from previous projects – drawing upon a past case for file input and another for file output. Through this recursive breakdown, the broader issue of data persistence is resolved by sequentially tackling its component parts. Similarly, in the field of design, recursive case-based reasoning facilitates the creation of innovative solutions by amalgamating partial solutions from diverse cases. Consider the design challenge of developing a “microbot” capable of stealthy underwater navigation. Inspiration might be drawn from the copepod, known for its slow, wake-minimizing movement, thereby addressing the aspect of stealth at lower speeds. However, this solution presents a new sub-goal: achieving stealth at higher velocities. A subsequent case, such as the squid's ability to blend its wake with surrounding water, provides a solution for high-speed stealth. By combining these partial solutions, a comprehensive design for the microbot emerges, illustrating the concept of compound analogy – a form of adaptation that employs recursive reasoning to integrate solutions from multiple cases into a cohesive whole.

Rule-Based Reasoning (Heuristics) for Adaptation

Exploring the third approach to case adaptation, we encounter the rule or heuristic method, which leverages rules of thumb to modify and apply past solutions to new situations. Heuristics, by their nature, offer general guidelines that are effective in many, but not all, circumstances. For instance, a common heuristic for locating a city's downtown area involves seeking out the tallest buildings, a strategy that tends to be reliable in North America but may not hold true globally. Applying this concept to case-based reasoning, consider the scenario of needing to return home from a restaurant. If a previous journey from home to the restaurant has been successfully completed and stored in memory as a case, this case can be retrieved when faced with the reverse journey. To adapt the original case to the new problem, a heuristic might suggest simply reversing the directions taken. While this rule of thumb can often provide a viable solution, as depicted in

the adaptation, it's important to acknowledge its limitations. Not all routes can be reversed due to one-way streets or other navigational constraints, underscoring the heuristic's nature as a generally reliable, but not infallible, method of adaptation.



Figure 8-7 Illustrative example showing the application of a rule of reversing a known answer to find the path from the restaurant to home.

Returning to the world of programming, especially when designing new software, efficiency can be paramount, particularly in processes like file input which can be resource-intensive. An illustrative scenario involves adapting a previously used method of file input to suit the efficiency needs of a new program. For instance, if an earlier solution involved reading data one byte at a time, and the new program prioritizes efficiency, a heuristic rule could be applied: “For more efficient file input, read entire arrays of data at once rather than byte by byte.” This heuristic guides the adaptation of the previous case, shifting the method to batch processing of data, thereby enhancing efficiency. This concept of applying heuristics extends beyond programming to the broader field of design. Designers frequently employ similar rules of thumb to address common challenges. For example, a heuristic might suggest that to reduce the weight of an artifact, one should consider using lighter materials. Such heuristics serve as practical guidelines, enabling designers to make informed decisions when adapting existing solutions to meet new requirements or constraints.

EVALUATION AND STORAGE IN CBR

Evaluation

In our exploration of Case-Based Reasoning, having explored the intricacies of adaptation, we now pivot our attention to the critical phase of evaluation. This stage is pivotal in determining the efficacy of the candidate solution derived from the adaptation phase, effectively assessing its fit for the problem at hand. A pragmatic approach to evaluating a case-based solution is through simulation. To illustrate, let's revisit the scenario where the objective is to navigate from home to a restaurant. The solution proposed by case-based reasoning now stands as a candidate awaiting validation. To ascertain its practicality, one could employ simulation, a methodical rehearsal of the solution's steps in a controlled environment, or even execute a real-world trial. Such a process allows for a comprehensive examination of the solution's feasibility.

In the instance of navigating from home to the restaurant, undertaking a simulation or an actual execution could reveal the solution's effectiveness in real-world conditions. Should the solution prove viable during this evaluative stage, it would be deemed acceptable, thereby successfully concluding the evaluation phase of case-based reasoning. This step ensures that the solutions generated are not only theoretically sound but also practically applicable, reinforcing the utility of case-based reasoning in problem-solving.

Let's reinforce this idea by looking further into the evaluation phase, particularly in scenarios where the direct application of a candidate solution might encounter unforeseen obstacles. For instance, envision a scenario where, upon attempting to navigate back from the restaurant to home by merely reversing the previous route, we encounter one-way streets that render our solution infeasible. In domains where the practical execution of a solution is relatively cost-free, one might directly implement the solution to test its validity. However, in situations where execution costs are prohibitive, simulation becomes an invaluable step prior to any real-world application. The essence of evaluation is also deeply ingrained within the programming paradigm. Each instance of running a program to verify its functionality essentially constitutes an evaluation of whether the adapted solution effectively addresses the new challenge. Failure prompts a return to either the adaptation stage for further refinement or the retrieval stage for an alternative case that may offer a more viable solution. In the broader context of design, evaluation can take multiple forms, from simulation and prototyping to peer review. Simulation allows for a theoretical test of the design's functionality, while prototyping brings the design into the tangible, offering a concrete sense of its practicality. Peer critique, on the other hand, leverages the collective expertise and diverse perspectives of fellow designers to refine and validate the design further. Each of these methods underscores the multifaceted nature of evaluation in case-based reasoning, ensuring that solutions are not only theoretically sound but also pragmatically viable.

Storage

In our exploration of case-based reasoning, we've traversed through the stages of retrieval (introduced in the last chapter), adaptation, and evaluation.⁸⁹ Now, we arrive at the final, yet equally crucial phase: storage. Upon successfully navigating a problem and its corresponding solution, the culmination of this process involves encapsulating this experience into a case for inclusion in the case memory. This act of storage not only serves as a testament to the problem-solving journey but also as a repository of wisdom for future encounters. Consider the journey from home to a restaurant, an experience meticulously stored within the case memory. This stored case becomes a beacon when the need arises to navigate from the restaurant back home, offering a foundation upon which adaptation might commence. Thus, storage is not merely an act of archiving; it's an integral component of the learning process, continuously enriching the case memory with new experiences.

To facilitate efficient storage and retrieval, two primary mechanisms come to the forefront: indexing and discrimination trees. Indexing serves as the method by which cases are organized and accessed, ensuring that relevant cases can be quickly and accurately located when similar problems arise.⁹⁰ Discrimination trees, on the other hand, offer a structured way to differentiate among cases, allowing for the precise selection based on specific problem characteristics.

Indexing

Indexing plays a pivotal role in the organization and retrieval of cases from memory. Let's return to the domain of navigation, a familiar scenario where cases A, B, C, and D have been previously encountered. Initially, one might employ a straightforward indexing strategy, categorizing each case by the coordinates of its starting point. For instance, case A could be indexed by its initial coordinates, such as 3_E and 9_N , and similarly for cases B, C, and D.

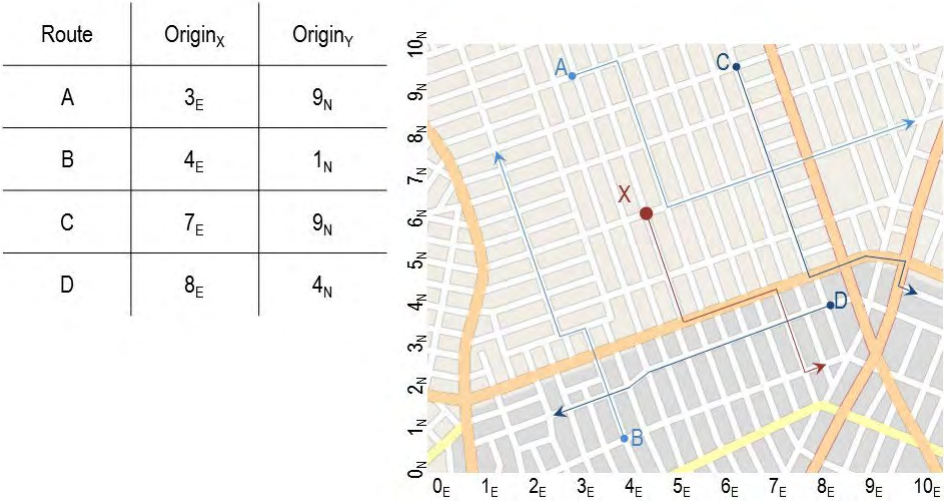


Figure 8-8 Storage indexing with four cases stored by their (x,y) coordinates.

Consider now the addition of a new case, X, which involves navigating from an office to a restaurant. Following the rudimentary indexing method, case X would be tagged based on the coordinates of its starting location. It's crucial to acknowledge, however, that this simple approach merely scratches the surface of indexing's potential. As discussed in the context of the K-Nearest Neighbor method, a more intricate indexing scheme that accounts for both the origin and destination points would offer a more nuanced and effective means of organizing and accessing cases.

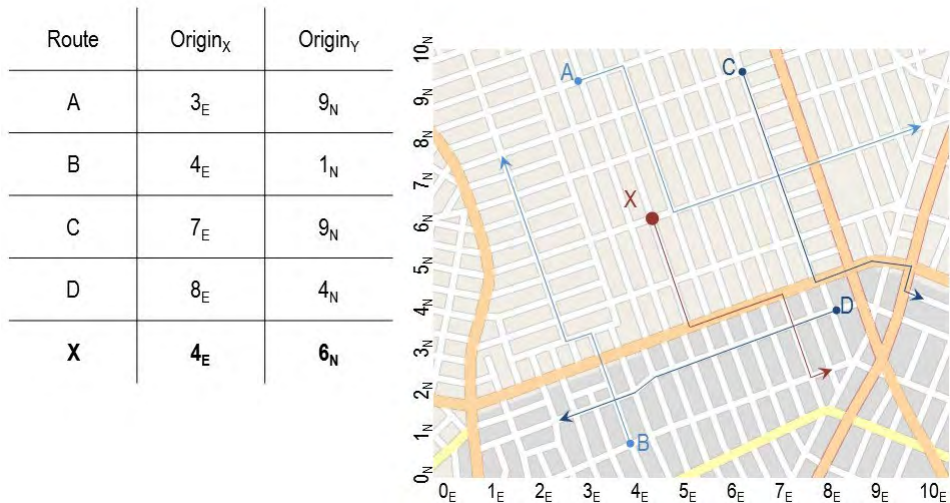


Figure 8-9 Storage indexing with case X added to storage.

This elementary example of indexing serves to introduce the core concept of tagging cases with identifiable markers. Yet, the scope of indexing extends far beyond mere geographical coordinates. In a navigation micro-world, for instance, indexes could encompass a variety of attributes, such as the scenic quality of routes or their speed. Such a rich indexing scheme opens the door to a more sophisticated and contextually aware retrieval of cases, enhancing the case-based reasoning process by providing a more comprehensive understanding of each case's unique characteristics and applicability to new problems.

As an example, in programming, especially when dealing with file input, the organization of cases can greatly benefit from a comprehensive indexing system. Cases can be meticulously categorized based on a variety of parameters and variables that define their unique attributes. For instance, cases might be tagged according to the programming language used, such as Java, Python, or C++. They could also be distinguished by their performance speed, identifying whether the file input process was fast or slow. Additionally, the type of file being read – be it text, XML, or another format – serves as another crucial indexing criterion. This nuanced approach to indexing ensures that each case is readily identifiable by its key characteristics. When faced with a new problem, the ability to match these detailed variables allows for the identification of the most relevant and similar past case. This precision in retrieval is vital, as the primary goal of storing cases is to ensure their utility in future problem-solving scenarios. In broader design contexts, the concept of indexing extends to encompass a wide range of factors, including the function of the design, its operational environment, and specific performance criteria. Such a rich indexical structure not only enhances the efficiency of case retrieval but also

ensures that the solutions drawn from these cases are as relevant and applicable as possible to the new challenges at hand.

To further illuminate the concept of indexing within the context of our navigation micro-world, let's consider a hypothetical scenario introduced by a new case, labeled as “Y.” In this instance, our indexing scheme is predicated on the XY coordinates that mark the initial location of each navigational case.

Ashok Let's apply our current indexing scheme to our new case Y in the navigation micro-world. Given the coordinates for the initial location, how would you index case Y?

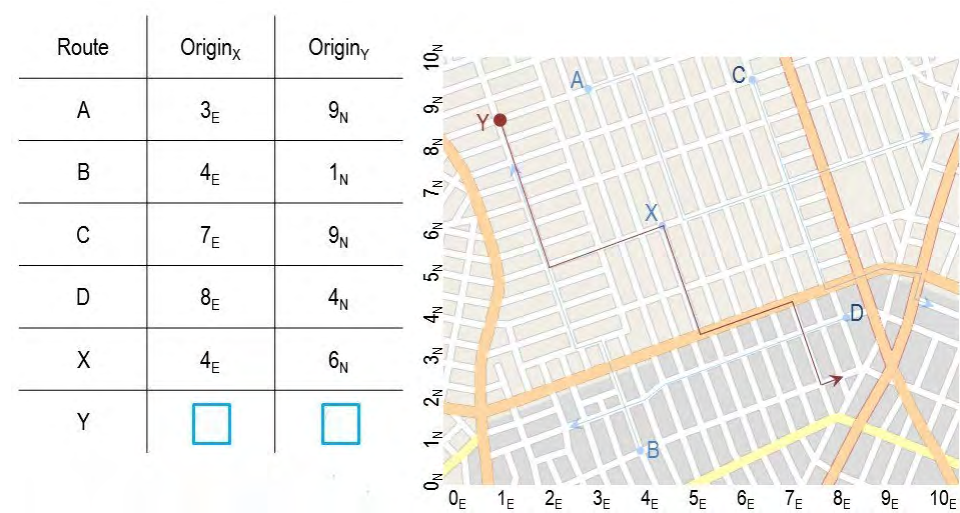


Figure 8-10 Example of tagging the location Y.

David Based on its position on the map, Y falls directly in line with 1_E on the horizontal axis and closely aligns with 9_N on the vertical axis. So, its index would be (1_E, 9_N).

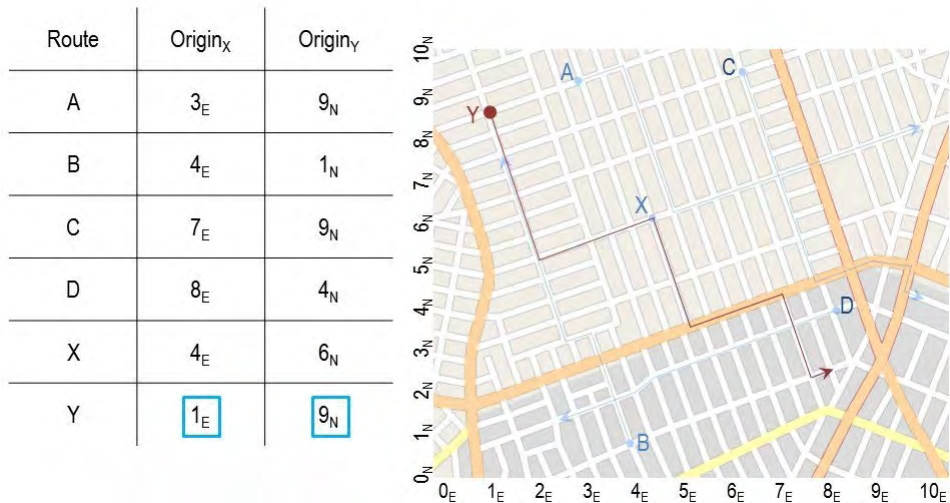


Figure 8-11 Example of tagging with the location Y given.

Ashok Exactly right, David. That's how we'd index case Y with our simple scheme. Now, let's shift our focus to a new case, Z, which involves the journey from the restaurant back to the home. This time, let's adjust our indexing approach. We'll index cases based on the X coordinates of the destination rather than the origin. With this new scheme, how would we index case Z? David, what do you think?

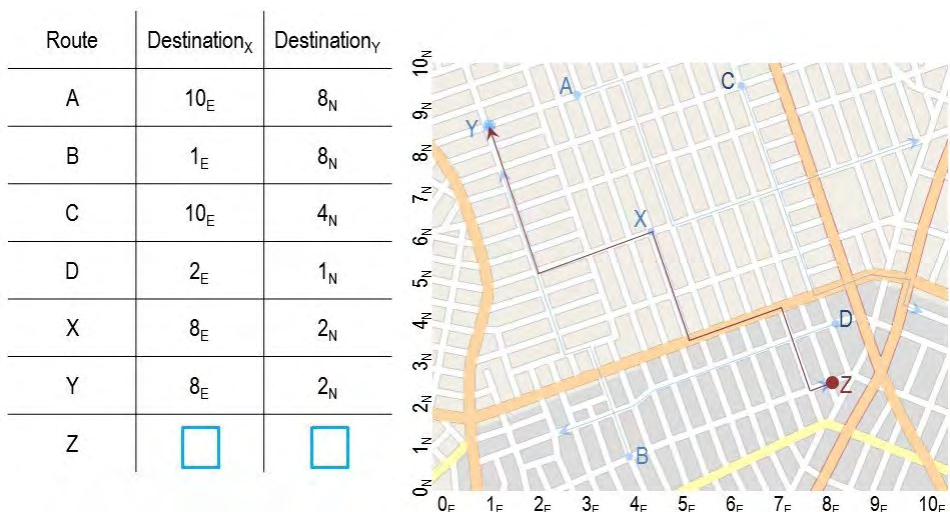


Figure 8-12 Example of tagging the location Z.

David Looking at the destination for case Z, it aligns with the starting point for case Y, which is at 1_E and 9_N . So, even with our changed indexing scheme, the indices for case Z would still be $(1_E, 9_N)$. But Ashok, as our case library grows and as we start dealing with a multitude of variables, I can foresee this becoming quite cumbersome and unwieldy. Searching through such a vast index could become quite challenging.

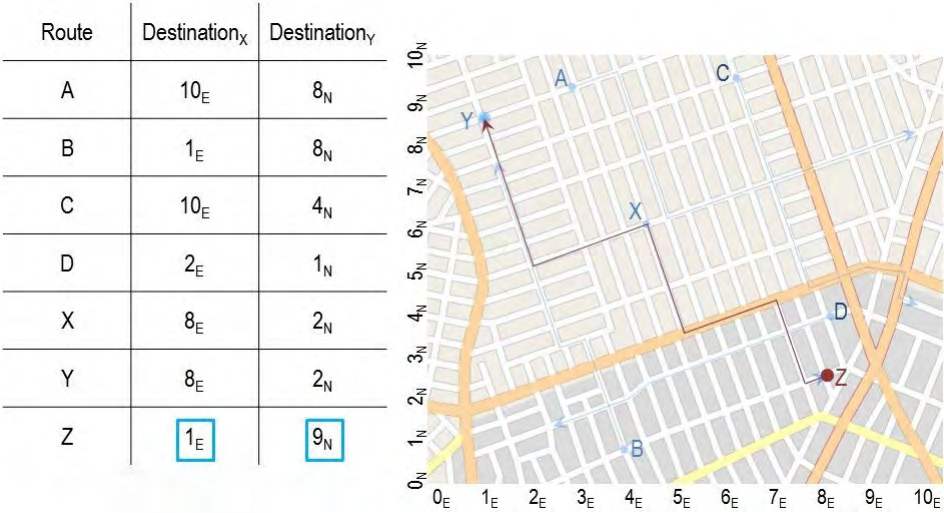


Figure 8-13 Example of tagging with the location Z given.

Ashok Indeed, David, that's a valid concern. As the case library expands and becomes more complex, we need to consider efficient ways to manage and navigate through it.

It's crucial to remember that the primary goal of storing cases is to facilitate efficient retrieval at a later time. An effective storage mechanism should enable quick and easy access to the stored cases, especially as the volume of cases and the dimensions for indexing grow. This is where the challenge lies: ensuring that our storage method remains efficient regardless of the scale and complexity of the case library. As the indexing approach may become cumbersome with an increasing number of cases and dimensions, we turn our attention to an alternative method known as discrimination trees. Discrimination trees offer a structured way to organize and store cases, potentially overcoming the limitations encountered with simple indexing and tagging methods.

Discrimination Trees

A discrimination tree serves as an advanced knowledge structure designed for efficient case storage and retrieval.⁹² At its core, the discrimination tree arranges cases as the terminal nodes, or leaf nodes, with a series of questions guiding the path from the root to these cases. Each question at a node is crafted to differentiate among cases based on their attributes, aligning with the indexing scheme of the cases. For instance, considering our earlier approach of indexing cases by their origin, a root question might inquire whether the origin is “north of 5_N ”. Affirmative answers lead down one branch, while negative responses take another path. Subsequent nodes pose further questions, such as “is the origin east of 5_E ?”, to progressively narrow down the selection, effectively sorting the cases by their geographical attributes.



Figure 8-14 Discrimination Tree knowledge structure.

This structured approach allows for the clear discrimination among cases, such as distinguishing case C from A and others, by following the tree's branches based on the answers to each posed question. The discrimination tree thus becomes a powerful tool for organizing case memory, enabling quick access to relevant cases by traversing through a series of simple decision points. The next step in leveraging discrimination trees involves understanding how to incorporate new cases into this structure. This process, known as incremental learning, is crucial for maintaining an up-to-date and efficient case library. Let's explore how new cases are integrated into the discrimination tree, ensuring that the tree evolves and adapts as additional experiences are encountered and stored.

Integrating a new case, labeled X, into an existing discrimination tree illustrates the dynamic nature of this knowledge structure and exemplifies incremental learning. As we navigate the tree

with X, we initially determine that its origin is north of 5_N , guiding us down the left branch. The next decision point asks whether X's origin is east of 5_E . With a negative response, we proceed down the right branch, leading to a junction where X and an existing case, A, are indistinguishable based on the current questions.



Figure 8-15 Learning a new case with Discrimination Trees.

To resolve this ambiguity and effectively discriminate between X and A, we introduce a new question tailored to their differing attributes: “Is the origin East of 3_E ?” For X, the answer is affirmative, contrasting with the negative response for A. This addition of a new node and question in the tree enables precise differentiation between the two cases, showcasing the tree's adaptability. This modified discrimination tree, enriched by the inclusion of case X, demonstrates incremental learning in action. With each new case addition, the tree's structure evolves, expanding the knowledge encapsulated within the case memory. This process not only enhances the tree's ability to discriminate among cases but also reflects the ongoing accumulation of knowledge, ensuring the case-based reasoning system remains current and effective. The concept of incremental learning, crucial for the continuous growth and refinement of the discrimination tree, will be further explored in subsequent discussions.

Transitioning our focus back to the programming example, specifically cases of file input, we can apply a similar conceptual structure to construct a discrimination tree. At the topmost level of this tree, a pivotal question might concern the programming language used in the case, such as Java, C++, or Python. Notably, discrimination trees are not restricted to binary formats; they can branch out into multiple paths based on the variety of potential answers. Therefore, at this initial juncture, branches could diverge based on the language involved, leading to distinct pathways for Java, C++, Python, and potentially others. Further down the tree, additional

questions could refine the categorization process, inquiring about the efficiency of the solution, the scale of the problem it addresses (big or small), and its intended application (personal or business use). This branching continues until reaching the leaf nodes, which represent specific cases that encapsulate the nuances one might need to consider when devising a new solution. Expanding on the flexibility of discrimination trees beyond binary structures is crucial for accommodating the complexity and diversity of cases. For instance, in a hypothetical micro-world of blocks with varying colors, a root question might inquire about the block's color. This would lead to an extensive array of branches, each corresponding to a different color. Such an approach exemplifies how discrimination trees can adeptly manage a wide range of distinctions, far beyond simple binary choices, thereby offering a rich, hierarchical structure for organizing and retrieving cases based on nuanced characteristics.

Ashok Let's tackle an exercise. Imagine we're presented with case Y, as depicted here. We also have this discrimination tree to the left. Where do you think case Y fits within this discrimination tree? David, what's your conclusion?



Figure 8-16 Discrimination Tree - Location case Y in the tree.

David I began with the root question: Is the origin North of 5_N ? The answer is yes, so I proceeded down the left path. The subsequent question asked: Is the origin East of 5_E ? The answer was no, so I veered right. The final query at that node was: Is the origin East of 3_E ? Again, the answer was no, leading me down the right path towards A. Therefore, I determined that case Y would align with the leaf node labeled A.



Figure 8-17 Discrimination Tree with the location for case Y found in the tree.

Ashok That's a solid analysis, David. However, we must now strategize on differentiating between case A and Y. Since A and Y occupy the same branch, we must devise a method to distinguish between A and Y. How might we accomplish this? David, what solution did you identify?



Figure 8-18 Extended Discrimination Tree with cases A and Y delineated from one another.

David Examining the map, A is situated to the right of Y, with Y to the left of all others. There's a vertical line intersecting A and Y, serving as a demarcation, aligning approximately with 2_E . Hence, my decision was 2_E .

Ashok That seems like an appropriate solution to me.

The concept of organizing case memory using a table raised concerns about scalability and search efficiency due to potential exponential growth in table size. Discrimination Trees offer a solution to this issue by enabling rapid exclusion of irrelevant branches through targeted questioning, thereby streamlining the search process.⁹² This hierarchical questioning framework significantly enhances search efficiency compared to the linear approach of table indexing. Specifically, the complexity of searching in a case library organized by indices, which is linear in nature, is contrasted with the logarithmic efficiency achieved through Discrimination Trees. This logarithmic efficiency is particularly advantageous for those familiar with computational complexity and big O notation, highlighting the superior search performance of Discrimination Trees in managing and accessing a growing case library.

ADVANCED CASE RETRIEVAL

In transitioning from the organizational strategies of case memory to the nuances of case retrieval, it's essential to recognize that the goal is to enhance retrieval methods to complement the previously discussed storage approaches. This enhancement aims to ensure efficient and effective access to cases that are most relevant to new problems, leveraging the structural foundations laid out by both tabular and discrimination tree storage methods. To achieve this, we will explore two distinct retrieval strategies: retrieval by index and retrieval by discrimination tree. Each of these strategies is tailored to work seamlessly with its corresponding storage method, ensuring that the retrieval process is both swift and precise. Retrieval by index involves navigating through a tabular array of cases, utilizing key identifiers or tags to quickly locate relevant cases. On the other hand, retrieval by discrimination tree employs a hierarchical decision-making process, where each node's question guides the path down the tree to the most pertinent case.

Retrieval by Index Example

Ashok Let's imagine our case library is laid out like a table, as we have here. Now, we're presented with a new challenge: we need to navigate from this starting point, marked by the black circle, to our destination, indicated by the chevron. Based on this setup, which case from our library should we pull up for this scenario? David, what's your take on this?

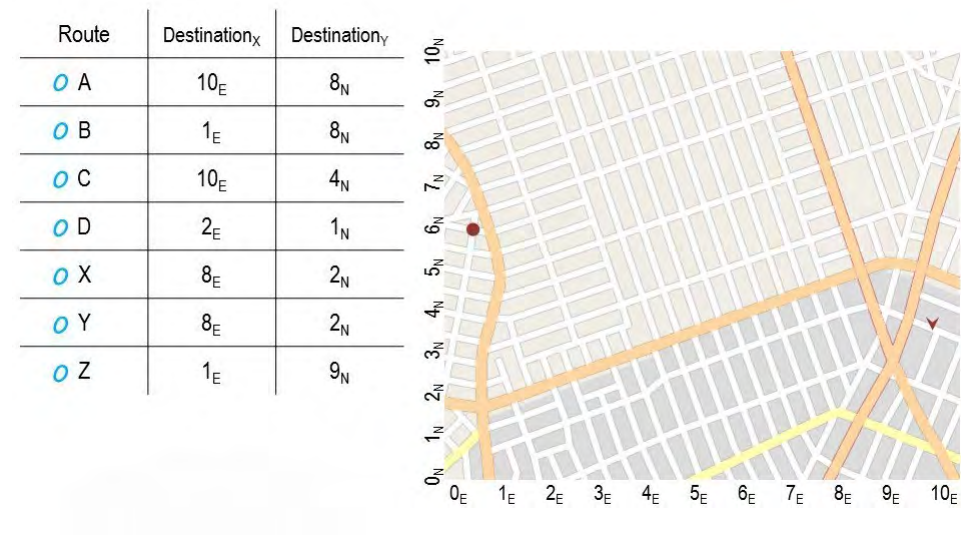


Figure 8-19 Retrieval by Index example.

David Well, when I looked at the X coordinates across our cases, I noticed that both A and C line up with the X coordinate of our new problem. But then, shifting focus to the Y coordinates, it's clear that A is way off target. So, my bet is on C, which aligns perfectly with both the X and Y coordinates of our new problem.

Ashok Precisely, David.

Retrieval by Discrimination Tree Example

Ashok Now, let's try the same exercise but with our case library organized as a discrimination tree. Here's the tree with our current cases, and here's the new problem we're facing, from this starting point to our destination. Using this tree, which case would we pull for this problem? David, what's your conclusion?



Figure 8-20 Retrieval by Discrimination Tree example.

David This time we're focusing on the starting point rather than the destination. Starting at the root, the first question is whether the starting point is north of 5_N , and it just makes it. Next, is it east of 5_E ? The answer's no. Moving on, is it east of 3_E ? Again, no. And is it east of 2_E ? It doesn't seem to be east of any reference point we have. So, the case we end up retrieving is Y.

Ashok Exactly, David. Y is indeed the case that best matches our new problem based on our discrimination tree.

ADVANCED CASE-BASED REASONING CONSIDERATIONS

Consider a scenario where the evaluation of a proposed solution, derived from an adapted case, falls short. Instead of discarding the case outright, a different adaptation strategy could be employed, giving the case another opportunity to provide a viable solution. Furthermore, there are instances where despite multiple attempts at adaptation, a case remains ill-suited to the problem at hand. In such situations, it might be prudent to set aside the ineffective case in favor of retrieving an alternative from the case memory that could offer a more fitting solution. Another complexity arises when an initially retrieved case seems promising but proves challenging to adapt to the new problem's specific requirements. This might necessitate abandoning the current case and initiating a fresh search for a more compatible one. Conversely, there may be times when a retrieved case aligns so closely with the new problem that it requires

no adaptation at all. This scenario mirrors the direct application of solutions seen in the Nearest Neighbor method, bypassing the adaptation phase and moving straight to evaluation. These examples highlight the inherent flexibility and non-linear nature of CBR, emphasizing the need for a dynamic approach that can navigate the complexities and nuances of real-world problem-solving.

The process of Case-Based Reasoning (CBR) extends beyond merely focusing on successful outcomes; it also involves a critical examination of failed cases. Understanding the utility of failed cases can be instrumental in refining problem-solving strategies, as they offer insights into potential pitfalls and challenges that may arise in future scenarios. Consider the experience of encountering repeated failures during file input operations due to exceeding the file's bounds – a common issue given the varying implementations of file handling across programming languages. Such recurrent failures form a repository of knowledge, highlighting the necessity of adapting and refining approaches to avoid similar mistakes. The value of failed cases lies in their ability to inform and guide the adaptation process, serving as cautionary tales that preemptively address potential issues in new problems. This preemptive insight allows for a more nuanced approach to adaptation, potentially circumventing repeat failures. However, this practice introduces a critical consideration known as the utility problem.⁹³ The indiscriminate storage of every successful case could lead to an unwieldy and bloated case library, complicating the retrieval process and diminishing efficiency. The crux of effective CBR lies in discerning which cases – both successful and failed – offer substantial learning value and contribute to a more versatile and comprehensive case library. This selective storage ensures that only those cases that expand the problem-solving repertoire or introduce novel solutions are retained, maintaining an optimal balance between breadth of knowledge and retrieval efficiency.

CASE BASED REASONING AND COGNITION

Case-Based Reasoning (CBR) is intricately linked with human cognition, particularly through the lens of analogical reasoning, a fundamental cognitive process. This form of reasoning operates on a spectrum of similarity, ranging from identical problems to those that are semantically distinct. At one extreme, when a new problem mirrors one previously encountered, the solution can be straightforwardly retrieved and applied. However, the far end of the spectrum involves problems vastly different from past experiences, requiring different cognitive strategies not covered by CBR. The most common cognitive challenges lie in the middle of this spectrum, where new problems bear a resemblance to, but are not identical to, past situations. Here, the cognitive task involves retrieving relevant past experiences, modifying them – tweaking, as it were – to suit the nuances of the current problem. This middle ground is where CBR shines and is most reflective of everyday human problem-solving.

Revisiting the concept of cognitive architecture, which comprises reasoning, learning, and memory, we see how these elements interact in CBR. Learning by recording cases, as discussed in a previous chapter, emphasized the roles of learning and memory by storing and retrieving

past cases. CBR extends this by integrating reasoning into the mix. It underscores the importance of learning, as we continuously accumulate new experiences and solutions. Memory plays a crucial role in allowing us to access these stored cases when faced with new problems. Reasoning is what enables us to adapt these recalled solutions to fit new contexts. Thus, CBR embodies a holistic approach to cognition, weaving together learning, memory, and reasoning in a process that mirrors the complex problem-solving undertaken by humans in their daily lives.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

Case-Based Reasoning (CBR), with its core cycle of retrieve, adapt, evaluate, and store, offers valuable perspectives and connections to various modern AI paradigms:

- **Machine Learning for Case Indexing, Retrieval, and Similarity: Modern ML techniques can significantly enhance CBR:**
 - **Enhanced Retrieval:** Instead of relying solely on predefined similarity metrics or manual indexing, features for cases can be learned using techniques like **deep learning** to create powerful embeddings. These embeddings can capture complex similarities, allowing for more nuanced and effective retrieval of the “nearest” or most relevant cases from a large case base. ML can also learn optimal indexing strategies or similarity functions directly from data.
 - **Discrimination Trees and Decision Trees:** The **discrimination trees** used in CBR for efficient case retrieval share structural similarities with **decision trees** from machine learning. Both use a tree structure with tests at internal nodes to partition items. However, their primary purpose and construction differ: discrimination trees are often built incrementally to organize and retrieve specific stored cases, while decision trees are typically learned in a batch process from training data for classification or regression on new instances. Understanding both can provide insights into structured knowledge organization and predictive modeling.
- **Reinforcement Learning (RL) and Learning from Experience:** RL agents learn from a sequence of experiences (state, action, reward, next state). Each such experience, or an entire trajectory, can be considered a “case.”
 - **Experience Replay:** Many RL approaches, like those incorporating **experience replay buffers**, effectively store and reuse past experiences (cases) to improve learning efficiency and stability.
 - **Model-Based RL & Simulation:** The evaluation phase of CBR, especially when it involves simulating an adapted solution, has conceptual parallels with model-based RL architectures like **Dyna-Q**. In Dyna-Q, an agent learns a model of the world from real experiences and then uses this model to *simulate*

further experiences for planning and policy improvement, akin to CBR's "what-if" evaluation.

- **Natural Language Processing (NLP) for Case Representation:** Many real-world cases are described in natural language (e.g., legal precedents, medical case reports, customer support tickets). NLP techniques are crucial for parsing these textual cases, extracting key features, and representing them in a structured format suitable for CBR systems. This allows CBR to be applied to a wider range of knowledge-intensive domains where information is primarily text-based.
- **Generative AI (GenAI) for Case Adaptation and Solution Generation:** LLMs and other generative models could potentially assist in the **adaptation** phase of CBR. Once a similar case is retrieved, an LLM could be prompted to suggest modifications to the old solution to fit the new problem's context, or even generate entirely new solution components inspired by the retrieved case. This blends the structured reasoning of CBR with the creative and flexible generation capabilities of GenAI.
- **Explainable AI (XAI):** CBR is inherently quite interpretable. When a CBR system proposes a solution, it can present the past case(s) it was derived from, offering a natural explanation for its reasoning (e.g., "I suggest this solution because it worked in this similar past situation..."). This transparency is a significant advantage over many "black-box" ML models and aligns well with the goals of XAI, making CBR a valuable approach where justifications are important.
- **Agentic AI and Lifelong Learning:** For AI agents designed to operate autonomously over long periods and in dynamic environments, the ability to learn from new experiences (store new cases) and adapt existing knowledge (adapt old cases) is crucial. CBR provides a natural framework for such lifelong learning, allowing an agent to continuously improve its problem-solving capabilities by accumulating and refining its case library in response to new encounters.

The principles of CBR – leveraging past experiences, adapting solutions, and learning from outcomes – are fundamental to building intelligent systems that can effectively solve novel problems by drawing upon what they already know. These principles are increasingly relevant as AI systems are expected to handle more complex, nuanced, and diverse tasks, often by integrating CBR concepts with other modern AI techniques.

CHAPTER SUMMARY

This chapter provided an in-depth look at Case-Based Reasoning (CBR), a significant KBAI method that mirrors human analogical reasoning by solving new problems based on solutions to similar past problems. We distinguished CBR from simple instance-based learning by emphasizing its core four-stage process: retrieval of relevant past cases (often using techniques like K-Nearest Neighbors), adaptation of the retrieved solution to fit the new problem's specifics,

evaluation of the adapted solution's effectiveness, and storage of the new problem-solution pair as a new case if successful.

We explored three key adaptation strategies: model-based reasoning (using a causal or structural model of the domain to guide modifications), recursive case-based reasoning (breaking the adaptation problem itself into smaller sub-problems to be solved by retrieving further cases), and rule-based or heuristic methods (applying general rules of thumb to transform a solution). These strategies allow CBR systems to handle problems that are not exact matches to stored cases.

The chapter also covered crucial aspects of managing the case library, including indexing methods (tagging cases with features for efficient lookup) and the use of discrimination trees as a more structured and scalable approach to organizing and retrieving cases through a series of differentiating questions. We touched upon advanced CBR considerations, such as the non-linear nature of the CBR cycle in practice, the learning value of failed cases, and the utility problem of deciding which cases to store to maintain an efficient and effective case base.

The strong cognitive connection of CBR was highlighted, particularly its link to human analogical reasoning and its integration of learning, memory, and reasoning processes.

Finally, the chapter established the relevance of CBR to modern AI paradigms. Its principles are evident in how machine learning enhances case retrieval, in the experience replay mechanisms of reinforcement learning, in NLP's role in representing textual cases, in the potential for Generative AI to assist with case adaptation, in CBR's inherent explainability (XAI), and its natural fit for lifelong learning in agentic AI. Case-Based Reasoning thus provides a robust and cognitively plausible framework for building AI systems that learn from experience and solve novel problems by intelligently reusing and adapting past knowledge.

ADDITIONAL READINGS & REFERENCES

- 89 Aamodt, A. & Plaza, E. Case-based reasoning: Foundational issues, methodological variations, and system approaches. *AI communications* 7, 39-59 (1994).
- 90 Richter, M. M. & Weber, R. O. *Case-based reasoning*. (Springer, 2016).
- 91 Goel, A. K. *Integration of case-based reasoning and model-based reasoning for adaptive design problem-solving*. (The Ohio State University, 1989).
- 92 Perner, P. in *Progress in Pattern Recognition, Image Analysis, Computer Vision, and Applications: 24th Iberoamerican Congress, CLARP 2019, Havana, Cuba, October 28-31, 2019, Proceedings* 24. 16-30 (Springer).
- 93 Craw, S. & Aamodt, A. in *Case-Based Reasoning Research and Development: 26th International Conference, ICCBR 2018, Stockholm, Sweden, July 9-12, 2018, Proceedings* 26. 62-77 (Springer).

Chapter 9

Incremental Concept Learning

INTRODUCTION TO INCREMENTAL CONCEPT LEARNING

In this chapter, we embark on an exploration of Incremental Concept Learning, a pivotal methodology within Knowledge-Based Artificial Intelligence (KBAI). Incremental Concept Learning is not only foundational to understanding how AI systems can evolve and adapt through experience but also central to the practices of case-based reasoning, where AI systems store new examples and learn from them over time.⁹⁴

We begin by defining the purpose of Incremental Concept Learning and its significance in the broader context of AI. This learning approach allows AI systems to abstract and refine concepts through the gradual accumulation of examples, each serving as a building block in the formation of a more comprehensive understanding.

The chapter examines key processes such as variabilization, specialization, and generalization.⁹⁵ Variabilization allows the AI to abstract specific instances into more general concepts, while specialization and generalization adjust the AI's concept definitions to more accurately fit new examples. These processes are crucial for the AI to adapt its understanding and approach to problems as new information becomes available.

Furthermore, we discuss heuristics for specialization and generalization, which provide AI systems with strategies to navigate the complex landscape of learning from incremental examples. These heuristics guide the AI in modifying its concept definitions in a way that

balances between being overly broad and unnecessarily restrictive. Through this exploration, we aim to provide a comprehensive understanding of how AI systems can incrementally learn concepts, mirroring aspects of human learning and cognition. Insights from Patrick Winston's "Artificial Intelligence" book complement our discussion, offering a rich context for the exploration of these ideas.⁹⁶

CORE CONCEPTS OF INCREMENTAL CONCEPT LEARNING: THE "FOO" EXAMPLE

In this section, we begin a journey to understand Incremental Concept Learning, a core technique in Knowledge-Based Artificial Intelligence. We start by introducing a hypothetical concept, for illustrative purposes dubbed "foo," through a series of examples. The first of these showcases an arrangement of four bricks: two positioned horizontally at the top and bottom, flanked by two vertical bricks on either side. This setup lays the groundwork for our exploration into how concepts can be abstracted from specific instances, a core aspect of incremental learning. As we proceed, we'll look into the intricacies of identifying, abstracting, and refining the concept of "foo" through varied examples, guiding the reader through the incremental learning process.

This is a foo.



Figure 9-1 Foo, First example (Positive)

Let's consider another example. This one is not a positive instance of the concept "foo." As in the previous case, we have four bricks: one at the bottom, one at the top, and two on the side. However, in this scenario, the two side bricks are in contact with each other.

This is not a foo.



Figure 9-2 Foo, Second example (Negative)

Let's examine a third example of the concept “foo.” This is a positive instance of “foo.” In this case, we also have four blocks, but instead of two vertical bricks as seen previously, we have two cylinders. Notably, these cylinders are not in contact with each other.

This is a foo.



Figure 9-3 Foo, Third example (Positive)

Ashok

I've presented you with three examples of the concept “foo.” Now, I'm about to show you another example. Based on your current understanding of the concept “foo,” what do you think, David? Does this new example align with your definition of the concept?



Figure 9-4 Foo, Fourth example

David Based on my existing knowledge, I would classify this as a “foo.” My understanding is that in the first example, bricks were vertically positioned; in the third, it was cylinders; and now, in this instance, we have special blocks positioned vertically. Since all these instances involve some form of block, they are interchangeable. This means that instead of a brick, one could feasibly use a cylinder or any similar object in a vertical orientation. Moreover, in all positive examples, these vertical blocks weren't touching each other.

Ashok I agree, David.

THE ROLE OF BACKGROUND KNOWLEDGE

In discussing the concept of “foo,” David's classification of the example as a “foo” was informed by his background knowledge. He observed that in various instances, objects positioned vertically – whether bricks, cylinders, or special blocks – were all considered interchangeable elements of the structure. This interchangeability stems from the understanding that these vertically oriented objects are all instances of a broader category, which we might term 'on blocks.' Such an interpretation hinges on the individual's background knowledge, which in David's case, allowed for the substitution of one type of block for another, provided they maintained a vertical orientation and were not in contact with each other. This example underscores the critical role that background knowledge plays in our ability to classify and understand concepts.⁹⁵ Depending on one's unique set of knowledge, the classification of an object or arrangement as an instance of “foo” could vary, highlighting the subjective nature of concept recognition.

In the context of Incremental Concept Learning, background knowledge refers to the pre-existing understanding and information that an individual or an AI system brings to the learning

process. It encompasses the concepts, categories, and experiences that have been previously acquired and are used to interpret and classify new information. For instance, understanding that bricks, cylinders, and special blocks can all be categorized as 'blocks' due to their functional or structural similarities is an example of background knowledge. This knowledge plays a crucial role in the learning process, as it influences how new examples are perceived, interpreted, and integrated into existing conceptual frameworks. It allows learners to draw analogies, make inferences, and generalize from specific instances, thereby facilitating the identification and abstraction of new concepts from given examples.

Ashok Let's consider another example. This time, we have four blocks arranged in a specific manner. There are two bricks: one positioned at the bottom and another at the top. In addition to these, there are two vertical cylinders placed adjacent to each other, in contact. Given our understanding and what we have learned so far, would this arrangement qualify as an instance of the concept “foo”? David, what do you think?



Figure 9-5 Foo, Fifth example

David Based on my understanding and background knowledge, this does not qualify as a “foo”. The arrangement of the vertical bricks is akin to that of the vertical cylinders. Therefore, if the criterion is that vertical bricks should not be in contact with each other, the same rule should apply to the vertical cylinders as well.

Ashok David's reasoning once again highlights the influence of background knowledge in the process of concept learning. He equates the vertical bricks with the vertical cylinders, applying the same criterion of non-contact to both. This interpretation is based on his understanding that the spatial arrangement and relational properties

between objects – whether bricks or cylinders – are crucial to identifying a “foo”. It's important to note that background knowledge varies among individuals, leading to different interpretations and conclusions about the same set of examples. Let's examine another instance in this sequence. In this scenario, we have four blocks again. There are three bricks: one positioned at the bottom and two on the sides, ensuring they are not in contact with each other. Additionally, there's a wedge placed at the top, replacing where a brick would typically be. David, would this configuration be considered an example of the concept “foo”?



Figure 9-6 Foo, Sixth example

David

I leaned towards “yes” on this one, though with less certainty than before. Two factors influenced my decision. Firstly, in previous examples, the top element was always a brick, but here we encounter a wedge in that position for the first time. However, from earlier observations, we've seen that various shapes – cylinders, bricks, arches – can fulfill the role of vertical supports. This leads me to believe that the type of block on top might also be interchangeable. Hence, considering the wedge as a type of block, I concluded this arrangement could still represent a “foo.” However, I acknowledge there's room for error in this interpretation.

Ashok

So to arrive at this answer, David again uses background knowledge. However, notice that someone else might say that, well, he does not think that this particular wedge here is the same kind of block as the brick. And therefore, this is not an example of foo. Specifically, individual perceptions can influence what someone considers a block, like a wedge or a brick. This can lead different people to different conclusions about whether an arrangement fits the concept of “foo.”

Foundational Insights from Learning Examples

In exploring the concept of Incremental Concept Learning, we encounter several key insights that illuminate the process of how agents, including humans, learn and generalize from examples. These insights, drawn from a practical exercise involving the abstract concept known as “foo,” underscore the nuanced nature of learning. They highlight the incremental nature of learning, the significance of labeled examples in supervised learning, the utility of both positive and negative examples, the distinction from case-based reasoning, the modest number of examples required for concept abstraction, and the challenges associated with finding the right balance in generalization. Each of these points sheds light on the intricate dance between acquiring knowledge, applying it, and refining it as new information becomes available.

1. **Incremental Learning:** Learning typically occurs one example at a time, reflecting the way cognitive agents, including humans, acquire knowledge incrementally rather than from large datasets all at once.
2. **Labeled Examples:** In many learning scenarios, examples are labeled as positive or negative by an instructor, a method akin to supervised learning in machine learning, where a “teacher” provides guidance on the categorization of each example.
3. **Order of Examples:** The sequence in which examples are presented can influence learning, with a common practice of introducing a positive example first, followed by a mix of both positive and negative examples.
4. **Contrast with Case-Based Reasoning:** This method of learning from examples differs significantly from case-based reasoning, where examples are stored in memory without abstraction. Here, the focus is on distilling broader concepts from specific instances.
5. **Small Sets of Examples:** The process often involves abstracting concepts from a relatively small number of examples, contrasting with approaches that rely on vast datasets.
6. **Challenges in Abstraction:** Deciding what to abstract and how to generalize from examples is complex. There is a risk of either overgeneralizing, leading to too broad an application of the concept, or overspecializing, where the concept is applied too narrowly. Finding the right balance in generalization is a critical and challenging aspect of learning.

These points emphasize the nuanced and incremental nature of learning, the importance of guidance in the form of labeled examples, and the challenges inherent in abstracting and generalizing concepts from limited data.

THE INCREMENTAL CONCEPT LEARNING ALGORITHM

The fundamental algorithm for incremental concept learning is presented through a visual diagram, illustrating the decision-making process when confronted with new examples.

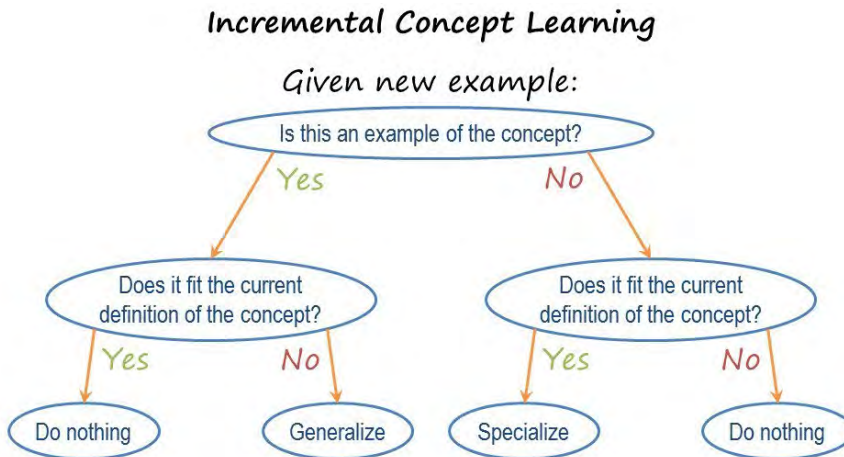


Figure 9-7 The Incremental Concept Learning Algorithm.

When an example is positive, the algorithm assesses whether the current conceptual definition encompasses this instance. If the definition already covers the example, no modification is required. However, if the definition fails to include the positive example, it necessitates a revision, prompting a generalization to ensure the example fits within the concept's scope.

Conversely, when dealing with a negative example, the algorithm determines whether the current definition erroneously includes it. If the negative example falls outside the concept's definition, the system remains unchanged. Yet, if the negative instance is covered by the current definition, it indicates a need for specialization to refine the concept, ensuring such examples are excluded in the future.

This process mirrors the learning patterns observed in human cognition, particularly in children, who often navigate the challenges of overgeneralization and overspecialization. For instance, a child might initially believe all cats must be black, based on their limited exposure. Encountering an orange cat necessitates a broadening of their concept of cats to include various colors. Similarly, a child's broad definition of dogs as furry, four-legged pets might lead to confusion upon meeting a cat fitting this description, requiring a more nuanced understanding to distinguish between cats and dogs. These examples highlight the dynamic nature of learning, where concepts evolve through continuous interaction with new information, balancing between generalizing to include diverse instances and specializing to maintain conceptual integrity.

KEY PROCESSES: VARIABILIZATION, GENERALIZATION, AND SPECIALIZATION

Examining the intricacies of incremental concept learning, let's explore a scenario involving an AI program learning about arches under the guidance of a teacher. Presented with an initial example comprising three bricks arranged to form an arch - two vertical bricks standing apart with a third resting atop - the AI faces the task of deciphering the underlying concept.



Figure 9-8 Arch, First example

Within the AI's perception, this setup is represented by four entities labeled A, B, C, and D, each denoting a brick. The spatial and support relationships among these bricks form the crux of the learning moment: Brick C is positioned to the left of Brick D, both C and D serve as supports for Brick B, which in turn supports Brick A. This configuration is visually captured in an accompanying diagram, illustrating the foundational structure of an arch as understood by the AI.

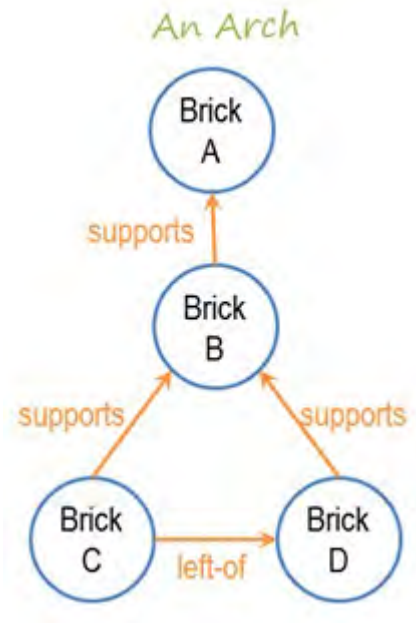


Figure 9-9 Arch Representation

Variabilization

At this juncture, the AI's learning is rudimentary, primarily centered around the concept of “variabilization.” This process involves transitioning from specific instances to more generalized notions; the bricks labeled A, B, C, and D are no longer seen as unique entities but as interchangeable instances of a broader category - bricks. By recognizing that any brick can fulfill the roles designated in the example, as long as the specified relationships are upheld, the AI grasps the essence of an arch. This pivotal step of variabilization sets the stage for further exploration into specialization and generalization, pivotal mechanisms in incremental concept learning.⁹⁷

Generalization

Building upon this, the AI program encounters a pivotal moment when presented with a second example by the teacher.⁹⁸ This example, distinctly labeled as a positive instance, introduces a new challenge: the task of accommodating this example within the existing framework of the arch concept.

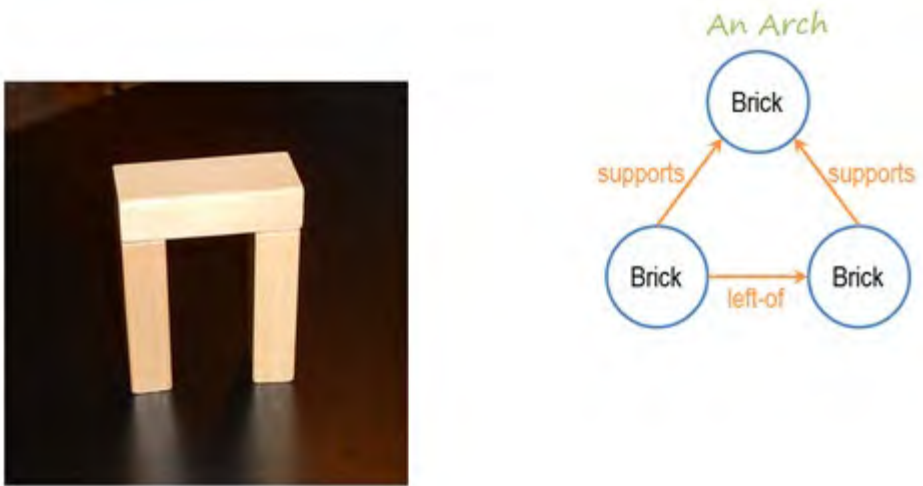


Figure 9-10 New example of an arch and its corresponding representation.

The teacher's presentation of this example, while maintaining the essence of an arch, subtly alters its composition. The AI program's task is to discern how to integrate this new information without disrupting the foundational understanding already established. Through the process of variabilization, the program generalizes the specific 'Brick A' to a more universal 'Brick', aligning with the principle that any brick, irrespective of its individual characteristics, can fulfill the required role within the arch structure.

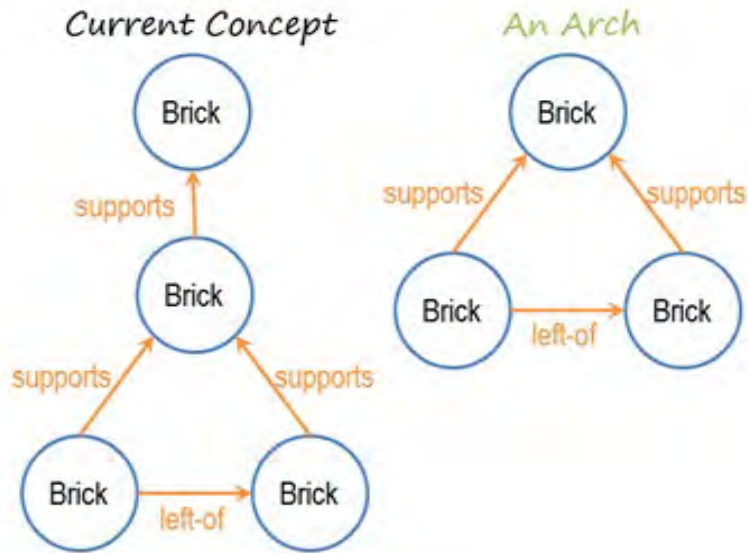


Figure 9-11 Current Arch Representation (left) and arch representation based on the new example (right).

Faced with the need to embrace this new example within the arch concept, the AI program leans towards generalization. A strategic move involves employing the 'drop-link' heuristic, a rule of thumb aimed at simplifying the concept to encompass a broader range of instances. By eliminating the link that necessitates the top brick's support by another, the program crafts a new, more inclusive definition of an arch. This revised concept not only embraces the new example but also retains compatibility with the initial one, showcasing the heuristic's effectiveness in bridging between specific instances and a generalized understanding.

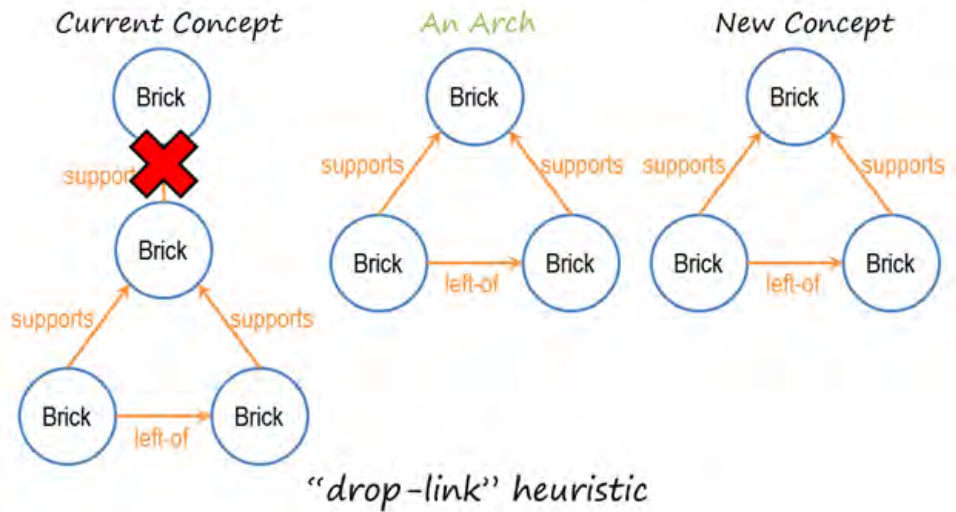


Figure 9-12 Arch Representation, Drop-Link heuristic.

The 'drop-link' heuristic exemplifies the AI program's navigation through the vast expanse of potential generalizations and specializations inherent in learning from a limited set of examples. It highlights the delicate balance between maintaining the integrity of existing concepts and adapting to accommodate new information, a fundamental aspect of incremental concept learning. This heuristic serves as a guiding principle, enabling the AI program to efficiently traverse the learning landscape, ensuring that each step of generalization is a measured stride towards a more comprehensive understanding.

Specialization

Excluded Negative Examples

As the AI program looks deeper into the nuances of incremental concept learning, it encounters a pivotal moment with the introduction of a new example by the teacher. This example presents a scenario where three bricks are aligned, but not in the traditional arch configuration – the third brick does not rest atop the first two, deviating from the established concept of an arch.



Figure 9-13 The AI agent is presented with a new negative example of an arch (left) with its corresponding representation also shown (right).

Confronted with this example, which the teacher explicitly labels as a negative instance, the AI program faces the challenge of reconciling this new information with its existing concept definition. The task at hand is not merely to acknowledge this example but to ensure that the program's concept of an arch excludes such configurations.

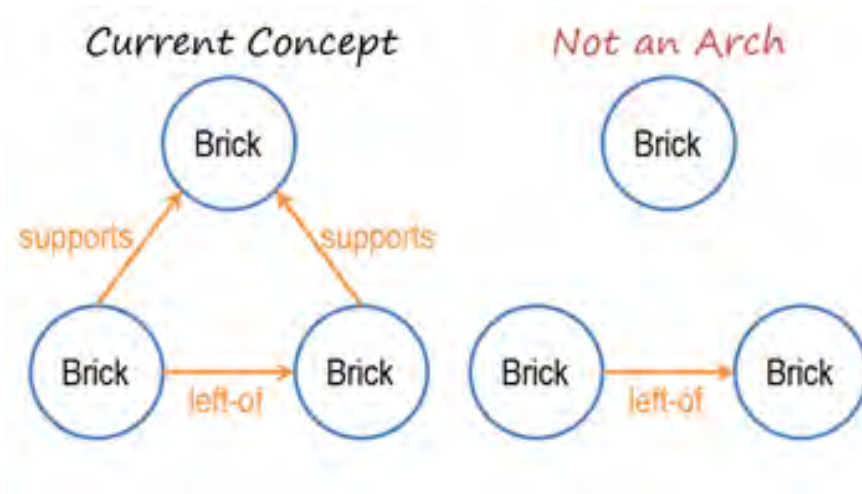


Figure 9-14 Current arch representation (left) and new negative example representation (right)

To achieve this, the AI program must undertake a process of refinement, adding specificity to the relationships between the bricks that constitute an arch. The solution lies in the application of the *require-link* heuristic. This heuristic mandates the presence of certain relationships – specifically, the supportive links between bricks that are fundamental to the arch's structure. These links are no longer optional but become essential components of the arch concept.

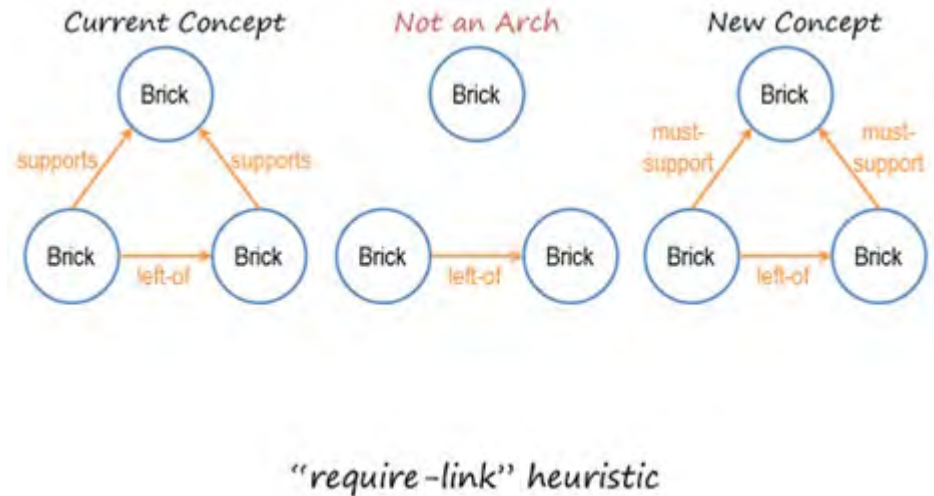


Figure 9-15 Revised arch representation (right) after adding the required links "require-link" (captured as "must-support").

The *require-link* heuristic operates on the principle that while certain elements of the concept and the negative example may overlap, the discrepancies provide an opportunity for refinement. By enforcing additional conditions on the supportive links, the AI program can effectively exclude the negative example from the concept definition. This nuanced approach to refining the concept underscores the dynamic nature of incremental concept learning, where each new example – positive or negative – serves as a catalyst for further conceptual evolution.

Specialization to Exclude Features

In an intriguing twist to the exercise, the teacher introduces a third example where, unlike previous instances, the vertical bricks are in contact with each other. This scenario presents the AI program with a unique challenge: the current concept definition of an arch, characterized by necessary support links, must now be reevaluated in light of this new, negative example.

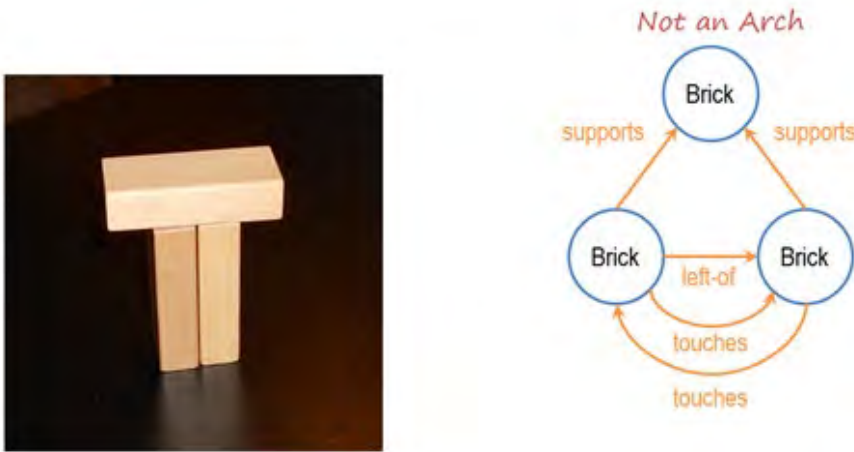


Figure 9-16 9-17 The AI agent is presented with another negative example of an arch (left) with its corresponding representation also shown (right).

The AI program's task is to refine its concept of an arch to ensure that such configurations, where vertical bricks touch, are explicitly excluded.

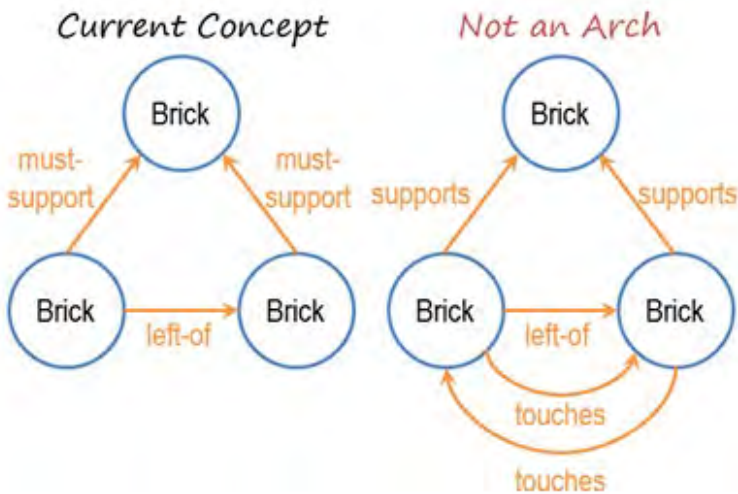


Figure 9-18 Current arch representation (left) and new negative example representation (right)

This leads to the implementation of the “forbid-link” heuristic. Through this heuristic, the AI program enforces a new rule within its concept definition: the two bricks positioned vertically

must not be in contact. The symbol used to represent this prohibition, “ $\neg$,” signifies that the presence of a touching relationship between these bricks is not permissible.

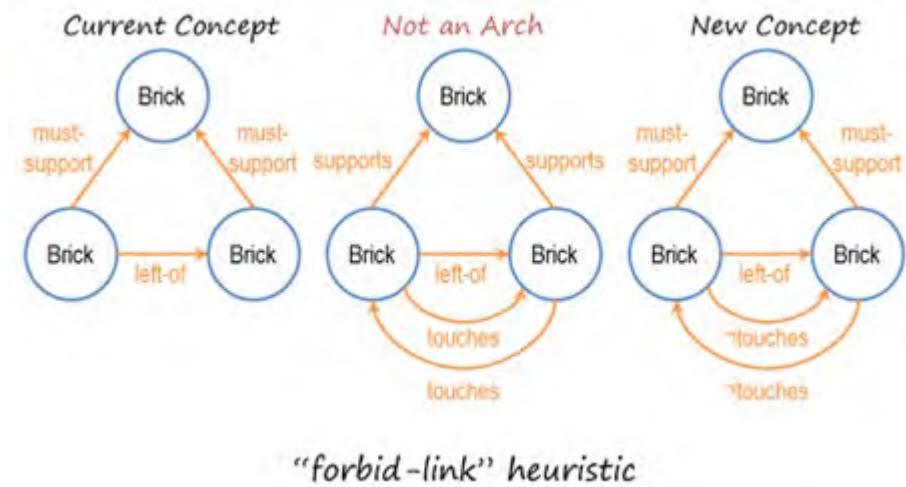


Figure 9-19 Revised arch representation (right) after adding the “forbid-link” heuristic (denoted by the “ $\neg$ ” symbol).

By introducing bidirectional links, the AI program establishes a mutual non-contact requirement between the two bricks, ensuring that neither brick touches the other. This refinement to the concept definition exemplifies the forbid-link heuristic's role in incremental concept learning, where specific relationships or conditions are systematically excluded to maintain the integrity of the concept being learned. Through such meticulous adjustments, the AI program incrementally hones its understanding, catering to the nuanced distinctions between positive and negative examples within the learning process.

Generalization to Abstract Features

As the exploration of incremental concept learning progresses, the AI program is presented with a fourth example that introduces a new element to the concept of an arch – a wedge. This example, confirmed by the teacher as a positive instance, showcases a configuration where two bricks support a wedge on top, diverging from the previous examples that only featured bricks.



Figure 9-20 The AI agent is presented with a positive example of an arch with a wedge top (left) with its corresponding representation also shown (right).

The current concept definition held by the AI program, which includes the essential “not touches” links between the supporting bricks, now faces a challenge. The introduction of a wedge, in addition to bricks, necessitates a revision of the concept to accommodate this broader range of elements. The AI program's response to this challenge is a strategic move towards generalization, specifically employing the “enlarge-set” heuristic.

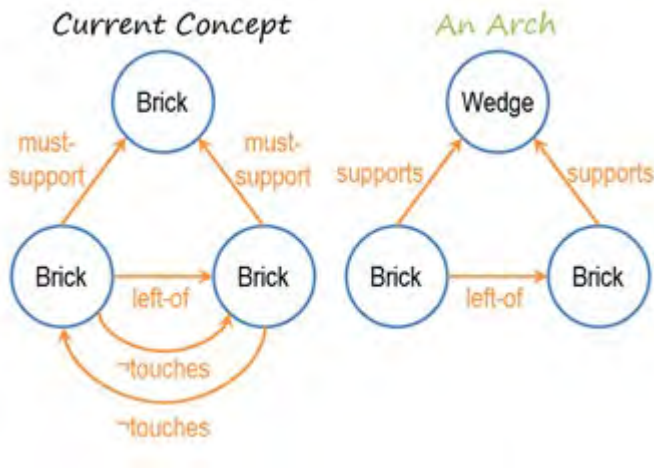


Figure 9-21 Current arch representation (left) and new positive example representation with a wedge top (right).

By broadening the definition to include not just a brick but also a wedge as the top element of the arch, the AI program ensures that this new example aligns with the concept of an arch.

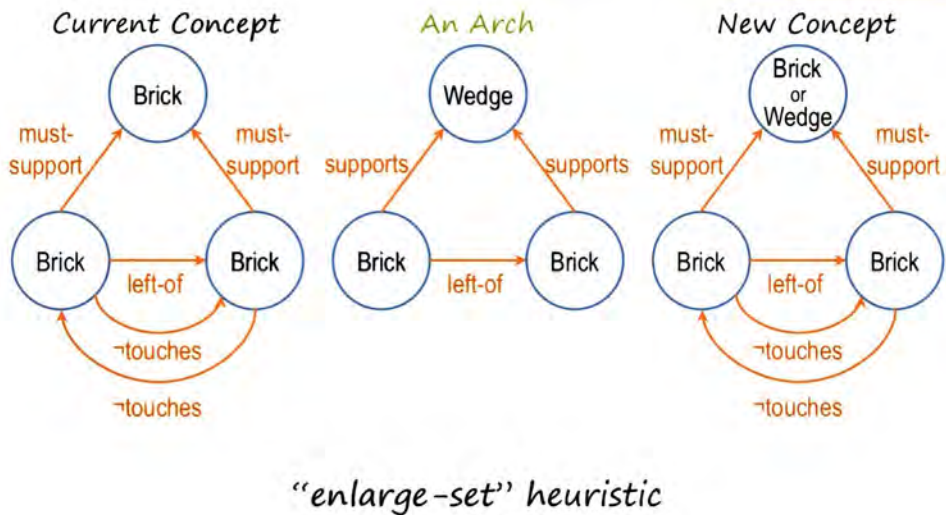


Figure 9-22 Revised arch representation (right) after adding the enlarge-set heuristic.

This enlargement of the set, from exclusively bricks to a combination of bricks or wedges, exemplifies the adaptability inherent in incremental concept learning. It highlights the AI program's capacity to integrate new information into existing concepts through subtle yet significant adjustments. The ‘enlarge-set’ heuristic, therefore, serves as a tool for expanding the concept's boundaries, making it more inclusive of variations while maintaining the core structural relationships that define an arch. This process underscores the dynamic interplay between new examples and the evolving concept definitions, guided by the principles of incremental learning.

Generalization with Background Knowledge

The use of background knowledge is a critical step for AI programs to generalize concepts more effectively.⁹⁵ Imagine an AI program that initially recognizes bricks and wedges as distinct elements in constructing an arch. Upon acquiring background knowledge that categorizes both bricks and wedges as blocks, the AI can revise its concept definition of an arch to include any block at the top, thus broadening the concept's applicability. This method is known as the “climb-tree” heuristic, where the AI “climbs” up a conceptual tree of knowledge to find more general categories that encompass specific instances.

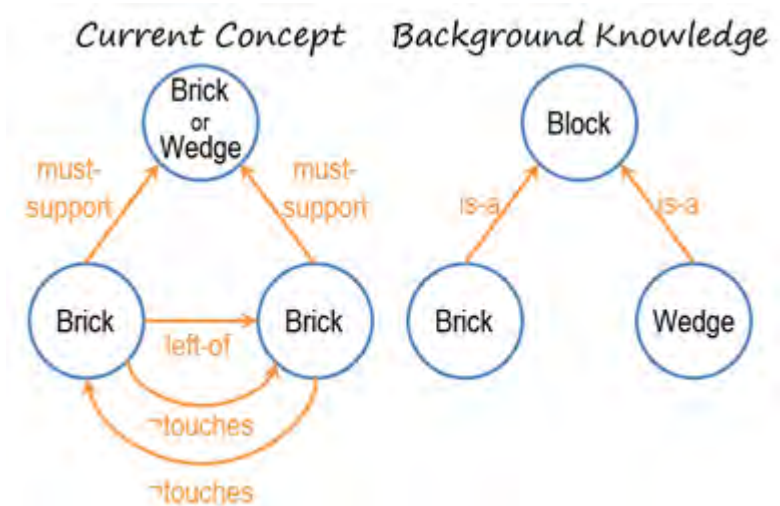


Figure 9-23 Current Arch Representation (left) and background knowledge about types of blocks (right).

This approach not only simplifies the concept definition but also prepares the AI for recognizing new instances that fit the broader category. For instance, if a new example features a cylinder at the top of the arch, the AI, equipped with its generalized concept, would identify this arrangement as a valid instance of an arch. This illustrates the power of background knowledge in expanding an AI's ability to generalize from specific examples to broader categories.⁹⁵

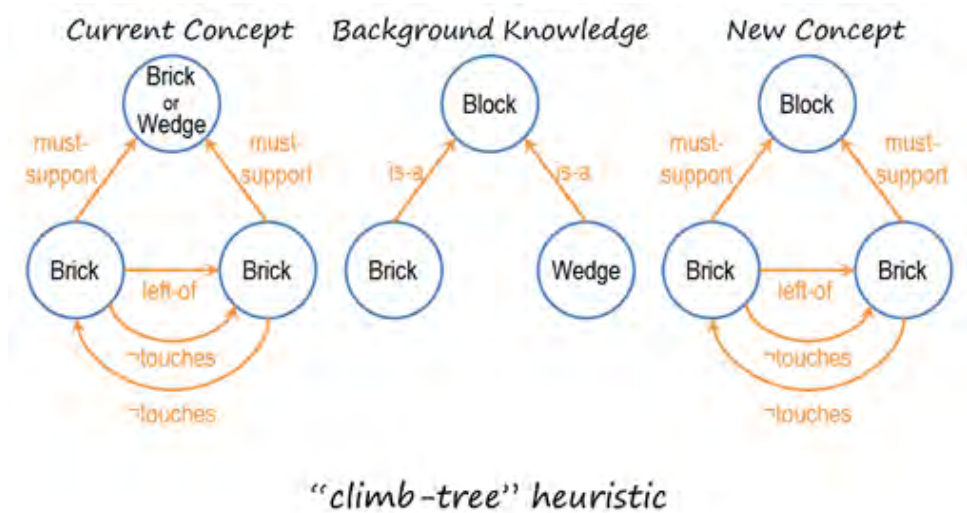


Figure 9-24 Revised arch representation (right) after applying the climb-tree (right) heuristic.

The 'climb-tree' heuristic demonstrates that the more background knowledge an AI possesses, the more adept it becomes at learning and generalizing from new examples. It highlights a fundamental aspect of learning, both artificial and human: the ability to apply knowledge from known categories to novel situations, thus enriching the learning experience and enhancing the adaptability of the learner. This principle underscores the importance of a rich knowledge base in facilitating more nuanced and flexible concept learning.

The process of generalizing from specific instances to broader categories, as demonstrated in the arch example, mirrors a fundamental aspect of human learning. Just as the AI program learned to generalize bricks and wedges to the broader category of blocks, humans often use similar reasoning to understand and categorize the world around them. For instance, upon encountering various objects that serve a similar function, such as bricks and cylinders both being used as supports in structures, a person might abstract these specific instances to the concept of 'blocks' capable of serving that function. This ability to generalize allows for flexibility in thought and application, enabling both AI and humans to apply known concepts to novel situations effectively. When faced with a new configuration, such as an arch with a wedge on top, the application of this generalized knowledge facilitates the recognition of this structure as fitting within the established concept of an arch, despite variations from previously encountered examples. This reflection on human learning highlights the cognitive connection between how AI and humans extrapolate from specific examples to broader understandings, emphasizing the intertwined nature of learning, knowledge application, and concept generalization.

AN EXAMPLE ALTERNATIVE REPRESENTATION (ITERATIVE REFINEMENT)

Understanding the incremental concept learning algorithm is crucial for grasping how AI agents adapt and refine their knowledge base. This can be visualized through a series of iterative adjustments to the concept definitions an AI agent holds. Initially, when an AI agent encounters a positive example, it formulates a concept definition that encompasses this example.



Figure 9-25 Positive example of an AI Agent

However, the learning process becomes more complex when the agent is presented with a negative example that falls under the current concept definition.

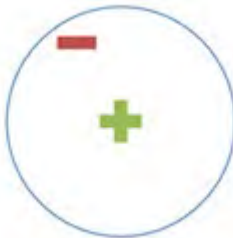


Figure 9-26 Positive and negative examples of an AI Agent

In such cases, the definition must be modified to exclude the negative example while still covering the positive ones. This adjustment process is depicted through the modification of the concept definition to both include new positive examples and exclude new negative ones.



Figure 9-27 AI Agent, Positive example with negative example excluded

The agent continually revises its concept definition, ensuring that it becomes increasingly accurate by including all positive examples and excluding all negative ones.



Figure 9-28 AI Agent illustrating Specialization and Generalization Iterations.

This iterative process may involve numerous cycles of refinement as the agent encounters additional examples (above) until arriving at a refined understanding of the concept (below).

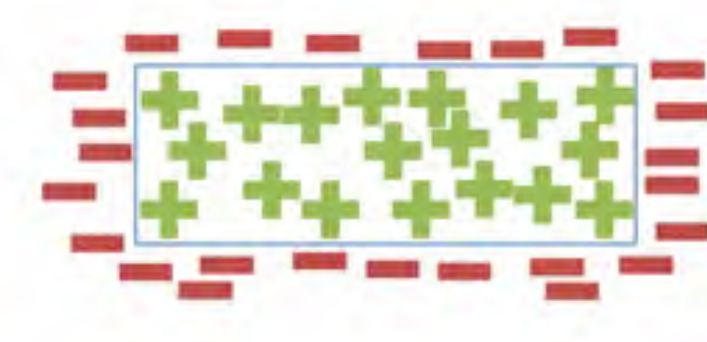


Figure 9-29 AI Agent, Final Concept

The challenge in incremental concept learning lies in navigating the vast potential for generalization and specialization, particularly when only a limited set of examples is available. To efficiently manage this expansive learning space, AI agents rely on heuristics and background knowledge. Heuristics serve as guiding principles that streamline the revision of concept definitions, ensuring that the agent makes appropriate adjustments. Meanwhile, background knowledge enriches the agent's understanding, providing a foundation upon which more nuanced and accurate concept definitions can be built. This combination of heuristics and background knowledge is essential for constraining and directing the learning process within this complex conceptual landscape.

HEURISTICS FOR CONCEPT LEARNING (SUMMARY OF HEURISTICS)

In incremental concept learning, AI agents employ various heuristics to navigate the complexities of generalizing and specializing concepts based on new examples. These heuristics are essential tools that guide AI agents in adjusting their concept definitions efficiently and effectively. Up until now, we've explored five primary heuristics:

- **Require-Link:** This heuristic mandates the inclusion of certain relationships or properties in the concept definition, ensuring that crucial elements are always present for a concept to be valid.
- **Forbid-Link:** Contrary to Require-Link, Forbid-Link excludes specific relationships or properties from the concept definition, ensuring that certain elements do not coexist within a valid concept.

- **Drop-Link:** Used for generalization, Drop-Link involves removing a condition or link from the concept definition to broaden its applicability, allowing it to encompass a wider range of examples.
- **Enlarge-Set:** This heuristic expands the set of elements that can fulfill a role within a concept, such as including additional objects or categories that can be part of the concept definition.
- **Climb-Tree:** Climb-Tree involves utilizing hierarchical background knowledge to generalize a concept further, replacing specific instances or categories with more general ones that encompass a broader range of elements.

Introducing a new heuristic, “Close-Interval”, we look into continuous values and their role in concept learning. This heuristic is particularly relevant when dealing with attributes that vary along a continuum, such as size, weight, or temperature. For example, if an AI agent, or a child in a learning scenario, initially encounters only small-sized examples of a concept (like dogs) and then encounters a significantly larger example, the Close-Interval heuristic would prompt an expansion of the concept's definition to include this wider range of sizes. Unlike the previous heuristics that primarily deal with discrete elements and relationships, Close-Interval addresses the need to accommodate continuous variations within a concept, thereby broadening the agent's ability to generalize and specialize based on the attributes that exhibit continuous variability.

Heuristics for Specializing and Generalizing

require-link: link must be present to be a positive example of the concept

enlarge-set: multiple objects or links may fit one role in the concept

forbid-link: link must be absent to be a positive example of the concept

climb-tree: generalize over multiple objects in the same role based on knowledge

drop-link: link is not necessary to be a positive example of the concept

close-interval: expand range of values to be a positive example of the concept

Figure 9-30 AI Heuristics for Specialization and Generalization.

Example: Re-Identifying “Foo” (Applying Heuristics)

Let’s join Ashok and David as they discuss the concepts of incremental learning.

Ashok David, as we embark on this exercise, we’re venturing into the world of foo, a concept we aim to unravel through a series of examples. Our journey begins with a solitary instance featuring four distinct bricks. Given this singular example, our primary task is to engage in variabilization, transforming specific instances into more generalized forms. David, based on this initial presentation, how would you approach the task of variabilizing these bricks?

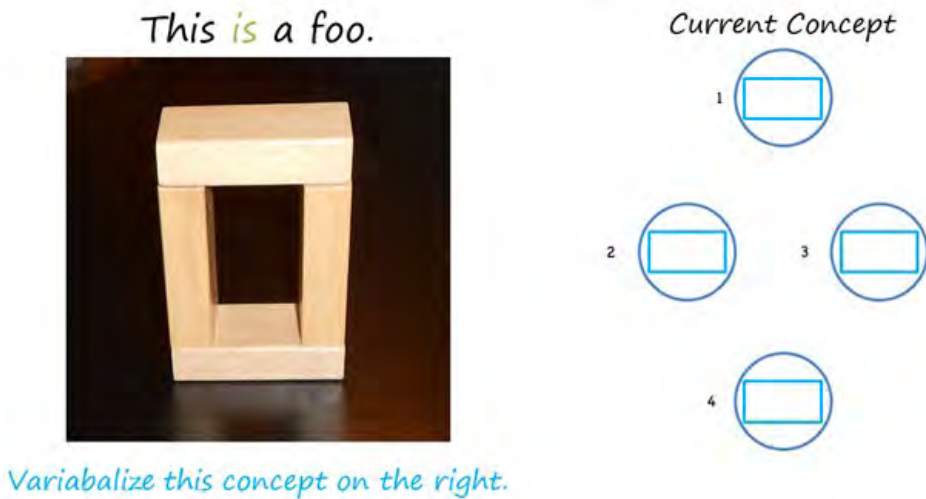


Figure 9-31 Foo, Current Concept using Variabalization

David Ashok, the example at hand presents us with four labeled bricks, each designated as brick one through four. To navigate through the concept of foo, I propose we generalize these specific labels, embracing a broader perspective where each brick is simply an instance of a generic brick. Our concept, thus, revolves around the universality of bricks within the framework of foo.

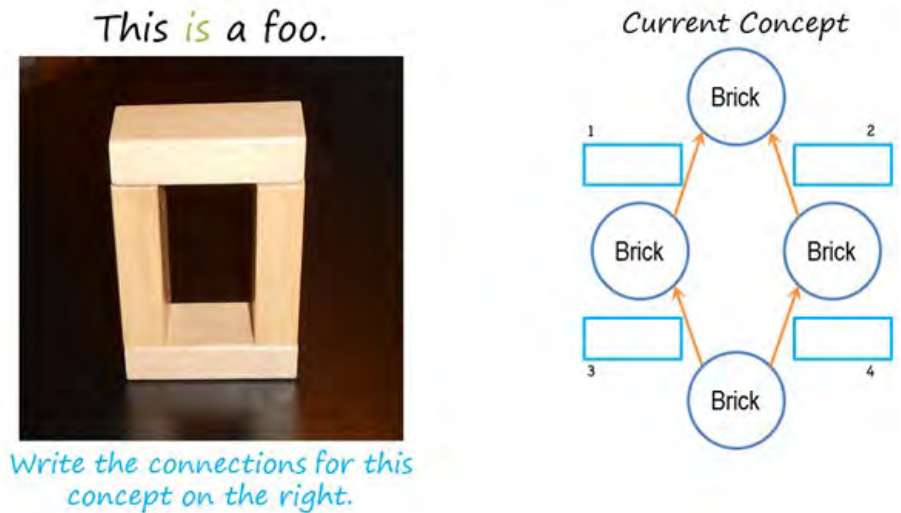


Figure 9-32 Foo, Current Concept with Variabalization without edge labels

- Ashok** That's right, David. Now, how should we describe the relationship with the concepts on the right?
- David** It appears the bottom brick supports the two middle bricks, and those, in turn, support the top brick. We don't have an example that specifies whether these links are obligatory, so they are "supports" not "must support", as seen earlier.

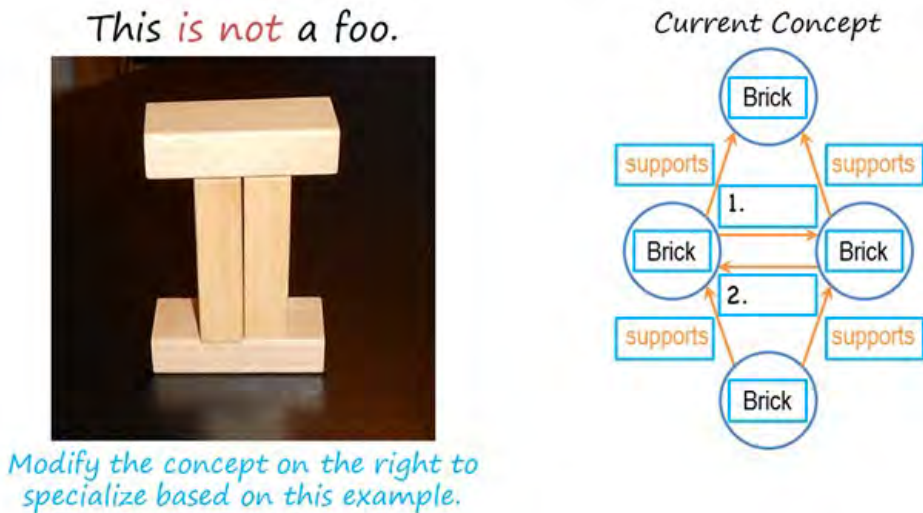


Figure 9-33 Foo, Current Concept Specialization

Ashok Now, imagine the teacher presents this as a second example, labeled as a negative instance of “foo” (above figure, left). How could the agent refine its current concept definition to exclude this negative example?

David Unlike our arches example, the main difference here is that the bricks are touching each other. We'll add a new forbidden link stating the two bricks cannot touch each other.

This *is not* a foo.



Modify the concept on the right to specialize based on this example.

Current Concept

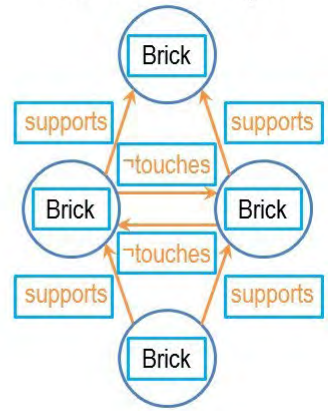


Figure 9-34 New negative example of a foo.

Ashok

Good job, David. Here's the next example, a positive one. How should the current definition be refined?

This *is* a foo.



Modify the concept on the right to generalize based on this example.

Current Concept

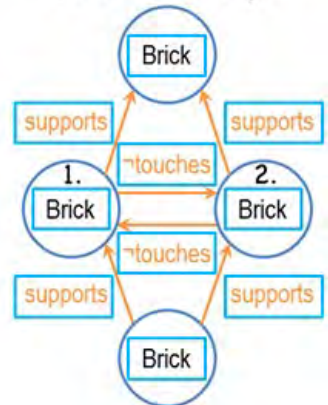


Figure 9-35 Foo, Refinement with a new positive example.

David

We've established that the side bricks can be cylinders. So I've replaced "brick" with "brick or cylinder." Currently, I haven't used the background knowledge that cylinders and bricks are types of blocks.

This is a foo.



Modify the concept on the right to generalize based on this example.

Current Concept

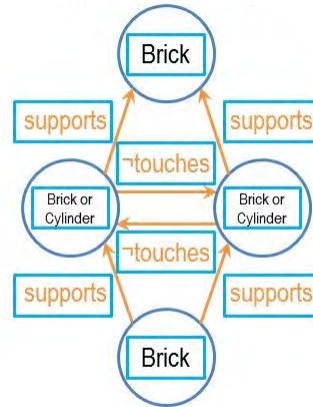


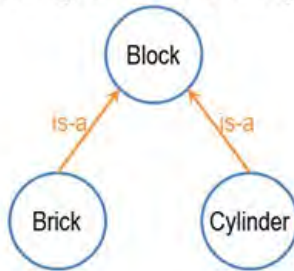
Figure 9-36 Foo, Generalization of Bricks with Bricks or Cylinders.

Ashok

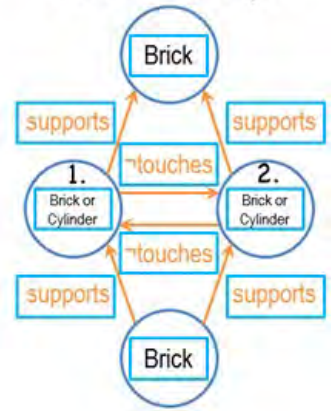
That's accurate, David. While humans have extensive background knowledge, we haven't attributed any to the AI agent yet. So, it might simply identify them as brick or cylinder.

Now, suppose the AI agent has background knowledge that bricks and cylinders are subclasses of blocks. How might the program refine its current concept?

Background Knowledge



Current Concept



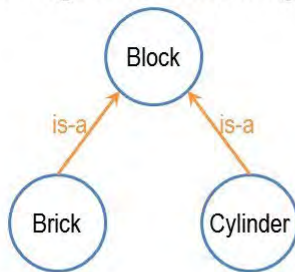
Modify the concept on the right to generalize based on this knowledge.

Figure 9-37 Foo, Incremental Concept Learning using Background Knowledge

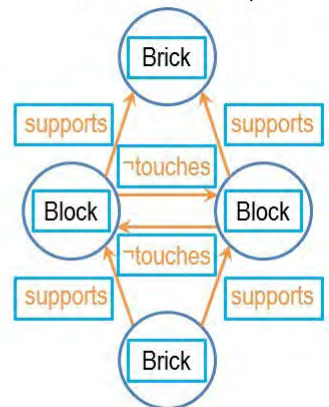
David

I used the “climb-tree” heuristic to generalize “brick or cylinder” to “block”, recognizing that if both can play this role, then “block” can as well.

Background Knowledge



Current Concept



Modify the concept on the right to generalize based on this knowledge.

Figure 9-38 Foo, Incremental concept learning using the climb-tree heuristic.

Ashok Excellent, David. This highlights how knowledge significantly influences learning. This incremental concept method is integral to knowledge-based AI, as the learning outcome depends on the initial knowledge. Consider this final example, labeled negatively. How might the AI agent refine its concept to exclude it?

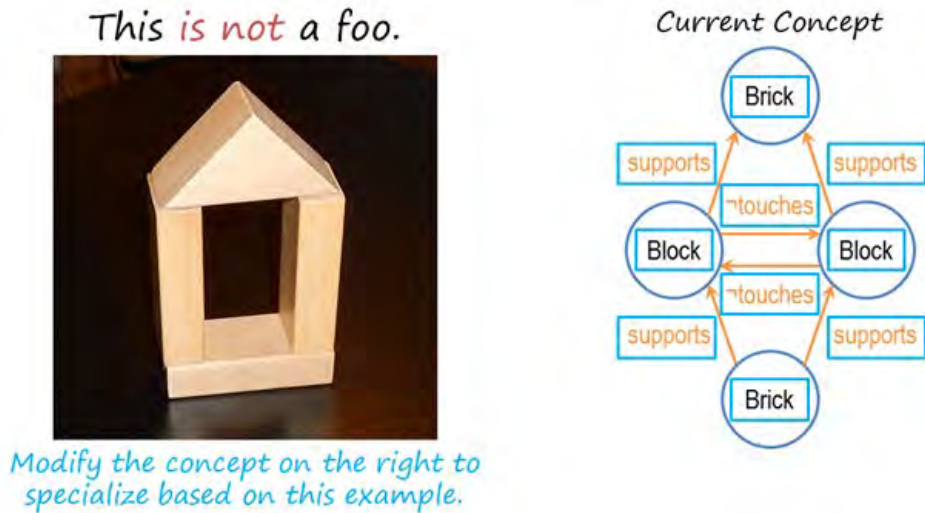


Figure 9-39 Foo, Refining to exclude the negative example

David This example doesn't fit our current "foo" concept, which has a brick on top, not a wedge. Since it already doesn't fit, we don't need to modify our concept.

Ashok Right. If a negative instance is already excluded by the current definition, no change is needed. Based on the examples and background knowledge, this is the AI agent's final "foo" concept. There are no "must-support" links as the examples didn't require them. Also, we didn't further generalize the top and bottom bricks or the side blocks due to lack of relevant background knowledge. The result of learning depends on both the input examples and the AI agent's background knowledge.

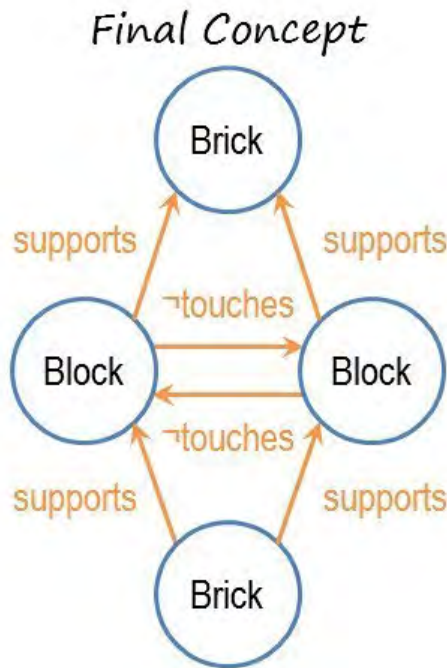


Figure 9-40 The final learned representation for a Foo.

This method of learning, where concepts are developed incrementally from a small set of examples, stands in contrast to traditional machine learning approaches that often rely on vast datasets to begin the learning process. In scenarios where examples are sparse and presented sequentially, the reliance on statistical methods to discern patterns becomes less feasible. Instead, the AI's background knowledge becomes a crucial asset, guiding the learning process and enabling the AI to extrapolate from limited data. This approach to learning, rooted in the principles of incremental concept learning and heavily influenced by the AI's background knowledge, offers a nuanced pathway to understanding and defining concepts in a way that mirrors human cognitive processes.⁹⁴

INCREMENTAL CONCEPT LEARNING AND COGNITION

Incremental Concept Learning stands as a testament to the profound interconnection between artificial intelligence and human cognition, embodying a learning approach that mirrors the gradual, example-by-example way in which humans assimilate and process new information. This method contrasts starkly with the bulk processing of massive datasets often associated with

machine learning, offering a nuanced perspective on learning that aligns more closely with the human experience.⁹⁴

At the heart of Incremental Concept Learning lies the principle that understanding and intelligence build progressively, with each new example serving as a stepping stone towards a more comprehensive grasp of a concept.⁹⁵ This reflects the everyday learning experiences of humans, where knowledge is not typically acquired in vast, sweeping influxes but rather through a series of individual, incremental encounters. Each new piece of information or experience adds to the existing cognitive framework, requiring adjustments, refinements, and sometimes, the redefinition of concepts based on newly acquired insights.

Consider the cognitive underpinnings of Incremental Concept Learning, drawing parallels between the ways AI systems and humans approach the task of learning from examples. It explores the dual perspectives on learning: one that involves sifting through a plethora of examples to discern patterns and another, more akin to human learning, where concepts are built and refined one example at a time. This incremental approach not only highlights the adaptability and flexibility required in learning but also underscores the importance of background knowledge and context in shaping understanding.

In essence, Incremental Concept Learning serves as a bridge between the computational methods of AI and the intricate, nuanced processes of human cognition. It underscores the potential for AI systems to not just learn but to do so in a manner that reflects the depth, adaptability, and incremental nature of human learning. By drawing from and contributing to our understanding of cognitive processes, Incremental Concept Learning enriches both the field of AI and our insights into the workings of the human mind.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

Incremental Concept Learning (ICL), with its focus on refining concepts from a sequence of examples, often using symbolic representations and background knowledge, has interesting connections and contrasts with modern AI paradigms:

- **Machine Learning (General):**
 - **Supervised Learning:** ICL is a form of supervised learning, as it relies on labeled positive and negative examples from a “teacher.” Classical ML algorithms like decision trees or Support Vector Machines (SVMs) also learn from labeled data, but typically in a batch fashion from larger datasets, whereas ICL emphasizes the one-by-one example processing and explicit concept representation.
 - **Online Learning:** ICL is inherently an online learning method, where the model updates its concept definition as each new example arrives. This contrasts with batch learning but aligns with many real-world scenarios where data streams in and concepts may evolve.

- **Inductive Logic Programming (ILP):** ILP is a subfield of ML that learns logical rules (similar to the symbolic concept descriptions in ICL, like the arch example) from examples and background knowledge, often expressed in first-order logic. There's a strong conceptual overlap here in terms of learning relational concepts.
- **Deep Learning:**
 - **Few-Shot & Zero-Shot Learning:** While ICL traditionally uses a small set of examples, deep learning often requires vast datasets. However, areas like few-shot learning in deep learning aim to learn concepts from very few examples, sometimes by leveraging pre-trained models that have acquired extensive “background knowledge” from large, diverse datasets. This has a conceptual parallel to ICL's use of explicit background knowledge (e.g., the “climb-tree” heuristic).
 - **Continual Learning / Lifelong Learning:** Deep learning models often suffer from “catastrophic forgetting” when trained sequentially on new tasks or evolving concepts. ICL, by explicitly refining concept definitions through generalization and specialization, aims to avoid this abrupt forgetting. Research in continual learning for deep networks seeks to enable models to learn new concepts incrementally without forgetting old ones, a challenge ICL addresses structurally.
 - **Neuro-Symbolic AI:** Combining ICL's symbolic concept refinement with the perceptual power of deep neural networks is a key direction in neuro-symbolic AI. A neural network might extract features or propose initial symbolic descriptions from raw data (like identifying objects and their basic properties in an image), which are then refined by an ICL-like process based on further examples and explicit rules.
- **Reinforcement Learning (RL):**
 - While RL is primarily about learning policies (what actions to take), an agent's understanding of its environment and the “concepts” within it (e.g., what constitutes a “safe state,” a “useful tool,” or a “threat”) can be learned or refined incrementally based on its experiences (which act as examples). Model-based RL might involve incrementally learning a symbolic model of the world's dynamics, which is a form of concept learning about cause and effect.
- **Natural Language Processing (NLP) & Generative AI (GenAI):**
 - **Concept Acquisition from Text:** LLMs implicitly learn a vast number of concepts from their training data. While not strictly ICL, the way they can sometimes generalize from a few examples in a prompt (in-context learning) has a surface similarity to learning from examples.

- **Interactive Learning with LLMs:** One could envision an interactive system where an LLM proposes a concept definition or a rule, and a human provides positive/negative examples, leading the LLM to refine its understanding incrementally, guided by ICL principles. This could make LLM knowledge more precise and controllable.
- **Structured Knowledge Extraction:** ICL principles could be applied to refine rules or templates for extracting structured information (concepts and their relationships) from unstructured text generated by GenAI or processed by other NLP systems.
- **Agentic AI:**
 - For an AI agent to adapt to new situations and learn about its environment or tasks over time, incremental learning of concepts is crucial. As an agent encounters new objects, situations, or outcomes of its actions, it needs to update its internal conceptual models. ICL provides a framework for how these conceptual updates might occur, allowing the agent to build a more accurate and useful understanding of its world.

ICL's emphasis on explicit concept representation, the use of background knowledge, and the incremental refinement process offer valuable perspectives for building more adaptive, transparent, and knowledge-rich AI systems, often in synergy with data-driven approaches from modern machine learning.

CHAPTER SUMMARY

This chapter explored Incremental Concept Learning (ICL), a key KBAI methodology where AI systems learn and refine concepts from a sequence of examples, one at a time. This approach mirrors human learning, where understanding often builds progressively rather than from analyzing large datasets simultaneously. We began by illustrating ICL with the “foo” example, demonstrating how an agent, guided by positive and negative instances, iteratively adjusts its concept definition.

A crucial element highlighted was the role of background knowledge, which significantly influences how an agent interprets examples and generalizes or specializes its concepts. The chapter detailed the core ICL algorithm, which involves generalizing the current concept definition when a new positive example is not covered, and specializing it when a negative example is incorrectly included.

We examined several key processes and heuristics that drive ICL:

- **Variabilization:** Abstracting specific entities in an example into more general variables.
- **Generalization Heuristics:** Such as “drop-link” (removing a condition to broaden the concept), “enlarge-set” (expanding the types of objects that fit a role), and “climb-tree” (using background knowledge hierarchies to move to more general categories).

- **Specialization Heuristics:** Like “require-link” (mandating a specific relationship) and “forbid-link” (prohibiting a relationship).
- The **Close-Interval** heuristic was introduced for handling continuous-valued attributes.

The iterative nature of ICL, applying these heuristics to refine concepts like “arch” and “foo,” was demonstrated through examples and dialogues. We emphasized that the final learned concept depends on both the sequence of examples and the agent's available background knowledge.

The cognitive connection of ICL was underscored, noting its similarity to human incremental learning and the importance of context and prior knowledge. Finally, we discussed the relationship between ICL and modern AI paradigms. ICL's principles of supervised, online learning, its use of background knowledge, and its explicit concept representation find parallels or offer complementary strengths to areas like few-shot and continual learning in Deep Learning, Inductive Logic Programming, model learning in RL, interactive learning with LLMs, and adaptive learning in Agentic AI. ICL provides a valuable framework for understanding how AI can build conceptual understanding incrementally, a vital capability for truly adaptive intelligent systems.

ADDITIONAL READINGS & REFERENCES

- 94 Bichindaritz, I. in *UK Workshop on Case-Based Reasoning*. 91-106 (Springer).
- 95 Gennari, J. H., Langley, P. & Fisher, D. Models of incremental concept formation. *Artificial intelligence* **40**, 11-61 (1989).
- 96 Winston, P. *Artificial Intelligence*. (Pearson, 1992).
- 97 Shi, Y., Mi, Y., Li, J. & Liu, W. Concept-cognitive learning model for incremental concept learning. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* **51**, 809-821 (2018).
- 98 Fisher, D. H. Knowledge acquisition via incremental conceptual clustering. *Machine learning* **2**, 139-172 (1987).

Chapter 10

Classification

INTRODUCTION TO CLASSIFICATION

In this chapter, we examine the fundamental problem of classification in artificial intelligence. Classification involves mapping diverse percepts from the world into distinct equivalence classes, enabling us to efficiently navigate and act within our environment. This chapter builds on the principles of incremental concept learning, exploring how we can abstract complex information into organized hierarchies of concepts.

We will look into the varied nature of these equivalence classes and their organization into conceptual hierarchies, discussing different types of concepts such as axiomatic and prototypical.⁹⁹ The chapter also introduces the methodologies for classification, encompassing both bottom-up and top-down approaches, and how these strategies aid in the efficient processing and understanding of information. Through this exploration, we aim to provide a comprehensive overview of classification in AI and its pivotal role in the interpretation and interaction with the world around us.

REVISITING CONCEPT LEARNING: THE FOUNDATION FOR CLASSIFICATION

In the previous chapter, we looked into techniques for deriving concepts from limited examples, focusing on relatively straightforward concepts such as the hypothetical “foo”. However, real-world concepts often present a higher level of complexity. To illustrate, let's consider a scenario involving eight animals, each depicted in a charming manner.

Which of these are birds?

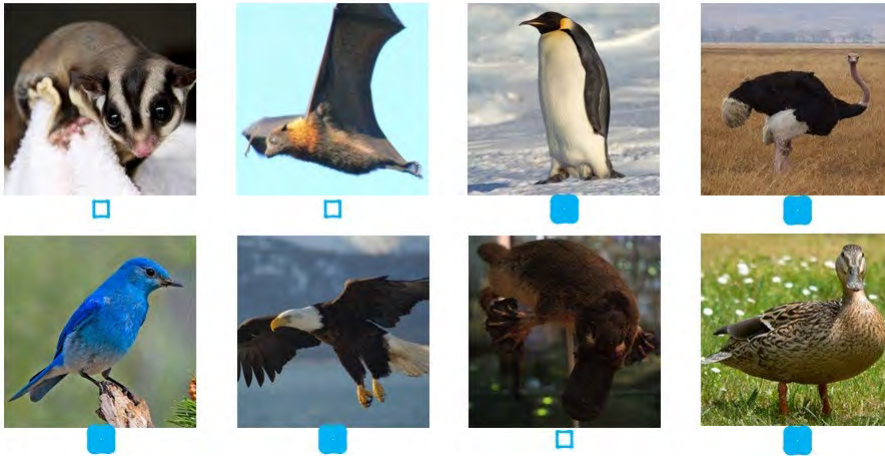


Figure 10-1 Animals that might be birds.

The question at hand is: how many of these animals would you classify as birds? This inquiry sets the stage for a deeper exploration of concept learning in the area of more intricate and diverse real-world examples. Let's join Ashok and David as they explore this further.

Ashok David, among these animals, which would you identify as birds, and could you share your reasoning behind each choice?

David I identified five of these as birds because they all possess wings, feathers, and I understand they lay eggs, which are typical bird characteristics. The remaining three present interesting cases. For example, the sugar glider resembles bird-like flight but lacks other avian features, such as feathers, and instead has fur. Similarly, while bats have wings, their lack of feathers and the fact they bear live young differentiate them from birds. The platypus, despite laying eggs and having a bill, also diverges from bird criteria due to its lack of feathers. Therefore, despite

variations in flight capabilities, as seen with penguins and ostriches, these five fulfill the essential bird criteria.

Ashok Excellent analysis, David. Your thoughtful approach to distinguishing birds based on specific criteria is appreciated.

The exercise demonstrated the ability to classify a group of animals into birds, discerning which animals fit into the bird category and which did not. The criteria used to make these distinctions were based on an understanding of typical bird characteristics. These criteria encompassed features that are generally expected of birds, such as wings, feathers, and the ability to lay eggs. The approach involved identifying fundamental conditions that an animal must meet to be classified as a bird. Animals that did not meet these essential criteria were not considered birds, illustrating the importance of specific features in the classification process.

DEFINING CLASSIFICATION: THE CORE TASK

Classification, in the context of both artificial intelligence and cognitive science, involves the process of categorizing objects, concepts, or entities based on shared characteristics or features. It is a fundamental aspect of human cognition and AI, enabling the efficient processing of information by grouping similar items together. This allows for more effective decision-making, problem-solving, and action planning.

In the exercise involving the identification of birds, several features were scrutinized to determine whether each animal fit within the bird category. The examination included observable characteristics such as the presence of wings, feathers, and a beak. These features are commonly associated with birds and serve as criteria for classification. While some of these features were explicitly mentioned, others were inferred based on general knowledge of avian attributes.

Classification is a pervasive activity not only in human thought processes but also in AI systems. It is essential because it simplifies the vast complexity of the world around us into more manageable and understandable categories. By classifying objects or entities, both humans and AI can make more informed decisions, predict outcomes, and interact with the environment more effectively. This ubiquitous nature of classification underscores its importance in organizing knowledge and guiding behavior in both natural and artificial systems.

The Fundamental Challenge of Classification

Classification stands as a cornerstone of artificial intelligence, serving as the process through which a myriad of percepts encountered in the world are organized into equivalence classes. This organization enables efficient action selection, a fundamental aspect of intelligent behavior. Within the context of a cognitive system, be it an AI agent or a human being, classification plays a pivotal role in navigating the complexities of the environment.

Consider a cognitive system's encounter with various animals at a zoo. This system is bombarded with numerous percepts – raw sensory inputs like the presence of wings, feathers, talons, or a beak. These percepts (which readers might also think of as initially observed attributes or features) form the basis of the system's understanding of the world. Based on these percepts, the system must decide on a course of action, which could range from approaching a friendly animal to avoiding a dangerous one. The term “percept” is often used in AI and cognitive science to emphasize the agent's process of sensing and interpreting its environment. The challenge lies in the vast number of possible percept combinations and the corresponding actions that could be taken.

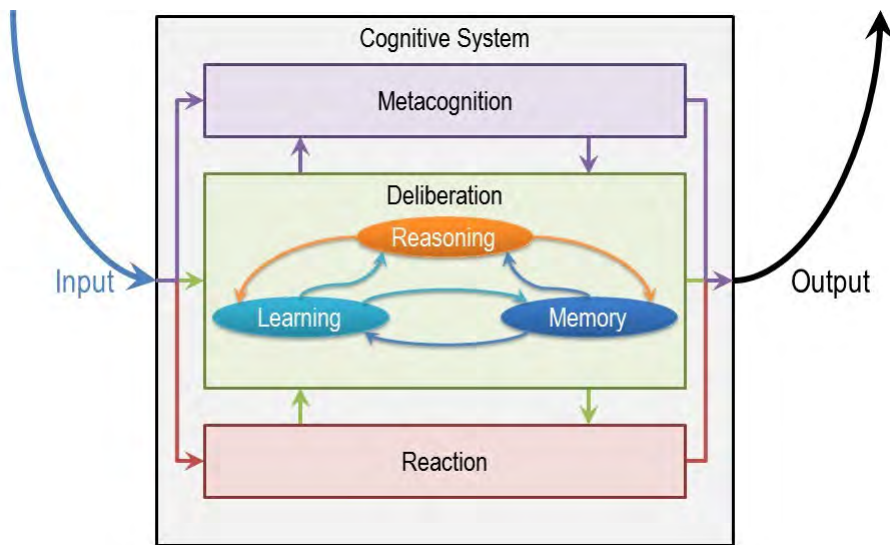


Figure 10-2 Cognitive Systems

The complexity of this mapping from percepts to actions is immense, especially considering the exponential growth in the number of combinations with each additional percept. For instance, with just ten percepts, there are already over a thousand possible combinations. This complexity escalates further with more percepts, quickly becoming unmanageable.

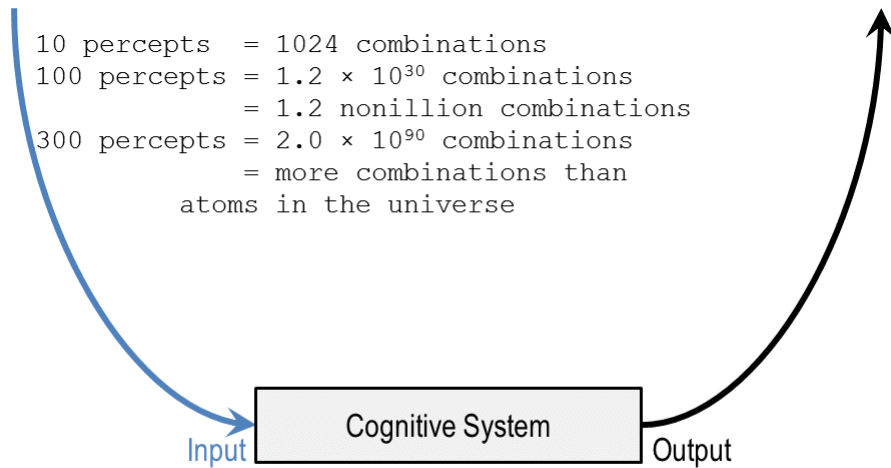


Figure 10-3 Percept growth leads to exponential combinations.

This is where the power of classification becomes evident. By grouping percepts into equivalence classes or concepts, a cognitive system can simplify this complex mapping. Actions are no longer indexed by an unwieldy array of individual percepts but by these broader equivalence classes. This approach not only makes action selection more manageable but also more efficient, enabling intelligent agents to operate effectively in real-time, despite the inherent complexity of their environments. Thus, classification underpins the cognitive architecture of intelligent agents, facilitating the crucial task of action selection amidst the vast and varied percepts of the world. It underscores the essence of intelligence as the ability to select appropriate actions based on a structured understanding of the environment, achieved through the systematic classification of percepts into meaningful concepts.

In navigating complex environments teeming with a multitude of percepts and potential actions, intelligent agents can achieve efficiency through an elegant method of classification. Imagine a scenario where an exponential number of percepts, symbolized as 2^N , are distilled into a significantly smaller set of K concepts. These concepts serve as equivalence classes, encapsulating vast combinations of percepts into more manageable groupings.

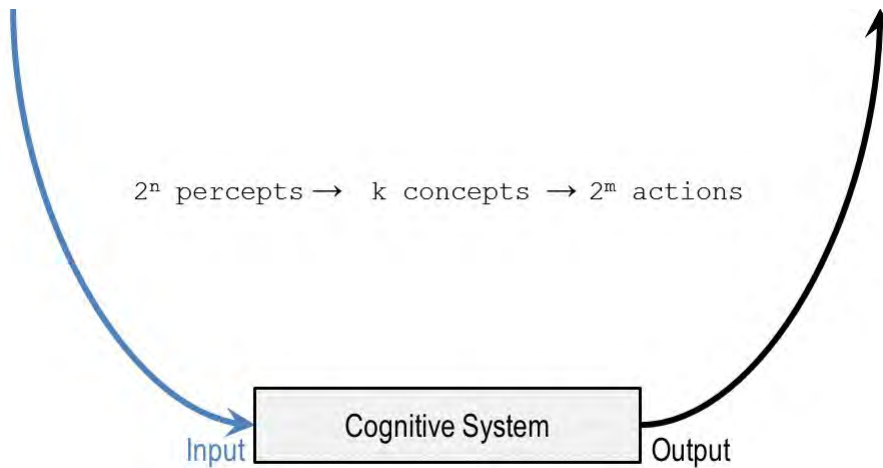


Figure 10-4 Using Classification to map percepts into a smaller number of concepts and actions.

Rather than basing actions on the direct and overwhelming array of percepts, actions are instead associated with these equivalence classes or concepts. This approach is not just theoretical but is applied in everyday scenarios. For instance, when visiting a doctor, a patient presents with certain signs and symptoms. The doctor's decision-making, such as prescribing a treatment, is not hinged on the raw symptoms alone but on a conceptual understanding of the underlying condition, significantly reducing the complexity of potential diagnoses. A similar principle applies in computer programming. A programmer faces numerous error messages (percepts) indicating various issues within the program. Rather than addressing each error in isolation, errors are categorized into broader conceptual problems, such as uninitialized variables, enabling a more systematic and efficient troubleshooting process.

The necessity of classification in artificial intelligence is underscored by its widespread study across different AI disciplines. Without the abstraction offered by concepts, an intelligent agent's decision-making process would resemble an unwieldy table with rows for every possible percept combination and columns for every possible action, an impractical and unfeasible model. Classification, therefore, is not merely a tool for simplification but a fundamental aspect of intelligence, transforming intricate challenges into a series of smaller, more tractable problems through the strategic application of knowledge.

EQUIVALENCE CLASSES: GROUPING PERCEPTS

The concept of “equivalence classes” plays a pivotal role in classification, particularly when it comes to organizing and understanding the vast array of objects or elements we encounter. An

Draft -Version:6/2/2025 3:06 PM

equivalence class groups items based on shared characteristics or percepts, allowing for a more structured and insightful analysis of their properties. To illustrate this concept, let's consider a practical example involving a set of animals: an eagle, a bluebird, and a penguin. Each of these animals can be characterized by a specific set of percepts, such as the ability to lay eggs, the presence of wings, or the possession of talons, among others. The task at hand is to determine a suitable equivalence class that encompasses these three animals, necessitating an evaluation of the pertinent percepts applicable to each. The process begins by identifying the crucial percepts that define the essence of these animals. For the eagle, bluebird, and penguin, we consider attributes like laying eggs, having wings, and having talons. The next step involves determining the actual values of these percepts for each animal, leveraging our collective background knowledge to make informed assessments. By meticulously analyzing the percepts and their values, we can establish an equivalence class that aptly categorizes these animals, highlighting their commonalities and distinctions. This methodical approach not only simplifies the complexity inherent in the natural world but also enhances our ability to make sense of the diverse entities within it.

Let's rejoin Ashok and David as they continue their conversation.

Ashok David, are you familiar with these three animals? Could you tackle this question?

For each of these three animals, choose the value for each percept that applies to that animal.

Eagle	Bluebird	Penguin
Lays eggs? ☐ Yes ☐ No ☐ Maybe	Lays eggs? ☐ Yes ☐ No ☐ Maybe	Lays eggs? ☐ Yes ☐ No ☐ Maybe
Has wings? ☐ Yes ☐ No ☐ Maybe	Has wings? ☐ Yes ☐ No ☐ Maybe	Has wings? ☐ Yes ☐ No ☐ Maybe
Has talons? ☐ Yes ☐ No ☐ Maybe	Has talons? ☐ Yes ☐ No ☐ Maybe	Has talons? ☐ Yes ☐ No ☐ Maybe
Flies? ☐ Yes ☐ No ☐ Maybe	Flies? ☐ Yes ☐ No ☐ Maybe	Flies? ☐ Yes ☐ No ☐ Maybe
Has fur? ☐ Yes ☐ No ☐ Maybe	Has fur? ☐ Yes ☐ No ☐ Maybe	Has fur? ☐ Yes ☐ No ☐ Maybe
Large? ☐ Yes ☐ No ☐ Maybe	Large? ☐ Yes ☐ No ☐ Maybe	Large? ☐ Yes ☐ No ☐ Maybe

Figure 10-5 Example of potential equivalence classes for birds.

David Well, I might have consulted Wikipedia to get a clearer picture, but it's common knowledge that eagles, bluebirds, and penguins all lay eggs and possess wings. Eagles and bluebirds are equipped with talons, contrasting with penguins, which

have flippers tailored for swimming, thus lacking talons. While eagles and bluebirds are capable of flight, penguins are not. Interestingly, none of these birds have fur. In terms of size, eagles and penguins are notably larger, whereas bluebirds are comparatively smaller. The concept of size can be quite subjective, but it's fair to categorize bluebirds on the smaller side.

Ashok Impressive, David. It seems you're quite informed about these animals.

Given a series of examples, such as the three animals discussed earlier, we revisit the principles of “incremental concept learning” from the previous chapter. These techniques allow us to derive equivalence classes and develop concept definitions based on the animals. However, the focus in this context extends beyond merely learning the concept. It examines the essence of these concepts, their structure, and how they interconnect and relate to each other within a broader framework. This exploration into the nature and organization of concepts is central to grasping the complexities involved in classification.

STRUCTURING KNOWLEDGE: CONCEPTUAL HIERARCHIES

In classification, the organization of concepts and their interrelationships play a crucial role, as previously discussed in the context of background knowledge during our exploration of incremental concept learning. For instance, consider how a brick and a wedge were categorized as subclasses of a broader category: blocks. This example only scratches the surface of how intricate and layered the organization of concepts can be.

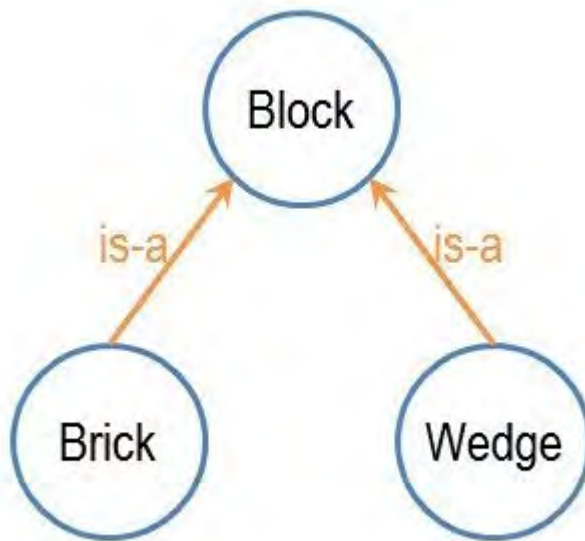


Figure 10-6 Concept Hierarchy using blocks.

Take, for example, the classification of animals, a concept familiar to most. Animals are broadly divided into vertebrates and invertebrates. Vertebrates, in turn, are categorized into various subclasses such as reptiles, birds, and mammals. Within birds, we find further subdivisions, such as eagles, bluebirds, and penguins, each with its distinct characteristics. This hierarchical arrangement of classes, sub-classes, and super-classes showcases the complexity of conceptual organization.

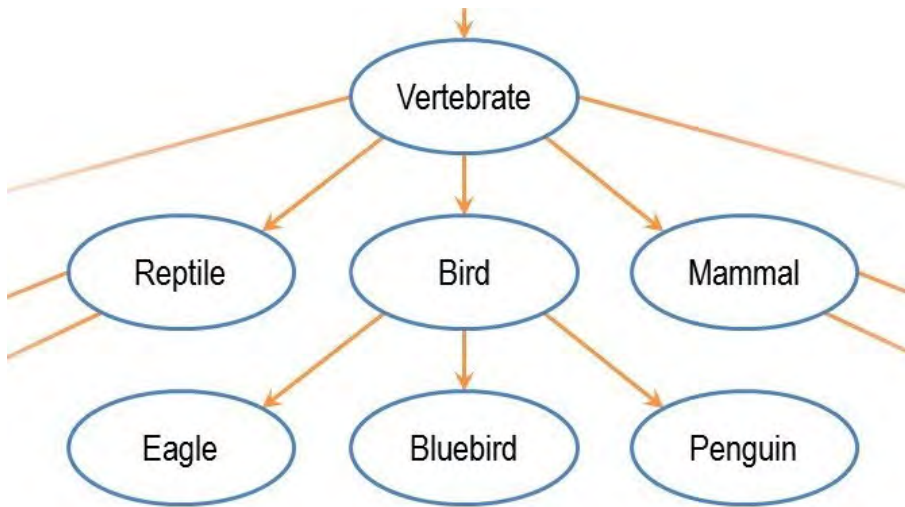


Figure 10-7 Concept Hierarchy using vertebrates.

The hierarchical organization allows for a systematic approach to classification, often starting from a broad category and narrowing down through a series of refinements. For instance, when identifying an animal, one might start by determining whether it is a vertebrate. If it is, the next step might be to ascertain whether it belongs to the bird category. From there, one could determine whether it is an eagle, a bluebird, or a penguin. This top-down approach of establishing a category and then refining it is not only intuitive but also efficient.

This methodical process, while seemingly instantaneous in human cognition, requires explicit definition when programming AI agents. The discovery of a new species by a scientist exemplifies this approach: the scientist must start at the top of the hierarchy and work down to find the appropriate category for the new species, asking questions like whether it is a vertebrate and, if so, whether it fits within the existing subclasses. The “establish and refine” approach also aids in focusing on relevant variables for classification. While the size of an animal might not distinguish between reptiles, birds, and mammals, it becomes a significant factor when differentiating between types of birds, such as eagles and bluebirds. Thus, the hierarchical organization not only structures knowledge but also directs the classification process, highlighting the crucial variables at each level. In essence, the hierarchical organization of concepts is not just a repository of knowledge but a dynamic framework that guides the process of classification.¹⁰⁰ It embodies the adage that both knowledge and its organization are forms of power, providing a structured pathway for navigating complex information spaces and making informed decisions.

Illustrative Example: Conceptual Hierarchies for Birds

Revisiting our previous exercise involving the eagle, bluebird, and penguin, we explored various features unique to each of these birds. Considering these distinctive sets of features and recognizing 'bird' as the encompassing superclass for these species, we're faced with a thought-provoking question: What common features would we assign to the 'bird' category within our classification hierarchy?

How would you characterize the class 'bird' given the characterization of its subclasses below?

Bird		
Lays eggs?	Has talons?	Has fur?
☐ Yes ☐ No ☐ Maybe	☐ Yes ☐ No ☐ Maybe	☐ Yes ☐ No ☐ Maybe
Has wings?	Flies?	Large?
☐ Yes ☐ No ☐ Maybe	☐ Yes ☐ No ☐ Maybe	☐ Yes ☐ No ☐ Maybe

Eagle	Bluebird	Penguin
Lays eggs? Yes	Lays eggs? Yes	Lays eggs? Yes
Has wings? Yes	Has wings? Yes	Has wings? Yes
Has talons? Yes	Has talons? Yes	Has talons? No
Flies? Yes	Flies? Yes	Flies? No
Has fur? No	Has fur? No	Has fur? No
Large? Yes	Large? No	Large? Yes

Figure 10-8 Example conceptual hierarchies for birds.

This question sets the stage for a deeper investigation into the nature of categorization and the principles that underlie the classification of entities into broader groups. It invites us to consider not only the observable characteristics of individual members but also the underlying attributes that unify them under a single, more general class. This process of abstraction and generalization is central to our understanding of classification and the construction of knowledge hierarchies.

Ashok David, what's your perspective on this?

David I took a close look at the shared characteristics among the three birds. They all lay eggs, which leads me to believe that laying eggs is a defining trait of birds. Similarly, they all possess wings, reinforcing the idea that wings are a fundamental bird attribute. Another observation is that none exhibit fur, suggesting birds typically have feathers instead. The presence of talons varies, with eagles and bluebirds equipped with them, but not penguins. Size also isn't a consistent factor, as eagles

and penguins are considerably larger than bluebirds. Therefore, the commonalities of laying eggs, having wings, and lacking fur seem essential to the bird classification.

Ashok Well-reasoned, David. It's important to highlight, though, that the method of deducing superclass features from subclass commonalities mainly applies to concepts with a more formalized structure. This approach might not extend universally across all concept types, something we'll look into shortly.

A SPECTRUM OF CONCEPTS: FROM AXIOMATIC TO EXEMPLAR

Concepts within human cognition and artificial intelligence can be envisioned on a continuum, ranging from highly formalized constructs to those that defy precise definition. At one end of this spectrum, we find axiomatic concepts, characterized by their strict logical parameters, which outline the necessary and sufficient conditions for their application. These are the concepts that we can neatly encapsulate within defined boundaries, such as those prevalent in mathematical theories.

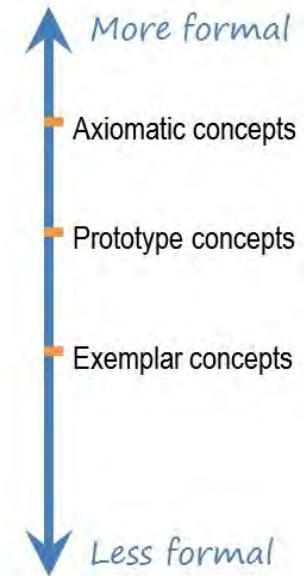


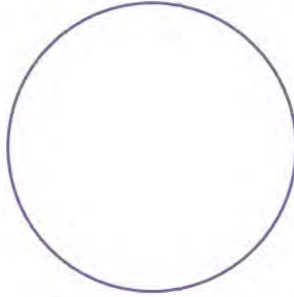
Figure 10-9 Types of Concepts.

Progressing along the spectrum, we encounter prototype concepts. Unlike their axiomatic counterparts, prototype concepts lack rigid defining conditions, instead embodying typical characteristics that may not be universally applicable. These are the everyday categories we navigate with relative ease, relying on a common understanding of shared attributes, even though exceptions and variations abound. At the far end of the spectrum lie exemplar concepts, which resist straightforward categorization altogether. These concepts, such as beauty or freedom, are understood more through instances and experiences than through any concrete criteria. They are inherently subjective and often deeply personal, making them the most challenging to communicate and, particularly, to encode within artificial intelligence systems. This gradient from axiomatic to exemplar concepts illustrates not only the diversity of human cognition but also the varying degrees of difficulty in translating human understanding into computational models. While axiomatic concepts lend themselves to clear programming and teaching within AI systems, prototype and exemplar concepts present increasing challenges, reflecting the intricate interplay between structure, flexibility, and abstraction in human thought and language.

Axiomatic Concepts: Necessary and Sufficient Conditions

Axiomatic concepts stand as the bedrock of formal logic and precision in various fields, particularly in mathematics and geometry. These concepts are meticulously defined by a set of necessary and sufficient conditions, ensuring clarity and unambiguity. Geometric shapes offer quintessential examples of axiomatic concepts. Take a circle, for instance, which is defined with mathematical rigor as the locus of all points in a plane equidistant from a central point. This definition encapsulates the essence of an axiomatic concept; it is precise, it is definitive, and it leaves no room for ambiguity. Any given object can be evaluated against this stringent criterion to determine if it qualifies as a circle. This level of formal definition is what characterizes axiomatic concepts, making them fundamental to the logical structuring of knowledge in various disciplines.

Axiomatic concepts:
Concepts defined by a formal set of necessary and sufficient conditions.



Circle: all points in a plane that are equidistant from a single point.

Figure 10-10 Axiomatic Concepts are defined by the set of necessary and sufficient conditions.

Prototypical Concepts: Typicality and Exceptions

Moving beyond the rigid confines of axiomatic concepts, we encounter the more fluid domain of prototypical concepts.¹⁰¹ This cognitive model diverges from classical views by introducing the idea of base concepts defined not by hard boundaries but by typical properties, which, importantly, can be subject to exceptions. Consider the concept of a chair, a familiar object that evokes a certain image in most minds – a structure with a back, a seat, and typically four legs. This image forms our prototypical chair, a concept that is flexible and can adapt to variations.

Prototype concepts:
Base concepts defined by a typical example with overridable properties.



```
Chair
  #-of-legs : four
  material  : metal
  has-back  : true
  has-arms  : false
  is-cushioned : false
  ...
```

Figure 10-11 Prototype Concept using a frame and slots representation.

This notion of a prototypical chair can be elegantly captured using the framework of frames, a concept previously introduced. Frames, with their slots and fillers, offer a structured way to represent these prototypes, embodying the stereotype with specific attributes like leg count, material, and the presence of a back. However, it is crucial to recognize that these attributes are not absolutes; they represent typical properties that can be overridden in certain instances. For example, a chair made of wood or a stool without a back still falls under the broad umbrella of “chair” despite deviating from the prototypical attributes.

The interplay between concepts and frames highlights the dynamic nature of prototype concepts. Through the lens of frames, we see how default values and inheritance play pivotal roles. A prototypical chair with default attributes like four metal legs and a back can give way to variations like stools, which inherit the chair's basic properties but modify others, such as the absence of a back.

Thus, prototypical concepts like chairs offer a fascinating glimpse into the flexible, adaptable nature of human cognition. They allow for a shared understanding of basic concepts while accommodating the rich diversity of the real world, illustrating the balance between the typical and the unique in our cognitive landscapes.

Exemplar Concepts: Understanding Through Instance

Venturing further along the spectrum of conceptual understanding, we arrive at the area of exemplar concepts, a territory where the typical conditions that guide our understanding of axiomatic and prototypical concepts no longer hold sway. Exemplar concepts are defined not by

a set of conditions, but through instances and examples, without the luxury of clear-cut definitions.¹⁰¹

Consider the concept of beauty – a notion that defies precise definition yet is universally recognized. Beauty can be encapsulated in a myriad of forms: the intricate brushstrokes of a Van Gogh painting, the ephemeral glow of a sunset, the delicate structure of a flower, or the fluid movements of a dance. Each example embodies beauty, yet none offers a concrete condition that encompasses the entire concept. This inherent ambiguity makes exemplar concepts particularly challenging to articulate or teach, especially in the context of AI programming. The nature of exemplar concepts extends beyond the world of abstract ideals like beauty, touching upon experiences and sensations unique to cultures or even individuals. This personal and cultural specificity adds another layer of complexity, making these concepts deeply subjective and resistant to universal categorization. Exemplar concepts thus represent the most elusive category in our cognitive spectrum, where understanding is rooted in individual experiences and examples rather than defined parameters. This domain reflects the rich diversity of human perception and the intricate ways in which we make sense of the world around us, highlighting the boundless capacity of the human mind to find meaning in the nuanced tapestry of life.

In the world of cognitive systems, concepts are not monolithic; they span a continuum from highly structured to notably fluid. Axiomatic concepts stand on one end of this spectrum, defined by rigid, formal conditions that delineate their boundaries clearly, akin to geometric figures like circles and triangles. Moving along the spectrum, we encounter Prototypical concepts, which, unlike their axiomatic counterparts, rely on typical properties rather than strict definitions. These concepts allow for a degree of variability, acknowledging that real-world instances may not always fit neatly within predefined parameters. Beyond exemplars lie even more abstract concepts known as “qualia,” which examine the area of sensory experiences. Qualia, such as the sensation of bitterness, are deeply personal and resist external articulation, encapsulating the subjective experience of perception. This hierarchical structuring of concepts – from the concrete and well-defined to the abstract and experiential – underscores the multifaceted nature of cognition. It highlights the varying degrees of abstraction and generalization that human thought can encompass, providing a framework for understanding the vast landscape of mental constructs.

Ordering Concepts by Formality: An Illustrative Exercise

In our exploration of concepts and their classification, we embark on an intriguing exercise to examine the spectrum of concept formality. Imagine a continuum where at one end lie the highly formalized concepts, and at the other, the more fluid and less defined ones.



Figure 10-12 Order of Concepts example.

Among the concepts we'll consider are "Inspirational," "Reptile," and "Foo," with "Foo" harking back to our discussions on incremental concept learning, serving as a bridge between abstract learning and tangible examples. Additionally, we'll examine "Right Triangle," "Holiday," and "Saltiness," each presenting unique challenges and insights into how we categorize and understand the world around us. Through this exercise, we aim to deepen our understanding of how different concepts fit within the broader landscape of cognitive categorization and the implications this has for both human cognition and artificial intelligence.

As we examine the nuances of concept formality, let's engage in an exercise that spans the spectrum from highly formalized to more fluid concepts. On one side of this spectrum, we encounter concepts like "Inspirational," "Reptile," and the familiar "Foo" from our discussions on incremental concept learning. On the other side, we find "Right Triangle," "Holiday," and "Saltiness," each presenting its own unique set of characteristics and challenges. This exploration will not only deepen our understanding of concept categorization but also highlight the diverse ways in which we, and potentially AI systems, perceive and organize information.

Ashok David, based on our discussions on the formality of concepts, how would you arrange these six concepts in order of formality?

David Well, Ashok, I placed "Right Triangle" at the very top because it epitomizes axiomatic concepts, defined by clear mathematical conditions. "Reptile" comes next, not just because of the common understanding, but due to its precise biological classification, which aligns it closer to axiomatic concepts, though it

might lean towards prototypes in everyday perception. “Foo,” as we explored, falls into the prototype category, shaped by various examples without strict definitions. “Holiday” is intriguing because it lacks a rigid definition, yet we recognize it by certain characteristics, making it a prototype as well. “Inspirational” is more abstract, hard to pin down with specific criteria, but we know it when we see it, placing it towards the exemplar end of the spectrum. And “Saltiness,” much like bitterness, is a sensory experience, challenging to convey and deeply subjective, aligning with exemplar concepts.

Ashok That's an insightful analysis, David. It underscores the importance of choosing appropriate knowledge representation and inference methods based on the nature of the concepts we're working with.

CLASSIFICATION STRATEGIES: FROM PERIODIC TABLES OF THE MIND TO ALGORITHMIC APPROACHES

In exploring the landscape of knowledge-based AI, it becomes evident that no single method suffices for all types of concepts and conditions. This diversity necessitates a spectrum of approaches, each tailored to the specific nature of the concept at hand. For instance, concepts like “Foo,” “Holiday,” or “Inspirational,” which defy easy abstraction into prototypical conditions, may be best navigated through case-based reasoning. This method leverages specific experiences, allowing for a nuanced understanding of these more fluid concepts. Conversely, when dealing with concepts like “Right Triangle” or “Reptile,” where formal definitions provide clear necessary and sufficient conditions, alternative strategies may prove more effective. This distinction underscores the importance of selecting the appropriate method based on the concept's characteristics, thereby optimizing the reasoning process.

This approach mirrors the foundational work of Dmitri Mendeleev, who categorized the elements based on their inherent properties, creating the Periodic Table. Drawing inspiration from this, our aim is to construct a metaphorical “Periodic Table of the Mind,” categorizing the fundamental elements of intelligence. This table would not list atoms and elements but would outline the various knowledge representations, organizational strategies, and reasoning methods fundamental to AI. In this context, case-based reasoning represents just one of many methods in our expanding “Periodic Table of Intelligence,” highlighting the necessity for a diverse array of strategies to address the myriad challenges presented by different concepts and conditions in the field of artificial intelligence.

Top-Down vs. Bottom-UP Classification

Expanding on the metaphor of the periodic table, we look deeper into the methodologies of classification, particularly highlighting the distinction between top-down and bottom-up approaches. The top-down method, often referred to as “establish-refine,” employs a hierarchical classification system.¹⁰⁰ This system starts with a broad concept and progressively narrows down through successive levels of specificity. This method is particularly effective in scenarios where a broad category is already known, and the task is to determine a more specific classification within that category, such as identifying a type of bird within the vertebrate class. (See Figure 10-7).

Conversely, the bottom-up approach offers a different control of processing, particularly beneficial in tasks where the goal is to synthesize a broader understanding from specific data points.¹⁰⁰ For example, consider the challenge faced by an AI agent tasked with predicting the Dow Jones Industrial Average for the next day. This agent wouldn't start with broad economic categories; instead, it would analyze specific indicators such as GDP, inflation rates, and employment figures, aggregating these data points to form a comprehensive prediction. This bottom-up processing involves a series of “identify and abstract” steps, where detailed information at the leaf nodes of a decision tree is synthesized upwards, layer by layer, until a prediction is formed at the root node. Such an approach is not about refining a pre-established broad category but about building up to it from detailed observations.

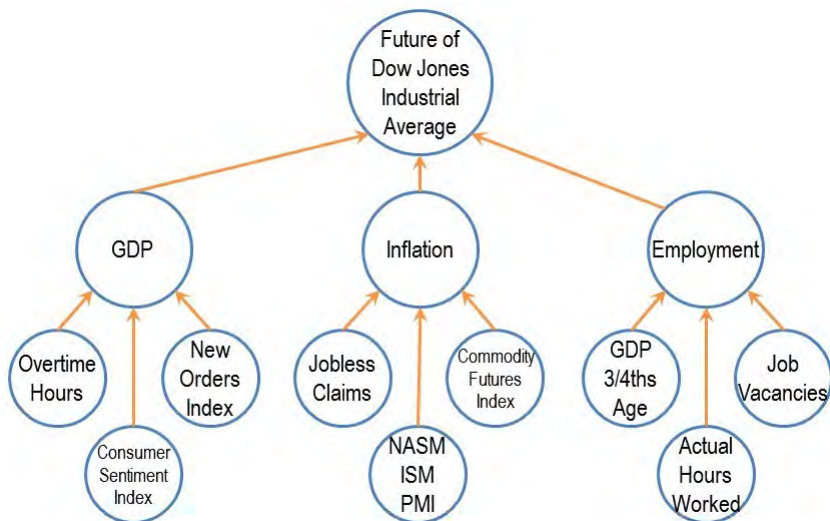


Figure 10-13 Example Bottom-up processing method.

By introducing these two distinct approaches to classification, we add new elements to our conceptual “Periodic Table of Intelligence.” Each approach represents a fundamental strategy in AI’s toolkit, emphasizing that the choice of method depends greatly on the nature of the task and the structure of the information available. The “bottom-up classification” illustrates the adaptability required in AI reasoning processes, demonstrating that the route to understanding and prediction can vary significantly based on the context and the specific challenges at hand.

THE COGNITIVE CONNECTION: CLASSIFICATION IN HUMAN THOUGHT

The interplay between classification and cognition underscores a fundamental aspect of how both humans and artificial intelligence systems navigate and make sense of the world. Classification is not merely an academic exercise; it is a core cognitive function that permeates every facet of our daily experiences. From the mundane act of recognizing a specific car make on the street to diagnosing a complex medical condition, classification underpins our ability to understand, categorize, and respond to the myriad stimuli that surrounds us.

Consider the act of driving: spotting a familiar vehicle and identifying its make involves classifying it based on certain perceptual features. This classification is instant and often subconscious, demonstrating how ingrained this cognitive process is in our daily lives. Similarly, when faced with a malfunctioning computer program, we engage in classification to diagnose the issue, labeling it as a specific type of bug based on its symptoms. This ability to classify enables us to take appropriate corrective actions, showcasing the direct link between classification and problem-solving. In the medical field, classification takes on a critical role. A doctor’s ability to classify symptoms into disease categories is the first step in the healthcare process, guiding subsequent decisions on treatment and care. This exemplifies the pivotal role of classification in action selection, a key component of intelligent behavior. It is through the lens of classification that we filter, interpret, and interact with the world, making it a cornerstone of both human and artificial cognition.

Understanding this intrinsic connection highlights why the study of classification is paramount in the field of artificial intelligence. By examining and replicating these cognitive processes, AI researchers aim to develop systems capable of similar levels of understanding and decision-making. This pursuit not only advances the field of AI but also offers profound insights into the nature of human intelligence itself, illuminating the cognitive mechanisms that enable us to navigate the complexities of our environment.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

The principles of classification discussed in this chapter, rooted in KBAI, have significant connections and offer valuable enhancements to modern AI paradigms, including Probabilistic Machine Learning, Deep Learning, Reinforcement Learning (RL), Natural Language Processing (NLP), Generative AI (GenAI), and Agentic AI.

- **Probabilistic Machine Learning:** Many machine learning classifiers (e.g., Naive Bayes, Logistic Regression) are inherently probabilistic, assigning likelihoods to different classes. KBAI's structured concept hierarchies and explicit classification rules can inform these probabilistic models. For instance, KBAI can help define prior probabilities for classes or structure the dependencies in a Bayesian network used for classification. Furthermore, the symbolic outputs of KBAI classification can be used to interpret or constrain the probabilistic outputs of ML models, leading to more robust and understandable systems.
- **Deep Learning (Neuro-Symbolic AI):** Deep neural networks excel at classification tasks by learning complex features from raw data. However, they often operate as “black boxes.” KBAI's explicit concept definitions and classification hierarchies can bring interpretability to neural network decisions. In Neuro-Symbolic AI, symbolic knowledge from KBAI can guide the training of neural classifiers, improve their data efficiency (especially in few-shot learning scenarios), verify their outputs against known constraints, or generate human-understandable explanations for why a particular classification was made. For example, a knowledge graph (a KBAI structure) could constrain the output classes of an image recognition model or provide a semantic context for its classifications.
- **Reinforcement Learning (RL):** RL agents learn to make sequences of decisions by interacting with an environment. A crucial aspect of this is the agent's ability to perceive and classify its current state to choose appropriate actions. KBAI classification methods can help define meaningful state abstractions or categorize observations, making the learning process for RL agents more efficient and targeted. The concepts learned through KBAI can serve as high-level features or symbolic state representations, enabling RL agents to learn more complex policies or transfer knowledge between related tasks.
- **Natural Language Processing (NLP) & Generative AI (GenAI):** Classification is a cornerstone of many NLP tasks, such as sentiment analysis (classifying text as positive, negative, or neutral), topic classification (assigning documents to predefined topics), intent recognition (identifying the user's goal from their query), and spam detection. KBAI's structured concept hierarchies and semantic knowledge can enhance the depth of understanding for these NLP classification tasks, allowing for more nuanced and context-aware categorizations. For Generative AI, including Large

Language Models (LLMs), classification plays several roles. It can be used to categorize user prompts to direct them to specialized models or fine-tuned versions of an LLM. The outputs of LLMs can be classified for safety, relevance, style, or factual accuracy. Moreover, KBAI-defined classes and concepts can guide or constrain the generation process, ensuring that the generated text or content adheres to specific categorical requirements or maintains logical consistency based on a predefined knowledge structure.

- **Agentic AI:** Autonomous agents operating in dynamic and complex environments must constantly classify objects, situations, and other agents to make informed decisions and interact effectively. KBAI classification provides a robust mechanism for these agents to map raw perceptual input to meaningful conceptual categories. This structured understanding then informs action selection and planning. Hierarchical classification, as discussed in this chapter, is particularly valuable for agentic AI, as it allows agents to reason and make decisions at different levels of abstraction, adapting their behavior to the specific context and complexity of the situation.

By integrating KBAI's structured and explicit approach to classification with the data-driven strengths of modern AI techniques, we can develop systems that are not only more powerful and accurate but also more interpretable, adaptable, and aligned with human-like understanding and reasoning.

CHAPTER SUMMARY

In this chapter, we examined the multifaceted area of classification, a cornerstone challenge and capability in the field of Artificial Intelligence. We embarked on this journey by revisiting the principles of incremental concept learning, which provides a foundational framework for abstracting complex concepts from simple examples. This method not only enriches our understanding but also serves as a stepping stone towards more intricate cognitive tasks. We further explored the concept of equivalence classes, a critical tool in the AI toolkit that allows us to categorize and make sense of diverse percepts by grouping them based on shared characteristics. This approach aids in simplifying and managing the overwhelming complexity of the world around us, making it a vital component of intelligent systems. Looking deeper, we examined the hierarchical structuring of concepts, drawing parallels with natural classification systems such as the biological taxonomy of the animal kingdom. This hierarchical organization, from broad categories like vertebrates down to specific classes like birds and penguins, illustrates the power of structured knowledge representation in enhancing cognitive efficiency and enabling top-down classification strategies.

The chapter also introduced us to a spectrum of concept types, from the rigidly defined axiomatic concepts (characterized by necessary and sufficient conditions) to the more fluid prototypical concepts (defined by typical features and allowing exceptions) and the subjective

exemplar concepts (understood through instances). Each type presents its unique challenges and advantages, highlighting the diversity of approaches needed to tackle the vast array of classification problems encountered in AI. We contrasted top-down (“establish-refine”) classification methods with bottom-up (“identify and abstract”) strategies. The former is useful when refining known categories, while the latter helps synthesize broader understandings from specific data points, such as in predictive tasks. This distinction underscores the adaptability required in AI to navigate the complexity of real-world environments.

This chapter also explored how KBAT's classification principles integrate with and enhance modern AI paradigms. We discussed its connections to Probabilistic Machine Learning, Deep Learning (Neuro-Symbolic AI), Reinforcement Learning, Natural Language Processing, Generative AI, and Agentic AI. This integration highlights KBAT's role in providing structure, interpretability, and conceptual grounding to contemporary data-driven AI methods.

Finally, the chapter reinforced the profound cognitive connection of classification, illustrating it as a core function in both human and artificial intelligence, essential for understanding, decision-making, and interacting with the world.

ADDITIONAL READINGS & REFERENCES

- 99 Fumagalli, M., Bella, G. & Giunchiglia, F. in *PRICAI 2019: Trends in Artificial Intelligence: 16th Pacific Rim International Conference on Artificial Intelligence, Cuvu, Yanuca Island, Fiji, August 26–30, 2019, Proceedings, Part I 16*. 71-84 (Springer).
- 100 Langley, P. *Elements of machine learning*. (Morgan Kaufmann, 1996).
- 101 Frixione, M. & Lieto, A. in *International Conference on Knowledge Engineering and Ontology Development*. 226-232 (SCITEPRESS).

Chapter 11

Logic

INTRODUCTION TO LOGIC

In this chapter, we examine logic as it applies to artificial intelligence. Logic serves as a formal language that provides us with the tools to make precise assertions about the world. This precision is not just academic; it forms the underpinning of practical applications in areas such as AI planning and decision-making systems. Understanding logic is fundamental because it helps us structure our approach to solving problems and developing algorithms that are not only efficient but also robust in their reasoning capabilities.¹⁰²

We begin our exploration with an introduction to the formal notation used in logic. This includes the use of conjunctions and disjunctions – basic yet powerful tools that allow us to combine simple statements into more complex expressions that can describe a wide range of situations and conditions. Following this, we will revisit truth tables, a method you might already be familiar with, to systematically evaluate the truth values of logical expressions. This technique is critical for understanding how different logical connectives interact within an expression. We will then discuss the rules of inference such as modus ponens and modus tollens, which are cornerstone techniques in logic for deriving valid conclusions from given premises. These rules form the backbone of logical reasoning and are essential for any system that aims to emulate human-like thinking processes. Finally, we will cover advanced methods for proving theories and theorems by refutation, with a focus on Resolution Theorem Proving. This method is

particularly powerful in the context of AI, where proving the validity of assertions can often be as important as finding the solution to a problem.

In addition to these technical aspects, we will examine the practical necessity of formal logic in the design of AI agents. An AI system typically consists of a knowledge base and an inference engine. The knowledge base stores information about the world in logical form, while the inference engine applies logical rules to derive new knowledge and make decisions. For an AI agent to function effectively and make decisions that are both sound (valid) and complete (comprehensive), it must operate under a framework of formal logic.

Despite its foundational role, the application of logic in AI will be discussed to the extent that it aids in clarifying and supporting other forms of knowledge representation like conceptual, experiential, and heuristic knowledge, which are often more aligned with the dynamic and varied nature of real-world environments. Through this chapter, we aim to provide a clear understanding of how logic is interwoven with artificial intelligence to enhance its capability to perform tasks that require high levels of cognitive functionality. This discussion sets the stage for subsequent topics in AI that build on these logical foundations to create more sophisticated and capable systems.

BUILDING BLOCKS OF PROPOSITIONAL LOGIC: THE ROLE OF LOGIC IN AI AGENTS

To construct meaningful logical statements, we need a foundational understanding of how information is structured and how basic assertions can be combined or modified. This section introduces the core components of propositional logic used in AI, starting with how AI agents utilize knowledge bases and predicates, then moving to the operators that connect or alter these predicates.¹⁰³

Knowledge Bases: Structuring Information for Reasoning

A knowledge base in AI is structured much like a database, but instead of merely storing data, it organizes information in a way that supports automated logical reasoning. For instance, consider a classification hierarchy in the natural world – vertebrates can be classified into various types, such as birds. Within this category, further distinctions can be made: eagles, bluebirds, and penguins, for example.

Logical inference involves using the structured rules within a knowledge base to derive new information or make predictions.¹⁰³ The process is fundamental for AI agents tasked with making decisions or classifications based on their programmed knowledge. In logical terms, we can represent knowledge such as “If an animal has feathers, then it is a bird.” This statement forms a simple logical rule that helps in classifying observations. Moreover, additional rules can refine this classification: “If an animal lays eggs and it flies, then it’s a bird.” These rules

collectively form part of the AI's knowledge base, allowing it to categorize and make decisions based on the given inputs. For example, an AI observing an animal that has feathers can infer, based on its knowledge base, that the animal is likely a bird. This is a direct application of modus ponens, a form of logical inference where a general statement (“All creatures with feathers are birds”) and a specific instance (“This creature has feathers”) combine to yield a conclusion (“This creature is a bird”).

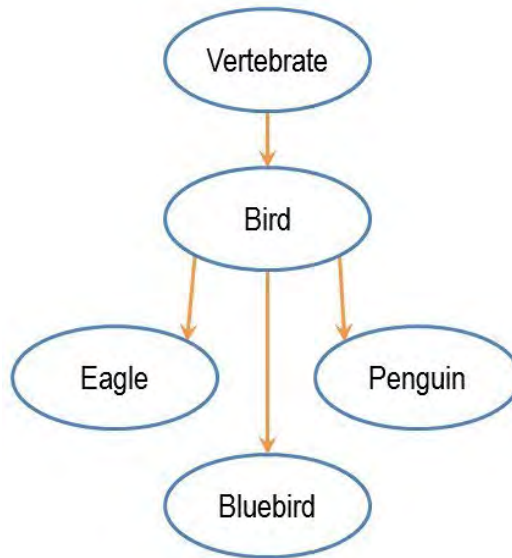


Figure 11-1 Classification of Birds. Inferences include 1) If an animal has feathers, then it is a bird, and 2) If an animal lays eggs and it flies, then it is a bird.

To illustrate the practical application of these concepts, let's examine a hypothetical construct known as 'foo.' Suppose 'foo' is defined within an AI system as an object constructed from building blocks arranged in a specific order: a block over a brick, supported by two other blocks that do not touch each other.

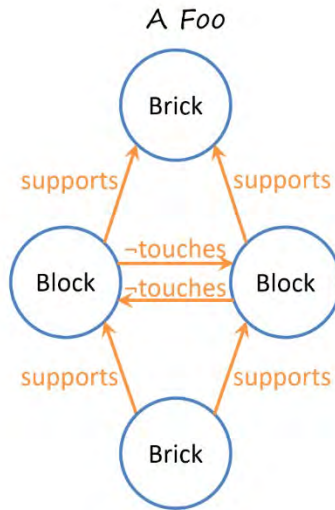
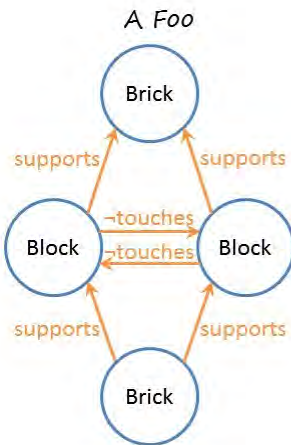


Figure 11-2 Logical Representation of “foo”.

Let’s rejoin Ashok and David as they make logical inferences about a conceptualized foo.

Ashok

David, given the structure we’ve been discussing, can you elaborate on the specific conditions that define what a foo is?



Mark the sufficient conditions:

If...

- ☐ A brick supports two bricks
- ☒ A brick supports two blocks
- ☐ Those two blocks touch
- ☒ Those two blocks do not touch
- ☐ Those two blocks support a block
- ☒ Those two blocks support a brick

...then the object is a foo

Figure 11-3 Inferences about foos.

- David

Certainly, Ashok. Starting from the base, the requirement is for a brick to support two blocks. It’s crucial that these aren't just any two blocks – they can't be touching each other. This separation is key. Then, these blocks themselves must support another brick on top. It's essential that the top element is specifically a brick and not just any block.
- Ashok

That’s a clear and concise summary. So, it sounds like the integrity of a foo's structure heavily relies on these specific spatial and material conditions. Does this configuration cover all the necessary criteria to logically classify an object as a foo?
- David:

Yes, that sums it up perfectly. If these conditions are met, then logically, we can confirm the object's classification as a foo. This precision helps in standardizing our definitions and ensuring consistent identification across any cases we might encounter.

Predicates: Describing Properties and Relationships

Building an effective knowledge base for a logic-based AI agent involves encoding information about the world into a structured, logical format. This foundational aspect of AI design involves parsing real-world elements and their interrelations into discrete, logical expressions that an AI can process and reason about. For example, consider how we might represent knowledge about different categories of animals in our AI's knowledge base. A knowledge representation scheme must capture the objects and the relationships between those objects. Using logic, we can define predicates that describe specific attributes of animals. A predicate, in this context, functions as a statement that assigns a *True* or *False* value to properties of objects.

Table 11-1 Example Predicates.

Predicates	Example
A function that maps object arguments to True or False.	<i>Has_Feathers(animal)</i>
A statement that maps predicates into True of False	<i>if Has_Feathers(animal): then Is_Bird(animal)</i>

Let's examine a practical instance, considering the classification of birds. As summarized by Table 11-1, we can establish a predicate "*Has_Feathers*" that evaluates whether a given animal object, like a bluebird, has feathers. In logical terms, if "*Has_Feathers(bluebird)*" evaluates to *True*, it confirms that bluebirds indeed have feathers.

By extending this logic, we can link attributes to broader class identities. For instance, having feathers might be a defining characteristic of all birds. Thus, another predicate "*Is_Bird*" can be associated with "*Has_Feathers*" such that if "*Has_Feathers(animal)*" is *True*, then "*Is_Bird(animal)*" must also be *True*. This implication helps the AI understand and categorize animals accurately based on their observed characteristics.

Notice how the "if" statement is one of assertion rather than one of function invocation. It can be used to establish a relationship between predicates. In this context, the statement "*if Has_Feathers (animal) then: Is_Bird(animal)*" consists of two clause: "*if Has_Feathers(animal)*" and "*then Is_Bird(animal)*". This establishes a relationship that states that if the first predicate "*Has_Feathers(Animal)*" is *True*, it *asserts* that the second predicate "*Is_Bird(animal)*" is *True* – indicating that the animal is indeed a bird. This *assertion* is distinct from using the predicate "*Is_Bird*" as a function to ascertain whether the animal qualifies as a bird. Statements where the truthfulness of one predicate asserts the truthfulness of another predicate is called an implicative relationship.¹⁰⁴

Additionally, it is customary to drop the leading verb from predicates for simplicity and clarity in logical expressions. Thus, "*Is_Bird*" is streamlined to "*Bird*". This convention is particularly useful when dealing with negations, as "*not Bird*" is more succinct and clearer than "*not Is_Bird*". This stylistic choice enhances readability and utility in constructing logical statements within the knowledge base.

Logical Connectives: Conjunctions, Disjunctions, and Negations

In the language of logic, conjunctions allow us to express relationships between multiple predicates.¹⁰⁰ Consider the example: "If an animal lays eggs and it flies, then it is a bird." To translate this into logical notation, we employ two predicates: "*Lays_Eggs*" and "*Flies*." These predicates describe the attributes of the animal.

To denote a conjunction (logical AND) between these conditions in logic, we use the symbol " $\wedge$ ". Therefore, the logical formulation of our initial natural language sentence would be structured as follows:

$$Lays_Eggs(animal) \wedge Flies(animal) \Rightarrow Bird(animal)$$

Here, the conjunction operator " $\wedge$ " connects the predicates "*Lays_Eggs(animal)*" and "*Flies(animal)*". The implication (denoted by " $\Rightarrow$ ", read as "implies") suggests that if both predicates are *True* for a given animal – that is, the animal lays eggs and flies – then the conclusion that the animal is a bird must also be *True*. This format not only simplifies

understanding but also aligns with the structured and precise requirements of logical expressions used in knowledge bases and reasoning processes.

Expanding our understanding of logical operators, let's explore the use of disjunction (logical OR), which is denoted by the symbol " $\vee$ " in logical notation. This operator allows us to express a choice between conditions, enhancing the flexibility of logical assertions.

Consider a nuanced variation of our previous example: "If an animal lays eggs or it flies, then it is a bird." This sentence introduces a choice between two conditions rather than a simultaneous requirement. To formulate this in logic, we once again utilize two predicates: "*Lays – Eggs*" and "*Flies*".

In logical terms, the sentence can be represented as:

$$Lays_Eggs(animal) \vee Flies(animal) \Rightarrow Bird(animal)$$

Here, the disjunction " $\vee$ " connects the predicates, indicating that if either condition – laying eggs or flying – is *True* for an animal, then it follows logically that the animal is a bird. Thus, implication continues to play a crucial role, asserting that the presence of either condition suffices to classify the animal as a bird. This logical structure enables more inclusive reasoning, accommodating diverse biological traits that might not strictly adhere to both conditions simultaneously.

Continuing our exploration of logic, we encounter the challenge of incorporating negations (logical NOT) within our logical statements, which is crucial for expressing the absence of certain properties. Let's analyze and reconstruct the sentence: "If an animal flies and is not a bird, then it is a bat." This statement introduces a negation to describe an exception to a general rule.

In logical form, we first establish the predicates involved: "*Flies*" indicating the ability to fly, and "*Bird*" indicating whether an animal is a bird. The negation of "*Bird*" is crucial here, as it specifies that despite the animal's ability to fly, it does not qualify as a bird.

The logical expression for this scenario can be structured as follows:

$$Flies(animal) \wedge \neg Bird(animal) \Rightarrow Bat(animal)$$

Once again, " $\wedge$ " denotes the conjunction, combining the predicates such that both conditions – the animal's ability to fly and its classification as not a bird – must simultaneously hold *True*. The negation " $\neg$ " (sometimes represented as "!" or " $\sim$ ") directly precedes the "*Bird*" predicate, articulating that the animal is not a bird. The implication leads us to conclude that if these conditions are met, then logically, the animal is a bat. This logical structure allows us to neatly express conditions that define exceptions, providing a clear framework for understanding categories that do not fit conventional classifications.

Implication: The Language of Conditionals

Building on our exploration of logical expressions, let's take a closer look at the concept of implication, which forms a critical backbone for constructing conditional statements in logic. Implication, often symbolized by an arrow " $\Rightarrow$ ", encapsulates the relationship between a condition (antecedent) and a consequence (consequent). This relationship is pivotal in logic because it allows us to assert that the presence of certain conditions necessarily leads to specific outcomes.

Consider the statement: "If an animal lays eggs and it flies, then it is a bird." To express this in the language of logic, we utilize the implication symbol to clearly define the conditional relationship. The statement can be formalized as follows:

$$\text{Lays_Eggs}(\text{animal}) \wedge \text{Flies}(\text{animal}) \Rightarrow \text{Bird}(\text{animal})$$

In this expression:

- The left-hand side of the implication $\text{Lays_Eggs}(\text{animal}) \wedge \text{Flies}(\text{animal})$ contains the conjunction of two predicates: " $\text{Lays_Eggs}(\text{animal})$ " and " $\text{Flies}(\text{animal})$ ". This conjunction establishes the conditions under which the implication holds.
- The right-hand side of the implication, " $\text{Bird}(\text{animal})$ ", specifies the conclusion that follows if the conditions on the left-hand side are *True*.

Thus, the logical implication expresses that if the specified conditions (the animal lays eggs and flies) are met, then it logically follows that the animal is a bird. This form of representation is essential in logic for constructing arguments, proving theorems, and designing logical models that accurately reflect causal and conditional relationships in diverse scenarios.

Logic Notation and Modern Equivalents

In practical applications, particularly when encoding logic within modern programming environments, the traditional symbols for logical operations might not be directly available on standard keyboards. Most programming languages like Java or Python utilize specific symbols to represent logical operations that are easily accessible during coding. For instance, logical "AND" is often denoted by `&&`, "NOT" by `!`, "OR" by `||`, and implication is typically handled through conditional structures (e.g., if-then statements) rather than a specific operator.

Operator	Symbol	Accepted Symbol
AND	$A \wedge B$	$A \& B$
		$A \&\& B$
OR	$A \vee B$	$A B$
		$A B$
NOT	$\neg A$	$!A$
		$\sim A$
IMPLIES	$A \Rightarrow B$	$A = B$
		$A == B$
		$A => B$

Figure 11-4 Logical Notation and their modern keyboard equivalent symbols

These substitutions are not only a matter of convenience but also of necessity, as the character maps of most programming languages are designed to work with a standard keyboard layout. When working within these environments or documenting logical expressions in digital format, you are encouraged to use these accessible symbols. This approach ensures that the logic you develop is both readable and executable within any standard programming environment.

Formal Logic Examples

Continuing our exploration of constructing a knowledge base in the language of logic, let's apply our understanding through practical examples. Here, we will consider four sentences and translate them into formal logical expressions. This exercise not only solidifies the concepts we've discussed but also demonstrates how these ideas manifest in actual logical formulations. Let's rejoin David and Ashok as they explore this further.

If an animal lays eggs and does not have feathers, it is a reptile.

If an animal has feathers or has talons, it is a bird.

If an animal lays eggs, has a beak, and flies, it is a duck.

If an animal lays eggs, has a beak, and do not fly, it is a platypus.

Figure 11-5 Expressing sentences in Logical Notation - Exercise

Ashok For each sentence provided, your task is to encode it using the logical notation we've covered. This exercise emphasizes the translation of everyday language into a structured format that adheres to the principles of logical design. By converting these sentences, you'll gain hands-on experience in how logical expressions are formed and how they reflect the underlying logical relationships. David, what did you come up with?

David For the first sentence, I defined predicates for laying eggs, having feathers, and being a reptile. I structured it like this: if the animal lays eggs and does not have feathers, then it is a reptile, which I've written as:

$$\text{Lays_Eggs}(\text{animal}) \wedge \neg \text{Feathers}(\text{animal}) \Rightarrow \text{Reptile}(\text{animal})$$

For the second sentence, I used the presence of feathers or talons to determine if an animal is a bird, so it's written as: if the animal has feathers or talons, then it is a bird. Logically, I wrote this as:

$$\text{Feathers}(\text{animal}) \vee \text{Talons}(\text{animal}) \Rightarrow \text{Bird}(\text{animal})$$

The third sentence was more complex; it involved an animal that lays eggs, has a beak, and flies, which together implies the animal is a duck. I wrote it as:

$$\text{Lays_Eggs}(\text{animal}) \wedge \text{Beak}(\text{animal}) \wedge \text{Flies}(\text{animal}) \Rightarrow \text{Duck}(\text{Animal})$$

And for the fourth, I noted: if the animal lays eggs, has a beak, and does not fly, then it must be a platypus, which I wrote as:

$$\text{Lays_Eggs}(\text{animal}) \wedge \text{Beak}(\text{animal}) \wedge \neg \text{Flies}(\text{Animal}) \Rightarrow \text{Platypus}(\text{animal})$$

This really highlights how distinctive a platypus is, given all these specific conditions.

Ashok Good David, that looks right to me.

This exercise showcases the translation of descriptive biological traits into logical formulations that are foundational for building knowledge bases in AI systems. Each statement David crafted adheres to logical structuring, illustrating how real-world observations are translated into formal logic that AI can process and utilize.

As we conclude our discussion on examples of how to structure predicates and implications within the language of logic, we've seen that individual predicates such as “*Flies*” can result in a binary state: *True* or *False*. However, the complexity increases significantly when these predicates are combined into more intricate sentences that also include implications. The natural question then arises: how do we determine the truth value of a sentence composed of multiple predicates and implications?

This leads us to our next topic: truth tables. Truth tables provide a systematic method to evaluate the truth values of logical expressions, particularly useful in handling complex logical sentences.¹⁰⁵ By breaking down each component of a sentence and assessing their combined effects, truth tables help us visualize and understand the logic behind AI reasoning processes. Let's explore this essential tool and see how it applies to the examples we've just discussed.

TRUTH TABLES: EVALUATING LOGICAL EXPRESSIONS

Understanding Truth Tables

In the pursuit of understanding how logical sentences are evaluated, truth tables emerge as an indispensable tool. They allow us to methodically assess the truth values of complex logical constructs, such as conjunctions, disjunctions, and negations, making them fundamental in both theoretical and practical applications of logic.

Truth tables are particularly valuable because they lay out a clear and concise visualization of how different logical operators interact within a sentence. By systematically breaking down each component of a sentence and examining the resultant truth values, we can gain deeper insights into the underlying structure of logical assertions and their implications.

This section will guide you through the construction of truth tables for various logical operators and their combinations. Whether you are revisiting familiar concepts or encountering

truth tables for the first time, this exploration will sharpen your understanding of logical expressions and their applications in reasoning processes. Let's explore how truth tables help us determine the veracity of complex sentences in logic, setting a solid foundation for more advanced topics like implication elimination.

Truth Table for Disjunction

To better understand the practical application of disjunction in logic, let us explore the truth table for the logical operation “A or B” (denoted as $A \vee B$). This operation is *True* if at least one of the operands, *A* or *B*, is *True*.

Here is a detailed breakdown of the truth table for disjunction:

- 1. **Both True:** If both *A* and *B* are *True*, then $A \vee B$ is naturally *True* because the disjunction requires only one *True* operand to be *True*.
- 2. **A True, B False:** If *A* is *True* and *B* is *False*, $A \vee B$ remains *True*. The truth of *A* alone is sufficient to satisfy the disjunction.
- 3. **A False, B True:** If *A* is *False* and *B* is *True*, $A \vee B$ is still *True* due to the truth of *B*.
- 4. **Both False:** When both *A* and *B* are *False*, $A \vee B$ results in *False*, as there are no *True* operands to fulfill the requirement of the disjunction.

This structure allows us to visually and logically verify the outcomes of different scenarios where *A* and *B* can either be *True* or *False*. The accompanying image illustrates this truth table, providing a clear and immediate reference for understanding how the disjunction operates across different conditions.

A	B	$A \vee B$
True	True	True
True	False	True
False	True	True
False	False	False

Figure 11-6 Disjunction Truth Table

By studying this truth table, one can appreciate how logical operators function and predict the outcome of more complex logical expressions built using basic logical operations like disjunction.

Truth Table Examples

In the exercise below, David and Ashok will explore how to determine the truth values for combinations of logical operations involving negation and disjunction. Let's begin by evaluating the truth table for the expression $A \vee \neg B$ and the expression $\neg A \wedge \neg B$, where A and B can be either *True* or *False*.

A	B	$A \vee \neg B$
True	True	
True	False	
False	True	
False	False	

A	B	$\neg A \wedge \neg B$
True	True	
True	False	
False	True	
False	False	

Figure 11-7 Truth Tables - Exercise 1.

Ashok David, if you start with the two logical expressions given in columns A and B, can you fill in the truth table in the image above?

David Sure, Ashok. For the first table I have *True*, *True*, *False*, and *True*. We know that when A is *True* and is combined using a disjunction, the logical value of the disjunction is *True*. When A is *False*, we must look at the value of B . However, since B is negated in the disjunctive statement, it flips the value of B . So, when B is *False*, the statement $\neg B$ is *True*, making disjunctive statement is *True*. This means that the only *False* disjunctive statement is where A is *False* and B is *True*, and that's because B is negated in the disjunctive statement.

For the second table, I have *False*, *False*, *False*, and *True*. Since this is a conjunctive statement, both the left-hand side and the right-hand side of the " $\wedge$ " operator must be *True*. Since A and B are both negated, the conjunctive statement is only *True* when A and B are both *False*.

In this next exercise, we examine a more intricate logical expression involving three predicates: A , B , and C . The objective is to determine the truth value for the compound sentence $A \vee (B \wedge \neg C)$. This expression evaluates to *True* if A is *True*, or both B is *True* and C is not *True*.

A	B	C	$A \vee (B \wedge \neg C)$
True	True	True	
True	True	False	
True	False	True	
True	False	False	
False	True	True	
False	True	False	
False	False	True	
False	False	False	

Figure 11-8 Truth Tables - Example 2.

Ashok David, please analyze the truth table and calculate the resultant truth values for each possible combination of A , B , and C .

David Let's break it down. The expression $A \vee (B \wedge \neg C)$ simplifies our task a bit. Due to the conjunction, if A is *True* in any scenario, the entire expression is *True*, irrespective of B and C . When A is *False*, the truth of the expression depends entirely on $(B \wedge \neg C)$. For this part to be *True*, both B and $\neg C$ must be *True* (which means C must be *False*). For example, if A is *False*, B is *True*, and C is *True*, the expression becomes $\text{False} \vee (\text{True} \wedge \neg \text{True}) = \text{False}$. However, if A is *False*, B is *True*, and C is *False*, then the expression turns *True*.

A	B	C	$A \vee (B \wedge \neg C)$
True	True	True	True
True	True	False	True
True	False	True	True
True	False	False	True
False	True	True	False
False	True	False	True
False	False	True	False
False	False	False	False

Figure 11-9 Truth Tables - Example 2 Answers.

Ashok Excellent breakdown, David. This exercise illustrates the need to carefully consider each part of the expression, especially as the sentences grow in complexity. It's an excellent demonstration of the logical operations and the importance of understanding how conjunction and disjunction interact, especially when negation is involved.

PROPERTIES OF LOGICAL OPERATIONS

In our study of logical relationships, we turn our attention to three fundamental properties that form the bedrock of logical algebra: the Commutative Property, the Distributive Property, and the Associative Property. These properties not only simplify our understanding of logical expressions but also enhance our ability to manipulate and reason about these expressions efficiently.

Commutative Property

This property states that the order of the operands does not affect the result of an operation. For logical operations, this means:

- $A \wedge B$ is equivalent to $B \wedge A$ (Commutation of conjunction)
- $A \vee B$ is equivalent to $B \vee A$ (Commutation of disjunction)

The Commutative Property allows for flexibility in how expressions are written and understood, emphasizing that the relationship between the elements is key, not their order.

Ashok In exploring the Commutative Property of logical operations through truth tables, let's conduct an illustrative exercise. Consider the predicates A and B , where we will compare the outcomes of $A \wedge B$ and $B \wedge A$. David, can you complete the following truth tables for these predicates?

Commutative Property

A	B	$A \wedge B$	$B \wedge A$
True	True		
True	False		
False	True		
False	False		

Figure 11-10 Truth Table - Commutative Property - Example.

David Yes, Ashok. I've filled out the boxes. It turns out that whenever either A or B is *False*, $A \wedge B$ results in *False*. This conjunction only holds *True* when both A and B are *True*, which only occurs in the top row of the table. Interestingly, $B \wedge A$ mirrors $A \wedge B$ exactly, as their truth values are identical.

Commutative Property

A	B	$A \wedge B$	$B \wedge A$
True	True	True	True
True	False	False	False
False	True	False	False
False	False	False	False

Figure 11-11 Truth Table - Commutative Property - Answers.

Ashok Excellent observation, David.

This illustrates the Commutative Property well. The property ensures that the order of the operands in logical conjunctions (and disjunctions) does not affect the outcome. Thus, $A \wedge B$ is effectively the same as $B \wedge A$, demonstrating the interchangeable nature of logical operations when it comes to the truth values they produce. This principle is fundamental in simplifying and reorganizing logical expressions, making it easier to understand and apply logical operations universally.

Distributive Property

This property is a rule that allows one operation to “distribute” over another operation contained within a bracket. In the context of logic, it is expressed as:

- $A \wedge (B \vee C)$ is equivalent to $(A \wedge B) \vee (A \wedge C)$
- Conversely, $A \vee (B \wedge C)$ is equivalent to $(A \vee B) \wedge (A \vee C)$

The Distributive Property is particularly useful for breaking down complex logical expressions into simpler, more manageable parts, which can then be analyzed or simplified separately.

<i>Distributive Property</i>				
A	B	C	$A \wedge (B \vee C)$	$(A \wedge B) \vee (A \wedge C)$
True	True	True		
True	True	False		
True	False	True		
True	False	False		
False	True	True		
False	True	False		
False	False	True		
False	False	False		

Figure 11-12 Truth Tables - Distributed Property - Example.

Ashok

Let's continue to look at logical operations by examining the Distributive Property through an engaging exercise involving three variables: *A*, *B*, and *C*. This exercise explores how the distributive property applies in logic by comparing two logically equivalent formulas. David, can you calculate the truth values for the two distributive formulas presented below?

David

Yes, Ashok. For the formula on the left, $A \wedge (B \vee C)$, it simplifies the evaluation since if *A* is *False*, the entire expression defaults to *False*, regardless of *B* or *C*. When *A* is *True*, the outcome hinges solely on whether either *B* or *C* is *True*. Now, the formula on the right, $(A \wedge B) \vee (A \wedge C)$, initially appears more complex. Here, each segment $A \wedge B$ and $A \wedge C$ must be evaluated separately, but the presence of 'or' means we need only one *True* segment for the overall expression to be *True*. Interestingly, whether *A* is *True* or *False* plays a critical role in both expressions, leading to the same truth values across corresponding rows in our truth table.

Distributive Property				
A	B	C	$A \wedge (B \vee C)$	$(A \wedge B) \vee (A \wedge C)$
True	True	True	True	True
True	True	False	True	True
True	False	True	True	True
True	False	False	False	False
False	True	True	False	False
False	True	False	False	False
False	False	True	False	False
False	False	False	False	False

Figure 11-13 Truth Tables - Distributive Property - Answers

Ashok That's an excellent breakdown, David.

This illustrates the Distributive Property where $A \wedge (B \vee C)$ is equivalent to $(A \wedge B) \vee (A \wedge C)$. The property allows us to distribute A across the operation inside the parentheses, akin to distributing a factor in a multiplication over addition in arithmetic. This property not only simplifies complex logical expressions but also ensures their flexibility in rearrangement without altering their truth values.

Associative Property

This property pertains to the grouping of operands with the same operation. It asserts that the way operands are grouped in an expression does not change the result. For logical operations, this translates to:

- $(A \wedge B) \wedge C$ is the same as $A \wedge (B \wedge C)$ (Association of conjunction)
- $(A \vee B) \vee C$ is the same as $A \vee (B \vee C)$ (Association of disjunction)

The Associative Property allows us to reorganize and regroup terms within an expression without altering the outcome, facilitating more streamlined computation and analysis.

Let's explore the Associative Property through a hands-on exercise involving truth tables and logical operations. This exercise demonstrates how altering the grouping of operations does not change the outcome when using associative logical operators.

Ashok David, can you share your results for the truth values of these two associative formulas for the table below?

Associative Property				
A	B	C	$A \vee (B \vee C)$	$(A \vee B) \vee C$
True	True	True		
True	True	False		
True	False	True		
True	False	False		
False	True	True		
False	True	False		
False	False	True		
False	False	False		

Figure 11-14 Truth Table - Associative Property - Example.

David

Absolutely, Ashok. For the first formula, $A \vee (B \vee C)$, as soon as A is *True*, the entire expression evaluates to *True*, no matter the values of B and C . When A is *False*, the result depends solely on B or C . If either B or C is *True*, the expression remains *True*. The scenario is similar for the second formula, $(A \vee B) \vee C$, where the position of the parentheses shifts but the evaluative process and outcomes remain the same. This rearrangement demonstrates that the grouping of A , B , and C does not affect the truth value, verifying the associative nature of the **or** operation.

Associative Property				
A	B	C	$A \vee (B \vee C)$	$(A \vee B) \vee C$
True	True	True	True	True
True	True	False	True	True
True	False	True	True	True
True	False	False	True	True
False	True	True	True	True
False	True	False	True	True
False	False	True	True	True
False	False	False	False	False

Figure 11-15 Truth Table - Associative Property - Answers.

Ashok Excellent explanation, David. The Associative Property, as you've shown, allows for the regrouping of elements in an expression without altering the result.

This property is applicable to both “or” and “and” operations, offering flexibility in the way expressions are structured and evaluated. This is crucial in simplifying complex logical formulas, ensuring that the interpretation remains consistent regardless of how the components are grouped.

De Morgan's Law

De Morgan's Law is a fundamental principle in logic that reveals a profound connection between conjunctions (ANDs), disjunctions (ORs), and negations (NOTs).¹⁰² This law is particularly useful when we need to simplify complex logical expressions or when transforming logical statements into a format more suitable for certain types of logical analysis or computation.

De Morgan's Law states that the negation of a conjunction is equivalent to the disjunction of the negations, and conversely, the negation of a disjunction is equivalent to the conjunction of the negations. Mathematically, these relationships can be expressed as follows:

- The negation of $A \wedge B$ is equivalent to $\neg A \vee \neg B$.
- The negation of $A \vee B$ is equivalent to $\neg A \wedge \neg B$.

These transformations are incredibly powerful because they allow us to express logical conditions in an alternative form, which can be more intuitive or practical depending on the context. This is particularly evident in computer science, where simplifying expressions can lead to more efficient algorithms. Understanding and applying De Morgan's Law not only aids in simplifying complex logical expressions but also enhances the understanding of the interplay between logical operators.

Let's explore de Morgan's Law in action through an interactive dialogue between Ashok and David, focusing on how this principle applies to logical predicates:

Ashok David, let's explore de Morgan's Law using these two predicates, A and B . We have their truth values listed below. Look at these two formulas where we apply negation. Could you calculate the truth values for these?

de Morgan's Law

A	B	$\neg(A \wedge B)$	$\neg A \vee \neg B$
True	True		
True	False		
False	True		
False	False		

Figure 11-16 Truth Table - De Morgan's Law - Example.

David

Certainly, Ashok. For the formula $\neg(A \wedge B)$, we know that $A \wedge B$ is only *True* when both A and B are *True*. Therefore, its negation will be *False* when both are *True*, and *True* in all other cases. Moving to $\neg A \vee \neg B$, this expression is *True* whenever either A or B is *False*, because the **or** operation only requires one true operand to be *True*. So, if A is *False*, $\neg A$ is *True*, and similarly for B .

de Morgan's Law

A	B	$\neg(A \wedge B)$	$\neg A \vee \neg B$
True	True	False	False
True	False	True	True
False	True	True	True
False	False	True	True

Figure 11-17 Truth Table - De Morgan's Law - Answers.

Ashok

Excellent explanation, David. So, essentially what you're illustrating is de Morgan's Law, which tells us that the negation of a conjunction – like our $\neg(A \wedge B)$ – is equivalent to the disjunction of the negations – $\neg A \vee \neg B$. This flips the conjunction into a disjunction when we distribute the negation across the predicates. And the inverse would hold if we started with a disjunction; negating that would yield a conjunction of the negations. How do the truth tables align with this understanding?

David

They align perfectly, Ashok. From the truth table, we see that $\neg(A \wedge B)$ and $\neg A \vee \neg B$ have identical truth values across all combinations of A and B . When both predicates are *True*, the result is *False*, and in all other cases,

it's *True*. This confirms that de Morgan's transformations hold universally, not just in theory but in practical scenarios as well.

Ashok That's good, David.

This demonstration effectively shows how De Morgan's law allows us to distribute negations and transform logical expressions, making it a powerful tool in logical reasoning and computational logic alike.

As we conclude our exploration of De Morgan's law and its implications for logical operations, let's reflect on the broader context and purpose behind these discussions. Recall that a logical AI agent is structured around a knowledge base and a set of formal rules of inference. The content of this knowledge base can vary widely, deriving from pre-set information or dynamically from sensory inputs, modeling a real-world scenario where an agent interacts with its environment.

The ability to rewrite logical expressions using principles like De Morgan's law is not merely academic; it serves a practical function in enhancing the flexibility and efficiency of these AI systems. By transforming sentences within the knowledge base, we can more effectively apply various rules of inference. Such transformations allow for more streamlined reasoning processes and facilitate the application of these rules in a more intuitive and computationally efficient manner.

This strategic rewriting of logical statements ensures that our logical agent can operate seamlessly, making reasoned decisions based on a robust framework of logical deductions. Next, we will examine the rules of implication, another fundamental aspect of logical reasoning, which plays a crucial role in how these agents derive conclusions from their knowledge bases.

RULES OF IMPLICATION AND INFERENCE

Understanding Implications

Understanding the concept of implication is fundamental to logical reasoning. An implication in logic represents a conditional relationship between two statements: if the first statement (the antecedent) is true, then the second statement (the consequent) must also be true. The truth table for implications helps us determine the truth value of such statements based on the truth values of their components.

Truth of Implications

A	B	$A \Rightarrow B$
True	True	True
True	False	False
False	True	True
False	False	True

Figure 11-18 Truth Table - Implications.

Let's examine how implication works with the help of the provided truth table. When both predicates A (antecedent) and B (consequent) are *True*, the implication $A \Rightarrow B$ is also *True* – this is straightforward. However, if A is *True* and B is *False*, the implication fails because the antecedent's truth does not lead to the truth of the consequent; hence $A \Rightarrow B$ is *False* in this case.

Interestingly, when A is *False*, the implication $A \Rightarrow B$ is always *True*, regardless of B 's truth value. This might seem counterintuitive at first but is a fundamental aspect of logical implication in classical logic. A false premise can imply anything.

Understanding the implications truth table is crucial as it underscores the essence of logical inference, highlighting how a correct implication hinges not merely on the truth of a proposition but on the relationship between the antecedent and consequent. Thus, implications are not about the absolute truth of their constituent propositions but about the preservation of truth from antecedent to consequent.

To explore the application of implications in logical reasoning, consider the following real-world examples where the truth of the implication hinges on the relationship between specific characteristics and classification:

1. **Feathers Imply Bird:** In the biological taxonomy, it's generally accepted that all birds have feathers. Thus, if an animal has feathers, we can logically conclude that it is a bird. This implication is straightforward and holds true under usual biological definitions.
2. **Scales Imply Bird:** This implication is interestingly *False*. Many animals, such as fish and reptiles, have scales but are not birds. Since the presence of scales does not exclusively pertain to birds, the implication that having scales would mean the animal is a bird does not hold. It is a clear example of how an implication fails when the antecedent is *True* (the animal has scales), but the consequent is *False* (the animal is a bird).
3. **Flight Implies Bird:** This implication presents a more nuanced scenario. While many birds fly, not all do, as exemplified by penguins. Furthermore, other creatures, such as bats and insects, also exhibit the capability of flight but are not birds. If we consider a penguin, which does not fly, yet is a bird, the implication might seem *False*. However,

logically speaking, the falsity of the antecedent (flight) (as in the case of a penguin), the implication is *True* regardless of the consequent. The only scenario that would invalidate “Flight implies Bird” is if the animal flies (antecedent is *True*) but it is not a bird (consequent is *False*). Since this occurs (e.g., bats fly but are not birds), the statement “Flight implies Bird” as a universal rule is *False*.

These examples highlight how implications work in logic: they are *True* unless a *True* antecedent leads to a *False* consequent. In logical discussions, understanding these nuances helps clarify how we interpret data and characteristics in reasoning tasks.

Implication Elimination

In logic, the practice of implication elimination is a pivotal strategy for simplifying and manipulating logical expressions. It is particularly useful when applying rules of inference to statements within a knowledge base, making the process of logical reasoning more straightforward and computationally feasible.

The principle of implication elimination is based on the equivalency between an implication and a disjunction involving negation. Specifically, if we have an implication such as ‘if *A* then *B*’ (symbolically, $A \Rightarrow B$), it can be transformed into a disjunction ‘not *A* or *B*’ (symbolically, $\neg A \vee B$). This transformation is grounded in the truth table equivalency between the two forms: both expressions yield the same truth values under all possible truth conditions for *A* and *B*.

This technique becomes invaluable in logical proofs and the construction of arguments because it allows for the simplification of complex logical formulas. For example, consider the biological statement “if an animal has feathers, then it is a bird” ($Feathers \Rightarrow Bird$). This reformulation can be more directly applied in proofs and logical operations, especially in automated reasoning systems where simplicity and computational efficiency are paramount.

Implication Elimination

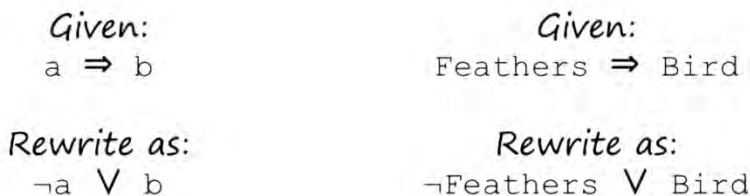


Figure 11-19 Implication Elimination

By employing implication elimination, we effectively decompose an assertion into simpler, more direct components, which can then be integrated into a broader logical framework or used in algorithmic procedures to assess truth values and make deductions. This foundational tool enriches the logical toolkit available for both theoretical exploration and practical application in diverse fields such as computer science, mathematics, and philosophy.

RULES OF INFERENCE: MODUS PONENS AND MODUS TOLLENS

In logical reasoning, the ability to correctly apply rules of inference is crucial for deriving valid conclusions from given premises. After understanding how to construct and deconstruct sentences within the language of logic, we now explore two fundamental rules of inference that are widely used: Modus Ponens and Modus Tollens.^{106,107}

Modus Ponens

Modus Ponens is a commonly encountered rule of inference that asserts if a conditional statement ($P \Rightarrow Q$) is *True*, and its antecedent (P) is also *True*, then its consequent (Q) must be *True*. This can be represented as:

- **Given:**
 - $P \Rightarrow Q$ (if P then Q)
 - P (P is *True*)
- **Infer:**
 - Q (therefore, Q is *True*)

Example of Modus Ponens: Imagine a scenario involving a logical agent, such as a robot, which has been programmed with the knowledge that “if an animal has feathers, it is a bird” ($Feathers \Rightarrow Bird$). If this robot encounters an animal with feathers, it can directly infer, based on Modus Ponens, that the animal must be a bird. This rule allows the robot to apply its knowledge effectively in real-world situations, making inferences that are both sound and reliable.

Modus Tollens

Modus Tollens is another critical rule of inference, which operates on the logic that if a conditional statement ($P \Rightarrow Q$) is accepted as *True* and the consequent (Q) is *False*, then the antecedent (P) must also be *False*. This can be formulated as:

- **Given:**

- $P \Rightarrow Q$ (if P then Q)
- $\neg Q$ (Q is *not True*)
- **Infer:**
 - $\neg P$ (therefore, P is *not True*)

Example of Modus Tollens: Continuing with the robot's example, suppose the robot has the same knowledge base where “feathers imply bird”. If the robot is told or observes that a certain animal is not a bird ($\neg Bird$), it can conclude, via Modus Tollens, that this particular animal does not have feathers ($\neg Feathers$). This application of Modus Tollens helps the robot adjust its understanding based on new information, ensuring its inferences remain consistent with its knowledge base.

These rules of inference are essential tools for any logical agent. They ensure that the agent's conclusions are logically sound, based on its programmed knowledge and any new data it perceives, thus facilitating reliable decision-making and reasoning in varied environments.

Applying the Rules of Inference

In the field of artificial intelligence, the application of logical rules such as Modus Ponens and Modus Tollens enables agents to make reliable inferences and decisions based on their knowledge bases. Let's explore two examples illustrating how these rules are applied in practice:

Example of Modus Ponens: Consider an AI agent equipped with the knowledge: “If an animal has feathers, it implies the animal is a bird.” When the agent encounters an animal named Harry, and observes that Harry has feathers, it can apply Modus Ponens:

- **Given:**
 - $Feathers \Rightarrow Bird$ (if an animal has feathers, then it is a bird)
 - $Feathers$ (Harry has feathers)
- **Infer:**
 - $Bird$ (therefore, Harry is a bird)

This scenario exemplifies how Modus Ponens helps the agent use its knowledge base to conclude that Harry is a bird, affirming the initial rule set.

Example of Modus Tollens: In a different scenario, the same AI agent aims to determine the attributes of another animal, Buzz. The agent knows the same rule: “If an animal has feathers, it implies the animal is a bird.” If the agent discovers that Buzz is not a bird, it can use Modus Tollens to infer:

- **Given:**

- $Feathers \Rightarrow Bird$ (if an animal has feathers, then it is a bird)
- $\neg Bird$ (Buzz is not a bird)
- **Infer:**
 - $\neg Feathers$ (therefore, Buzz does not have feathers)

This example demonstrates how Modus Tollens allows the agent to deduce that Buzz must not have feathers based on the absence of the consequent (being a bird).

Methods for Proving Truth Values

When proving the truth values of logical sentences, there are two primary methods:

1. Truth Tables:
 - Method: Construct a truth table for each sentence, analyzing how combinations of truth values for predicates affect the truth value of the sentence as a whole.
 - Utility: This method is particularly effective for straightforward logical relationships and serves as a visual aid in understanding how different logical operators interact.
2. Rules of Inference:
 - Method: Apply logical rules of inference, such as Modus Ponens and Modus Tollens, to derive conclusions from a set of premises.
 - Utility: This approach is more dynamic and suitable for complex reasoning tasks, allowing agents to adapt and respond based on changing information or varying scenarios.

Both methods are crucial in artificial intelligence, providing robust frameworks for decision-making and reasoning. Truth tables offer a foundational understanding, while rules of inference deliver the practical tools needed for real-time logical processing.

CHALLENGES OF TRADITIONAL LOGICAL INFERENCE

In the application of logic within computer science and artificial intelligence, particularly when dealing with complex or large-scale systems, traditional methods such as truth tables and rules of inference like Modus Ponens and Modus Tollens encounter significant challenges:

Scalability Issues with Truth Tables

The method of constructing truth tables is straightforward and effective for analyzing the logical structure of sentences. However, this approach quickly reaches its limits when applied to complicated sentences or large sets of conditions. As sentences increase in complexity with multiple variables and nested logical operations, the size of the truth tables expands exponentially (e.g., 2^n rows for n variables). This expansion makes it computationally infeasible to handle, especially for systems requiring real-time analysis and decision-making. The computational overhead of managing large truth tables is simply too great for practical applications in dynamic and responsive AI systems.

Limitations of Basic Rules of Inference in Large Knowledge Bases

Applying rules of inference such as Modus Ponens and Modus Tollens provides a powerful tool for deducing conclusions from known premises. Nonetheless, when the knowledge base grows to encompass a vast number of sentences, the task of drawing inferences becomes overwhelmingly complex. Each additional sentence potentially multiplies the number of inferences that must be considered, leading to a combinatorial explosion in the number of possible inferential paths to evaluate. This vast increase not only strains computational resources but also extends the processing time, which is impractical for applications needing swift responses.

Computational Feasibility Concerns

The core of the problem with traditional logical methods in modern AI applications lies in their computational demands. For AI systems operating under constraints of time and computational power – typical in scenarios like autonomous driving, real-time financial analysis, or dynamic robotics – the traditional logical approaches may fall short. These methods, while theoretically sound, do not always translate effectively into practice when faced with the realities of processing speed and resource availability.

While the foundational logical techniques of constructing truth tables and applying basic rules of inference have been cornerstones of theoretical computer science and artificial intelligence, their practical application is limited by issues of scalability and computational feasibility. These challenges necessitate the development of more efficient logical reasoning algorithms and techniques that can handle the complexities and dynamic nature of modern AI systems.

PREDICATE CALCULUS: EXTENDING LOGIC WITH VARIABLES AND QUANTIFIERS

In our study of logic as applied to artificial intelligence, we have thus far engaged primarily with propositional logic, also referred to as zeroth-order logic. This foundational form of logic operates without the use of variables, focusing solely on specific, static statements. For instance, a typical propositional logic statement might declare, “If an animal lays eggs and flies, then it is a bird,” referring explicitly to a singular, specified animal.

From Propositional to Predicate Logic

However, real-world applications often demand a more flexible approach, where we speak not just of specific entities but of categories and classes involving multiple or even an unlimited number of entities. This requirement leads us to Predicate Calculus (also known as first-order logic), a more sophisticated layer of logic that incorporates variables into logical statements.

In predicate calculus, a statement could be transformed to accommodate variables, allowing us to generalize assertions. For example, rather than discussing a specific animal, we could state, “If x lays eggs and x flies, then x is a bird.” Here, x represents a variable that can stand for any animal, introducing a broad scope to the statement.

Universal and Existential Quantifiers

To further refine the capability of predicate calculus, we introduce quantifiers that define the extent to which the variables apply within the logical framework:

- **Universal Quantifier ($\forall$):** This quantifier is used when the statement must apply to all possible instances of the variable. The notation $\forall x$ can be interpreted as “for all x ,” which would make our previous example read: “For all x , if x lays eggs and x flies, then x is a bird,” which is written symbolically as:

$$\forall(x)[Lays_Eggs(x) \wedge Flies(x) \Rightarrow Bird(x)]$$

This statement claims a universal truth across all instances that meet the conditions.

- **Existential Quantifier ($\exists$):** Whereas the universal quantifier speaks of all instances, the existential quantifier addresses the existence of at least one instance where the statement holds *True*. For instance, $\exists x$ means “there exists an x ” such that the conditions are met. As an example:

$$\exists x(Flies(x) \wedge \neg Bird(x))$$

means that there is at least one x (where x represents animals) such that x flies and x is not a bird. This could apply, for example, to bats, which are capable of flight but are not classified as birds.

Benefits: Scope and Efficiency

Utilizing variables and quantifiers in predicate calculus not only broadens the applicative scope of our logical operations but also enhances computational efficiency. By employing a single generalized statement with quantifiers, we can encapsulate what would otherwise require numerous individual statements in propositional logic. This consolidation significantly reduces the complexity and volume of the logical expressions we must handle, facilitating faster and more effective processing in AI systems.

Predicate calculus thus represents a powerful tool in the logician's arsenal, enabling the construction of more dynamic, versatile, and computationally feasible models for artificial intelligence applications.

RESOLUTION THEOREM PROVING: AN EFFICIENT METHOD FOR AUTOMATED REASONING

In artificial intelligence and logical reasoning, efficiency in proving theorems and validating truth values is paramount, especially when dealing with complex systems and large data sets. While traditional methods such as truth tables and standard rules of inference like Modus Ponens and Modus Tollens are foundational, they often fall short in terms of computational efficiency when scaled to real-world applications. To address these limitations, AI has adopted more robust techniques, one of which is Resolution Theorem Proving.¹⁰⁸

Introduction to Resolution Theorem Proving

Resolution Theorem Proving is a rule of inference leading to a decision procedure for propositional logic and a sound and refutation-complete inference procedure for first-order logic. It is particularly well-suited for automated theorem proving because of its suitability for algorithmic implementation. The method operates under the principle that to prove a statement, we can show that its negation leads to a contradiction when combined with a set of known true sentences (axioms or premises). A key step is transforming all sentences into a standardized form called Conjunctive Normal Form (CNF).

Conjunctive Normal Form (CNF)

Resolution theorem proving requires that every sentence within the logical system be converted into Conjunctive Normal Form (CNF). This form is crucial for applying the resolution technique effectively. A sentence in CNF is a conjunction (AND) of one or more clauses, where each clause is a disjunction (OR) of literals.

Characteristics of Conjunctive Normal Form

1. **Literal:** This is the simplest form of an expression in logic, representing a single propositional variable (an atom) or its negation. It can be either a positive literal (e.g., *Can_Move*) or a negative literal (e.g., $\neg \text{Liftable}$).
2. **Clause (Disjunction of Literals):** A clause is a disjunction (OR) of one or more literals, such as $\text{Can_Move} \vee \neg \text{Liftable}$.
3. **Sentence (Conjunction of Clauses):** A sentence in CNF is a conjunction (AND) of one or more clauses.
 - Example: $(\text{Can_Move} \vee \neg \text{Liftable}) \wedge (\neg \text{BatteryFull} \vee \neg \text{Liftable})$.

Step-by-Step Conversion to CNF

1. **Eliminate Implications:** The first task in converting sentences to CNF is the elimination of implications ($\Rightarrow$). The general rule for this conversion is: $A \Rightarrow B$ is rewritten as $\neg A \vee B$.
 - Example: For the implication $\neg \text{Can_Move} \Rightarrow \neg \text{Liftable}$, it is rewritten as $\neg(\neg \text{Can_Move}) \vee \neg \text{Liftable}$, which simplifies to $\text{Can_Move} \vee \neg \text{Liftable}$.
2. **Move Negations Inwards (De Morgan's Law):** Apply De Morgan's laws and double negation elimination to ensure negations only apply to atomic propositions.
 - $\neg(A \wedge B)$ becomes $\neg A \vee \neg B$
 - $\neg(A \vee B)$ becomes $\neg A \wedge \neg B$
 - $\neg(\neg A)$ becomes A
3. **Distribute OR over AND:** Apply the distributive law $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$ to convert the expression into a conjunction of disjunctions.

The Resolution Principle (Proof by Refutation)

Resolution Theorem Proving typically uses a technique called proof by refutation. To prove a statement (the goal or query) is true given a knowledge base (KB), we follow these steps:

1. Add the negation of the goal to the KB.
2. Convert all sentences in this augmented KB into CNF.

3. Apply the resolution rule repeatedly to derive new clauses. The resolution rule states: From $(A \vee B)$ and $(\neg B \vee C)$, we can infer $(A \vee C)$. A simpler form for propositional logic: $(P \vee Q_1 \vee \dots \vee Q_n)$ and $(\neg P \vee R_1 \vee \dots \vee R_m)$, infer $(Q_1 \vee \dots \vee Q_n \vee R_1 \vee \dots \vee R_m)$. If P and $\neg P$ are the only literals in their respective clauses, they resolve to the empty clause (representing a contradiction).
4. If the empty clause (denoted as $\square$ or False) is derived, it means the augmented KB is unsatisfiable (contains a contradiction). Therefore, the original goal must be true.

Example: The Robot and the Widget (Simple scenario)

To understand how resolution theorem proving works in practice, let's consider a practical scenario involving a factory robot. This robot's task is to pick up widgets from an assembly line. Occasionally, workers glue a widget to the assembly line. When the robot encounters such a widget, it needs to determine that the widget is not liftable.

Initial Knowledge Base (KB)

- Rule 1: If a widget cannot be moved, it implies the widget is not liftable.
 - In logic: $\neg \text{CanMove}(\text{widget}) \Rightarrow \neg \text{Liftable}(\text{widget})$
- Observation (Sensory Input): The robot attempts to move a widget and finds it immovable.
 - In logic: $\neg \text{CanMove}(\text{widget})$

Goal to Prove: The widget is not liftable ($\neg \text{Liftable}(\text{widget})$).

Resolution Steps

1. Negate the Goal and Add to KB: Assume the opposite of the goal: $\text{Liftable}(\text{widget})$.
2. Convert to CNF:
 - KB Rule 1: $\neg \text{CanMove}(\text{widget}) \Rightarrow \neg \text{Liftable}(\text{widget})$ becomes $\text{CanMove}(\text{widget}) \vee \neg \text{Liftable}(\text{widget})$ (Sentence S1)
 - Observation: $\neg \text{CanMove}(\text{widget})$ (Sentence S2)
 - Negated Goal: $\text{Liftable}(\text{widget})$ (Sentence S3)
3. Apply Resolution:
 - Resolve S1 ($\text{CanMove}(\text{widget}) \vee \neg \text{Liftable}(\text{widget})$) and S3 ($\text{Liftable}(\text{widget})$): The $\neg \text{Liftable}(\text{widget})$ from S1 and $\text{Liftable}(\text{widget})$ from S3 cancel out, leaving $\text{CanMove}(\text{widget})$ (Sentence S4).
 - Resolve S4 ($\text{CanMove}(\text{widget})$) and S2 ($\neg \text{CanMove}(\text{widget})$): The $\text{CanMove}(\text{widget})$ from S4 and $\neg \text{CanMove}(\text{widget})$ from S2 cancel out, leaving the empty clause $\square$.

Conclusion: Since we derived the empty clause, our initial assumption ($Liftable(widget)$) must be *False*. Therefore, the goal ($\neg Liftable(widget)$) is proven *True*. The robot logically deduces that the widget is not liftable.

Example: The Robot, Widget, and Battery (Complex scenario)

Let's expand the example. Imagine the robot has concluded that a box is not liftable. The factory workers suggest the issue might be the robot's battery rather than the box's immovability.

Refined Knowledge Base (KB):

- Rule 1: If the box cannot move AND the battery is full, then the box is not liftable.
 - $(\neg CanMove(box) \wedge BatteryFull) \Rightarrow \neg Liftable(box)$
- Observation 1: The box cannot move.
 - $\neg CanMove(box)$
- Observation 2: The robot's battery is full.
 - $BatteryFull$

Goal to Prove: The box is not liftable ($\neg Liftable(box)$).

Resolution Steps:

1. **Negate the Goal and Add to KB:** Assume $Liftable(box)$.
2. **Convert to CNF:**
 - KB Rule 1: $(\neg CanMove(box) \wedge BatteryFull) \Rightarrow \neg Liftable(box)$
 - Eliminate implication: $\neg(\neg CanMove(box) \wedge BatteryFull) \vee \neg Liftable(box)$
 - Apply De Morgan's: $(\neg\neg CanMove(box) \vee \neg BatteryFull) \vee \neg Liftable(box)$
 - Simplify: $CanMove(box) \vee \neg BatteryFull \vee \neg Liftable(box)$ (Sentence S1)
 - Observation 1: $\neg CanMove(box)$ (Sentence S2)
 - Observation 2: $BatteryFull$ (Sentence S3)
 - Negated Goal: $Liftable(box)$ (Sentence S4)
3. **Apply Resolution:**
 - Resolve S1 ($CanMove(box) \vee \neg BatteryFull \vee \neg Liftable(box)$) and S4 ($Liftable(box)$): This yields $CanMove(box) \vee \neg BatteryFull$ (Sentence S5).
 - Resolve S5 ($CanMove(box) \vee \neg BatteryFull$) and S3 ($BatteryFull$): The $\neg BatteryFull$ from S5 and $BatteryFull$ from S3 cancel, leaving $CanMove(box)$ (Sentence S6).

- Resolve S6 ($CanMove(box)$) and S2 ($\neg CanMove(box)$): These cancel, leaving the empty clause $\square$.

Conclusion: The empty clause is derived, so the assumption $Liftable(box)$ is *False*. Thus, $\neg Liftable(box)$ is proven *True*.

This detailed example illustrates how resolution theorem proving allows a logical agent to handle complex scenarios with multiple conditions.

Focus of Attention and Horn Clauses in Resolution

An essential aspect of using resolution theorem proving, especially in complex logical environments, is the management of the agent's focus of attention. When dealing with large and intricate sets of sentences within a knowledge base, it can become challenging for the agent to effectively navigate and prioritize tasks.

Benefits of Conjunctive Normal Form (CNF): By converting all statements in the knowledge base into CNF, we streamline the problem space significantly. CNF simplifies the structure of sentences to a uniform format that aids in systematic analysis and resolution processes.

Implementing Resolution Step-by-Step: In resolution theorem proving, the process is clear:

1. Identify the Target Literal: The agent always begins by focusing on a specific literal from the (negated) goal.
2. Search for Contradictory Evidence: It then searches for clauses containing the negation of this target literal.
3. Apply Resolution: Upon finding such a clause, the agent applies the resolution rule, seeking to eliminate the contradictory pair of literals and generate a new clause (resolvent).
4. Proceed Sequentially: This new clause is added to the set of clauses, and the process repeats, selecting new pairs of clauses to resolve, until the empty clause is derived or no more new clauses can be generated.

Efficiency Through Horn Clauses: The use of Horn clauses can significantly enhance the computational efficiency of resolution theorem proving. A Horn clause is a clause with at most one positive literal.

- Example: $\neg A \vee \neg B \vee C$ (implication form: $(A \wedge B) \Rightarrow C$)
- Example: $\neg A \vee \neg B$ (implication form: $(A \wedge B) \Rightarrow False$)
- Example: C (implication form: $True \Rightarrow C$)

Reasoning with Horn clauses (e.g., using an inference algorithm like forward chaining or backward chaining) can be done in polynomial time, which is much more efficient than general resolution. Prolog, a logic programming language, is based on Horn clauses.

The structured approach of resolution theorem proving, aided by the use of CNF and potentially specialized strategies for Horn clauses, provides a robust framework for logical agents to handle complex reasoning tasks.

Walkthrough Example: Has-Wings, Has-Fur, Bird

Let's rejoin David and Ashok as they walk through an example using resolution theorem proving.

Ashok Let's ensure we understand resolution theorem proving by going through a practical exercise. Consider the statement: "If an animal has wings and does not have fur, it is a bird." Let's write this down in formal logic using the predicates *has-wings*, *has-fur*, and *bird*.



If an animal has wings and does not have fur, it is a bird.

Write in formal logic:

*(Use the predicates *has-wings*, *has-fur*, and *bird*)*

How do we prove this is a bird?

Figure 11-20 Resolution Theorem Proving - Formal Logic

David I started by translating the statement into logic. "*has-wings*" translates to the predicate *has – wings*, and "does not have fur" translates to the negation $\neg$ *has – fur*. These two are combined using a conjunction. Thus, the full implication in formal logic becomes:

$$(has - wings(x) \wedge \neg has - fur(x)) \Rightarrow bird(x)$$

Ashok That's correct, David. Now, we need to put this into a conjunctive normal form to utilize it in resolution theorem proving. Remember, to eliminate the implication, we can use the implication elimination technique, which transforms an implication $P \Rightarrow Q$ into $\neg P \vee Q$. Could you do that transformation?

David Of course. Applying the implication elimination here, we take the negation of the conjunction and combine it with 'bird' using a disjunction. Thus, the transformed statement becomes:

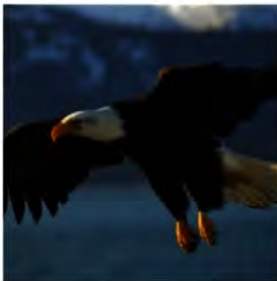
$$\neg(has - wings(x) \wedge \neg has - fur(x)) \vee \neg bird(x)$$

Ashok Now, apply de Morgan's Law to the negation of the conjunction, which turns it into a disjunction of negations.

David Right, by de Morgan's Law, the negation of the conjunction $\neg has - wings(x) \wedge \neg has - fur(x)$ is equivalent to $\neg has - wings(x) \vee has - fur(x)$. So, the entire statement now in conjunctive normal form is:

$$(\neg has - wings(x) \vee has - fur(x)) \vee bird(x)$$

Ashok Excellent, David. That's precisely the conjunctive normal form we need for resolution theorem proving. We are now ready to use this form in our theorem proving exercises to deduce logical conclusions about animals based on their characteristics. David, considering the robot has the initial knowledge and now perceives an animal with wings that does not have fur, how should we proceed with resolution theorem proving? What sentence should we write in S4 to facilitate this proof?



S1:
 $\neg has - wings \vee has - fur \vee bird$
 S2: $has - wings$
 S3: $\neg has - fur$

What sentence would be assumed to facilitate the proof?

S4:

Figure 11-21 Resolution Theorem Proving - Example.

David: Right, Ashok, in resolution theorem proving, we often employ proof by refutation. This means we start by assuming the opposite of what we're trying to prove. Since we want to demonstrate that the animal is a bird ($bird(animal)$), we should assume in S4 that it is not a bird. The new information is: S2: $has - wings(animal)$ S3: $\neg has - fur(animal)$.

Ashok: Exactly, David. Assuming the negation of our desired conclusion allows us to use resolution to find contradictions, thereby validating our original assumption if the contradiction is resolved. So, the sentence for S4 should be:

$$S4: \neg bird(animal)$$

This assumption will help us conduct the resolution with the existing sentences from the knowledge base:

$$S1: \neg has - wings(animal) \vee has - fur(animal) \vee bird(animal)$$

$$S2: has - wings(animal)$$

$$S3: \neg has - fur(animal)$$

$$S4: \neg bird(animal)$$

David: With these sentences set, we can now look for a resolution.

- Resolve S1 and S4: $(\neg has - wings(animal) \vee has - fur(animal) \vee bird(animal))$ and $(\neg bird(animal))$ resolve to $(\neg has - wings(animal) \vee has - fur(animal))$ (Let's call this S5).
- Resolve S5 and S2: $(\neg has - wings(animal) \vee has - fur(animal))$ and $(has - wings(animal))$ resolve to $(has - fur(animal))$ (Let's call this S6).
- Resolve S6 and S3: $(has - fur(animal))$ and $(\neg has - fur(animal))$ resolve to the empty clause $[]$.

Ashok Once the resolution process is complete, if all contradictory elements are resolved, leaving no unresolved literals (the empty clause), the negation we assumed in S4 is disproved, thus proving our initial proposition that the animal is a bird, based on its features of having wings and no fur.

This scenario illustrates the power and methodological precision of resolution theorem proving in artificial intelligence applications, particularly in automated reasoning and decision-making processes.

LOGIC AND COGNITION

The intricate relationship between logic and human cognition is not only fascinating but also crucial in understanding both artificial intelligence and the human mind. Logic, renowned for its rigorous structure and precision, offers a clear and defined way of reasoning that is highly valued in AI development. It provides a framework for constructing formal notations that articulate how intelligent agents should reason, whether they adhere strictly to logical paradigms or not. This structured approach to reasoning is essential for developing reliable and predictable AI systems.

While logic is instrumental in AI, its role in human cognition is more nuanced. Humans often employ logic in everyday decision-making, as seen in straightforward scenarios where conclusions follow logically from premises. For example, the reasoning “If I receive a large bonus, I will take a long vacation; I received a large bonus; therefore, I will take a long vacation” mirrors the structure of Modus Ponens – a fundamental logical form.

However, the assumption that humans fundamentally operate on logic is oversimplified. Human reasoning frequently involves analogies and heuristic approaches that might appear logical but do not necessarily follow formal logical rules. For instance, solving a problem by relating it to a previously encountered one uses analogical reasoning, which might mimic logical structure without actually engaging formal logical processes.

Beyond Deductive Logic: Inductive and Abductive Reasoning

The type of logic discussed so far in this book has primarily been deductive, focusing on deriving specific conclusions from general rules. However, much of human reasoning operates on inductive and abductive logic, which are essential for navigating real-world complexities where information is incomplete or ambiguous.

- **Inductive Reasoning:** This involves generalizing from specific instances to broader generalizations.¹⁰⁹ For example, observing that a medication relieves symptoms in several individual cases might lead one to conclude it is effective more broadly.
- **Abductive Reasoning:** Often referred to as inference to the best explanation, abduction is about forming plausible hypotheses for observed phenomena.¹⁰⁹ A doctor diagnosing a disease from symptoms exemplifies abductive reasoning – deriving likely causes from the observed effects.

These forms of reasoning are critical in everyday human thought processes, where decisions and conclusions must often be drawn from incomplete data sets or partial observations.

While deductive logic offers a clean, mathematical neatness in AI, human cognition is messier, incorporating a mix of deductive, inductive, and abductive reasoning. This mixture allows for flexibility and adaptability in uncertain environments. Understanding this blend is crucial for developing AI that more accurately mimics human thought processes and for appreciating the complexity of human reasoning.

Future Explorations

As we progress in our exploration of AI and cognition, we will investigate deeper into how these various forms of reasoning can be effectively modeled and integrated into AI systems. The goal is to enhance AI's reasoning capabilities to be more aligned with human-like thinking, thereby making AI systems more intuitive and effective in real-world applications.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

Formal logic, as explored in this chapter, provides foundational principles for explicit knowledge representation and structured reasoning. These KBAI tenets, far from being outdated, offer significant enhancements and complementary strengths when integrated with modern AI paradigms such as Probabilistic Machine Learning, Deep Learning, Reinforcement Learning, Natural Language Processing (NLP), Generative AI (GenAI), and Agentic AI.

- **Probabilistic Machine Learning:** While probabilistic models excel at handling uncertainty, KBAI's logical frameworks can provide the underlying structure or constraints. For instance, logic can define the causal relationships in a Bayesian network, guide the selection of relevant variables, or impose logical consistency on probabilistic inferences. This can lead to more robust and interpretable probabilistic models.
- **Deep Learning (Neuro-Symbolic AI):** The fusion of deep learning's pattern recognition capabilities with KBAI's symbolic reasoning is a burgeoning field known as Neuro-Symbolic AI. Logic can be used to:
 - Inject domain knowledge into neural networks, improving data efficiency and generalization, especially in few-shot learning scenarios.
 - Verify the outputs of “black box” neural models against logical constraints.
 - Provide interpretable explanations for neural network decisions by translating learned representations into symbolic rules.
 - Enable more complex, multi-step reasoning by combining neural perception with logical deduction.
- **Reinforcement Learning (RL):** RL agents learn optimal policies through interaction. KBAI's logical representations can be used to define complex goal specifications, describe state abstractions in a more human-understandable way, or shape reward functions based on logical conditions. This can accelerate learning and make learned policies more transparent.
- **Natural Language Processing (NLP) & Generative AI (GenAI):** Logic has historical ties to NLP through semantic parsing and formal grammars. In the era of Large Language Models (LLMs):

- Knowledge graphs (structured logical knowledge) can ground LLM outputs in factual data, reducing hallucinations and improving accuracy.
- Logical reasoning can verify the consistency of text generated by LLMs or ensure adherence to specific constraints.
- LLMs can be prompted to perform logical reasoning tasks or generate outputs that conform to logical structures.
- Formal logic can help in representing and reasoning about the meaning extracted by NLP systems.
- **Agentic AI:** For autonomous agents to plan, reason about their actions, and interact intelligently in complex environments, explicit knowledge representation and logical inference are crucial. KBAI provides foundations for:
 - Representing beliefs, desires, and intentions (BDI architectures).
 - Developing sophisticated planning algorithms that can reason about preconditions and effects of actions (as seen in classical planning, which has strong logical underpinnings).
 - Ensuring an agent's behavior is consistent with its knowledge and goals.

The integration of KBAI's explicit, symbolic approaches with the data-driven, sub-symbolic methods of modern AI is leading to hybrid systems that are more powerful, robust, interpretable, and trustworthy. Logic provides the tools for precision, consistency, and explanation, which are becoming increasingly important as AI systems tackle more complex and safety-critical tasks.

CHAPTER SUMMARY

In this chapter, we have undertaken a comprehensive examination of formal logic, establishing a robust foundation for sophisticated reasoning and decision-making in various applications, especially within artificial intelligence. Our study began with the essentials of formal notation – exploring conjunctions, disjunctions, negations, and the precise structuring of logical sentences using predicates and implications. This groundwork enabled us to effectively utilize truth tables, a fundamental tool for systematically assessing the veracity of logical statements and understanding operator interactions.

We further expanded our toolkit by introducing and applying critical logical properties such as the Commutative, Associative, and Distributive properties, alongside De Morgan's law. These principles simplify complex logical expressions, enhance our understanding of logical equivalences, and are pivotal in streamlining logical deductions for clear, concise reasoning.

Our discussion on the rules of inference, including Modus Ponens and Modus Tollens, illustrated how we can derive valid conclusions from given premises, reinforcing the power of formal logic. We also looked into implication elimination as a technique for transforming logical statements. We then addressed the challenges of traditional inference methods, such as scalability and computational feasibility, which led to an introduction to Predicate Calculus, extending logic with variables and quantifiers ($\forall, \exists$) for more expressive and efficient representation. The chapter

culminated in an exploration of Resolution Theorem Proving, a powerful automated reasoning technique based on converting sentences to Conjunctive Normal Form (CNF) and using refutation to prove statements.

We also considered the relationship between formal logic and human cognition, acknowledging that while deductive logic is a cornerstone of AI, human reasoning often incorporates inductive and abductive approaches.

Finally, we explored the crucial role of KBAI's logical principles in enhancing modern AI paradigms. Integrating logic with Probabilistic Machine Learning, Deep Learning (Neuro-Symbolic AI), Reinforcement Learning, NLP/Generative AI, and Agentic AI can lead to systems that are more robust, interpretable, and capable of complex reasoning.

As we transition from the constructs of formal logic to their practical applications in planning and decision-making, it's clear that the principles studied are not merely academic. They are crucial for enabling AI agents to reason about, and interact with, their environments more effectively. Planning, our focus in the next chapter, builds directly on this foundation, employing formal logic to strategize and determine actionable paths from initial states to desired outcomes. The logical frameworks and methodologies discussed are instrumental in advancing AI's capabilities, shaping how agents infer, plan, and execute tasks.

ADDITIONAL READINGS & REFERENCES

- 100 Langley, P. *Elements of machine learning*. (Morgan Kaufmann, 1996).
- 102 Cropper, A. & Dumančić, S. Inductive logic programming at 30: a new introduction. *Journal of Artificial Intelligence Research* **74**, 765-850 (2022).
- 103 Russell, S. & Norvig, P. *Artificial Intelligence: A Modern Approach*. (Pearson, 2015).
- 104 Cintula, P. & Noguera, C. *Logic and implication*. (Springer, 2021).
- 105 Shosky, J. Russell's use of truth tables. *Russell: The Journal of Bertrand Russell Archives* **17**, 11-26 (1997).
- 106 Schechter, J. & Enoch, D. Meaning and justification: The case of modus ponens. *Noûs*, 687-715 (2006).
- 107 Adams, E. W. Modus tollens revisited. *Analysis* **48**, 122-128 (1988).
- 108 Bachmair, L. & Ganzinger, H. Resolution Theorem Proving. *Handbook of automated reasoning* **1** (2001).
- 109 Johnson, S. G., Merchant, T. & Keil, F. C. in *Proceedings of the Annual Meeting of the Cognitive Science Society*.

Chapter 12

Planning

INTRODUCTION TO PLANNING

Planning is a crucial process for any intelligent agent tasked with navigating the world and making decisions. As we've discussed previously, an intelligent agent is one that maps its perceptual histories into actions. Planning is one of the most powerful methods for selecting those actions, as it provides a structured way for the agent to move from its current state to a desired goal state. In this chapter, we will explore the mechanics of planning in depth and learn how logical reasoning can be applied to formulate and execute plans effectively.

We'll start by building on the knowledge from the previous chapter on logic. The syntax of logic will serve as the foundation for specifying goals, initial states, and operators – essential elements that describe the actions an agent can take to move between states. This will give us the tools we need to define the problem space in which the agent operates, making it possible to outline how the agent can plan to achieve its goals.

However, planning doesn't come without challenges. One of the key issues arises when an agent is faced with multiple goals. These goals may sometimes conflict with one another, forcing the agent to determine the best way to prioritize and sequence actions. For example, painting both a ladder and a ceiling presents a conflict: if the ladder is painted first, it becomes impossible to climb the wet ladder to paint the ceiling. Planning methods need to account for such conflicts and offer strategies to avoid them.

To address these challenges, we'll explore Partial-Order Planning, a method designed specifically to handle situations where goals are interdependent and may conflict. This technique allows the agent to avoid goal clashes by not committing to a rigid sequence of actions upfront, instead maintaining flexibility as goals are pursued.

Lastly, we'll explore Hierarchical Task Networks (HTNs), a representation that helps create more complex, multi-level plans. HTNs break down large problems into smaller, manageable tasks, enabling the agent to plan hierarchically - starting with higher-level goals and progressively working down to the detailed actions required. This not only simplifies the planning process but also mirrors how human cognition often tackles complicated tasks by thinking at multiple levels of abstraction.

In this chapter, we'll walk through the mechanics of planning, identify the problems it addresses, and understand how it is essential to both artificial intelligence and human cognition. Through examples and formal representations, you will learn how intelligent agents navigate their environments and make decisions, even in the face of conflicting goals or complex tasks.

THE NATURE OF PLANNING AND GOAL PRIORITIZATION

Let's explore the concept of planning through two examples that highlight the challenges of managing multiple goals and selecting intelligent actions.



Figure 12-1 Blocks world illustration

The first example, called Blocks World, involves a robot tasked with rearranging blocks to achieve a desired configuration. In the current state, blocks A, B, and C are stacked, while block D sits separately on the table. The goal state, however, requires the blocks to be reordered, with

A on B, B on C, and C on D. The robot, constrained by having only one arm, can move only one block at a time and only if no other block is stacked on top of it.

This problem was introduced when we discussed Means-Ends Analysis, where it can be used to help the robot determine the necessary actions to achieve its goal. However, a critical question arises: how does the agent select which goal to prioritize when multiple goals are at play? Earlier, we glossed over this issue by suggesting that the agent “might” choose a goal. Now, we will take a closer look at how the robot can systematically break down its goals and decide which to pursue first.

Let's move on to a second, more realistic example: you hire a robot to paint both the ceiling and the ladder in your room. Here, the robot has two goals: 1) paint the ceiling, and 2) paint the ladder. At first, this seems like a simple task, but there's an inherent conflict: if the robot paints the ladder first, it will be wet and unusable for climbing, making it impossible for the robot to paint the ceiling afterward.

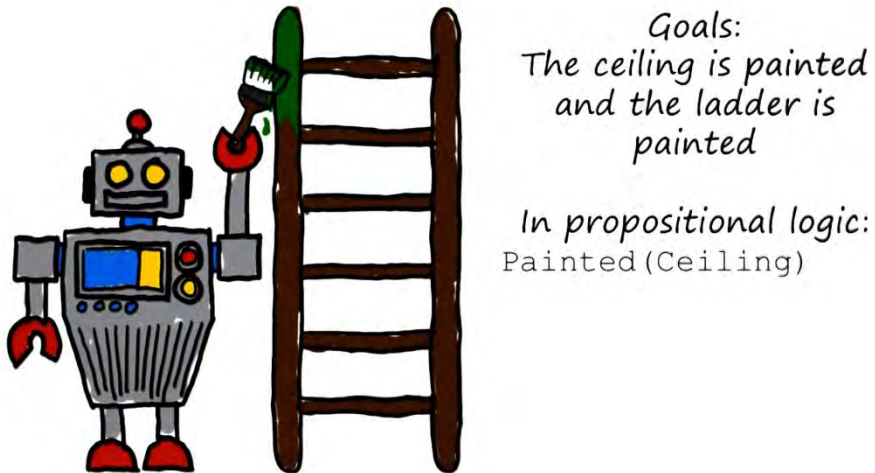


Figure 12-2 Illustration of a robot that must paint a ceiling and a ladder.

For humans, the solution is almost instinctive - you paint the ceiling first, then the ladder. This straightforward decision resolves the conflict. But for a robot, resolving such goal conflicts requires more deliberate reasoning. It needs to recognize how to properly represent and prioritize its goals to avoid complications.

To formalize this process, we use propositional logic to express the goals:

- *Painted(Ceiling)*: represents the goal of painting the ceiling.
- *Painted(Ladder)*: represents the goal of painting the ladder.

By using logic, the robot can reason about the sequence in which it must act. First, it paints the ceiling, ensuring that it can still climb the ladder, then it proceeds to paint the ladder. This

logical flow prevents the conflict that would arise if the robot tackled the tasks in the wrong order. In this case, the robot needs a planning technique called Partial-Order Planning, which helps it prioritize its goals without one action conflicting with another. This method enables the robot to manage multiple goals, ensuring that it completes both tasks efficiently and in the correct sequence.

In both examples - the Blocks World and the painting robot - the central issue is the same: intelligent agents must plan their actions carefully while considering potential conflicts between goals. Whether it's rearranging blocks or painting a room, intelligent planning requires a detailed understanding of how goals interact, and a strategy for resolving conflicts to achieve the desired outcomes.

Now we'll explore how logical representations and goal decomposition provide the foundation for such intelligent planning, along with conflict-resolution methods like partial-order planning. These approaches allow an agent to navigate complex tasks, ensuring that its actions are aligned with both its immediate and long-term objectives.

REPRESENTING PLANNING PROBLEMS IN LOGIC

Specifying Goals Using Propositional Logic

When defining goals for intelligent agents, it is crucial to use clear and precise representations. In our example, the robot has two conflicting goals: painting the ceiling and painting the ladder. To avoid conflicts and ensure the robot prioritizes the correct task, we can represent these goals in propositional logic.

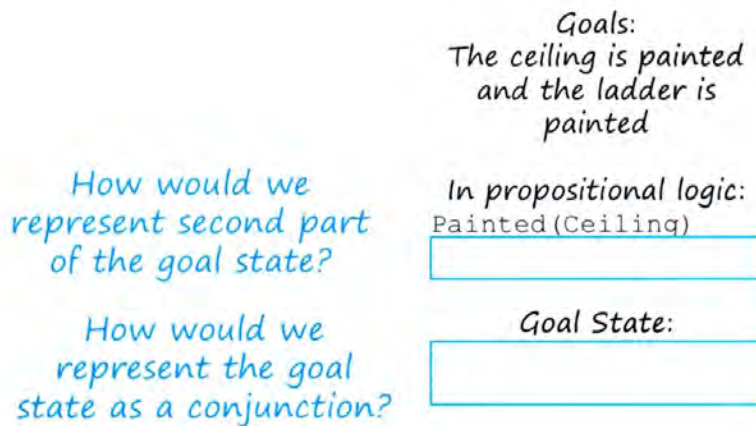


Figure 12-3 Propositional logic and goal states

- Ashok** David, what do you think?
- David** The second goal is straightforward. We're using the same proposition – *Painted* – but now we're applying it to the “Ladder” instead of the “Ceiling.” To represent the goal state, we'll just link the two goals together using a conjunction. So, our goal state becomes *Painted(Ceiling) $\wedge$ Painted(Ladder)*.
- Ashok** Good. David correctly applies the same logical framework to both tasks. By using the predicate *Painted* for both the ceiling and the ladder, and linking them with the conjunction operator $\wedge$, he formulates a clear goal state that reflects both tasks.

This dialogue highlights the importance of using formal logic to specify goals in a structured manner. Now that both tasks are logically represented, the next step will be to specify not only the goal state but also the initial state. With this information, the robot will be equipped to plan its actions effectively and resolve potential conflicts between goals.

Defining States: Initial, Goal, and Intermediate

In the context of planning, a crucial first step is to clearly define both the initial state and the goal state of the world. Let's work through an example involving a robot tasked with painting a ceiling and a ladder. To succeed, the robot must move between these two tasks in a way that avoids conflicts, like painting the ladder first, which would make it wet and unusable for climbing to reach the ceiling.

To represent this problem in logical terms, we define the initial state and the goal state as follows:

- **Initial State:** *On(Robot, Floor) $\wedge$ Dry(Ladder) $\wedge$ Dry(Ceiling)*
- **Goal State:** *Painted(Ceiling) $\wedge$ Painted(Ladder)*

In this formulation:

- “*On(Robot, Floor)*” specifies that the robot begins on the floor.
- “*Dry(Ladder)*” and “*Dry(Ceiling)*” indicate that both the ladder and ceiling are dry in the initial state.
- The goal is to have both the ceiling and the ladder painted, expressed with “*Painted(Ceiling)*” and “*Painted(Ladder)*.”

By defining these states in propositional logic, we have now fully described the task. The robot's job is to figure out a sequence of actions that will transform the world from the initial state to the goal state without introducing conflicts.

Representing the Robot's State After Painting the Ceiling

Once we've specified the initial and goal states, we can start to think about the intermediate states that the robot will pass through as it works toward the goal. For example, consider what happens after the robot has climbed the ladder and painted the ceiling.

Ashok How did you describe the state of the world after the robot climbs the ladder and paints the ceiling?

David Since our goal includes the ceiling being painted, I'll begin by writing "*Painted(Ceiling)*" to represent that change. In the initial state, the robot was on the floor, but now it's on the ladder, so I'll change that to "*On(Robot, Ladder)*." Since no other information was given, I'll just combine these two facts with a conjunction.

Ashok That's good, David.

David's response provides a clear example of how to describe an intermediate state in the planning process. After the robot paints the ceiling, the two key changes are:

- The robot is no longer on the floor, but on the ladder.
- The ceiling is now painted.

Thus, we represent the state as:

- **State:** *On(Robot, Ladder) $\wedge$ Painted(Ceiling)*

Carrying Information Across States

One important concept in planning is how information from earlier states can propagate to later states. For example, if the ladder and ceiling were both dry in the initial state, and nothing happens in the meantime to make them wet, this fact can be carried forward to subsequent states. However, if an action like "paint-ceiling" causes the ceiling to no longer be dry, we must update the state accordingly by negating that fact.

This raises another good point. Information about earlier states, such as the ladder and ceiling being dry, can be carried forward as long as no action has occurred to negate it. As we'll see later, it's essential to keep track of which facts remain true and which facts have changed as the robot moves through each stage of the plan.

This ability to propagate unchanged facts is critical for maintaining consistency in the planning process.¹¹⁰ It ensures that the robot's actions are based on a correct understanding of the world's current state at each step.

In planning, the accurate specification of states – initial, goal, and intermediate – is foundational. As we've seen, the robot must understand the state of the world before it can generate a plan. Defining the problem in propositional logic allows for precise reasoning about

how the robot can progress through its tasks, step by step, without creating conflicts. Through this structured approach, we ensure the robot can systematically and efficiently move from the initial state to the goal state, all while maintaining a clear and correct understanding of the world at each point in the process.

Defining Operators: Actions and Their Consequences

After specifying states in planning, the next step is to focus on operators – actions that move the agent from one state to another. These operators define the transitions by establishing two things: preconditions, which outline what must be true before the action can occur, and postconditions (or effects), which describe what becomes true (or false) after the action has been applied.

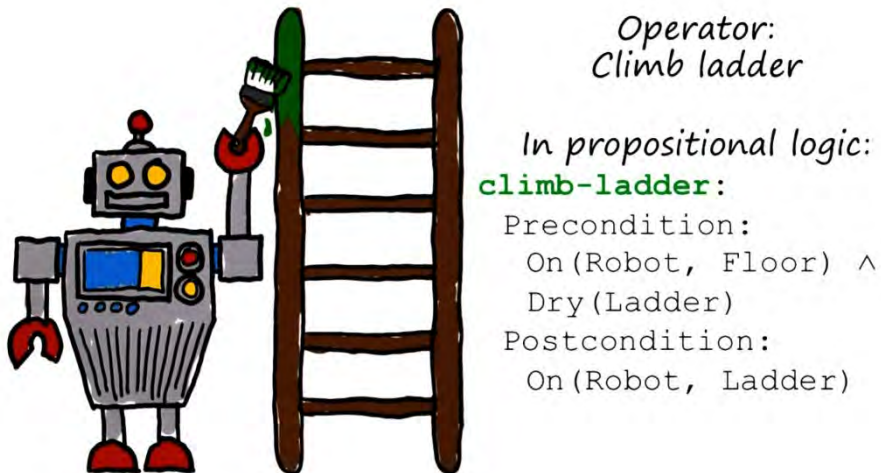


Figure 12-4 The climb-ladder operator

Take, for example, the operator climb-ladder. Here's how we could specify it in propositional logic:

- **Precondition:** $On(Robot, Floor) \wedge Dry(Ladder)$
- **Postcondition:** $On(Robot, Ladder)$ (and implicitly, $\neg On(Robot, Floor)$ if we use a more complete representation like STRIPS, which includes add and delete lists)

The precondition tells us that for the robot to execute the “climb-ladder” action, two conditions must be met: the robot must already be on the floor, and the ladder must be dry. Once those conditions are satisfied, the action can be executed, and the postcondition becomes true – the robot is now on the ladder.

There are several key points to note when specifying operators:

1. **Preconditions Use Positive Literals:** By convention in many classical planning systems (like STRIPS, which stands for Stanford Research Institute Problem Solver), the preconditions of an operator include only positive statements (literals).¹¹¹ In our example, the precondition is written as “*On(Robot, Floor) ∧ Dry(Ladder)*,” both of which describe positive conditions that must be true in the world for the robot to climb the ladder.
2. **Postconditions Can Include Negative Literals (Effects):** Unlike preconditions in the strict STRIPS sense, postconditions describe the changes to the world. These are often represented as an “add list” (literals that become true) and a “delete list” (literals that become false). So, after climb-ladder, *On(Robot, Ladder)* is added, and *On(Robot, Floor)* would be deleted. A literal like $\neg \text{Dry}(\text{Ladder})$ in the postcondition of paintladder means *Dry(Ladder)* is deleted and $\neg \text{Dry}(\text{Ladder})$ is added (or simply understood as the state changing).
3. **Application and Semantics of Operators:** The operator can only be applied when all of its preconditions are satisfied in the current state of the world. If one of the preconditions is not met (e.g., if the ladder isn’t dry), the action cannot be performed. Once applied, the postconditions modify the state of the world accordingly. In the case of the “climb-ladder” operator, if the preconditions are satisfied, the postcondition “*On(Robot, Ladder)*” will hold true, meaning the robot has successfully climbed the ladder.

The precise definition of operators - via preconditions and postconditions - ensures that the robot or intelligent agent makes logical decisions. The operator transitions the world from one state to another until the goal state is achieved. By clearly defining each operator, the planning system avoids mistakes, ensuring that each action aligns with the goal and no conflicting steps are taken.

Specifying Operators: Additional Examples

Now that we've learned how to specify an operator like climb-ladder, let's continue by applying the same logic to other operators in our scenario, such as descend-ladder, paint-ceiling, and paint-ladder. Each of these operators has preconditions and postconditions, much like the climb-ladder operator.

climb-ladder:

Precondition:
 On(Robot, Floor) $\wedge$
 Dry(Ladder)
 Postcondition:
 On(Robot, Ladder)

descend-ladder:

Precondition:

 Postcondition:

paint-ceiling:

Precondition:

 Postcondition:

paint-ladder:

Precondition:

 Postcondition:

Figure 12-5 Completing the pre- and post-conditions for the operators

Ashok David, what did you come up with for these operators?

climb-ladder:

Precondition:
 On(Robot, Floor) $\wedge$
 Dry(Ladder)
 Postcondition:
 On(Robot, Ladder)

descend-ladder:

Precondition:

 Postcondition:

paint-ceiling:

Precondition:

 Postcondition:

paint-ladder:

Precondition:

 Postcondition:

Figure 12-6 Completed pre- and post-conditions for the operators

David Well, the descend-ladder operator is similar to climb-ladder. In both cases, the ladder needs to be dry before any climbing action takes place, whether going up

or down. So, for descend-ladder, the preconditions are: $On(Robot, Ladder) \wedge Dry(Ladder)$. After descending, the robot will be on the floor, so the postcondition is: $On(Robot, Floor)$. It's essentially the reverse of climbing the ladder.

For the paint-ceiling operator, the robot needs to already be on the ladder in order to reach the ceiling, so the precondition is: $On(Robot, Ladder)$. After painting the ceiling, two changes occur: the ceiling is now painted, and it's no longer dry. So, the postconditions are: $Painted(Ceiling) \wedge \neg Dry(Ceiling)$, since the ceiling becomes wet after being painted.

As for the paint-ladder operator, the robot must remain on the floor to paint the ladder - standing on the ladder while painting it would be impractical. The precondition is: $On(Robot, Floor)$. After painting the ladder, the postconditions are: $Painted(Ladder) \wedge \neg Dry(Ladder)$, reflecting the fact that the ladder is now painted and wet.

Ashok That's correct, David. You've captured the core logic.

There are a few things to keep in mind here. First, propositional logic provides precision when specifying operators, making the requirements for these actions clear and unambiguous. This precision is what makes logic such a powerful tool in AI.

Second, it's important to note that preconditions often only include positive literals (statements about what is true), while postconditions may include both positive and negative literals (or add/delete effects). This means that while the preconditions simply state what must be true for an action to occur, the postconditions can describe what changes about the world as a result of the action, including what is no longer true.

David So, Ashok, why don't we use negative literals in the preconditions? Why can't we just say $\neg Painted(Ladder)$ instead of $Dry(Ladder)$?

Ashok That's an excellent question, David. It comes from the history of these notations. The convention of using only positive literals in preconditions originated with an early AI planning system known as STRIPS, developed in the late 1960s at Stanford Research Institute. STRIPS was one of the first planners, used by a robot called Shakey. The system used a restricted form of logic for operators, and it turned out that using only positive literals in the preconditions simplified the planning algorithms and theorem-proving process. This practice has been influential in AI ever since, stemming from the work on STRIPS and Shakey, though modern planners can often handle more expressive operator definitions.

PLANNING APPROACHES AND CHALLENGES

As we move forward with planning, it's crucial to understand how it differs from search and heuristic methods like Means-Ends Analysis, which we covered earlier in the book. In fact, the figure shown here may seem familiar, as it mirrors the one we used when discussing Means-Ends Analysis.

The Problem with Uninformed Search

Imagine this: starting from an initial state, you apply every operator that's applicable. Each operator generates a new state, and then you apply all the available operators to each of those states, expanding further and further. What happens next? A combinatorial explosion, where the number of states grows rapidly, creating an overwhelming number of possibilities, most of which are irrelevant. This is where uninformed search methods (like breadth-first or depth-first search without heuristics) struggle. They lack the ability to efficiently focus on what really matters.

To illustrate: think about planning a route in a city. If you considered every possible turn at each intersection, followed by every possible turn at the next intersection, and so on, you'd end up with far more routes than you'd ever need, many leading far from your destination. Search, in this sense, would be incredibly inefficient.

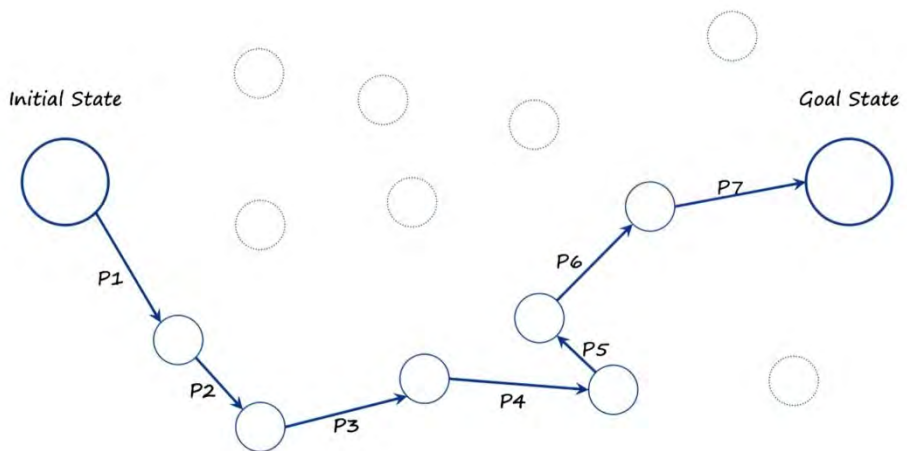


Figure 12-7 Planning using search

Control Knowledge: The Key to Efficient Planning

But when you actually plan a route, you don't try every possible turn, do you? Instead, you use goal knowledge. The goal tells you which turns to take at each intersection, guiding you toward your destination. This is where control knowledge comes into play. Control knowledge helps in selecting actions that push you closer to your goal, filtering out irrelevant paths.

There are two types of knowledge at work here:

- **Explicit knowledge:** Facts about the world, like knowing which streets lead where, or the preconditions and effects of operators.
- **Control knowledge:** Tacit or heuristic knowledge that guides the selection of actions or strategies to reach a goal efficiently.

Means-Ends Analysis: A Heuristic Approach to Control

Earlier in the book, we talked about Means-Ends Analysis as a method for using goals to guide action selection. In Means-Ends Analysis, you compare the current state to the goal state, identify the differences, and choose the operator that reduces the largest difference between the two states. This provides a way of using goals as control knowledge to guide the selection of the next action, but it's heuristic, not always systematic or guaranteed to find the optimal plan.

Planning: A Systematic Approach

Planning, particularly the methods we will discuss, offers a more systematic approach to action selection.¹¹² It doesn't rely solely on local heuristics but instead uses a structured framework for determining which operator to apply at each stage to construct a sequence of actions leading to the goal. And, if you think back to our earlier discussion on intelligent agents, we defined them as systems that map perceptual histories into actions. Since action selection is a key part of intelligent behavior, planning becomes a core problem to solve.

Let's look at the structure of a plan using the language of planning we've been developing throughout this chapter. A plan can be visualized as a sequence of states, each transformed by an operator that moves us closer to the goal.

Take a look at the illustration. On the left, you see a sequence of states leading from an initial state to a goal state. Each state is connected by operators like climb-ladder and paint-ceiling. To the right, we've expanded the operator paint-ceiling, showing its preconditions and postconditions.

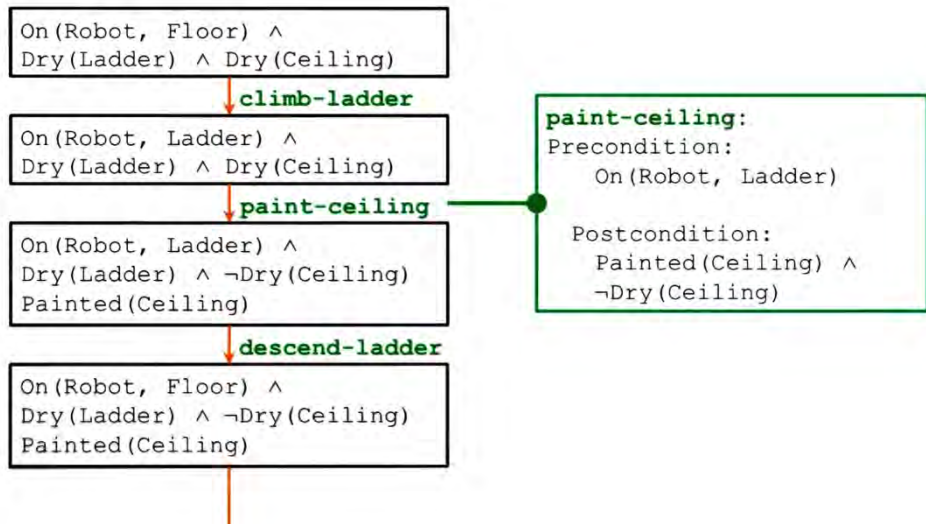


Figure 12-8 Representing a plan

Note something important here: the preconditions of an operator must match the current state for that operator to be applied. In this case, the precondition for paint-ceiling is $On(Robot, Ladder)$, which matches the postcondition of the previous operator, climb-ladder. This ensures the operator can be applied.

Once the operator is executed, the world moves to the next state, where the postconditions of paint-ceiling become true. For instance, $Painted(Ceiling)$ and $\neg Dry(Ceiling)$ now describe the world after the ceiling has been painted. This precise matching of preconditions and postconditions ensures a clear and logical progression of states.

The Challenge of Goal Clobbering in Simple Planners

Let's revisit the challenge presented by using Means-Ends Analysis in the scenario where a robot is tasked with painting both a ceiling and a ladder. The goal state is represented as the conjunction: $Painted(Ladder) \wedge Painted(Ceiling)$. At the outset, the initial state is: $On(Robot, Floor) \wedge Dry(Ladder) \wedge Dry(Ceiling)$.

Goal State:

Painted(Ladder) $\wedge$
Painted(Ceiling)

On(Robot, Floor) $\wedge$
Dry(Ladder) $\wedge$ Dry(Ceiling)

Figure 12-9 Planning with Means-Ends Analysis

Means-Ends Analysis would begin by evaluating the operators necessary to achieve these goals. For the ladder, the robot can apply the paint-ladder operator, which has the preconditions *On(Robot, Floor)* and *Dry(Ladder)*, both satisfied by the initial state. By executing this operator, the robot paints the ladder, reaching the state: *On(Robot, Floor) $\wedge$ $\neg$ Dry(Ladder) $\wedge$ Painted(Ladder) $\wedge$ Dry(Ceiling)*.

The planner now turns its focus to achieving the second goal of painting the ceiling. However, to paint the ceiling, the robot must first climb the ladder, requiring the application of the climb-ladder operator. Here lies the issue: the precondition for climbing the ladder is *Dry(Ladder)*, but since the ladder was just painted, this precondition is no longer satisfied. The ladder is now wet ($\neg$ *Dry(Ladder)*), creating a conflict that prevents the robot from climbing.

Goal State:

Painted(Ladder) $\wedge$
Painted(Ceiling)

On(Robot, Floor) $\wedge$
Dry(Ladder) $\wedge$ Dry(Ceiling)

$\downarrow$ paint-ladder

On(Robot, Floor) $\wedge$
 $\neg$ Dry(Ladder) $\wedge$ Dry(Ceiling)
 $\wedge$ Painted(Ladder)

paint-ladder:
Precondition:
On(Robot, Floor)
Postcondition:
Painted(Ladder) $\wedge$
 $\neg$ Dry(Ladder)

climb-ladder:
Precondition:
On(Robot, Floor) $\wedge$
Dry(Ladder)
Postcondition:
On(Robot, Ladder)

Figure 12-10 Goal pursuit with Means-Ends Analysis

At this point, Means-Ends Analysis has painted the ladder but rendered it impossible for the robot to achieve the second goal of painting the ceiling, at least until the ladder dries. This

type of conflict – where the pursuit of one goal prevents the achievement of another – is called “goal clobbering.”¹¹³ In this example, the act of painting the ladder introduced a condition ($\neg \text{Dry}(\text{Ladder})$) that interfered with the robot's ability to pursue the second goal of painting the ceiling.

This example highlights a significant limitation of Means-Ends Analysis and other simple, linear planners. Linear planners treat goals as independent and attempt to achieve them one at a time without considering how the steps taken to achieve one goal might negatively impact the achievement of another. In our scenario, this led to an inefficient plan where the first goal (painting the ladder) clobbered the second goal (painting the ceiling).

The fundamental problem here is that linear planners, including Means-Ends Analysis, do not reason about conflicts between goals. They don't anticipate that actions taken to achieve one goal may introduce conditions that block the achievement of other goals. This lack of foresight results in plans that either fail outright or require significant rework, like waiting for the ladder to dry before the robot can continue with its task.

To solve this, we need a more sophisticated approach to planning – one that can systematically reason about how different goals interact and ensure that achieving one goal doesn't interfere with another. This leads us into more advanced planning techniques, where conflicts like goal clobbering are anticipated and avoided by carefully sequencing actions. These techniques, unlike linear planning, can reason about the interactions between different goals and organize operators in a way that prevents conflicts from arising in the first place.

PARTIAL-ORDER PLANNING (POP)

Partial-Order Planning (POP) becomes essential when dealing with multiple goals, especially in cases where the plan to achieve one goal interferes with or “clobbers” the progress toward another. This need for conflict management brings us to POP, which is also referred to as nonlinear planning. The ability to manage the sequence and relationship of actions is what separates POP from simpler linear approaches.

A Real-Life Example of Partial-Order Planning

Ashok David, can you think of an example of partial-order planning?

David Actually, I have a funny example. Recently, we bought a couch, and to make things easier, we paid the delivery team to assemble it for us. Once it was fully assembled, we realized it wouldn't fit through the apartment door! So, we had to disassemble the couch completely, move the parts inside one by one, and reassemble it inside the apartment. In this case, the plan to assemble the couch clobbered the goal of getting it inside the apartment.

Ashok Good example. Thank you, David.

This example highlights exactly why POP is needed – to detect conflicts and avoid these kinds of setbacks.

Applying Partial-Order Planning to the Ladder and Ceiling Problem

Let's now return to the example of painting the ladder and ceiling. Imagine the robot tasked with completing both tasks, starting from an initial state where the robot is on the floor, and both the ladder and ceiling are dry.

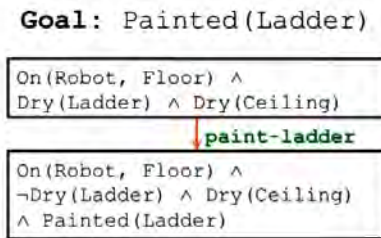


Figure 12-11 Partial Order Planning example

The goals are:

- Goal 1: *Painted(Ladder)*
- Goal 2: *Painted(Ceiling)*

We'll begin by focusing on how the robot achieves the first goal of painting the ladder. To select the right operator, we use goal knowledge as control knowledge. This approach helps the robot prioritize and choose between available operators in the world.

Step 1: Achieving Painted(Ladder)

In this case, the only operator that satisfies the goal *Painted(Ladder)* and whose preconditions align with the initial state is paint-ladder. So, we'll select this operator and apply it to the initial state, resulting in a new successor state:

- Postconditions of paint-ladder: *Painted(Ladder)* and $\neg$ *Dry(Ladder)*
- Propagated conditions from the initial state: *On(Robot, Floor)* and *Dry(Ceiling)*

Since the operator paint-ladder only affects the ladder, the conditions *On(Robot, Floor)* and *Dry(Ceiling)* remain unchanged. This is important because the paint-ladder operator does not specify anything about the robot's location or the state of the ceiling.

Now, let's move on to the second goal: *Painted(Ceiling)* as we rejoin David's and Ashok's conversation.

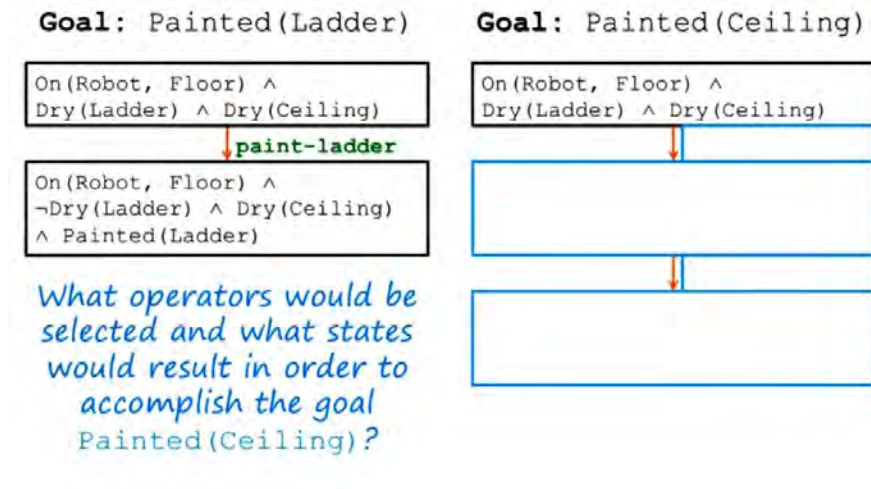


Figure 12-12 Planning for the Painted(Ceiling) goal

Ashok David, how would you achieve the goal?

David The first thing the robot will have to do is climb the ladder, so I applied the climb-ladder operator. This results in *On(Robot, Ladder)*, but nothing else changes yet. After that, the robot will apply the paint-ceiling operator, resulting in *Painted(Ceiling)* and $\neg$ *Dry(Ceiling)*.

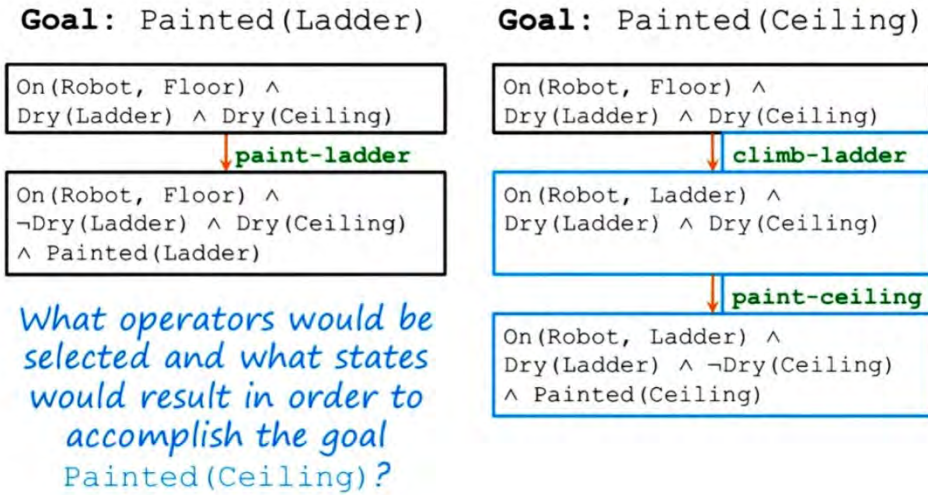


Figure 12-13 Completed plan for Painted(Ceiling)

Ashok That's good, David.

Now that we've seen the structure of this plan, the next question is: How does a robot come up with such a plan?

Working Backwards: A Key Feature of Partial-Order Planning

One of the key techniques in partial-order planning is working backwards from the goal to select the right operators.¹¹⁴ This is what makes POP powerful – it allows the robot to systematically reason through operator selection by focusing on the end goal.

Let's take the goal *Painted(Ceiling)* as an example. To achieve this goal, we need an operator whose postcondition matches *Painted(Ceiling)*. In this case, the operator is paint-ceiling. However, paint-ceiling has a precondition: *On(Robot, Ladder)*. This precondition doesn't match the initial state, which is *On(Robot, Floor)*. So, the robot needs to achieve the subgoal *On(Robot, Ladder)* first.

To achieve the subgoal, the robot selects the climb-ladder operator, whose postcondition is *On(Robot, Ladder)*. Since the preconditions of climb-ladder match the initial state, the robot can apply this operator, followed by paint-ceiling.

Thus, the robot constructs the following plan:

1. Apply climb-ladder to achieve *On(Robot, Ladder)*
2. Apply paint-ceiling to achieve *Painted(Ceiling)*

This systematic method of working backwards from the goal, using goal knowledge as control knowledge, ensures that the robot selects the correct operators in the right order..

Connecting Back to Problem Reduction

As explained above, partial-order planning has strong ties to problem reduction, a concept introduced earlier when discussing Means-Ends Analysis. Problem reduction involves breaking down a large problem into smaller, manageable subgoals, much like we did with the robot needing to climb the ladder before painting the ceiling.

When we first talked about problem reduction, we focused on breaking big problems into smaller ones. Here, with Partial Order Planning, we see a practical application of how an agent can identify those subgoals – like climbing the ladder – on its way to achieving the larger goal of painting the ceiling. By identifying and solving smaller subproblems in the right order, the agent is able to avoid the goal-clobbering conflicts that arise in simpler, linear planning methods.

Partial-Order Planning is a powerful tool that ensures intelligent agents can systematically work through multiple goals without creating conflicts between them, all while achieving the final objective in an efficient manner.

Conflict Detection in Partial-Order Planning

So far, the partial-order planner has treated the two goals – painting the ladder and painting the ceiling – as independent tasks, creating partial plans for each without recognizing the potential conflicts between them. However, the next step is critical: examining the relationship between these plans to detect and resolve any conflicts that may arise.

Let's begin by considering how the partial-order planner evaluates the situation. As shown in the figure, we have Plan A (to paint the ladder) and Plan B (to paint the ceiling). The partial-order planner needs to examine each precondition of the operators in both plans. If a precondition for an operator in Plan A is clobbered (made false) by an effect (postcondition) of an operator in Plan B (or vice versa), then a conflict is identified.

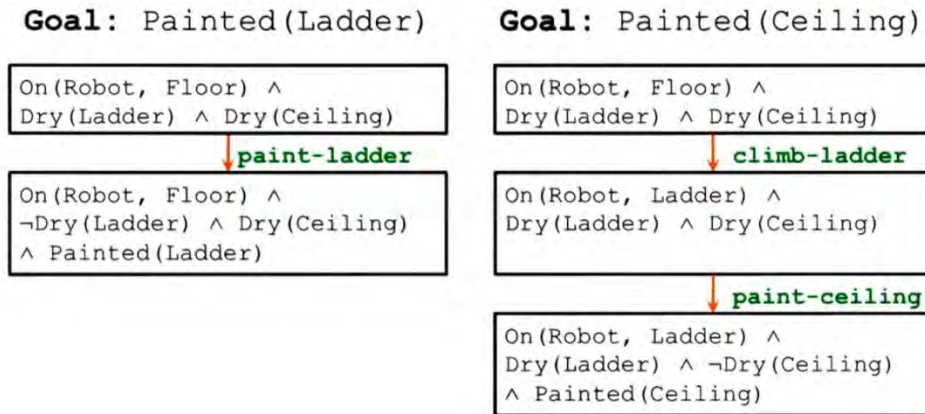


Figure 12-14 Detecting conflicts (goal 1 - left, goal 2, right)

The conflict detection algorithm is simple: for each precondition of an operator in the current plan, if that precondition is threatened (i.e., an action in another part of the plan could make it false before it's needed), the planner identifies the conflict. Once the conflict is detected, the planner can resolve it by adding ordering constraints (promoting or demoting goals/actions) or by other means to ensure that the clobbered precondition can be satisfied before another action invalidates it. In essence, if one plan interferes with another, the planner reorders the goals to ensure that operators are executed in a sequence that avoids the conflict.

Detecting Conflicts

For each precondition in current plan:

If precondition for an operator in the current plan is clobbered by a state in another plan:

Promote current plan's goal above other plan's goal

Figure 12-15 Algorithm for detecting and addressing goal conflicts

Now, let's explore how this process works in our ladder and ceiling example. First, the planner evaluates Plan A (painting the ladder) and checks whether the preconditions of the operator paint-ladder are clobbered by any states in Plan B (painting the ceiling). In this case, there is no conflict with the preconditions for painting the ladder that Plan B would cause before paint-ladder executes.

Next, the planner turns its attention to Plan B. It checks the operator climb-ladder, which has the precondition $On(Robot, Floor) \wedge Dry(Ladder)$. As the planner compares this precondition with the effects in Plan A, it identifies a conflict: if paint-ladder (from Plan A) is executed before climb-ladder (from Plan B), then the effect $\neg Dry(Ladder)$ from paint-ladder contradicts the precondition $Dry(Ladder)$ required for the climb-ladder operator in Plan B.

This is where the partial-order planner's conflict resolution comes into play. The planner must add an ordering constraint to resolve the conflict. It could order Plan B (painting the ceiling) to occur before Plan A (painting the ladder).

Another issue arises: if Plan B (paint ceiling) is executed first, its final state includes $On(Robot, Ladder)$. However, the precondition for paint-ladder (from Plan A) is $On(Robot, Floor)$. This "open condition" means the required condition for paint-ladder has not been met if Plan B simply finishes. The planner must ensure the robot descends the ladder before it can apply paint to the ladder. This is not a conflict between the plans in the same way as the ladder-drying issue, but rather a gap that needs filling to connect the two sub-plans if Plan B is to precede Plan A.

By systematically detecting conflicts and adding ordering constraints or new actions to resolve open conditions, the partial-order planner ensures that all preconditions are satisfied without clobbering one another, leading to a final plan where both goals – painting the ladder and painting the ceiling – are successfully achieved.

Plan Completion in Partial-Order Planning

In our previous discussion on detecting conflicts, we saw how the partial-order planner identifies and resolves conflicts between goals by promoting or demoting them based on their preconditions. Let's now focus on the final steps of plan development, resolving the remaining open condition, and completing the plan.

As we recall, the partial-order planner detected a conflict between the two goals: painting the ladder and painting the ceiling. If paint-ladder happens before climb-ladder, the ladder becomes wet, clobbering climb-ladder's precondition. To resolve this conflict, the planner decided to promote the goal of painting the ceiling (Plan B) to occur before painting the ladder (Plan A). However, when attempting to connect these two plans in that order, the planner encountered an open condition problem. Specifically, after Plan B (ending with the robot $On(Robot, Ladder)$), the precondition $On(Robot, Floor)$ for paint-ladder (from Plan A) was not met.

To address this, the planner must now select an operator whose preconditions align with the state after Plan B ($On(Robot, Ladder)$), and whose post-conditions satisfy the desired precondition for Plan A ($On(Robot, Floor)$). The only operator available to satisfy these conditions is descend-ladder. By choosing this operator, the planner ensures the robot descends from the ladder after completing the task of painting the ceiling, thereby positioning the robot correctly on the floor to paint the ladder next.

With descend-ladder inserted into the plan, we now have a complete and functional plan. The final plan is structured as follows:

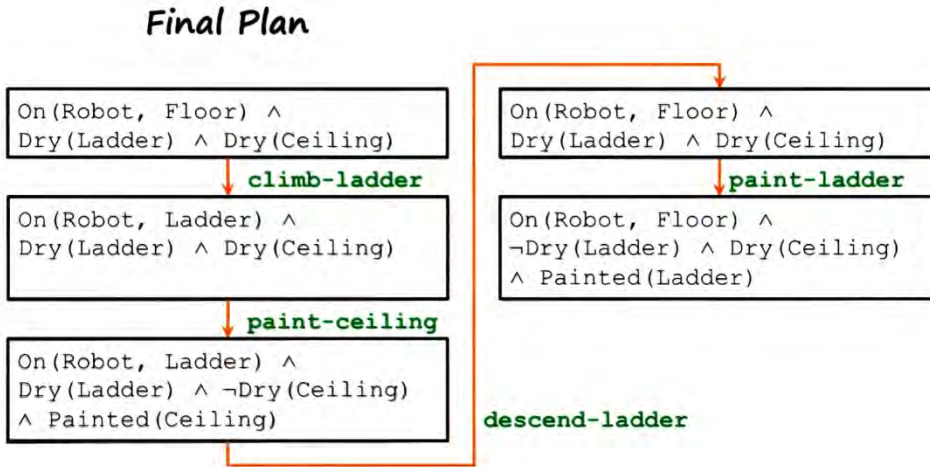


Figure 12-16 Final plan with Climb Ladder, Paint Ceiling, and Descent Ladder

In this final plan, the robot successfully accomplishes both goals: painting the ceiling and painting the ladder, without any conflicting preconditions left unresolved. The partial-order planning process is now complete, having navigated and resolved all conflicts in goal achievement.

Intelligence through Partial-Order Planning

Now that we've explored how the partial-order planning algorithm operates in practice, what insights can we derive from this process about intelligence itself?

Let's explore a few key postulates that emerge:

1. **Knowledge is more than just facts about the world.** In this context, knowledge also encompasses control knowledge – the tacit understanding of how to choose between various operators to achieve goals. In our planning process, control knowledge enabled

the planner to select between operators like paintladder and paintceiling, guiding the robot towards its objectives. This shows that control knowledge plays a central role in intelligent decision-making.

2. **Goals provide essential control knowledge.** Goals are not just end states; they act as a guide in the decision-making process, helping to select the most appropriate operators. For instance, the goals of Painted(Ladder) and Painted(Ceiling) served as control knowledge that informed which operators – paintladder or paintceiling – were most relevant for the robot's next steps.
3. **Small abilities working together create complex intelligence.** Partial-order planning can be understood as an interaction between several specialized agents or abilities. Each of these “agents” is responsible for a specific, small task. For example, one agent generates a plan for each goal independently, another agent detects conflicts between those plans, and yet another resolves those conflicts. Though these agents are simple in isolation, their coordinated efforts produce the overall intelligent behavior that solves the problem.

This concept closely mirrors Marvin Minsky’s theory of the Society of Mind.¹¹⁵ Minsky proposed that intelligence arises from the interaction of many smaller, simpler agents working together. In our example, one agent handles conflict detection, another handles conflict resolution, and yet another constructs basic plans for achieving simple goals. These small tasks, when combined, give rise to sophisticated problem-solving capabilities.

The Power of Explicit Problem-Solving

When humans solve problems like painting a ladder and ceiling, the process seems intuitive and effortless. However, this perceived simplicity can be deceptive. What artificial intelligence does is make this process explicit. To write a computer program that solves the same problem, we must specify every operator, precondition, state, and goal with exact precision.

By building AI agents that can tackle such problems, we make the implicit processes humans use explicit. This allows us to generate hypotheses about how human cognition might work. It is through this process that AI not only helps us develop smarter machines but also provides powerful insights into how humans solve complex problems.

POP Example: The Blocks World

Now that we've seen partial-order planning in action, let's practice with a series of exercises to solidify our understanding. The problem we'll work on is one we've encountered earlier: the blocks world. This problem involves moving blocks from an initial state to a goal state, only moving one block at a time.

The figure above shows the initial and goal states. Our task is to figure out how to transition between these states using logical reasoning and partial-order planning. Write the initial and goal states in propositional logic to clearly define the problem.

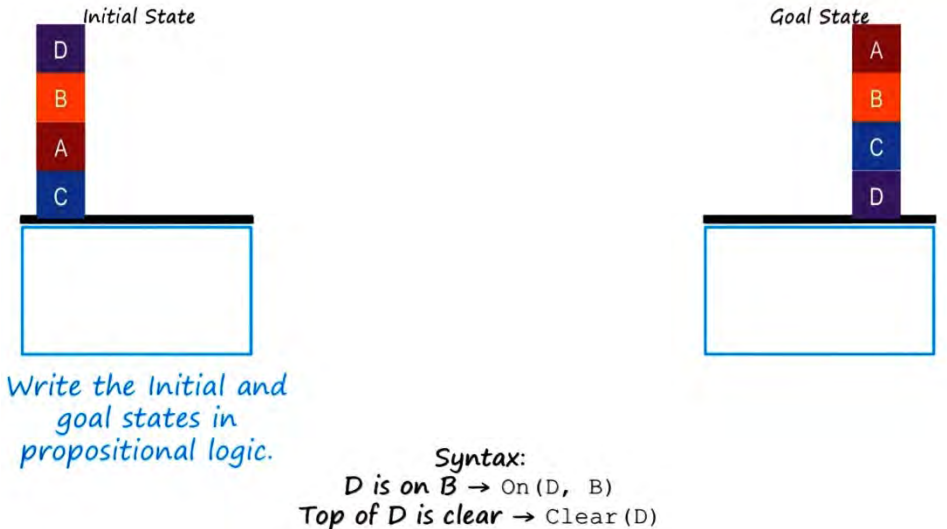


Figure 12-17 Partial Order Planning with blocks world

Ashok What did you get, David?

David So, our initial state is as follows: D is on B, B is on A, A is on C, and C is on the table. The goal state rearranges these blocks into alphabetical order: A is on B, B is on C, C is on D, and D is on the table.

Although this may seem simple for us as humans to solve, where we know immediately to move the blocks in a certain order, the challenge is in writing an AI program that can do the same. By writing out the operators and the conditions they rely on, we can gain insights into human intelligence and problem-solving by making these processes explicit.

There are only two operators in this blocks world:

1. Move block x to block y : $\text{Move}(x, y)$
2. Move block x to the table: $\text{Move}(x, \text{Table})$

Note two important things here. First, we use variables such as x and y to represent blocks, rather than naming specific blocks like A, B, or C. This allows the operators to be more general. Second, for the operator to work, both block x (the one being moved) and block y (the

destination, if it's another block) must be clear – meaning nothing is on top of them. With this setup in mind, we can define the preconditions and postconditions of these operators.

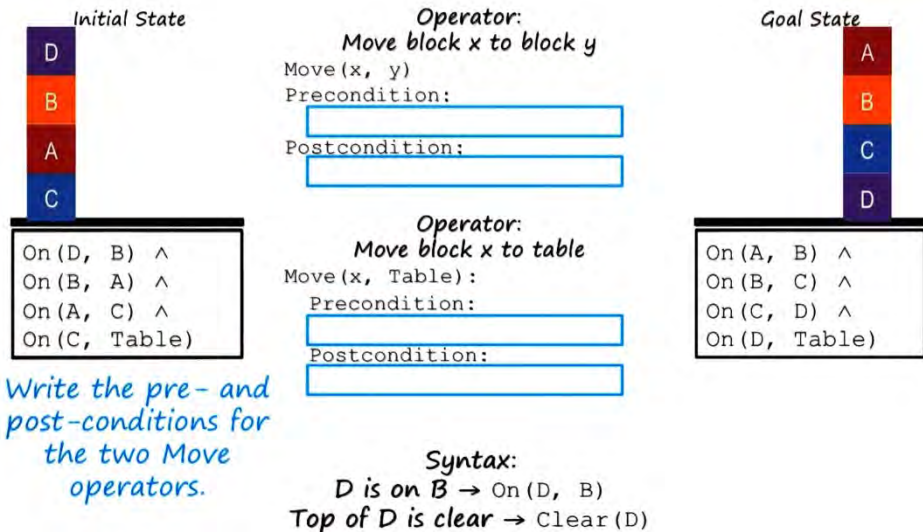


Figure 12-18 Partial Order Planning - Operator Conditions Example

Ashok What did you write down, David?

David The precondition for moving block x to block y is that both x and y must be clear. We can't move x if there's something on top of it, and we can't place it on y if y already has something on top of it. The postcondition is that x will now be on y. For moving block x to the table, the precondition is simply that x must be clear, and the postcondition is that x will be on the table.

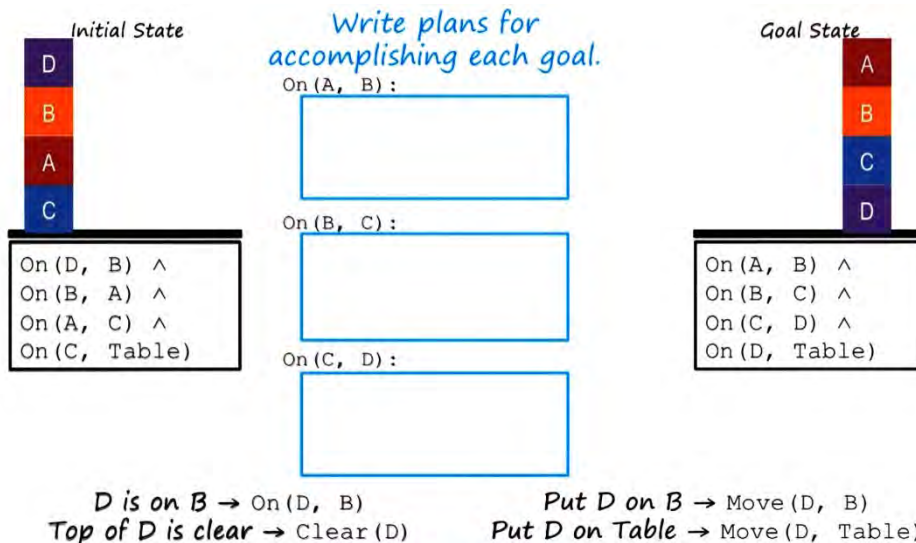


Figure 12-19 Partial Order Planning - Plan development

Ashok Now that we have our operators defined, let's break down the problem into three main goals from the goal state: putting A on B, B on C, and C on D. (The goal D on Table is often achieved implicitly or as a starting point for other actions). Write down the plan for each of these goals as if they were independent of each others.

David For the goal of putting A on B, the plan is to move D to the table, move B to the table, and then place A on top of B. For the second goal, putting B on C, we start by moving D to the table, then B, then A, and finally place B on top of C. For the third goal, placing C on D, we move D, B, and A to the table before putting C on top of D.

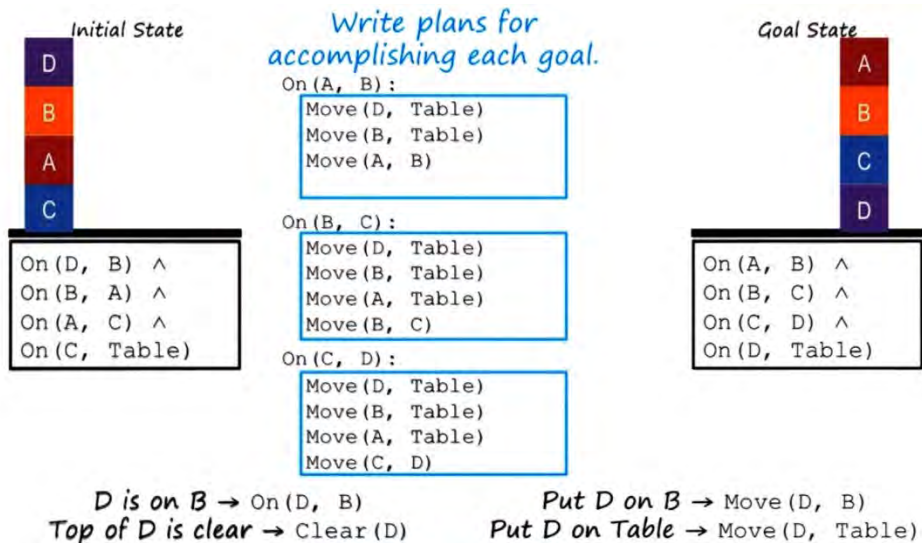


Figure 12-20 Partial Order Planning - Completed Individual plan development

Now that we've written plans for each goal, we need to identify any conflicts between them. This is where partial-order planning becomes essential, as it helps detect and resolve conflicts between goals.

Ashok What conflicts did you find, David?

David Each plan conflicts with the next if executed in the order A on B, then B on C, then C on D. For example, the postcondition of the plan to move A onto B is that A is on B. But for the plan to move B onto C, B needs to be clear, and it won't be if A is on top of it. Similarly, once B is on C, C will no longer be clear, so we can't place C onto D. To resolve these conflicts, we need to reorder the goals: first achieve C on D, then B on C, and finally A on B.

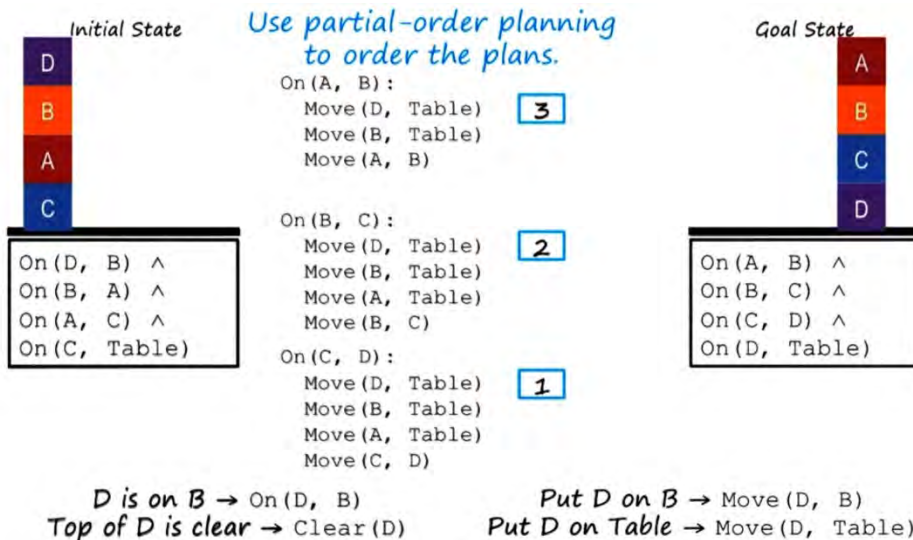


Figure 12-21 Partial Order Conflicts - Resolving conflicts

Ashok Good. Now that we've identified the conflicts and reordered the goals, write down the final plan for achieving the goal state.

David First, we achieve C on D. To do this, we move D to the table (if it's not already there and clear), then B to the table, then A to the table. Then place C on D. Once that's done, many of the steps required for moving B onto C have already been completed (A and B are on the table, D has C on it). So, we place B on top of C. Finally, since most steps for moving A onto B have also been completed (A is on the table, C has B on it), we can go ahead and place A onto B directly.

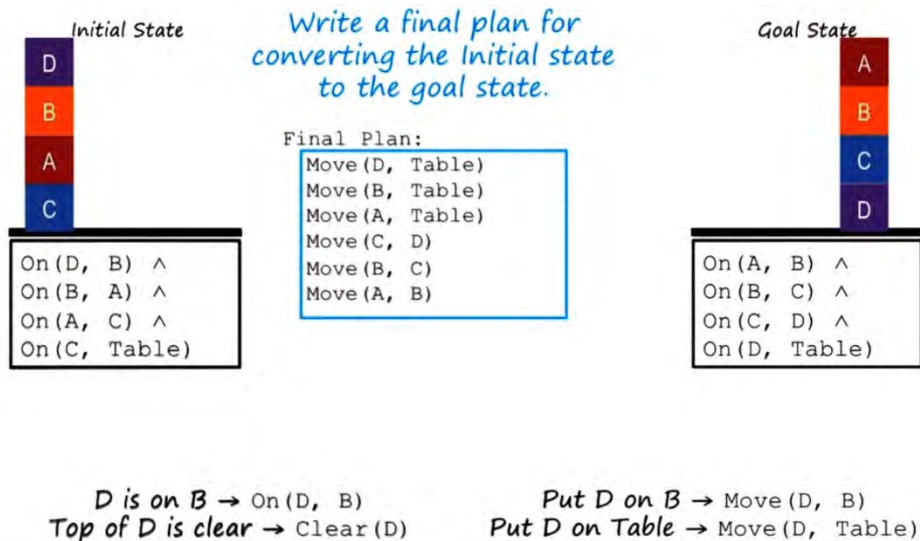


Figure 12-22 Partial Order Planning - Final plan

Ashok Well done, David.

This exercise demonstrates the power of partial-order planning. When we attempted this problem earlier using means-ends analysis, we ran into difficulties because different plans interfered with each other. Now, using partial-order planning, we can avoid those issues by reordering goals and resolving conflicts systematically.

One of the great strengths of partial-order planning is that it makes implicit human reasoning explicit.¹¹⁶ By breaking the problem into smaller tasks and specifying the necessary steps and conditions, AI can mimic human problem-solving processes with precision. This gives us valuable insights into how human intelligence might work and how we can replicate it in machines.

HIERARCHICAL TASK NETWORK (HTN) PLANNING

Our next topic in planning is hierarchical planning, a method that simplifies complex problems by breaking them into smaller, more manageable components.¹¹⁷ To represent this process, we use a formalism called a Hierarchical Task Network (HTN), which enables reasoning at multiple levels of abstraction.¹¹⁸

Let's revisit our blocks world to understand HTN better. Here, we're faced with a more complicated initial state and goal state than we've seen before (see image below for the initial state, left, and goal states, right). Moving from this initial state to the goal state involves a series

of operations. If we use partial-order planning, we could generate a detailed step-by-step plan that eventually reaches the goal state.

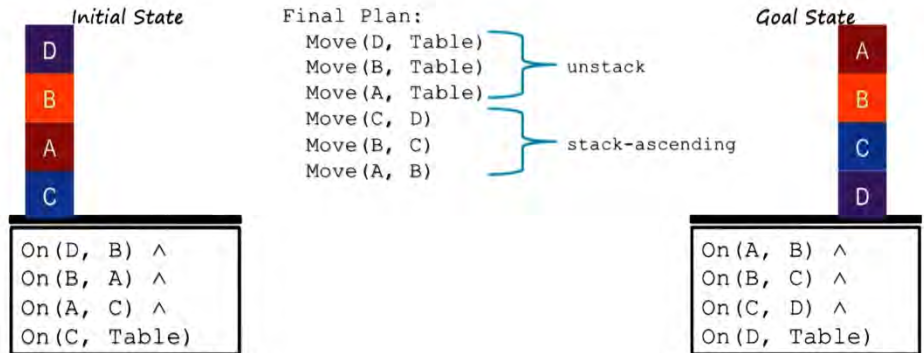


Figure 12-23 Planning with hierarchical task networks - block world example

However, as shown in the above image (center), the final plan derived using this method is long and filled with numerous operations. This raises an important question: can we simplify the plan by abstracting groups of low-level actions into higher-level macro operations? By thinking at a higher level of abstraction, we make the problem space much smaller and simpler, allowing us to navigate it more easily.

Macro Operators: Simplifying the Problem

To illustrate this idea, let's zoom in on a smaller part of the problem. Consider the simple task of rearranging a set of blocks. In this case, we want to move from an initial configuration to a goal configuration using as few moves as possible. Rather than listing each individual operation, we can group several of them into macro operators.



D is on B $\rightarrow$ On (D, B)
Top of D is clear $\rightarrow$ Clear (D)

Put D on B $\rightarrow$ Move (D, B)
Put D on Table $\rightarrow$ Move (D, Table)

Figure 12-24 Grouping operations into macro operators

For example, the task of unstacking blocks from a stack and placing them on the table can be represented by a macro operator called unstack. Similarly, stacking blocks in a particular order can be represented by the stack ascending macro operator. These higher-level operations encapsulate several smaller moves, making it easier to reason about the task. As shown in the image above, the final plan for this simpler block problem uses these two macro operators.

The specifications for these macro operators are outlined in Figure 12-25. The macro operator unstack includes preconditions and postconditions that describe how blocks should be moved from a stack onto the table. Similarly, the stack ascending operator includes the conditions for stacking the blocks in a particular order. These macro operators represent abstractions of the underlying move operations, and they can be expanded into the detailed steps when necessary.

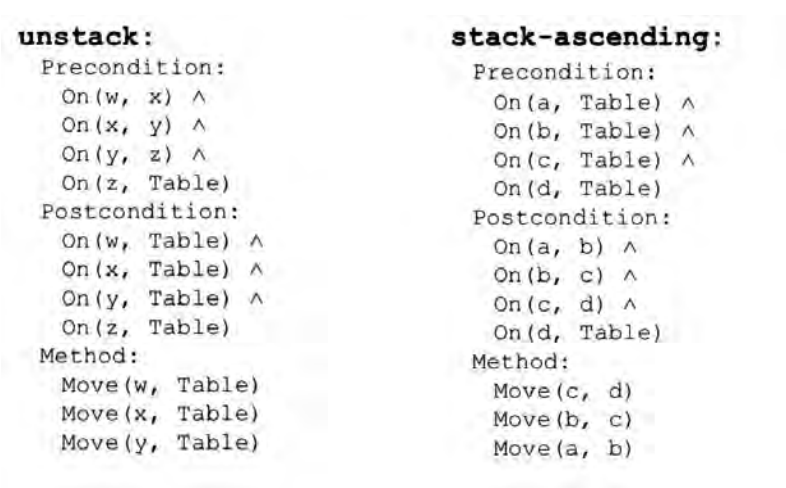


Figure 12-25 Specification of the unstack and stack-ascending macro operators

Abstraction and Intelligence

This approach to planning reveals an important aspect of intelligence – both in humans and artificial agents. Hierarchical reasoning enables us to manage complex tasks by breaking them down into simpler subtasks and reasoning at multiple levels of abstraction. Instead of getting bogged down in low-level details, we focus on the larger picture and use higher-level abstractions to simplify the problem.

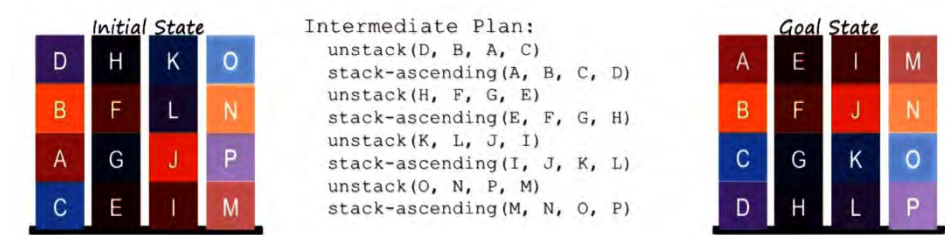


Figure 12-26 Abstraction at the macro operators level

For hierarchical reasoning to be effective, agents need knowledge at multiple levels of abstraction. In our example, the agent not only knows how to move individual blocks, but also understands how to perform higher-level tasks like unstacking and stacking in ascending order.

This multi-level knowledge allows the agent to tackle large, complex problems more efficiently by organizing them into manageable chunks.

This ties back to a fundamental concept in knowledge-based AI: intelligent agents use knowledge to solve complex problems. By reasoning at multiple levels of abstraction, they reduce the complexity of the problem space and can solve even the most difficult tasks. This is the power of Hierarchical Task Network (HTN) planning.

PLANNING AND COGNITION

Planning is a fundamental process in cognition, intricately tied to the selection of actions, which lies at the heart of intelligent behavior.¹¹⁹ Human beings are constantly faced with decisions that require careful consideration of various courses of action. Whether the decision involves simple daily activities – Where should one go for dinner? What should be cooked? How should it be prepared? – or more complex life choices like, What should be done with an unexpected bonus? Should one save it or indulge in a vacation, and if so, where and how should that vacation unfold? Each scenario exemplifies the core problem of action selection. To navigate these moments of choice, planning is essential.

For cognitive agents, the challenge often goes beyond just choosing actions for a single goal. These agents typically juggle multiple, often competing objectives. For example, a professor may aim to deliver effective instruction while simultaneously harboring personal goals, like achieving financial security. However, the professor may recognize the inherent limitations of academia when it comes to accumulating wealth. This underscores an important reality: cognitive agents pursue numerous goals at once, some of which may be synergistic, while others may create conflicts.

These goals can interact in both positive and negative ways. In cases of positive interaction, achieving one goal can facilitate the accomplishment of another. For instance, if someone decides to cook a meal, completing this task not only satisfies the immediate goal of preparing dinner but also furthers a long-term goal, such as developing culinary expertise. On the other hand, interactions between goals can also be negative, resulting in conflicts. A common example is the tension between pursuing a demanding career while also dedicating sufficient time to personal relationships or self-care.

A hallmark of cognitive agents is their ability to detect and navigate these conflicts. When goals align positively, agents can seize the opportunity to achieve multiple outcomes efficiently. Conversely, when goals conflict, agents must adjust their plans, reprioritize, or find compromises to avoid frustration or failure. In this way, planning becomes a central mechanism for balancing multiple objectives, guiding agents toward the most effective paths to success.

The interplay between planning and cognition reveals the complexity of human thought. Planning is not just about solving isolated problems – it is about managing the interconnections between numerous goals, recognizing when they can reinforce one another or when they compete for resources, such as time and attention. Cognitive agents must skillfully navigate these

intersections, continually adapting their strategies and selecting actions that help fulfill their broad spectrum of goals.

Moreover, the ability to plan highlights key characteristics of intelligence: adaptability, problem-solving, and foresight. Whether planning a meal, organizing a vacation, or making major career decisions, the cognitive processes involved reflect a nuanced ability to handle both immediate needs and long-term aspirations. Effective planning requires not only an awareness of the current situation but also an understanding of how different actions impact the achievement of multiple goals.

In conclusion, planning plays a pivotal role in cognition because it enables agents to select actions that balance competing priorities, avoid conflicts, and leverage synergies. It helps them navigate the dynamic complexity of real-world situations by providing a framework for achieving multiple, often overlapping, goals. Through planning, cognitive agents demonstrate their ability to adapt to challenges, resolve conflicts, and ultimately act in ways that are both strategic and intelligent.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

The classical planning techniques explored in this chapter, such as representing problems in logic, Partial-Order Planning (POP), and Hierarchical Task Network (HTN) Planning, provide foundational concepts that continue to inform and enhance modern AI paradigms.

- **Reinforcement Learning (RL):** While RL agents learn policies (which are essentially plans) through trial-and-error interaction, KBAI planning concepts offer valuable integrations.
 - **Model-Based RL:** Agents can learn a model of the environment's dynamics (akin to KBAI operators and state transitions) and then use planning algorithms (sometimes derived from classical planning) to compute optimal policies within that learned model.
 - **Hierarchical Reinforcement Learning (HRL):** This directly mirrors HTN planning. HRL agents learn policies at multiple levels of abstraction, breaking down complex tasks into sequences of simpler sub-tasks or options. This makes learning more efficient and policies more interpretable.
 - **Reward Shaping & Goal Specification:** Logical representations from KBAI can be used to define complex goals or to shape reward functions in RL, guiding the agent towards desired behaviors more effectively than sparse rewards alone.
- **Automated Planning in Robotics and Autonomous Systems:** Classical planning languages like PDDL (Planning Domain Definition Language) and algorithms based on POP or HTN principles are still actively researched and applied in robotics, logistics,

autonomous vehicles, and space exploration. These systems require robust, verifiable plans to operate safely and effectively in complex real-world environments.

- **Large Language Models (LLMs) for Planning:** LLMs are increasingly being explored for their potential in planning:
 - **Plan Generation from Natural Language:** LLMs can interpret high-level goals described in natural language and generate sequences of actions or sub-goals.
 - **Commonsense Reasoning for Planning:** LLMs possess vast amounts of commonsense knowledge that can inform the feasibility and appropriateness of planned actions, helping to fill gaps in formal models.
 - **Plan Verification and Refinement:** KBAl's logical frameworks can be used to verify the logical consistency or feasibility of plans generated by LLMs. LLMs can also be prompted to refine or critique plans.
 - **Tool Use:** Agentic LLMs often need to plan sequences of tool invocations to solve problems, a task that directly involves planning.
- **Neuro-Symbolic AI for Planning:** This area seeks to combine the strengths of neural networks (e.g., for learning from perception, or learning heuristic guidance) with symbolic planning methods (for structured reasoning, guarantees, and explainability). For example, a neural network might identify relevant objects and predicates from sensor data, which are then fed into a symbolic planner.
- **Explainable AI (XAI):** Plans generated by KBAl methods, with their explicit representation of states, goals, operators, and causal links, are inherently more interpretable than policies learned by many “black-box” machine learning models. This makes it easier to understand *why* an agent chose a particular sequence of actions, which is crucial for debugging, trust, and safety.
- **Agentic AI Architectures:** Sophisticated AI agents, whether single or multi-agent systems, rely heavily on planning to achieve long-term goals, coordinate actions, manage resources, and adapt to dynamic environments. The principles of goal decomposition (problem reduction), conflict resolution (as in POP), and hierarchical abstraction (HTN) are fundamental building blocks for such agent architectures.

The structured reasoning, explicit knowledge representation, and goal-directedness inherent in KBAl planning provide a powerful complement to modern data-driven AI. As AI systems tackle increasingly complex tasks requiring foresight, coordination, and understanding, the integration of these classical planning principles will be essential for developing robust, reliable, and intelligent solutions.

CHAPTER SUMMARY

In this chapter, we explored the role of planning in artificial intelligence and cognition, examining how agents plan actions to achieve goals. We began by discussing the foundations of planning

using formal logic, focusing on how states (initial, goal, intermediate) and operators (with preconditions and postconditions) interact to drive decision-making processes.

We then moved into understanding the importance of detecting and resolving conflicts that may arise between multiple goals or actions, introducing the concept of Partial-Order Planning (POP). POP was shown to be particularly useful when dealing with complex problems where the plan for one goal could potentially interfere with another (goal clobbering). By allowing for flexibility in the order of actions and employing techniques like working backwards from goals, POP helps detect and avoid conflicts, ensuring that multiple goals can be achieved coherently. The Blocks World problem served as a key example to illustrate POP's mechanisms.

Next, we examined Hierarchical Task Networks (HTNs), which enable hierarchical planning. HTNs help simplify complex problems by organizing actions at different levels of abstraction. Instead of focusing solely on low-level actions, we can group these into higher-level macro-operators (e.g., unstack, stack ascending). This hierarchical structure mirrors how intelligent agents approach complex problems in the real world, allowing them to reason across multiple levels of abstraction and manage large problem spaces more effectively.

Throughout the chapter, we emphasized how planning is central to cognition, as it addresses the problem of action selection – a constant in everyday decision-making. We highlighted how cognitive agents juggle multiple goals, often with conflicting demands, and how planning helps to detect and resolve these conflicts. This capability to manage multiple, sometimes conflicting, goals is a hallmark of intelligent behavior.

Finally, we discussed the integration of KBAI planning principles with modern AI paradigms such as Reinforcement Learning, LLMs, Neuro-Symbolic AI, and Agentic AI, underscoring the enduring relevance of structured planning in contemporary AI research and applications. The ability of classical planning to provide interpretable and verifiable solutions makes it a valuable component in building trustworthy AI systems.

ADDITIONAL READINGS & REFERENCES

- 110 Bercher, P., Höller, D., Behnke, G. & Biundo, S. in *ECAI 2016* 225-233 (IOS Press, 2016).
- 111 Fikes, R. E. & Nilsson, N. J. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial intelligence* **2**, 189-208 (1971).
- 112 Ghallab, M., Nau, D. & Traverso, P. *Automated planning and acting*. (Cambridge University Press, 2016).
- 113 Liu, P. in [1990] *Proceedings of the 2nd International IEEE Conference on Tools for Artificial Intelligence*. 2-8 (IEEE).
- 114 Bercher, P., Keen, S. & Biundo, S. in *Proceedings of the International Symposium on Combinatorial Search*. 35-43.
- 115 Minsky, M. *Society of mind*. (Simon and Schuster, 1986).
- 116 Biundo, S. & Schattenberg, B. in *Proc. of ECP*. 157-168.
- 117 Bercher, P., Alford, R. & Höller, D. in *IJCAI*. 6267-6275.

- 118 Stock, S., Mansouri, M., Pecora, F. & Hertzberg, J. in *2015 IEEE/RSJ International*
 Conference on Intelligent Robots and Systems (IROS). 6459-6464 (IEEE).
- 119 Patra, S. *et al.* in *Proceedings of the international conference on automated planning and scheduling*.
 478-487.

Chapter 13

Understanding

INTRODUCTION TO UNDERSTANDING

Understanding is a fundamental yet expansive concept, encompassing various aspects of cognitive and artificial intelligence systems. It involves interpreting events and stories from the world around us – a process that significantly relies on the structured methodologies provided by frames. This chapter aims to dissect the mechanics of understanding by exploring how frames, particularly Thematic Role Systems, contribute to building a robust infrastructure for AI's comprehension capabilities.

Thematic Role Systems represent the most structured type of frame representation. These systems categorize information into specific roles that help clarify the relationships and actions within a sentence or narrative. By defining and assigning these roles, AI can begin to parse and understand complex scenarios and narratives with greater accuracy.

Moreover, understanding in AI is not just about parsing text but also about resolving ambiguities that frequently arise in language. Thematic roles play a pivotal role in this aspect by helping to distinguish between multiple meanings based on context, significantly reducing the potential for misinterpretation.

Lastly, this chapter will examine how grammar and other linguistic constraints aid AI systems in navigating the subtleties of language. These constraints guide the AI's interpretation, ensuring that it adheres to logical and grammatical norms, which are crucial for accurate comprehension.

By the end of this discussion, we will have laid down a comprehensive framework that not only supports current AI understanding capabilities but also sets the stage for more advanced topics such as common-sense reasoning, which will be covered in the following chapter. This exploration into understanding serves as a cornerstone for further discussions on how AI can more effectively mimic human cognitive processes, enhancing its ability to interact with and make sense of the complex world it inhabits.

THE CHALLENGE OF STORY UNDERSTANDING IN AI

In this chapter, we examine the complexities of story understanding through the lens of two distinct narratives from Lower Slabovia. Each narrative employs the word “killed” in vastly different contexts: one describing the tragic loss of 25 lives due to a severe earthquake, and the other using the term metaphorically to denote the rejection of 25 scientific proposals by the President. These stories illustrate the nuanced ways in which humans interpret language based on context, and the challenges this poses for artificial intelligence.

Story 1:

Today, an extremely serious earthquake of magnitude 8.5 hit Lower Slabovia, killing 25 people and causing \$500 million in damage. The President of Lower Slabovia said that the hard-hit area near the Sadie Hawkins fault has been a danger zone for years.

Story 2:

Today, the President of Lower Slabovia killed 25 proposals totaling \$500 million for research in earthquake prediction. Our Lower Slabovian correspondent calculates that 8.5 research proposals are rejected for every one approved. There are rumors that the President's science advisor, Sadie Hawkins, is at fault.

Figure 13-1 Story understanding example.

Understanding in humans is intuitive, leveraging contextual cues and background knowledge to differentiate meanings. For AI to achieve a comparable level of comprehension, it must be equipped with robust semantic frames that help parse the roles words play within sentences. It also requires extensive background knowledge to contextually analyze word usage and sophisticated disambiguation techniques to distinguish between literal and metaphorical meanings.

To illustrate, consider an AI system tasked with understanding these narratives. It would begin by identifying key verbs, subjects, and objects through syntactic parsing. Applying semantic frames, it would then interpret “killed” in the earthquake story as causing physical death, while in the political narrative, it would recognize the termination of initiatives. This dual interpretation

underpins the AI's ability to process and differentiate the thematic roles of words based on the narrative context.

This chapter underscores the importance of frames in providing structure to data, allowing both humans and AI to make coherent sense of diverse information. As we advance into discussions on common sense reasoning, we'll explore how AI can further harness this capability to interpret and reason about the world in ways that mimic human cognitive processes.

THE GENERAL PROCESS OF UNDERSTANDING

Understanding is an intricate process that involves deciphering the layered meanings embedded within communication, be it verbal, written, or non-verbal. At its core, the act of understanding extends beyond merely processing information; it requires constructing coherent interpretations from complex arrays of signals and contexts. In narrative forms, whether they are everyday conversations, literature, or technical descriptions, understanding hinges on the ability to connect and synthesize information according to linguistic rules, cultural norms, and personal experiences.

For artificial intelligence systems, replicating this process demands a multifaceted approach. AI must integrate various levels of analysis to mimic human comprehension effectively. These levels range from identifying basic linguistic structures to interpreting the intent and implications behind the expressed thoughts. The ultimate goal is to enable AI to not only recognize words and syntax but to infer and reason about the underlying messages and their relevance in a given context.

Analyzing Sentence Structure and Meaning

Let's examine a practical example to illustrate different layers of analysis that contribute to understanding. Consider the sentence: "Ashok made pancakes for David with a griddle." This sentence is immediately comprehensible to most humans, but let's unpack what we actually understand and how.

- **Lexical Analysis:** This initial analysis categorizes each word into its respective lexical class. In our example, "Ashok" is identified as a noun, "made" as a verb, "pancakes" also as a noun, and so forth. Lexical analysis forms the foundation by defining the basic building blocks of the sentence.
- **Syntactic Analysis:** Moving deeper, syntactic analysis explores the structure of the sentence. Here, "Ashok" functions as a noun phrase, while "made pancakes for David with a griddle" serves as a complex verb phrase. This phrase can be further broken down into sub-phrases, each playing a specific role in conveying the action being described.

- **Semantic Analysis:** At this level, we interpret the roles and relationships between the elements of the sentence. “Ashok” is the agent performing the action, “made” is the verb describing the action, “pancakes” represent the object of the action, “David” is the beneficiary of the action, and the “griddle” is the instrument used in the action. Semantic analysis not only parses who is doing what to whom and with what tool but also builds an understanding of the motivations and outcomes associated with the sentence.

In Knowledge-Based Artificial Intelligence (KBAI), semantic analysis is often prioritized because it integrates the deeper meanings and implications behind words and phrases. While syntactic and lexical analyses are critical, they primarily serve to support the more complex semantic understanding, which is key to grasping the full significance of narratives and instructions as humans naturally do. This semantic layer is where AI begins to bridge the gap between mere word recognition and true comprehension, paving the way for more sophisticated interactions and reasoning capabilities.

FRAMES AND SEMANTIC ROLES IN UNDERSTANDING

In understanding language, frames provide a crucial method for representing semantic knowledge, helping to delineate the meaning embedded within sentences through predefined categories. Consider the sentence: “Ashok made pancakes for David with a griddle.” This simple sentence encapsulates various semantic roles that contribute to our comprehension of the event it describes.

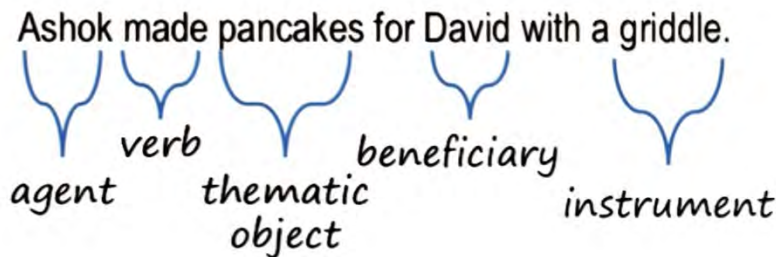


Figure 13-2 Sentence represented using Frames.

The framework for analyzing this sentence involves assigning each word or phrase to a specific semantic category: Ashok is the 'agent' – the doer of the action; 'made' represents the 'action' or verb; 'pancakes' are identified as the 'thematic object' – the item being acted upon; David is the 'beneficiary' – the one for whom the action is performed; and the 'griddle' serves as the 'instrument' – the tool used to carry out the action.

These roles help construct a 'frame' – a structural tool that outlines the various elements of the sentence in relation to each other based on their semantic functions. This frame isn't just a static representation; it actively informs our understanding by allowing us to infer information not explicitly stated in the sentence. For instance, if asked “Who ate the pancakes?” we might reasonably infer that David did, since he is the beneficiary in the context of the sentence where pancakes were made for him. Such inferences highlight how frames facilitate a deeper level of comprehension beyond the surface elements of text.

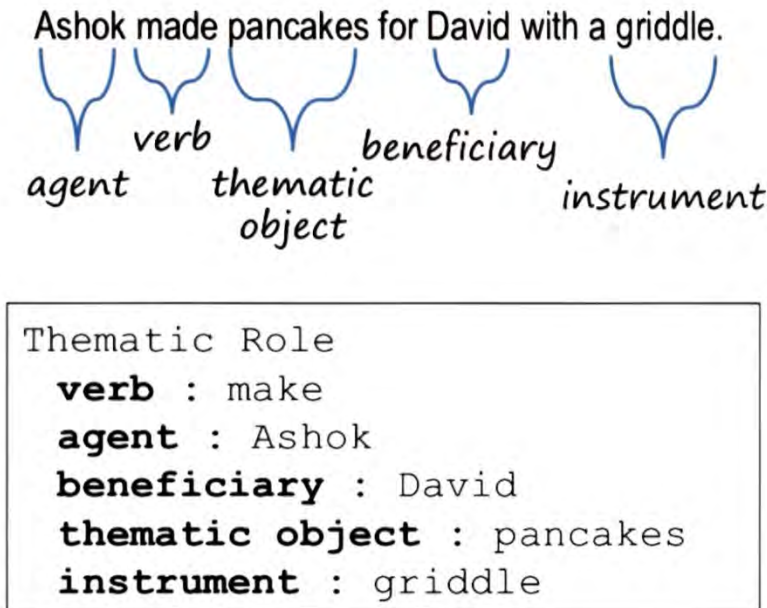


Figure 13-3 Thematic Role Frame.

The strength of using frames lies in their ability to allow inferences about unstated but implied information. For example, while it is not directly mentioned, we might guess that Ashok was content after cooking, similar to how we might deduce the state of the frog in the sentence “Ashok ate a frog” based on typical outcomes of such actions. These deductions are based on 'default values' assumed within the frame – common expectations like satisfaction from completing a task or the demise of the frog due to being eaten.

Understanding, then, according to this framework, hinges on the ability to draw correct inferences. Misunderstandings or incorrect interpretations often stem from drawing the wrong inferences or failing to draw any at all. The process of extracting meaning from a sentence through frames not only helps clarify what is being communicated but also enhances interaction by framing a coherent narrative around the bare facts presented.

As we progress, we will investigate deeper into how this knowledge is extracted and structured within frames, enhancing our capability to understand and interact with complex narratives and instructions. This exploration will reveal the intricacies of semantic roles and how they are pivotal in shaping our understanding of language and its application in artificial intelligence.

Thematic Role Systems

To effectively address the ambiguities similar to those in the Slabovian earthquake stories, it becomes essential to adopt an advanced form of knowledge representation known as a thematic role system. This system is a sophisticated type of frame architecture specifically designed to represent actions or events that are prompted by specific words within a sentence. These words instigate a set of predictable expectations about the roles associated with the event, thereby clarifying the actions described.

For instance, let's consider the action 'throw,' a term that readily brings to mind several associated roles and expectations, even without a direct reference to a specific instance of throwing. When we hear this word, we automatically infer that there must be an agent performing the action (the one who throws), an object being thrown, and potentially a target or destination towards which the object is directed. These roles are delineated by prepositions such as 'to' or 'at,' which help specify the direction of the action – whether it is “throwing the ball to Ashok” or “throwing the ball at Ashok” hinges on the contextual nuances conveyed by these small yet significant words.

This ability to generate expectations based on the verbs used in a sentence is the core function of a thematic role frame. By framing the action 'throw,' we don't just understand that something is being propelled through the air; we also begin to form a mental picture of the scenario, including who is involved, what is being thrown, and where it is likely to end up. This predictive capacity of thematic role frames allows us to grasp the full implications of a sentence rapidly and with a high degree of accuracy.

Thematic role systems are particularly effective because they not only catalog the various elements of an action but also provide the relational context that binds these elements together in a meaningful way. This framework is not just a passive repository of definitions; it actively shapes our understanding by aligning our expectations with the logical structure of the information presented. In the following discussions, we will explore how these systems are implemented in practical scenarios and how they help resolve ambiguities by providing a clearer, more structured interpretation of complex sentences and stories.

As part of their ongoing discussion about how thematic role systems can enhance the understanding of language in AI systems, Ashok proposes an exercise to deepen their exploration into the practical application of this concept. He provides a sentence and asks David to apply his knowledge of thematic roles to dissect and explain the sentence structure.

Ashok David, let's apply what we've learned about thematic role systems. Take the sentence: "David went to the meeting with Ashok by car." Can you break this down using the thematic roles we discussed? Identify the agent, co-agent, destination, and conveyance.

David went to the meeting with Ashok by car.

Thematic Role	
verb :	
agent :	
coagent :	
destination :	
conveyance :	

Figure 13-4 Representing a sentence as a Thematic Role Frame.

David So, we start with the verb, and here the verb is "went," which is the past tense of "go." The agent in this sentence is myself, David, because I'm the one performing the action; I'm the one who is going. Ashok, here, serves as the co-agent because I'm going with him. The destination is the meeting, and the conveyance is by car, which means the car is the conveyance.

Ashok That's right, David. But let's investigate deeper. How did we know that David was the agent? How did we determine that the destination was the meeting? And how did we establish that the car was the conveyance?

This dialogue between Ashok and David exemplifies the application of thematic role frames in parsing and understanding the structure of a sentence. By identifying the verb, agent, co-agent, destination, and conveyance, they have effectively applied a thematic role frame to clarify the relationships and roles within the sentence "David went to the meeting with Ashok by car."

Their discussion highlights how thematic roles help in dissecting sentences to understand the underlying structure and meaning. This approach is crucial for AI systems to grasp human language nuances, allowing them to interpret and respond appropriately in various contexts. In the next part of their conversation, they will explore how this knowledge is intuitively recognized and the implications for AI in mimicking this human capability.

The Role of Prepositions and Ontologies in Determining Thematic Roles

In the intricate architecture of human language, each component plays a pivotal role in shaping meaning. An essential aspect of this structure is the use of prepositions, which are crucial in assigning thematic roles to different elements within a sentence. Thematic roles, akin to semantic categories, help in parsing the relational dynamics between the elements of a sentence, facilitating a deeper comprehension beyond the superficial string of words.

For instance, consider the sentence “David went to the meeting with Ashok by car.” The preposition “by” here is instrumental in determining the role of “car.” In thematic role systems, prepositions like “by” can indicate various roles such as an agent, a conveyance, or a location. When we encounter “by” followed by “car,” the structured use of language guides us to understand “car” as a conveyance. This is not because the word inherently carries this meaning, but because the prepositional context positions it so.

The role of prepositions extends to others like “for,” “from,” “to,” and “with,” each playing distinct roles in framing the thematic context. “By” can associate with an agent, conveyance, or location; “for” might denote a beneficiary or a duration; “from” typically points to a source; “to” indicates a destination; and “with” might define a co-agent or an instrument.

David went to the meeting with Ashok by car.

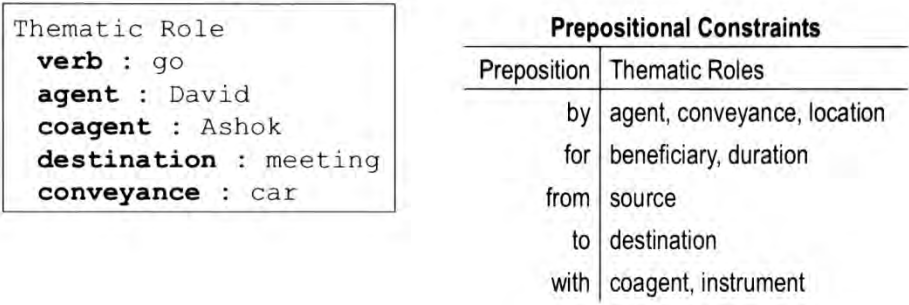


Figure 13-5 Thematic Roles

Understanding these roles is not about categorizing words as nouns or verbs but about grasping their semantic implications which are crucial for inference. However, the interpretation of these prepositions isn't always straightforward. The linguistic structure provides constraints but does not unambiguously resolve every ambiguity. It is here that additional knowledge – contextual or encyclopedic – becomes essential. For example, knowing that “car” is typically not used as an agent or a location helps solidify its interpretation as a conveyance in this context.

Thus, thematic role systems are foundational in enabling intelligent agents and humans alike to navigate the complexities of language, enhancing both understanding and communication by clarifying how elements within a sentence relate to one another based on their defined roles.

Prepositional Constraints and Semantic Roles

Understanding the role of prepositions in structuring language is essential for interpreting the meaning of sentences. The preposition “by,” for example, plays a pivotal role in indicating thematic roles within a sentence, which can vary based on its usage context. Here are three illustrative examples demonstrating how the preposition “by” guides our understanding of different semantic roles:

- **Agent:** In the sentence “That was written by Ashok,” the preposition “by” identifies Ashok as the agent who performed the action of writing. This usage aligns with one of the typical roles associated with “by,” indicating the doer of an action.
- **Conveyance:** In “David went to New York by train,” “by” introduces the train as the mode of transportation used by David to travel to New York. Here, “by” specifies the conveyance, highlighting the means by which the action (going) is achieved.
- **Location:** The sentence “David stood by the statue” uses “by” to denote proximity, placing David in a specific location relative to the statue. In this context, “by” refers to a spatial relationship, positioning the subject near a landmark.

These examples underscore the flexible nature of prepositions like “by,” which, depending on context, can refer to an agent, a conveyance, or a location. This flexibility, however, also introduces ambiguity, as the preposition alone does not always definitively resolve the intended semantic role. Additional contextual knowledge or background information is often required to accurately discern the role “by” plays in any given sentence. This necessity for further information highlights the complex interplay between linguistic structure and the broader knowledge required for full comprehension.

Understanding Ontologies in Artificial Intelligence

The concept of “ontology” within artificial intelligence significantly deviates from its philosophical roots where it pertains to the nature of reality. In AI, ontology refers to a structured framework representing the world; it involves categorizing and organizing information into a hierarchy that defines the relationships between different entities.

For instance, consider how we interpret the preposition “by” across different contexts, which could point to an agent, a conveyance, or a location. This interpretation isn't just about linguistic function – it requires a deeper understanding of the roles these entities play within a given scenario. The sentences “That was written by Ashok,” “David went to New York by train,” and “David stood by the statue,” demonstrate varying uses of “by,” each activating different semantic categories within our mental model of the language.

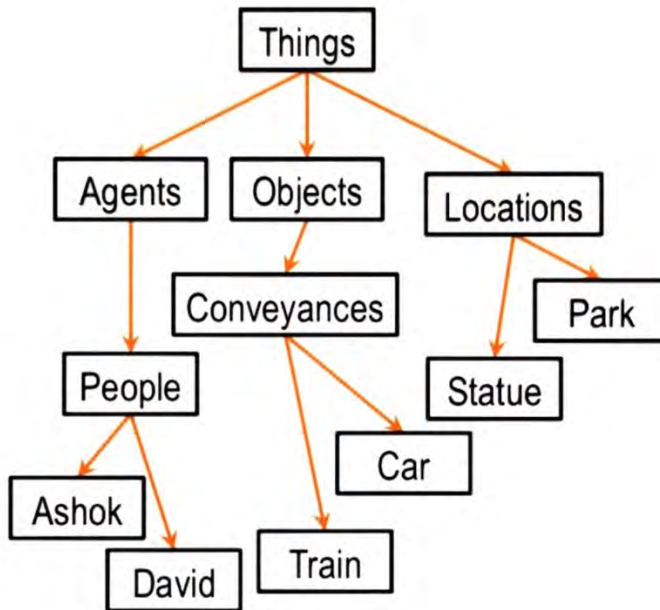


Figure 13-6 Example Ontology

An AI's ontology acts as a conceptual map of kinds, where various categories and their relationships are predefined. For example, in a simple ontology:

- Things are divided into Agents, Objects, and Locations.
- Agents could be further classified into People, such as Ashok and David.
- Objects might include Conveyances, like trains and cars.
- Locations could be specific places like parks or more general like statues.

This hierarchical organization helps in defining how entities interact or relate to one another, making it easier for an AI system to interpret complex inputs and draw inferences accordingly. For example, understanding that in the sentence “David went to New York by train,” “train” functions as a conveyance rather than a location or agent, relies on this structured knowledge.

Ontologies are crucial for AI as they provide not just a vocabulary but also a framework for understanding the relationships and functions of various elements within the world model. This organization not only supports better data processing and interpretation but also enhances the AI's ability to interact with the world in a more human-like manner.

Semantic Categories and Ontologies

Understanding the semantic roles of words in sentences hinges crucially on the structured knowledge termed as ontology within artificial intelligence. This ontology is not just a dictionary but a conceptual map that outlines how different entities relate to each other and the roles they typically play in the real world.

Consider the examples where the preposition “by” is used in sentences to denote different roles:

1. “That was written by Ashok” – Here, “by” indicates an agent.
2. “David went to New York by train” – Here, “by” suggests a conveyance.
3. “David stood by the statue” – In this case, “by” refers to a location.

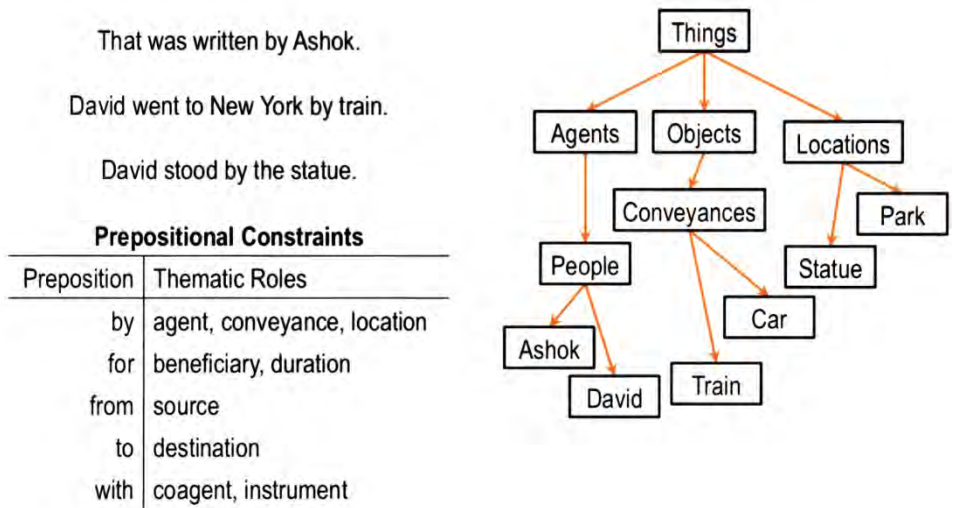


Figure 13-7 Ontology mappings

The ontology enables us to navigate through these interpretations. For instance, when we read “by Ashok,” our understanding of the ontology where people are classified under agents helps us recognize “Ashok” as an agent, rather than a conveyance or a location. This kind of categorization is crucial for AI to process language in a meaningful way.

Similarly, the train in the second example is classified as a conveyance within our ontology, making it clear that “by train” describes the mode of transport rather than a location or agent. Likewise, the statue, being part of the locations category, anchors the meaning in the third example.

Top-down and Bottom-up Processing In Semantic Analysis

This understanding process starts with bottom-up processing where the basic grammatical structures are identified – David as a noun, went as a verb. However, the interpretation quickly shifts to top-down processing influenced by the ontology. This background knowledge guides the interpretation of “by” and other prepositions, linking them with the correct semantic category – agent, conveyance, or location.

For instance, if the sentence had been “That was written with Ashok,” the preposition “with” could imply that Ashok is a co-agent or an instrument, depending on the context provided by the ontology.

Such dynamic between bottom-up data-driven processing and top-down knowledge-driven interpretation forms the crux of natural language understanding in AI systems. It illustrates how deeply AI's ability to understand and interact with human language is entwined with its underlying representations of the world. This comprehensive framework not only helps in identifying the literal meanings but also in drawing sophisticated inferences that are not explicitly stated, thereby enriching the interaction between humans and machines.

Resolving Word Ambiguity

Ambiguity in language is not limited to prepositions; it frequently occurs at the level of individual words, posing a significant challenge for natural language understanding systems.

Ashok David, can you think of ambiguity in a word?

David Certainly, Ashok. Consider the word “hit” in the sentence, “I was wondering why the ball was getting bigger and then it hit me.” Here, “hit” is ambiguous because it could mean that the realization dawned on me – ‘it occurred to me’ – or it could literally mean the ball struck me. This dual potential creates a rich ground for misinterpretation without context.

Another example is the word “take,” which is often used in puns. For instance, it’s difficult to explain puns to kleptomaniacs because they always take things literally. Here, “take” could mean to understand something in a literal sense or to physically pick something up. The humor in such sentences often stems from the ability of a word to assume multiple roles simultaneously, enriching the sentence with playful ambiguity.

Ashok Did you know that I was going to say this will be much fun? Our goal in exploring natural language understanding is to see how AI agents can resolve ambiguities where only one correct interpretation is possible. But we should also appreciate

that sometimes, embedding a bit of humor might involve allowing a sentence to sustain multiple thematic roles at once.

This ongoing dialogue between David and Ashok underscores how AI systems need to employ sophisticated strategies to decipher and choose between multiple potential meanings of a word based on context, enhancing their ability to interact in human-like ways. Understanding these nuances allows AI to not only communicate effectively but also engage in the playful and creative use of language that characterizes much of human interaction.

In natural language, ambiguity is not confined solely to prepositions. It extends broadly into other lexical categories, including verbs, which can manifest a wide array of meanings based on the context in which they are used. A prime example of such a verb is “take.”

“Take” is an everyday word with a plethora of meanings, each contextualized by its use in a sentence. For instance, if someone says, “Ashok took an aspirin,” the verb “take” clearly implies the action of consuming medication. However, the very same word can shift meanings dramatically in different contexts. For example, in the sentence “My doctor took my blood pressure,” “take” does not involve acquiring or consuming anything physically but refers to measuring or assessing blood pressure.

take. verb. /tāk/

- to reach for and hold
- to bring or transport
- to remove
- to medicate
- to capture
- to steal
- to cheat or swindle
- to subtract
- to measure
- to accompany
- to assume control
- to remove from the body

Figure 13-8 Multiple meanings of the verb “take”.

This versatility of the verb “take” illustrates the inherent challenge in parsing and understanding natural language. Each meaning, though distinct, shares a conceptual core that relates to the act of taking but diverges in application from physical actions like removing or transporting, to abstract concepts such as assuming control or measuring. Determining the intended meaning of “take” in any given sentence requires a nuanced understanding of its context, which is a significant task for AI systems engaged in natural language processing.

In the broader scope of natural language understanding, such ambiguity necessitates sophisticated inference mechanisms within AI systems to interpret verbs accurately based on surrounding textual cues. This capability is crucial for effectively handling various language tasks, from translation and content summarization to interactive dialogue systems, where understanding the precise meaning of words like “take” can significantly influence the response quality and relevance.

Exploring the Multifaceted Meanings of “Take”

Language is inherently complex and ambiguous, a characteristic that is not limited to structural elements like prepositions but extends to various lexical categories, including verbs. Verbs, in particular, can carry a multitude of interpretations depending on their contextual usage. Consider the verb “take,” a simple yet profoundly versatile term that encapsulates multiple meanings.

“Take” is a commonplace verb encountered frequently across diverse contexts, each instance potentially altering its interpretation. For example, when someone says, “Ashok took an aspirin,” the verb “take” straightforwardly implies the action of consuming medication. However, the interpretative scope of “take” broadens significantly with different syntactic constructions. If the statement changes to “My doctor took my blood pressure,” “take” transitions from an act of consumption to one of measurement, illustrating the verb’s capacity to adapt its meaning based on the surrounding context.

To systematically dissect the varied interpretations of “take,” we employ a framework where each distinct meaning is associated with specific thematic roles. This approach helps in structuring our understanding of how “take” operates within sentences. From the first to the twelfth meaning, each connotation of “take” is tied to a unique set of roles:

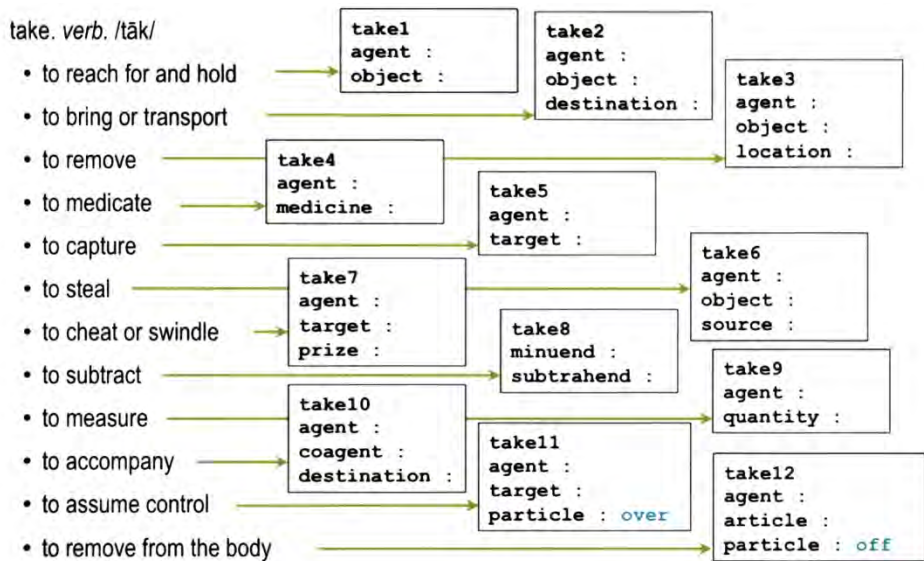


Figure 13-9 Nine of twelve thematic role frames for the verb “take” illustrated.

Exploring these thematic roles associated with each meaning of “take” not only aids in distinguishing between potential interpretations but also enhances our ability to predict verb usage in various contexts. This method of semantic categorization, where words like “take” are dissected into their constituent thematic roles, proves invaluable in natural language processing, providing clarity and precision in language comprehension and AI interactions.

Semantic Ambiguity: Understanding the Meaning of “Take” in Context

To further our exploration into story understanding, let's analyze a straightforward narrative: “I took the candy from the baby.” This example illustrates how the verb “take” operates within a specific context, which may initially seem apparent to us but requires deeper analysis to be understood by an AI agent.

In examining this sentence, it's important to acknowledge how our background knowledge significantly influences semantic interpretation. For instance, we understand that candy is neither a medicine nor a measurable quantity. Such knowledge allows us to dismiss certain meanings of “take” that would otherwise fit the verb's broader semantic range but are irrelevant in this context. Similarly, the preposition “from” primarily indicates a source, further narrowing down the potential meanings by eliminating those that do not involve a source component.

Given these constraints, we focus on the relevant semantic frames for “take,” specifically those where the verb implies removing something from a source. The most fitting interpretation in this context is that “take” implies an action of stealing, particularly because the item in

question, candy, typically would not be willingly given up by a baby. Thus, among the various meanings of “take,” we pinpoint “to steal” as the correct interpretation for this scenario. This conclusion is reached not merely through the direct application of language rules but through a sophisticated combination of lexical understanding and contextual knowledge, showcasing the complex interplay of bottom-up and top-down processing in natural language understanding.

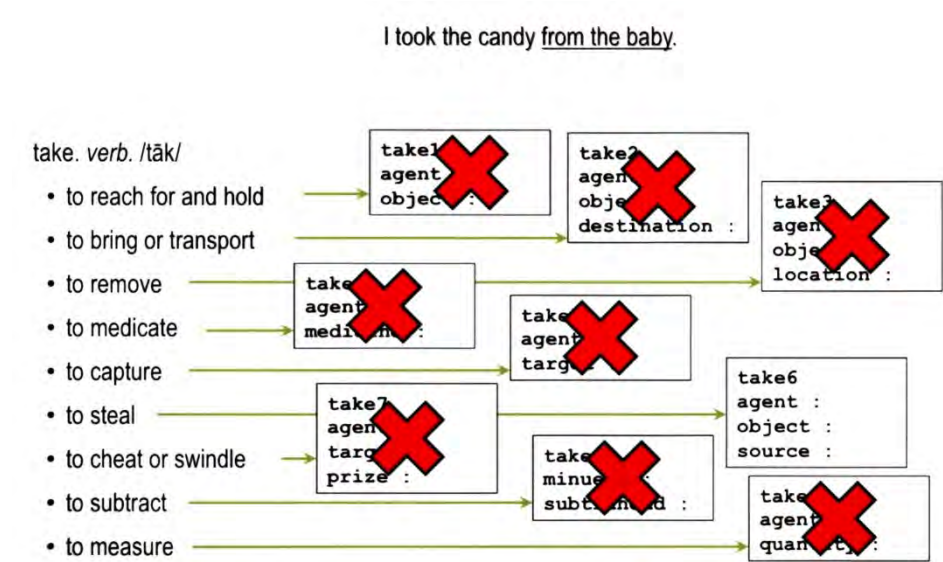


Figure 13-10 Thematic role frame reduction using background knowledge.

Thematic Role Frames: Exercises in Context

After investigating the complexities of thematic role frames and their ability to differentiate among the varied meanings of “take,” let us now engage in some practical exercises to further illustrate these concepts. We will evaluate three sentences to determine the most applicable meanings of “take” in each context, using the thematic role frames as our guide. The sentences are:

1. Take 3 from 5
2. I took my briefcase to work today.
3. I took the casino for \$5 million.

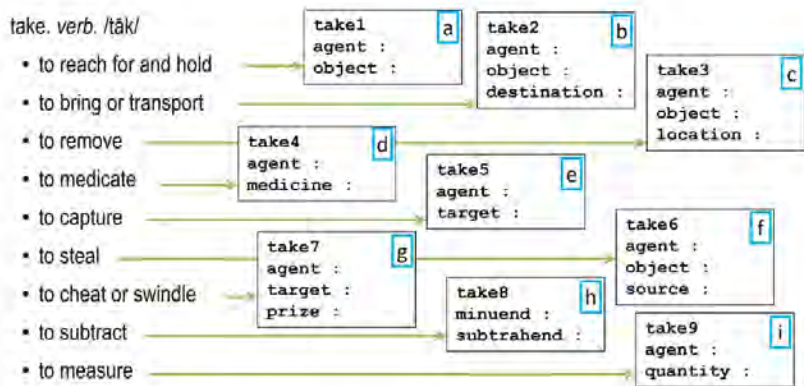


Figure 13-11 Thematic Role - Example Problem.

Ashok What did you decide on, David?

David For the first sentence, considering that the entities involved are merely numbers – specifically, 3 and 5 – and not objects, destinations, locations, or targets, the only frame that fits is “to subtract.” This frame looks exclusively for numerical entities, aligning perfectly with our needs here. Thus, I selected frame h for the meaning of “take” in this sentence.

Moving on to the second sentence, my role as an agent, my briefcase as an object, and work as the destination clearly direct me to the frame b to determine the meaning of “take,” where the action involves transporting an object to a particular location.

The third sentence is a bit more intricate. Here, I recognize myself as an agent, the casino as a place of business, and \$5 million as either a prize or an object. The challenge arises in determining whether the action is “to steal” or “to cheat or swindle.” Given the lack of the preposition “from,” which would typically indicate a source, I lean away from interpreting the casino as a source. Therefore, I opt for the meaning of “take” from frame g, which suggests swindling, as it does not require the source designation that stealing would.

Ashok That’s an intriguing analysis, David. Thank you. And congratulations on your hypothetical windfall – I wasn't aware that teaching could be so lucrative!

This exercise highlights the importance of background knowledge in interpreting verb meanings – knowledge that may vary significantly depending on one's linguistic and cultural

background. This variability sometimes complicates the task of distinguishing between meanings, especially when they are closely related, such as stealing versus swindling. Such exercises demonstrate not only the utility of thematic role frames in natural language processing but also the nuanced understanding required to navigate language's inherent ambiguities.

Implications of Thematic Role Frames Across Languages

In our exploration of thematic role frames, it becomes evident that the effective disambiguation of verb meanings often hinges on the speaker's background knowledge. For instance, native speakers of English may possess inherent understandings of certain prepositions like “for,” which aids in distinguishing between closely related meanings. This nuanced comprehension can make it significantly easier to differentiate between actions as distinct as stealing and medicating.

However, the challenges in verb disambiguation are not unique to the English language. This analytical approach applies universally across various languages, suggesting its potential utility in linguistic studies and natural language processing worldwide. For example, despite my origins in India and native fluency in Hindi, the techniques we apply to English can be equally effective in Hindi. This universality prompts intriguing possibilities for machine translation and cross-lingual applications. Could this method help bridge language gaps by translating sentences while maintaining their thematic integrity?

Moreover, this discussion leads us to consider whether all languages share a structural foundation robust enough to support this type of semantic analysis. If so, this could revolutionize our approach to computational linguistics, enabling more effective and nuanced translations and interpretations across diverse linguistic landscapes. This consideration opens up a plethora of research opportunities to explore the commonalities and unique aspects of language structures globally.

Revisiting the Slabovia Example

Returning to the Slabovian scenarios provides a clear demonstration of how semantic analysis and thematic role frames can effectively distinguish between different meanings of the verb “kill.” Like the verb “take,” “kill” is polysemous, carrying multiple meanings depending on the context. In our examples, “kill” can denote either “causing the death of someone” or “putting an end to something.”

Sentence 1: A serious earthquake killed 25 people in Lower Slabovia.

Sentence 2: The President of Lower Slabovia killed 25 proposals for earthquake prediction.

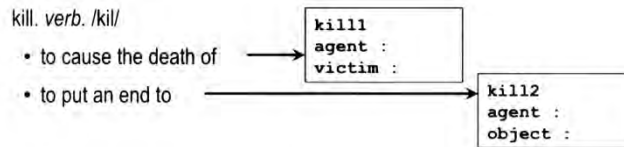


Figure 13-12 Thematic role frames for the Slabovia example.

In the first scenario where a serious earthquake resulted in casualties, the verb “kill” is linked with a tragic loss of life. Here, the thematic roles are straightforward: the earthquake serves as the agent and the unfortunate victims as the objects. Conversely, in the second scenario concerning the President of Lower Slabovia, “kill” refers to the rejection of proposals – an entirely figurative use of the term where the proposals are the objects and the President acts as the agent.

This distinction hinges significantly on background knowledge that aligns the verb with its contextually appropriate meaning. In the case of the earthquake, the common understanding of “kill” as causing death is applicable, while in the bureaucratic context, “kill” metaphorically describes the cessation of proposals.

Such examples underscore the nuanced capability of thematic role frames in parsing sentences according to the verb's role and associated agents and objects. It's this sophisticated understanding that not only enriches our grasp of language mechanics but also showcases the adaptability of semantic frameworks across different narrative contexts. The elegance of this analytical approach lies in its ability to provide clarity and precision in interpretation, revealing the depth and flexibility of language understanding.

Examining the Limitations of Semantic Analysis

While the theory of thematic role frames and semantic analysis offers profound insights into language understanding, it's crucial to acknowledge its limitations, particularly when applied to increasingly complex or varied linguistic contexts. Consider the earlier example sentence, “I took the candy from the baby.” Here, the interpretation relies heavily on a set of rules that associate the meanings of “take,” “candy,” and “from” to imply theft. However, the adaptability of this approach is tested as we introduce slight variations to this sentence.

For instance, altering the sentence to “I took the candy for the baby,” “I took the toy from the baby,” “I took the medicine from the baby,” or even “I took a smile from the baby,” each

introduces new nuances that demand additional rules for accurate interpretation. These variations demonstrate how quickly the number of necessary rules can expand, complicating the task of disambiguation. As sentences diverge from the original structure or context, the semantic framework must accommodate an ever-growing set of interpretations, each requiring unique considerations.

This proliferation of rules poses a significant challenge. While the framework handles a substantial array of typical interactions effectively, it struggles with the sheer volume and variability of natural language. The task of enumerating all possible rules for all potential sentence variations is daunting and, in practical terms, often unfeasible.

Despite these challenges, the foundational principles of semantic analysis remain invaluable. They equip us with a robust toolset for understanding a wide range of narratives and dialogues that we encounter in everyday communication. However, this example serves as a reminder of the inherent complexity of language and the ongoing need to refine and expand our analytical approaches to keep pace with its subtleties and variations.

UNDERSTANDING AND COGNITION

This chapter primarily focused on Natural Language Understanding (NLU), but it is crucial to recognize that understanding is a broader cognitive task, transcending the verbal and textual confines of language. Understanding encapsulates our ability to make sense of the world, a world that incessantly bombards us with diverse forms of data – acoustic, visual, verbal, and numerical. Grappling with such a complex array of information presents a formidable challenge: how do we process and interpret this incessant influx of data?

- **Harnessing World Constraints:** One fundamental strategy involves leveraging the inherent constraints of the world around us. These constraints are not just physical but also social and grammatical. They guide our expectations and interactions, providing a structured framework within which the world operates. By understanding these constraints, we can predict and interpret behaviors and outcomes, reducing the complexity of our environment into more manageable segments.
- **Structured Knowledge Representations:** Another critical aspect is the role of structured knowledge representations. These are not mere repositories of facts but are organized systems of information that align with how knowledge itself is structured and interconnected. This organization is not trivial; it empowers our cognitive processes, enabling us to retrieve and apply knowledge efficiently and effectively. The structuring of knowledge helps in transforming raw data into meaningful information, allowing us to navigate through and make sense of the world.
- **Activating Knowledge Structures:** The third source of power in understanding is the activation of these knowledge structures through low-level processing. This involves the initial, often subconscious, stages of processing that retrieve relevant knowledge from memory. Once these structures are activated, they enable top-down processing,

where our understanding and expectations guide our perception and interpretation of new information. This ability to generate and apply expectations is crucial – it means that our understanding of the world is not always reactive but often proactive, shaped by what we already know and anticipate.

In sum, understanding is a dynamic interplay between the structured knowledge we have accumulated and the continuous processing of new information through both bottom-up and top-down processes. It's a testament to the cognitive complexity inherent in making sense of our multifaceted world, highlighting the sophisticated nature of human thought and the systems we've developed to support it. As we explore further into cognitive tasks beyond natural language, these principles continue to underscore the sophisticated, intricate nature of human cognition and its capacity to navigate and interpret the vast array of stimuli that define our reality.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

The principles of understanding explored in this chapter, particularly the use of frames and thematic role systems, are foundational KBAI concepts that offer significant value when integrated with modern AI paradigms.

- **Natural Language Processing (NLP) & Understanding (NLU):** This is the most direct area of connection.
 - **Semantic Parsing:** Thematic role systems directly inform semantic parsing, where the goal is to convert natural language sentences into formal meaning representations. Modern NLP systems, including those based on deep learning, often aim to identify semantic roles (e.g., agent, patient, instrument) to understand “who did what to whom.” KBAI frames provide explicit structures for these roles.
 - **Information Extraction:** Frames can serve as templates for extracting structured information from unstructured text. For example, an “earthquake event” frame can guide an NLP system to find and fill slots for location, magnitude, casualties, etc., from news articles.
 - **Disambiguation:** As discussed, ontologies and structured knowledge (frames) help resolve lexical (word meaning) and prepositional attachment ambiguities, a persistent challenge in NLU.
 - **Dialogue Systems:** Understanding user intent and extracting key entities from user utterances in a dialogue system heavily relies on identifying semantic roles and filling frame slots.
- **Large Language Models (LLMs) & Generative AI (GenAI):**
 - **Structured Output Generation:** While LLMs excel at generating fluent text, their output can sometimes lack structure or factual consistency. Frames can

be used as a target output structure, prompting LLMs to fill predefined slots, thus generating more organized and verifiable information.

- **Knowledge Grounding & Explainability:** KBAI frames and ontologies can provide a structured knowledge base to ground the outputs of LLMs, helping to verify factual claims or provide more explicit reasoning traces for how an LLM arrived at an answer.
- **Prompt Engineering:** Understanding semantic roles and frame-like structures can inform better prompt engineering, guiding LLMs to produce more targeted and contextually appropriate responses.
- **Knowledge Graphs:** Thematic roles and frame structures are fundamental to the design of knowledge graphs. Entities in a knowledge graph (nodes) often correspond to concepts or instances, and relationships (edges) often represent thematic roles or slot-filler relationships. Building and querying knowledge graphs often involves NLU techniques enhanced by frame semantics.
- **Reinforcement Learning (RL) for Language Tasks:** In RL agents that interact using natural language or need to understand instructions, representing the meaning of linguistic inputs using frames can provide a more structured state representation, potentially making it easier for the agent to learn effective policies.
- **Explainable AI (XAI):** When an AI system's understanding is based on explicit frame structures and thematic roles, its interpretations become more transparent. It's easier to explain *why* the system understood a sentence in a particular way by pointing to the activated frame and the filled slots, compared to “black-box” NLU models.

The KBAI approach to understanding, with its emphasis on structured knowledge, explicit semantic roles, and the interplay of bottom-up and top-down processing, offers a complementary perspective to purely data-driven methods in modern AI. Integrating these symbolic structures can lead to NLP and NLU systems that are more robust, interpretable, and capable of deeper comprehension.

CHAPTER SUMMARY

In this chapter, we have investigated the complex cognitive task of understanding – how agents, whether human or artificial, interpret stories, events, and various stimuli from the world around them. Central to our discussion was the development and application of thematic role systems, a sophisticated framework rooted in frame-based knowledge representation, which aids in the formal representation of verbs and their expected roles in sentences. This approach not only clarifies the function of verbs within events but also provides a structured method to anticipate outcomes and interpretations in narrative contexts.

We explored the inherent ambiguity of language, particularly how verbs like “take” and “kill” can carry multiple meanings based on their usage. Thematic role frames, supported by ontologies that categorize concepts (like agent, object, conveyance, location), emerged as a vital

tool in distinguishing these meanings. This involves a dynamic interplay of bottom-up processing (lexical and syntactic analysis) and top-down processing (using frames and ontological knowledge to generate expectations and fill semantic roles).

Further, we examined the role of constraints in language – how specific words (especially prepositions) and syntactic structures limit or specify possible interpretations, thus aiding in the resolution of ambiguities. These constraints are not merely linguistic but are deeply tied to our cognitive processes, reflecting how structured knowledge and memory play pivotal roles in language comprehension. The limitations of purely rule-based semantic analysis for handling the vast variability of natural language were also acknowledged.

The chapter emphasized that understanding is a broad cognitive capability, extending beyond NLU to making sense of diverse data from the world by leveraging world constraints, structured knowledge, and the activation of these knowledge structures.

Finally, we discussed the integration of KBAl's principles of understanding with modern AI paradigms. Frame-based representations and thematic role analysis enhance NLP/NLU tasks like semantic parsing and information extraction, can structure and ground the outputs of LLMs, are fundamental to knowledge graph design, and contribute to explainable AI by making linguistic interpretations more transparent.

This chapter laid the groundwork for recognizing and interpreting the subtleties of language, setting the stage for the next chapter where we will approach the inverse: how different expressions can converge to share identical meanings. We'll investigate the cognitive strategies that allow us to discern this sameness and adapt our responses to communicate effectively and efficiently. By understanding the tools and theories discussed, readers can better appreciate the intricate dance between language form, meaning, and cognitive processing.

By understanding the tools and theories discussed, readers can better appreciate the intricate dance between language form, meaning, and cognitive processing, enriching their insights into both human cognition and the mechanics of communication. This foundation is crucial not only for linguists and cognitive scientists but for anyone interested in the deeper workings of language and thought.

ADDITIONAL READINGS

REFERENCES

Chapter 14

Commonsense Reasoning

INTRODUCTION TO COMMONSENSE REASONING

Welcome to our exploration of commonsense reasoning, a fundamental aspect of cognitive functioning that allows both humans and potentially AI agents to navigate and interpret the world.¹²⁰⁻¹²² Commonsense reasoning involves making intuitive judgments and inferences that seem natural and obvious to humans, but are profoundly complex when we attempt to instill similar capabilities in machines.

Consider a scenario where a robot is asked to determine if it is raining outside. Unlike humans, who would simply look out a window, a robot without commonsense reasoning might consider jumping out the window to check the weather – a clearly risky and unnecessary action. This chapter investigates how we can equip AI agents with the tools necessary to avoid such illogical decisions and make inferences that humans consider routine.

Our discussion will utilize a framework of commonsense reasoning structured around a concise set of primitive actions. These 14 fundamental actions, though limited in number, are powerful in their capacity to structure a wide array of knowledge into hierarchical actions and sub-actions, enabling AI to make safer and more effective decisions.¹²³ By understanding these primitives and how they interact to form complex decision-making processes, AI can be programmed to anticipate outcomes and avoid actions that might lead to harm or inefficiency.

Through this chapter, we will explore the intricate dance between these primitive actions and the broader contexts in which they are employed, aiming to illuminate the paths through

which AI can not only mimic human reasoning but do so in a way that feels natural and intuitive. This exploration is not just about preventing a robot from making a literal leap out a window but about refining AI's ability to process and respond to the myriad subtleties of human interaction and the physical world.

UNDERSTANDING STORIES: THE FOUNDATION OF COMMONSENSE

Stories are the bedrock of human communication, offering a structured way to convey experiences, ideas, and emotions. Whether told around a campfire or shared through the latest digital media, stories shape our understanding of the world and each other. But what happens when the listener is not human, but a machine? In this section, we investigate the challenges and strategies of teaching machines to understand stories, particularly focusing on the nuances of language that come to life in storytelling.

Machines like IBM Watson or Siri are designed to interact with humans in ways that seem intuitive, processing our language and responding in kind. However, the complexity of natural language, with its multiple layers of meaning and context, presents unique challenges in natural language understanding (NLU). This section explores how AI can interpret simple, one-sentence stories, such as “Ashok ate a frog,” and discern the varied meanings of verbs that appear straightforward at first glance.

Through a deeper look into the verb “eat,” we illustrate the diverse interpretations this simple action can assume, from the literal consumption of food to metaphorical uses that describe emotional or physical processes, such as “eating away” at something.¹²⁴ Each example underscores the importance of context and structured knowledge in helping AI systems not just hear, but truly understand, the stories we tell.¹²⁵⁻¹²⁹

Navigating the Nuances of Language

Okay. Have you ever wondered how you could write the equivalent of the IBM Watson program or a Siri program for your own computer? Just imagine if you could talk to your machine in terms of stories – simple stories – perhaps just one-sentence stories like “Ashok ate a frog.” Now, we’ve already seen how a machine can make sense of this particular sentence: Ashok ate a frog. We’ve seen this earlier in the book. But, of course, eat or ate can occur in many different ways in sentences.

Here are some of the other ways in which I can use the verb eat:

- “Ashok ate a frog.”
- “Ashok ate out at a restaurant.”
- “I could tell something was really eating him.”
- “The manager would rather eat the losses.”

- “The river gradually ate away at the riverbank.”

The sense of eating in the first sentence is very different from the sense of eating in the second. There’s no physical object that is being eaten here. So just like we had in the previous chapter, the word “take” had so many different meanings, here, eat has so many different meanings.

Now when we were discussing the word take, we discussed how we can use both the structure of sentences as well as background knowledge to disambiguate between different interpretations of the word take. We could do something similar with eat. We could enumerate all the different meanings of eat, then for each meaning of eat, we could ask ourselves what is it about the structure of the sentence and what is it about the background knowledge that I have about things like rivers and riverbanks which tells me what the meaning of ate here is? To summarize this thought, if you were to start talking to your machine in stories, in terms of simple stories designated by a single sentence, then one problem that will occur is, that words like ate or take will have a large number of meanings.¹³⁰ And we have seen how your machine can be programmed in such a way so as to disambiguate between different kinds of meanings of the same word.

In this exploration of language and understanding, we encounter a fascinating reversal of the ambiguity challenge discussed earlier. Here, the focus shifts from a single word with multiple meanings to multiple words that share a single meaning within their respective contexts. Consider the sentences:

- “Ashok ate a frog.”
- “Ashok devoured a frog.”
- “Ashok consumed a frog.”
- “Ashok ingested a frog.”
- “Ashok swallowed a frog.”

Despite the diversity in verb usage, the underlying narrative remains remarkably consistent across all instances.

In each of these examples, the sequence of events unfolds identically: initially, the frog is outside of Ashok’s body. Subsequently, through the action described – whether it’s eating, devouring, consuming, or ingesting – the frog ends up inside Ashok’s body, deceased by the end of the process, with Ashok presumably satisfied. This illustrates a fundamental aspect of storytelling where different verbs, while varied in their connotations and usual contexts, converge in meaning due to the narrative’s demands.

The challenge then arises: how can we enable a machine to recognize that all these varied expressions essentially convey the same outcome? This issue stands in contrast to the previous problem of a single word like “eat,” which could signify numerous actions depending on the context. Here, the task is to understand that despite the lexical variety, the semantic intent remains unchanged across different formulations. This understanding is crucial for developing AI systems capable of interpreting human language with the nuance and depth required for genuine interaction.¹³¹

The Role of Context in Story Understanding

A pivotal question arises in artificial intelligence: How can machines be taught to recognize that different expressions in language might convey identical meanings? This challenge is rooted in understanding the context – a concept that proves to be one of AI's most formidable hurdles.

Context plays dual yet distinct roles depending on the scenario.¹³² In cases where a single verb might have multiple meanings, context dictates which meaning is appropriate based on the surrounding text or situation. Conversely, when different verbs share a common meaning within their specific contexts, it is the uniformity of these contexts that guides our understanding, reinforcing that despite lexical diversity, the semantic intent remains unchanged.

For instance, consider simple one-sentence stories such as “Bob shot Bill” and “Bob killed Bill with a gun.” Despite the structural differences between these sentences, the outcome and interpretation remain consistent – Bill is deceased, and Bob is responsible for the act. In both scenarios, the essence of the action is the same, highlighting how diverse linguistic structures can still guide us to a unified understanding due to the shared context.

These examples illustrate the complexity of incorporating context into AI systems. While currently, our focus may be on single-sentence stories, we are poised to explore more intricate narratives composed of multiple sentences in subsequent chapters. This progression will further demonstrate how the structural elements of language interact with context to convey meaning, offering a deeper insight into the nuanced challenges of teaching machines to process human language effectively.

BUILDING AI AGENTS WITH PRIMITIVE ACTIONS

The challenge now shifts to constructing AI agents capable of meaningfully interpreting narratives, such as “Bob shot Bill” or “Bob killed Bill with a gun.” A potential strategy involves using a structured approach similar to the one we used for the verb “take.” For each verb in a story, we could theoretically develop a distinct frame – creating frames for “ate,” “devoured,” “consumed,” “ingested,” and more. However, this would result in a multitude of frames, given the extensive variety of verbs present in English and other modern languages.

An alternative approach could be to recognize the underlying similarities in these verbs' interpretations and apply a uniform internal representation for each. This concept introduces the idea of “primitive actions.” A primitive action simplifies the multitude of potential verb interpretations into a single, fundamental action that encapsulates the essence of these verbs in a uniform manner.

For instance, while the English language employs a diverse array of verbs for communication, an AI agent could be designed to recognize and process these verbs through a single, streamlined conceptual framework. Each verb, despite its surface variation, would link back to a core primitive action that aligns with our interpretation of that verb's effect in the real world.¹³³ This approach reduces the complexity of understanding and processing language by

standardizing the interpretation mechanism, making it a potentially powerful tool for enhancing AI's narrative comprehension.

The Ontology of Primitive Actions

Roger Schank, quite some time ago, proposed a set of fourteen primitive actions, though the terms he used have since evolved into what we now recognize in modern terminology.¹³⁴ Let's briefly explore these actions which range from the physically tangible to the abstract:

1. **Move-body-part:** For example, Ashok moving his hand from one position to another.
2. **Move-object:** Such as Ashok moving his car from his office to his home.
3. **Expel:** Like Ashok exhaling.
4. **Ingest:** Ashok eating something.
5. **Propel:** Ashok kicking a ball, where his body is in contact with the ball.
6. **Think-about:** Ashok forming an idea in his mind.
7. **Move-concept:** Ashok suggesting to Susie they go to a movie.
8. **Conclude:** Related to decision-making processes.
9. **Feel:** Experiencing emotional or physical sensations.
10. **See:** The act of perceiving visually.
11. **Smell:** Detecting odors.
12. **Speak:** Vocal communication.
13. **Hear:** Auditory reception.
14. **Move-possession:** Transferring ownership or possession of an object.

These primitive actions, while simple in description, encapsulate a broad range of human activity, from the physical movements we make to the mental activities we engage in. Collectively, they form an ontology – a framework for categorizing and understanding the myriad actions that populate human behavior.

Ontologies, as introduced in the previous chapter, are conceptualizations used to describe and categorize the world in a structured way, enabling us to make sense of diverse inputs. An ontology, in AI, demands precise definitions of these concepts to enable machines to understand the world in human-like terms.

These 14 actions don't just stand alone; they interact and combine to form the complex tapestries of behavior we observe in daily life. By defining these primitive actions within an ontology, we aim to empower AI systems to interpret and respond to human actions with a level of understanding previously confined to the domain of human cognition. This structured approach to action and interpretation is foundational in teaching AI to process and react to the world in a way that mimics human reasoning, making it a crucial step in the development of intelligent systems.

While the set of fourteen primitive actions provides a robust framework for interpreting many human behaviors and interactions, it's important to recognize the limits of any such system. These actions, as comprehensive as they might seem, do not encompass every possible human

story or action. It is entirely feasible that alternative ontologies could be developed, featuring a different set of primitive actions tailored to specific applications or cultural contexts. Engaging with the development of such ontologies not only broadens the scope of AI's understanding but also enriches our own perspectives on human behavior.

Indeed, some situations pose unique challenges for categorization within this framework. Consider a common activity such as walking across a room. In this scenario, the individual is both the agent initiating the movement and the object being moved. This dual role complicates the application of our existing primitive actions. However, a potential solution lies in the application of the “move-object” action, where the individual is considered both the mover and the object moved. This interpretation allows the action to be captured within the existing framework, albeit with a nuanced understanding that recognizes the individual's dual role.

This example illustrates the flexibility and adaptability of the primitive actions ontology. While initially designed to capture straightforward actions, the framework can be extended to interpret more complex behaviors with a bit of creative application. Thus, these primitive actions are not just tools for building AI systems capable of understanding simple narratives; they are also stepping stones towards more sophisticated interpretations of human actions and stories. This capability is crucial for developing AI agents that can interact more naturally and effectively with humans, understanding a wide array of activities and contexts in a manner that feels intuitive and logical.

Application of Primitive Actions in AI Story Understanding

The concept of primitive actions significantly enhances an AI agent's ability to comprehend and interpret stories succinctly expressed through single sentences, such as “Ashok ate a frog” or “Ashok devoured a frog.” These sentences, while varying in their surface structure and verb usage – ate, devoured, consumed, ingested, partook – all fundamentally describe the action of ingestion, which can be classified under the primitive action termed “ingest.”

Despite superficial differences, the core meaning of these sentences remains consistent. The act of devouring may imply a more intense engagement, like ravishing with hands, whereas dining might suggest a more formal eating setting involving utensils. Yet, the primitive action “ingest” adequately captures the essence of these varied expressions. It conveys that Ashok, as the agent, consumes the frog, the object, which transitions from being outside Ashok's body to inside, ultimately resulting in the frog's demise.

This understanding is facilitated by a structured frame associated with each primitive action, a concept repeatedly discussed in this text. The “ingest” frame, for example, is equipped with slots for agents, objects, co-agents, beneficiaries, and other roles that clarify the relationships and dynamics within the story. These slots are pre-filled with default values that help resolve ambiguous or complex situations.

When an AI encounters a sentence like “Ashok ate a frog,” it doesn't merely process the words in isolation but activates the corresponding frame for “ingest.” This triggers a top-down

processing approach where the AI begins to allocate elements from the sentence into appropriate slots within the frame: Ashok fills the agent slot, and the frog occupies the object slot. This method not only streamlines the understanding process but also ensures consistency and depth in the AI's interpretation of stories, grounding its comprehension in a well-defined framework of primitive actions and their associated frames. This structured approach empowers AI agents to parse narratives with precision and nuanced understanding, reflecting a significant advancement in natural language processing and artificial intelligence story comprehension.

Exercise: Identifying Primitive Actions in Sentences

As we investigate deeper into the nuances of commonsense reasoning in AI, let's engage with some practical examples to better understand how to apply the theory of primitive actions to everyday sentences. Here are four sentences, each highlighting a specific verb that we need to analyze for its underlying primitive action:

1. John "pushed" the cart.
2. John "took" the book from Mary.
3. John "ate" ice cream with a spoon.
4. John "decided" to go to the store.

For each of these examples, the challenge is to determine the most appropriate primitive action that accurately captures the essence of the verb used.

Ashok David, what did you think?

David For the first sentence, I debated for a minute between 'propelled' and 'move-object'. Pushing the cart seems to align more with moving the object. However, the specific action of pushing implies initiating movement rather than continuous movement, suggesting a temporary action rather than relocation. Therefore, I chose 'propelled' as it captures the essence of launching the cart into motion without defining a destination. For "John took the book from Mary," the scenario clearly involves a transfer of possession. The book was in Mary's possession and transitioned to John's, which aligns with the primitive action 'move-possession'. In the third example, "John ate ice cream with a spoon," it is quite straightforward. This is a classic case of ingestion, so the primitive action 'ingest' fits perfectly as it describes the act of consuming food. The fourth sentence poses a more complex scenario as it involves internal decision-making. "John decided to go to the store" reflects a cognitive process resulting in a decision, best captured by the primitive action 'conclude', which indicates the finalization of a thought process.

Ashok That sounds right, David.

Each sentence here serves as a compelling exercise in distinguishing the correct primitive actions based on the verbs. This approach not only simplifies the understanding of actions but also enhances an AI's ability to process and interpret human language with greater accuracy and context.

Notice that there are several key observations and challenges to consider. Notably, sentences with multiple verbs introduce a complex layer to the analysis, requiring a nuanced understanding of how each verb interacts within the sentence. For example, the sentence “John decided to go to the store” contains two verbs, “decided” and “go”, each necessitating its own analysis.

Moreover, choosing the most fitting primitive action for a verb is not always straightforward. The verb “pushed”, discussed earlier, serves as a prime example. The selection between 'propelled' and 'move-object' was influenced by the specific interaction of the body with the object – propulsion indicates a form of contact that initiates movement. This subtle distinction highlights the intricacy of determining the right primitive action based on the physical dynamics described in the sentence.

THEMATIC ROLES AND ACTION FRAMES

Building on our discussion of primitive actions, we now examine thematic roles and structured knowledge representations known as action frames. Thematic roles serve as a fundamental concept in understanding how sentences are processed by AI agents, especially in terms of identifying the roles that different concepts play within a sentence.¹³⁵ This understanding is critical for developing AI systems capable of interpreting and responding to natural language effectively.

Exploring Action Frames

Consider the sentence “John pushed the cart.” To process this sentence, an AI agent typically begins parsing from left to right, reflecting common practice in the analysis of English sentences. The first concept identified, “John,” is not a verb and is therefore initially categorized simply as a concept within the agent's memory. The focus then shifts to the next word, “pushed,” which is recognized as a verb.

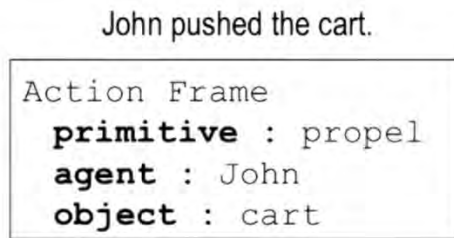


Figure 14-1 Action Frame for the Propel primitive.

This verb activates a specific action frame from the agent's long-term memory – the frame for “propel.” This frame is structured to expect certain thematic roles: an agent and an object. These roles are not just placeholders; they are filled based on specific rules encoded within the frame system:

- **Agent Slot Rule:** If the preceding word to the verb is animate, it is classified as the agent. In our example, “John” precedes “pushed” and is animate, thereby filling the agent slot.
- **Object Slot Rule:** If the word following the verb refers to an inanimate object, it fills the object slot. In this sentence, “the cart” follows “pushed” and is inanimate, thus occupying the object slot.

This structured approach to parsing and understanding sentences illustrates how AI leverages both bottom-up (data-driven) and top-down (knowledge-driven) processing. Initially, the sentence is dissected piece by piece – a bottom-up approach. However, once the verb activates a specific frame, the processing becomes top-down, pulling in knowledge from memory to fill in the thematic roles based on predefined rules. This method allows the AI to generate expectations and make sense of the input effectively, akin to using a hook to catch a fish; once the right frame is activated, the agent can “hook” the correct meanings out of the sentence.

This dynamic interplay between data-driven parsing and knowledge-driven interpretation underpins the agent's ability to understand and respond to natural language, showcasing the sophisticated nature of AI cognition in handling human language.

Deeper Insights into Thematic Roles and Action Frames

The structural integrity of knowledge representations plays a crucial role in how AI systems interpret language. These representations, known as structured knowledge representations, are akin to molecules formed from the linkage of multiple knowledge “atoms.” Unlike simpler forms of knowledge like predicates and logic, which offer minimal structural complexity, structured

knowledge representations provide a robust framework that guides the AI in understanding and placing each piece of information accurately within a sentence's context.

While the example of “John pushed the cart” illustrates a straightforward case of sentence processing using action frames, not all sentences are this simple. Consider a situation where the selected frame does not accurately match the verb in a sentence. For instance, if the verb “push” were mistakenly framed under “move-object” instead of “propel,” the AI might struggle to fit the sentence elements into the incorrect frame, leading to potential errors in interpretation. In such cases, the AI might need to abandon the initial frame and select a more appropriate one, highlighting the flexibility required in AI processing.

Furthermore, sentence processing is rarely linear, especially with complex constructions. An AI system must be adaptable, able to shift frames and interpretations as needed. This adaptability is essential not only for understanding complex or ambiguous sentences but also for appreciating language nuances such as puns, where words may carry multiple meanings depending on the context. For example, the phrase “I was wondering why the ball was getting bigger. Then it hit me.” leverages the dual meanings of “hit,” blending humor with linguistic insight.

The capacity to interpret multiple meanings from a single word or phrase also lays the groundwork for understanding humor. Consider the joke, “Two men walk into a bar; the third one ducks.” Here, the word “bar” serves two roles, playing on the physical and social meanings of the term. This dual interpretation underscores how AI might one day grasp and generate humor based on the structural and contextual nuances of language.

This exploration of thematic roles and action frames not only deepens our understanding of AI's language processing capabilities but also opens the door to more sophisticated interactions, where AI can engage with both the literal and the figurative dimensions of human language.

Exercise on Thematic Roles and Action Frames

Let's engage in a practical exercise to apply the concept of primitive actions and understand how they can elucidate the interactions in simple narratives. Consider the sentence: “John took the book from Mary.” This exercise aims to identify the appropriate primitive action and specify the roles involved in this transaction.

John took the book from Mary.

Action Frame
primitive :
agent :
coagent :
object :

Write the action frame corresponding to the above sentence, using a primitive action.

Primitive Actions			
Move-body-part	Move-object	See	Heart
Expel	Ingest	Smell	Feel
Propel	Speak	Move-possession	Move-concept
	Conclude	Think-about	

Figure 14-2 Example of Thematic Roles for Object Transference.

Ashok What did you write, David?

David For the primitive action of this sentence, I chose “move-possession” as the most suitable. This primitive effectively captures the essence of the sentence, which involves the transfer of ownership of the book from Mary to John. While “move-object” might initially seem appropriate since it involves the physical movement of an object, it does not address the transfer of ownership implied in the action. Thus, “move-possession” more accurately reflects the high-level meaning of the transaction. In filling out the frame for “move-possession”: The agent is John, as he is the one initiating the action, indicated by his placement immediately before the verb. The object is the book, which directly follows the verb, marking it as the item being transferred. The source, as indicated by the preposition “from,” is Mary, from whom the book is being transferred.

Ashok That’s good, David.

This structured approach allows us to dissect and understand the underlying dynamics of simple sentences, highlighting the importance of selecting the appropriate primitive action to capture the full meaning of interactions in narratives. Through this methodology, we not only parse the sentence but also enrich our understanding of how different elements within a sentence contribute to its overall semantic structure.

Integrating Lexical, Syntactical, and Semantical Analysis

In our exploration of thematic roles and action frames, it is essential to recognize how lexical, syntactical, and semantical analyses converge to form a cohesive understanding of sentence structure and meaning. This convergence is pivotal, with semantic analysis playing a critical role.

The semantic framework of the sentence “John took the book from Mary” captures the core semantics, enabling us to draw precise inferences about the narrative. This frame is detailed with slots for agent, object, and source, each filled based on rules derived from lexical and syntactical insights.

- **Lexical Analysis:** This involves understanding the properties of words in isolation and how they interact in phrases. For example, “John” is identified as a concept, specifically an animate one, information typically retrieved from a lexical database.
- **Syntactical Analysis:** This deals with how words combine to form phrases and sentences, adhering to the rules of grammar. In the sentence at hand, “from” operates as a preposition, indicating that the following word, “Mary,” should be interpreted as the source of the action.

Together, these analyses ensure that every element of the sentence is correctly interpreted:

- **Semantics:** Determines the overall meaning conveyed by the sentence.
- **Lexicon:** Provides definitions and roles for individual words.
- **Syntax:** Guides the arrangement of words to ensure grammatical correctness and clarity of meaning.

By integrating these approaches, we can systematically decipher the meaning behind sentences and apply this understanding to more complex texts, enhancing both human and machine comprehension of language. This integration not only clarifies the individual contributions of words and phrases but also solidifies our grasp of their collective impact within the narrative.

It should be evident how vital the structured representation of knowledge through frames and rules is in understanding and interpreting stories. Each frame is not just a static repository of information; rather, it is dynamic, equipped with rules tailored to extract and allocate pieces of information into designated slots.

Understanding Frames and Rules

1. **Rule-Based Slot Filling:** Each slot in a frame has associated rules that dictate how information from the text is interpreted and placed into the frame. For instance, a rule might specify how to identify an agent in a sentence and assign it to the appropriate slot. This ensures that the semantic roles are correctly identified and associated with the right elements of the sentence.
2. **Complexity with Complexity:** As sentences and their structures become more intricate, so do the frames and their rules. A simple sentence might need only a

straightforward rule for each slot, but more complex sentences can require multiple, nuanced rules per slot. This complexity allows the frames to handle a wide variety of linguistic constructions effectively.

3. **Semantic Precision:** The power of frames in semantic analysis is also demonstrated in their ability to facilitate detailed questioning and understanding. By querying a frame, we can retrieve specific information about the story or text. Questions like “Who has the book?”, “Who had the book?”, or “What did John take from Mary?” can be precisely answered. This capability is crucial for implementing commonsense reasoning in AI systems, as it allows machines to generate and verify factual responses based on the structured knowledge.

Frames do not merely store information; they enable AI agents to perform logical reasoning and make informed decisions based on the context provided by the frame. This structured approach to knowledge representation is instrumental in advancing AI’s understanding of language beyond mere word recognition to grasping deeper meanings and implications.

By harnessing the combined strength of frames and rules, we lay the groundwork for sophisticated AI systems capable of engaging in meaningful dialogues and answering complex queries, bringing them a step closer to human-like comprehension and interaction. This structured knowledge approach underscores the importance of semantic accuracy and the dynamic interplay between different elements of language understanding.

IMPLIED ACTIONS AND COMMONSENSE REASONING

Understanding natural language often requires more than just literal interpretation; it involves recognizing implied actions that aren’t directly stated but are understood through context and background knowledge. This deeper level of interpretation is essential for machines to truly grasp the subtleties of human communication.

The structure of a sentence and the background knowledge of the listener or reader play crucial roles in comprehending the full meaning behind words. Take the sentence “John fertilized the field.” At first glance, the verb “fertilized” might not directly correspond to any specific primitive actions like those discussed in prior sections. However, a closer examination with commonsense reasoning reveals an implied action. What the sentence essentially conveys is that “John put the fertilizer on the field,” which aligns more straightforwardly with the primitive action known as “move-object.”

The initial step in processing such a sentence within an AI system involves attempting to match the verb with a known primitive action. If this direct approach fails, the next step is to rethink the sentence to highlight any implied actions that might be inferred from the context. This adjustment is vital because it allows the AI to apply a more suitable primitive action that it can understand and execute.

For instance, transforming “John fertilized the field” to “John put the fertilizer on the field” simplifies the mapping process for the AI. The verb “put” directly corresponds to “move-

object,” which is a part of the AI's repertoire of primitive actions. Once this rephrasing is done, the AI can employ the move-object frame, filling in the slots with “John” as the agent, “fertilizer” as the object, and “field” as the destination.

```
Action Frame
primitive : move-object
agent : John
object : fertilizer
destination : field
```

Figure 14-3 Action frame constructed from the sentence “John fertilized the field.”

This method illustrates how AI can use commonsense reasoning to deduce the implied actions within a sentence. Such reasoning is not about adhering strictly to the literal text but rather about interpreting what the text aims to convey based on common understanding and contextual knowledge. By recognizing that certain actions might be implied rather than explicitly stated, AI can more effectively parse and understand complex sentence structures.^{136,137}

This approach not only aids in comprehension but also enhances the AI's ability to interact in a human-like manner, acknowledging that language is often nuanced and metaphorical. The ability to infer implied actions is a significant stride towards achieving more sophisticated and intuitive AI communication.

Exploring Implied Actions Through Primitive Action Frames

As we investigate deeper into understanding language and its nuances, it becomes apparent that not all verbs directly map to the predefined primitive actions we have explored. This exercise will help us grasp how implied actions can be inferred from a sentence, enhancing our comprehension of complex verbal interactions.

Exercise: Analyzing “Bill shot Bob” through Primitive Actions

Ashok Let's examine the sentence “Bill shot Bob,” which at first glance does not clearly align with any of our listed primitive actions. This requires us to think about the implied actions that might be represented within the sentence. What did you write, David?

David I reinterpreted the sentence as 'Bill propelled a bullet into Bob.' Analyzing the verb 'shot,' it essentially involves the action of pulling a trigger, resulting in the high-velocity propulsion of a bullet from a gun. Thus, converting 'shot' to 'propelled a bullet' fits our understanding. In this frame, 'propel' becomes the primitive action. Here's how I filled the slots:

Primitive Action: Propel

Agent: Bill (as the one who initiates the action)

Object: Bullet (the entity being propelled)

Destination: Bob (where the bullet ends up)

Ashok That's a thorough analysis, David. But consider another perspective. What if Bill had a camera instead of a gun?

David: Interesting point, Ashok. Had 'shot' referred to photography, I would interpret it as 'Bill took a picture of Bob,' transforming 'shot' into a different context. This would likely shift the primitive action from 'propel' to perhaps 'capture' or 'record,' depending on how we classify photographic actions within our primitive actions framework. This ambiguity highlights the flexibility needed in parsing language and underscores the role of context in determining meaning.

Ashok Exactly, David. This ambiguity and the necessity for context highlight how crucial it is to consider multiple interpretations. The ability to toggle between these interpretations is fundamental in building AI systems that can understand and interact with human language effectively.

This exercise not only demonstrates the application of primitive actions to interpret sentences but also emphasizes the complexity of language where words like “shot” can have multiple meanings based on context. Such exercises are vital in advancing our understanding and development of AI systems capable of nuanced language comprehension and interaction. The exploration of multiple potential interpretations based on the context highlights the intricacies involved in AI's approach to human-like understanding and the need for sophisticated models that consider prior and subsequent context for accurate interpretation.

SUB-ACTIONS AND COMMONSENSE REASONING

This section explores the intricate connection between action frames and the sub-actions that are often implied but not explicitly stated in language, a key aspect of commonsense reasoning.

Let's analyze the sentence "Ashok put the wedge on the block" to uncover the deeper layers of commonsense reasoning that an AI agent can utilize.

Imagine an AI agent tasked with interpreting the sentence: "Ashok put the wedge on the block." The agent employs the action frame associated with the primitive action 'move-object'. This action frame, as we know, prompts the agent to fill in specific slots: the agent (Ashok), the object (wedge), and the destination (block).

Ashok put the wedge on the block.

```
Action Frame
primitive : move-object
agent : Ashok
object : wedge
destination : block
```

Figure 14-4 Action Frame for move-object.

However, commonsense reasoning extends beyond merely filling in these slots. Let's pose a question that probes deeper into the context of the action: "Did Ashok's hand move when he was putting the wedge on the block?" The expected response would be "yes," indicating not just an understanding of the direct action but also the implied actions – Ashok grasping the wedge, moving his hand, and then releasing the wedge onto the block. This sequence of actions demonstrates a level of reasoning that goes beyond the literal words, tapping into the background knowledge about how such actions are typically performed.

To enhance an AI's capability to perform such nuanced reasoning, it's crucial to consider not just the primary actions but also the sub-actions involved. These sub-actions represent the finer details of the main action, such as the movement of Ashok's fingers or the manner in which the wedge is positioned. For instance, the micro-story for this action might include that fingers first close around the wedge, then the hand moves, and finally, the fingers release the wedge onto the block.

This level of detail enriches the AI's understanding and allows it to make more sophisticated inferences about the world. It showcases the essential knowledge the AI must possess to not just understand but also predict and interact with the world in a manner akin to human reasoning.

The understanding of sub-actions fosters an AI's ability to infer and reason in ways that mirror human thought processes. It allows the AI to anticipate actions and their outcomes, enabling it to engage in what we perceive as commonsense reasoning. By equipping an AI with

knowledge not only of primary actions but also of the subtler, often unspoken steps that comprise these actions, we enable it to construct more accurate and contextually appropriate responses and interactions.

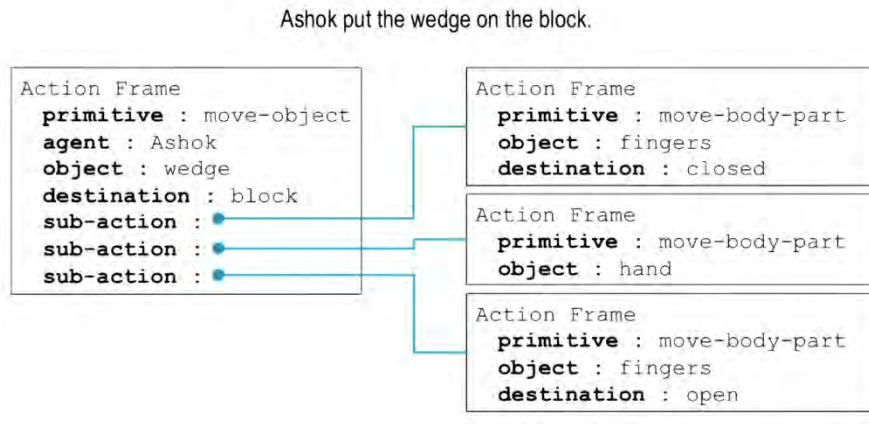


Figure 14-5 Sub-actions for move-object.

Thus, the inclusion of sub-actions within the framework of action frames is not merely a technical enhancement but a significant leap towards imbuing AI systems with a robust layer of commonsense reasoning, bridging the gap between mechanical response and intuitive understanding.

The Complexity of Primitive Actions and Sub-Actions

In the exploration of AI and commonsense reasoning, the classification and utility of primitive actions warrant a deeper analysis. A notable point of discussion centers around the primitive action “move-object,” traditionally linked to actions like transporting, carrying, or moving within the AI’s conceptual framework.

Although “move-object” is considered a primitive action, it can be decomposed into more granular actions that reveal a detailed choreography of movement. For example, when considering the action described as “Ashok puts the wedge on the block,” this decomposition helps illustrate not just the action itself but the mechanics – how each part of the body cooperates to accomplish the task.

The discussion on the decomposition of primitive actions into sub-actions highlights several challenging aspects and limitations of current AI models in interpreting and understanding complex human activities:

1. **Multiplicity of Stories:** The vast number of potential narratives and scenarios an AI might encounter poses a significant challenge. Can an AI system feasibly have a pre-modeled response for every action or scenario it might face?
2. **Scope of Theory:** Current theories and models do not cover all possible narratives or actions, indicating a gap between the theoretical models and their practical applicability. Many complex stories remain beyond the scope of existing AI capabilities.
3. **Computational Tractability:** Expanding stories into detailed sub-actions raises significant computational challenges. As the number of conceivable scenarios increases, the computational demand can become overwhelming, questioning the practicality of such detailed models in real-world applications.

The nuanced discussion about the nature and application of primitive actions in AI systems underscores the depth of analysis required to develop AI systems capable of commonsense reasoning. It also highlights the complexities involved in scaling these systems to handle a broad spectrum of human-like understanding and interactions. This exploration into the granularity of actions within AI frameworks continues to push the boundaries of what is computationally and theoretically feasible in the field of artificial intelligence.

STATE CHANGES IN COMMONSENSE REASONING

In the domain of commonsense reasoning within AI, understanding the dynamics of state changes through action frames presents a significant theoretical advancement. Consider the sentence: “Ashok enjoyed eating the frog.” This sentence introduces more than the act of ingestion; it embeds an emotional response to the action.

To decipher this, we first identify the primitive action associated with “eating” – ingest. Here, Ashok is the agent, and the frog serves as the object of this action. However, the crux of the sentence lies in the emotional response it encapsulates, which is reflected in a state-change frame. This frame extends beyond the physical action to include the resultant emotional state – Ashok's happiness from the action.

Ashok enjoyed eating the frog.

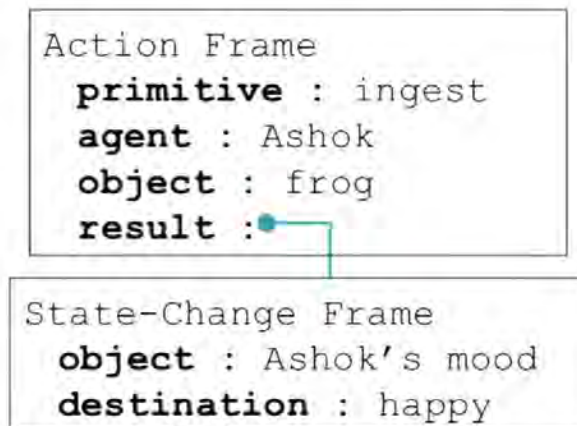


Figure 14-6 State-change frame.

This state-change is cataloged in what we term as a “result” slot within the action frame, indicating a transition from a neutral state to happiness as a consequence of the action. This transformation is crucial as it integrates the emotional outcome directly tied to the physical action, highlighting the complex layering of human actions and reactions that AI seeks to understand.

The challenge then arises in dual interpretations: how does an AI discern the primary focus of a sentence – whether on the action (eating) or the emotional response (enjoying)? To address this, separate but related frames can be constructed: one for the action of ingestion and another for the emotional response, linked through their relational impact on Ashok. This approach allows the AI to flexibly interpret the emphasis of the sentence based on contextual cues and predefined semantic rules, ensuring a nuanced understanding that aligns more closely with human cognition.

By engaging with these dual frames, AI can navigate the intricate landscape of language where verbs like “enjoy” signify an internal state change rather than an outward action, demanding a deeper integration of semantic and syntactic analysis to fully capture the sentence's meaning. This method illustrates how advanced AI systems can employ detailed frameworks to process and respond to complex human expressions, bridging the gap between mere word recognition and true contextual understanding.

Generic Actions in Commonsense Reasoning

In the exploration of commonsense reasoning, there are instances where the specific actions described by verbs are not immediately clear or directly mappable to known primitive actions. Consider the sentence: “Susan comforted Jing.” The action of comforting is intuitive to humans – we understand it as an act that improves another's emotional state, in this case making Jing feel happier. However, pinning down the exact nature of the comforting action in computational terms presents a challenge.

Susan comforted Jing.

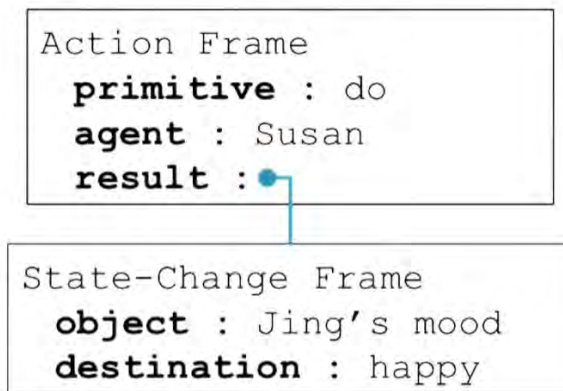


Figure 14-7 Generic Actions.

When the specific actions are ambiguous or broadly defined, AI systems can employ a generic primitive action termed “do.” This approach is used when the AI cannot decisively determine the precise action from the verb alone. For instance, while the verb “comforted” suggests a positive interaction, the exact mechanics of what Susan did to comfort Jing might be complex and varied, involving multiple possible actions that are not explicitly stated in the sentence.

The generic action “do” allows AI agents to handle such vagueness by enabling them to perform commonsense reasoning even with partial information. It effectively allows the agent to acknowledge that something significant occurred that altered Jing's emotional state, without requiring a detailed breakdown of the action. Thus, the agent concludes that Susan's actions resulted in Jing feeling happier, capturing the essence of the event.

This method reflects a sophisticated level of interpretation that mirrors human reasoning, where the understanding of actions can sometimes rely more on the outcomes than on the detailed processes leading to those outcomes. This flexible approach helps in developing AI

systems that can better interact with and understand human-like narratives where not every detail is always known or necessary to infer the intent or result of actions.

Understanding Complex Sentences with Multiple Verbs

In the study of commonsense reasoning, complex sentences containing multiple verbs present unique challenges for AI interpretation. Consider the sentence: “Maria told Ben to throw the ball.” This sentence involves two distinct actions – telling and throwing – which must be understood and processed separately yet cohesively by an AI agent.

The sentence begins with the subject “Maria,” which is not a verb, hence it is cataloged under a concept list temporarily ignored during the initial verb analysis. The first verb encountered is “told,” which triggers the extraction of the first action frame. Here, the primitive action is identified as “speak,” reflecting the communicative action Maria performs.

Action Frame for 'Told':

- **Primitive:** Speak
- **Agent:** Maria
- **Result:** Instruction for subsequent action

Upon processing the verb “told,” the AI then encounters the next key verb, “throw,” indicating a physical action. This shifts the processing to a new frame where the primitive action is “propel,” corresponding to the act of throwing. In this context, Ben becomes the agent of this action, and the ball is the object being acted upon.

Action Frame for 'Throw':

- **Primitive:** Propel
- **Agent:** Ben
- **Object:** Ball

The crucial part of processing such sentences lies in linking these two action frames effectively. The result of Maria’s action (speaking) effectively sets the stage for Ben’s action (throwing). This interconnection illustrates how one action frame can influence or trigger another within a complex sentence structure, demonstrating the AI’s ability to handle layered instructions.

Maria told Ben to throw the ball.

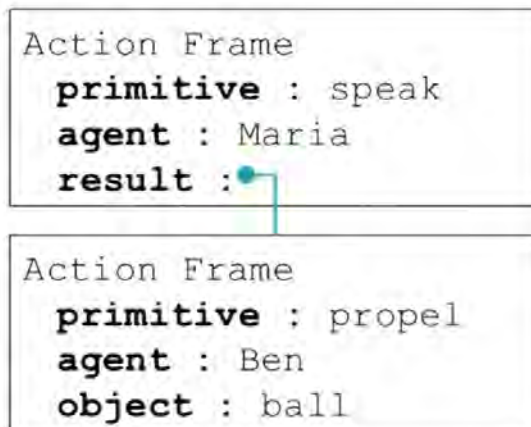


Figure 14-8 Action frames for sentences with two verbs.

This method showcases an advanced level of AI processing, where separate actions are not only recognized and parsed but also connected to reflect the sequence and causality present in human language. This capability is fundamental in enabling AI to understand and interact with more dynamically structured human communications.

Analyzing Sentences with Multiple Verbs

In exploring the capabilities of AI in understanding complex sentences with multiple verbs, let's dissect the sentence "Ashok enjoyed eating a frog" to illustrate how different verbs can be captured within action frames.

The sentence incorporates two distinct actions: enjoying and eating. Each action can be represented by its own frame, demonstrating the multifaceted nature of commonsense reasoning in AI. For the frame that describes the physical act of eating we have *Ingest* as the primitive, Ashok as the agent, and the Frog as the object. For the frame that describes the emotional state resulting from the first action, we have *Feel* as the primitive, Ashok as the agent, and *Enjoyment* as the result.

Despite these representations, challenges persist in fully capturing the nuances of such sentences. The dual verbs introduce complexities in determining whether to treat the enjoyment and the act of eating as sequential or simultaneous actions, or whether they should be integrated into a single frame that encompasses both the action and its emotional impact.

Further complications arise when considering the narrative context or the sequence of events. Understanding what occurred before or after “Ashok enjoyed eating a frog” can provide crucial insights that resolve ambiguities inherent in standalone sentences. This approach shifts from analyzing isolated sentences to considering entire stories or discourses, enhancing the AI's ability to derive meaningful interpretations based on broader narrative structures.

This examination underscores the ongoing challenges and developments in AI's ability to parse and make sense of complex human language, particularly when multiple verbs and actions are involved. As AI continues to evolve, its capacity to understand and reason about such complexities will significantly impact its applications in natural language processing and beyond.

Exercise on Constructing Action and State-Change Frames

In this exercise, we will investigate constructing both action and state-change frames for two sentences, enhancing our understanding of how primitive actions and their results can depict complex meanings.

Complete the frame representations for these sentences.

Anika decided to have a glass of water.

☐ Action ☐ State Change

result : ●

☐ Action ☐ State Change

Marc loved watching TED talks.

☐ Action ☐ State Change

result : ●

☐ Action ☐ State Change

Primitive Actions

Move-body-part	Move-object	See	Hear
Expel	Ingest	Smell	Feel
Propel	Speak	Move-possession	Move-concept
	Conclude	Think-about	

Figure 14-9 State Change Example

Ashok Consider the two sentences: “Anika decided to have a glass of water.” And “Marc loved watching TED Talks.” How did you fill in these frames, David?

David For the sentence about Anika, I started from the beginning. Anika, clearly the agent, encounters the verb “decided,” which we align with the conclude primitive action. Thus, the action frame for Anika is:

Primitive: Conclude

Agent: Anika

Result: Decision to take a new action

The result of this decision leads to another action, described by the ingest primitive, with the object being the glass of water. Thus, Anika's decision leads directly to her ingesting a glass of water.

For the second sentence regarding Marc, the primary action is his engagement with TED Talks. Here, the primitive is see, where:

Primitive: See

Agent: Marc

Object: TED Talks

The verb “loved” enhances this scenario, indicating a change in Marc's mood as a result of the action, transitioning to a state of happiness. Hence, the state-change frame would look like:

Object: Marc's mood

Destination: Happy

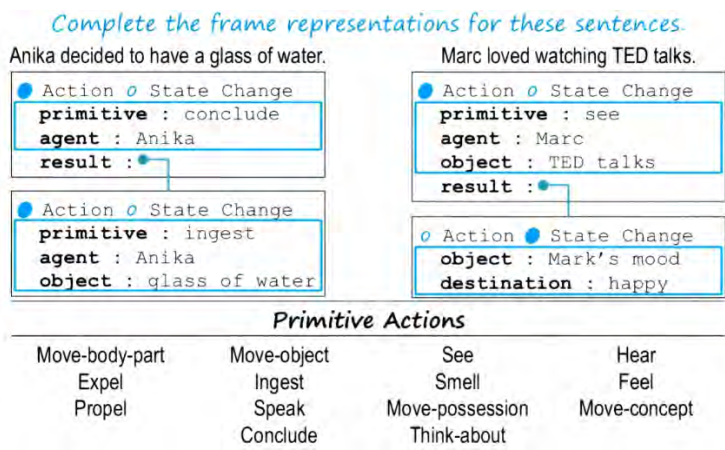


Figure 14-10 State Change illustrating David's answers.

Ashok That's insightful, David. Note how this approach mirrors our earlier analysis of "Ashok enjoyed eating the frog." In both instances, the sentences encompass two interlinked actions or states – Marc's watching and his enjoyment, similar to Ashok's eating and his enjoyment. This duality in representation captures the nuanced relationship between the actions and their emotional outcomes, showcasing the depth of commonsense reasoning needed to fully interpret such sentences.

In exploring how AI agents can comprehend and interact with human-like stories, it becomes evident that our engagement with AI is evolving far beyond traditional interfaces. Consider how Watson and Siri process information; their methodologies provide a blueprint for future interactions.

The essence of our interaction with machines is rapidly shifting. In the near future, rather than relying on keyboards and mice, we will communicate with AI through narratives and dialogues. We will share stories – like Anika deciding to have a glass of water or Ashok enjoying his meal – and in doing so, embed these tales with layers of context and ambiguity that challenge the AI to engage in commonsense reasoning.

This serves as an introduction to harnessing the power of narrative to enable machines to understand and participate in human experiences more deeply. As we progress, the discussions will examine more intricate stories, further exploring the sophistication of commonsense reasoning. This journey into the narrative-driven interactions between humans and machines will continue in the subsequent chapter, expanding upon the foundational concepts introduced here.

COMMONSENSE REASONING AND COGNITION

The intricate link between commonsense reasoning and human cognition underscores our profound yet partially grasped understanding of cognitive processes. Consider a simple scenario where you're asked to check the weather outside. Naturally, you wouldn't contemplate jumping out the window to fulfill this task. Your decision not to leap stems from commonsense reasoning, guided by a clear understanding of goals and context – key elements that determine the appropriateness of actions in given situations.

This same principle applies to natural language understanding, where context helps disambiguate meanings and guide decision-making processes. For instance, the word "take" could have multiple interpretations depending on its usage within a context, influencing how we understand and react to instructions.

Furthermore, our interactions within the social sphere are also permeated by commonsense reasoning. This extends to the psychological concept known as the "theory of mind," where individuals attribute thoughts, desires, and intentions to others. This ability not only facilitates social interactions but also allows us to make educated guesses about others' behaviors based on the goals and desires we ascribe to them.

Through these examples, we see how commonsense reasoning bridges the gap between simple physical actions and complex social interactions, playing a crucial role in both navigating the physical world and understanding the intricate web of human relationships.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

Commonsense reasoning, as explored in this chapter using primitive actions and action frames, provides a structured way to represent and infer everyday knowledge. This KBAI approach offers valuable synergies with modern AI paradigms:

- **Natural Language Processing (NLP) & Understanding (NLU):**
 - **Deepening Semantic Interpretation:** While modern NLP models (like transformers) excel at capturing statistical patterns in language, they often lack robust commonsense understanding. Integrating KBAI's primitive actions and thematic role frames can provide a more explicit semantic layer, helping NLU systems to better grasp the underlying meaning, infer implied actions, and resolve ambiguities that require real-world knowledge (e.g., understanding “Bill shot Bob” with a gun vs. a camera).
 - **Story Comprehension:** For AI to understand narratives beyond surface-level text, it needs to infer causal relationships, character motivations, and likely outcomes – all hallmarks of commonsense reasoning. Primitive action ontologies can provide a vocabulary for representing these deeper narrative structures.
 - **Question Answering:** Answering “why” or “how” questions often requires commonsense inferences. An AI agent equipped with action frames and knowledge of typical sub-actions can generate more plausible and complete answers.
- **Large Language Models (LLMs) & Generative AI (GenAI):**
 - **Grounding and Constraint:** LLMs can generate fluent and creative text, but their outputs may sometimes be nonsensical or factually incorrect from a commonsense perspective. KBAI's structured commonsense knowledge (e.g., typical sequences of actions, state changes associated with primitives) can be used to ground LLM outputs, verify their plausibility, or constrain generation to be more aligned with real-world expectations.
 - **Controllable Generation:** By specifying goals or desired state changes using a framework of primitive actions, it might be possible to guide LLMs to generate more targeted and coherent narratives or plans.
 - **Explanation of LLM Behavior:** If an LLM's reasoning can be partially mapped to an underlying commonsense framework (like primitive actions), it could offer a pathway to more interpretable explanations for its outputs.

- **Reinforcement Learning (RL) & Robotics:**
 - **Action Abstraction:** Primitive actions can serve as a basis for action abstraction in RL. Instead of learning policies over low-level motor commands, an agent could learn policies over a set of meaningful primitive actions, simplifying the learning problem.
 - **Goal-Oriented Behavior:** For robots to act intelligently in human environments, they need commonsense understanding of objects, actions, and their consequences. Knowing that “fertilizing the field” implies “moving fertilizer to the field” is crucial for a robot to plan and execute the task correctly.
 - **Safe AI:** Commonsense reasoning about the likely outcomes of actions (including sub-actions and state changes) is vital for ensuring robots and other AI agents operate safely and avoid harmful or absurd behaviors (like jumping out a window to check the weather).
- **Knowledge Graphs:** Ontologies of primitive actions, their thematic roles, and associated state changes can be explicitly represented in knowledge graphs. This allows for structured storage and retrieval of commonsense knowledge, which can then be queried by various AI systems, including NLP models and planners.
- **Explainable AI (XAI):** When an AI system's reasoning is based on an explicit framework of commonsense principles like primitive actions, its decisions and inferences become more transparent and explainable to humans. We can trace how an understanding of a situation was built up from these fundamental components.

By integrating the explicit, structured approach of KBAI's commonsense reasoning with the pattern-recognition strengths of modern machine learning, AI systems can achieve a more robust, nuanced, and human-like understanding of the world. This is particularly important for tasks requiring interaction in complex, dynamic environments or deep comprehension of human language and behavior.

CHAPTER SUMMARY

This chapter examined the foundational aspects of commonsense reasoning and its crucial role in artificial intelligence. We explored how AI systems, much like the renowned Watson and Siri, interpret simple narratives, facilitating human-like interactions beyond the confines of conventional input devices like keyboards and mice.

At the core of our discussion were primitive actions, the elemental units of understanding that allow AI to process and respond to human language. We introduced a set of fourteen such primitives (e.g., move-object, ingest, propel, speak, conclude) that form an ontology for representing a wide range of human activities. This framework helps AI agents to standardize the interpretation of diverse verbs that might convey similar underlying meanings (e.g., “ate,” “devoured,” “consumed” all mapping to “ingest”).

We further examined how these primitives are assembled into more complex structures using action frames, which include slots for thematic roles (agent, object, destination, etc.). This structured approach, combining bottom-up parsing with top-down frame activation, enables machines to comprehend sentences and infer implied actions (e.g., “John fertilized the field” implies “John put fertilizer on the field”). The importance of sub-actions in enriching this understanding was also highlighted, along with the concept of state-change frames to capture the results of actions, including emotional or physical changes. The use of generic primitives like “do” was discussed for handling ambiguous actions.

The complexity of commonsense reasoning was also highlighted through its application in both physical actions and social interactions. We touched upon the theory of mind, which posits how humans attribute mental states to others, a concept AI attempts to mimic to better understand and predict human behavior.

Finally, we discussed the integration of KBAl's commonsense reasoning principles with modern AI paradigms. This includes deepening semantic interpretation in NLP, grounding LLM outputs, providing action abstractions for RL in robotics, populating knowledge graphs, and enhancing explainable AI.

By linking commonsense reasoning to human cognition, we established a profound connection that underpins the way AI systems are designed to interpret the world. The chapter concluded by setting the stage for further exploration into how these principles are applied in more intricate scenarios, promising to investigate how scripts – complex sets of interrelated actions – enable AI to handle extended dialogues and broader story contexts. This exploration not only broadens our understanding of AI's capabilities but also enhances our anticipation of its future evolution.

ADDITIONAL READINGS & REFERENCES

- 120 Davis, E. & Marcus, G. Commonsense reasoning and commonsense knowledge in artificial intelligence. *Communications of the ACM* **58**, 92-103 (2015).
- 121 McCarthy, J. Artificial intelligence, logic, and formalising common sense. *Machine Learning and the City: Applications in Architecture and Urban Design*, 69-90 (2022).
- 122 Mota, T., Sridharan, M. & Leonardis, A. Integrated commonsense reasoning and deep learning for transparent decision making in robotics. *SN Computer Science* **2**, 242 (2021).
- 123 Piantadosi, S. T., Tenenbaum, J. B. & Goodman, N. D. The logical primitives of thought: Empirical foundations for compositional cognitive models. *Psychological review* **123**, 392 (2016).
- 124 Blass, J. & Forbus, K. in *Proceedings of the AAAI Conference on Artificial Intelligence*.
- 125 Lebowitz, M. in *Proceedings of the Annual Meeting of the Cognitive Science Society*.
- 126 Tarau, P. & Figa, E. in *Proceedings of the 2004 ACM symposium on Applied computing*. 39-44.
- 127 Guan, J., Huang, F., Zhao, Z., Zhu, X. & Huang, M. A knowledge-enhanced pretraining model for commonsense story generation. *Transactions of the Association for Computational Linguistics* **8**, 93-108 (2020).

- 128 Liu, H. & Singh, P. in *AAAI/LAAI*. 957-958.
129 Yang, P. *et al.* in *IJCAI*. 7.
130 Maurya, H. C., Gupta, P. & Choudhary, N. Natural language ambiguity and its effect
on machine learning. *International Journal Of Modern Engineering Research* **5**, 25-30 (2015).
131 Wälchli, B. & Cysouw, M. Lexical typology through similarity semantics: Toward a
semantic map of motion verbs. *Linguistics* **50**, 671-710 (2012).
132 Habel, C. Stories—An artificial intelligence perspective (?). *Poetics* **15**, 111-125 (1986).
133 Shridhar, K. *et al.* in *2019 International Joint Conference on Neural Networks (IJCNN)*. 1-6
(IEEE).
134 Schank, R. C. *The fourteen primitive actions and their inferences*. Vol. 183 (Citeseer, 1973).
135 Davis, A. R. Thematic roles. *Semantics. An international handbook of natural language meaning*
1, 399-419 (2011).
136 Tandon, N. *et al.* Reasoning about actions and state changes by injecting commonsense
knowledge. *arXiv preprint arXiv:1808.10012* (2018).
137 Yang, S., Gao, Q., Saba-Sadiya, S. & Chai, J. in *Proceedings of the 2018 Conference on
Empirical Methods in Natural Language Processing*. 2627-2637.

Chapter 15

Scripts

INTRODUCTION TO SCRIPTS

In this chapter, we investigate a sophisticated form of knowledge representation known as scripts, which play a crucial role in deciphering complex discourses, stories, and everyday scenes. Building upon the concepts explored in previous discussions – frames, understanding, and commonsense reasoning – scripts represent a natural progression in our journey to equip artificial intelligence with human-like interpretative skills.

Scripts synthesize the foundational principles of frames with the dynamic aspects of understanding and reasoning, allowing us to model how events typically unfold in real-world scenarios. This chapter aims to unfold the intricate layers of scripts – starting with their definition and moving towards a deeper exploration of their structure and utility.

We will explore both the form and the content of scripts, understanding how they are constructed and how they function within varied contexts to generate predictable patterns and expectations. By examining how scripts enable us to anticipate outcomes and behaviors in familiar settings, we can see their power in making sense of the narratives that permeate our daily lives.

Additionally, we will discuss the hierarchical nature of scripts, illustrating how complex interactions are often built upon simpler, underlying structures that guide behavior in both humans and machines. Through this exploration, you will gain insights into how scripts not only

enhance the comprehension abilities of AI systems but also mirror the cognitive processes humans use to navigate the social world.

Join us as we dissect the role of scripts in advancing the capabilities of AI, making interactions with machines more intuitive and aligned with human expectations. This chapter promises a detailed examination of scripts, enriching your understanding of both artificial intelligence and the cognitive frameworks that underpin human reasoning.

To set the stage for our exploration of scripts, consider the current state of interaction between humans and machines, exemplified by AI systems like Watson and Siri.^{138,139} These programs typically understand and respond to queries that are confined to one sentence, reflecting a limited capacity for narrative comprehension. However, the real world operates largely through stories – complex, multi-sentence constructs that convey richer, more nuanced information.

UNDERSTANDING STORIES AND THE NEED FOR SCRIPTS

Stories are fundamental to the way cognitive agents – including both humans and AI – navigate social contexts and make decisions. They play a pivotal role in enabling and enhancing commonsense reasoning, a crucial aspect of cognitive processing that allows agents to make sense of the world around them. For instance, take a simple narrative composed of two sentences: Ali asks, “Do you think we’ll have many customers in the next half hour?” to which Sarah responds, “Go ahead and grab your lunch.” This brief exchange, while simple, encapsulates a scenario filled with implicit cues and context that inform our understanding of the situation.

Ashok What do you think, David, does this story make sense to you?

David Yes, it does make sense to me. I envision a scenario where Ali and Sarah are coworkers, and Sarah holds some authority over Ali. Ali glances around, noting the current lull in customer activity, and wonders if this could be an opportune moment to take his lunch break. Yet, he's concerned about the potential for a sudden influx of customers which would leave Sarah to manage alone. His question to Sarah, “Do you think we’ll have many customers in the next half hour?” reflects this deliberation. Sarah, aware that it's around lunchtime and recalling Ali's earlier remark about skipping breakfast, interprets his query as not just about customer volume but as hinting at his need for a break. She responds affirmatively, encouraging him to take his lunch now. This interpretation, though it links two seemingly disconnected sentences, is grounded in an understanding of their work dynamics and mutual awareness of each other's schedules and habits.

Ashok That's a great interpretation, David. You've clearly got a knack for reading between the lines. Your ability to draw inferences that weren't explicitly stated in the dialogue is a prime example of commonsense reasoning. How did you deduce that it might be lunchtime and that Ali was motivated by his hunger, or that Sarah has a supervisory role over Ali? This kind of reasoning is crucial for understanding narratives and interactions in a deeper, more contextual way.

We have encountered similar narratives before. Consider another instance: "Ashok wanted to become rich. He got a gun." This brief narrative might trigger immediate assumptions. You might think that Ashok, likely a professor, would not typically become wealthy through academic endeavors, even if he teaches a lucrative online Master's course. Alternatively, you might draw on more conventional narratives of quick wealth acquisition involving dubious means, such as bank robbery, which obviously involves weapons. This leads you to make a connection between Ashok acquiring a gun and his aspiration for sudden wealth.

As humans, we effortlessly piece together such stories. Given just two sentences, we can link them causally and coherently, sometimes relying on our vivid imaginations. The real question then is: How can we replicate this ability in AI agents? What type of knowledge and inferential capabilities must an AI possess to seamlessly connect and understand such narratives? This challenge forms a core part of developing AI systems that can engage in human-like reasoning and story comprehension.

SCRIPTS: STRUCTURED KNOWLEDGE FOR SEQUENCES OF EVENTS

Until now, we have predominantly utilized the metaphor of human-computer interaction to discuss how humans might communicate with machines through storytelling. However, consider a slightly different scenario: intelligent agents coexisting with humans. These agents encounter human interactions and world events not as orderly sequences, but as disparate occurrences that they must logically connect.

For instance, consider the Roomba – a familiar autonomous vacuum that operates independently without manual guidance. Imagine a more sophisticated version of this device, one that remains dormant until needed.¹⁴⁰ Suppose it detects a sudden noise, such as the sound of something crashing and a subsequent call for paper towels. Although these sounds don't immediately form a coherent story, an advanced Roomba, equipped with the right contextual knowledge, could deduce that a spill has occurred and thus infer its services are required. This capability to make causal connections from fragmented inputs illustrates a significant leap towards integrating AI more seamlessly into everyday human environments.

Consider a hypothetical scenario: Bob visits a restaurant, where he waits a considerable time before being served. When his order finally arrives, the hamburger is overcooked and unsatisfactory. Bob is understandably displeased and leaves without finishing his meal. Would

you surmise that Bob left a generous tip? Likely not. Most would infer that the combination of poor service and subpar food would lead to a small or nonexistent tip. But how do we arrive at this conclusion?

This intuitive understanding stems from our collective experiences and societal norms about dining in restaurants. We recognize patterns – service quality and customer satisfaction often dictate the tip left behind. This example illustrates how stories are not just sequences of events; they are frameworks within which we contextualize and predict outcomes based on embedded social scripts.

Scripts are a structured form of knowledge representation that encapsulates typical sequences of events or behaviors in familiar settings.¹⁴¹ Scripts enable us to predict and understand outcomes by providing a scaffold for interpreting complex interactions. Through scripts, we can seamlessly connect disparate events and anticipate future occurrences, enhancing our ability to navigate and make sense of the world around us.

Ashok So, David, let's imagine you go to a coffee house. What do you typically do there?

David Well, the first thing I do is walk through the door and get in line. I wait for the line to clear. Then I approach the barista and give him my order. He processes it, and I pay. After that, I wait for my drink to be prepared.

Notice that I didn't need to ask David which coffee house he visits, what time of day it is, what exactly he orders, or who the cashier is. David has a mental script for this scenario. He knows the sequence of actions expected at a coffee house, and this script guides his behavior. It also sets his expectations, such as receiving a total from the cashier and then receiving his drink. If these expectations are not met, David immediately knows something is amiss. This is the power of a script – it helps us generate and manage expectations in familiar situations, illustrating how scripts guide our interactions in everyday environments.

DEFINING SCRIPT COMPONENTS

A script, in the context of knowledge representation, is a structured framework that captures a causally coherent sequence of events.^{142,143} “Causal” here implies that one event triggers another. For example, when David visits a coffee house and approaches the counter, his action prompts the barista to engage and ask what he would like to order. Each event naturally and logically leads to the next in a sequence that makes sense within the given context. This coherence ensures that the events align with real-world expectations; for instance, ordering coffee leads to receiving coffee, not an illogical response like being slapped by the barista. Scripts mainly involve observable events, although some, such as decisions or conclusions, might occur internally within the actor's mind. This structure allows scripts to effectively model both the visible and cognitive actions in everyday scenarios.

Scripts, as structured knowledge representations, are composed of six integral parts, each contributing to the script's functionality and coherence:

1. **Entry Conditions:** These are the prerequisites required to initiate the script. For instance, in a restaurant script, the entry conditions might include the presence of a hungry customer who has money to spend.
2. **Result:** This component outlines the outcomes that emerge after the script's execution. In the restaurant scenario, typical results would be the restaurant owner earning more money, the customer spending money, satisfying their hunger, and experiencing pleasure from the meal.
3. **Props:** Props refer to the physical or conceptual objects involved in the script. In a restaurant, props would include items like tables, menus, and food, which play roles in the script's execution.
4. **Roles:** These are the participants or agents involved in the script. For a restaurant, roles would include the customer, restaurant owner, and service staff such as waiters or waitresses.
5. **Tracks:** Tracks represent variations or specific versions of a script based on the context. For example, a restaurant script could have different tracks for a coffee house, a fast food outlet, or a fine dining establishment, each with unique characteristics and expectations.
6. **Scenes:** These are the detailed sequences of events that unfold during the script. A restaurant script might include scenes such as entering the restaurant, ordering food, and receiving the meal.

Together, these elements form a comprehensive script, akin to a large molecule in a knowledge representation metaphor, where smaller, atomic representations come together to form a complex and functional whole. This molecular structure allows scripts to effectively model real-world interactions and scenarios in a detailed and predictable manner.

Here's how the “Restaurant Script” is elaborated within the context of formal dining:

```
Script
script : restaurant
track  : formal dining
props : tables, menu, check,
        money, F = food, P = place
roles : S = customer, W = waiter,
        C = cook, M = cashier,
        O = owner
entry : S is hungry, S has money
result : S has less money,
        O has more money,
        S is not hungry,
        S is pleased
scenes :
```

Figure 15-1 Restaurant Script Example.

Script Name: Restaurant

- **Track:** Formal Dining
- **Props:** Tables, menu, check, money, food, place (abbreviated as F for food and P for place)
- **Roles:**
 - o S = Customer
 - o W = Waiter
 - o C = Cook
 - o M = Cashier
 - o O = Owner
- **Entry Conditions:** The customer (S) is hungry and has money.
- **Results:**
 - o S ends up with less money.
 - o S is no longer hungry and feels pleased.
 - o O (Owner) has more money.
- **Scenes:** The script includes various scenes that follow a logical sequence within the setting of a restaurant, such as entering, ordering, eating, and paying.

The concept of using symbols as variables allows for flexibility within the script. For instance, different kinds of food (F) and places (P) within the restaurant can be specified, adapting the script to various scenarios without losing the structural integrity of the overall script framework. This methodical approach not only organizes the expected sequence of events but also delineates the roles and outcomes associated with a typical restaurant visit, particularly in a formal dining context.

Scenes: Detailing Script Execution

The initial scene in the structured narrative of our “Restaurant Script” is aptly titled the **Entering Scene**. This scene is composed of a sequence of action frames that detail the customer's journey from the entrance to being seated. Here's how the scene unfolds:

1. **Movement to the Venue:** The first action frame captures the customer, denoted as “S”, moving to the venue. In this frame, the action is simply described as the customer moving himself or herself to a particular place, referred to here as “P.”
2. **Observing the Setting:** Upon entering, the customer encounters the restaurant's interior. The second frame illustrates that the customer, still referred to as “S”, notices a table. This observation triggers the decision to approach the table.
3. **Action Towards the Table:** The ensuing action involves the customer moving towards the selected table. This frame encapsulates the walking action of the customer from their current position to the table, marking a straightforward but essential part of the entering sequence.

Each frame represents a distinct phase in the customer's progression within the restaurant, starting from entering the establishment to making their way to a table. This structured breakdown of actions highlights the sequential and logical flow of events typical in a restaurant setting, providing a clear narrative framework for understanding customer behavior upon arrival.

Continuing with our illustration of the first scene within the “Entering Scene” of a restaurant script, we investigate deeper into the actions involved as the customer, denoted by “S”, transitions from identifying a table to settling down.

1. **Sitting Down:** The customer, “S”, maneuvers his body into a sitting position at the chosen table. This move is captured within the action frame as “S” performing the move-body-part action with the destination being sitting at the table.
2. **Waiter's Interaction:** Concurrently, the waiter, denoted as “W”, notices the customer. This observation triggers the waiter's movement towards the customer, which is another action frame where “W” performs the move-object action, with the object being himself, directed towards the customer “S”.
3. **Menu Handover:** As the waiter reaches the customer, he then proceeds to hand over the menu. This is represented in another action frame where the waiter, “W”, moves the object, which in this case is the menu, to the customer “S”.

4. **Completion of the Scene:** This sequence of actions completes the representation of the first scene upon entering the restaurant. The scene portrays the typical initial interactions between a customer and staff within the context of a restaurant, marking the beginning of the dining experience.

It is envisaged that subsequent scenes would follow this initial setup, such as the customer ordering food, receiving the food, and eventually, the process of payment and departure. This structured script provides a stereotypical narrative of a dining experience, though it is acknowledged that variations might exist based on cultural differences or specific restaurant protocols. The script serves as a fundamental structure to encapsulate what is generally known about dining out, reflecting common expectations and the sequence of events in a dining setting.

As we explore the application of scripts in a practical setting, we consider the transformation from a general, abstract script of a restaurant scenario to its specific instantiation. This process is analogous to object-oriented programming, where a general class is instantiated with specific attributes.¹⁴⁴

- **Instantiation of the Script:** In our example, we instantiate the general restaurant script by assigning roles to specific entities. Salwa becomes the customer, and Lucas assumes the role of the waiter. This instantiation is crucial for operationalizing the script in real-world settings.
- **Application to Intelligent Agents:** Consider Salwa, not merely a customer, but a robot designed to function autonomously in social environments like a restaurant. Programming Salwa involves embedding her with scripts that guide her actions in specific scenarios. When Salwa enters a restaurant, she activates the restaurant script, which prescribes every step from entry to exit based on predefined roles and actions.
- **Expectation Generation:** This scripted approach enables Salwa to anticipate events and react appropriately. By knowing what typically follows upon entering a restaurant or when approaching a waiter, Salwa can predict and prepare for subsequent interactions, enhancing her functionality and interaction quality.
- **Composition of Scripts:** The construction of Salwa's operational scripts utilizes primitive actions previously discussed, such as moving to a seat or ordering food. These primitives are the building blocks of more complex scripts, each action linked causally to form a coherent sequence that guides Salwa's behavior.
- **Integration of Knowledge Structures:** This example also illustrates how complex knowledge structures like scripts are built from simpler structures like frames. Frames define individual actions or events; scripts connect these into logical, functional sequences applicable to broader scenarios. This layered construction not only simplifies programming but also enriches the robot's interaction capabilities by providing a structured yet flexible approach to handling real-life situations.

Thus, the use of scripts encapsulates a sophisticated method of encoding behavior and expectations, crucial for intelligent agents designed to operate autonomously in human-centric environments.

EXPECTATION GENERATION AND VIOLATION

In our previous discussion, we touched upon the concept of expectation generation. Let's explore how this plays a crucial role in our interactions with the world. In daily life, things often don't unfold as anticipated, and sometimes, events take us completely by surprise. But what underpins our expectations, and how do we manage when they are not met?

Consider a real-life example involving a script – a structured sequence of expected actions based on common social settings. For instance, imagine I visit Olive Garden, and the assigned waiter is Andrew. According to the restaurant script, I find a table, seat myself, and Andrew notices me. At this point, my expectation is that Andrew will approach me and present a menu. However, what if Andrew never comes? As I sit waiting, the script I anticipate is disrupted – no one attends to me, leading to what we term an 'expectation violation'.^{145,146} This discrepancy signals that something in the expected sequence has deviated, alerting me to a potential issue in the service workflow.

Now, let's consider an alternative scenario where, instead of Andrew bringing a menu, he unexpectedly presents a bill upon approaching me. Such a scenario would naturally elicit surprise, deviating sharply from the established script of a dining experience. This deviation highlights a critical aspect of expectation generation: it not only sets the stage for normal interactions but also sharpens our awareness of irregularities.

These instances illustrate how scripts serve as frameworks within which we generate expectations about routine social interactions. When actual events align with these scripts, our experiences feel seamless and predictable. Conversely, deviations from these scripts can lead to confusion, surprise, or reassessment of the situation, demonstrating the pivotal role of expectation management in understanding and navigating the social world.

As we consider the implications of scripts, several intriguing questions emerge. For instance, could our experiences at the restaurant signify the inception of a theory of surprise? Surprises manifest in two primary ways: first, when events do not proceed as anticipated, and second, when entirely unexpected outcomes occur. In our example, the surprise came twice: once when Andrew failed to provide a menu and again when he unexpectedly delivered a bill instead. This dual aspect of surprise – both the absence of an expected action and the presence of an unanticipated one – illustrates the complex interplay between our expectations and reality.

This concept is vividly demonstrated in the context of cinema, particularly horror films, which excel in manipulating viewer expectations. A typical horror movie might set the scene for safety only to shatter this illusion with a sudden, startling event, such as a ghost appearing unexpectedly. This manipulation of expectations is not confined to horror; it extends across genres. Whether it's a twist in a science fiction narrative or an unexpected turn in a romance, the element of surprise can significantly enhance the viewing experience, making a film feel innovative and engaging.

Such surprises are not merely entertaining; they may also foster a broader understanding of creativity. In previous discussions, we considered how balance and humor contribute to creative

expressions. The same can be said for surprises, which are integral to creativity. According to some theories, creativity involves three core attributes: novelty, utility or value, and unexpectedness. The ability of scripts to generate and then violate expectations touches on this third attribute, illustrating how unexpectedness fuels creative storytelling.

As we continue to explore scripts and their applications, we will revisit these concepts, particularly how they intersect with computational creativity. By understanding the mechanics behind scripts and expectation management, we can better appreciate not only the narratives found in films but also the broader implications for AI and human interaction. This exploration into the unexpected and its role in creativity will be further developed as we examine computational creativity later in the book.

TRACKS: VARIATIONS WITHIN SCRIPTS

An integral component of scripts, which we haven't fully explored until now, is the concept of “tracks.” Tracks represent variations within a script that account for different scenarios or environments. For instance, in the context of a restaurant script, we distinguish between different dining experiences through tracks.

Consider the restaurant script, which is versatile enough to include several distinct tracks, each corresponding to different types of dining establishments: a coffeehouse, fast food, casual dining, and formal dining. While the core events – entering the restaurant, ordering food, eating, paying the bill, and leaving – are consistent across all tracks, the specific nature of these events can vary significantly from one track to another.

For example, the experience at a coffeehouse, with its casual come-and-go atmosphere, differs markedly from that of formal dining, where the service and dining process are more structured and prolonged. Fast food, with its quick-service and often self-service format, presents another unique set of interactions.

These tracks allow the script to adapt to various real-world situations, creating a semantic hierarchy of scripts. Within this hierarchy, each track tailors the general script to specific circumstances, reflecting the unique aspects of different dining experiences.^{147,148} This hierarchical structure not only enhances the script's flexibility but also its applicability to diverse social settings, making it possible to extend the script framework to broader contexts, such as various social events. This adaptability is crucial for understanding and navigating the multifaceted nature of human activities in different cultural and social environments.

SCRIPT SELECTION AND INVOCATION IN AI AGENTS

Understanding how an artificial intelligence (AI) agent utilizes scripts provides a window into its decision-making processes. Let's consider an AI agent equipped with basic needs and desires,

similar to a human. For instance, an AI that feels 'hungry' and knows it has money might decide it's time to address its hunger.

This decision-making is akin to solving a classification problem. The AI has a repository of scripts stored in its long-term memory, each representing a blueprint for handling specific situations based on past learning or pre-programmed knowledge. The task for the AI is to accurately identify which script fits the current context.

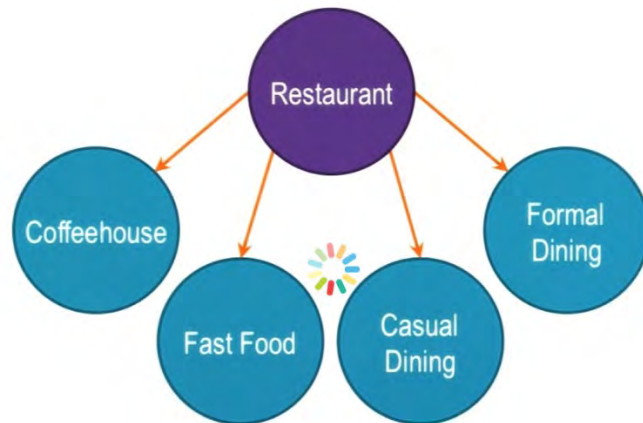


Figure 15-2 Restaurant Script Hierarchy.

Imagine the AI selects a generic restaurant script because it decides to eat out. Upon entering a specific restaurant, it assesses the environment. If the setting aligns more closely with a fast food venue – quick service, counter ordering, minimal table service – the AI would then switch to the fast food script from its broader restaurant script. This step-by-step refinement in script selection allows the AI to tailor its behavior to the specifics of the current scenario, effectively navigating through its semantic hierarchy of scripts.

In contrast to dynamic, real-time planning, scripts offer a more resource-efficient method. While an AI could theoretically generate a plan on the fly by considering its initial conditions and objectives, doing so requires significant computational power and time – resources that are often limited, especially in real-time interactions. Scripts circumvent this challenge by providing pre-compiled, ready-to-execute plans based on typical scenarios. This capability ensures that AIs can respond promptly and appropriately in dynamic environments without the need for extensive computation at every turn.

This approach highlights a critical aspect of modern AI systems: their ability to leverage structured knowledge representations (like scripts) to manage complex interactions within

constrained operational contexts. This method not only streamlines decision-making but also enhances the AI's ability to predict and react to changes in its environment effectively.

Learning and Using Scripts: Connections to KBAI Methods

We have extensively discussed how scripts are integral to AI agents, particularly in understanding and acting within their environment. Let's engage in an exercise to explore which of the previously discussed topics could aid an agent in learning a script, and which apply when an agent is using a script.

How an Agent Learns a Script

Ashok David, from the KBAI topics we've covered (Semantic Networks, Frames, Production Systems, Learning by Recording Cases, Incremental Concept Learning, Planning, Commonsense Reasoning), which do you believe would be most beneficial for an agent learning a script, and why?

David For learning a script, I identified five key areas from our discussions that would significantly help an agent. Firstly, semantic networks and frames are closely related and can be seen as representationally equivalent. We observed this when addressing the Raven's progressive matrices by transitioning from semantic networks to frames. Frames are particularly potent for storing essential information needed to build comprehensive scripts. If we extend this equivalence, scripts could also be structured using semantic networks.

Furthermore, incremental concept learning is crucial. Imagine an AI navigating daily events, constantly categorizing and re-categorizing experiences. This ability allows the AI to discern what is critical to a script's context – for instance, distinguishing between fast food and fine dining based on environmental cues rather than specific brands.

Planning is another critical component. It involves an AI determining a path from an initial state to a desired goal state. Once this pathway is established, it effectively becomes a script that can be reused in similar situations, reducing the need for constant recalibration.

Commonsense reasoning is the foundation that allows an AI to acquire scripts. It provides a language of primitive actions, enabling the AI to parse and understand new and complex scenarios.

However, production systems and learning by recording cases are less applicable for learning the structure of a script. These methods operate at a more granular level of knowledge representation, which doesn't align well with the abstract nature of scripts. Scripts require a higher level of generalization, whereas these systems focus on specific instances or basic logic rules.

Ashok Nicely explained, David.

Additionally, considering how we use semantic networks to interpret narratives can enrich our understanding. For example, in the story where “Ashok wanted to become rich, he got a gun,” the semantic network activates connections between nodes like “Ashok,” “wanted,” “rich,” and “gun.” This activation and the pathways formed provide a narrative structure, akin to a script. This example illustrates how semantic networks facilitate story understanding, serving as a backbone for script learning.

How an Agent Uses a Script

Ashok Let's consider a second example to identify which of our previously discussed topics are directly applicable to an agent actively using a script. Here are the topics for consideration: Generate & Test, Means-Ends Analysis, Problem Reduction, Case-Based Reasoning, Classification, Logic, Understanding. David, from these topics, which do you think apply to an agent's use of a script?

David I've identified four of the seven topics that are particularly relevant to using a script. Starting with Problem Reduction, this technique aligns closely with how scripts function. Scripts inherently break down complex scenarios into manageable actions or smaller scenes. This decomposition helps pinpoint where a script might falter during execution, allowing for immediate troubleshooting and understanding where expectations may not align with outcomes.

Classification is crucial as it helps an agent determine the appropriate script (or track within a script) to execute based on the observed environment. For example, recognizing the setting of a restaurant can trigger the specific dining script suited for that environment.

The concept of Logic also plays a significant role. We've previously discussed how scripts could be likened to pre-executed plans laid out in logical sequences. Rewriting these plans in formal logic could help formalize script actions and validate the coherence of the script's flow in various scenarios.

Understanding is indispensable as it enables the agent to differentiate and appropriately respond to similar events occurring under different circumstances. It assists in disambiguating events that might otherwise seem contradictory or out of place, ensuring the script adapts appropriately to real-world interactions.

Ashok Those are insightful selections, David.

It's worth considering that other methods like Generate and Test and Case-Based Reasoning could still play a role, albeit less directly. In scenarios where an agent must decide from a multitude of available scripts, these methods could facilitate testing and refining scripts in dynamic environments. Case-Based Reasoning, in particular, emphasizes the use of past experiences to handle new situations, which, while more instance-focused, can inform the selection and adaptation of scripts based on historical effectiveness.

This discussion underscores the diverse toolkit that AI agents can leverage in script utilization, each contributing uniquely to the robustness and adaptability of script-based reasoning in artificial intelligence.

SCRIPTS AND COGNITION

Scripts play a pivotal role in the domain of human cognition, echoing current psychological theories that view the human brain as a somewhat predictable entity.¹⁴⁹ This predictability is rooted in our cognitive processes, which typically begin with rapid bottom-up processing that assimilates basic sensory inputs. This is swiftly followed by top-down processing, where the brain applies broader understanding and expectations to interpret these inputs, leading to actions based on these formulated expectations.

The robustness of this cognitive framework becomes evident when expectations, as dictated by our internal scripts, are not met. Such deviations can provoke reactions ranging from amusement and surprise to outright frustration and anger, illustrating the strong influence of expectations on emotional responses. This brings forth a compelling inquiry: Do we inherently carry these scripts within us, or are they dynamically generated in response to our experiences?

Further integrating with the concept of mental models, scripts extend beyond simple reactions to encompass a broad spectrum of knowledge. These models, or scripts, are not confined to social interactions, such as dining out or movie-going. They also govern our understanding of complex systems like computer operations, economic theories, and mechanical functionalities, shaping our interactions with almost every aspect of the physical, social, and economic environment.^{150,151}

An intriguing aspect of scripts is their cultural specificity, which highlights the diversity of human cognition across different societies. For instance, in the United States, it is customary to leave a tip for service at a restaurant, a script deeply ingrained in the social fabric. However, this script varies globally; in many cultures, tipping may be seen as unnecessary or even offensive.

This variation suggests that scripts are not only shaped by immediate social interactions but are also the product of extensive cultural evolution and interaction.

The power of scripts lies in their ability to encapsulate complex chains of events into comprehensible and predictable patterns, thus serving as a profound source of knowledge that influences both individual behavior and collective cultural practices.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

Scripts, as structured representations of stereotypical event sequences, offer significant value when integrated with modern AI paradigms, enhancing their capabilities in understanding, prediction, and interaction.

- **Natural Language Processing (NLP) & Understanding (NLU):**
 - **Story Comprehension and Narrative Coherence:** Scripts provide a framework for understanding multi-sentence narratives by defining expected event sequences. This helps NLP systems infer missing information, resolve anaphora (pronoun references), and determine the coherence of a story. For example, knowing a “restaurant script” helps an AI understand that “ordering” typically precedes “eating.”
 - **Discourse Analysis:** Scripts can model typical conversational flows or event progressions in discourse, aiding in tasks like summarization or question answering where understanding the overall structure is key.
 - **Intent Recognition and Dialogue Management:** In dialogue systems, scripts can represent common user goals and the steps involved in achieving them (e.g., a “booking a flight” script). This helps the system predict user needs, manage the conversation flow, and handle deviations from the typical path.
- **Large Language Models (LLMs) & Generative AI (GenAI):**
 - **Structured Story Generation:** While LLMs can generate creative text, scripts can provide a high-level structure or plot outline to ensure generated stories are coherent and follow a plausible sequence of events. An LLM could be tasked with “fleshing out” the scenes within a given script.
 - **Controllable Narrative Generation:** Scripts can constrain LLM outputs to adhere to specific scenarios or genres, making their creative generation more targeted and predictable.
 - **Commonsense Inference for Generation:** LLMs often struggle with implicit commonsense knowledge embedded in typical event sequences. Scripts explicitly encode this, and this knowledge could be used to fine-tune or guide LLMs to produce more commonsense-aligned text. For example, an

LLM generating a story about a restaurant visit should adhere to the typical script unless a deviation is intentionally introduced for plot purposes.

- **Reinforcement Learning (RL) & Robotics:**
 - **Hierarchical Task Execution:** Scripts can define high-level tasks for robots or RL agents, which can then learn or execute policies for each scene or sub-task within the script. This is akin to Hierarchical Reinforcement Learning where an agent learns a policy over a set of options or sub-goals.
 - **Predictive Modeling in Dynamic Environments:** By understanding common scripts (e.g., traffic patterns, pedestrian behavior), an autonomous vehicle can better predict the actions of other agents and navigate more safely.
 - **Human-Robot Interaction:** Robots equipped with scripts for common social interactions (greetings, service encounters) can interact more naturally and effectively with humans.
- **Explainable AI (XAI):** If an AI agent's behavior is guided by a script, its actions become more interpretable. The script itself can serve as an explanation for why the agent performed a certain sequence of actions in a given situation.
- **Knowledge Graphs:** Scripts can be represented within knowledge graphs, where events are nodes and their causal or temporal relationships are edges. This allows for complex querying and reasoning about event sequences.

The explicit, structured nature of scripts provides a powerful way to encode stereotypical knowledge about event sequences, complementing the data-driven pattern recognition of many modern AI techniques. Integrating scripts can lead to AI systems that have a deeper understanding of context, can predict future events more reliably, and can interact in more human-like and coherent ways.

CHAPTER SUMMARY

In this chapter, we investigated the concept of scripts as a sophisticated knowledge representation framework that allows us to interpret and predict events within the natural world and human interactions. Scripts embody more than mere sequences of events; they are foundational elements in our understanding of stories, which are ubiquitous in our daily lives – not only in narratives but also in art, music, architecture, and more.

We began by defining scripts as causally coherent series of events that provide a structured expectation of how scenarios unfold. This coherence helps in crafting a predictable model for various situations, serving as a blueprint for behavior and expectation management. The exploration of scripts covered their structure, including their six key components: entry conditions, props, roles, results, tracks, and scenes, which collectively establish a comprehensive storyline for any given situation.

The discussion highlighted the hierarchical and dynamic nature of scripts, where they can be specialized into tracks (e.g., different types of restaurant experiences) to suit particular

contexts. This demonstrates the script's flexibility and applicability across a spectrum of scenarios.

Further, we explored how scripts facilitate expectation generation, a critical cognitive process where individuals anticipate outcomes based on established scripts. This process becomes particularly salient when expectations are not met (expectation violation), leading to reactions such as surprise, amusement, or frustration, which underscore the deep integration of scripts into cognitive and emotional responses. The connection between expectation violation, surprise, and creativity was also noted.

The chapter also touched upon the cultural specificity of scripts, noting that what is considered normative behavior in one culture can be perceived differently in another, thus emphasizing the adaptability and cultural embeddedness of scripts. We also discussed how AI agents might learn and utilize scripts, connecting this to KBAI methods like frames, planning, commonsense reasoning, classification, and understanding.

Finally, we considered the integration of scripts with modern AI paradigms. Scripts can enhance NLP/NLU by providing structure for story comprehension, guide LLMs in generating coherent narratives, offer hierarchical task definitions for RL and robotics, contribute to explainable AI by making behavior transparent, and be represented within knowledge graphs.

By dissecting scripts, we gain insights into the underlying mechanisms of commonsense reasoning, which prepares us for future discussions on enhancing, differentiating, and refining these structured knowledge representations to better capture the nuances of human experiences and expectations.

ADDITIONAL READINGS & REFERENCES

- 138 Campbell, J., Ansell, K. & Stelzer, T. Evaluating IBM's Watson natural language processing artificial intelligence as a short-answer categorization tool for physics education research. *Physical Review Physics Education Research* **20**, 010116 (2024).
- 139 Natale, S. in *NATALE: A CRITICAL ANALYSIS OF AI VOICE ASSISTANTS* 1-17 (University of Bremen, 2020).
- 140 Jones, J. L. Robots at the tipping point: the road to iRobot Roomba. *IEEE Robotics & Automation Magazine* **13**, 76-78 (2006).
- 141 Modi, A. Modeling common sense knowledge via scripts. (2017).
- 142 Herman, D. Scripts, sequences, and stories: Elements of a postclassical narratology. *pmla* **112**, 1046-1059 (1997).
- 143 Abbott, V., Black, J. B. & Smith, E. E. The representation of scripts in memory. *Journal of memory and language* **24**, 179-199 (1985).
- 144 Patel, A. & Jain, S. Formalisms of representing knowledge. *Procedia Computer Science* **125**, 542-549 (2018).
- 145 Lew, Z. & Walther, J. B. Social scripts and expectancy violations: Evaluating communication with human or AI chatbot interactants. *Media Psychology* **26**, 1-16 (2023).

- 146 Bovolenta, G. & Marsden, E. Expectation violation enhances the development of new
abstract syntactic representations: Evidence from an artificial language learning study.
147 *Language Development Research* **1**, 193-243 (2021).
- 148 Weber, N., Shekhar, L., Balasubramanian, N. & Chambers, N. Hierarchical quantized
representations for script generation. *arXiv preprint arXiv:1808.09542* (2018).
- 149 Fatimah, D. in *IOP Conference Series: Materials Science and Engineering*. 012053 (IOP
Publishing).
- 150 Szilas, N. in *6th Workshop on Computational Models of Narrative (CMN 2015)*. 133-141
(Schloss Dagstuhl–Leibniz-Zentrum für Informatik).
- 151 Jabbari, M., Recker, J., Green, P. & Werder, K. How do individuals understand multiple
conceptual modeling scripts? *Journal of the Association for Information Systems* **23**, 1037-1070
(2022).
- Tikhonov, R. & DeDeo, S. Prediction, explanation, and control: The use of mental
models in dynamic environments. *Open Mind* **7**, 894-916 (2023).

Chapter 16

Explanation-Based Learning

INTRODUCTION TO EXPLANATION-BASED LEARNING

This chapter examines the significant area of Explanation-Based Learning (EBL), an approach where learning is centered not on acquiring entirely new concepts, but on deepening the understanding of how existing concepts interconnect and apply to new situations. This method contrasts with other forms of learning by emphasizing the integration and application of known knowledge rather than the accumulation of new information. EBL equips an agent to learn from a single example by explaining why the example is an instance of a concept, and then generalizing this explanation.

Explanation-Based Learning is particularly pertinent as we prepare to investigate more complex cognitive mechanisms, such as analogical reasoning, in subsequent chapters. By understanding EBL, agents learn to transfer knowledge from familiar contexts to novel scenarios, thereby enhancing their problem-solving capabilities without requiring extensive additional data.

Our exploration of this chapter commences with the idea of a “Concept Space” – a theoretical framework where known concepts and their interrelationships are mapped.¹⁵² This space serves as the foundation for understanding how different ideas link and support one another, providing a scaffold from which more sophisticated reasoning can emerge.

We then investigate the role of Abstraction in EBL, a critical process through which agents generalize from specific instances.¹⁵³⁻¹⁵⁵ Abstraction allows for the distillation of broad principles

from particular examples, facilitating the transfer of these principles to new, seemingly unrelated problems. This capability is essential for AI systems tasked with navigating the complex, variable-rich real world.

Through the lens of EBL, we will also engage with practical exercises and scenarios that illustrate how AI agents can utilize their existing knowledge base to forge solutions in unfamiliar territories. These exercises will highlight the intuitive ease with which humans often solve problems and contrast it with the challenges faced by AI, underscoring the need for robust, creative problem-solving frameworks within artificial intelligence systems.

By understanding and implementing EBL, we aim to equip AI agents with the ability not only to recall information but to reason profoundly about it and apply it creatively. This chapter sets the stage for a deeper examination of how knowledge is transferred and transformed, paving the way for future discussions on analogical reasoning and beyond.

UNDERSTANDING EBL THROUGH A PRACTICAL SCENARIO

To better understand explanation-based learning, let's consider a practical exercise. Imagine needing to transport soup from the kitchen to the dining table. However, a dilemma arises: all the usual utensils for such a task – like ladles and soup tureens – are either dirty or unavailable. Surveying the available alternatives, you identify four potential objects: a backpack, a pitcher, a box, and a car. The question is: which of these objects would you choose to complete the task of transporting soup?



Figure 16-1 Objects under considerations for carrying soup.

Ashok Which one would you use, David?

David To me, the answer seems obvious. The only thing among all these objects that could feasibly transport soup would be the pitcher.

Ashok I think most of us would give the same answer. The pitcher, not the backpack or the car or this box.

This task, seemingly simple for humans, presents a significant challenge for a machine or a robot. How does a robot determine that a pitcher is an appropriate utensil for transporting soup from the kitchen to the dining table, while a backpack or a box is not? This highlights the complexity that robots face in making decisions that humans find instinctively easy. The core of EBL lies in using existing knowledge to explain *why* the pitcher is suitable (e.g., it can hold liquids, it's portable, it won't spill easily) and then generalizing this explanation to understand the properties of a “soup transporter.”

THE NATURE OF INCREMENTAL LEARNING IN EBL

The exercise of selecting the most suitable object for transporting soup, while straightforward for humans, poses a complex challenge for robots. This raises a critical question: why is this task so easy for humans yet difficult for robots, and how can we design AI agents to handle such decisions with similar ease?

One key aspect is the concept of incremental learning as it applies within EBL. While EBL can learn from a single example, the broader knowledge base it relies upon is often built incrementally. This contrasts with some traditional machine learning methods that require large datasets to discern patterns.¹⁵⁶ For instance, in previous discussions on incremental concept learning, learners might focus on a single positive example, such as an “arch,” to build an understanding of that concept. EBL takes such a specific example and, using its domain knowledge, explains why it is an instance of the concept (e.g., “arch”). This explanation is then generalized to create a new rule or schema that can recognize other instances of the concept, effectively refining the agent's understanding from one or few examples.

Furthermore, explanation-based learning intersects intriguingly with creativity. Faced with the absence of typical utensils for soup transportation, an AI agent using EBL must devise a novel solution by applying its existing knowledge. Choosing a pitcher involves recognizing that the pitcher shares critical functional properties with known soup-carrying utensils, even if it's not a perfect match. This scenario exemplifies how AI can engage in creative problem-solving within its learning framework, highlighting the potential for AI to adapt creatively to new situations by leveraging learned concepts and their underlying explanations. This synthesis of learning from specific instances and creative application is vital for advancing AI's capability to function autonomously in varied and unforeseen environments.^{157,158}

Let's extend our examination of AI capabilities through another household example. Imagine purchasing a household robot designed to assist with everyday tasks. Its typical routine involves entering the kitchen, brewing coffee, and serving it in a cup. However, the situation complicates following a large party at your home, resulting in a shortage of clean cups.

Confronted with this dilemma, the robot does not shut down or fail. Instead, it engages its EBL process to identify a viable alternative. Scanning the kitchen, it spots an object that deviates from the usual coffee cups but shares several functional and physical traits with them. The object is light, made of porcelain, features a decorative design, possesses a concave shape, and includes a handle with a flat bottom – attributes that resemble those of a typical cup.

Faced with the task of determining the suitability of this unusual object for drinking, the robot seeks to establish whether this object can serve as a stable vessel for liquids – essentially, whether it can function as a cup. The robot must analyze the object's features and, through a logical deduction based on previously learned concepts and rules about what constitutes a “cup” (its operational criteria), conclude whether the object can indeed fulfill the role of a cup. This is achieved by constructing an explanation: “This object can be a cup *because* it is stable (due to its

flat bottom) and can hold liquid (due to its concavity) and is liftable (due to its handle and light weight).” This explanation is then generalized.

This instance illustrates how robots can utilize explanation-based learning to adapt to new situations by reapplying known concepts to novel scenarios, thereby solving problems creatively and effectively without human intervention.

The Explanation-Building Process

To understand the process of explanation-based learning more deeply, let's consider how an AI agent might tackle the task of identifying whether a particular object qualifies as a cup based on its learned characteristics and prior knowledge. This examination tests not only the robot's cognitive functions but also its ability to utilize its stored information effectively.

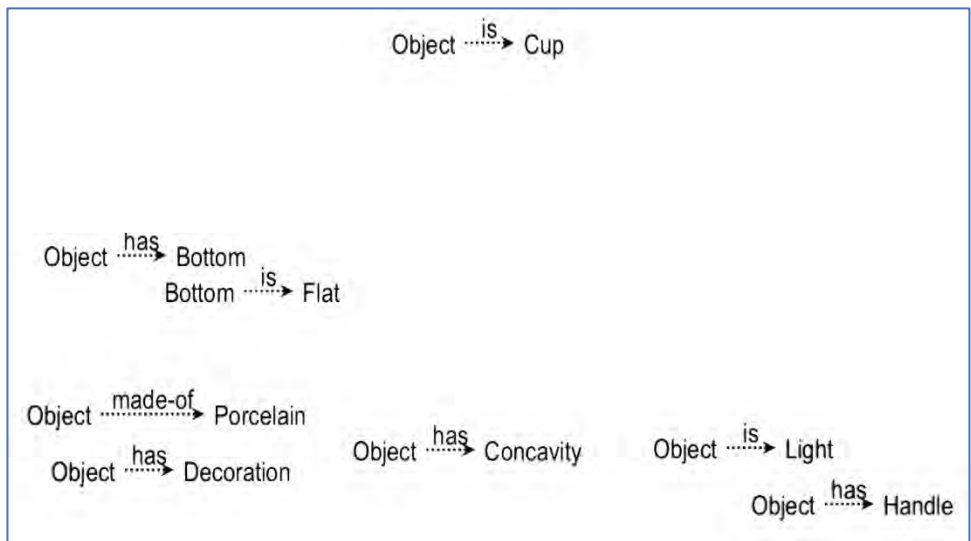


Figure 16-2 Definition of a cup as an explainable concept.

The challenge for the robot is to confirm that an object, which at first glance appears suitable for drinking, conforms to the defined attributes of a 'cup.' The robot's initial knowledge indicates that the object possesses a flat bottom, is made from porcelain, has a concave shape, is adorned with a decoration, is lightweight, and features a handle. These characteristics align closely with the traditional components of a cup.

Given these attributes, the robot needs to build a logical explanation that connects these observed features to the established concept of a cup. This process involves the robot analyzing each trait and understanding how these contribute to the function and stability of a cup. For

instance, the flat bottom offers stability when placed on a surface, while the concavity is crucial for containing liquids.

To effectively construct this explanation, the AI agent must rely on its prior knowledge, which includes understanding materials, object functionality, and their typical uses. The robot must engage in a step-by-step reasoning process, where it evaluates if the combination of these attributes can functionally support the roles of a cup. This scenario illustrates how the robot applies its understanding of physical properties and their implications to decipher the practical usability of the object in question, leveraging its conceptual knowledge to bridge the gap between theory and practical application. The output of EBL is a generalized rule or schema that captures the essential features of a cup, derived from the explanation of this specific instance.

THE ROLE OF PRIOR KNOWLEDGE IN EBL

In explanation-based learning, an AI agent's ability to draw from a well-defined reservoir of prior knowledge is crucial. Let us consider how an AI agent leverages this knowledge across various objects it may encounter. For this discussion, we will focus on four fundamental concepts: a brick, a glass, a bowl, and a briefcase. This prior knowledge consists of facts and rules about the world.

Prior Knowledge

A Brick

The brick is stable because its bottom is flat. A brick is heavy.

A Glass

The glass enables drinking because it carries liquids and is liftable. It is pretty.

A Briefcase

The briefcase is liftable because it has a handle and is light. It is useful because it is a portable container for papers.

A Bowl

The bowl carries liquids because it has a concavity. The bowl contains cherry soup.

Figure 16-3 Prior Knowledge for four objects.

- **The Brick:** The brick serves as a prime example of how structural characteristics lead to functional outcomes. The agent knows that the brick's stability is a result of its flat bottom. This causal relationship – *is_stable(X) IF has_flat_bottom(X)* – is a critical

piece of knowledge. The brick's weight further contributes to its stability, showcasing how multiple attributes interact to define an object's utility.

- **Causality and Conceptualization:** Each object the AI encounters is understood not just in isolation but as a collection of properties that lead to certain capabilities. For the brick, the causality is clear: its flat bottom directly contributes to its stability. This kind of reasoning is essential for the AI to navigate and interact with the physical world effectively. By understanding and applying these causal relationships, the AI can infer how other similar objects may behave in various situations, enhancing its problem-solving abilities.¹⁵⁹

By examining these concepts in detail, the AI builds a robust framework of knowledge that not only aids in identifying objects but also in predicting their functionality based on their characteristics. This foundational knowledge is indispensable as the AI encounters novel situations and needs to apply learned concepts in creative ways.

Representing Knowledge: The Brick

As we examine how an AI agent conceptualizes and utilizes knowledge about everyday objects, consider the example of a brick – a simple yet illustrative case. The AI agent's representation of a brick is grounded in both observable properties and inferred causality, crucial for explanation-based learning.

A Brick

The brick is stable because its bottom is flat. A brick is heavy.

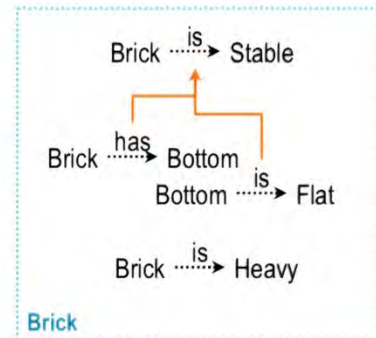


Figure 16-4 Prior knowledge of a brick textually (left) and as an illustrative representation (right).

Observable Properties of the Brick

- **Weight:** The AI agent recognizes that the brick is heavy. This characteristic is crucial for various functional roles that the brick might play, especially in scenarios requiring stability and support.
- **Structure:** The brick has a bottom, and this bottom is flat. The flatness of the bottom is not merely a physical attribute but ties directly into the brick's functionality.

Causality in the AI's Conceptualization

- **Stability:** The key causal relationship here is between the brick's flat bottom and its stability. This isn't just an observed fact; it's a functional reality. The AI understands that the flat bottom makes the brick stable (*is_stable(X) IF has_flat_bottom(X)*), allowing it to be used effectively in building or as a supportive object. It might also know that being heavy contributes to stability (*is_stable(X) IF is_heavy(X)*).

The AI uses these facts and rules to build a causal framework, represented visually with arrows. These arrows do not just represent the flow of information but the logical connections that define causality: the brick is stable *because* its bottom is flat and it is heavy. This causal linkage allows the AI to apply this knowledge to solve problems or make predictions about the brick's behavior in different scenarios.

Through this structured representation, the AI encapsulates both the structural and functional aspects of the brick, enabling it to reason about and utilize this object effectively in a variety of contexts.

Representing Knowledge: The Briefcase

Continuing with our examples, we consider the briefcase. The AI agent models the briefcase through its observable properties and their causal relations, highlighting how it can leverage this understanding in practical scenarios.

A Briefcase

The briefcase is liftable because it has a handle and is light. It is useful because it is a portable container for papers.

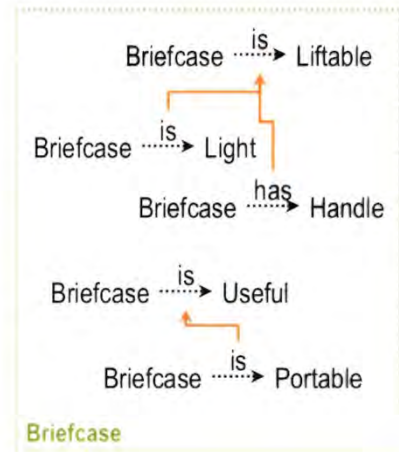


Figure 16-5 Prior knowledge of a briefcase textually (left) and as an illustrative representation (right).

Observable Properties of the Briefcase

- **Handle and Weight:** The briefcase is recognized by the AI as having a handle and being light. These are foundational to the briefcase's functionality.
- **Capacity:** It is known to contain papers, defining its use as a container.

Causality in the AI's Conceptualization

- **Liftability:** The causal connection here is straightforward: the briefcase is easy to lift because it has a handle and is lightweight (*is_liftable(X) IF has_handle(X) AND is_light(X)*). This makes it a practical choice for transporting documents.
- **Utility:** The utility of the briefcase stems from its role as a portable container for papers.

The AI uses causal mapping to link these structural features with functional outcomes.^{160,161} This integration of structure and function into the agent's knowledge base enables it to understand why a briefcase is used in certain ways and can predict or infer its use in new situations.

Representing Knowledge: The Bowl

The third example is the conceptual characterization of a bowl, further illustrating how AI agents use structural features to infer functionality.

Observable Properties of the Bowl

- **Concavity:** The bowl is notably concave, a key structural feature responsible for its ability to hold contents.
- **Contents:** In this example, the bowl contains cherry soup, providing a context-specific application.

Causality in the AI's Conceptualization

- **Functionality to Carry Liquids:** The causal connection is derived from the bowl's concavity, which facilitates the containment of liquids (*can_carry_liquid(X) IF is_concave(X)*).

The visual representation illustrates how the AI agent connects the observable feature of concavity directly to the bowl's functionality to carry liquids. This understanding is pivotal for applying knowledge to new scenarios.

Representing Knowledge: The Glass

The fourth example is the concept of a glass, illustrating how AI agents utilize observable features to infer functionality.

Observable Properties of the Glass

- **Carries Liquids:** A primary function, evident through its design.
- **Liftable:** Typically light enough to be easily lifted.
- **Aesthetic Appeal:** Often possesses an aesthetic element (e.g., 'pretty'), which can influence human preferences.

Causality in the AI's Conceptualization

- **Enables Drinking:** This core functionality is directly tied to the glass's ability to carry liquids and its liftable nature (*enables_drinking(X) IF can_carry_liquid(X) AND is_liftable(X)*).

This detailed examination allows AI agents not only to recognize objects but also to anticipate their utility based on specific features, a step towards sophisticated interaction with the physical world.

ABSTRACTION: GENERALIZING FROM SPECIFICS

A pivotal concept in explanation-based learning is the AI agent's ability to abstract knowledge from specific instances to generalize across situations. This process is vividly illustrated using the example of a bowl, which we now extend to a generic object to establish a foundational understanding. The core of EBL involves creating an explanation for why a training example is an instance of a concept and then generalizing this explanation.

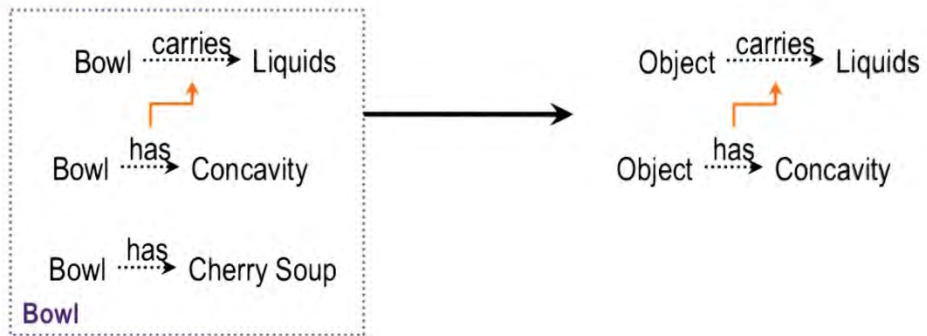


Figure 16-6 Abstraction of a bowl (left) into a generalized object (right) retaining the causal relationship).

Abstraction Process Explained

1. **Explanation Construction:** The AI agent begins by constructing an explanation for why the specific bowl can carry liquids. Using its prior knowledge (e.g., *can_carry_liquid(X) IF is_concave(X)* and the fact *is_concave(bowl_example)*), it deduces *can_carry_liquid(bowl_example)*. This is the explanation.
2. **Generalization of the Explanation:** The AI then generalizes this explanation. It identifies the features in the explanation that are specific to the training example (e.g., *bowl_example*) and replaces them with variables, while keeping the structure of the explanation intact. The goal is to find the most general set of conditions under which the explanation holds. In this case, the rule *can_carry_liquid(X) IF is_concave(X)* is already quite general. If the explanation involved multiple steps, the generalization would prune away unnecessary details, retaining only the causally relevant aspects. For

instance, the fact that the bowl contained “cherry soup” is irrelevant to its ability to carry liquids in general; its concavity is what matters.

3. **Formation of a New Rule/Schema:** The generalized explanation becomes a new, learned rule or schema. For example, the AI learns that *any* object X can carry liquids if X is concave. This abstracted causal model (*can_carry_liquid(X) IF is_concave(X)*) can then be applied to other objects.

This abstraction mechanism allows AI systems to operate with a higher level of efficiency and adaptability. By focusing only on causally significant features identified in the explanation, the AI reduces the complexity of its decision-making processes, enabling it to adapt learned behaviors to new but related scenarios. This ability to generalize from specific instances to broader contexts is foundational in teaching AI systems to navigate our unpredictable world more intelligently.¹⁶²

KNOWLEDGE TRANSFER AND CAUSAL EXPLANATIONS

In explanation-based learning, AI agents build explanations that connect abstract concepts to specific instances. This is crucial for teaching AI agents to recognize and categorize objects based on learned properties and their causal relationships. This process often involves transferring knowledge learned from one concept to help understand another.

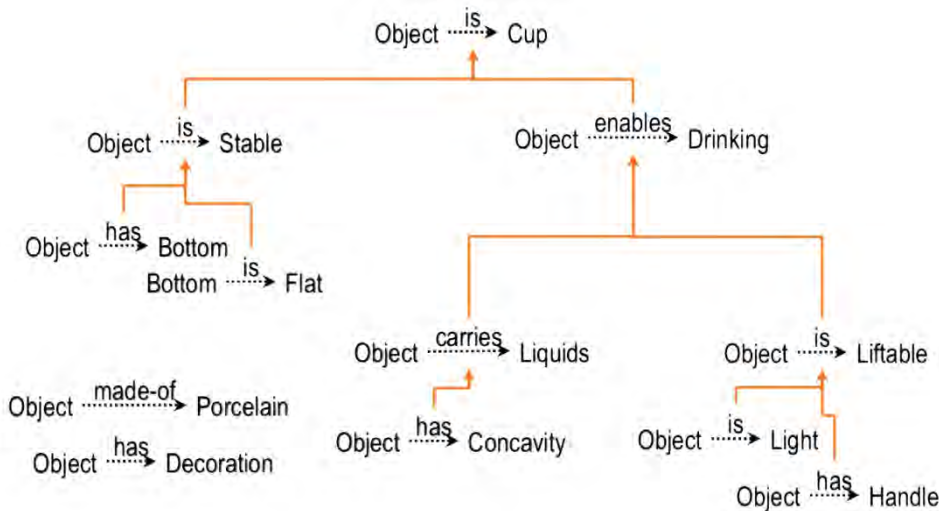


Figure 16-7 Cup representation build using transference of prior knowledge.

Constructing Causal Explanations for a “Cup”

Suppose the AI is given an example of a new object and told it is a “cup.” The goal of EBL is to learn a general definition of “cup.”

1. **Operational Criteria for “Cup”:** The AI needs a definition of what makes something a cup. Let's say the operational criteria are: *is_cup(X) IF is_stable(X) AND enables_drinking(X)*.
2. **Explaining the Example:** The AI must explain why the given example object satisfies these criteria using its background knowledge.
3. **Explaining *enables_drinking(example_cup)*:**
 - It might use the rule: *enables_drinking(Y) IF can_carry_liquid(Y) AND is_liftable(Y)*.
 - To prove *can_carry_liquid(example_cup)*, it uses: *can_carry_liquid(Z) IF is_concave(Z)*. It observes *is_concave(example_cup)*.
 - To prove *is_liftable(example_cup)*, it uses: *is_liftable(W) IF has_handle(W) AND is_light(W)*. It observes *has_handle(example_cup)* and *is_light(example_cup)*.
4. **Explaining *is_stable(example_cup)*:**
 - It might use the rule: *is_stable(V) IF has_flat_bottom(V)*. It observes *has_flat_bottom(example_cup)*.
5. **Generalizing the Explanation:** The AI traces the proof structure, collecting the “leaf” nodes of the explanation — the observed features of the *example_cup* that were necessary for the proof. These are: *is_concave(example_cup)*, *has_handle(example_cup)*, *is_light(example_cup)*, and *has_flat_bottom(example_cup)*.
6. **Forming a New Rule:** By replacing *example_cup* with a variable *X*, the AI forms a new, generalized rule for “cup”:
 - *is_cup(X) IF is_concave(X) AND has_handle(X) AND is_light(X) AND has_flat_bottom(X)*.

This method demonstrates how AI systems use EBL to analyze a specific instance, explain why it's an example of a concept using prior knowledge, and then generalize that explanation to form a new, more operational rule for recognizing future instances of the concept. This process effectively transfers and refines knowledge, illustrating a significant advancement in AI's ability to learn efficiently.

The Process of Reverse Engineering Explanations

In practice, when AI agents use explanation-based learning to classify a new object or understand a situation, they effectively work to construct an explanation. If the goal is to determine if an object *Obj* is a Cup, the system attempts to prove *is_cup(Obj)* using its

background knowledge and the operational definition of a cup. This can be seen as a form of “reverse engineering” an explanation for why *Obj* would be a cup.¹⁶³

Reverse Engineering the Explanation for an Object being a Cup

1. Goal-Driven Reasoning:

- **Goal:** *is_cup(Obj)?* The AI starts with the target concept.
- **Subgoal 1:** *is_stable(Obj)?* Based on the definition *is_cup(X) IF is_stable(X) AND enables_drinking(X)*, the AI needs to prove *Obj* is stable. It might look for rules like *is_stable(Y) IF has_flat_bottom(Y)*. If *Obj* has a flat bottom, this subgoal is satisfied. This involves retrieving knowledge about stability, perhaps from its understanding of a “brick.”
- **Subgoal 2:** *enables_drinking(Obj)?* The AI then tackles the second part of the definition. It might use *enables_drinking(Z) IF can_carry_liquid(Z) AND is_liftable(Z)*.
 - **Sub-subgoal 2a:** *can_carry_liquid(Obj)?* This could be proven if *Obj* is concave, drawing on knowledge perhaps abstracted from a “bowl.”
 - **Sub-subgoal 2b:** *is_liftable(Obj)?* This could be proven if *Obj* has a handle and is light, drawing on knowledge perhaps abstracted from a “briefcase” or “glass.”

2. Integration and Application of Knowledge:

The agent synthesizes information from different sources or generalized rules (e.g., stability from brick-like properties, liquid-carrying from bowl-like properties, liftability from briefcase/glass-like properties) to construct a comprehensive explanation for why the object in question could be a cup.

3. Problem Reduction and Planning Analogy:

- **Problem Reduction:** This process is analogous to problem reduction in AI, breaking down the complex task of identifying a cup into simpler, more manageable sub-problems (proving stability, proving it enables drinking, etc.). Each sub-problem becomes a subgoal.
- **Planning with Open Preconditions:** Similar to planning, where actions have preconditions that must be met, the rules used in EBL have conditions (the IF parts). The AI searches for a sequence of rule applications that satisfies the main goal, effectively finding a “plan” to prove the object is a cup. The observed features of the object serve to satisfy the most basic preconditions.

Through this method, AI agents utilize their stored knowledge efficiently, aligning it with goal-oriented tasks. This backward reasoning from the goal, combined with the decomposition of the problem, exemplifies a sophisticated level of AI reasoning. By setting subgoals and retrieving relevant rules and facts from memory, the AI constructs a logical and causally coherent explanation, bridging the gap between theoretical knowledge and practical application.

ADVANCING WITH EXPLANATION-BASED LEARNING

As we conclude this examination of explanation-based learning, it is essential to reflect not only on its current applications but also on its potential to profoundly shape our future interactions with technology. Throughout this chapter, we have seen examples demonstrating the versatility and effectiveness of this approach in problem-solving.

Future Technologies and Practical Applications

Consider the potential of robotic assistants in our homes. While robots capable of identifying and retrieving items like cups might still seem advanced, EBL provides a mechanism for such capabilities. These robots, equipped with EBL, could navigate dynamic environments, making intelligent decisions based on abstracted knowledge derived from explanations of single examples, allowing them to interact and assist effectively.

Similarly, envision an intelligent word processor. This processor, rather than merely searching by filenames, could understand the context and relevance of user queries. For instance, if you were to ask for “that important document about the Q2 budget I worked on last Tuesday,” the word processor could utilize EBL principles. It might have learned what makes a document “important” to *you* (e.g., by explaining a few examples you designated as important, noting features like specific keywords, recipients, or document structure) and combine this with date information to retrieve the correct document.

Explanation-based learning is not just theoretical – it mirrors aspects of everyday human problem-solving. Take, for example, the simple act of keeping papers from blowing away. An everyday object like a cup, which is heavy and has a flat bottom, can be repurposed as a paperweight. This practical application demonstrates how we routinely apply reasoning similar to EBL: we understand *why* a cup works (it's stable, it has some weight) and generalize this to other objects.

In another scenario, when a door needs to be kept open and a traditional door stopper isn't available, creativity comes into play. Items like an eraser or a chair can be repurposed. Their stability and size make them effective makeshift door stoppers. These everyday improvisations are vivid examples of how EBL-like reasoning enables us to adapt and reuse objects based on their properties and our immediate needs.

By abstracting and applying knowledge from explanations of known situations to novel ones, both AI agents and humans exhibit a remarkable capacity for adaptation and innovation. As this learning paradigm continues to evolve, its integration into increasingly sophisticated technologies promises to enhance our interactions with both digital and physical environments.

PRACTICAL EXERCISES IN EBL

Let's consider an exercise that reinforces our understanding and challenges the AI's capabilities in applying learned concepts. We'll attempt to determine whether an object can be identified as a mug, expanding on the criteria typically used to identify a cup.

Exercise 1: Identifying a Mug

We are presented with an object in the kitchen. The object is light, crafted from clay, features a concave design with a handle, a flat bottom, and notably thick sides. Given our prior discussions and the AI's knowledge base (which includes properties of a glass, bowl, brick, briefcase, and perhaps a pot), we want to determine if this object meets the criteria of a mug.

A Mug

A mug is an object that is stable, enables drinking, and protects against heat.

A Pot

The pot carries liquids because it has a concavity. It limits heat transfer because it has thick sides and is made of clay.

An Object

This object is light and made of clay. It has a concavity and a handle. The bottom is flat, and the sides are thick.

Can we prove this object is a mug?
o Yes o No

Figure 16-8 Determining if an object is a mug.

A mug, for this exercise, is defined by the operational criteria: *is_mug(X) IF is_stable(X) AND enables_drinking(X) AND protects_against_heat(X)*. This definition adds complexity by introducing thermal insulation as a criterion.

Ashok What do you think, David? Is this provable with our current knowledge?

David Reflecting on the challenge, it's clear that we can likely substantiate the object's stability (due to its flat bottom) and its ability to facilitate drinking (concave, has

handle, is light) – attributes similar to those of a cup. The crucial test lies in its heat protection capability: *protects_against_heat(Obj)*. Our object has thick sides and is made of clay. Let's say our knowledge base contains *limits_heat_transfer(Y) IF has_thick_sides(Y) AND is_made_of_clay(Y)*. So, we can prove *limits_heat_transfer(Obj)*. However, if our knowledge base does *not* contain a rule directly linking *limits_heat_transfer(Z)* to *protects_against_heat(Z)*, we have a missing link. Our current understanding and definitions do not equate limiting heat transfer directly with protecting against heat without such an explicit rule. This highlights a gap that prevents us from conclusively proving the object is a mug.

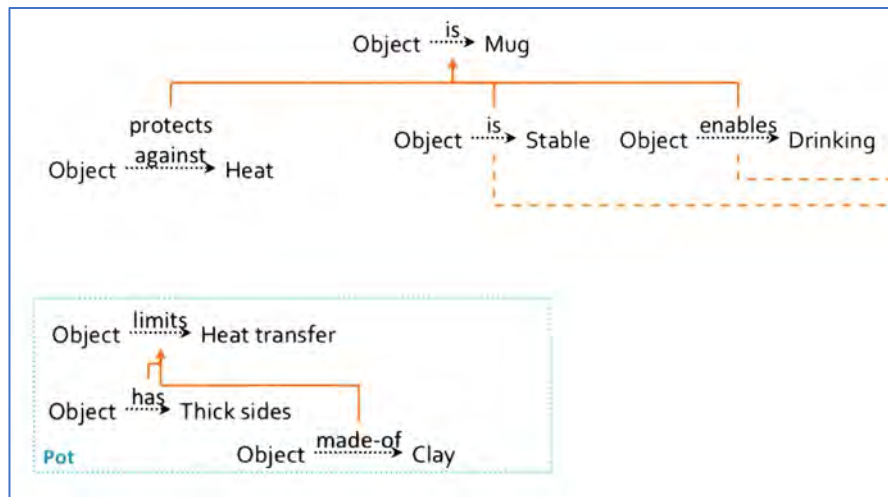


Figure 16-9 Illustration that we cannot connect limits heat transfer to protects against heat.

Ashok That's an insightful analysis, David. It appears that while we can establish certain properties of the object, the specific attribute of heat protection, as required for a mug, remains unproven with the available evidence.

This exercise illuminates the intricacies of explanation-based learning, particularly how AI systems depend on the completeness of their background knowledge to construct explanations. David's process involved deconstructing the problem and seeking evidence. The absence of a direct rule linking the known properties to one of the operational criteria for “mug” underscores the challenges AI faces.

Exercise 2: Incorporating Additional Knowledge

Building on our prior exercise, we now explore how additional knowledge can complete our understanding. This next exercise demonstrates the strategic selection of knowledge to finalize our proof that the object is a mug.

We are again in the kitchen, this time considering which of four different concepts (or rather, knowledge about them) could help us conclude that our earlier object qualifies as a mug. The key is to find knowledge that bridges the gap for *protects_against_heat(X)*.

Suppose we are considering:

1. An Oven Mitt: We know *protects_against_heat(oven_mitt)*. If we also know *why* it protects (e.g., *protects_against_heat(A) IF limits_heat_transfer(A)* and we know *limits_heat_transfer(oven_mitt)* because it's made of insulating material), then adding the general rule *protects_against_heat(A) IF limits_heat_transfer(A)* to our AI's knowledge base would be helpful.
2. A Wooden Spoon: Perhaps less direct. If we knew *protects_against_heat(wooden_spoon_handle)* because wood is a poor conductor (limits heat transfer), and we had the rule *protects_against_heat(A) IF limits_heat_transfer(A)*, this could also help.
3. A Metal Rod (recently in a fire): Known *not* to protect against heat. This is a negative example and might help refine concepts but not directly prove our mug.
4. A Thermos Flask: Known to protect against heat (keeps things hot/cold by limiting heat transfer). This is very similar to the oven mitt example.

What knowledge would finish our proof that this object is a mug?

o <u>A Serving Plate</u> The serving plate protects against heat because it is made of clay and is shaped like an oval.	o <u>A Wooden Spoon</u> The spoon is long and made of wood. It protects against heat because it has a handle.
o <u>An Oven Mitt</u> The oven mitt is made of cloth. It protects against heat because it limits heat transfer.	o <u>A Baking Dish</u> A baking dish protects against heat because it is made of porcelain and has a handle.

Figure 16-10 Additional objects from which Prior Knowledge is obtained.

Ashok David, which of these concepts, or knowledge derived from them, do you think could help us complete the proof?

David Ashok, if we can acquire the general rule *protects_against_heat(X) IF limits_heat_transfer(X)*, then knowledge about the oven mitt or the thermos flask would be most helpful. Since we already established that our clay object with thick sides *limits_heat_transfer(Obj)*, adding this new rule would allow us to prove *protects_against_heat(Obj)*. For instance, observing that an oven mitt protects against heat *because* it limits heat transfer, and then generalizing this to the rule *protects_against_heat(X) IF limits_heat_transfer(X)*, would bridge the gap.

Ashok Indeed, that makes sense. It's fascinating to see how a single piece of generalized knowledge can complete the reasoning chain.

This exercise illuminates a critical aspect of knowledge-based AI: the importance of having the right abstract rules in the knowledge base.¹⁶⁴ EBL is not just about the specific example but also about the richness and applicability of the background theory. An interesting takeaway is how the AI's ability to form an explanation depends on these available general rules. If the agent learns the rule *protects_against_heat(X) IF limits_heat_transfer(X)* (perhaps through EBL itself from an example of an oven mitt), it can then complete the mug proof.

THE POWER OF EXPLORATION-BASED LEARNING

In our day-to-day lives, humans continually adapt to and manage new challenges by understanding *why* things work the way they do and generalizing that understanding. This process involves utilizing everyday objects in unconventional ways to solve problems by recognizing that the underlying principles of their function can be applied in new contexts. This is akin to the core idea of explanation-based learning.

For example, using a chair to prop open a door or placing a coffee mug on a stack of papers to prevent them from blowing away are actions often based on an implicit explanation: the chair works because it's heavy and stable enough to counteract the door's tendency to close; the mug works as a paperweight because it has sufficient weight and a flat bottom. These might seem trivial, but they represent a fundamental type of reasoning: using existing objects and knowledge about their properties and the causal relations that govern them in new and useful ways. This method of problem-solving is rooted in our ability to quickly form explanations and generalize them.

This approach to learning and problem-solving offers a potent framework for developing AI systems. By implementing EBL, we can design AI agents that emulate this aspect of human ingenuity. These agents would not merely operate within the confines of their initial

programming but would be capable of adapting their responses based on explanations derived from specific situations.

“Speed-up learning” is a term often associated with EBL. EBL systems can expedite their problem-solving processes not by acquiring entirely new fundamental concepts from scratch, but by creating new, more efficient rules (generalizations of explanations) that allow them to solve similar problems faster in the future. This form of learning is particularly powerful as it leverages existing knowledge bases extensively, transforming inefficient, declarative knowledge into more efficient, operational rules.

In practice, this means creating AI systems that can, for instance, recognize that an object's stability or weight could be used for purposes other than those it was originally designed for, such as using a book to hold down papers. The AI would understand not only the primary function of an object but also its potential utility in a variety of contexts based on explanations of its physical characteristics and their causal consequences.

Building AI agents with the capability for explanation-based learning involves teaching systems to construct explanations for why an example is an instance of a concept, and then to generalize these explanations. These systems need to assess their environments and draw from a rich database of domain knowledge (facts and rules) to construct these explanations and derive useful generalizations. This method stands to significantly enhance the flexibility and effectiveness of AI systems, allowing them to perform more human-like reasoning tasks.

EXPLANATION-BASED LEARNING AND COGNITION

Explanation-based learning (EBL) mirrors an integral aspect of human cognition and is a fundamental element in how humans learn and reason.¹⁶⁵⁻¹⁶⁷ It is a process we engage in daily, often without conscious effort. For instance, when using a chair to keep a door open, most people can quickly rationalize *why* the chair is suitable: it is stable, has sufficient weight, and can be wedged. Similarly, placing a coffee mug on a stack of papers to prevent them from scattering is another practical application where the rationale behind the mug's effectiveness as a paperweight (its weight, flat bottom providing stability) is readily understood. These everyday examples underscore the routine nature of explanation-based reasoning in our lives.

In cognitive science and knowledge-based artificial intelligence (KBAI), EBL is pivotal because it strives to model this human capacity for making spontaneous explanatory connections and learning from them.^{168,169} Unlike some branches of AI that might focus more on data-driven and probabilistic approaches from large datasets, KBAI, and EBL within it, emphasizes understanding and replicating human-like reasoning processes, making explanations a core component of its methodology.

Humans, while adept at explaining many interactions and phenomena, often struggle with introspective explanations for processes that are highly practiced or subconscious. For example, explaining the precise biomechanics of a tennis serve can be challenging even for an expert player. These limitations highlight that not all processes are equally accessible to our conscious,

explicit understanding, and our generated explanations may not always perfectly align with the actual underlying cognitive or physical processes.

In AI, the ability to generate coherent and understandable explanations for its conclusions (as distinct from the internal explanations EBL uses for learning) is becoming increasingly important, especially for user trust and acceptance. For instance, a medical diagnostic system gains credibility not only from its diagnostic accuracy but also from its ability to present an understandable rationale for its conclusions. This stresses the importance of transparency and explainability in AI, particularly in critical decision-making domains.

It's crucial to acknowledge that while humans can generate explanations for a vast array of actions and decisions, these explanations are sometimes post-hoc rationalizations; they are constructed after the fact and may not accurately represent the initial underlying decision-making process. This disparity can sometimes obscure the true causal mechanisms, potentially leading to misleading conclusions about our own reasoning.

Moreover, the process of formulating an explanation can itself influence and alter the reasoning process, introducing biases or changing decision pathways. This phenomenon poses challenges in both human cognition and the development of AI systems, especially when the goal is to create transparent and unbiased AI.

To cultivate trust and ensure the ethical deployment of AI, systems must be developed with the capability to not only perform tasks effectively but also, where appropriate, to provide clear and rational explanations for their actions and decisions.^{168,170} This capacity for explanation is a fundamental component of advanced AI systems aimed at achieving robust intelligence and functionality. EBL, by its very nature, is grounded in the construction and generalization of explanations, making it a key paradigm in the pursuit of more understandable and adaptable AI.

INTEGRATING KBAI: ENHANCING MODERN AI WITH KNOWLEDGE-BASED APPROACHES

The principles of Knowledge-Based Artificial Intelligence (KBAI), including Explanation-Based Learning (EBL), offer significant contributions to the broader landscape of modern AI, which is often dominated by data-intensive machine learning models. While deep learning and other statistical methods have achieved remarkable successes, they also exhibit limitations, particularly in areas where KBAI excels: explainability, learning from sparse data, and incorporating common-sense reasoning.

- **Complementing Data-Driven Approaches:** Modern AI systems can benefit greatly from hybrid architectures that integrate KBAI techniques with data-driven models. For instance, while a neural network might learn complex patterns from vast datasets, KBAI components can provide a framework for:
 - **Explainability:** EBL, by its nature, generates an explanation for why a conclusion is reached. This explicit reasoning chain can be invaluable in making the decisions of opaque models (like deep neural networks) more

transparent and interpretable. If a complex model classifies an image, an EBL component could potentially explain the classification based on symbolic features and learned rules, providing a rationale beyond statistical correlations.

- **Knowledge Injection and Refinement:** Prior knowledge, structured in a symbolic form as used in KBAI, can guide the learning process of data-driven models, especially in scenarios with limited training data. EBL can take high-level domain knowledge and operationalize it, creating more specific rules that can then constrain or inform a statistical learner. Conversely, patterns discovered by machine learning can suggest new symbolic knowledge or help refine existing knowledge bases.
- **Learning from Few Examples:** EBL is designed to learn from one or a few examples by deeply explaining them against a background theory. This contrasts sharply with many deep learning models that require thousands or millions of examples. In domains where data is scarce, expensive, or sensitive (e.g., rare disease diagnosis, critical system failure analysis), EBL offers a viable learning mechanism.
- **Addressing Common-Sense Reasoning:** A significant challenge for current AI is imbuing systems with common-sense reasoning – the vast body of implicit knowledge humans use to navigate the world. KBAI approaches, with their emphasis on symbolic representation of concepts, relationships, and causalities, provide a natural framework for encoding and reasoning with common-sense knowledge. EBL can help operationalize and generalize common-sense principles from specific examples of their application. For example, understanding *why* a pitcher is better than a backpack for carrying soup relies on common-sense physics and functional knowledge, which EBL can leverage.
- **Enhancing Robustness and Adaptability:** Knowledge-based systems, including those using EBL, can exhibit greater robustness when faced with novel situations that are outside the specific distribution of training data seen by purely statistical models. By reasoning from underlying principles and causal explanations, an EBL-enhanced system may be better able to adapt its knowledge to slightly different contexts. The generalization process in EBL aims to capture the core reasons why a concept holds, making the learned knowledge more broadly applicable.
- **The Path Forward: Hybrid Intelligence:** The future of AI likely lies in hybrid systems that harness the strengths of both knowledge-based reasoning and data-driven learning. KBAI and EBL provide mechanisms for deeper understanding, more efficient learning from limited data, and improved explainability. Integrating these approaches with the pattern-recognition power of modern machine learning can lead to AI systems that are not only more capable but also more reliable, transparent, and aligned with human reasoning. For instance, research into neuro-symbolic AI explicitly aims to

combine neural networks with symbolic reasoning, where EBL could play a role in the symbolic learning and reasoning components.

By leveraging the structured knowledge and explanatory power inherent in KBAI paradigms like EBL, we can work towards AI systems that possess a more holistic and human-like intelligence, capable of not just performing tasks but understanding the world in a more meaningful way.

CHAPTER SUMMARY

In this chapter, we investigated the process of explanation-based learning (EBL), a learning paradigm that emphasizes forming new, generalized rules by explaining why a specific training example is an instance of a concept, using an existing domain theory. This approach is crucial for advancing both human cognitive modeling and the development of sophisticated AI systems that can learn efficiently from limited data.

We introduced the idea that EBL operates within a 'concept space,' utilizing a background of prior knowledge (facts and rules) to construct explanations. This prior knowledge is mapped onto new situations to form the basis for reasoning and ultimately, for generalizing the explanation into a new, operational rule. Through abstraction, essential features of a concept, as revealed by the explanation, are distilled. This abstraction allows for the 'transfer' of these refined concepts or rules to new problems, facilitating creative solutions and the application of knowledge to different contexts.

The principles of explanation-based learning find wide application, from enabling AI to understand object functionality in everyday tasks to more complex professional activities. This adaptability makes it a powerful tool in AI, especially in areas requiring cognitive flexibility and learning from few examples. EBL is often described as a form of “speed-up learning” because it transforms declarative knowledge into more efficient, operational forms.

We also explored how EBL integrates with and enhances modern AI. By complementing data-driven approaches, EBL can contribute to improved explainability, facilitate learning from sparse data, and help incorporate common-sense reasoning into AI systems, paving the way for more robust and adaptable hybrid intelligence.

Looking ahead, the principles of EBL, particularly its focus on understanding underlying structures and causal relationships, connect to other forms of reasoning, such as analogical reasoning, which we will explore further. Analogical reasoning extends these ideas to connect seemingly unrelated concepts through similarities in their relational structures.

By engaging with explanation-based learning, we equip ourselves with a framework for understanding a powerful form of machine learning that mirrors aspects of human learning and innovation, preparing AI systems to tackle new challenges by leveraging existing knowledge more effectively. This chapter sets the stage for deeper explorations into enhancing AI systems with human-like reasoning capabilities, emphasizing the importance of explanation in achieving transparent, trustworthy, and efficient AI.

ADDITIONAL READINGS & REFERENCES

- 152 Teso, S., Alkan, Ö., Stammer, W. & Daly, E. Leveraging explanations in interactive machine learning: An overview. *Frontiers in Artificial Intelligence* **6**, 1066049 (2023).
- 153 De Martino, B. & Cortese, A. Goals, usefulness and abstraction in value-based choice. *Trends in Cognitive Sciences* **27**, 65-80 (2023).
- 154 Ho, M. K., Abel, D., Griffiths, T. L. & Littman, M. L. The value of abstraction. *Current opinion in behavioral sciences* **29**, 111-116 (2019).
- 155 Timperley, M., Mokhtar, M., Bellaby, G. & Howe, J. Explanation-based learning with analogy for impasse resolution. *Expert Systems with Applications* **61**, 181-191 (2016).
- 156 Gepperth, A. & Hammer, B. in *European symposium on artificial neural networks (ESANN)*.
- 157 Jennings, K. E. Developing creativity: Artificial barriers in artificial intelligence. *Minds and Machines* **20**, 489-501 (2010).
- 158 De Pisapia, N. & Rastelli, C. Creativity as an information-based process. *Rivista internazionale di Filosofia e Psicologia* **13**, 1-18 (2022).
- 159 Schölkopf, B. *et al.* Toward causal representation learning. *Proceedings of the IEEE* **109**, 612-634 (2021).
- 160 Sbaï, A. H. & Chaari, W. L. Extended causal map for reasoning explanation in multi-agent systems. *International Journal of Intelligent Systems Technologies and Applications* **12**, 301-315 (2013).
- 161 Sbaï, A. H., Chaari, W. L. & Ghédira, K. Intra-agent explanation using temporal and extended causal maps. *Procedia Computer Science* **22**, 241-249 (2013).
- 162 Yang, S. C. H., Folke, T. & Shafto, P. Abstraction, validation, and generalization for explainable artificial intelligence. *Applied AI Letters* **2**, e37 (2021).
- 163 Oh, S. J., Schiele, B. & Fritz, M. Towards reverse-engineering black-box neural networks. *Explainable AI: interpreting, explaining and visualizing deep learning*, 121-144 (2019).
- 164 Sakcak, B., Weinstein, V. & LaValle, S. M. in *International Workshop on the Algorithmic Foundations of Robotics*. 256-272 (Springer).
- 165 Khosravi, H. *et al.* Explainable artificial intelligence in education. *Computers and education: artificial intelligence* **3**, 100074 (2022).
- 166 Fiok, K., Farahani, F. V., Karwowski, W. & Ahram, T. Explainable artificial intelligence for education and training. *The Journal of Defense Modeling and Simulation* **19**, 133-144 (2022).
- 167 Chen, L., Chen, P. & Lin, Z. Artificial intelligence in education: A review. *Ieee Access* **8**, 75264-75278 (2020).
- 168 Miller, T. Explanation in artificial intelligence: Insights from the social sciences. *Artificial intelligence* **267**, 1-38 (2019).
- 169 Taylor, J. E. T. & Taylor, G. W. Artificial cognition: How experimental psychology can help generate explainable artificial intelligence. *Psychonomic Bulletin & Review* **28**, 454-475 (2021).
- 170 Bickley, S. J. & Torgler, B. Cognitive architectures for artificial intelligence ethics. *Ai & Society* **38**, 501-519 (2023).

Chapter 17

Analogical Reasoning

INTRODUCTION TO ANALOGICAL REASONING

In this chapter, we explore the fascinating area of Analogical Reasoning, a cognitive process that allows us to tackle new challenges by drawing parallels with familiar problems. Analogical Reasoning is not just about recognizing similarities; it fundamentally involves the transfer of knowledge about relationships and structures from well-understood problems to new, unfamiliar scenarios. This transfer is critical for solving problems that, at first glance, may appear unrelated but are connected through underlying principles.

Previously, we examined the concept of transfer in the contexts of Explanation-Based Learning and Case-Based Reasoning. These discussions set the groundwork for a deeper examination of transfer, which we will investigate in a much broader scope in this chapter. Our journey through the topic will begin with a close look at the concept of similarity – how it is defined and identified in various contexts.

Building on our understanding of similarity, we will revisit Case-Based Reasoning to see how previously encountered situations inform new decisions. From there, we will progress to a comprehensive discussion on the stages of analogical reasoning: retrieval, where we identify analogous cases; mapping, where we establish correspondences between elements of the known and the new problems; and transfer, where the insights from familiar scenarios are adapted to solve new problems.

To illustrate the practical applications of these concepts, the chapter will conclude with an examination of 'Design by Analogy'. This specific application of analogical reasoning showcases how insights from one domain can inspire innovative solutions in another, demonstrating the power and versatility of this approach in problem-solving. By understanding and applying analogical reasoning, we can enhance our ability to think creatively and solve complex challenges in various fields.

UNDERSTANDING SIMILARITY IN ANALOGICAL REASONING

At the heart of analogical reasoning lies the concept of similarity – a fundamental component that enables us to draw meaningful connections between disparate situations. Understanding new problems in terms of familiar ones hinges critically on our ability to discern how they are similar. This goes beyond superficial resemblances, extending into deeper structural, relational, and functional similarities.¹⁷¹

The concept of similarity in analogical reasoning encompasses more than just a straightforward comparison of observable characteristics. It involves a nuanced assessment of relationships between elements, the roles they play, and the contexts in which they operate. For example, understanding how a situation involving a woman climbing a ladder can inform a variety of scenarios requires us to think about what it means to “climb,” what a “ladder” represents in different contexts, and how these elements interact dynamically within each scenario.

Within Case-Based Reasoning, which we discussed earlier, similarity plays a pivotal role. We've seen it operationalized through methods like finding the nearest neighbor based on Euclidean distance or organizing cases in a discrimination tree. Such structures help in identifying which past cases are most relevant to a new problem based on a detailed comparison of their features.

By examining the concept of similarity, we are not only able to see the obvious connections but also the subtle, less immediate ones. This deeper look into similarity allows us to engage with analogical reasoning more effectively, using it to solve problems by transferring knowledge from known cases to novel situations. This transfer is not random; it is guided by the degree of similarity and the relevance of the relationships between the elements involved.

As we proceed further into this chapter, we will explore various dimensions of similarity – how it is defined, measured, and applied in different scenarios of analogical reasoning. This understanding will lay the groundwork for subsequent sections where we examine the mechanisms of retrieval, mapping, and transfer that characterize effective analogical reasoning in both everyday problem-solving and complex scientific reasoning.

A woman is climbing a ladder.

Which of the situations on the right is most similar to the situation above?

Rank them from 1 (most similar) to 7 (least similar)

- A woman climbing a set of stairs.
- An ant walking up the wall.
- A woman painting a ladder.
- A woman climbing the corporate ladder.
- A water bottle sitting on a desk.
- A woman climbing a step ladder.
- A plane taking off into the sky.

Figure 17-1 Similarity example of a “Woman climbing a ladder”.

Continuing our exploration of similarity in analogical reasoning, let's consider a practical exercise that demonstrates how we evaluate similarity in various scenarios. This exercise involves ranking different situations based on their similarity to the situation of “a woman climbing up a ladder.” Here are seven situations. Can you please rank these seven situations by their order of similarity to the given situation?

Ashok David, what was your answer?

David So, I said that the two most similar were “the woman climbing up a step ladder” and “a woman climbing a set of stairs.” I said the woman climbing the step ladder was a little more similar because a step ladder is more similar to a ladder than a set of stairs is to a ladder. But in both of these situations, we still have a woman and she's still going up some kind of physical artifact.

For third, I put “an ant walking up the wall”. I ranked that as a little bit more similar than “a woman climbing the corporate ladder” because of its physical motion up something rather than metaphorical. But there's also similarity between a real ladder and a corporate ladder, so it could have been ranked higher. For the fifth one, I picked “a plane taking off into the sky”. While it's physically similar to an ant climbing up a wall in that they're both rising, the degree to which they rise is different; a plane's rise is much more significant. The mechanism by which they rise is different as well.

Then there's "a woman painting a ladder". I didn't think that was very similar except that it involved a woman and a ladder.

Finally, there is "a water bottle sitting on a desk", which I don't think is similar at all.

Ashok Interesting answer, David.

It's insightful to see the factors influencing David's assessments. In the two situations he deemed most similar to a woman climbing a ladder, there was a clear similarity in the activity or relationship – climbing up – as well as the objects involved: the woman and some form of a ladder. In stark contrast, the scenario involving a woman painting a ladder, although involving similar objects (woman and ladder), diverged significantly in the nature of the activity, highlighting how different relationships between similar objects can lead to activities that are perceived very differently.

This exercise underscores the complexity of assessing similarity in analogical reasoning, where not only the physical attributes but also the actions and contexts play crucial roles in how we draw analogies across different situations.

INDEXING SCHEMES FOR CASE RETRIEVAL

Understanding similarity in analogical reasoning involves the application of indexing schemes that facilitate the retrieval of related cases based on defined metrics. These indexing schemes, such as the nearest neighbor method and the use of discrimination trees, are pivotal in the effective implementation of case-based reasoning, which we've previously discussed.

One foundational method we explored is the **nearest neighbor technique**. This method is straightforward: it evaluates the proximity of a new situation to known cases using Euclidean distance. For example, in the context of urban navigation, if you have a series of cases each marked by coordinates (x and y locations of destinations), the nearest neighbor to a new coordinate point can be identified by calculating which existing point it is closest to in terms of physical distance on the map. This proximity is assumed to correlate with similarity, meaning the nearest case likely has the most in common with the new situation concerning the aspects measured – here, geographic location.

Finding the Nearest Neighbor

Given existing case at (x_c, y_c)
and new problem at (x_n, y_n)

$$d = \sqrt{(y_c - y_n)^2 + (x_c - x_n)^2}$$

Block	x_c	y_c	x_n	y_n	d
Blue	0.5	2.0	1.1	1.6	0.72
Red	0.5	0.5	1.1	1.6	1.25
Black	1.0	1.5	1.1	1.6	0.14
Green	1.5	1.5	1.1	1.6	0.41
Orange	1.5	0.5	1.1	1.6	1.17
Purple	2.0	1.0	1.1	1.6	1.08

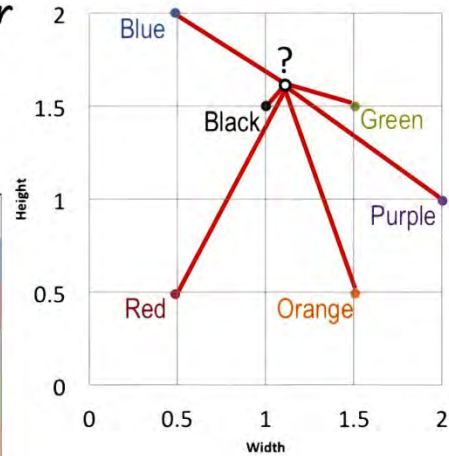


Figure 17-2 Indexing using Nearest Neighbors.

A more complex and structured approach to organizing cases is through a **discrimination tree**. This method categorizes cases into a decision tree where each node represents a decision point that branches based on the values of specific features. For instance, in an urban navigation scenario, cases might be divided at a root node based on whether they are east or not east of 5th Street. Further branching could differentiate cases based on additional criteria like proximity to landmarks or type of terrain.

Route	Destination _x	Destination _y
A	10 _E	8 _N
B	1 _E	8 _N
C	10 _E	4 _N
D	2 _E	1 _N
X	8 _E	2 _N
Y	8 _E	2 _N
Z	1 _E	9 _N

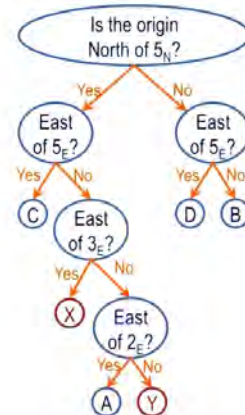


Figure 17-3 Indexing using Discrimination Trees.

The discrimination tree allows for a hierarchical sorting of cases, where the path followed from the root to a leaf – where actual cases are represented – determines how similar a new situation is to the stored cases based on multiple features' values. This method not only considers simple proximity but integrates a variety of features, making it suitable for complex domains where multiple factors influence case similarity.^{172,173}

Both indexing methods highlight different dimensions of similarity:

- **Nearest neighbor** focuses on proximity in a straightforward, often single-dimension metric.
- **Discrimination tree** incorporates a multi-faceted view, considering various attributes and their combinations to assess similarity.

These methods underscore that similarity in analogical reasoning is not a one-size-fits-all evaluation but depends significantly on the specific attributes and relationships relevant to the domain in question. They also reflect the importance of how cases are stored and retrieved, impacting the efficiency and relevance of the analogies drawn between new problems and known cases. Understanding and applying these indexing schemes correctly is crucial for building systems that can effectively use past experiences to navigate new challenges.

CROSS-DOMAIN ANALOGIES

In analogical reasoning, cross-domain analogies represent a sophisticated form of cognition where similarities are identified and leveraged between scenarios that do not share the same domain. This ability to bridge distinct domains is crucial for creative problem-solving and innovation.

Typically, when considering problems within the same domain, such as navigation in an urban area or arranging colored blocks, the connections and similarities are more straightforward because the elements involved share contextual and often physical properties. However, when the problems span different domains, the challenge intensifies due to the dissimilarity of the contexts and the elements involved. For example, a woman climbing a ladder and an ant walking up a wall involve very different scales and creatures, but both share the action of ascending vertically, albeit on different surfaces.

A compelling illustration of cross-domain analogies can be found in the well-known medical problem introduced by psychologist Karl Duncker in 1926.¹⁷⁴ In this scenario, a physician faces a dilemma: a powerful radiation source can destroy a tumor inside a patient's stomach, but unfortunately, it also poses a threat to healthy tissue. This medical problem, while specific to healthcare, echoes challenges found in completely different domains, such as military strategy or even urban planning, where focused actions may have unintended widespread effects.

The medical solution is to arrange the radiation sources so that each beam, individually examined, is insufficient to damage tissue, but when the beams combine at the tumor site, their intensities combine to destroy the tumor.

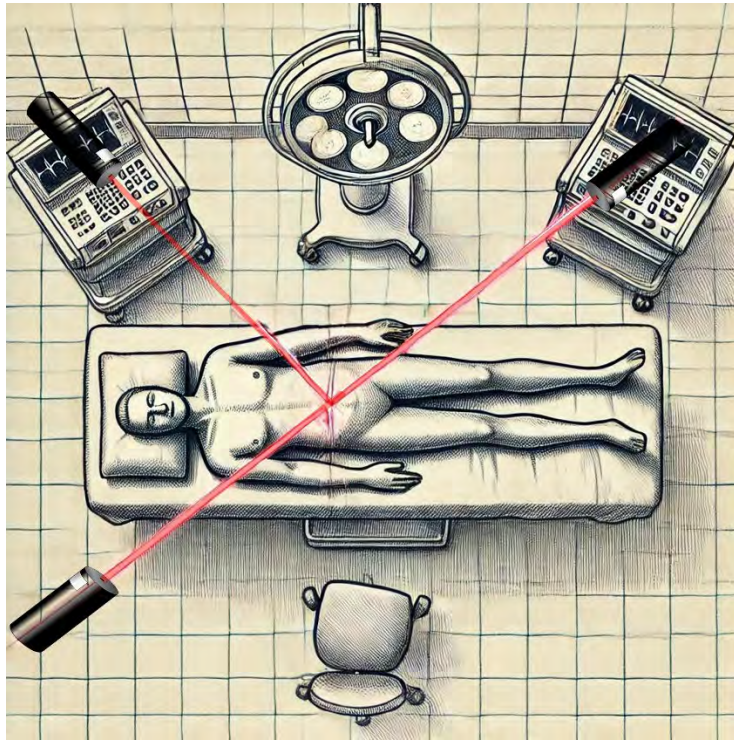


Figure 17-4 Karl Duncker's patient tumor problem is solved by using three lasers, each with a lower intensity as not to damage tissue on their paths when considered individually, but when converged are sufficiently powerful to destroy the tumor.

The Kingdom's Strategic Dilemma: An Illustrative Analogy

Let's take a closer look at how these analogies can be applied to seemingly unrelated problems to derive innovative solutions. A poignant example to illustrate this is the story of a kingdom. In this fictional narrative, a ruthless king's fortress was approached by a rebel army. The roads leading to the fort were mined, posing a lethal threat to any large group attempting an assault.

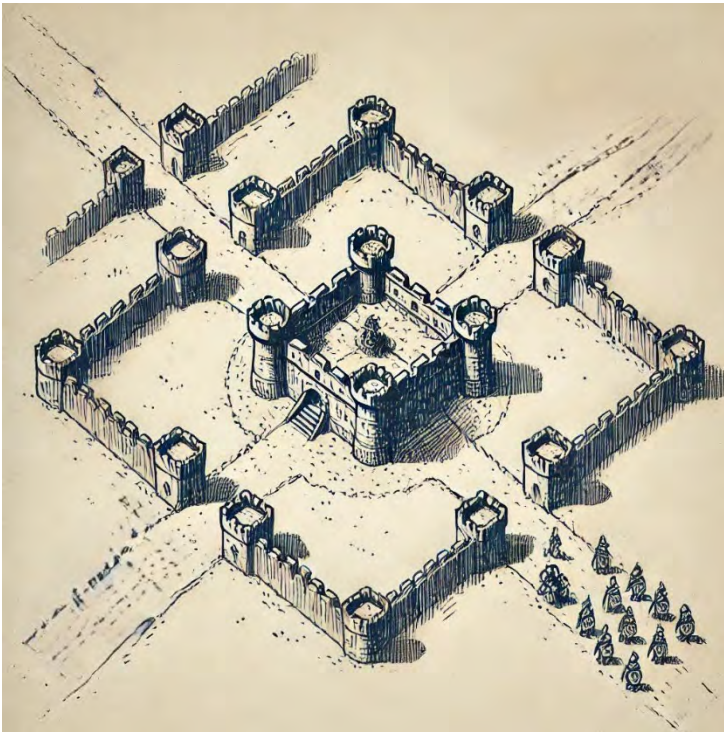


Figure 17-5 The military source problem occurs when too many soldiers approach along a single path, trigger defenses that would immobilize the force.

To overcome this problem, we can return to the medical scenario where the physician faces a similar challenge: a powerful radiation source that can destroy a tumor but also harm healthy tissue if used directly. Inspired by the medical tactic of dividing the radiation into multiple smaller beams, the rebel army ingeniously split into smaller groups, each taking a different route. This strategy prevented the mines from detonating, allowing the soldiers to converge on the fort simultaneously and successfully overthrow the king. This story exemplifies strategic decomposition – dividing a large force into smaller units to achieve a common goal without triggering defenses. This solution demonstrates how a strategy from a completely unrelated domain – medical treatments – can provide a viable solution in military tactics, highlighting the power of cross-domain analogical reasoning.¹⁷⁵

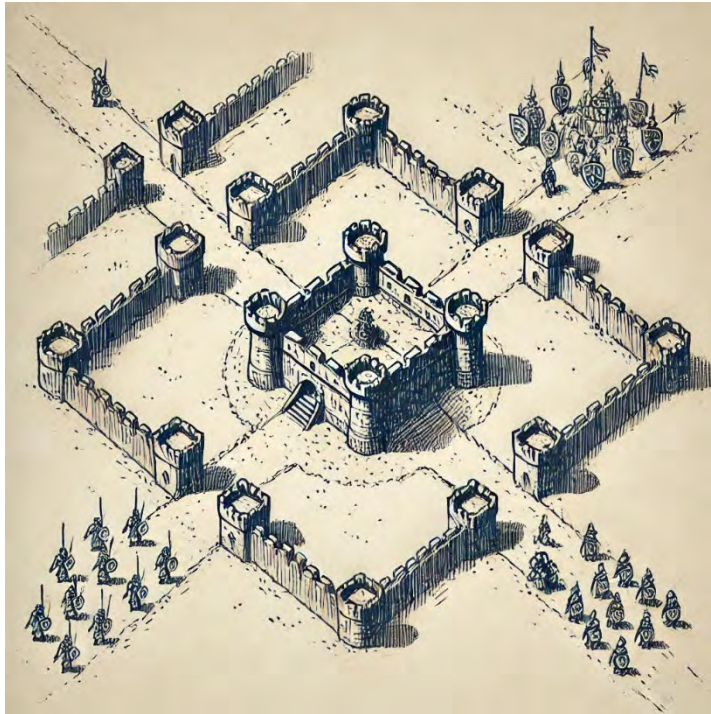


Figure 17-6 Transferring analogical knowledge from Duncker's tumor patient solution and applying it to the military problem where approaching simultaneously from multiple directions renders the defensive measures ineffective.

In both cases – the siege of the fortress and the medical treatment – the fundamental strategy involves decomposing a powerful resource into smaller units to achieve a goal safely and effectively. The scenarios involve different domains (military and medical) and different objects (armies and radiation beams), yet the relational structure – the way resources are managed to circumvent a critical problem – is strikingly similar.

This type of reasoning underscores the importance of relationships over the physical or contextual characteristics of the elements involved. Whether it's soldiers navigating a minefield or physicians targeting a tumor, the underlying relational strategy is what bridges these disparate domains. Such analogies not only enrich our understanding of different fields but also expand the possibilities for innovative problem-solving by transferring strategic insights across domain boundaries.

The key to analogical reasoning lies in identifying abstract similarities that transcend the surface-level differences between the domains. The task is to find a common relational structure or a similar set of dynamics that govern interactions in both the source and the target scenarios.

For instance, in both the medical scenario and the military scenario, the underlying principle involves distributing a potent force in a manner that achieves the goal without collateral damage.

This approach also explains the conceptual link between seemingly unrelated actions, such as an ant climbing a wall and a woman climbing a ladder. While the entities and their contexts differ greatly, the action – ascending vertically – forms the basis of their similarity. It is through these relational similarities that we can apply solutions from one domain to problems in another, showcasing the robustness and versatility of analogical reasoning in human cognition.¹⁷⁶

This kind of thinking – drawing parallels from different areas to address a problem – highlights the essence of cross-domain analogies. It shows how analogical reasoning is not just about matching similar situations but about abstracting the underlying principles that can guide actions across varied scenarios. This ability is fundamental in developing innovative solutions to complex problems where direct answers are not apparent.

The Spectrum of Similarity

We often conceptualize analogical reasoning within a spectrum of similarity that ranges from perfect alignment to complete divergence between the target problem and the source case. This spectrum is crucial for evaluating the connections between new and known problems, assessing how closely they align based on relationships, objects, features, and the values of these features.

At one end of the spectrum, the target problem and the source case are nearly identical across all dimensions. This situation is typical in recording cases, where similar past scenarios are recalled based on their striking similarities. For example, scenarios involving colored blocks in a blocks world might be indexed and retrieved based on exact matches in relationships, objects, features, and values.

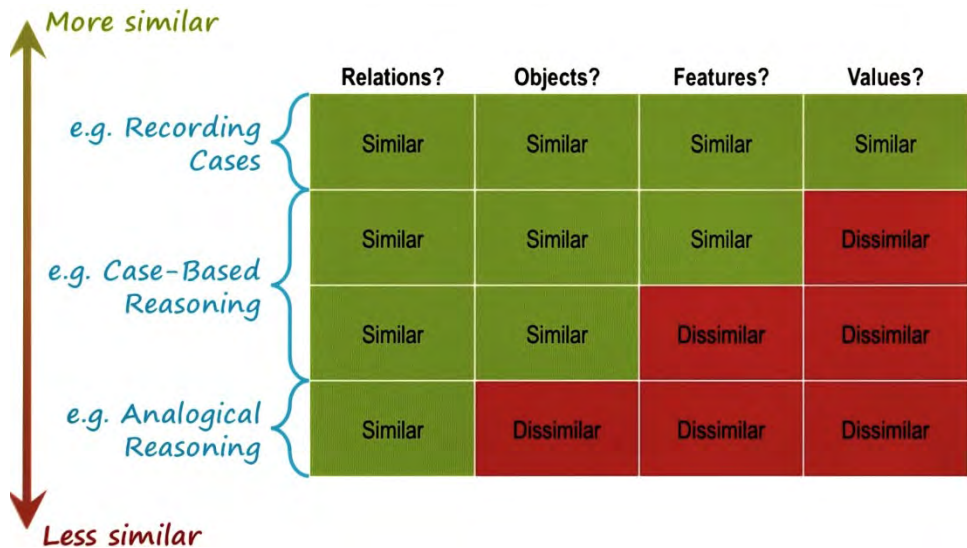


Figure 17-7 Similarity table.

As we move along the spectrum, we encounter scenarios where the relationships and objects align, but the features and values may begin to diverge. This setup is often found in case-based reasoning, especially within specific domains like urban navigation. Here, the fundamental objects and their interactions remain consistent, even though the specific conditions or attributes may vary. The objects of the concept between the target problem and the source case being the same suggests that the domains are similar, so the reasoning is applied within the same domain framework.

At the far end of the spectrum, only the relationships might align, with significant differences in the objects, features, and values involved. This scenario is where cross-domain analogical reasoning becomes critical. A prime example of this is Duncker's radiation problem, where the strategic concept of distributing resources for a coordinated effect – whether it be soldiers attacking a fort or radiation beams targeting a tumor – demonstrates how strategies can be transferred across vastly different contexts.

This understanding highlights the profound utility of analogical reasoning: it allows us to approach new problems by drawing on familiar ones, transferring knowledge across different contexts to innovate and solve. By navigating this spectrum of similarity, we can extend our problem-solving capabilities, applying known relationships in new ways to unlock creative solutions. This process not only enhances our understanding of both familiar and new phenomena but also enriches our cognitive and practical engagements with the world around us.

A HIERARCHICAL PROCESS FOR ANALOGICAL REASONING

Analogical reasoning, a sophisticated cognitive process, unfolds through a series of distinct yet interconnected phases. Each phase is integral to how we understand and apply knowledge from one context to another, allowing us to tackle novel problems by drawing on familiar experiences. The hierarchy of these phases, as depicted in the accompanying diagram, outlines a structured approach to analogical reasoning.



Figure 17-8 Analogical Reasoning Process Hierarchy.

3. **Retrieval:** The first step in analogical reasoning is retrieval, where relevant past experiences or known cases are recalled. This involves identifying a source case that has aspects in common with the current problem. The effectiveness of this stage depends heavily on the breadth and depth of one's knowledge base and the ability to access relevant memories or stored cases.
4. **Mapping:** Once a suitable source case is retrieved, the next step is mapping. This involves determining the correspondences between the elements of the source case and

those of the target problem. Mapping is crucial for identifying which aspects of the previous situation align with the current context. For instance, in Duncker's radiation problem, the conceptual relationship of distributing a resource strategically to avoid harm is mapped from a military tactic to a medical treatment strategy.

5. **Transfer:** Following successful mapping, the transfer phase involves applying the insights or solutions from the source case to the target problem. This may require abstracting certain elements from the source to fit the new context. In the radiation problem example, the strategy of dividing forces to minimize impact was abstracted and then applied to the radiation beams targeting a tumor.
6. **Evaluation:** Once the transferred solution is implemented, it must be evaluated to determine its effectiveness. This stage assesses whether the analogical application solves the problem as intended and whether it introduces any new issues. Evaluation can lead to a refinement of the solution and may necessitate revisiting earlier stages for adjustments.
7. **Storage:** The final stage involves incorporating the new knowledge and experience gained from the analogical reasoning process into one's knowledge base. This includes not only the solution but also insights about the mapping and transfer processes. Successful cases are stored for future reference, enhancing the repository of cases available for retrieval in future analogical reasoning tasks.

By examining these stages in the context of analogical reasoning alongside case-based reasoning, we observe similarities and differences. In both processes, retrieval, evaluation, and storage play pivotal roles. However, analogical reasoning can extend beyond the domain constraints typical of case-based reasoning, requiring more complex mappings and transfer stages due to the potential for cross-domain applications. This flexibility allows analogical reasoning to adapt concepts from vastly different domains to solve new problems creatively and effectively.¹⁷⁷⁻¹⁷⁹

The framework of analogical reasoning outlined above represents just one of several theories within the field. Different theoretical models can configure the processes of reasoning in varied ways, highlighting the flexibility and depth of this cognitive strategy. For instance, in some theories, the boundaries between the phases are less distinct, integrating processes like mapping and retrieval into a single, intertwined step.¹⁸⁰

DEGREES OF SIMILARITY: SUPERFICIAL VS. DEEP

When discussing analogical reasoning, a key concept to grasp is the evaluation of similarity, which allows for effective retrieval of relevant cases or knowledge. This evaluation is not straightforward, especially when the target problem and the source case span different domains, presenting a challenge in identifying applicable analogies.

The idea of similarity in analogical reasoning can be categorized into two distinct types: superficial similarity and deep similarity. Superficial similarity focuses on the observable features

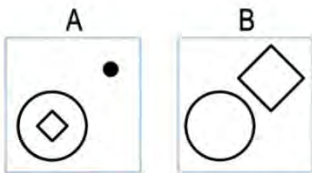
of objects, such as their physical appearance, counts, or the explicit characteristics that are immediately evident. For example, in a visual test, superficial similarity might be identified through the number of shapes present, their sizes, or their explicit forms like squares or circles. This type of similarity is often the easiest to recognize because it deals with direct, observable elements.

Deep similarity, on the other hand, explores the more abstract connections between objects or even the relationships between these connections. It transcends the immediate features to consider how objects interact or relate to one another within a system. For instance, deep similarity would not just note the presence of shapes but would explore how these shapes are positioned in relation to each other or how they interact within the overall structure. This kind of similarity is crucial for solving complex problems where the link between elements isn't overtly visible but is critical for understanding the system's behavior or for generating solutions that are not immediately obvious from the surface characteristics.

The accompanying image visually represents these concepts, distinguishing between superficial and deep similarities with clarity. Consider the left side of the image, which shows two scenarios (A and B). In both, the relationships of position between the shapes are maintained – the dot is outside and above the circle in both scenarios. This consistency in relational placement exemplifies deep similarity, where the relationship between the objects is the focus.^{181,182}

Superficial Similarity

- *Features*
- *Counts*
- *Objects*



Deep Similarity

- *Relationships between objects*
- *Relationships between relationships*

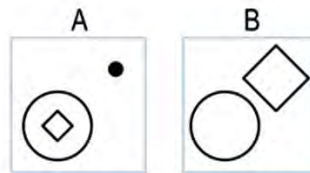


Figure 17-9 Characteristics of Superficial and Deep Similarity.

Moving to a more complex analysis, we examine the relationships between relationships. This advanced concept is about understanding changes or consistencies in relational dynamics from one scenario to another. For instance, if a dot transitions to a square while maintaining its Draft -Version:6/2/2025 3:06 PM

external and superior position relative to a circle, this transformation highlights a higher order of relational consistency. This kind of analysis is akin to what one might encounter in Raven's Progressive Matrices, where understanding how object relationships transform across sequences is crucial.

This layered approach to similarity – from superficial attributes to the relationships between objects, and further into the relationships between those relationships – enables a deeper analytical perspective. It allows one to infer similarity based not just on what is visible but on the underlying principles governing the interactions within the scenario. As we progress from recognizing unary relationships (simple presence or absence of attributes) to binary and tertiary relationships (how objects relate to each other and how these relationships themselves interact), the depth of similarity understanding increases, enabling more sophisticated reasoning and problem-solving capabilities.

In analogical reasoning, determining which type of similarity to apply – superficial or deep – depends significantly on the problem at hand. For problems where the domain and context are similar, superficial similarities might suffice. However, in cases where the problems are from vastly different domains, as in cross-domain analogies, deep similarities become essential. These involve understanding the underlying principles or dynamics that govern the relationships within the cases, regardless of the differences in the superficial attributes of the objects involved.

This nuanced approach to similarity allows for a more flexible and profound application of analogical reasoning, enabling thinkers and problem-solvers to bridge the gap between seemingly disparate domains and to innovate or adapt solutions from one context to another.

TYPES OF SIMILARITIES: SEMANTIC, PRAGMATIC, AND STRUCTURAL

In analogical reasoning, discerning similarity between cases is pivotal, particularly when these cases are from varied domains. Understanding the types of similarity helps in establishing a more nuanced approach to reasoning and solving new problems. There are three primary types of similarity that are instrumental in analogical reasoning: semantic, pragmatic, and structural similarity. Each type plays a unique role in how we connect different scenarios and apply learned solutions to new challenges.

- **Semantic similarity** focuses on the conceptual parallels between the target problem and the source case. This type of similarity is concerned with the inherent meaning and the conceptual overlap between the two situations. For instance, if we consider the example of a woman climbing a ladder and another climbing a step ladder, the semantic similarity lies in the shared concept of 'climbing' a 'ladder.' Despite the slight variation in the type of ladder, the fundamental activity and the object involved remain conceptually aligned. This similarity allows us to draw analogies based on the shared meanings of the elements involved in the scenarios.

- **Pragmatic similarity** extends beyond the immediate presentation of objects and relationships to consider the goals or intentions behind actions within the scenarios. It examines how the purposes of the actions in different scenarios align. For example, in the scenario where a physician uses radiation to target a tumor, the goal is to eliminate the tumor without harming the patient, akin to the strategic goal in a military scenario where a rebel army aims to capture a fort without triggering mines. Although the contexts are vastly different – one medical, the other military – the underlying goal of achieving an objective without collateral damage is the same. This similarity in objectives across different scenarios illustrates pragmatic similarity.
- **Structural similarity** relates to the organization and configuration of relationships within the scenarios, focusing on the structure of the representational models rather than the physical structures involved. This type of similarity assesses how the elements within each scenario are related to each other structurally. For instance, if we analyze a situation in which strategic movements in a battle are structured similarly to approaches in a medical procedure to isolate a tumor, the structural patterns in planning and execution reveal a deep, abstract similarity that transcends the surface details of the scenarios.

Different theories of analogy may emphasize different types of similarity based on the context or the nature of the problem at hand. Some may prioritize structural similarity, especially in cases where the relationships and their configurations provide critical insights. Others might focus more on semantic or pragmatic similarities, particularly when the conceptual understanding or the goals of the scenarios are paramount. The variation in emphasis on these types of similarity often explains why different individuals or systems might interpret or solve problems differently based on the same set of data.¹⁸³

By understanding and applying these types of similarity, we can enhance our ability to use analogical reasoning effectively, allowing for more sophisticated and targeted approaches to problem-solving across diverse domains.

Exercise: Assessing Degrees of Similarity

In this exercise, we further explore the concept of deep and superficial similarities by analyzing a range of scenarios in relation to a baseline situation: a woman climbing a ladder. Understanding these similarities aids in grasping how analogical reasoning can be applied across varied situations, even when they initially appear unrelated. Please consider the following diagram and indicate the degree of similarity of each of the sentences.

A woman is climbing a ladder.

Which of the situations on the right is most similar to the situation above?

Rank them from 1 (most similar) to 7 (least similar)

- A woman climbing a set of stairs.
- An ant walking up the wall.
- A woman painting a ladder.
- A woman climbing the corporate ladder.
- A water bottle sitting on a desk.
- A woman climbing a step ladder.
- A plane taking off into the sky.

Figure 17-10 Degree of Similarity Example.

Ashok What do you think, David?

David So perhaps unsurprisingly, I found that the two most similar situations in my original answer were both deeply and superficially similar. A woman climbing a step ladder is deeply similar because the fundamental activity – climbing a ladder – is unchanged. Superficially, it's similar too because it involves a woman and an object that functionally resembles a ladder. I also believe that a woman climbing a set of stairs shares both deep and superficial similarities. Both scenarios involve ascending, and stairs are structurally similar to a ladder, facilitating upward movement.

Then I considered the scenarios involving ascending movements in different contexts. The plane taking off into the sky, the woman climbing the corporate ladder, and the ant walking up the wall all shared a deep similarity with the original scenario because they all involve a rising action, albeit in vastly different contexts and scales. However, I found that the woman painting a ladder was only superficially similar – it involved a ladder, but the activity was entirely different, focusing on painting rather than climbing.

Finally, the situation with the water bottle sitting on a desk lacked both deep and superficial similarities to the scenario of climbing a ladder. It neither involves ascent nor shares any contextual or physical resemblance to the ladder-climbing scenario.

Ashok This is good, David. Your analysis highlights how deep and superficial similarities can guide our understanding and categorization of situations in analogical reasoning. It demonstrates how relationships and object features play critical roles in determining the degree of similarity and hence the potential for analogical transfer of knowledge or strategies across different situations.

Once again, different people may give different answers to this exercise. Why did we do so? Well, let's examine this next.

In educational contexts, the analogy between the solar system and the atomic structure is frequently employed to help students grasp complex scientific concepts by relating them to more familiar systems. This approach utilizes well-known dynamics of celestial bodies to clarify the interactions within an atom. Let's look at the representations of the solar system and the atomic structure to identify their deep similarities.

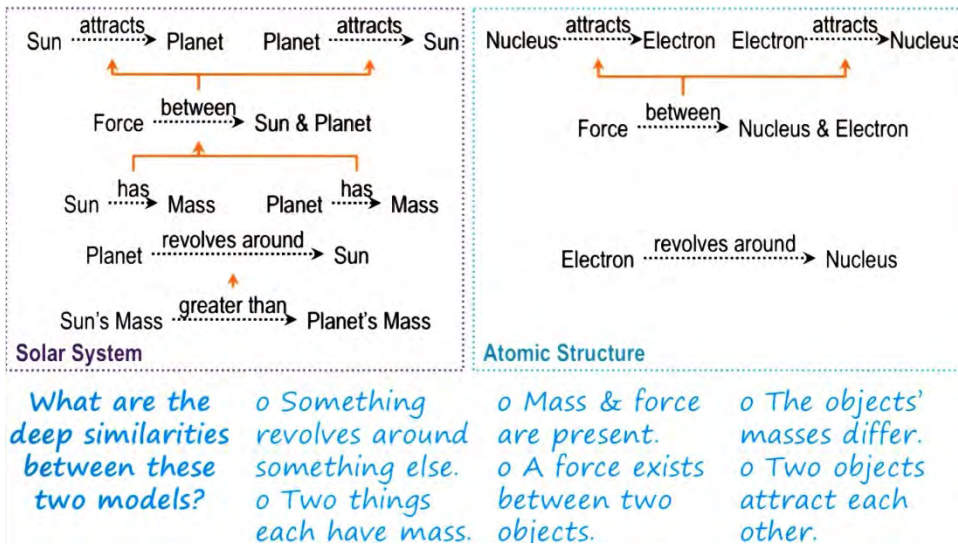


Figure 17-11 Identifying solar system and atomic structure deep representations.

Ashok What do you think, David? What are the deep similarities here?

David Looking at these models, there are clear deep similarities that help us understand the underlying principles governing both systems. First, there's something revolving around something else: in the solar system, planets revolve around the Sun; in the atomic structure, electrons revolve around the nucleus. This commonality of revolution is a foundational dynamic in both models.

Next, the interaction of forces is evident in both systems. The solar system operates under the gravitational force, where the Sun's mass exerts a pull on the planets. Similarly, in the atomic structure, electrostatic forces dictate the attraction between the nucleus and electrons. This shows how forces maintain order and structure at both the cosmic and atomic levels.

Additionally, the mutual attraction between objects – planets to the Sun and electrons to the nucleus – underscores a fundamental aspect of how entities interact within these systems. However, it's important to note why I did not choose other potential similarities. While it might seem that other relationships could align, they don't quite capture the essence of what's represented in these models. For instance, some aspects might be technically true but aren't explicitly represented in the models we use, so they don't form part of our core understanding or teaching of these systems.

Ashok That's right, David.

By highlighting these relationships, textbooks aim to make the atomic structure comprehensible by drawing parallels with the solar system. This analogy not only aids in learning but also emphasizes the deep connections between different scientific domains, illustrating how similar principles can govern very different phenomena. This method not only enhances understanding but also highlights the elegance of the natural world, where such profound relationships exist across vastly different scales.

UNDERSTANDING ANALOGICAL MAPPING: THE CORRESPONDENCE PROBLEM

Analogical mapping involves a complex process known as the “correspondence problem.” This problem entails identifying which elements of the target problem align with elements in the source case, highlighting the connections that make the relationships in each scenario deeply similar.

At the heart of analogical mapping is determining correspondences that illuminate these relationships. For example, consider if we can establish that a radiation beam in a medical scenario corresponds to a rebel army in a historical battle scenario. Establishing such a correspondence allows us to align the target problem (medical treatment with radiation) with the source case (a military strategy), making the underlying strategies salient and applicable across domains.

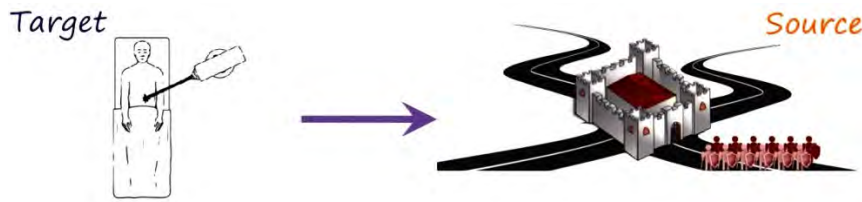


Figure 17-12 The correspondence problem involves mapping a solution to address a different problem.

- **Correspondence Challenges:** In any complex problem, multiple elements could potentially correspond between the target and the source. Ideally, a physician's use of radiation to target a tumor might be mapped to a general's strategy of deploying troops. However, this m-to-n mapping can become computationally intensive and less efficient as the number of potential correspondences increases.
- **Utilizing Higher-Order Relationships:** Effective analogical mapping goes beyond simple one-to-one correspondences (unary relationships) and examines more complex relational structures:
 - **Unary Relationships:** Basic level correspondences, like identifying similar roles or objects across scenarios (e.g., a patient and a king as central figures).
 - **Binary Relationships:** These involve two entities interacting directly, such as the interaction between a physician and radiation or a general and an army.
 - **Tertiary Relationships:** More complex interactions that involve an intermediary or an additional layer, such as the strategic placement of resources against barriers like healthy tissue or minefields.
- **Practical Decision-Making in Mapping:** In practice, deciding on correspondences involves prioritizing certain relationships over others, often focusing on higher-order relationships. For instance, when comparing a physician using radiation and a general using an army, we might prioritize the strategic use of resources against obstacles. Here, the radiation and the army both represent resources aimed at overcoming a barrier – healthy tissue in one case and minefields in another.
- **Deeper Connections:** We look beyond superficial similarities to deeper relational structures. While it might be tempting to draw direct parallels between visibly similar elements (e.g., comparing a patient to a king because both are central figures), more profound insights are gained by examining how these elements interact within their systems. For instance, the goal of a medical treatment (eliminating a tumor) mirrors the military objective (capturing or defeating a strategic target), with both strategies needing to navigate around significant obstacles.

This approach to analogical mapping not only helps in drawing useful parallels but also in applying known strategies from one domain to solve problems in another, fundamentally

enhancing the capability of AI systems to tackle new challenges by leveraging historical or known solutions.

EXERCISE: MAPPING DEEP RELATIONSHIPS - SOLAR SYSTEM AND ATOMIC STRUCTURE

As we investigate the intricacies of analogical mapping further, let's revisit the models of the solar system and the atomic structure to explore how these two can be mapped against each other.

Consider the given representations of the solar system and the atomic structure. The challenge is to determine how elements of the solar system correspond to those of the atomic structure, focusing on their relational dynamics.

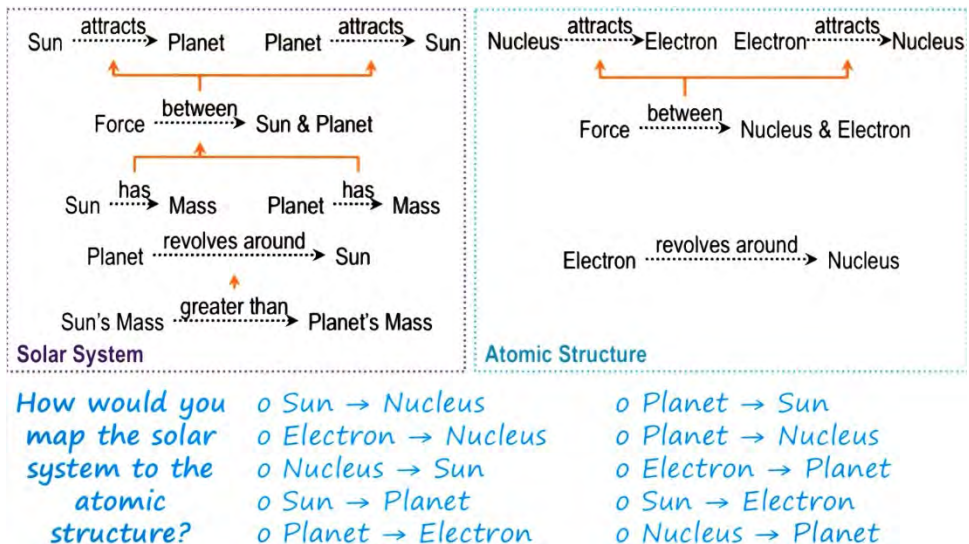


Figure 17-13 Analogical mapping between the solar system and the atomic structure.

Ashok David, how would you map the solar system to the atomic structure based on the representations provided?

David There are a few different ways we might approach this. Initially, if we only consider the top half of each diagram, it appears ambiguous whether the Sun should map to the nucleus or the electron, and the same ambiguity applies to the planet. However, when we examine the full context of both diagrams, it becomes evident

how these should align. The planet revolves around the Sun, just as the electron revolves around the nucleus. This makes it clear that the planet correlates with the electron and the Sun with the nucleus, as each pair engages in a similar relational dynamic of revolution and attraction.

Ashok That's a good observation, David.

David The key takeaway here is the depth of understanding required to make accurate correspondences.

Without a proper grasp of the fundamental relationships in the solar system and atomic structure models, accurate mapping is challenging. These models do more than visually represent; they encapsulate deep relationships that are critical for aligning and translating one system's behavior to another. This alignment is crucial for understanding the atomic structure through the lens of the solar system, emphasizing the importance of detailed and accurate models to facilitate effective analogical transfer.

Analogical Transfer: Applying Mapped Knowledge

Analogical transfer involves applying a solution or strategy from one domain (the source case) to solve a problem in a different domain (the target problem), leveraging the mappings established through analogical reasoning.

Let's revisit the target problem involving a physician treating a tumor with radiation, and the source case of a rebel army aiming to capture a fort without triggering mines. Through analogical mapping, we've established that the king in the source case corresponds to the tumor, and the rebel army corresponds to the radiation source in the target problem.

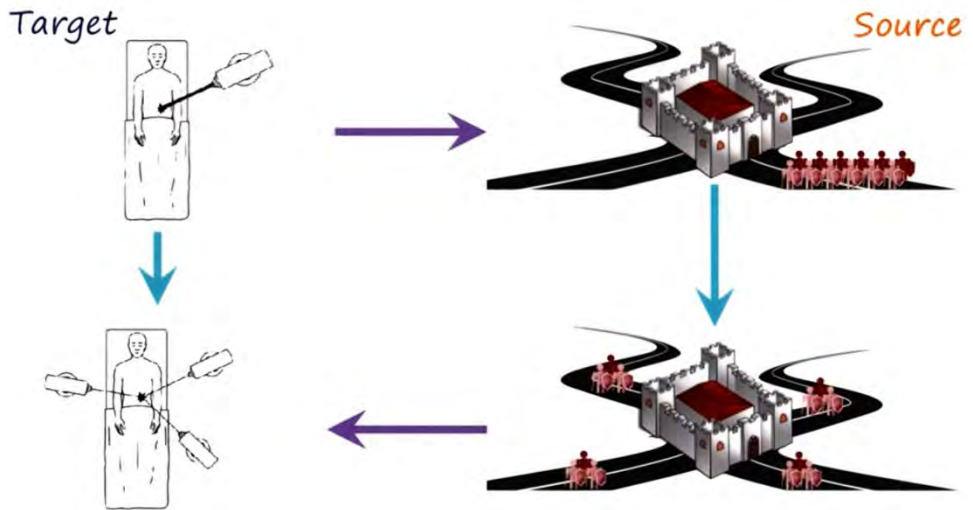


Figure 17-14 Analogical Transfer between the source problem and target solution.

In the source scenario, the rebel army's strategy was to divide into smaller groups to simultaneously approach the fort from different directions, thus avoiding triggering the mines. This approach allowed them to achieve their goal without activating the mines.

When we apply this strategy to the target problem, we recognize a parallel solution: the physician can divide the intense radiation source into several smaller, less intense beams. These beams can then be directed to converge on the tumor from different angles, minimizing damage to surrounding healthy tissues while effectively targeting the tumor.

The abstraction of the solution from the source case involves decomposing the resource (in this case, the radiation) into multiple components.⁷⁸ These are then coordinated to achieve the goal (destroying the tumor) while navigating the obstacle (the healthy tissue). This strategic approach is directly inspired by the rebel army's tactic of attacking the fort without setting off mines.

The underlying similarity here extends beyond superficial attributes, aligning deeply at the level of goals and strategies. Both cases involve a critical goal blocked by a potentially lethal obstacle, and both solutions require a careful, strategic distribution of resources to overcome these barriers. This highlights a key aspect of pragmatic similarity, where the overarching goals – capturing a fort and eradicating a tumor – drive the analogical transfer, reinforcing the strategy's applicability across diverse scenarios.

This example illustrates not only the effectiveness of analogical transfer in solving complex problems by borrowing strategies across domains but also underscores the importance of accurate mapping and deep understanding of both the source and target contexts to enable such transfers.¹⁸⁴⁻¹⁸⁷

Exercise: Analogical Transfer with the Solar System Model

In exploring how analogical reasoning aids in understanding complex concepts, we engage in an exercise that translates attributes from the solar system model to the atomic structure.

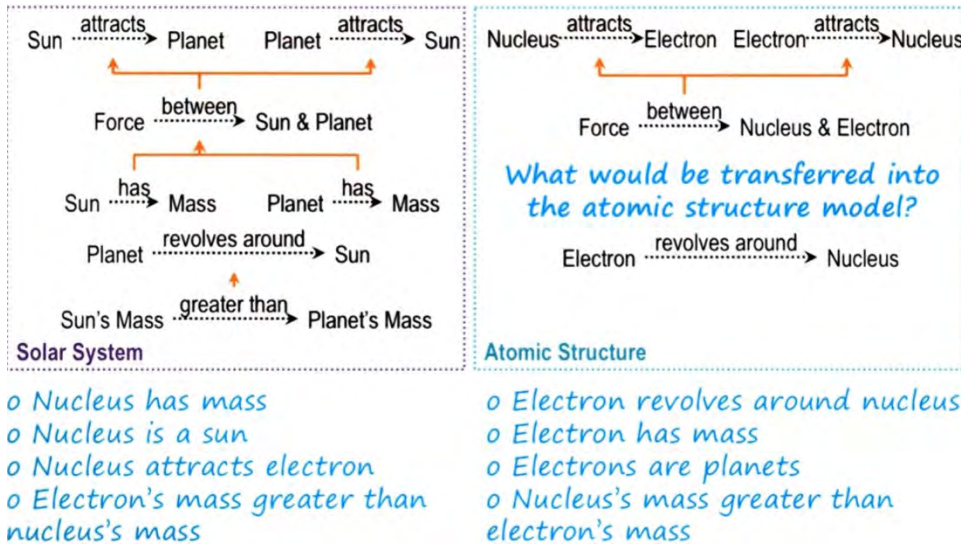


Figure 17-15 Analogical Transfer Solar System Example.

Ashok David, what was your answer?

David Well, I said that we transfer three things. First, we're going to transfer the idea that the force between nucleus and an electron is caused by the fact that the nucleus and electron have mass. We notice that the relationship here was similar to the relationships we have here. So, we're going to infer that the explanation is the same as well. So, the nucleus has mass and the electron has mass, and those are going to cause the force between the nucleus and the electron.

We're also going to transfer the idea that because the electron revolves around the nucleus, that means that the nucleus must have a greater mass than the electron. The planet revolves around the sun because the sun's mass was greater than the planet, so, the electron must revolve around the nucleus because the nucleus' mass is greater than the electron's.

We would not transfer the idea that the nucleus is a sun or that electrons are planets because those take the transfer too literally. We also wouldn't transfer the idea that the nucleus attracts electrons or that electrons revolve around the nucleus because those were already present in our original representation. So, we end up with this: The nucleus and the electron, both having mass, causes the force between them. And the nucleus's mass being greater than the electron's mass causes the electron to revolve around the nucleus.

Ashok That's a smart answer, David.

In this dialogue, David carefully chooses which elements to transfer based on their functional similarities and conceptual relevance, avoiding overly literal or redundant attributes. This discussion not only highlights the discernment needed in analogical transfer but also reinforces the educational value of understanding core scientific principles through comparative analysis.

THE ROLE OF STRUCTURAL SIMILARITY

In addressing structural similarity, we examine the nuanced relationship between the solar system model and the atomic structure. These two systems, though vastly different in their components and scales, provide a remarkable example of how structural similarity transcends apparent differences.

The solar system and the atomic structure do not share much in terms of semantic similarity. The elements that make up the solar system – planets, the sun, and other celestial bodies – are quite distinct from the subatomic particles like protons, neutrons, and electrons that form the atomic structure. Similarly, there is no direct pragmatic similarity; the solar system does not have an analogous 'goal' like the atomic structure might in terms of scientific exploration or understanding.

However, when we examine these systems through a structural lens, a different picture emerges. Both the solar system and the atomic structure can be represented as graphs where vertices (nodes) represent objects and their properties, and edges denote the relationships between these objects. For instance, in the solar system, the gravitational forces cause the planets to revolve around the sun, drawing a parallel to electrons orbiting the nucleus in an atom due to electrostatic forces.

This graphical representation underscores the structural similarity between the two models. The relationships, such as attraction and revolving dynamics in the solar system, align with the interactions within the atomic structure. Despite their disparate natures and the lack of tangible similarity in objects or goals, the fundamental relationships – how components interact with one another – remain strikingly similar.

By viewing these systems as graphs, we highlight the structural similarity. This approach is not concerned with the physical properties of the objects involved but rather with how the relationships are organized and function. Thus, the solar system model provides a valuable framework for understanding atomic interactions, not through direct comparison of components but through the mapping of their relational structures.

Structural similarity thus plays a crucial role in analogical reasoning, especially in scientific education and research, where it helps bridge conceptual gaps between different domains by highlighting how underlying relationships can inform one another, even in vastly different contexts.^{181,188}

Design by Analogy: Evaluation and Storage

In the application of analogical reasoning, the steps of evaluation and storage play pivotal roles, akin to their counterparts in case-based reasoning. This similarity underscores the essential processes that support the learning mechanisms of AI systems, particularly in how they handle new and complex problems.

Analogical reasoning does not inherently assure the correctness of its outcomes. Therefore, each solution proposed must undergo a thorough evaluation to verify its efficacy and safety. This evaluation often involves simulations that are critical in contexts such as medical treatments, where theoretical solutions must be tested in controlled environments before any real-world application. For instance, in Duncker's radiation problem, the suggested strategy of splitting a radiation source into multiple, less intense beams to target a tumor simultaneously from various angles needs to be simulated.¹⁷⁴ This simulation helps verify whether the beams can effectively treat the tumor without damaging surrounding healthy tissues.

Post-evaluation, if a solution proves successful, it is crucial to capture and store this knowledge. This involves encapsulating both the problem and its effective solution into a new case within the system's memory. Such storage serves not only as a record but also as a potential future resource. This archived case can later be retrieved and adapted to solve new, similar problems, enhancing the system's efficiency and adaptability over time.

For example, once the decomposed radiation beam approach is validated through simulation, it is stored as a new case. This case not only represents a specific solution but also embodies the strategic pattern that can be generalized to other scenarios requiring targeted intervention without collateral damage.

Each stored case contributes to the AI system's cumulative knowledge, allowing it to draw on past experiences to address future challenges. This process of learning from each solved case and storing the solution exemplifies incremental learning, where each step forward builds on the previous, gradually enhancing the system's problem-solving capabilities.

Through such structured evaluation and storage, AI systems not only improve in solving specific problems but also in their overall ability to reason, adapt, and apply learned strategies to a broader range of challenges. This ongoing cycle of analogical reasoning, evaluation, and

memory enrichment is foundational in the evolution of intelligent systems, making them capable of handling increasingly complex tasks with greater autonomy and precision.

Design by Analogy: Biomimicry in Modern Engineering

In exploring the application of analogical reasoning, particularly within design, one innovative approach stands out – biologically inspired design, also known as biomimicry. This design philosophy is driven by the quest for environmental sustainability and is further propelled by the pursuit of creativity, innovation, and groundbreaking design solutions.

A prime example of biomimicry is seen in the design of the Shinkansen 500 train, commonly referred to as the bullet train, due to its distinctive nose shape. The design of this train's nose was inspired by the beak of the Kingfisher bird. Originally, Japanese railway engineers faced a significant challenge: as faster trains traveled through tunnels, they produced loud shock waves, disturbing local communities. These shock waves occurred because the train moved from one medium (air) to another (the tunnel atmosphere) at high speeds.

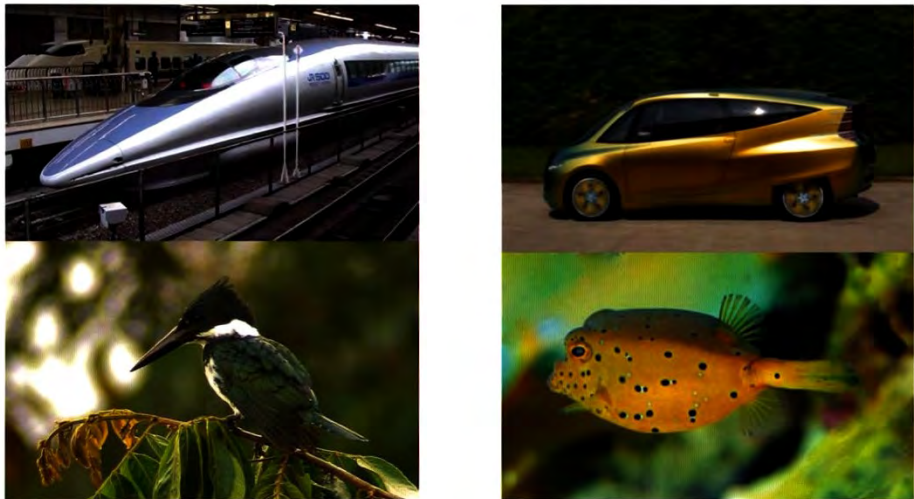


Figure 17-16 Examples of biomimicry in design by analogy.

The breakthrough came from observing the Kingfisher, which dives from air into water to catch prey without causing much splash. The bird's beak shape allows it to transition smoothly between two different mediums. This observation led engineers to redesign the train's nose to mimic the Kingfisher's beak, resulting in a design that not only reduced the noise by minimizing shock waves but also allowed the train to travel faster.¹⁸⁹

Another notable example is the Mercedes Benz concept car, whose design was inspired by the Boxfish. This car's design adopts the boxy yet aerodynamic shape of the fish, which is known for its efficient movement through water.¹⁹⁰ The car's structure reflects a similar efficiency in air, highlighting how solutions from biological entities can be transferred to solve engineering challenges.

Biomimicry involves identifying a target problem within the design space and then finding a natural system or organism that solves a similar problem through its biological functions. The essence of this process is analogical reasoning, where a solution from one domain (biology) is analogically transferred to another domain (engineering). This cross-domain transfer not only solves design problems but also fosters innovation by looking at nature's time-tested strategies and adaptations.

These instances demonstrate how biomimicry, underpinned by analogical reasoning, harnesses biological inspirations to address engineering challenges, leading to more sustainable and innovative design solutions. The field continues to expand as more designers and engineers turn to nature for inspiration, showing the profound impact of analogical thinking in practical applications.

ANALOGICAL DESIGN IN ROBOTICS: THE CHALLENGE OF WALKING ON WATER

Let us consider the theoretical challenge of designing a robot capable of walking on water. Nature itself provides numerous precedents, such as the basilisk lizard, renowned for its ability to sprint across water surfaces to evade predators or capture prey.

The process of designing such a robot hinges significantly on the principles of analogical design, where a profound understanding of both the source case – here, the locomotion of the basilisk lizard – and the target problem – the creation of a water-walking robot – is crucial. This approach mirrors the broader methodology of analogical reasoning where both analogical mapping and transfer are predicated on deep insights into the analogous relationship between the source and the target.

To systematically capture and utilize the basilisk lizard's capabilities in a robotic design, one might employ a structure-behavior-function model. While this discussion does not go into detailed visual representations, the model would typically articulate the lizard's initial and goal states to define its function, and detail the behaviors through a sequence of states and transitions that enable the lizard to achieve its function – walking on water. These behaviors and functions then guide the robotic design, ensuring that the robot mimics the lizard's efficient water locomotion mechanisms.

Designing a Robot for Aquatic Locomotion Through Compositional Analogy

In exploring the challenge of designing a robot capable of walking on water, we examine a process known as compositional analogy, leveraging both existing robotic designs and biological inspirations.

Initially, we consider a robot designed for terrestrial locomotion. This existing model, while effective on land, needs significant adaptations to navigate aquatic environments.

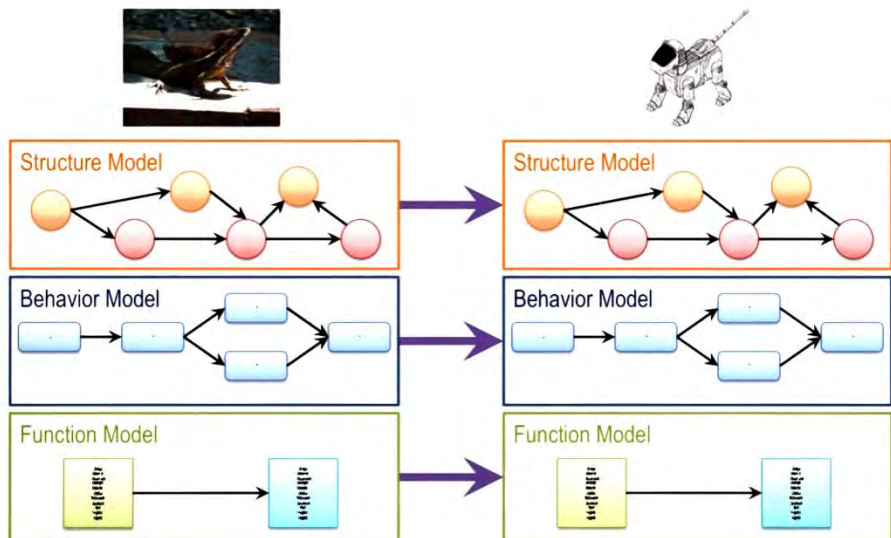


Figure 17-17 Design by Analogy: Transferring models from the basilisk lizard to the robot.

The discovery process begins by probing our case memory for relevant designs that could inform our new challenge. Intriguingly, we retrieve the design of the basilisk lizard, known for its ability to sprint across water surfaces. This organism's design is indexed by its remarkable functional model – its ability to walk on water – establishing a pragmatic similarity with our target problem.

Given the stark differences in the environments and the mechanisms involved, the task then is to align the structural, behavioral, and functional aspects of the basilisk lizard's model with our terrestrial robot's design. This alignment hinges on identifying and mapping similarities in their relational structures. For instance, although the robots and lizards are vastly different in form (objects), the relationships their structures support – necessary for movement – can be analogous.

The transfer process involves adapting specific features from the basilisk lizard, such as the unique shape of its feet, which are crucial for its water-walking ability, into the robotic design. This adaptation is not straightforward and requires a deep understanding of the lizard's locomotion mechanics. By translating these structural elements into the robot's design, we can then proceed to modify its behavioral model to mimic those of the lizard.

Finally, by integrating these adaptations, we refine the robot's function model, ensuring it aligns with the new capabilities required for water locomotion. This step-by-step process exemplifies how structural mapping leads to behavioral adjustments, which in turn refine the overall function of the design. This method not only enhances the robot's ability to perform its intended function – walking on water – but also expands our understanding of how different models can be integrated to solve complex design problems through analogical reasoning.

This example underscores the essence of analogical design: leveraging biological inspirations to solve complex engineering challenges, thereby fostering innovation through a cross-disciplinary synthesis of nature's proven strategies and modern technological capabilities.

INTEGRATIVE DESIGN USING COMPOUND ANALOGIES

In exploring the dynamics of analogical reasoning within biologically inspired design, we often encounter scenarios that deviate from the linear procedural model typically described by the sequence of retrieval, mapping, transfer, evaluation, and storage. Rather, the process exhibits multiple iterative loops where initial mappings can lead to transfers of information, which in turn may prompt further mappings and subsequent transfers.

Consider the design challenge of creating a robot capable of moving underwater stealthily. This task conjures images of the copepod – a small aquatic organism known for its ability to move with minimal wake, particularly at low speeds. This movement pattern is crucial for maintaining stealth, as low wake generation equates to a less detectable presence. At higher speeds, however, the copepod's wake increases, diminishing its stealth.

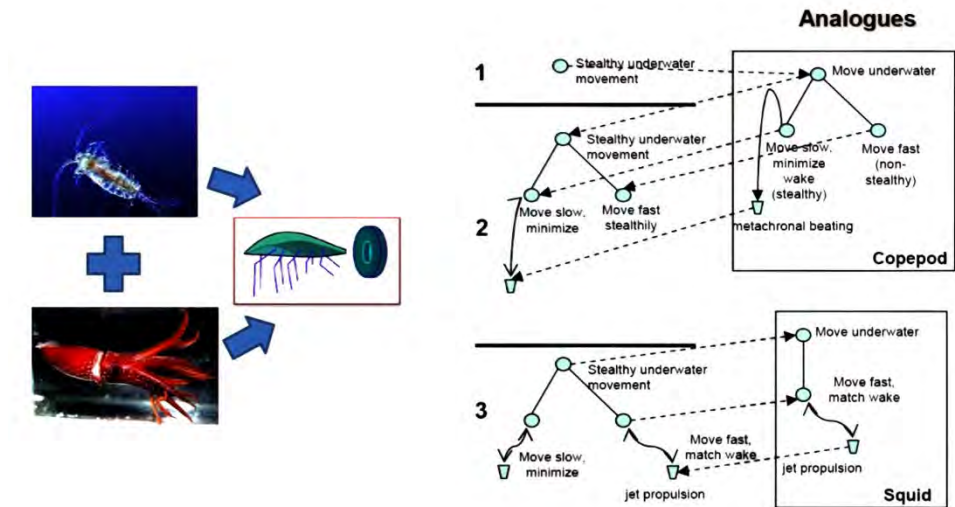


Figure 17-18 Compound analogies using iterations of design by analogy.

From this biological inspiration, we derive an initial design model for our robot, specifically tailored to operate at slow speeds with minimal wake, mirroring the copepod's locomotion. Yet, this solution only partially addresses our design challenge; we still need a mechanism for high-speed, stealthy movement.

This gap in the design leads us to consider another marine creature, the squid, renowned for its jet propulsion mechanism that allows for rapid, yet stealthy, underwater movement. This observation inspires a segment of our robot's design that incorporates a similar propulsion system, enabling effective stealth operation at higher velocities.

By combining elements from both the copepod and squid, we formulate a hybrid design. This approach – termed **compound analogy** – utilizes aspects of multiple source cases to address different facets of a complex problem. It represents a significant evolution from our original design problem, which began with the singular challenge of stealthy movement and expanded to include varying velocities.

This layered approach not only illustrates the non-linear nature of analogical reasoning in design but also highlights the process of problem evolution. As we refine our understanding of the challenge through iterative cycles of analogical retrieval, mapping, and transfer, new solutions emerge that may lead to further explorations and refinements.

Such a strategy underscores the dynamic interplay between retrieval and transfer, mapping and re-mapping, illustrating that the path to innovation is often as fluid and intricate as the aquatic environments we aim to navigate.

Refinement and Iteration in Analogical Reasoning

The iterative nature of analogical reasoning is particularly evident in the evaluation phase, which plays a crucial role in refining the design process. Evaluation methods can vary depending on the specific requirements and constraints of the project. For instance, in the case of designing a robot capable of walking on water, evaluations might include simulations or constructing physical prototypes to test the practicality of the design.

Successful evaluations validate the design, allowing it to be encapsulated as a new case within the case memory for future reference. Conversely, if the evaluation indicates failures or shortcomings, it necessitates a return to earlier stages of the analogical reasoning process. This could involve adjusting the knowledge transferred, realigning the mapping between the source and target, or even retrieving an entirely new source case.

For example, if during evaluation it is discovered that the robot designed for walking on water is excessively heavy, this might prompt several possible adjustments:

- **Transfer Reevaluation:** Revisit the transfer stage to modify or swap out elements that contribute to the excessive weight.
- **Mapping Adjustment:** Realign the mapping to emphasize or de-emphasize certain traits or functionalities, potentially adopting features that ensure a lighter design.
- **Retrieval Revision:** Return to the retrieval phase to look for a new source case – perhaps a different biological organism known for its lightweight structure that better meets the design requirements.

This dynamic revisiting of the stages underscores that analogical reasoning is inherently non-linear and often requires multiple iterations to refine the solution to a problem effectively. Each iteration brings us closer to a viable solution, enhancing our understanding of both the problem at hand and the potential solutions offered by nature or other domains.

Biologically inspired design remains a fertile ground for innovation, continually pushing the boundaries of what is achievable by integrating lessons from the natural world into technological applications. Through this process, analogical reasoning not only supports the development of novel solutions but also contributes significantly to our broader understanding of complex design principles.

ADVANCED OPEN ISSUES IN ANALOGY RESEARCH

Analogy, a cornerstone of human thought and a powerful tool in both AI and cognitive science, presents several advanced and still open research questions that push the boundaries of how we understand and implement analogical reasoning.

Vocabulary Uniformity Across Domains: One major question in analogical reasoning involves the necessity of a common vocabulary across different domains to facilitate effective transfer. For instance, when discussing the solar system and atomic structure, the terms “revolve” and “orbit” might be used to describe similar actions in different contexts. This raises

the issue: if different terminologies are used, how can effective alignment between these domains be achieved? Is a uniform vocabulary essential, or are there alternative methods to bridge the linguistic gaps?

Problem Abstraction and Transformation: Analogical reasoning inherently involves abstraction and transformation of problems. Traditional discussions often assume a static problem that finds its solution via transfer from a retrieved source case. However, in practice, problems are dynamic; they often need to be abstracted or transformed for effective source case retrieval. This dynamic nature of problem handling challenges the static view and requires more sophisticated mechanisms for abstraction and transformation in analogical reasoning.¹⁹¹

Compound and Compositional Analogies: The discussion often focuses on simple analogies – transferring knowledge from a single source to a single target. However, real-world problems frequently require drawing from multiple sources simultaneously. For example, designing a car might involve borrowing the engine design from one model and the chassis from another – a scenario known as compound analogy. Compositional analogy takes this further by applying analogies at multiple levels of abstraction. For instance, in comparing two organizations, one might align analogies at the levels of personnel, processes, and organizational structures. These layered analogies pose significant challenges: How do we effectively manage these multi-layered mappings and transfers, especially in AI systems?

Visuospatial Analogies: While many analogies discussed are causal, relying on explicit cause-and-effect relationships, a vast array of analogies operate primarily on visuospatial relations, where causality is implicit or irrelevant. These visuospatial analogies are crucial in fields such as architecture and art, where the spatial arrangement holds more significance than causal dynamics.^{184,192}

Conceptual Combination: Conceptual combination represents another advanced facet of analogical reasoning. It involves creating new concepts by merging elements from known concepts, akin to hybridizing features from two distinct domains to synthesize a new understanding. An example would be integrating certain aspects of the solar system's dynamics into the atomic model to generate a novel conceptualization of atomic structures.

These open issues highlight the complexity and depth of analogical reasoning as a research area. Each issue not only underscores the challenges but also the immense potential of analogy to drive forward understanding and innovation across various domains of knowledge and technology. The continuous exploration of these problems will likely yield crucial insights into both human cognitive processes and the development of more sophisticated AI systems.

ANALOGICAL REASONING AND HUMAN COGNITION

Analogical reasoning is widely regarded as a fundamental cognitive process, deeply interwoven with the way humans think, reason, and understand the world. This cognitive connection is evident in everyday language, problem-solving, and our ability to learn and adapt to new situations.

Metaphors and Everyday Reasoning: Metaphors, a ubiquitous form of analogy, illustrate how analogy permeates everyday thinking. Consider the expression “We had grown very far apart” used to describe the end of a relationship. Here, “far apart” is not a literal spatial description but a metaphorical expression of emotional distance. This usage highlights how spatial concepts are often analogically applied to describe abstract ideas, helping us conceptualize and communicate complex emotions.

Shakespeare and the Power of Metaphorical Analogy: Shakespeare's famous line from “As You Like It,” “All the world's a stage, and all the men and women merely players,” serves as a profound example of metaphorical analogy in literature. Here, the world is likened to a stage, and life to a play, suggesting that our actions are performances and that various stages of life are akin to different acts in a play. This analogy enriches our understanding of human behavior and societal roles, showcasing the power of analogical reasoning in enriching literary themes and offering insights into human nature.¹⁹³

Analogical Reasoning in Intelligence Testing: The cognitive significance of analogical reasoning extends into intelligence testing, exemplified by the Raven's Progressive Matrices. This test evaluates a person's ability to discern patterns and solve problems through analogical and abstract reasoning. The test's reliance on analogy underscores the central role of analogical reasoning in assessments of cognitive function and intelligence.

These examples underline that analogy is not merely a linguistic or artistic device but a core component of cognition. Analogical reasoning enables us to bridge the gap between the familiar and the unfamiliar, facilitating learning and understanding across various contexts. It allows for the transfer of knowledge and concepts from one domain to another, helping us navigate and make sense of new situations. By drawing parallels between disparate domains, analogical reasoning fosters creative problem-solving and innovation, making it indispensable in both everyday cognition and complex intellectual endeavors.^{194,195}

INTEGRATING KBAI: ANALOGICAL REASONING IN THE LANDSCAPE OF MODERN AI

Analogical Reasoning, a core component of Knowledge-Based Artificial Intelligence (KBAI), offers unique strengths that can significantly enhance and complement modern AI approaches, particularly those dominated by data-intensive machine learning. While statistical models excel at pattern recognition from large datasets, analogical reasoning provides mechanisms for deeper understanding, innovation, and learning from sparse data.

- **Addressing Limitations of Purely Data-Driven Models:**
 - **Learning from Few Examples (Few-Shot Learning):** Unlike models requiring vast datasets, analogical reasoning can facilitate learning from one or a few examples by transferring knowledge from a known source to a new

target. This is invaluable in domains where data is scarce, expensive, or rapidly evolving.

- **Explainability and Interpretability:** Analogies provide intuitive explanations. When an AI system solves a problem by analogy, the source case and the mapping can make the reasoning process transparent. This contrasts with “black box” models and is crucial for building trust and understanding in AI decision-making.
- **Innovation and Creative Problem Solving:** Analogical reasoning is a cornerstone of human creativity. By drawing parallels between disparate domains (e.g., biomimicry), AI systems equipped with robust analogical capabilities can propose novel solutions that go beyond mere pattern extrapolation.
- **Handling Out-of-Distribution Data:** Purely statistical models can struggle when faced with inputs significantly different from their training data. Analogical reasoning, by focusing on relational structures and abstract principles, can offer a path to reason about novel situations by relating them to known analogous ones, even if superficially dissimilar.
- **Synergies with Modern Machine Learning:**
 - **Guiding Representation Learning:** Analogies can help define what constitutes a “good” representation. If a system can form meaningful analogies, its internal representations likely capture essential relational structures. This insight can guide the design of representation learning objectives in neural networks.
 - **Augmenting Training Data:** Analogies can be used to generate plausible new training examples or to transform existing data in meaningful ways, thereby enriching datasets for machine learning models.
 - **Hybrid Architectures:** The future likely involves hybrid AI systems that combine the strengths of KBAI (like analogical reasoning) with data-driven methods. For example, a neural network might identify potential source analogs from a large database, while a symbolic analogical reasoning engine performs the detailed mapping and transfer. Knowledge learned through analogy can also be compiled into forms usable by statistical learners, and vice-versa.
- **Incorporating Common-Sense and Abstract Knowledge:** Analogical reasoning inherently deals with abstract relationships and can be a powerful tool for encoding and applying common-sense knowledge. Many common-sense inferences rely on understanding analogous situations (e.g., “if X worked in situation A, something similar to X might work in situation B, which is similar to A”).

By integrating robust analogical reasoning capabilities, modern AI can move towards systems that are not only proficient at specific tasks but also more adaptable, creative,

explainable, and capable of reasoning in a more human-like fashion, especially when faced with novelty or limited information.

CHAPTER SUMMARY

In this chapter, we undertook a comprehensive exploration of analogical reasoning, a cognitive process integral to human thinking, problem-solving, and creativity. We began by examining the concept of similarity, fundamental to how analogies operate, noting the human facility for evaluating it and the challenge of replicating this in artificial agents. We explored indexing schemes like nearest neighbor and discrimination trees used for retrieving similar cases.

The process of analogical reasoning was broken down into key stages: retrieval of relevant knowledge from a source domain, mapping correspondences between source and target, and the crucial transfer phase where knowledge is applied to the unfamiliar target. Examples like Duncker's radiation problem and the solar system-atom analogy illustrated these processes, particularly highlighting cross-domain transfer. We differentiated between superficial and deep similarity, and explored semantic, pragmatic, and structural types of similarity, emphasizing their roles in forming robust analogies across a spectrum of similarity.

We discussed the evaluation of analogies for effectiveness and the storage of successful analogies for future use, enhancing learning. The chapter then focused on design by analogy, including biomimicry, showcasing how nature inspires innovative solutions to engineering challenges through analogical transfer from biological systems. Concepts like compositional and compound analogies demonstrated the complexity and iterative nature of design, often involving multiple source analogs and refinements.

The discussion extended to advanced open issues in analogy research, such as vocabulary uniformity, problem abstraction, multi-source analogies, visuospatial reasoning, and conceptual combination, indicating the richness of the field. We also affirmed the cognitive significance of analogy, evident in everyday metaphors and its role in intelligence.

Finally, we considered how analogical reasoning, as a KBAI principle, can enhance modern AI. By integrating knowledge-based approaches like analogical reasoning, AI systems can better address challenges in few-shot learning, improve explainability, foster creative problem-solving, and handle out-of-distribution data, suggesting a path towards more adaptable, robust, and human-like hybrid AI systems. This chapter underscored that analogical reasoning is a dynamic, iterative, and powerful tool, foundational to both human cognition and the advancement of artificial intelligence.

ADDITIONAL READINGS & REFERENCES

- 78 Mitchell, M. Abstraction and analogy-making in artificial intelligence. *Annals of the New York Academy of Sciences* **1505**, 79-101 (2021).
- 171 Kokinov, B. in *Artificial Intelligence V* 3-12 (Elsevier, 1992).

- 172 Brims, F. J. *et al.* A novel clinical prediction model for prognosis in malignant pleural
mesothelioma using decision tree analysis. *Journal of Thoracic Oncology* **11**, 573-582 (2016).
- 173 Goh, P. K. *et al.* A decision tree to identify children affected by prenatal alcohol
exposure. *The Journal of pediatrics* **177**, 121-127. e121 (2016).
- 174 Duncker, K. & Lees, L. S. On problem-solving. *Psychological monographs* **58**, i (1945).
- 175 Gick, M. L. & Holyoak, K. J. Analogical problem solving. *Cognitive psychology* **12**, 306-
355 (1980).
- 176 Goswami, U. Analogical reasoning and cognitive development. *Advances in child
development and behavior* **26**, 91-138 (1996).
- 177 Yang, L. *et al.* Cross-domain analogical reasoning ability links functional connectome
to creativity. *Thinking Skills and Creativity* **57**, 101808 (2025).
- 178 Leuzzi, F. & Ferilli, S. A multi-strategy approach to structural analogy making. *Journal
of Intelligent Information Systems* **50**, 1-28 (2018).
- 179 Gick, M. L. & Holyoak, K. J. Schema induction and analogical transfer. *Cognitive
psychology* **15**, 1-38 (1983).
- 180 Hall, R. P. Computational approaches to analogical reasoning: A comparative analysis.
Artificial intelligence **39**, 39-120 (1989).
- 181 Blanchette, I. & Dunbar, K. How analogies are generated: The roles of structural and
superficial similarity. *Memory & cognition* **28**, 108-124 (2000).
- 182 Honda, H. & Hagiwara, M. Analogical reasoning with deep learning-based symbolic
processing. *IEEE Access* **9**, 121859-121870 (2021).
- 183 Macagno, F. The logical and pragmatic structure of arguments from analogy. *Logique et
analyse* **240**, 465-489 (2017).
- 184 Navarrete-Ulloa, J. A. & Trench, M. Transfer Across Episodes of Analogical
Reasoning: The Role of Visuo-Spatial Schemas. *Journal of Cognition* **8**, 4 (2025).
- 185 Pratama, A. & Rahmawati, S. Cognitive Models of Analogical Reasoning for
Knowledge Transfer and Generalization: Simulating Human-Like Knowledge
Acquisition. *International Journal of Applied Machine Learning* **2**, 22-40 (2022).
- 186 Leuzzi, F. & Ferilli, S. in *International Workshop on New Frontiers in Mining Complex Patterns*.
147-162 (Springer).
- 187 Macagno, F., Walton, D. & Tindale, C. W. Analogical reasoning and semantic rules of
inference. *Revue internationale de philosophie* **270**, 419-432 (2014).
- 188 Schockaert, S., Ibáñez-García, Y. & Gutiérrez-Basulto, V. A description logic for
analogical reasoning. *arXiv preprint arXiv:2105.04620* (2021).
- 189 Foo, C. T., Omar, B. & Taib, I. in *MATEC Web of Conferences*. 00019 (EDP Sciences).
- 190 Chowdhury, H. & Loganathan, B. in *Biomimicry for Aerospace* 211-235 (Elsevier, 2022).
- 191 Mostek, T., Forbus, K. D. & Meverden, C. in *AAAI/LAAI*. 323-329.
- 192 Yaner, P. W. & Goel, A. K. Visual analogy: Viewing analogical retrieval and mapping
as constraint satisfaction problems. *Applied Intelligence* **25**, 91-105 (2006).
- 193 Whall, H. M. "As You like It": The Play of Analogy. *The Huntington Library Quarterly*,
33-46 (1984).
- 194 Malkiński, M. & Mańdziuk, J. Deep learning methods for abstract visual reasoning: A
survey on raven's progressive matrices. *ACM Computing Surveys* **57**, 1-36 (2025).
- 195 Lovett, A. & Forbus, K. Modeling visual problem solving as analogical reasoning.
Psychological review **124**, 60 (2017).

Chapter 18

Generalization and Learning with Version Spaces

INTRODUCTION TO GENERALIZATION AND LEARNING WITH VERSION SPACES

In this chapter, we explore the fundamental concepts and practical applications of version spaces, a pivotal technique in the domain of artificial intelligence for addressing the challenges associated with generalization and learning. The core of version spaces lies in its ability to manage and refine conceptual understanding incrementally, as new examples are introduced one at a time – a process akin to human learning from sequential experiences.

Generalization and learning, particularly in the absence of a comprehensive background knowledge base, are critical issues in both artificial and natural cognitive systems. Unlike traditional learning models that rely on extensive pre-existing data, version spaces thrive in environments where information unfolds gradually, and learners must adapt without prior knowledge.¹⁹⁶ This mirrors real-world learning where often, information is incomplete, and learners must make educated guesses about new concepts.

This chapter begins by revisiting incremental concept learning, a precursor to understanding version spaces. In incremental concept learning, the sequence and structure of examples are typically controlled, and learners apply background knowledge to integrate new examples into

their existing frameworks. However, version spaces extend this by allowing learning to proceed even when examples are not pre-arranged in an optimal sequence and when background knowledge is minimal or absent.

We begin by defining version spaces – an abstract technique designed to refine and adapt models of concepts as new data becomes available. The essence of version spaces is its dual model system, where both specific and general conceptual models are adjusted dynamically as new examples are introduced. This adjustment occurs through a systematic algorithm which we will explore in detail, illustrating how new examples lead to either the specialization or generalization of the conceptual model.

Later, we also introduce an alternative methodology that complements the version spaces approach: organizing examples into decision trees or identification trees. This method shares some similarities with Incremental Discrimination Tree Learning discussed earlier in the context of Case-Based Reasoning.

This chapter aims to provide a comprehensive understanding of version spaces and their application in incremental learning scenarios, setting the stage for a deeper exploration of advanced AI techniques and their cognitive parallels. Through this discussion, we illuminate the challenges and solutions in crafting AI systems that learn and generalize effectively in dynamically changing environments.

VERSION SPACES AND INCREMENTAL CONCEPT LEARNING

The Version Spaces technique is a powerful method for learning concepts incrementally. This means it can handle a small set of examples arriving one at a time, refining its understanding as it encounters more information. To better understand this, we first revisit a similar approach known as Incremental Concept Learning.

Incremental Concept Learning Revisited

In Incremental Concept Learning, learning begins with a current definition of a concept, such as “arch.” As more examples are encountered, the concept’s definition is updated. Here’s how it works:

- A negative example arrives, one that is not an arch.
- The system refines its understanding by revising the concept’s definition to exclude the negative example. This process leads to **specialization**, narrowing the concept.

For example, look at the *Current Concept* in the image below. It shows the learner’s initial understanding of an arch, where the relationship between bricks includes “must-support” and “left-of.” When a negative example arrives – a configuration that is not an arch – this leads to a refinement of the concept. In this case, the bricks in the new example touch each other, an interaction missing from the original concept. This difference causes the learner to specialize the

concept by excluding such cases, leading to the *New Concept* in the second image, which reflects a revised understanding that excludes the new non-arch example.

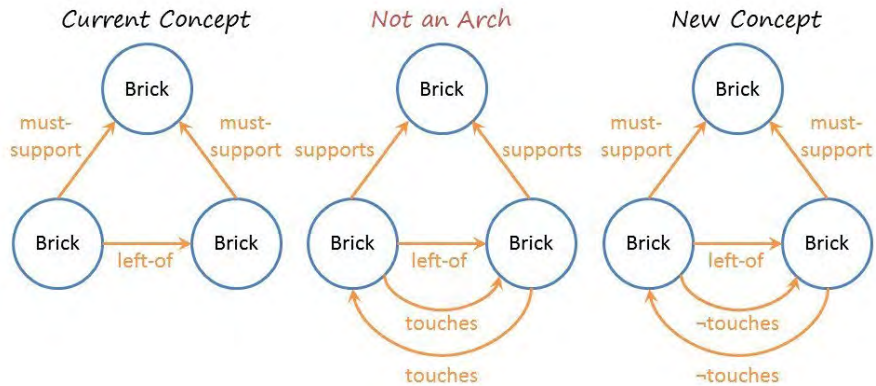


Figure 18-1 An example of Incremental Concept Learning from a new example.

In this method, negative examples help narrow or specialize the concept, while positive examples lead to generalization. The order in which these examples are presented plays a crucial role in learning. You generally want to begin with a positive example that lays the foundation for the concept, followed by both positive and negative examples to refine the understanding.

Additionally, each example should differ from the current concept by just one significant feature. This allows the learner to focus on the relevant change – whether to generalize or specialize – and adjust the concept accordingly.

For instance, in the example provided, the only important difference between the new negative example and the current concept is that the two bricks now touch. This difference is key because it helps the learner focus on exactly how to revise its understanding.

The technique is especially effective if a teacher can provide examples in an optimal order, guiding the learner through a structured process of generalization and specialization.¹⁵⁶ However, background knowledge can also play an important role in refining the learner’s understanding.

The Role of Background Knowledge in Incremental Learning

In Incremental Concept Learning, background knowledge helps guide how far a concept should be generalized. For instance, if the current concept includes “brick” or “wedge,” and the learner knows that both are types of blocks, this background knowledge allows the learner to generalize “brick” and “wedge” to the broader category of “block.”

Consider the second image, which shows how background knowledge can assist in generalization. In this case, if the learner understands that bricks and wedges are both types of blocks, it can generalize its concept to “block,” improving the flexibility and applicability of the concept.

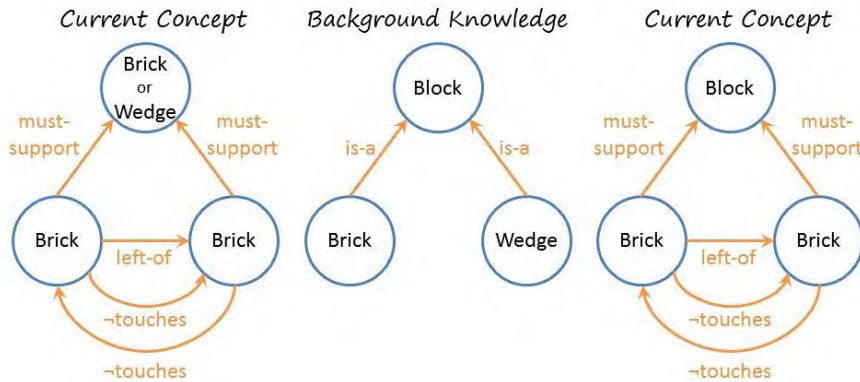


Figure 18-2 An example of background knowledge with Incremental Concept Learning.

This process works well when the examples are given in a structured order, and when background knowledge provides further guidance on how much to generalize.

Challenges in Generalization: Overgeneralization and Undergeneralization

One of the most significant challenges in learning is determining how far to generalize. Learners can face two opposite problems:

- **Overgeneralization:** This happens when a learner extends the concept too broadly. For example, if a child encounters a dog named Buddy – who is four-legged, furry, and black – and concludes that all four-legged, furry animals are dogs, they may mistakenly include other animals, such as cats, under the definition of “dog.” This broad overgeneralization leads to incorrect conclusions.
- **Undergeneralization:** In this case, the learner doesn’t extend the concept far enough. If the same child concludes that a dog is defined as only four-legged, furry, and black animals named Buddy, their definition of a dog is too narrow. While technically correct, this understanding won’t apply to most other dogs, making the definition less useful.

In Incremental Concept Learning, background knowledge helps guide the learner to avoid both under- and overgeneralization.¹⁹⁷ Additionally, the teacher's structured presentation of examples further helps the learner balance generalization and specialization.

Introducing Version Spaces for Learning Without Guidance

But what happens if neither a teacher nor background knowledge is available? This is where Version Spaces comes in.

Version Spaces offers a technique for learning when examples arrive in random order and background knowledge is unavailable. Unlike Incremental Concept Learning, which depends on structured examples, Version Spaces allows for learning through incremental refinement of concepts, even when examples are unordered.

The technique operates by maintaining two models of a concept:

- **The specific model:** The narrowest definition of the concept, which matches only one or very few examples.
- **The general model:** The broadest definition, which covers all possible examples.

As new examples arrive, the specific and general models are continually refined. For a positive example, the specific model is generalized to include the new example. For a negative example, the general model is specialized to exclude the unwanted example. This process continues until the two models converge, producing the correct level of abstraction for the concept.

This technique ensures that the learner converges on the right generalization under certain conditions, even without guidance from background knowledge or a teacher-provided example order. Over time, the specific and general models narrow down toward the most accurate representation of the concept.

Version Spaces provides a solution to the problem of learning without guidance. It allows learners to make sense of random examples by maintaining both a highly specific and a highly general version of the concept. Through the careful refinement of these two models, Version Spaces enables convergence on the correct level of generalization, helping the learner avoid both undergeneralization and overgeneralization.

This approach offers a valuable method for concept learning in a wide variety of scenarios, including cases where background knowledge is unavailable and the order of examples cannot be controlled.¹⁹⁸

VISUALIZING THE VERSION SPACE LEARNING PROCESS

In the Version Spaces technique for incremental concept learning, we maintain two models at all times: a specific model and a general model. These models evolve as we encounter new

examples, which are categorized as either positive or negative instances of the concept we are trying to learn.

Defining Specific and General Models

- **Specific Model:** The specific model is the most narrow or precise model of the concept, and it matches exactly one example. For instance, if we are learning the concept of a “dog,” our most specific model might describe a dog as “a four-legged, furry, black animal named Buddy.” This model would only apply to one very specific dog.
- **General Model:** In contrast, the general model is the broadest possible version of the concept. It encompasses everything, whether or not it fits the actual concept. In the same “dog” example, the general model could be as vague as “any animal is a dog,” which clearly includes far more than actual dogs.

Adjusting the Models: Generalizing and Specializing

Now, as we encounter new examples, we ask ourselves: is this a positive example (meaning it fits the concept we're learning) or a negative example (meaning it does not)?

- **Positive Example – Generalizing the Specific Model:** When a new positive example arrives, we generalize the specific model. For example, if our current specific model of a dog only includes “Buddy,” and now we see a different dog named “Scout,” which is blond and bigger than Buddy, we need to adjust. We generalize the specific model to cover this new instance, modifying it to say something like “a four-legged, furry animal.” This updated specific model now encompasses both Buddy and Scout.
- **Negative Example – Specializing the General Model:** On the other hand, when we encounter a negative example, like a cat, which is not a dog, we specialize the general model to exclude the wrong example. Our overly general model that previously defined a dog as “any animal” now needs to be adjusted. Perhaps we refine it to say “four-legged, furry animals that are not cats,” removing the cat from the potential definition of a dog.

The Iterative Learning Cycle

This process continues iteratively. With each new positive example, the specific model is generalized to include it, and with each new negative example, the general model is specialized

to exclude it. Over time, the specific model becomes less restrictive, while the general model becomes more refined.

The following steps provide more clarity:

1. **Start with the Most Specific and General Models:** As shown in Image 1, the specific model begins by matching only one very precise example, while the general model applies to everything.

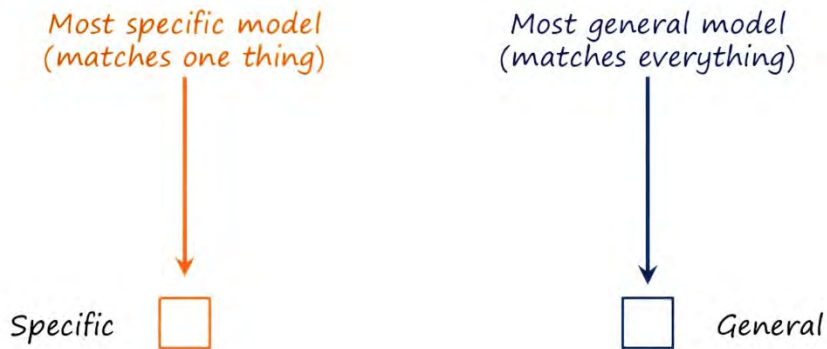


Figure 18-3 Start with the most specific and general models

2. **Generalizing and Specializing the Models:** Image 2 illustrates the adjustments we make when we receive positive and negative examples. Positive examples lead us to generalize the specific model (making it less restrictive), and negative examples prompt us to specialize the general model (making it more precise).

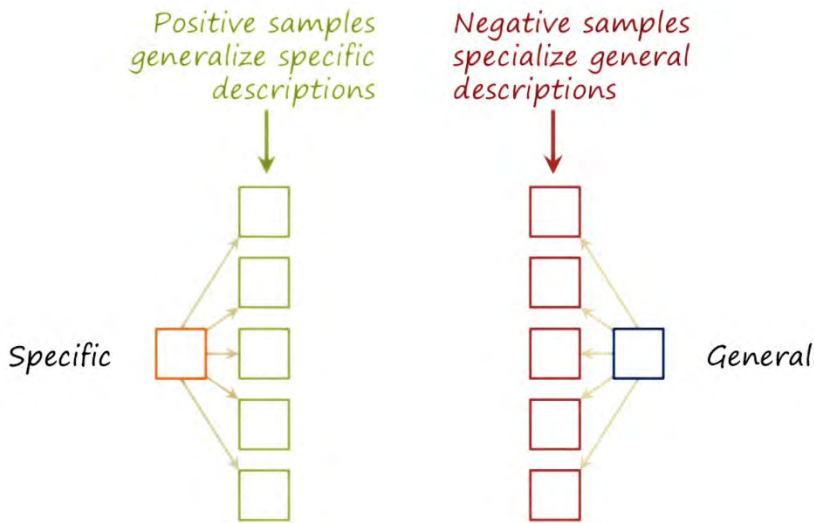


Figure 18-4 Generalize and specialize the model

3. **Pruning the Models:** As shown in Image 3, some of the generalizations or specializations we create might no longer match the current set of data. When this happens, we prune those models – essentially removing the incorrect or irrelevant models from our consideration.

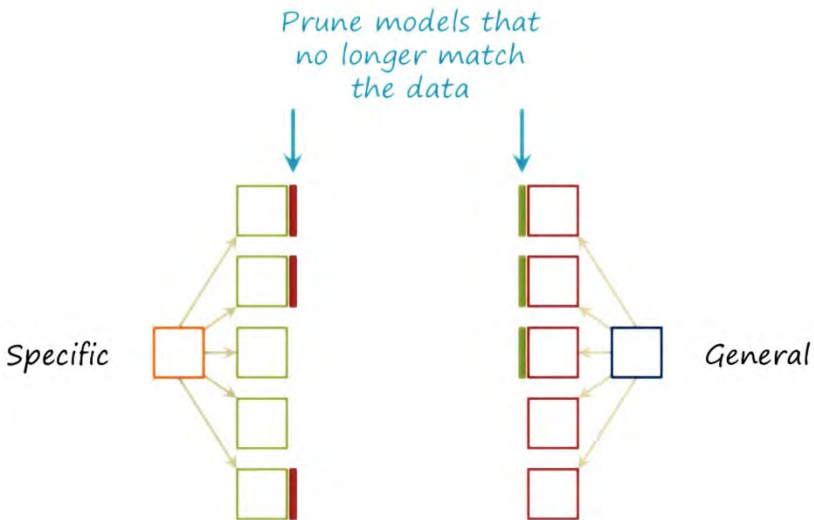


Figure 18-5 Prune the models

4. **Convergence:** Eventually, as more examples are processed, the two models – the specific and the general – move toward one another. Image 4 illustrates this final step of convergence. Once the two models merge, we have successfully learned the correct generalization of the concept.

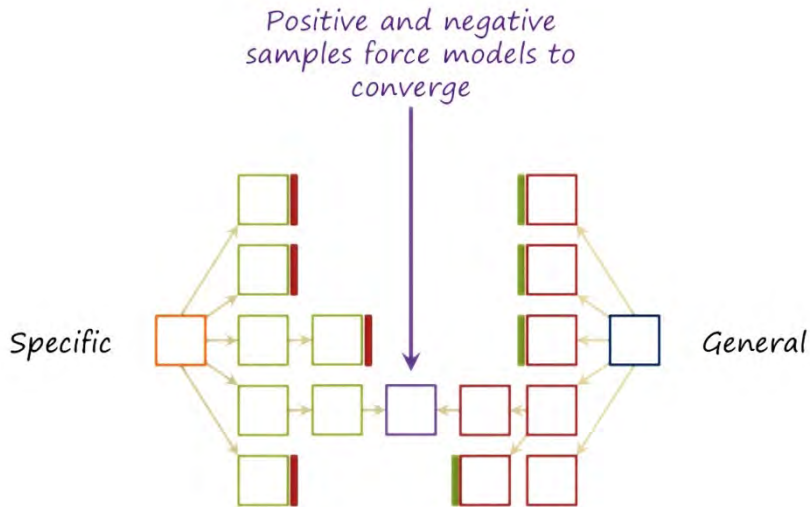


Figure 18-6 Converge the models

Convergence: Reaching the Final Concept Model

The goal of this process is convergence, where the specific and general models eventually meet. When the two models merge, they represent the correct concept, capturing all the positive examples and excluding all the negative ones.

At this stage, the two models no longer need further adjustment because they have reached the point where they fully describe the concept, given the set of examples we've processed. The final model is neither too specific (covering just one case) nor too general (encompassing unrelated examples).

The Version Spaces method ensures that we gradually and systematically refine our understanding of a concept through incremental learning. With each new piece of data – whether a positive or negative example – the specific and general models adjust, and we prune incorrect models to maintain a focused path toward the correct concept.

This method is highly effective in converging to an accurate understanding of the concept, even when the examples come in arbitrary order or without prior background knowledge. The process is resilient, as it forces the model to converge, guaranteeing a stable and reliable conclusion as long as we encounter enough examples.

COMPARING INCREMENTAL CONCEPT LEARNING AND VERSION SPACES: A VISUAL EXPLANATION

So far, we've discussed both incremental concept learning and version spaces in somewhat abstract terms. Now, let's use a more concrete illustration to understand the core differences and nuances between these two methods of learning.

Understanding Incremental Concept Learning Visually

In incremental concept learning, we begin with a model somewhere on a spectrum that ranges from specific to general (as shown in the first image). This spectrum helps visualize how concepts evolve based on the input examples we receive over time.

- **Specific Model:** The specific model is a narrow definition that represents a tightly constrained concept.
- **General Model:** The general model is much broader and includes many possibilities, some of which may not be correct.

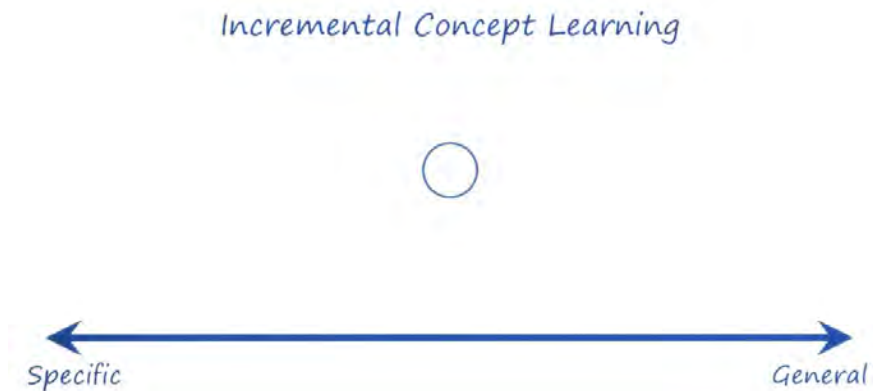


Figure 18-7 Incremental Concept Learning works with the placement of one model while learning.

In the initial stages, our concept might be quite specific, based on a limited number of examples. For instance, a child might encounter a single dog and conclude that a dog must be a black, four-legged animal named Buddy. As they encounter more examples, the model shifts along the spectrum from specific to general.

When we receive a new example, two things can happen:

- **Positive Example** (as depicted in the second image): If the new example isn't currently subsumed by our existing concept, we generalize the concept. For instance, if the child sees a bigger, blonde dog named Scout, they might adjust their concept of “dog” to include animals of different sizes and colors.

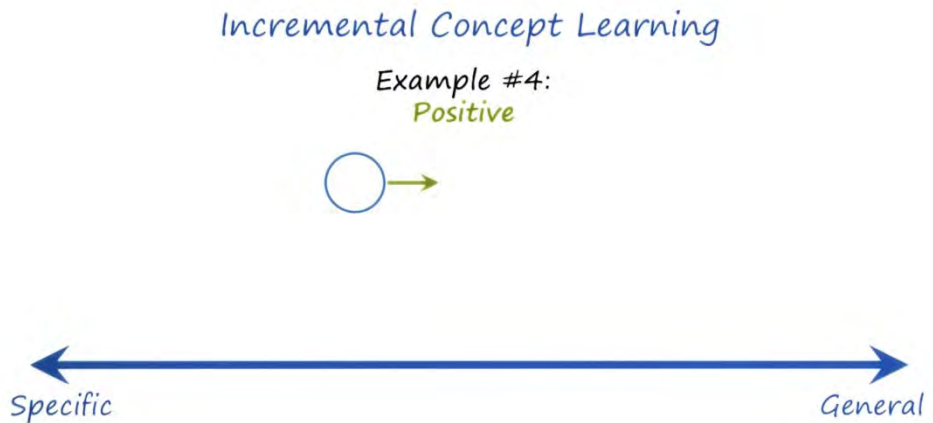


Figure 18-8 Incremental Concept Learning with a positive example.

- **Negative Example** (illustrated in the third image): If the new example is already included in the current concept but is actually a negative case, we specialize the concept to exclude it. For instance, if the child sees a cat and is told it's not a dog, they need to refine their understanding to avoid overgeneralization (i.e., not all furry, four-legged creatures are dogs).

Incremental Concept Learning

Example #6:
Negative



Figure 18-9 Incremental Concept Learning when a negative example.

This back-and-forth movement along the spectrum, adjusting based on both positive and negative examples, helps refine the concept over time. In the earlier image (Incremental Concept Learning), you see this dynamic process as the concept shifts between specific and general, eventually landing somewhere that represents a more accurate model of the concept.

Incremental Concept Learning



Figure 18-10 Incremental Concept Learning final concept

The Version Spaces Approach Visually

Now, version spaces take this process a step further by maintaining two simultaneous models at all times: one specific and one general (illustrated in the fourth image).

- **Specific Model** (on the left): This model represents the narrowest interpretation that matches exactly one instance. For example, the specific model for the child's concept of a dog might start with a black, furry animal named Buddy.
- **General Model (on the right)**: This model represents the broadest possible interpretation, which could encompass everything, such as all animals.



Figure 18-11 Version Spaces involves learning with multiple models until they converge.

Unlike incremental concept learning, which adjusts the concept incrementally, version spaces use both specific and general models simultaneously. As new examples come in, we apply these examples to both models, adjusting them accordingly.

Applying New Examples in the Version Spaces Framework

The process of updating the specific and general models works as follows:

- **Positive Example** (as shown in the image below): When we receive a positive example, we generalize the specific model. For instance, if the child sees a new dog named Scout, the specific model is expanded to include other dogs beyond just Buddy.

Version Spaces

Example #1:
Positive



Figure 18-12 Version Spaces - learning with a positive example.

- **Negative Example** (illustrated in the image below): If we encounter a negative example, such as a cat, we specialize the general model to exclude animals that don't fit the concept of a dog. For instance, the general model might now exclude animals with pointy ears or certain other characteristics that apply to cats.

Version Spaces

Example #2:
Negative



Figure 18-13 Version Spaces - learning with a negative example.

The Convergence Process in Version Spaces

The key innovation in version spaces is the convergence of the specific and general models over time. As more examples are processed, the specific model is gradually generalized, and the general model is gradually specialized. Eventually, these two models converge, forming a stable, accurate concept that works for all the examples encountered (shown below).

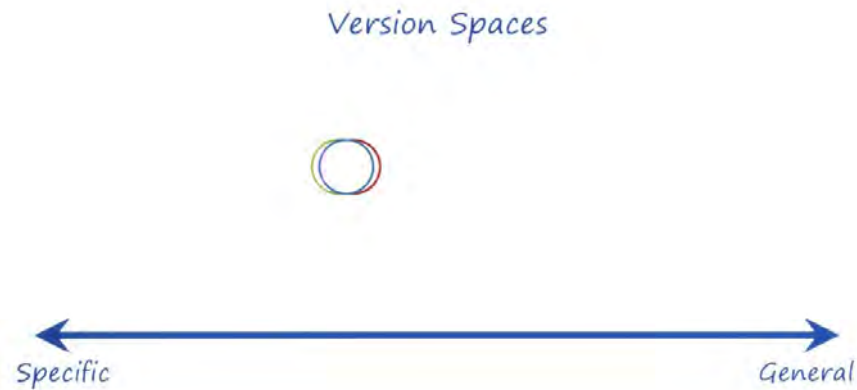


Figure 18-14 Version Spaces - Convergence of the internal models.

For example, the child's understanding of a dog starts very narrowly (e.g., Buddy, the black dog) but expands to include more dogs, such as Scout, without overgeneralizing to include cats. As both the specific and general models are refined, they merge into a well-defined concept of what constitutes a dog.

Models and Concepts: Clarifying Terminology

It's important to note that, in this context, we use the terms **model** and **concept** almost interchangeably. This is quite common in knowledge-based AI and cognitive learning. A model is essentially a representation of the world, which maps closely to the concept we are learning.

For instance, in earlier examples involving blocks, we discussed prototypical concepts like arches. A model of an arch is a representation that corresponds one-to-one with the real-world object. In this case, the concept of an arch and the model of an arch can be considered interchangeable because both serve as representations of the same idea.

Advantages of the Version Spaces Approach

The key advantage of version spaces over traditional incremental concept learning is that it helps learners converge to the correct level of generalization more efficiently. It provides a structure for handling both positive and negative examples without needing an intelligent teacher to present them in a specific order or rely heavily on prior background knowledge.

In fact, version spaces solve one of the major challenges in learning: how far to generalize. Whether the learner is undergeneralizing (creating a concept that’s too specific, like “a dog is a black, four-legged animal named Buddy”) or overgeneralizing (such as “all four-legged animals are dogs”), version spaces help find the right balance between the specific and general models.

By keeping both models and using new examples to refine them, learners can ensure that their concept of the world is both accurate and applicable to a wide range of situations.

ILLUSTRATIVE EXAMPLE: LEARNING ALLERGIC REACTION CONDITIONS WITH VERSION SPACES

Imagine I go to various restaurants, eat different meals, and occasionally experience allergic reactions. However, I don’t know under what specific conditions I get these allergic reactions. So, I ask an AI agent: “Dear AI agent, can you tell me under what conditions I get allergic reactions?” I provide the AI with the table below, which contains details about each restaurant visit – whether I had an allergic reaction, the restaurant name, meal type, day of the week, cost, and whether I had an allergic reaction or not.

Number	Restaurant	Meal	Day	Cost	Allergic Reaction?
Visit1	Sam’s	Breakfast	Friday	Cheap	Yes
Visit2	Kim’s	Lunch	Friday	Expensive	No
Visit3	Sam’s	Lunch	Saturday	Cheap	Yes
Visit4	Bob’s	Breakfast	Sunday	Cheap	No
Visit5	Sam’s	Breakfast	Sunday	Expensive	No

Figure 18-15 Dataset of meal details and allergic reactions.

Notice that the data contains only five examples, with a mix of positive and negative examples. This is important in knowledge-based AI (KBAI), which aims to learn concepts

Draft -Version:6/2/2025 3:06 PM

incrementally from small datasets, just like humans do. The positive and negative examples allow the AI to construct both generalizations and specializations to determine the conditions that trigger my allergic reactions.

Step 1: The First Example (Positive)

The AI first processes Visit 1, where I had breakfast at Sam's on Friday, with a cheap meal, and experienced an allergic reaction.



Figure 18-16 Version Spaces restaurant example - Visit 1

At this point, the AI constructs two models:

1. **Specific Model:** This model represents the exact scenario of the first visit. The specific model states:
 - o “I get allergic reactions when I eat breakfast at Sam’s on Friday, and the meal is cheap.”
 - o This is the most specific model possible, as it includes every detail of this example.
2. **General Model:** The general model, on the other hand, is the broadest possible hypothesis. It states:
 - o “I get allergic reactions regardless of where I eat, what meal I have, which day it is, or the cost.”
 - o This is the most general model, assuming that every condition leads to an allergic reaction.

These two models represent the extremes: the specific model accounts only for this exact instance, while the general model assumes that every situation results in an allergic reaction.

Step 2: The Second Example (Negative)

Next, the AI processes Visit 2, which is a negative example – I had lunch at Kim's on Friday, with an expensive meal, but did not have an allergic reaction.

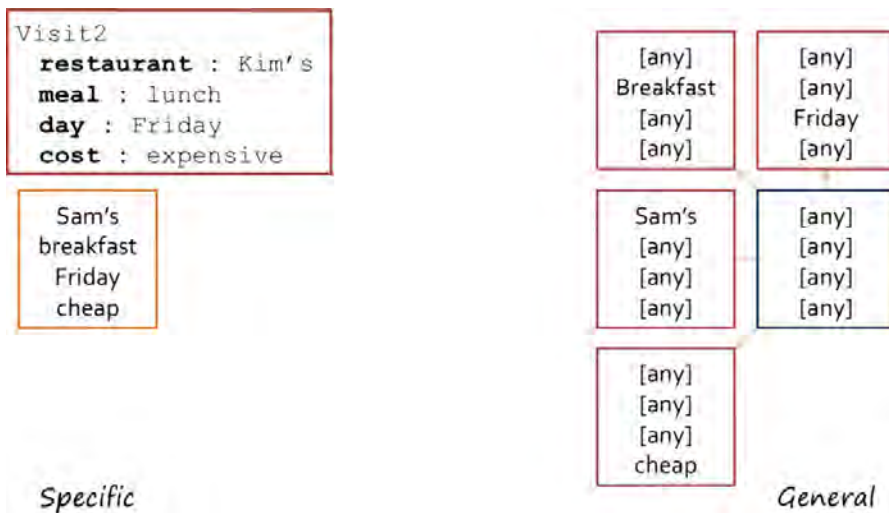


Figure 18-17 Version Spaces restaurant example - Visit 2

At this stage, the AI needs to specialize the general model (to rule out this negative example) and generalize the specific model (to expand its scope). Here's how this works:

- Specializing the General Model:** The AI needs to exclude this negative example by narrowing down the general model. It can do this in several ways:
 - Specify that the reaction happens at a particular restaurant (e.g., only at Sam's).
 - Specify that it only happens for a particular meal (e.g., only for breakfast).
 - Specify that it only happens on certain days (e.g., only on Friday).
 - Specify that it only happens for meals of a certain cost (e.g., only for cheap meals). Each of these specializations is a way of narrowing down the general model to exclude negative examples while maintaining the possibility that the positive examples (like Visit 1) still apply.
- Generalizing the Specific Model:** The AI must now consider how to make the specific model more flexible while still including the conditions of the positive example (Visit 1). It can do this by loosening constraints:

- For instance, generalizing the meal from “breakfast” to include all meals, or generalizing the restaurant to include other restaurants besides Sam’s.

This process leads to new candidate models that balance specificity and generality, but we must prune the ones that include the negative example. Since the negative example happened at Kim’s, the AI rules out any model that includes Kim’s or expensive meals, as I didn’t have a reaction there.

Step 3: The Third Example (Positive)

The AI now receives Visit 3, another positive example – I had lunch at Sam’s on Saturday, with a cheap meal, and experienced an allergic reaction.

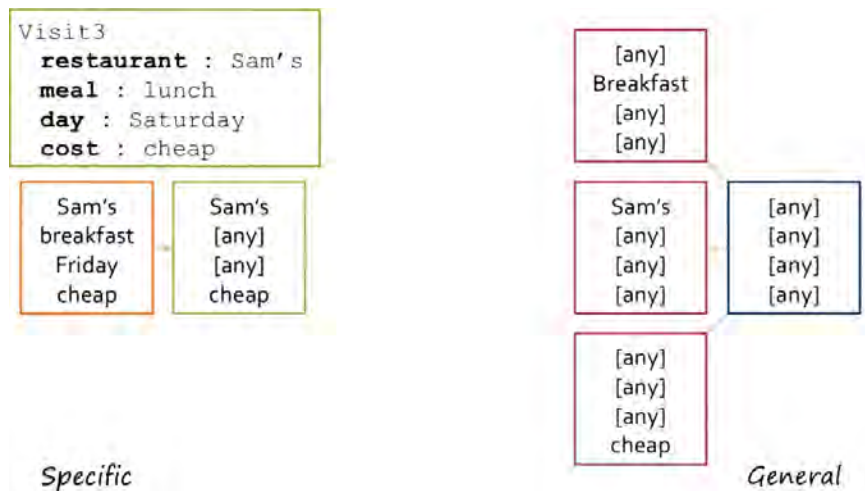


Figure 18-18 Version Spaces restaurant example - Visit 3

Since this is a positive example, the AI needs to generalize the specific model further to account for it. Here’s how it works:

- The previous specific model only included breakfast, but now it needs to generalize to cover both breakfast and lunch.
- The previous specific model only included Friday, but now it needs to generalize to any day, as both Friday and Saturday are now relevant.

At the same time, the AI needs to check the specialized general models to ensure they remain consistent with the new positive example. If a specialized model says I’m only allergic to breakfast, it must be pruned since lunch is now also a trigger for allergic reactions. Thus, the AI gradually refines the models, adjusting to new evidence while excluding contradictions.

Step 4: The Fourth Example (Negative)

The fourth example, Visit 4, is a negative example – I had breakfast at Bob’s on Sunday, with a cheap meal, and did not have an allergic reaction.

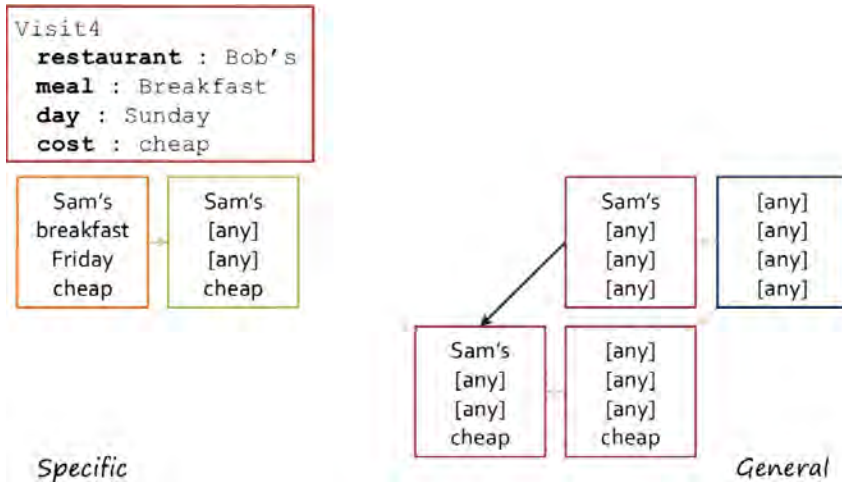


Figure 18-19 Version Spaces restaurant example - Visit 4

In this case, the AI must specialize the general model to exclude this scenario:

- It doesn't need to modify the restaurant part of the general model because Bob's is already excluded (we're focusing on Sam's).
- However, the AI must ensure that it excludes cheap meals at Bob's specifically, so it specializes the general model to exclude this case.

Additionally, the AI reviews the specific model to ensure it remains consistent with both positive and negative examples. Since the negative example occurred at Bob's and not Sam's, this case doesn't require further generalization.

Step 5: The Fifth Example (Negative)

Finally, the AI processes Visit 5, another negative example – I had breakfast at Sam's on Sunday, with an expensive meal, but did not have an allergic reaction.

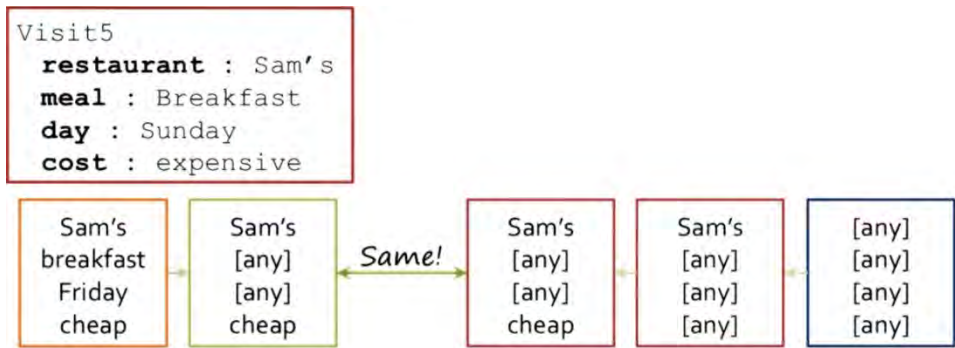
*Specific**General*

Figure 18-20 Version Spaces restaurant example - Convergence with Visit 5

This example helps the AI further refine its understanding:

- Since I didn't have a reaction to an expensive meal at Sam's, the AI specializes the general model further to clarify that only cheap meals at Sam's trigger allergic reactions.

At this point, the AI finds that the specific model and the general model converge. The AI now understands the conditions: I get allergic reactions only when I eat cheap meals at Sam's.

Interpreting the Learned Concept

The AI agent, through incremental learning, has determined that I experience allergic reactions only when I eat cheap meals at Sam's. This learning process mirrors how humans learn through both positive and negative examples, gradually adjusting and refining their understanding.

By processing each example one at a time and continually generalizing and specializing the models, the AI was able to hone in on the specific conditions that trigger the allergic reactions. The incremental learning approach allowed the AI to develop a correct understanding without needing a vast dataset – just like how humans learn from limited experience.

THE PRINCIPLE OF CONVERGENCE IN VERSION SPACES

The concept of convergence is central to learning in AI. Without convergence, a learning agent might continually oscillate between different models, zigzagging across the learning space without ever settling on a stable, correct concept characterization. This is why it is important to

ensure that the learning agent converges on a concept characterization and remains stable once it reaches that point.

The version space technique we've illustrated guarantees convergence, provided that there is a sufficient number of examples. In the specific case of our example, we required five examples to achieve convergence. Importantly, this convergence would have occurred regardless of the order in which the examples were presented, as long as all five examples were included.

This ability to converge, regardless of the order of the examples, is one of the greatest advantages of the version space technique. It provides a clear path to learning, even in situations where the examples are given in a disorganized or random order.¹⁹⁹

Learning Without Background Knowledge or Ordered Examples

Unlike other techniques, such as incremental concept learning, version space learning does not require background knowledge to guide the agent in its decision-making. This makes version space learning particularly powerful because it doesn't need a teacher or expert to provide clues about how to generalize or specialize. The learning happens purely by evaluating the examples.¹⁹⁷

Moreover, there is no assumption that the examples are provided in the correct order. This makes the version space method more flexible than techniques where the sequence of examples is important for guiding the learning process. In version space learning, the AI agent can learn effectively even when the examples are presented in a random sequence.

Handling Variability in Examples

A crucial difference between version space learning and incremental concept learning is how they deal with the differences between examples. In incremental concept learning, the agent is expected to process examples that differ by only one feature from the current concept. This allows the agent to focus on a single change, making learning more straightforward.

In contrast, version space learning can handle examples that differ in many features. This added flexibility allows the agent to process more complex scenarios where examples may have multiple varying elements.

For example, consider the first two visits in our restaurant scenario: the examples differ in several features (the restaurant name, the meal type, and the cost). Despite these multiple differences, the agent can still learn effectively from them, adjusting both its general and specific models accordingly.

The Version Space Algorithm Explained

Let's now take a closer look at the algorithm behind version spaces. This algorithm ensures that, as new examples are presented, the AI agent alternates between generalizing and specializing its models based on whether the example is positive or negative.

Algorithm for Version Spaces

For each example:

If the example is **positive**:

Generalize all **specific** models to include it

Prune away **general** models that cannot include it

If the example is **negative**:

Specialize all **general** models to include it

Prune away **specific** models that cannot include it

Prune away any models **subsumed** by other models

Figure 18-21 The Version Spaces Algorithm

1. **For each example:**
 - 0 **If the example is positive:**
 - Generalize all specific models to include the new positive example.
 - Prune away any general models that cannot account for the positive example.
 - 0 **If the example is negative:**
 - Specialize all general models to exclude the negative example.
 - Prune away any specific models that incorrectly include the negative example.
2. **Prune away models subsumed by other models:**
 - 0 This step is crucial to optimizing the learning process by removing redundant models that are already covered by more specific or more general models. This ensures that the agent focuses on the most relevant models.

This version space algorithm guarantees that the agent learns effectively, moving closer to the correct concept characterization with each new example. Over time, the search space narrows, and the learning agent converges on the best possible model, whether the examples are positive or negative.

A Note on Single vs. Multiple Pathways

In our specific implementation of the version space technique, there was a single pathway coming from the most specialized model. Because of this, we didn't need to prune away any specific models during the process. However, in more complex scenarios, multiple generalizations may arise from the most specialized models, and in such cases, pruning becomes essential to streamline the learning process and ensure efficiency.

In our restaurant allergy example, the agent ultimately converged on the correct concept characterization: I get allergies whenever I eat cheap meals at Sam's. The agent's ability to move between generalization and specialization, combined with pruning irrelevant models, allowed it to efficiently learn from the examples and converge on the correct answer.

Recap: Ensuring Convergence

This brings us back to the power of the version space technique and its guaranteed convergence. The algorithm works as follows:

1. **Positive Examples:** When the learning agent receives a positive example, it generalizes the specific model to include this example. Simultaneously, it prunes away any general models that cannot include the positive example.
2. **Negative Examples:** When the agent encounters a negative example, it specializes the general model to exclude this example. At the same time, it prunes away any specific models that incorrectly include the negative example.
3. **Pruning Models:** The pruning step ensures that models that are subsumed by other models (i.e., redundant models) are removed. This step is critical to streamlining the learning process and preventing unnecessary complexity in the agent's decision-making.

The version space algorithm we followed in our example guarantees that the learning agent converges on a stable and accurate concept, provided there are enough examples. In our case, the agent required five examples to identify the exact conditions under which allergic reactions occurred.

In summary, the version space technique offers a powerful method for incremental learning without the need for background knowledge or reliance on the order of examples. The agent processes both positive and negative examples, generalizing or specializing as needed and pruning away irrelevant models. Over time, this allows the agent to converge on an accurate, stable model of the concept it is learning, as illustrated in our restaurant allergy example.¹⁹⁹

INTERACTIVE EXAMPLE: LEARNING ALLERGIC REACTIONS WITH VERSION SPACES (EXTENDED)

In this exercise, we will build on what we learned earlier by exploring how version spaces help us narrow down possible hypotheses about an allergic reaction. We've added a new feature to the examples: vegan. Meals can either be vegan or not vegan. As we go through the exercise, we'll reason step-by-step to discover what exactly triggers the allergic reaction.

Let's start by looking at the first visit.

First Example: Initial Observation

Here's the first positive example, meaning there was an allergic reaction during this visit. This is indicated by the green outline.

- Restaurant: Kim's
- Meal: Breakfast
- Day: Friday
- Cost: Cheap
- Vegan: No

```
Visit1
restaurant : Kim's    meal : Breakfast    day : Friday
cost : Cheap         vegan : no
```


Figure 18-22 Version Spaces - Visit 1

Ashok David, can you tell me what you wrote down for the most specific and most general models based on this first example?

David Sure! The most specific model would be exactly what's observed in the example: "I'm allergic to breakfast at Kim's on Friday if it's cheap and not vegan." For the

most general model, it would be “I’m allergic to everything,” meaning I could have an allergic reaction no matter the restaurant, meal, day, cost, or whether it’s vegan or not.



Figure 18-23 - Version Spaces example - Visit 1 (continued)

Ashok Excellent, David. So, we now have both the most specific model, which is grounded on all the details from the first example, and the most general model, which covers all possible scenarios.

Second Example: Positive Example and Generalization

Let's look at a second positive example, meaning another instance where there was an allergic reaction:

- Restaurant: Kim's
- Meal: Lunch
- Day: Friday
- Cost: Cheap
- Vegan: No

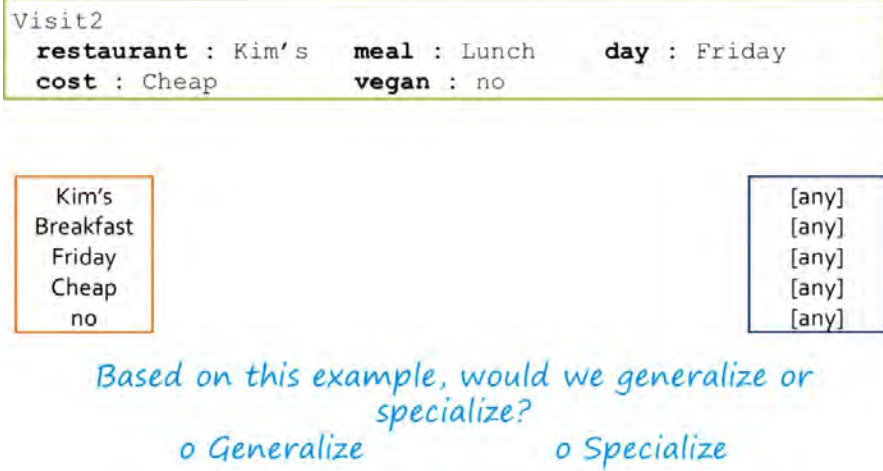


Figure 18-24 Version Spaces example - Visit 2

Ashok Based on this second example, David, would you generalize or specialize?

David Since this is another positive example, I'd generalize from the most specific model. We've got new information here – now it's lunch instead of breakfast – so I need to expand the specific model to cover both meals. I'll generalize by saying that "I'm allergic to any meal at Kim's on Friday, as long as it's cheap and not vegan."

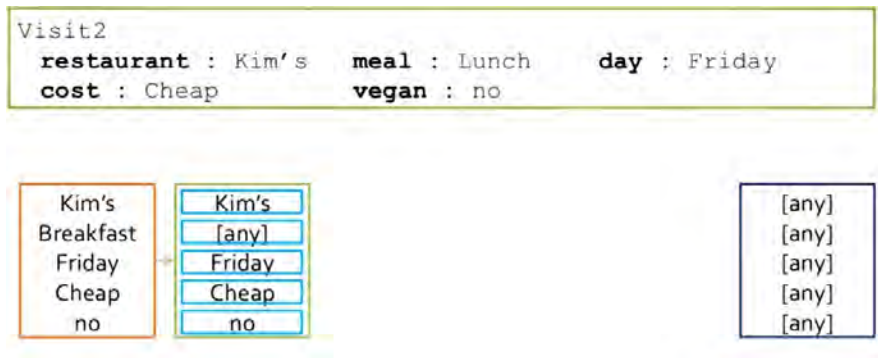


Figure 18-25 Version Spaces Example - Visit 2 (continued)

Ashok Well done. Notice how, for simplicity, you generalized the model to include any meal, rather than distinguishing between breakfast and lunch.

Third Example: Negative Example and Specialization

Now, let's consider a third example. This one is negative, meaning no allergic reaction occurred:

- Restaurant: Sam's
- Meal: Lunch
- Day: Saturday
- Cost: Cheap
- Vegan: No



Figure 18-26 Version Spaces examples - Visit 3

- Ashok** What would you do this time, David? Would you generalize or specialize?
- David** This time, we need to specialize the most general model because we know I didn't have an allergic reaction during this visit. This rules out some of the broader possibilities we considered earlier. I'll specialize by excluding scenarios that don't match the data.
- Ashok** Exactly. We'll specialize the most general model by pruning away any that don't fit the negative example. After the pruning, how many specializations are left, David?
- David** After pruning, I think we'll have three potential models left. One possibility is that I could be allergic to all meals at Kim's. Another is that I'm allergic to breakfast specifically. The last possibility is that I'm allergic to meals on Fridays. I would eliminate any models related to the cost or whether the meal is vegan because, in this example, I ate a cheap, non-vegan meal and didn't have an allergic reaction. So, those attributes can't be the cause.

Final Examples and Convergence

We've walked through the first three examples, but there are more visits we should consider:

Number	Meal	Meal	Day	Cost	Vegan	Reaction?
Visit1	Kim's	Breakfast	Friday	Cheap	No	Yes
Visit2	Kim's	Lunch	Friday	Cheap	No	Yes
Visit3	Sam's	Lunch	Saturday	Cheap	No	No
Visit4	Kim's	Breakfast	Sunday	Cheap	Yes	No
Visit5	Sam's	Breakfast	Sunday	Expensive	Yes	No
Visit6	Kim's	Lunch	Saturday	Cheap	No	Yes
Visit7	Kim's	Lunch	Monday	Expensive	No	No

What model did you converge on?

Kim's	[any]	[any]	Kim's	Kim's	[any]
[any]	[any]	Lunch	Lunch	[any]	[any]
[any]	[any]	Friday	Friday	[any]	[any]
Cheap	[any]	[any]	Cheap	Cheap	Cheap
[any]	[any]	No	No	No	No

Figure 18-27 Version Spaces examples - convergence

Ashok Now that you've seen all seven examples, which model do you think you converged on, David?

David I converged on the model that says, "I'm allergic to any cheap, non-vegan meal at Kim's." We had started with broader possibilities, but as more examples came in, we ruled out models based on further negative evidence. For instance, the fourth visit ruled out the idea that I'm allergic to all meals at Kim's or all cheap meals. I didn't have a reaction to a cheap, non-vegan breakfast on Sunday, which meant the cause isn't just any breakfast or any cheap meal. Later, visits also helped eliminate other possibilities. Visit 6 showed that my allergic reaction wasn't restricted to Fridays, as I also reacted to a meal on Saturday. This left the final model: "any cheap, non-vegan meal at Kim's."

Number	Meal	Meal	Day	Cost	Vegan	Reaction?
Visit1	Kim's	Breakfast	Friday	Cheap	No	Yes
Visit2	Kim's	Lunch	Friday	Cheap	No	Yes
Visit3	Sam's	Lunch	Saturday	Cheap	No	No
Visit4	Kim's	Breakfast	Sunday	Cheap	Yes	No
Visit5	Sam's	Breakfast	Sunday	Expensive	Yes	No
Visit6	Kim's	Lunch	Saturday	Cheap	No	Yes
Visit7	Kim's	Lunch	Monday	Expensive	No	No

What model did you converge on?

Kim's	[any]	[any]	Kim's	Kim's	[any]
[any]	[any]	Lunch	Lunch	[any]	[any]
[any]	[any]	Friday	Friday	[any]	[any]
Cheap	[any]	[any]	Cheap	Cheap	Cheap
[any]	[any]	No	No	No	No

Figure 18-28 Version Spaces examples - convergence (continued)

Ashok Nice work, David. It's important to note that we were able to do this by hand because there were only seven examples and five features. But what do you think would happen if there were many more examples and features?

David Well, if there were a larger number of examples and features, we'd need much more computing power to process all the data. However, the algorithm would still work, though it might take longer. In some cases, the algorithm might not be able to converge on a single model. For instance, if I were allergic to different meals at different restaurants – say breakfast at Kim's and lunch at Sam's – it might be harder to find a single cause. Even in those cases, though, the benefit of this algorithm is that it would show when convergence isn't possible, even after many examples.

Ashok Exactly, David. The algorithm allows us to systematically explore the version space until we converge on the most likely model or learn that there isn't a single explanation.

Through this exercise, we demonstrated how version space learning works by generalizing and specializing based on examples. With each step, we ruled out or expanded possible explanations, gradually converging on a model that explains the allergic reactions. By following this process, we ensure that the learning agent finds the most plausible hypothesis while pruning out those that don't fit the data. This approach helps in identifying a stable concept characterization, even as more examples are added.¹⁹⁹

ALTERNATIVE APPROACH: DECISION TREE LEARNING FOR CLASSIFICATION

In this section, we will explore a method known as decision tree learning, a powerful approach for processing data. This builds naturally on our earlier discussion of case-based learning and discrimination tree learning. In discrimination tree learning, the algorithm incrementally constructs a tree as new cases are presented. For each new case, it identifies a discriminating feature that sets the new case apart from existing ones. However, discrimination trees don't guarantee the optimality of the tree. This means that when a new problem arises, the retrieval process through the tree can be inefficient, as the tree may not be organized optimally for those cases.



Figure 18-29 Example of a discrimination tree

Decision tree learning addresses this inefficiency by offering a more structured and optimal approach, but with a tradeoff: all examples must be given at the start, unlike discrimination tree learning, which builds the tree incrementally.²⁰⁰

Constructing a Decision Tree: Feature Selection

Let's return to our restaurant allergy example to illustrate the decision tree construction process. We have five examples, and our goal is to build a decision tree that classifies these examples efficiently. To do this, we must select a feature – restaurant, meal, day, or cost – that will help us

separate the cases into categories containing either only positive (allergic) or only negative (non-allergic) reactions.

Number	Restaurant	Meal	Day	Cost	Allergic Reaction?
Visit1	Sam's	Breakfast	Friday	Cheap	Yes
Visit2	Kim's	Lunch	Friday	Expensive	No
Visit3	Sam's	Lunch	Saturday	Cheap	Yes
Visit4	Bob's	Breakfast	Sunday	Cheap	No
Visit5	Sam's	Breakfast	Sunday	Expensive	No

Figure 18-30 Decision Tree Learning dataset

Step 1: Start with the Feature “Restaurant”

Suppose we choose **restaurant** as the decisive feature. Here’s how the decision-making process plays out:

- The restaurants in our dataset are Kim’s, Bob’s, and Sam’s.
- Whenever the restaurant is Kim’s or Bob’s, there is no allergic reaction, as these are represented by negative examples.
- However, whenever the restaurant is Sam’s, the reactions vary: sometimes there is an allergic reaction (positive), and other times there isn’t (negative).

This initial split by restaurant is valuable because it effectively divides our examples into two groups:

- One group (Kim’s and Bob’s) contains only negative reactions.
- The other group (Sam’s) contains a mix of positive and negative reactions.

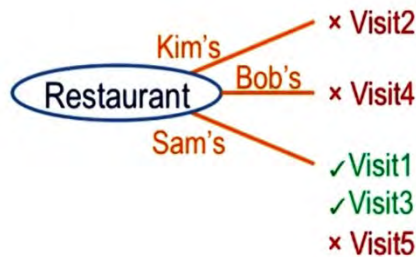


Figure 18-31 Decision Tree using restaurant as the split feature

Step 2: Refining with the Feature “Cost”

Now that we’ve separated the cases into those involving Sam’s (which have mixed reactions) and those involving other restaurants (which are all negative), we need to further refine the classification of the Sam’s examples. To do this, we can select another feature, such as **cost**, which helps distinguish positive and negative examples more clearly.

- When the cost is cheap, the examples show a positive allergic reaction.
- When the cost is expensive, the examples show a negative reaction.



Figure 18-32 Decision Tree using restaurant and cost as split features

By combining the features restaurant and cost, we’ve constructed a decision tree that classifies the examples with efficiency. This tree first examines the restaurant and, if the restaurant is Sam’s, it then checks the cost to predict whether the reaction will be positive or negative.

Efficiency and Optimality of Decision Trees

This decision tree is highly efficient because it allows us to classify new examples with minimal effort. For instance, if a new example comes in – Sam’s, lunch, Friday, expensive – we can quickly determine whether there will be an allergic reaction by simply traversing the tree from restaurant to cost. The tree efficiently leads us to the right classification without unnecessary steps.

This method of inductive decision tree learning is intuitive and provides a clear advantage over methods like discrimination tree learning, where the tree might not be as structured. Here, we quickly find the best feature splits for classification. The tradeoff, of course, is that we need to have all the examples upfront in order to create an optimal tree.

While this decision tree example seems simple, that’s largely because we are working with only five examples and four features. As the number of examples or features grows, decision tree learning becomes more complex. Deciding which feature to split on requires careful consideration, especially with large datasets.

Fortunately, machine learning algorithms have evolved to efficiently handle larger datasets. These algorithms automatically optimize the tree-building process, ensuring that decision trees can still classify data efficiently, even with many features and examples.²⁰¹

Illustrative Example: Decision Tree for Sunburn Prediction

Let’s take a closer look at another example of decision tree learning. This time, we have a dataset of people who go to the beach, where some of them get sunburned, while others don’t. In this dataset, there are nine examples, and each person is described by four features: hair color, height, age, and whether they used lotion. Our task is to construct an optimal decision tree to classify these examples effectively.



Figure 18-33 Decision Tree beach example

A good starting point would be to discriminate based on **hair color**. Hair color can divide the examples into three distinct categories: brown, red, and blonde. The interesting thing about this choice is that for people with brown hair, all of the cases are negative – none of them got sunburned. On the other hand, for people with red hair, every case is positive – they all got sunburned. This makes hair color a strong candidate for our first decision point, as it cleanly separates groups where some categories contain only negative or only positive instances.

However, the blonde-haired group isn't as straightforward. In this case, there are both positive and negative examples – some individuals with blonde hair got sunburned, while others did not. So, to further classify these examples, we need to apply another discriminating feature. **Lotion** could be our second feature.

When we use lotion as the next discriminating feature, it classifies the remaining blonde-haired examples into two groups: those who used lotion and those who did not. People who used lotion did not get sunburned, while those who did not use lotion got sunburned. Thus, we successfully classified the blonde-haired individuals using this feature.

By using just two features, hair color and lotion, we managed to classify all nine examples in a straightforward and efficient manner. However, it's important to note that decision trees can vary depending on the order of feature selection. This is another decision tree for the same dataset, but because we used a different order for the features, this tree turned out to be less optimal. For instance, had we first discriminated based on height, followed by hair color, and then age, we would have ended up with a much “bushier” tree – a more complex and less efficient classification.

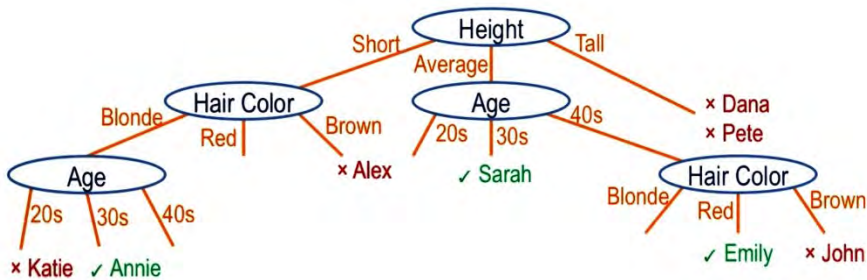


Figure 18-34 Decision tree beach example of a suboptimal tree

This brings us to an important point about the trade-offs in decision tree learning. Decision tree learning can lead to more optimal classification trees, but it requires all the examples upfront to build the best possible tree. On the other hand, in discrimination tree learning, as covered in case-based reasoning, the tree can be built incrementally, meaning new examples can be added over time. However, this approach often leads to suboptimal trees, as it doesn't always produce the most efficient structure.

Concluding Thoughts on Decision Tree Learning

Decision tree learning is an incredibly useful technique when you have access to all your data from the outset. It produces optimal trees that can classify new problems efficiently and with minimal effort. Unlike other methods that build trees incrementally or sub-optimally, decision tree learning provides a structured approach that ensures each split maximizes the separation of positive and negative examples. While the method works well with small datasets, it scales effectively with the help of modern algorithms, making it a valuable tool for large-scale data classification.

VERSION SPACES AND COGNITION

Cognitive agents, like learning algorithms, encounter the critical problem of determining how broadly or narrowly to generalize from their experiences. This issue is pivotal in learning, as striking the right balance between undergeneralization and overgeneralization is essential for forming useful and accurate knowledge.

When an agent undergeneralizes, the knowledge it acquires is too specific to be applied to future scenarios. For example, suppose I'm a Martian visiting Earth and I encounter my first human being. If I undergeneralize, I might conclude that this specific human has two arms, but

that doesn't help me understand anything about other humans. My conclusion is too narrow to be useful beyond this one observation.

On the other hand, if I overgeneralize, I might conclude that every human on Earth has two arms. While this may apply to most humans, it overlooks exceptions, such as people with fewer or more limbs. This generalization would be inaccurate because it fails to account for diversity and exceptions in the population.

Version Spaces is a learning technique designed to help agents navigate this balance, allowing them to converge toward the right level of abstraction. It systematically explores multiple possible concept definitions, progressively narrowing down the space of hypotheses. By pruning incorrect or overly specific models, the agent moves closer to the most accurate generalization possible. This process prevents both overgeneralization and undergeneralization by maintaining flexibility and pruning away what doesn't fit.

The ability of an agent to entertain multiple hypotheses simultaneously in this way is closely related to the notion of cognitive flexibility. Just as version spaces maintain multiple possible concept definitions that converge over time, cognitive agents can flexibly switch between different perspectives or understandings of a concept. This ability to adjust one's thinking, or to entertain various possibilities before settling on a solution, is crucial for effective problem-solving and learning.

Another way to approach learning is to make a single generalization and then test it in the real world. In this method, the agent relies on trial and error, where it begins with a hypothesis and adjusts that hypothesis based on feedback from its successes or mistakes. If a generalization fails, the agent learns by modifying the generalization or forming a new one. This is a feedback-driven learning process, where every mistake refines the agent's understanding. While this method is effective, it doesn't have the built-in flexibility that version spaces offer, which consider multiple possibilities simultaneously and systematically prune them based on new information.

The two methods – systematic convergence using version spaces and trial-and-error learning through feedback – both underscore the importance of balancing exploration and refinement in cognitive agents. In both approaches, the agent works to avoid premature conclusions while remaining open to learning from new information. Version spaces, with their structured approach to hypothesis refinement, provide a model for how cognitive agents can systematically converge on accurate knowledge without becoming too narrow or too broad in their conclusions.

In essence, version spaces offer more than a machine learning technique – they model the cognitive processes humans use to learn and adapt. By maintaining multiple hypotheses and converging through experience, both learning algorithms and human cognition can avoid the extremes of overgeneralization and undergeneralization, refining their understanding of the world incrementally and flexibly.²⁰²⁻²⁰⁴

INTEGRATING KBAI: GENERALIZATION AND VERSION SPACES IN MODERN AI

Generalization, the ability to apply learned knowledge to new, unseen situations, is a cornerstone of intelligence. Version Spaces, as a KBAI technique, offer a systematic approach to generalization by explicitly representing and refining the space of all plausible hypotheses consistent with observed examples. This principled method of hypothesis space search provides valuable insights and capabilities that can complement and enhance contemporary AI, particularly data-driven machine learning approaches.

- **Enhancing Learning from Limited or Incremental Data:**
 - **Few-Shot and Incremental Learning:** Version Spaces are inherently designed to learn from a small number of examples and to update hypotheses incrementally as new data arrives. This contrasts sharply with many deep learning models that require vast datasets for training. Integrating Version Space principles could lead to more data-efficient learning systems.
 - **Structured Concept Acquisition:** The explicit maintenance of Specific (S) and General (G) sets provides a structured way to acquire concepts, which can be particularly useful in domains where data is sparse or expensive to obtain.
- **Improving Transparency and Explainability:**
 - **Interpretable Learning Process:** The state of the S and G sets at any point provides a clear snapshot of what the learner knows and the range of consistent hypotheses. The process of specializing G and generalizing S in response to examples is a transparent reasoning mechanism, offering explainability often lacking in “black box” models.
 - **Identifying Ambiguity:** When the S and G sets do not converge to a single hypothesis, the Version Space explicitly indicates ambiguity or that more data is needed to define the concept uniquely. This is valuable diagnostic information.
- **Synergies with Modern Machine Learning Techniques:**
 - **Guiding Hypothesis Search in Complex Models:** While Version Spaces are typically applied to simpler hypothesis languages, the core idea of maintaining boundaries of consistent hypotheses could inspire methods to guide or constrain the search in more complex model spaces, such as those used in deep learning.
 - **Active Learning:** The state of the Version Space can inform active learning strategies. Queries could be chosen to maximally reduce the size of the version space (i.e., the gap between S and G sets), leading to more efficient learning.

- **Hybrid Architectures:** Version Space learning could act as a symbolic reasoning layer in a hybrid system. For example, a neural network might extract features or propose candidate hypotheses, which are then refined and validated by a Version Space-like mechanism.
- **Addressing Robustness and Consistency:**
 - **Logical Consistency:** Version Spaces strive for logical consistency with all observed (noise-free) examples. While the standard algorithm assumes no noise, extensions can handle noisy data, and the core principle of maintaining consistent hypotheses is valuable for robust learning.
 - **Foundation for More Complex Reasoning:** The systematic exploration of a hypothesis space is a foundational element for more advanced reasoning, including abductive reasoning or learning relational concepts.
- **Formalizing Generalization:**
 - **Explicit Bias:** Version Spaces make the learning bias (the choice of hypothesis space) explicit. Understanding this bias is crucial for understanding what a system can and cannot learn. This contrasts with the often implicit biases in complex statistical models.

While classic Version Space algorithms have limitations (e.g., computational cost with large hypothesis spaces, sensitivity to noise in their basic form), their conceptual framework for generalization through bounded, incremental hypothesis refinement offers enduring value. Integrating these KBAI principles can lead to AI systems that are not only powerful pattern recognizers but also more data-efficient, transparent, and robust learners capable of acquiring well-defined concepts.

CHAPTER SUMMARY

In this chapter, we explored the concept of version spaces, a method for systematically refining our understanding of a concept, even in the absence of prior knowledge or guidance from a teacher. Version spaces serve as a framework for narrowing down hypotheses through the careful balance of generalization and specialization, ultimately converging on a precise characterization of the concept. We began by contrasting this with incremental concept learning, highlighting the role of background knowledge and the challenges of over- and undergeneralization.

We detailed the core mechanism of version spaces: maintaining both a maximally specific model (S-set) and a maximally general model (G-set). Through exposure to positive and negative examples, the algorithm generalizes the S-set and specializes the G-set, pruning hypotheses that become inconsistent. This iterative process ensures that what is learned remains both accurate and broadly applicable, ideally leading to convergence where S and G sets meet. The chapter illustrated this with concrete examples, including learning conditions for allergic reactions, and formally presented the Version Space algorithm.

The versatility of version spaces enables them to handle more complex learning problems, as they support flexible concept formation without requiring examples in a specific order or differing by only one feature. We connected this to the broader notion of cognitive flexibility, where multiple perspectives on the same concept are maintained and refined simultaneously.

We also introduced decision tree learning as an alternative method for classification, particularly when all data is available upfront, offering a way to build optimal trees for efficient decision-making, contrasting it with the incremental nature of discrimination trees.

The discussion then considered the relationship between version spaces and human cognition, particularly how they model the process of navigating generalization challenges. Finally, we examined how the principles of version spaces and structured generalization can be integrated with modern AI, enhancing data efficiency, explainability, and robustness in machine learning systems. This sets a strong foundation for understanding how cognitive agents, including AI, converge on accurate knowledge through structured learning processes.

ADDITIONAL READINGS & REFERENCES

- 156 Gepperth, A. & Hammer, B. in *European symposium on artificial neural networks (ESANN)*.
- 196 Hirsh, H. Generalizing version spaces. *Machine learning* **17**, 5-46 (1994).
- 197 Peters, U. & Chin-Yee, B. Generalization bias in large language model summarization of scientific research. *Royal Society Open Science* **12**, 241776 (2025).
- 198 Mudrow, M. in *Analogical Modeling: An exemplar-based approach to language* 225-264 (John Benjamins Publishing Company, 2008).
- 199 Hirsh, H., Mishra, N. & Pitt, L. Version spaces and the consistency problem. *Artificial Intelligence* **156**, 115-138 (2004).
- 200 Navada, A., Ansari, A. N., Patil, S. & Sonkamble, B. A. in *2011 IEEE control and system graduate research colloquium*. 37-42 (IEEE).
- 201 Charbuty, B. & Abdulazeez, A. Classification based on decision tree algorithm for machine learning. *Journal of applied science and technology trends* **2**, 20-28 (2021).
- 202 Vanlehn, K. & Ball, W. A version space approach to learning context-free grammars. *Machine learning* **2**, 39-74 (1987).
- 203 Chen, Y., Singla, A., Mac Aodha, O., Perona, P. & Yue, Y. Understanding the role of adaptivity in machine teaching: The case of version space learners. *Advances in Neural Information Processing Systems* **31** (2018).
- 204 Kautz, H., Arnstein, L., Borriello, G., Etzioni, O. & Fox, D. in *AAAI-2002 Workshop on Automation as Caregiver: The Role of Intelligent Technology in Elder Care*. 6065.

Chapter 19

Constraint Propagation

INTRODUCTION TO CONSTRAINT PROPAGATION

In this chapter, we'll explore constraint propagation, a fundamental and widely-used method in knowledge-based AI. Constraint propagation refers to the process by which an agent assigns values to variables in a way that satisfies specific conditions, called constraints.²⁰⁵ It is a highly adaptable and general-purpose method applied across various AI domains, including planning, natural language processing, understanding, and visual-spatial reasoning.

At its core, constraint propagation involves finding solutions to problems by making sure all given rules or conditions are met. For instance, when an AI agent is tasked with understanding a visual scene or interpreting a sentence, it uses constraints from its knowledge – such as rules of grammar or properties of physical objects – to arrive at a solution.^{206,207} These constraints help the agent eliminate possibilities that do not fit and focus on those that do.

This chapter begins by formally defining constraint propagation, followed by an exploration of how it enables AI agents to make sense of the world. We'll draw examples from both understanding natural language sentences and interpreting visual scenes to demonstrate how this method works in practice. Through these examples, you'll see how agents can group components, identify patterns, and meet the rules defined by constraints to achieve successful outcomes.

For example, when interpreting a sentence, an agent must ensure that grammatical rules are followed even if the meaning of the sentence seems nonsensical. Likewise, when recognizing a

3D shape from a 2D drawing, the agent uses its knowledge of geometric constraints to understand how the different parts fit together. Both tasks involve satisfying constraints – whether they are about the structure of language or the properties of shapes.

In this chapter, we will also explore some advanced aspects of constraint propagation. Although the process is a robust way to reason through problems, it can also lead to ambiguity, or even conflicts, when multiple interpretations of a situation can satisfy the same set of constraints. We'll touch on how these complexities arise and how AI systems can handle them.

By the end of this chapter, you will gain a thorough understanding of how constraint propagation functions as a core technique in AI problem-solving. It is a mechanism that allows agents to process information in a structured way by assigning values to variables and ensuring that all constraints are satisfied. Furthermore, as we will see, constraint propagation mirrors aspects of human cognition, helping both AI and humans make sense of complex environments by using rules and constraints to guide decision-making.²⁰⁸

UNDERSTANDING 3D OBJECTS THROUGH CONSTRAINT PROPAGATION

To introduce the concept of constraint propagation, let's begin with an example from visual perception. Take a look at the figure shown here. While this is a simple drawing on a two-dimensional surface, most of us almost immediately recognize it as a three-dimensional cube. The question is: how do we do this? How does our brain process this flat image and interpret it as a 3D object? And, perhaps more importantly, how can we help machines achieve the same recognition?

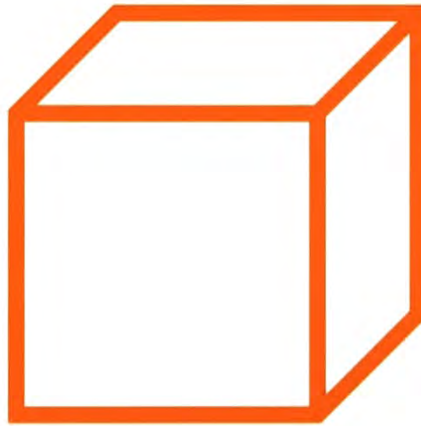


Figure 19-1 2D representation of a 3D shape

The answer lies in the idea that this cube is a trihedral object. A trihedral object is defined as one in which three surfaces meet at a particular point. This geometric property provides useful constraints that allow us to interpret the object as three-dimensional. In broader terms, a polyhedral object is one where multiple surfaces meet at the same point. For example, think of a soccer ball: the white and black patches represent polyhedral surfaces that converge at different points. Another example is a pyramid, where four surfaces meet at the apex.

Recognizing these shapes, even on a flat, two-dimensional surface, depends on understanding the constraints provided by the geometry of the object. Through the method of constraint propagation, our brains – or an AI system – can process the visual information and arrive at the conclusion that this is a 3D cube. This ability to infer depth and structure from simple lines on a flat surface demonstrates the power of constraint propagation in both human cognition and artificial intelligence, especially when it comes to visual-spatial reasoning.

Next, we'll explore an example that takes us further into the process of how constraints are propagated to determine the nature of objects.

Recognizing 3D Figures: A Visual Exercise with Ambiguity

Ashok and David are about to explore the complexities of recognizing 3D objects from 2D representations. Let's follow along in their dialogue to understand how different shapes can sometimes challenge our visual system. Their dialogue offers insights into how our visual systems interpret shapes and the role ambiguity plays in perception.

Which of these wireframes depicts a 3D figure that could exist in the real world?

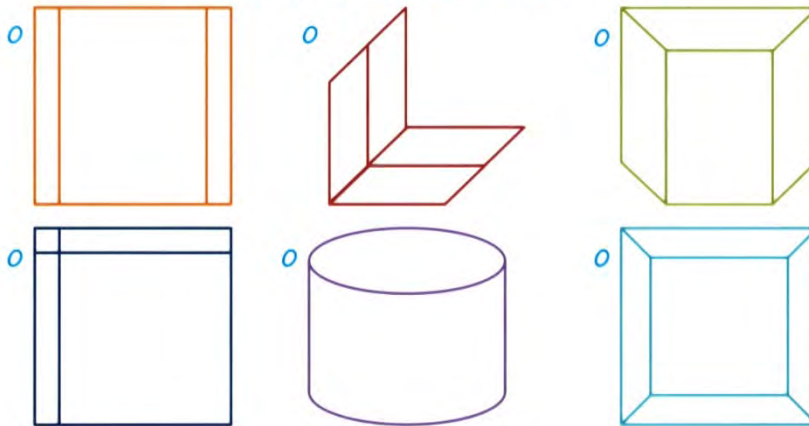


Figure 19-2 Identifying 3D objects when represented in 2D

Ashok Consider the following image of six figures. Which of the figures represent 3D objects? What do you think, David?

David I believe four of these could represent 3D objects. The one in the bottom center could be a cylinder. The upper right one looks like a trapezoidal prism, and the lower right could be a pyramid from above, though it's as if the top is blocked off. We'd have to interpret it as either looking down into a pyramid or a box. The shape in the upper left is more interesting – it doesn't as easily lend itself to a 3D interpretation, but we could imagine that this is the result of looking down on the upper-right shape directly.

Ashok It's interesting you didn't pick the upper center as a 3D object, David. I would have. I see it as depicting the x-axis, y-axis, and z-axis. Since all three axes are visible here, I would have thought of this as a 3D object as well.

David That's a good point, Ashok. Now that you mention it, I can't stop seeing it as a 3D object. It now looks like a folded piece of paper with a line drawn down the middle. Initially, I didn't see it as 3D because I thought the lines represented folds or intersections of planes. But when I reinterpret the middle line as simply drawn on the figure, it pops out as a 3D object.

Ashok And that's the main takeaway of this exercise. It shows that some kind of processing occurs within our visual system that helps us group these lines and

surfaces in ways that let us decide which shapes represent 3D objects and which don't.

What this exercise reveals is that the processing we use to interpret these shapes is not always definitive. There is often some ambiguity in our perception. You might come up with one interpretation, and someone else might arrive at a slightly different conclusion. This is because the processing our brain uses leaves room for individual variations in interpretation.

This ambiguity shows that our visual system often uses constraints to make sense of shapes, but even when those constraints are applied, different valid interpretations can coexist. Next, we'll explore how constraint propagation can assist in resolving these ambiguities and allow us to make more definitive conclusions about the shapes we see.

Understanding Grammatical Constraints through Nonsensical Sentences

Next, we'll examine how our cognitive system processes language, particularly when identifying grammatically correct sentences from those that are nonsensical. Consider the following six sentences:

1. Colorless green ideas sleep furiously.
2. Soft drinks due from thank bills insurance.
3. The treating physician persons the following excluded tasty.
4. Wall décor notifies business cards of nonsensical whims.
5. Tuesday brought a sharp-edged suite of pumpernickel.
6. Go search or revoke to writing this present understand.

None of these sentences make much sense in terms of meaning, or semantics. However, some are grammatically correct, and our minds can quickly detect which ones follow proper grammar rules. Let's see if you can identify which of these sentences are grammatically correct.

Ashok What did you think, David?

David The first sentence, "Colorless green ideas sleep furiously," immediately caught my attention. It's a well-known example introduced by Noam Chomsky to show that a sentence can follow grammatical rules while still being nonsensical. This sentence makes sense structurally – it has a subject (ideas), a verb (sleep), and an adverb (furiously) describing the action. But the concept of colorless green ideas or sleeping furiously makes no sense in the real world. Still, grammatically speaking, it's correct.

The second sentence, "Soft drinks due from thank bills insurance," is an entirely different case. While it appears to have a subject (soft drinks) and a verb (due), the

rest falls apart grammatically. The verb is immediately followed by a preposition (from), which creates confusion, and the rest of the sentence tries to string together random nouns (thank, bills, insurance) that don't fit into any coherent structure. This sentence is both nonsensical and grammatically incorrect.

The other two sentences that are incorrect have similar problems. They either lack the basic subject-verb-predicate structure or violate other fundamental grammatical rules, making them completely incoherent.

For the two sentences that are grammatically correct but still nonsensical, they at least maintain a recognizable sentence structure. For example, in "Wall décor notifies business cards of nonsensical whims," the phrase follows the proper subject-verb-object format (wall décor notifies). Though wall décor cannot literally notify business cards, the sentence obeys the rules of grammar. Similarly, in "Tuesday brought a sharp-edged suite of pumpernickel," the structure is also correct, even though a sharp-edged suite of pumpernickel makes no sense in reality.

Ashok Thank you, David.

David's method of analyzing the sentences reveals how he applied his understanding of English grammar. Instead of focusing solely on individual words, he examined whether the sentence structure fulfilled certain conditions, or constraints, he expected based on his knowledge of the language. Essentially, he was performing constraint processing – determining if the sentence met the grammatical requirements, regardless of whether the sentence conveyed meaningful ideas.

This process illustrates how we use constraints to parse and make sense of language. Even when semantics fail to provide meaning, our brain can still rely on grammatical constraints to determine correctness. In the same way, constraint propagation can help us disambiguate or interpret possible and impossible scenarios in other cognitive tasks, from language processing to visual recognition.

CONSTRAINT PROPAGATION: DEFINITION AND CHARACTERISTICS

Constraint propagation is a key method in knowledge-based AI, used in various contexts such as planning, visual reasoning, natural language understanding, and more.²⁰⁹⁻²¹³ At its core, it is a method of inference that assigns values to variables within a problem while ensuring that certain conditions, called constraints, are satisfied.

Let's define it more formally:

- **Constraint propagation** is a method of inference that assigns values to variables characterizing a problem in such a way that global constraints are satisfied.

In any problem, there are usually several variables, each of which can take on different values. The challenge is to assign specific values to these variables so that the solution satisfies the global constraints – meaning that the entire solution must adhere to certain predefined conditions.

Example 1: A 2D Representation of a 3D Object

Let's consider the figure in the image above, which is drawn on a 2D surface. The problem here is to determine whether this 2D figure represents a 3D object.

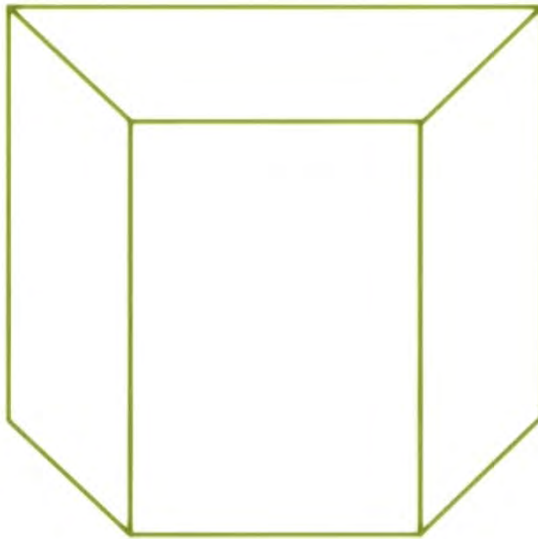


Figure 19-3 Determining if 2D diagram represents a 3D shape

The variables in this problem are the surfaces and their orientations. One way to interpret this figure is as a simple 2D image composed of lines, but there's also a deeper interpretation: we can view it as having four distinct surfaces (one, two, three, four), each with a specific orientation. The orientation of each surface can be described by the perpendicular to that surface.

Constraint propagation can help us identify these surfaces and their orientations. The constraints here are defined by the junctions where the surfaces meet. In the case of trihedral

objects (objects with three surfaces meeting at a single point), these junctions follow specific properties. The goal is to assign values to the surfaces and their orientations in such a way that all the junction constraints are satisfied.

There are multiple ways to interpret this particular figure:

1. **Looking inside a box** – One possible interpretation is that we are viewing the inside of a box from a certain perspective.
2. **Looking at a building** – Another possibility is that we are observing a building, depending on how we assign values to the surfaces and their orientations.

This example demonstrates that constraint propagation doesn't always result in a single, definitive solution. In some cases, multiple interpretations can satisfy all the constraints simultaneously. On the other hand, it's also possible that no assignment of values to the variables will satisfy all the constraints, making interpretation difficult.

The Importance of Junctions

The constraints in this example are tied to the junctions where surfaces meet. Trihedral objects, like cubes or pyramids, have junctions where three surfaces converge. These junctions have properties that must be respected when assigning values to the variables (i.e., the surfaces and their orientations). Whether we view this figure as a box or a building depends on how these constraints are applied to the junctions.

Thus, constraint propagation plays a critical role in helping an AI system make sense of ambiguous or complex visual information by ensuring that surfaces and their orientations are assigned values that satisfy all the relevant constraints.

Example 2: Semantic Meaninglessness vs. Grammatical Correctness

Let's explore how constraint propagation operates in language, by examining how we can determine the grammatical correctness of a sentence even when it lacks semantic meaning.

Consider the famous sentence:

- “Colorless green ideas sleep furiously.”

At first glance, the sentence is nonsensical – something cannot be both colorless and green, and ideas certainly don't sleep, let alone furiously. Yet, we still recognize it as grammatically correct. How can we determine this?

In this example, the variables are the grammatical roles assigned to each word – such as whether a word is a noun, verb, adjective, or adverb. The values are the specific grammatical categories we assign to the words in the sentence:

- Green functions as an adjective (describing the noun),
- Ideas is a noun,
- Sleep acts as a verb,
- Furiously functions as an adverb (modifying the verb).

The constraints are the rules of English grammar. As we assign words to their respective categories, we ensure these assignments follow grammatical rules. If they do, the sentence is grammatically correct, regardless of whether it makes sense semantically.

However, if the sentence were grammatically incorrect, we would not be able to assign values to the variables that meet all the constraints. For example, putting an adjective where a verb belongs would violate grammatical rules, thus breaking the constraints and rendering the sentence incorrect.

This concept of using constraints within grammar is not new. In our earlier discussions about understanding language, we touched on how prepositions, like “from,” limit or define what kinds of words can follow them, setting expectations about meaning. For example, after the word “from,” we expect a source to follow (e.g., a location, person, or thing). In those cases, we used grammatical constraints to assist with semantic analysis.

Here, however, the focus is strictly on determining whether a sentence is grammatically correct without concern for its meaning.

The Dual Nature of Constraint Propagation

There is an intriguing parallel between visual and linguistic constraint propagation, which reveals how the method allows for multiple valid interpretations. Just as a 2D shape can be interpreted as either protruding outwards or receding inwards (as we saw in the earlier example), some sentences can hold multiple interpretations, even while adhering to grammatical rules.

Consider the following pun: “It’s hard to explain puns to kleptomaniacs because they always take things literally.” Here, the word “take” can be interpreted in two ways:

1. Interpret (as in “take” meaning understand),
2. Physically remove (as in “take” meaning to steal).

Both meanings fit within the grammatical structure of the sentence, showcasing how constraint propagation in language can support multiple valid interpretations, just as in visual processing.

In both the visual and linguistic examples, the underlying mechanism is the same. We assign values to variables – whether those variables are surfaces and orientations for visual objects or words and grammatical categories for sentences. The goal is to ensure those assignments satisfy the given constraints. When they do, we reach a valid interpretation, whether in terms of recognizing a 3D shape or determining that a sentence is grammatically correct.

UNDERSTANDING CONSTRAINT PROPAGATION IN COMPUTER VISION: MARR'S FRAMEWORK

To explore the mechanics of constraint propagation, let's examine an example from the domain of computer vision. Consider the first image, which is composed of a large number of pixels. The intensity of light at each pixel is represented by varying shades of grey. While it might initially

appear as just a collection of shaded squares, you and I can immediately recognize this as a cube. But how does this recognition happen? And more importantly, how can we enable a machine to recognize it in the same way?

David Marr, a key figure in computer vision, proposed a theory to break down the process of 3D object recognition into several stages. According to Marr, the brain – or in the case of AI, the machine – works through a series of subtasks that allow it to progress from simple pixel data to a full 3D object.

In the first subtask, the visual system detects edges or lines, as represented in the first image. At this stage, no surfaces or objects have been identified. All the system has is a set of lines derived from the contrast in light intensity between pixels. There is no understanding of depth or three-dimensionality yet, just an assembly of lines.

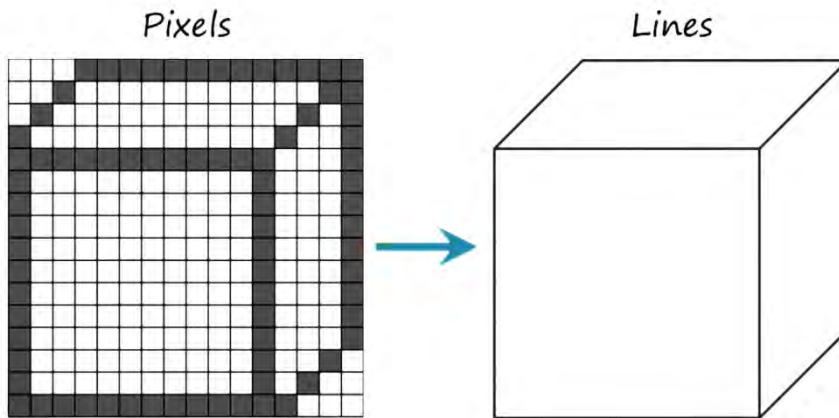


Figure 19-4 Recognizing pixels as lines

Next comes the second subtask: grouping the lines into surfaces. As shown in the second image, the visual system begins to recognize that these lines are connected and form planes, or surfaces. The system now starts to assign orientations to these surfaces. For example, the surfaces of the cube are defined by their orientation in space, and the system understands that these surfaces are parts of the same object.

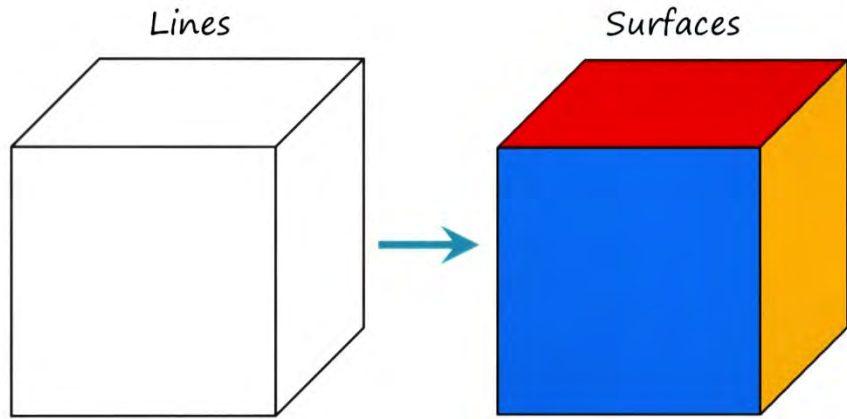


Figure 19-5 Recognizing lines and surfaces

The final stage, shown in the third image, is the grouping of surfaces into a complete 3D object. Now, the visual system recognizes that these surfaces form a cube. What was initially a collection of pixels and lines is now fully understood as a three-dimensional shape.

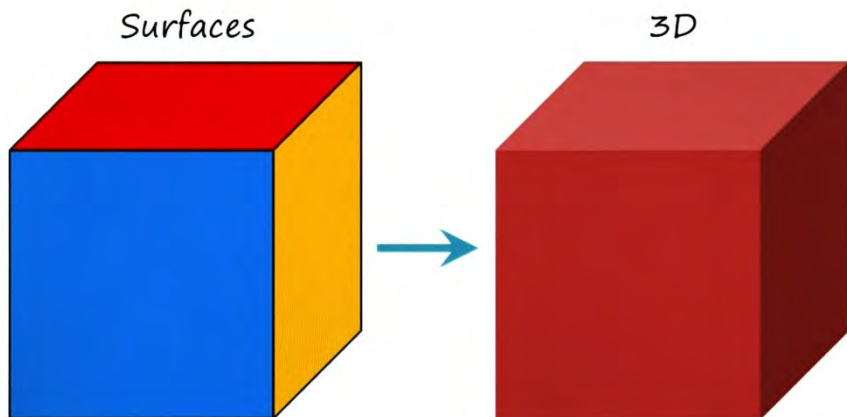


Figure 19-6 Recognizing surfaces and 3D objects

Marr's theory has had a significant impact on computer vision and AI as a whole. One of the core insights from Marr's work is the importance of breaking a task down into smaller, manageable subtasks before diving into the algorithms used to solve it. This approach applies beyond vision and is a common theme across many areas of knowledge-based AI.

Throughout this book, we've emphasized the power of task decomposition. In earlier chapters, we saw how complex tasks like understanding a sentence can be divided into smaller steps. For example, surface-level cues can act as triggers for memory, retrieving a relevant frame. The slots within the frame then set expectations. Lexical and grammatical analysis helps identify objects and predicates that fulfill those expectations, and the corresponding slots are filled in. Problem decomposition, like this, is a general-purpose method that applies regardless of the specific algorithm or task.

By understanding how constraint propagation works in tasks like vision, we can apply the same principles to other AI problems, breaking them down into smaller tasks that can be tackled step by step. This approach is not only effective in computer vision but also in language processing, problem-solving, and other cognitive tasks AI systems perform.

LINE LABELING AND JUNCTION CONSTRAINTS IN VISUAL PERCEPTION

In this section, we focus on constraint propagation through the task of line labeling, which plays a key role in identifying the surfaces and orientations of objects by using real-world constraints.^{214,215} Specifically, we'll use the example of recognizing trihedral objects – three-dimensional objects like cubes, which have multiple surfaces meeting at a point.

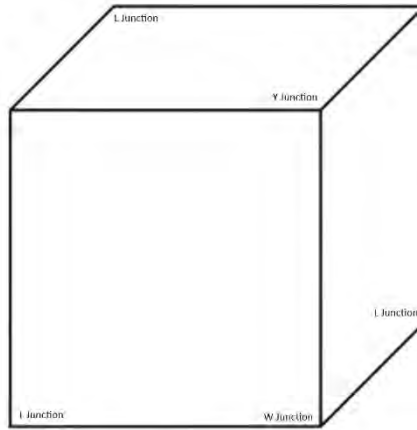


Figure 19-7 Cube with junction identification

Take a look at the cube depicted in the image above. The first thing to note is that the cube has several junctions, where lines meet and form different shapes. These junctions can be categorized into four main types: Y junctions, L junctions, W junctions, and T junctions. Each type of junction carries distinct constraints, which help us determine how the surfaces of the object are organized and how its overall structure is inferred.

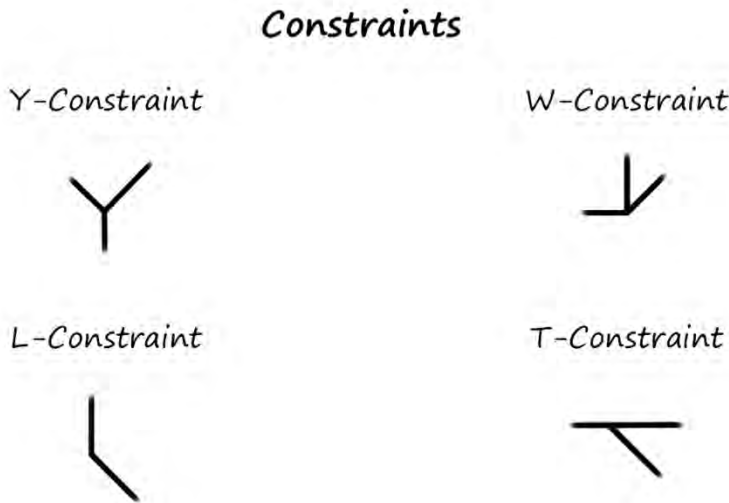


Figure 19-8 Type of junctions as constraints

Y Junctions

Let's start with the Y junction, which occurs when three surfaces meet at a single point. You'll notice a Y junction at the top of the cube in the image. The important feature of a Y junction is that each of the lines radiating from the point represents a fold. A fold is a line where two surfaces physically meet. The key idea here is that once we recognize a Y junction, we can propagate the information – meaning that if one line is identified as a fold, the others must also be folds. This constraint helps us define how the surfaces of the object are connected. In the real world, Y junctions typically occur in objects like cubes, pyramids, or other polyhedral shapes where multiple surfaces converge.

L Junctions

Next, consider the L junction, which is formed when two lines meet at a right angle. In the case of L junctions, the constraint tells us that the two lines represent blades rather than folds. A blade refers to an edge where two surfaces don't necessarily meet directly. This distinction is important because it tells us that the two surfaces adjacent to the L junction are not connected. Just like with Y junctions, the constraints of the L junction propagate through the object, so when we identify a line as a blade, any other line connected to it at an L junction must also be a blade. Though the L junction can take on various forms depending on the specific geometry of the object, the basic constraint of labeling the lines as blades remains.

W Junctions

Now, let's examine the W junction, which has a unique blade-fold-blade arrangement. In this junction, one of the lines connected to the point is a fold, while the other two are blades. This means that at the fold, two surfaces are meeting, whereas at the blades, the surfaces are not connected. By identifying a W junction, we apply this specific constraint to the lines, helping us understand how the surfaces and edges interact. For example, this type of junction might appear in objects with more complex geometries where one surface folds into another but the adjacent surfaces do not connect.

T Junctions

Lastly, there's the T junction, which occurs when one line is interrupted by another at a right angle, creating a blade on the continuous line and a fold on the intersecting line. T junctions often appear in scenes where one object is partially occluding another, meaning one object is blocking part of the view of another object. The constraints of a T junction give us critical information about the depth and spatial relationships of objects in the scene, helping to differentiate between what is in the foreground and what is in the background.

Understanding Spatial Grammar through Junctions

In effect, these junction types and their constraints form a kind of spatial grammar – a set of rules that help us understand the structure of objects based on their edges and surfaces. Just as grammar in language helps us construct and understand sentences (e.g., a noun phrase followed by a verb phrase), the constraints in spatial grammar help us identify how surfaces are connected through the various types of junctions. For instance, just as a sentence can have a noun phrase followed by a verb phrase, an image can be decomposed into surfaces connected by folds, blades, and junctions.

By categorizing and labeling the junctions, we can propagate these constraints throughout the entire image, allowing both human observers and AI systems to infer the three-dimensional structure of an object, even when presented with a two-dimensional representation. This method is crucial for making sense of visual information in both computer vision and AI reasoning tasks.

Next, we'll walk through an example of applying these junction constraints to propagate information throughout an image, further demonstrating the power of constraint propagation in visual reasoning.

EXERCISE: IDENTIFYING JUNCTION TYPES IN A CUBE

Now that we've introduced the basic constraints for the world of trihedral objects, let's see how we can propagate these constraints to group edges into surfaces. We'll do this through an exercise where we analyze the junctions present in a cube and identify what type of junction each one is, using the established constraints.

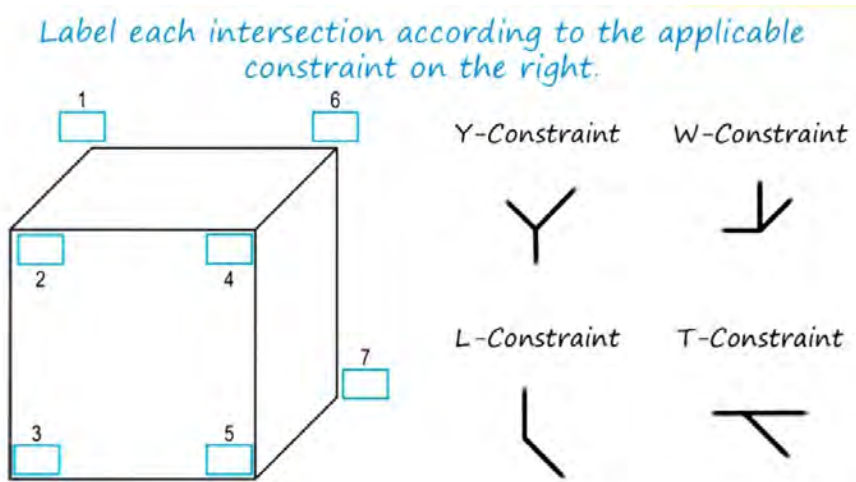


Figure 19-9 Identification of junctions

Take a look at the cube above, where seven junctions are labeled from 1 through 7. Your task is to determine whether each junction is a Y, W, L, or T junction by referencing the constraint types shown to the right of the cube.

Ashok What did you write down, David?

David Going down the outside, we have alternating L and W junctions. At the bottom left, we have an L junction (position 3), then a W junction (position 5), and another L junction (position 7). At the top left, there's another L junction (position 2), and at the top right, we have a W junction (position 4). In the center (position 6), we have a Y junction.

We can differentiate a Y junction from a W junction because in a W junction, one of the three angles must be over 180 degrees. In a Y junction, all three angles are less than 180 degrees. The T junction (at position 1) occurs where one of the angles is exactly 180 degrees.

Ashok This sounds good, David. Now, let's look at how we'll apply these constraints to identify the surfaces.

Exercise: Labeling Edges as Folds or Blades

We've identified the junction types in the previous exercise. Now, let's use the constraints associated with each junction to determine the type of each edge – whether it's a fold or a blade. Take a look at the cube again and label each edge based on the provided constraints.

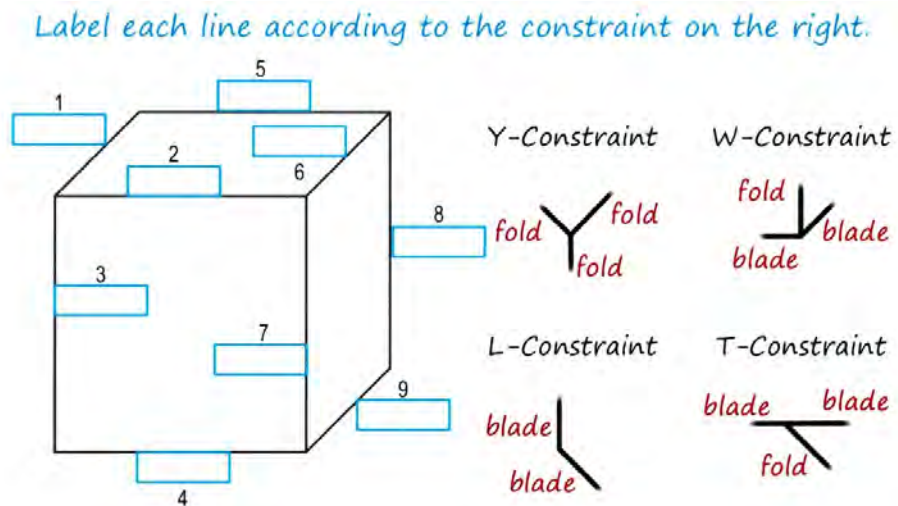


Figure 19-10 Identification of edge types

Ashok What did you come up with, David?

David Starting from the top left, I applied the L-Constraint at junction 5, which told me that both edges 5 and 1 are blades. Then I moved to the W-Constraint at junction 6 and labeled the edges as blade, fold, and blade for edges 1, 2, and 3, respectively.

What I noticed is that the constraints at different junctions are consistent. For example, both the L-Constraint and the W-Constraint identified edge 1 as a blade, which was confirmed as I worked my way around the figure. When I reached the center Y-junction at junction 2, I saw that edges 2, 6, and 7 are all folds. These folds were also consistent with the W-Constraint for those same edges, showing that the constraints propagated across the entire figure without contradiction.

Ashok That's correct, David.

It's also important to note that David started from the top left, but he could have begun at any other junction – say here or here – and arrived at the same result. This consistency happens because we've simplified the constraints in this exercise.

Now that we've determined that some of the lines are folds, it follows that two surfaces must meet at each fold. For example, since edge 1 is a fold, two surfaces must meet at that line. Similarly, because edge 2 is also a fold, two surfaces meet at that line as well. By repeating this process for each edge, we can group the lines into distinct surfaces.

What this exercise shows is that the visual system uses constraints at each junction to recognize which lines form the boundaries of surfaces. Instead of perceiving the cube as one continuous surface, the system identifies it as composed of three distinct surfaces. This process helps us recognize the figure as a three-dimensional cube.

Handling Ambiguities in Constraint Propagation

Ashok Now that we've discussed the constraint propagation technique, David, why do you think you didn't immediately recognize this as a 3D object?

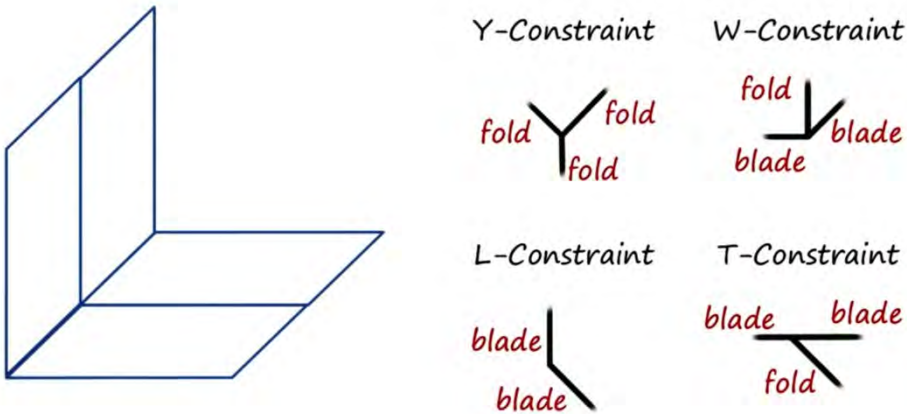


Figure 19-11 Recognition of an object as a folded book or page

David I think there are two main reasons. First, there's a junction here that doesn't exist in my dictionary of junctions. It involves four lines coming together at a single point, but I can only handle junctions with up to three lines. Because I don't recognize it as a junction, I don't naturally perceive the shape as a realistic one. The second reason is that when I label these three intersections with their respective junctions, I get an L, a Y, and another L. When I propagate the

constraints from the L junction, I see blades. When I propagate from the Y junction, I see folds. But notice that the Y junction tells me that two edges are folds, while the L junctions tell me that the same edges are blades. This creates a conflict, which makes it hard to propagate the constraints throughout the shape, and as a result, I don't see it as a valid 3D object.

Ashok That's an insightful analysis, David. And yes, you're right. Some people do see this as a 3D shape, but there's a trick to it. You can imagine it like a piece of paper folded in two – one plane here and another plane there, somewhat like an open book. This particular line here then can be ignored. It's just a line between two planes, not signifying a surface by itself.

By thinking of this line as simply a line, and not an edge marking a surface boundary, you solve the first problem of not recognizing a junction. But how do we resolve the second problem, where the fold vs. blade ambiguity arises depending on where constraint propagation starts?

The answer lies in the fact that we've been using a simplified ontology of junctions and constraints to demonstrate the idea of constraint propagation. In reality, the ontology is much more complex. For example, a Y-constraint doesn't always result in fold, fold, fold – it can also result in blade, blade, blade, depending on the situation. Similarly, the L-constraint isn't always blade and blade. It can be blade and fold, or fold and blade.

With this more complex ontology, we can eliminate the conflict you noticed. The Y junction can allow for a blade, and the L junction can also suggest a blade.

It's important to note that even though we've expanded the ontology, what we're working with here is still a simplified version of the complete set of Y, W, L, and T constraints. The T junction, in particular, can be more intricate, adding further complexity to the system.

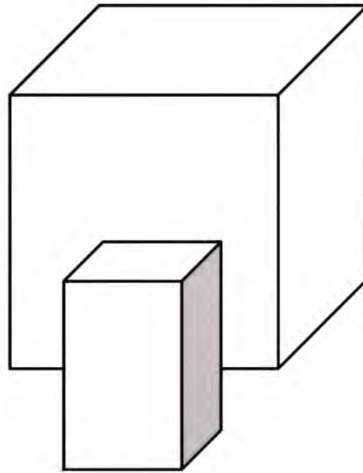


Figure 19-12 Two blocks with one partially occluded

One advantage of having a more complete ontology is that it allows us to interpret more complicated scenes. For instance, consider the image of two rectangular objects, one of which partially occludes the other. The more complex ontology helps us disambiguate junctions in such scenes, though it does come with new challenges. Specifically, a junction could be labeled as blade, blade, blade, or fold, fold, fold, and both are permissible under the more complete ontology.

To address these ambiguities, we can adopt additional conventions. One useful convention is to label all edges adjacent to the background as blades. So, we could label these edges as blade, blade, blade, blade. Once that decision is made, it becomes easier to propagate the constraints.

Take the W junction, for example. It could potentially be fold, blade, fold, or blade, fold, blade. But if we apply the background-edge convention, the W junction can only be blade, fold, blade. Once we've labeled it that way, the Y junction here can only be fold, fold, fold, and so on. This helps us resolve the ambiguity.

Ultimately, the task of interpreting images like this is an instance of abduction. Abduction involves forming the best possible explanation for the available data. In this case, we're trying to interpret the visual data by labeling the edges and junctions with the most appropriate constraints. We'll discuss abduction in more depth when we explore diagnosis later on, but for now, it's worth noting that we begin with something we know – such as blade, blade, blade – and propagate the constraints outward to disambiguate the other junctions.

EXPLORING GRAMMAR THROUGH CONSTRAINT PROPAGATION

Let's return once again to the sentence: "Colorless green ideas sleep furiously." While the sentence is semantically meaningless, you and I can still immediately recognize that it is grammatically correct. But how exactly do we determine this?

Colorless green ideas sleep furiously.

Constraints:
Sentence = Noun Phrase + Verb Phrase
Noun Phrase = [Adjectives] + (Noun or Pronoun)
Verb Phrase = Verb + [Adverb]

Figure 19-13 Using constraint propagation with NLP

To answer this, let's consider a simplified grammar – a small subset of the English language rules – that helps us evaluate grammatical correctness using constraints. This mini grammar consists of just three basic rules:

1. A sentence can be broken down into a noun phrase followed by a verb phrase.
2. A noun phrase may consist of optional adjectives, followed by a noun or pronoun. (The square brackets indicate that adjectives are optional.)
3. A verb phrase is composed of a verb followed by an optional adverb.

In this sentence, the variables are the individual words, and the values we assign to them are their lexical categories, such as adjectives, nouns, verbs, and adverbs. If we can create a parse tree that assigns these values to the variables in a way that adheres to the constraints of our simplified grammar, we can confidently determine the sentence is grammatically correct.

Let's begin by constructing the parse tree for this sentence. According to the first rule, a sentence is composed of a noun phrase followed by a verb phrase. So, we divide the sentence into these two parts: "Colorless green ideas" as the noun phrase and "sleep furiously" as the

verb phrase. But how do we know where exactly to make this split, and how can we verify that our division is correct?

To check this, we rely on the second rule: a noun phrase can consist of one or more adjectives followed by a noun. By consulting our lexicon, we can confirm that “ideas” is a noun, while “colorless” and “green” are adjectives. Since this satisfies the constraints of a noun phrase, we know that “Colorless green ideas” is correctly categorized.

Next, we look at the verb phrase: “sleep furiously.” According to the third rule, a verb phrase is composed of a verb followed by an optional adverb. Again, using the lexicon, we verify that “sleep” is a verb and “furiously” is an adverb. Since this satisfies the constraints of the verb phrase, we know that “sleep furiously” is properly categorized as a verb phrase.

What’s crucial to note here is that the processing isn’t entirely top-down – it also involves a bottom-up component. By verifying the lexical categories at the word level, we confirm the higher-level structure of the sentence. This approach allows us to determine that the sentence is grammatically correct, even though its meaning remains nonsensical.

General Purpose of Constraint Propagation Across Domains

What makes this example so valuable is that it demonstrates how constraint propagation works across domains, not just in language. We’ve used constraints to process visual information in earlier examples, and now we’re applying them to language processing. In this case, constraint propagation has helped us build a parse tree for the sentence, which in turn could support further analysis, such as semantic understanding. Similarly, in visual processing, once we’ve applied constraints to label lines and recognize surfaces, we can proceed to interpret the object in its full 3D form.

Thus, constraint propagation is a domain-independent, general-purpose method. Whether you’re analyzing sentence structure or interpreting a visual scene, once the constraints are correctly propagated, they help disambiguate and interpret the information effectively. The method not only supports grammatical analysis in natural language processing but also enables deeper cognitive tasks like understanding and reasoning.

CONSTRAINT PROPAGATION AND COGNITION

Constraint propagation is not just a powerful method in artificial intelligence – it’s also deeply connected to human cognition. This technique, similar to means-ends analysis, serves as a general-purpose method that allows both AI systems and humans to interpret the world using pre-existing knowledge.²¹⁶ Whether we’re applying this technique in visual perception, language understanding, or even planning, constraint propagation allows us to reason about information by ensuring that certain conditions, or constraints, are met. These constraints, as we’ve explored, can be symbolic, numeric, or even a combination of both.

Earlier, we examined symbolic constraints when working with examples like 3D object recognition and sentence structure. In visual processing, we identified surfaces and edges by using constraints derived from trihedral objects, such as the Y, W, and L junctions. These junctions act as rules that guide how we interpret lines and surfaces. In language processing, we used a simplified grammar to categorize words and phrases into their respective lexical categories. Here too, constraints served as the backbone of our analysis, allowing us to determine the grammatical correctness of a sentence despite its nonsensical meaning.

But constraint propagation isn't limited to symbolic reasoning. It's equally relevant in dealing with numeric data, as illustrated by the example of an Excel spreadsheet. Many of us are familiar with how spreadsheets work: when columns are connected by formulas, a change in one column propagates through all other related columns. This automatic adjustment is a clear example of numerical constraint propagation in action. The relationships between cells are governed by numeric formulas, and any update to one value cascades through the entire system to ensure that all the constraints – set by the formulas – are met.

The Broader Cognitive Role of Constraints

We've actually encountered constraint propagation in other areas of human cognition as well. Consider planning: when we create a plan, we set a series of steps and conditions that must be satisfied to reach a particular goal. If one step changes or doesn't go as planned, adjustments are made to ensure the overall goal can still be achieved. This type of cognitive process reflects how our minds propagate constraints to maintain coherence in our plans.

The same principle applies to understanding. When interpreting new information, whether it's in the form of language, a visual scene, or a real-world event, we use constraints to make sense of it. In the example of sentence parsing, we use syntactic and semantic constraints to determine whether the words and phrases in a sentence form a coherent structure. Similarly, when we encounter a familiar scenario – a “script” in cognitive terms – we use constraints from that script to interpret the roles and actions within that scenario.

INTEGRATING KBAI: CONSTRAINT PROPAGATION IN THE LANDSCAPE OF MODERN AI

Constraint Propagation, a cornerstone of Knowledge-Based Artificial Intelligence (KBAI), offers powerful mechanisms for reasoning and problem-solving that remain highly relevant and can significantly enhance modern AI systems, including those based on machine learning. While statistical approaches excel at learning patterns from data, constraint propagation provides a robust framework for incorporating explicit knowledge, ensuring consistency, and efficiently pruning search spaces.

- **Enhancing Machine Learning with Explicit Knowledge:**

- **Guiding Learning and Inference:** Constraints derived from domain knowledge (e.g., physics, grammar, logical rules) can guide the learning process of statistical models, helping them converge faster, generalize better from less data, and avoid implausible solutions. In inference, constraints can filter or refine the outputs of machine learning models.
- **Structured Prediction:** Many real-world problems involve predicting structured outputs (e.g., parse trees, molecular structures, schedules). Constraint satisfaction is inherent in these tasks and can be integrated with machine learning models to ensure outputs are valid and coherent.
- **Improving Explainability and Trustworthiness:**
 - **Transparent Reasoning:** Constraint propagation operates on explicit rules and variables. When a solution is found (or not found), the satisfied (or violated) constraints provide a clear explanation for the outcome, which is crucial for debugging and building trust in AI systems. This contrasts with the often opaque nature of deep learning models.
 - **Verification and Validation:** Explicit constraints allow for easier verification that a system is behaving as intended and for validation against known ground truths or rules.
- **Addressing Complex Problem Solving and Reasoning:**
 - **Combinatorial Problems:** Constraint Satisfaction Problems (CSPs) are a formalization of constraint propagation and are highly effective for solving combinatorial problems found in scheduling, resource allocation, configuration, and planning. These are areas where purely data-driven approaches might struggle with the vast search spaces.
 - **Integrating Symbolic and Sub-symbolic Reasoning:** Constraint propagation can serve as a bridge between symbolic KBAI approaches and sub-symbolic machine learning. For instance, a neural network might identify candidate objects in an image, and constraint propagation could then reason about their spatial relationships and consistency with scene knowledge.
- **Applications in Specialized AI Domains:**
 - **Robotics and Planning:** Robots operate under physical and task-based constraints. Constraint propagation is essential for motion planning, task planning, and ensuring safe interaction with the environment.
 - **Natural Language Processing:** Beyond basic parsing, constraints are vital for semantic interpretation, discourse coherence, and ensuring that generated text is grammatically correct and contextually appropriate.
 - **Knowledge Graphs and Semantic Web:** Reasoning over knowledge graphs often involves propagating constraints to infer new facts or check consistency.
- **Efficiency and Scalability:**

- **Pruning Search Space:** A key strength of constraint propagation is its ability to dramatically reduce the search space by eliminating inconsistent assignments early. This makes intractable problems feasible.
- **Hybrid Solvers:** Modern CSP solvers often combine constraint propagation techniques with heuristic search and machine learning methods to tackle large-scale problems efficiently.

While constraint propagation is a mature field within KBAI, its principles are not outdated. Instead, they offer complementary strengths to data-driven AI. The future of robust, explainable, and effective AI will likely involve a tighter integration of knowledge-based reasoning, like constraint propagation, with the pattern recognition capabilities of machine learning. This synergy allows AI systems to leverage both explicit knowledge and learned patterns to solve complex real-world problems.

CHAPTER SUMMARY

In this chapter, we explored constraint propagation, a fundamental method of inference where values are assigned to variables in a way that satisfies specific constraints. By adhering to these constraints, we can make strong inferences about problems in various domains, whether identifying shapes in images or parsing the structure of sentences.

We began by defining constraint propagation and demonstrated its application in visual processing, using David Marr's framework to illustrate how AI can recognize 3D objects from 2D images by breaking the task into sub-problems: detecting lines, grouping lines into surfaces, and then surfaces into objects. We examined how constraints, such as the Y, L, W, and T junctions of trihedral objects (forming a “spatial grammar”), allow us to label edges as folds or blades, thereby building a structured understanding of visual scenes. We also acknowledged the potential for ambiguity and the need for more complex ontologies and conventions to resolve conflicts.

Next, we turned to natural language understanding. Here, constraint propagation is applied through grammatical rules and lexical categories. Even when sentences are semantically nonsensical, like the famous “Colorless green ideas sleep furiously,” we can still determine if they are grammatically correct by assigning parts of speech to words in a way that satisfies the grammar's constraints. This process illustrates how constraint propagation can help us structure language similarly to how it structures visual scenes, highlighting its domain-independent nature.

The discussion emphasized that constraint propagation is a general-purpose method, applicable to both symbolic and numeric constraints, and is deeply connected to human cognition, playing a role in planning and understanding.

Finally, we considered how constraint propagation, as a KBAI principle, integrates with and enhances modern AI. It can guide machine learning, improve explainability, solve complex combinatorial problems, and bridge symbolic and sub-symbolic reasoning, proving essential in fields like robotics, NLP, and knowledge representation. This underscores that constraint

propagation is not only integral to human cognition but also a vital component for building more robust, transparent, and effective AI systems. The next chapter will expand on these ideas, building on how constraint propagation is applied in design tasks through configuration.

ADDITIONAL READINGS & REFERENCES

- 205 Bessiere, C. in *Foundations of Artificial Intelligence* Vol. 2 29-83 (Elsevier, 2006).
- 206 Lu, Z. & Peng, Y. Exhaustive and efficient constraint propagation: A graph-based learning approach and its applications. *International journal of computer vision* **103**, 306-325 (2013).
- 207 Wang, D., Gao, X. & Wang, X. Semi-supervised nonnegative matrix factorization via constraint propagation. *IEEE transactions on cybernetics* **46**, 233-244 (2015).
- 208 Konar, A. *Artificial intelligence and soft computing: behavioral and cognitive modeling of the human brain*. (CRC press, 2018).
- 209 Tiezzi, M., Marra, G., Melacci, S. & Maggini, M. Deep constraint-based propagation in graph neural networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **44**, 727-739 (2021).
- 210 Cong, R. *et al.* Review of visual saliency detection with comprehensive information. *IEEE Transactions on circuits and Systems for Video Technology* **29**, 2941-2959 (2018).
- 211 Puranik, Y. & Sahinidis, N. V. Domain reduction techniques for global NLP and MINLP optimization. *Constraints* **22**, 338-376 (2017).
- 212 Lee, J. Y., Mehta, S. V., Wick, M., Tristan, J.-B. & Carbonell, J. in *Proceedings of the AAAI Conference on Artificial Intelligence*. 4147-4154.
- 213 Madkour, M., Benhaddou, D. & Tao, C. Temporal data representation, normalization, extraction, and reasoning: A review from clinical domain. *Computer methods and programs in biomedicine* **128**, 52-68 (2016).
- 214 Wolff, A., Knipping, L., van Kreveld, M., Strijk, T. & Agarwal, P. K. in *GIS and GeoComputation* 159-172 (CRC Press, 2000).
- 215 Zucker, Hummel & Rosenfeld. An application of relaxation labeling to line and curve enhancement. *IEEE Transactions on Computers* **100**, 394-403 (1977).
- 216 Wittocx, J., Denecker, M. & Bruynooghe, M. Constraint propagation for first-order logic and inductive definitions. *ACM Transactions on Computational Logic (FOCL)* **14**, 1-45 (2013).

Chapter 20

Configuration

INTRODUCTION TO CONFIGURATION

In this chapter, we explore the concept of configuration, a fundamental design task where all components are already known.²¹⁷ This task involves assigning values to the variables of these components so they can be arranged according to specific constraints.

Configuration marks the first topic within our unit on Design and Creativity. We begin by discussing the broader concept of design, which involves generating specifications for artifacts – ranging from physical products to processes, programs, and policies – that meet certain needs, goals, or functions.

We will define configuration in detail, tracing its process through what is known as “Planned Refinement.” This method emphasizes the step-by-step refinement of designs to meet predefined constraints, highlighting the iterative nature of configuration.

Additionally, this chapter connects configuration with previously explored topics such as Classification, Case-Based Reasoning, and Planning. By integrating these concepts, we aim to enrich our understanding of configuration and illustrate its role as a critical bridge between theoretical design principles and their practical applications across various domains. This approach demonstrates how configuration facilitates the transition from abstract design challenges to tangible, functional outcomes in daily tasks.

UNDERSTANDING DESIGN

Design is a fundamental activity that spans various disciplines, transforming specific needs, goals, or functions into detailed specifications for an artifact. Unlike physical products alone, these artifacts can also be processes, programs, or policies, each tailored to satisfy designated objectives.

Here are diverse examples illustrating the extensive reach of design:

- Engineers may design a robot specifically to navigate over water surfaces.
- Software developers create search engines optimized to fetch the most relevant search results.
- Economists at the Federal Reserve Bank craft monetary policies aimed at regulating and optimizing economic performance.

This diversity underscores the inherently broad, open-ended, and somewhat ambiguous nature of design tasks compared to more traditional problem-solving, where the problem itself typically remains static as solutions evolve. In design, however, both the problem and the solution evolve concurrently – the understanding of the problem deepens as solutions are conceptualized and refined.

The study of design within artificial intelligence is driven by the potential for AI agents to engage in sophisticated design tasks, including the capacity to design other AI agents.^{218,219} This exploration positions AI not merely as a tool but as an active participant in creative and innovative processes, paving the way for advancements in how we approach design across fields.

EXPLORING CONFIGURATION THROUGH PRACTICAL APPLICATION: BASEMENT DESIGN

Let's explore the concept of configuration with a practical example focused on designing a basement layout. In this scenario, we have specific requirements and constraints that guide the arrangement of the basement's spatial elements.

Requirements:

- Total area must equal sum of areas of individual rooms.
- All rooms must be rectangular.
- Utility closet and stairwell must each be at least 100 square feet.
- No length or width can be under 10 feet.
- Length is 44, width is 30.
- Bathroom must be at least 200 square feet.
- Two other rooms, each at least 400 square feet.

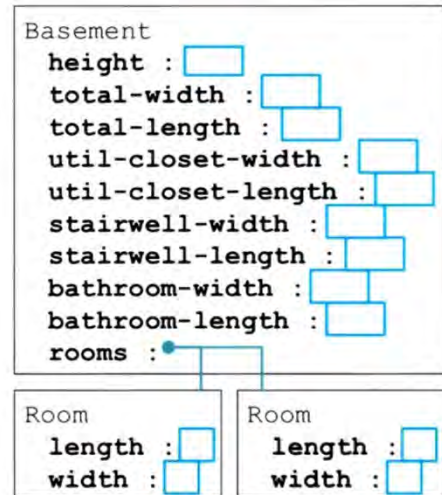


Figure 20-1 Configuration example.

The requirements for configuring the basement include:

- The total area must match the sum of the areas of individual rooms.
- All rooms, including utility spaces, must be rectangular.
- The utility closet and the stairwell should each cover at least 100 square feet.
- No dimension of any room, whether length or width, should be less than 10 feet.
- The basement's dimensions are fixed, with a length of 44 feet and a width of 30 feet.
- A bathroom is required, taking up at least 200 square feet.
- Additionally, there are two other rooms, each needing at least 400 square feet.

The challenge is to assign specific dimensions to various elements like the widths and lengths of the utility closet, stairwell, bathroom, and the two other rooms. This exercise demands efficient space allocation within the given constraints, ensuring every requirement is meticulously met. This example not only illustrates the configuration process but also highlights the detailed attention needed to effectively utilize space and meet predefined standards.²²⁰

Practical Configuration: Designing a Basement Step-by-Step

In this section, we explore the concept of configuration through a practical example of designing a basement within predefined constraints, illustrating the step-by-step approach to spatial arrangement and variable assignment.

Requirements:

- Total area must equal sum of areas of individual rooms.
- All rooms must be rectangular.
- Utility closet and stairwell must each be at least 100 square feet.
- No length or width can be under 10 feet.
- Length is 44, width is 30.
- Bathroom must be at least 200 square feet.
- Two other rooms, each at least 400 square feet.

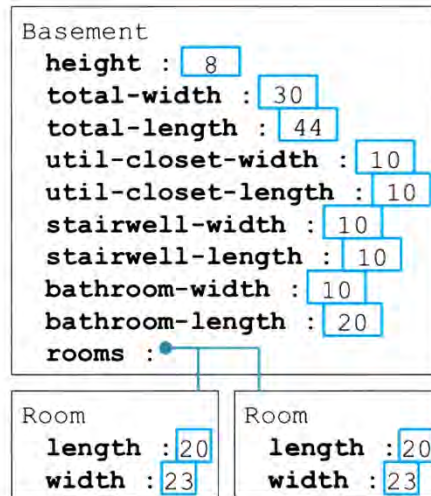


Figure 20-2 Walking through the configuration process using a completed example

The configuration begins with setting baseline parameters: the basement is designated a standard height of 8 feet – suitable for most residential constructions, ensuring ample headroom. The overall layout is defined by the structure above, with a length of 44 feet and a width of 30 feet, determining the available space for further planning.

Key utility areas are addressed next. The utility closet and the stairwell are allocated equal spaces of 100 square feet, standardized at dimensions of 10 feet by 10 feet. This choice simplifies the configuration while ensuring each space meets minimum requirements.

The bathroom, necessitating at least 200 square feet, is configured to dimensions of 10 feet by 20 feet, adhering to the constraint that no single dimension falls below 10 feet. This methodical assignment ensures compliance with spatial requirements while maintaining a straightforward layout.

With primary utilities allocated, focus shifts to the living spaces. The remaining area, 920 square feet, is equally divided into two substantial rooms, each measuring 460 square feet. Opting for dimensions of 20 feet by 23 feet for these rooms effectively utilizes the space, balancing generous room sizes with the architectural constraints of the basement.

This proposed configuration, while meeting area specifications on paper, highlights a common challenge: theoretical arrangements may not seamlessly translate into viable physical layouts. The calculated room areas align numerically, but fitting them within the basement's dimensions necessitates additional adjustments to ensure a practical and feasible layout.

Analyzing the Configuration Strategy

This basement configuration exercise underscores several key aspects of configuration in design:

- **Spatial Planning:** The primary objective is to devise a layout where all elements not only satisfy size requirements but also cohesively fit within the actual dimensions of the basement.
- **Prioritization of Variables:** The configuration process begins with variables that have strict limitations or significant impacts on the design, facilitating decision-making for subsequent variables.
- **Application of Heuristics:** The approach to prioritizing variables might include choosing the most restricted variables first, those that impose the most restrictions on others, or those deemed most critical to the project's success.

This detailed walkthrough of basement configuration demonstrates the application of theoretical principles in a real-world scenario, emphasizing the routine yet complex nature of configuration tasks in design projects. It showcases the necessity for adaptive strategies that respond to both the theoretical design requirements and the practical constraints of physical space.

EXPLORING CONFIGURATION AS A DESIGN PROCESS

Configuration is a focused aspect of design where all the elements are predetermined and the challenge lies in assigning values to variables within set constraints. This type of design is less about creativity and more about the efficient arrangement of known components. Unlike more open-ended design tasks, configuration deals with the methodical organization of elements to satisfy specific operational requirements.

Real-World Applications of Configuration

Consider the example of arranging furniture in a room, a practical exercise in configuration. The furniture and room dimensions are given, but the layout, such as placing a couch in optimal view of the TV, requires careful consideration of spacing and functional use. This task involves strategic placement of the primary elements first, followed by the arrangement of additional pieces to enhance the living space's functionality and aesthetic.

Further extending the concept of configuration:

- **Airplane Interiors:** Configuring an airplane involves meticulously placing seats, windows, and exits. This task balances passenger comfort with stringent safety standards and physical constraints, showcasing the complexity even in routine configurations.

- **Photographic Settings:** Adjusting a camera phone's settings for a specific shot involves configuring focus, flash intensity, and zoom to suit the subject and lighting conditions. For instance, capturing a distant landmark necessitates specific adjustments to these settings to accommodate for the physical distance, ensuring the photograph meets desired specifications.

These instances highlight the pervasive nature of configuration across various domains, demonstrating its critical role in both everyday and specialized settings.

Configuration's Role in Systematic Design

Configuration is a prevalent design activity in information processing, characterized by its systematic approach to arranging pre-defined elements. It simplifies the design process by focusing solely on the optimal placement and settings of components to meet predefined constraints, thereby enhancing functionality and compliance with required standards.

As we look deeper into the methods employed to tackle configuration tasks, it's clear that the process involves a detailed understanding of the components involved. This includes recognizing how to best utilize the available space and resources to meet design objectives efficiently. By exploring various strategies to optimize component arrangement, configuration proves to be an essential skill in both mundane and complex design scenarios, facilitating effective solutions in diverse settings.

THE CONFIGURATION PROCESS: FROM SPECIFICATIONS TO ARRANGEMENT MODEL

The configuration process in knowledge-based AI is a structured approach that begins with detailed specifications and culminates in a comprehensive arrangement model. This systematic method illustrates the transition from initial constraints to a fully realized design, showcasing the iterative and adaptive nature of design within AI.

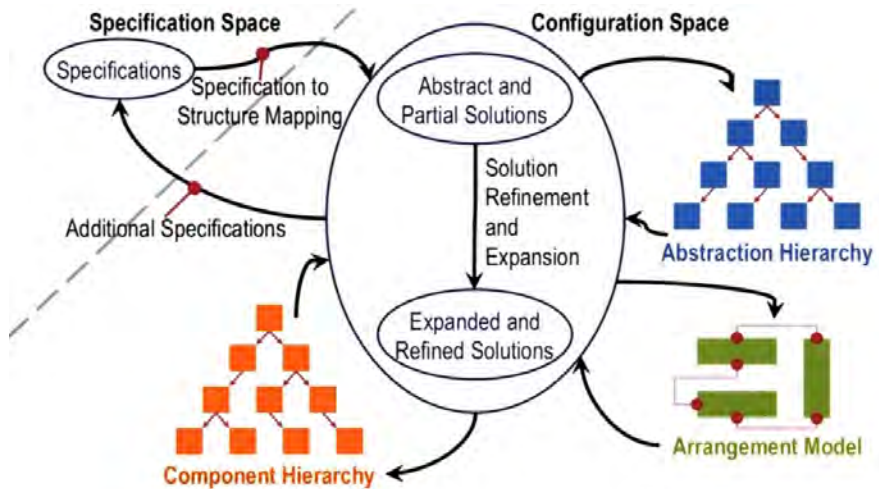


Figure 20-3 The Configuration Process

Initial Specifications to Solution Development

The process initiates in the Specification Space, where constraints and requirements for the configuration task are precisely defined. For instance, when configuring the basement of a house, specifications might include the total area, height requirements, and specific needs for areas like utility closets and stairwells.

These initial specifications lay the groundwork for developing Abstract and Partial Solutions. During this phase, designers draft general plans that set preliminary spatial arrangements and resource allocations. These solutions are adaptable, designed to be refined as more information becomes available or as requirements evolve.

Refinement and Expansion of Solutions

Moving forward, these abstract solutions enter a phase of Solution Refinement and Expansion within the Configuration Space. Each plan is incrementally detailed, specifying additional variables at each level. This process of refinement is iterative, continuing until a detailed and actionable Arrangement Model is achieved, which accurately reflects the layout of all components in compliance with the established constraints.

In a practical example, such as house configuration, an initial abstract plan might allocate spaces into functional areas – living rooms, bedrooms, kitchens, and bathrooms. Subsequent refinements would detail the exact dimensions, interior layouts, and amenities for each area, tailored to specific living requirements and aesthetic preferences.

Iterative Feedback and Dynamic Adjustments

A pivotal element of this configuration process is the feedback mechanism between the evolving arrangement model and the ongoing configuration activities. This iterative feedback allows for continuous reassessment and realignment of the plans to ensure they meet all requirements and adapt to any new constraints that may arise.

For example, adjustments may be necessary if the spatial arrangement of furniture does not meet ergonomic standards or if aesthetic considerations demand changes. This capability to modify plans based on assessment results ensures that the final design is both functional and aligned with user expectations.

Component Selection within Configuration

While the main focus of configuration is on the arrangement of predefined components, there is also the opportunity to select specific types of components that best fit the project's needs. This could involve choosing between various appliance models or furniture styles, each affecting the overall functionality and appearance of the space.

Example: Configuration in Computer Processor Design

Consider the task of configuring a computer processor to achieve a specific operational speed. Initially, the objective may simply be to optimize the processor's speed. However, upon constructing an arrangement model and assessing its performance, it might become apparent that the processor tends to overheat at higher speeds.

This discovery leads to a revision of the initial specifications. The design challenge now expands to include not only achieving the desired speed but also ensuring the processor does not overheat. This shift exemplifies the dynamic nature of design where both the problem and the solution evolve together. The revised specifications then guide the development of a new solution, which might involve adjustments in the processor's cooling system or changes in its architectural design to manage heat more efficiently.

This example underscores how configuration tasks often lead to iterative revisions and enhancements, reflecting the complex interplay between design objectives and practical constraints in real-world applications.

Evolutionary Nature of Design Through Configuration

Distinguishing configuration from simple problem-solving is the evolutionary nature of the design process, where the problem definition and the solution develop concurrently. As the

design is tested and implemented, it may reveal new challenges or opportunities, leading to revisions in the original specifications and, consequently, an evolved design solution.

This co-evolution of problem and solution is essential in design, particularly evident in configuration tasks where the iterative adjustments enhance the suitability and effectiveness of the design to meet complex user needs and contextual constraints.

By navigating these stages – from setting specifications to refining plans and making iterative adjustments – configuration stands out as a crucial tool in knowledge-based AI. It enables designers to craft precise, functional, and customized solutions across various applications, demonstrating the depth and flexibility of AI in practical design tasks.

DETAILED EXPLORATION OF CHAIR CONFIGURATION

In the domain of configuration, our goal is to represent knowledge in a manner that facilitates effective reasoning and decision-making. Let's examine the configuration process through the example of designing a chair. This process starts with a structured representation known as a frame, which organizes all pertinent knowledge about the chair into designated categories or slots. Each component of the chair – legs, seat, arms, and back – is represented in this frame, with specific attributes like size, material, and cost associated with each.

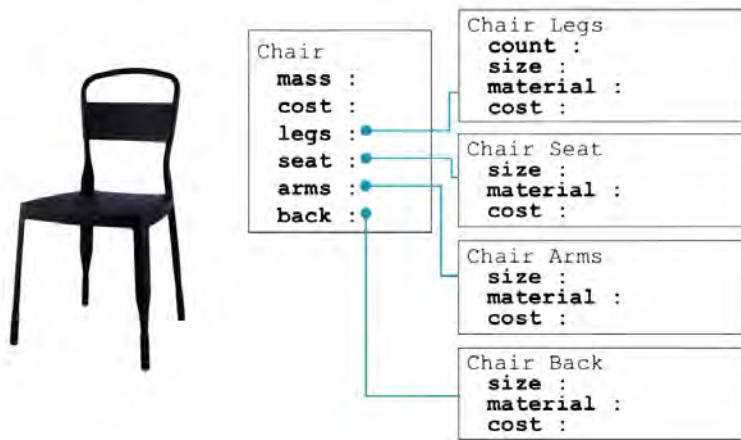


Figure 20-4 Designing a chair using slots and configuration.

For example, the frame for the legs of the chair includes slots for the number of legs (count), their dimensions (size), the material from which they are made, and the cost implications of these choices. This frame isn't just a repository of information; it's an active tool for reasoning. It allows us to explore various configurations by altering the values assigned to these slots, ensuring each configuration adheres to both specific requirements and broader constraints.

Here's how the configuration might unfold:

1. **Defining Attributes:** We initiate by defining the attributes for each component:
 - Legs: Number of legs, material (e.g., wood, metal), size, and cost.
 - Seat: Material, dimensions, and cost.
 - Arms: Presence or absence (as some chairs are armless), material, size, and cost.
 - Back: Material, height, and cost.
2. **Applying Constraints:** The process involves applying constraints that might affect all decisions about the chair's design. For instance:
 - Global Constraints: Such as the total mass and cost of the chair. These constraints ensure the chair is not only functional and comfortable but also economical to produce.
 - Material Constraints: Specific requirements like the use of a sustainable material or a material that meets certain durability standards.
3. **Iterative Refinement:** Each choice might lead to a series of refinements. For example, if the initial selection of wood for the legs makes the chair too heavy or expensive, alternative materials or adjustments to the design might be considered.
4. **Integration and Finalization:** The final step involves integrating all selected attributes into a cohesive design that meets all the functional and aesthetic requirements, as well as cost and material constraints.

This structured approach allows for a meticulous exploration of various design possibilities. By filling out each slot in the frame with appropriate values and continuously refining these choices, the configuration process evolves from abstract ideas to concrete solutions. The framework is dynamic; as solutions are evaluated, specifications can be adjusted – a two-way process that exemplifies the iterative nature of design. For instance, if a prototype reveals that the chair is uncomfortable, adjustments can be made to the back or seat dimensions, or a different material can be chosen to enhance comfort without compromising on cost or aesthetics.

Through this detailed configuration process, what begins as a simple set of specifications evolves into a complex, refined product. This method showcases how structured representation and reasoned adjustments converge to solve practical design challenges, making it a cornerstone technique in fields ranging from furniture design to complex engineering projects.

Implementing Configuration Design in Chair Manufacturing

In configuration design, detailed knowledge of each component is paramount. This approach allows for each aspect of a chair – legs, seat, arms, and back – to be understood not only in basic terms but also in the variety of their possible attributes such as size, material, and cost.

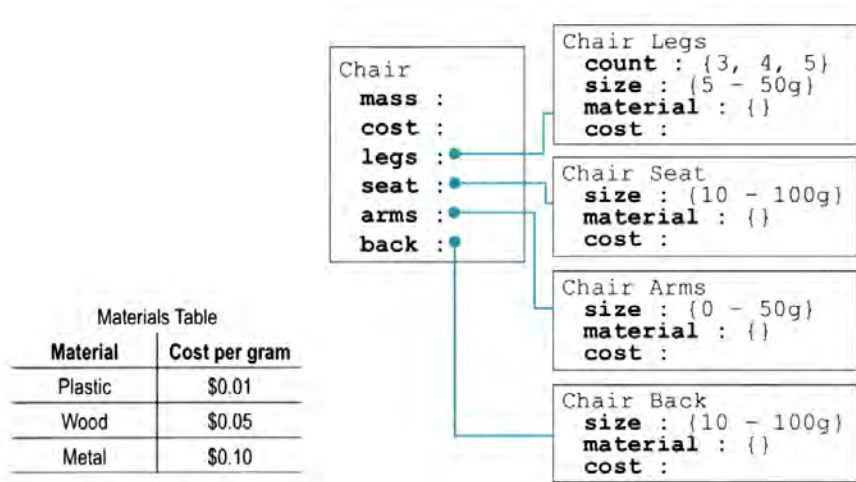


Figure 20-5 Using configuration find fillers for the slots representing a chair

Consider the specific attributes for the components of a chair:

- Legs: The number of legs can vary from 3 to 5, with each leg weighing between 5 and 50 grams.
- Seat: The weight of the seat ranges from 10 to 100 grams.
- Arms: These components can weigh between 0 and 50 grams, accommodating designs both with and without arms.
- Back: Similar to the seat, the back's weight ranges from 10 to 100 grams.

Materials for these components are defined as plastic, wood, or metal, with costs associated per gram outlined in a materials table:

- Plastic: \$0.01 per gram
- Wood: \$0.05 per gram
- Metal: \$0.10 per gram

This structured definition enables a dynamic configuration process where each component can be adjusted within its range to meet overall design specifications. The process effectively links the cost and mass to the chosen material and size specifications. By specifying the size in grams, it simplifies calculating the cost based on the selected material, thereby facilitating a clear correlation between component size and cost. This aids in managing the budget and material selection efficiently.

Example Application: For instance, if the chosen specification for a chair's seat is 50 grams and wood is selected as the material, the cost calculation is straightforward – 50 grams at \$0.05 per gram results in a cost of \$2.50 for the seat. This method allows for the optimization of design decisions, balancing functional requirements with financial considerations.

The configuration design framework is essential in manufacturing, enabling a systematic approach to product customization that aligns practicality with innovation. This structured process not only supports the creation of a product that meets predefined specifications but also enhances the designer's ability to experiment within set parameters, resulting in a functional, market-ready product that adheres to both consumer needs and manufacturing capabilities.

Configuring a Chair to Meet Customer Specifications

In configuration design, the process begins by clearly understanding and defining the constraints of the project based on customer specifications. Here, the customer has ordered a chair with specific requirements: it must weigh over 200 grams, cost no more than \$20, and have exactly four legs. The configuration process utilizes this information to methodically assign values to each component's variables to ensure all specifications are met.

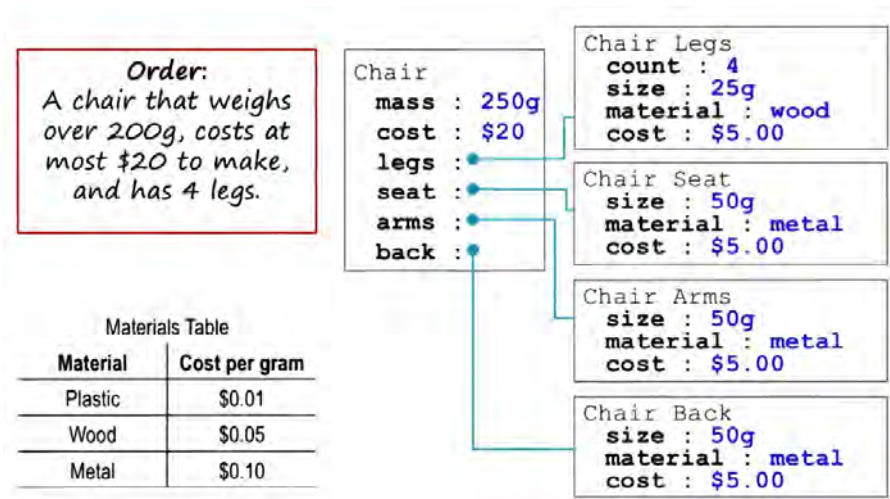


Figure 20-6 Using Configuration to customize the design of a chair

- 1. **Setting Initial Constraints:** The process starts by documenting all constraints from the customer's order. These constraints include the chair's minimum weight (over 200 grams), the maximum cost (\$20), and the specific requirement for four legs.
- 2. **Distributing Costs:** An abstract plan is developed where the total allowable cost of \$20 is preliminarily distributed among the main components – legs, seat, arms, and back. This plan aims to allocate no more than \$5 per component to stay within the budget.
- 3. **Detailed Component Specification:**

- Legs: The legs of the chair are determined to be four in number. Choosing wood as the material, each leg is assigned a weight of 25 grams. Given the cost of wood at \$0.05 per gram, this allocation results in a total cost of \$5 for the legs.
 - Seat, Arms, and Back: The seat, arms, and back are designated to be made from metal, weighing 50 grams each, and also costing \$5 each. This selection not only fulfills the weight requirements but also respects the budget constraints while ensuring durability and style.
4. **Refinement and Expansion:** The configuration process transitions into refining and expanding each component's specifications. This phase involves detailed adjustments to the sizes, materials, or costs based on the initial assembly and any discrepancies that may arise during the fitting of parts.
 5. **Evaluation and Iterative Adjustments:** After all components are assembled according to the configuration plan, the complete chair is evaluated against the original specifications. If the chair meets the set constraints for mass and cost, the configuration is deemed successful. Otherwise, adjustments are revisited, which might include altering materials or component dimensions to better align with cost and weight specifications.
 6. **Dynamic Adjustments and Feedback:** It is possible during the evaluation to find that the initial specifications may need revision – for example, if the chair is too close to the upper cost limit or if the weight is not adequately distributed. This may lead to further iterations where materials or designs are adjusted. For instance, if the initial material choice for a component makes it too expensive or too heavy, alternative materials or design adjustments may be considered to achieve compliance with all specified constraints.

This example demonstrates the iterative and dynamic nature of configuration in design, where specifications not only guide the initial setup but also the ongoing adjustments and refinements necessary to achieve a final product that satisfies all requirements. Through this systematic approach, each component's contribution to the chair's functionality and cost-effectiveness is meticulously calculated and adjusted, showcasing the practical application of configuration principles in manufacturing.

Configuration Exercise: Designing a Chair with New Constraints

In this exercise, we'll design a chair based on specific requirements. The input specification is a chair that costs at most \$16 to make and has a 100-gram metal seat. Please fill out the values of all of these boxes. Try to use the configuration process we just discussed. Let's join David and Ashok as they walk through this specific order.

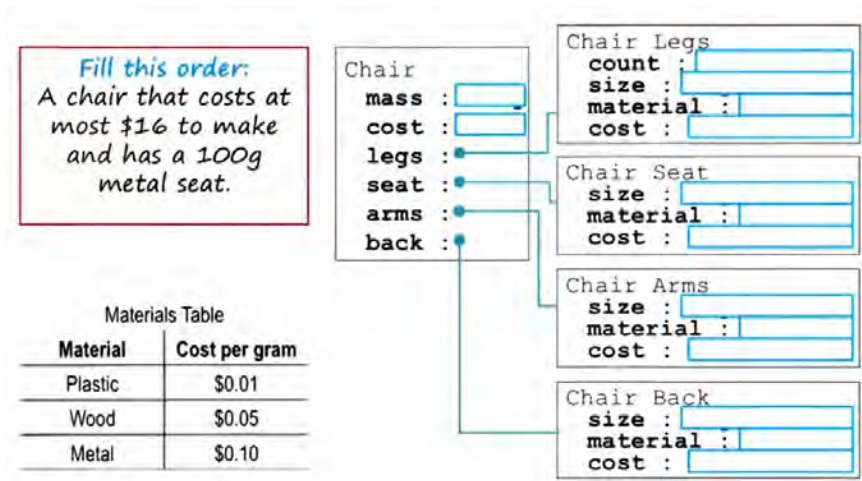


Figure 20-7 Example with a new constraint with a cost maximum of \$16.

Ashok So David, how did you configure the chair?

David Here’s the final setup for this chair. It weighs 160 grams and cost exactly \$16, with the structure supported by four metal legs. I omitted arms to reduce costs and added a metal back. The crucial element, a 100-gram metal seat, complies with the specifications. Initially, I jotted down the essential constraints: the chair must cost less than \$16 and include a 100-gram metal seat. Calculating the cost of the seat, 100 grams at \$0.10 per gram amounts to \$10.

With most of our budget already allocated, I used a cost-minimization heuristic. Since the arms, ranging from 0 to 50 grams, weren't mandatory, I decided against them to conserve funds, bringing their cost to zero. This decision directly influenced the remaining budget allocations.

For the legs, the planning heuristic suggested four metal legs weighing 10 grams each, amounting to \$4. This left us with \$2 for the back. Aligning materials for aesthetic and structural consistency, I chose metal for the back as well, which, given the remaining budget, meant it could only weigh 20 grams.

With all components assigned and costs meticulously calculated, the chair's total mass stands at 160 grams, costing exactly \$16, meeting all set constraints including the necessity for a 100-gram metal seat.

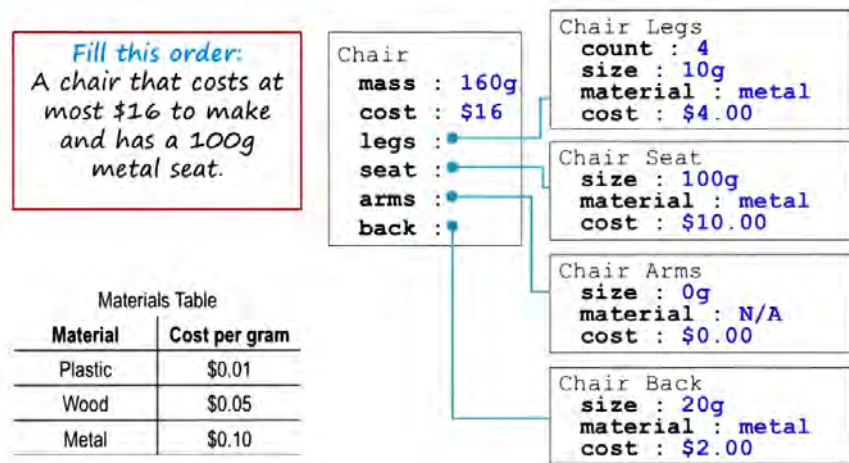


Figure 20-8 Completed example with the constraint of \$16 in total cost.

Ashok That's well executed, David.

David's approach to configuring the chair highlights the diverse kinds of knowledge required to solve a design problem effectively. First, he drew upon his foundational understanding of a generic chair. This includes knowing what components the chair consists of – legs, seat, arms, and back – and the various slots, or attributes, each component can have, such as size, material, and cost. While not all the specific values were predetermined, David's familiarity with the structure of a typical chair gave him the necessary framework to begin assigning values to these variables. This foundational knowledge is essential in configuration tasks, allowing for a structured starting point even when not all information is immediately available.

In addition to this foundational knowledge, David also relied heavily on heuristic methods to guide his decision-making. A heuristic, as we've discussed before, is essentially a rule of thumb or a practical approach to problem-solving based on experience and general principles. In this case, David applied heuristic knowledge in balancing costs and assigning priorities to various parts of the chair. For example, he recognized that reducing the size or eliminating the arms could help stay within the budget constraints. Heuristics allow a designer to navigate the complexity of design decisions by simplifying choices, particularly when resources such as time, money, or materials are limited. These decision-making shortcuts are a crucial tool for efficient problem-solving.

A deeper layer of knowledge at play here is how David decomposed the chair into its individual components and tackled them one by one. Rather than viewing the chair as a single entity, he broke it down into parts – legs, seat, arms, back – and addressed each component's variables separately. This is a key principle in knowledge-based AI and problem-solving in

general. Decomposing a problem allows for each part to be addressed more efficiently, as it simplifies what could otherwise be an overwhelming set of choices. This methodical breakdown is critical when configuring objects with many interconnected variables, as it ensures no element is overlooked while remaining within the constraints provided.

Finally, David's process closely mirrors the concept of constraint propagation, which was discussed earlier in this book. As he progressed through the design, constraints from the specification, such as the maximum allowed cost and the weight of the chair, cascaded down through his decisions at each level. For instance, once the material and weight of the seat were set, the cost of the other components had to be adjusted accordingly to meet the overall \$16 budget. This process of adjusting variables based on higher-level constraints is a key part of configuration design. The interaction between the plan's abstraction levels allows the designer to refine and adjust choices iteratively, ensuring that the final product satisfies all specified requirements.

David's use of these different kinds of knowledge demonstrates how effective configuration design is a blend of structured foundational understanding, practical heuristics, and systematic problem decomposition. By combining these strategies, he was able to navigate the constraints of cost, material, and component size to produce a chair that met the customer's specifications. This example underscores how critical it is in knowledge-based AI to apply multiple types of knowledge in parallel to achieve a solution that is both efficient and meets all requirements.

INTEGRATION WITH CONFIGURATION AND OTHER KBAI TECHNIQUES

In this section, we will explore how configuration interacts with key concepts in Knowledge-Based AI (KBAI). Specifically, we will look at how configuration connects with Classification, Case-Based Reasoning, and Planning. Each of these approaches plays a significant role in shaping the way we think about and implement configuration, whether by organizing knowledge into prototypes, leveraging past solutions, or planning the steps needed to achieve a goal. By examining these links, we gain a deeper understanding of how configuration serves as both a task and a method within the broader KBAI framework.

Configuration and Classification

Configuration and classification share many similarities, particularly in their hierarchical structures and their step-by-step processes. Both approaches rely on building up from more abstract, general concepts to specific details. In classification, we begin by defining broader categories, then gradually refine those categories as we encounter new data, progressively working toward more specific classifications. Configuration, on the other hand, begins with an

abstract plan that is instantiated, assigning values to variables, and then refined and expanded as more constraints and details are considered.

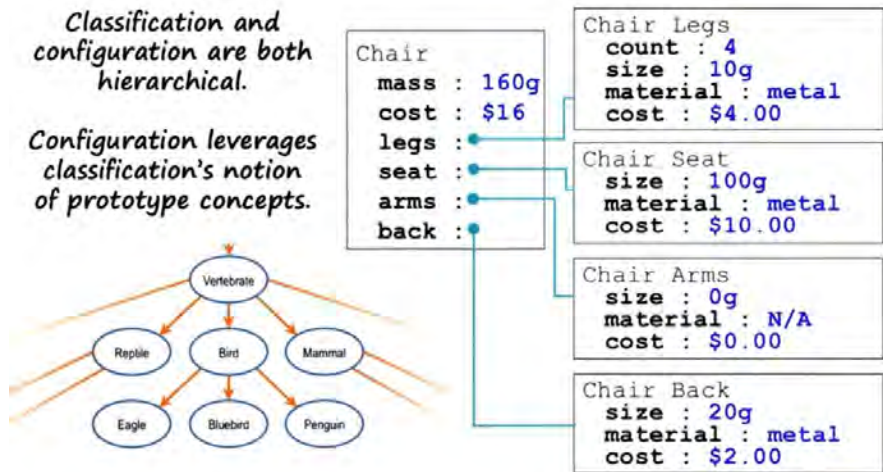


Figure 20-9 Classification and Configuration.

One key connection between the two is how configuration leverages classification's idea of prototypical concepts. For instance, in configuring a chair, there are common, prototypical components such as legs, a seat, arms, and a back. These components, like the classification categories in a hierarchy, help structure the design process. Each component has a range of possible values – such as the material or size for the legs – that act as the default or expected values for that component. The hierarchy of plans in configuration organizes around these prototypical concepts, allowing the designer to fill in specifics based on constraints.

Despite these similarities, classification and configuration serve different purposes. Classification is fundamentally about making sense of the world by organizing and mapping sensory data or observations into predefined equivalence classes. It is a cognitive process of perception. Configuration, on the other hand, is about acting on the world by designing or assembling components to meet certain specifications. It focuses on creation and problem-solving. In a sense, classification is about understanding the world as it is, while configuration is about transforming the world to meet a particular goal.

Classification provides the conceptual framework for understanding and organizing knowledge, while configuration builds upon that knowledge to create functional objects or solutions. This relationship highlights how both processes complement each other in problem-solving, with classification helping to make sense of what is given and configuration focusing on constructing what is needed.

Configuration and Case-Based Reasoning

Configuration and case-based reasoning are two distinct but related approaches to solving routine design problems.²²¹ These are the kinds of problems that are frequently encountered in the real world – whether it's designing chairs or configuring a computer system – making them predictable and familiar. However, while both methods leverage past knowledge and experience, they approach problem-solving differently, each with its unique strengths.

In configuration, the process begins with a general, prototypical concept. Take the example of designing a chair: we already know the basic components – legs, seat, arms, back – and the task is to assign specific values to these variables, such as the material, size, and quantity. This is a structured process where, once the prototypical components are in place, the design is refined and expanded to meet given constraints like cost, mass, or material limits. Configuration assumes that we've already designed enough chairs (or similar objects) in the past that we can abstract a hierarchy of plans from those experiences. The AI agent can then follow this plan hierarchy, instantiating and adjusting as necessary. It's essentially a process of assembling predefined elements in a way that adheres to the given constraints, relying on past designs to guide the creation of new ones.

Case-based reasoning, on the other hand, takes a more adaptive approach.⁹³ Instead of starting from a prototypical plan, it retrieves a specific, previously designed chair (or another object) from memory. The AI system examines the attributes and variables of that prior design and tweaks it to fit the current problem. For instance, if a chair design from the past closely matches the current requirements but needs slight modifications in terms of material or cost, the system will adjust the existing design rather than start from scratch. Case-based reasoning assumes that there's a rich library of previous designs in memory and focuses on modifying a past success to fit new specifications. This approach is less about building from a predefined template and more about adapting what already works.

When faced with a new design problem, an AI agent must decide which approach to use. If it applies configuration, it draws on its plan abstraction hierarchy to build a solution by assigning values to variables and refining them as needed. If it employs case-based reasoning, it retrieves the closest match from memory and tweaks that previous case to fit the new constraints. The method selected will depend on the nature of the problem and the resources available in memory.

Both methods have their advantages. Configuration is systematic and structured, while case-based reasoning is adaptable and flexible.²²² These two approaches represent different ways of utilizing past knowledge to solve routine problems, but both are powerful tools when it comes to design tasks.

In our broader exploration of AI, we can liken this process to constructing a metaphorical periodic table of intelligence. Just as the chemical periodic table was a groundbreaking discovery in organizing elements based on their properties, we aim to categorize the various tasks and methods that AI agents use. Configuration and case-based reasoning are just two of these methods, each addressing specific kinds of tasks. By understanding how these different

approaches fit into the larger framework of AI problem-solving, we can better grasp how AI systems can select the best method for any given task.

Configuration and Planning

The process of configuration is closely related to planning, as both involve organizing actions or decisions into a structured sequence. In the context of configuration, we can think of planning as the foundation upon which the entire process is built. A planner typically generates an abstract plan that resides within a plan abstraction hierarchy. This initial plan is skeletal – it specifies variables but does not yet assign specific values. The skeletal plan outlines the structure of what needs to be accomplished but leaves the details to be filled in later by the configuration process.

The configuration process works by taking these skeletal plans, organizing them into a hierarchy, and iteratively refining and expanding them until the final arrangement is complete. It assigns specific values to the variables, ensuring that the configuration satisfies all given constraints. As the configuration progresses, the abstract plan becomes increasingly concrete, with the variables instantiated and refined step by step.

An important part of this process is how closely configuration mirrors planning. Just as a planner may devise an initial broad strategy that is later fleshed out, the configuration process begins with a high-level plan that eventually gets filled in with all the necessary details. The configuration approach also benefits from the abstraction hierarchy, which allows for organized refinement. Each level in the hierarchy corresponds to a more detailed specification of the task. For instance, the initial plan might specify components like “legs” or “seat” for a chair, but the refinement process assigns specific attributes like “metal legs weighing 10 grams.”

This relationship between configuration and planning highlights how configuration can serve as a kind of script for designing physical objects or systems. The plan is initially broad, but through configuration, it is transformed into something more concrete and actionable. This process is not one-way; configuration can sometimes reveal the need to modify the original plan, much like planning itself may be adjusted as new information comes to light. Thus, planning and configuration are dynamic processes that work together, evolving in tandem to meet the needs of the task.

Furthermore, plans themselves are not static entities. In knowledge-based AI, these plans can be learned and refined over time, using methods such as incremental concept learning. This allows an AI agent to improve its problem-solving capabilities through experience. As the AI agent encounters new challenges, it can adapt its plans or create new ones based on what it has learned. The plan abstraction hierarchy – essential to configuration – grows richer and more comprehensive as the AI gains more experience with different kinds of tasks.

This leads to one of the broader goals of knowledge-based AI: identifying what kinds of knowledge need to be learned and determining how to best learn them. By examining the configuration process, we uncover different types of knowledge that an AI agent must acquire – knowledge about components, abstract plans, and the process of assigning values to variables.

Configuration, therefore, becomes an essential task in understanding the various forms of knowledge that AI must target for learning.

In essence, the deep connection between planning and configuration serves as a key mechanism for solving complex problems in knowledge-based AI. As AI systems become more adept at learning plans, refining configurations, and adjusting to new situations, they gain the ability to operate more efficiently and flexibly in a wide range of design tasks.

Configuration and the KBAI Cognitive Architecture

To link configuration with the broader Knowledge-Based AI (KBAI) architecture, we refer back to the cognitive system diagram. Here, memory plays a crucial role, storing both knowledge of prototypical concepts – such as a chair’s components – and the abstraction hierarchy of design plans. When a new design problem is presented, this stored knowledge is activated by the reasoning component, which then begins the process of instantiation, refinement, and expansion.

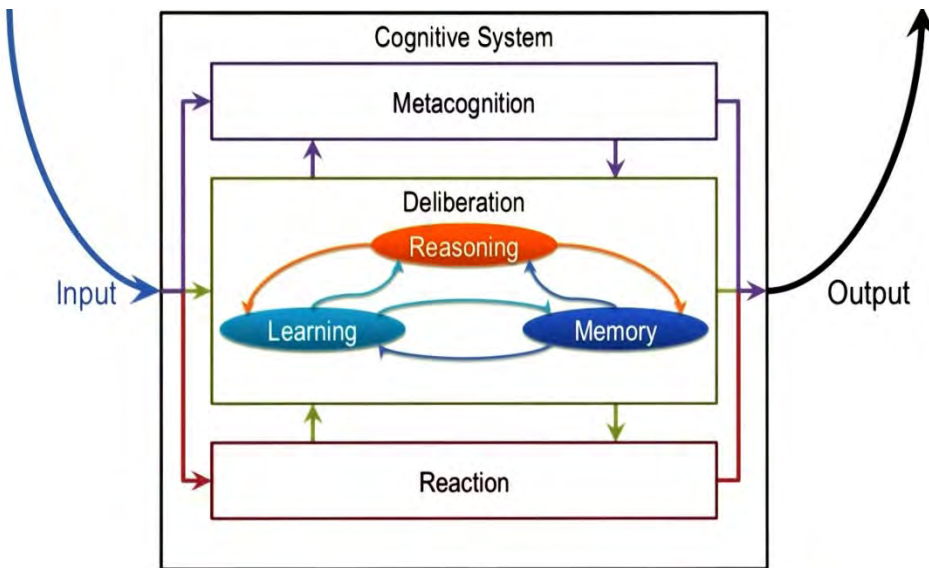


Figure 20-10 The KBAI Architecture and Configuration

For example, when tasked with configuring a chair, the reasoning system retrieves relevant stored information from memory, including the prototypical variables (like the size of the seat, number of legs, or type of material). Using this knowledge, the reasoning component proceeds

through the configuration process, adjusting these variables to meet the input constraints (such as weight or cost).

This iterative process doesn't occur in isolation – learning is a key part of the overall architecture. Through repeated examples of configuration tasks, the system refines its knowledge. For instance, after solving several similar problems, the AI system builds a more robust understanding of the relationships between different components and how to optimize variables in future configurations. This accumulated knowledge is then stored in memory for future use, ensuring that the system becomes more efficient with each new task.

In essence, the configuration process involves a dynamic interaction between reasoning, memory, and learning. The reasoning component draws on past knowledge, memory retrieves stored plans and concepts, and learning continuously updates the knowledge base with new insights. This integration allows the system to solve routine design problems with increasing sophistication, handling new configurations with greater speed and accuracy over time.

CONFIGURATION AND COGNITION

Design is an essential cognitive activity, often cited as one of the most valuable in terms of its economic and practical impact. Within the broad scope of design, configuration is a type of routine design we perform every day. Configuration involves determining the arrangement of known components to satisfy specific constraints, a process that is so common we often overlook its significance. Whether we're driving to work, organizing our workspace, or cooking a meal, we're constantly engaged in tasks of configuration.

For instance, when you run errands, you already know the roads, the vehicle you'll be driving, and the general traffic patterns. These elements are the constants in your problem space. But you still need to configure the best route – perhaps to minimize travel time or gas consumption. The process of planning your route is an act of configuration, adjusting the variables (which roads to take and in what order) to meet the constraints of time, traffic, or fuel efficiency.

Another common example of configuration is cooking. When you follow a recipe, you're working with a high-level plan and a set of known ingredients. The recipe itself is a configuration task where you're asked to combine and adjust specific variables – such as ingredient quantities, cooking times, and temperature settings – under the constraint of optimizing for taste or dietary preferences. The components are familiar, but the configuration varies depending on the desired outcome.

A key insight in understanding configuration from a cognitive perspective is the separation of task from method. The task of configuration involves organizing components within known constraints, while the method refers to how we go about solving the configuration problem. Multiple methods can be used to address the same configuration task. For example, you might use Case-Based Reasoning, where you draw upon past experiences of similar configurations to inform your current decisions. Alternatively, you could use Plan Refinement, starting with an

abstract plan and iteratively refining it based on the constraints. Another approach could be Generate-and-Test, where different configurations are created and evaluated until the best one is found.

Cognition is deeply involved in the configuration process because it requires abstract thinking, the application of past experiences, and reasoning to adjust variables within specific constraints. The knowledge we draw upon in these scenarios can be generalized knowledge, such as a common understanding of traffic patterns or basic cooking techniques. Or it can be specific knowledge, such as how best to configure a recipe to suit a particular dietary need.

What makes configuration particularly valuable as a cognitive task is its flexibility. Even though the components are fixed, the way we arrange them can differ greatly depending on the goals and constraints at hand. This ability to configure solutions in real-time to meet various objectives is a fundamental aspect of human cognition. Every day, whether we are aware of it or not, we engage in numerous acts of configuration, solving practical problems by optimizing and arranging the world around us to meet our needs.

By recognizing configuration as both a cognitive task and a design method, we can better appreciate the routine but powerful mental processes that enable us to navigate complex environments and optimize for various constraints in real-time.

INTEGRATING KBAI: CONFIGURATION IN THE MODERN AI LANDSCAPE

Configuration, as a core KBAI task, involves assembling a set of pre-defined components to satisfy a given set of constraints and achieve specific goals. This structured approach to design has significant implications and offers valuable synergies when integrated with modern AI methodologies, particularly in areas dominated by machine learning.

- **Bridging Symbolic AI and Machine Learning:**
 - **Knowledge-Infused Learning:** Configuration relies on explicit knowledge about components, their attributes, and constraints. This knowledge can be used to guide machine learning models, for example, by defining valid output spaces for generative models or by providing a symbolic framework to interpret learned representations. A neural network might learn to propose component parameters, while a configuration engine ensures these parameters satisfy all hard constraints.
 - **Explainable AI (XAI):** The step-by-step refinement process in configuration, often based on explicit plans and constraint satisfaction, offers a degree of transparency. If a configuration fails or succeeds, the reasoning path and the constraints involved can often be traced, contributing to more explainable AI systems compared to purely “black-box” approaches.
- **Enhancing Complex System Design and Automation:**

- **Automated Product Customization:** Configuration is central to mass customization (e.g., configuring cars, computers, or software). AI-driven configuration systems can handle vast numbers of components and complex interdependencies, enabling highly personalized products.
- **System Composition and Service Orchestration:** In areas like cloud computing or IoT, configuring complex systems from available services or microservices is a critical task. KBAI configuration techniques can automate this, ensuring that the composed system meets performance, security, and cost constraints.
- **Robotics and Autonomous Systems:** Configuring robot behaviors, toolsets, or operational parameters for specific tasks in dynamic environments is an ongoing challenge. Configuration methods can help in adapting robot capabilities to new tasks and constraints.
- **Improving Efficiency and Optimality in Design:**
 - **Constraint Satisfaction and Optimization:** Configuration tasks are often a form of Constraint Satisfaction Problem (CSP) or Constraint Optimization Problem (COP). Advanced KBAI techniques, including sophisticated search algorithms and constraint propagation (as discussed in the previous chapter), are crucial for finding valid and optimal configurations efficiently.
 - **Generative Design:** When combined with machine learning, configuration principles can support generative design, where AI explores a vast design space to propose novel configurations that humans might not have considered, while still adhering to fundamental constraints.
- **Learning and Adapting Configuration Knowledge:**
 - **Learning from Past Configurations:** Similar to Case-Based Reasoning, AI systems can learn from successful (and unsuccessful) past configurations to improve future design choices, refine abstract plans, or learn new heuristics for prioritizing variables.
 - **Interactive Configuration:** Modern AI can facilitate interactive configuration systems where the user and the AI collaborate. The AI can manage constraints and propose solutions, while the user provides preferences and high-level guidance.

While machine learning excels at pattern recognition and learning from data, KBAI principles like those underlying configuration provide the structure, explicit knowledge representation, and reasoning mechanisms necessary to tackle complex design problems systematically. The integration of these approaches allows for AI systems that are not only data-driven but also knowledge-guided, leading to more robust, explainable, and efficient solutions in a wide array of design and engineering domains. Configuration, therefore, remains a vital concept for building intelligent systems that can actively shape and organize their world.

CHAPTER SUMMARY

This chapter focused on configuration, a type of routine design task encountered when modifying existing plans or assembling known components to meet specific new constraints. Configuration is essential when working with established designs – such as buildings, cars, or computers – where key elements are predefined, but variables like size, cost, or performance need adjustment. The core of configuration lies in arranging these known components to satisfy all requirements and constraints.

We began by defining design broadly as a dynamic process where the problem and solution co-evolve. In contrast, configuration is more constrained, dealing with a fixed set of components and the task of optimizing their arrangement. This is a common activity, seen in everyday tasks from arranging furniture to planning routes.

The process of configuration was detailed, highlighting its similarity to constraint propagation. It typically starts with an abstract or partial solution (a skeletal plan) and proceeds through iterative refinement and expansion. Variables are assigned values, and the design is progressively detailed until all constraints are met. We emphasized the dynamic, two-way nature of this process, allowing for adjustments as the design unfolds.

Key connections between configuration and other KBAI topics were explored. Configuration leverages prototypical concepts from Classification to structure design components. It relates to Case-Based Reasoning in solving routine problems by adapting past solutions, though configuration often starts from abstract plans rather than specific past cases. The strong link to Planning was highlighted: configuration often instantiates and refines skeletal plans from an abstraction hierarchy. The role of heuristics in guiding variable assignment and prioritizing decisions within configuration was also discussed, using examples like chair and computer processor design.

We then considered configuration from a cognitive perspective, noting its prevalence in everyday human problem-solving. The distinction between the task of configuration and the various methods (e.g., CBR, plan refinement, generate-and-test) used to achieve it was emphasized.

Finally, the chapter explored how configuration, as a KBAI principle, integrates with and enhances modern AI. It can infuse machine learning with explicit knowledge, improve explainability, aid in complex system design (like automated customization and service orchestration), and leverage constraint satisfaction techniques for efficiency. The potential for AI to learn configuration knowledge and participate in interactive design further underscores its importance.

This chapter laid the groundwork for understanding how AI can address tasks requiring dynamic, flexible thinking and iterative solution refinement. The next chapter will explore diagnosis, which can be seen as approaching similar problem structures from an analytical rather than a generative direction.

ADDITIONAL READINGS & REFERENCES

- 93 Craw, S. & Aamodt, A. in *Case-Based Reasoning Research and Development: 26th International Conference, ICCBR 2018, Stockholm, Sweden, July 9-12, 2018, Proceedings* 26. 62-77 (Springer).
- 217 Schede, E. *et al.* A survey of methods for automated algorithm configuration. *Journal of Artificial Intelligence Research* **75**, 425-487 (2022).
- 218 Yang, Q., Steinfeld, A., Rosé, C. & Zimmerman, J. in *Proceedings of the 2020 chi conference on human factors in computing systems*. 1-13.
- 219 Wang, H. *et al.* Scientific discovery in the age of artificial intelligence. *Nature* **620**, 47-60 (2023).
- 220 Van Beek, P. (Elsevier, Amsterdam, 2006).
- 221 Tseng, H.-E., Chang, C.-C. & Chang, S.-H. Applying case-based reasoning for product configuration in mass customization environments. *Expert Systems with Applications* **29**, 913-925 (2005).
- 222 Eggensperger, K., Lindauer, M. & Hutter, F. Pitfalls and best practices in algorithm configuration. *Journal of Artificial Intelligence Research* **64**, 861-893 (2019).

Chapter 21

Diagnosis

INTRODUCTION TO DIAGNOSIS

In this chapter, we explore the critical concept of diagnosis within the context of Knowledge-Based Artificial Intelligence. Diagnosis involves identifying the faults that cause a system to malfunction, whether it's a mechanical system like a car, a software program, a biological organism, or even complex economic systems.

Building on our discussions about classification and configuration, we will deepen our understanding by introducing two specific areas: the data space and the hypothesis space. The data space consists of observable information about the system's malfunction, while the hypothesis space encompasses potential explanations for these malfunctions. Our exploration will focus on mapping information from the data space into the hypothesis space, which is central to the process of diagnostic reasoning.

Additionally, we will examine diagnosis from two distinct perspectives: as a form of classification and as a process of abduction. While classification in diagnosis involves categorizing faults based on observed symptoms or malfunctions, abduction involves inferring the most likely causes based on the effects observed. This dual approach not only broadens the scope of diagnosis but also deepens the understanding and application of these concepts.

Throughout this chapter, our goal is to provide a comprehensive framework that not only defines and explains diagnosis but also equips you with the methodologies to apply these

concepts across various domains, enhancing both your theoretical understanding and practical application skills.

INTRODUCTION TO THE CONCEPT OF DIAGNOSIS
THROUGH A MEDICAL EXERCISE

Diagnosis is a fundamental concept that extends across various fields, but it is most commonly associated with medicine. To better understand the diagnostic process, we engage in a simulated exercise, mirroring the complex decision-making that doctors face.

Patient:

A: Normal

B: High

C: Low

D: Normal

E: Normal

F: Normal

G: Normal

H: Low

What illness (or set of illnesses) would you use to diagnose this patient?

Illnesses:

- Alphaitis: Elevated A, Reduced C, Elevated F
- Betatosis: Elevated B, Reduced C, Elevated E, Reduced H
- Gammanoma: Elevated D, Elevated E, Elevated F
- Deltacol: Elevated B, Reduced C
- Epsicusus: Reduced H
- Zetad: Elevated B, Reduced C, Reduced E, Reduced F
- Etaemia: Elevated A, Reduced D, Reduced H
- Thetadesis: Elevated B, Reduced C, Reduced H
- Iotalgia: Elevated A, Reduced E, Elevated F, Elevated G
- Kappacide: Reduced A, Reduced F, Reduced G
- Lambdacrite: Reduced A, Reduced E, Reduced F, Reduced G
- Mutenson: Elevated A, Elevated G

Figure 21-1 Diagnosing fictional diseases.

In this exercise, we consider a hypothetical patient who presents with specific symptoms, alongside a list of fictional diseases, each associated with distinct symptom patterns. For example, the disease “Alphaitis” is identified by elevated levels of symptom A, reduced levels of symptom C, and elevated levels of symptom F. The list includes other diseases such as “Betatosis,” “Gammanoma,” and “Deltacol,” each contributing its unique set of symptoms.

Your challenge is to analyze the symptoms exhibited by the patient and match them to the potential diseases. This task will introduce you to the diagnostic concepts of data and hypothesis spaces. You will need to map the observed symptoms (data space) to potential causes (hypothesis space), engaging in a process that involves both classification and abduction – terms we will explore in depth throughout this chapter.

This exercise not only serves to deepen your understanding of medical diagnostic reasoning but also sets the foundation for discussing diagnosis in broader contexts, such as mechanical

repairs, software troubleshooting, and economic analysis. Through this example, we aim to illustrate the intricate nature of diagnosis, demonstrating its application in various domains beyond healthcare.

Let's join David and Ashok as they consider a challenging diagnostic problem involving a patient and a list of fictional diseases, each associated with specific symptoms.

Ashok What do you think, David?

David So, I chose Thetadesis as the only illness necessary to explain these symptoms. I did notice that Betatosis explains three of the four symptoms, but the patient doesn't exhibit its other symptoms. Additionally, the combination of Detacol and Epsiculus could also explain exactly those three symptoms. But generally, I think I would prefer to have one illness that explains the symptoms instead of two. I could also explain it with a combination of Betatosis, Iotalgia, and Kappacide because, although the patient has a normal level of A, Iotalgia elevates A and Kappacide reduces A. But again, that approach is much more complicated than explaining everything through a single illness, Thetadesis.

Ashok That's a good answer, David.

Understanding David's Diagnostic Strategy

David's approach in diagnosing the hypothetical patient highlights several foundational principles of diagnostic reasoning, essential for understanding the complexities and methodologies involved.

Firstly, David's selection of Thetadesis as the sole necessary illness exemplifies the principle of coverage. He ensures that his diagnosis accounts for all presented symptoms. This comprehensive coverage is crucial as it prevents any symptoms from being overlooked, thereby avoiding misdiagnosis.

Secondly, David preferred a single hypothesis over a combination of possible diseases, such as Betatosis combined with Iotalgia and Kappacide. This preference aligns with the principle of parsimony, advocating for simplicity in diagnostic explanations. Opting for a simpler hypothesis that still covers all symptoms is favored because it minimizes complexity and potential errors in the diagnostic process.

Thirdly, David's choice reflects the understanding that hypotheses can interact in complex ways. These interactions, where symptoms could be explained by different combinations of diseases, significantly complicate the diagnostic task. Such complexity necessitates a careful evaluation of how different conditions might interrelate and impact each other's manifestation in symptoms.

Lastly, the emphasis on providing a coherent explanation that links symptoms to their underlying causes is a critical component of diagnostic reasoning. It is essential not just to match symptoms with conditions but also to articulate a convincing rationale for why these symptoms manifest under a specific diagnosis.

This example, though seemingly straightforward due to the single disease covering all symptoms, raises important questions about handling scenarios where no single hypothesis fully explains all data, or multiple hypotheses could plausibly account for the data equally well. These situations introduce significant challenges to the diagnostic process, requiring a deeper and more nuanced understanding of potential diseases and their interactions.

THE PROCESS OF DIAGNOSIS ACROSS DOMAINS

Diagnosis is the critical process of identifying the underlying cause or causes of a malfunctioning system. This process is essential not only in medical settings but also extends to various other domains where systems are expected to function according to predefined behaviors. Diagnosis begins by establishing what a system is designed to do – the expected behavior – and observing the actual performance to identify any deviations, which indicate malfunctions.

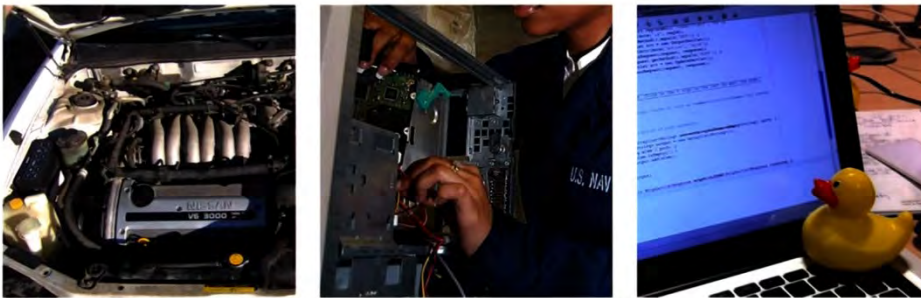


Figure 21-2 Examples of problems requiring diagnosis.

Each system, be it mechanical, biological, or digital, operates based on certain expected behaviors. When these expectations are not met, it signifies a problem. The essence of diagnosis lies in the detection and analysis of discrepancies between what is expected and what is observed. For example, if turning the ignition key in a car doesn't start the engine, there's a clear misalignment between the expected outcome (the engine starts) and the observed outcome (the engine remains off), signaling a malfunction.

Broad Application of Diagnostic Principles

Diagnosis transcends beyond just medical practice into areas like automotive repair, computer hardware troubleshooting, and software debugging:

- In **automotive repair**, diagnosing a car involves a systematic check of components such as the carburetor and spark plugs to isolate the fault. This process often follows a rule-based approach, where each potential problem is methodically tested until the issue is resolved.
- **Computer hardware issues**, such as overheating, prompt diagnostics that might lead to the discovery and replacement of a malfunctioning fan, based on the observed behavior of the system.
- In the domain of **software**, unexpected outputs from a program prompt debugging. Techniques such as rubber duck debugging involve the programmer explaining their logic out loud, potentially to an inanimate object like a rubber duck, to find faults in their reasoning.

Diverse Methods for Effective Diagnosis

The methods employed in diagnosis vary significantly across domains:

- **Rule-based Reasoning** is commonly utilized where systematic checks are applicable, such as in mechanical systems including cars and computer hardware.
- **Case-based Reasoning** leverages past similar instances to diagnose and solve current problems, applying learned solutions from historical data.
- **Model-based Reasoning**, which involves a comprehensive understanding of a system's model, is part of broader systems thinking that we will explore later in this book.

Each diagnostic scenario, whether it concerns car engines, computer malfunctions, or software bugs, involves identifying a discrepancy between expected and actual behaviors and systematically determining the cause through various reasoning methods.

MAPPING BETWEEN DATA SPACE AND HYPOTHESIS SPACE IN DIAGNOSIS

Diagnosis is essentially about mapping observable data from a data space to a hypothesis space. This process is crucial in various fields such as medical diagnosis, automotive repair, and computer software troubleshooting, each employing this mapping to pinpoint the source of a malfunction.

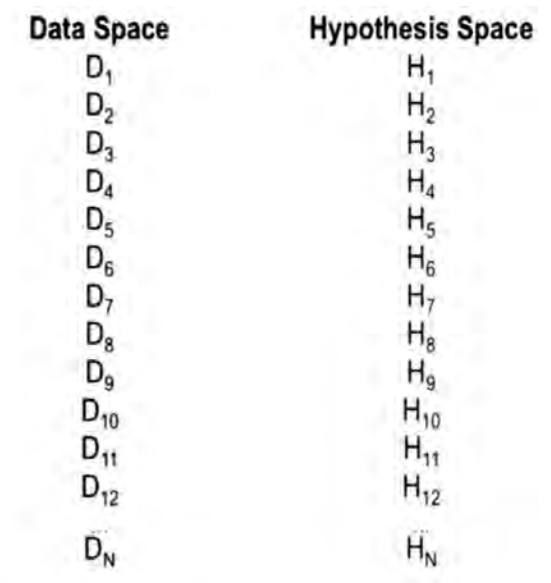


Figure 21-3 Mapping the Data and Hypothesis spaces.

In medical diagnosis, the data space includes all the signs and symptoms a patient presents. For instance, a specific data point might be that Ashok’s temperature is 104 degrees Fahrenheit, indicating a high fever, which is a more generalized symptom. The hypothesis space consists of potential medical conditions that could explain these symptoms. A doctor might hypothesize that Ashok's high fever suggests he is suffering from the flu.

In the context of car repairs, the hypothesis might address particular mechanical failures. For example, if a car doesn’t start, the hypothesis space may include issues like a malfunctioning carburetor. This hypothesis is tested against the observed behavior – namely, the car not starting when the key is turned.

Similarly, in software troubleshooting, hypotheses may involve specific functionalities or components failing to perform as expected. If a software application crashes or returns unexpected outputs, the diagnostic process might explore potential bugs or logical errors in the code.

Complexity of the Diagnostic Process

The diagnostic mapping process is inherently complex due to:

- The broad and diverse nature of data points within the data space.
- The extensive range of possible hypotheses within the hypothesis space.

- The complexity of relationships between data points and hypotheses, where one hypothesis may invalidate another or multiple hypotheses may be needed to fully explain a single set of data points.

Interactions within the hypothesis space further complicate diagnostics. For instance, if hypothesis H3 is confirmed, it may automatically negate the validity of hypothesis H4. Conversely, if hypothesis H5 is validated, it might ensure the presence of H6.

EFFECTIVE DATA ABSTRACTION IN DIAGNOSIS

In diagnosis, handling raw data effectively is essential, yet it is often beneficial to engage with abstractions of that data. This approach simplifies the complex information, making it more manageable to form and refine hypotheses about underlying issues. Here's a detailed exploration of how abstraction aids the diagnostic process across different domains.

Abstraction in Medical Diagnosis

Consider a typical scenario in medical diagnosis. A patient named Ashok presents with a specific symptom: a fever of 104 degrees Fahrenheit. Initially, this specific detail is abstracted to "Ashok has a high fever" for the initial diagnostic purposes. This broadened category serves as a bridge to a more extensive hypothesis space. For example, "Ashok has a high fever" might lead to hypothesizing that "Ashok has a bladder infection." This hypothesis can then be refined further, perhaps to "Ashok suffers from the flu" or more specifically, "Ashok is suffering from a specific strain of the flu." The aim is to refine the hypothesis to precisely match all observed and abstracted data.

Processes of Classification in Diagnosis

The formation of these hypotheses mirrors two key approaches in classification:

- **Bottom-up Classification:** This method begins with the granular, specific data – like individual symptoms or test results – and groups them into broader categories. This aggregated data forms the basis for initial hypotheses.
- **Top-down Classification:** This approach starts with a broad hypothesis or a high-level category and refines it based on additional specific data, tightening the hypothesis to align more closely with the detailed data at hand.

In practice, the diagnostic process typically involves a dynamic interplay between these methods, often termed heuristic classification. This strategy integrates both bottom-up and top-down approaches, navigating efficiently from a broad dataspace to a focused hypothesis space.

Heuristic Classification in Diagnostic Tasks

Heuristic classification is particularly effective in diagnostic contexts because it allows for flexible interpretation of data and generation of hypotheses. It accommodates real-world complexities where data may be incomplete or ambiguous, and initial hypotheses may require significant adjustment as new information becomes available.

This methodological framework is not only applicable in the medical field but extends across various domains, such as auto mechanics and software troubleshooting. Regardless of the domain, the goal is to develop a well-supported hypothesis that comprehensively accounts for all data, refined through iterative processes of abstraction and specification. This approach not only streamlines the diagnostic process but also enhances the accuracy and reliability of the outcomes, leading to more effective solutions and interventions.

THE INTRICACIES OF DIAGNOSTIC CLASSIFICATION IN AI

Diagnosis in artificial intelligence (AI) is a complex task that involves multiple variables and conditions, reflecting the sophisticated interplay between observed data and potential hypotheses. This process, while central to AI's problem-solving capabilities, encounters numerous challenges that can complicate the path to a correct diagnosis.

One significant challenge in diagnosis is the possibility of multiple hypotheses explaining a single data point. For example, if Ashok visits a doctor with a high fever – a symptom labeled as D5 – it could be attributed to various diseases. Deciding which of these diseases, or hypotheses, actually explains the high fever is a crucial step in the diagnostic process. The complication arises when the chosen hypothesis must explain not only the high fever but also any other concurrent symptoms Ashok might have, such as fatigue or shivering.

The process of diagnosis involves mapping from a set of data points to a set of hypotheses and vice versa. This mapping is complex due to the vastness of the data and hypothesis spaces and the multifaceted nature of potential mappings. For instance, hypothesis H3 suggesting Ashok has influenza might cover symptoms like fever and tiredness, but additional symptoms expected from influenza might lead the doctor to probe further, querying other related symptoms to confirm the diagnosis.

Interactions between hypotheses add another layer of complexity. Mutual exclusion is a common interaction where one hypothesis being true makes another false. If two hypotheses, say H3 and H6, explain different sets of symptoms but are mutually exclusive, the doctor faces a dilemma if Ashok shows symptoms explained by both hypotheses. The decision on which hypothesis to pursue depends on a careful evaluation of all presented symptoms.

Cancellation occurs when the effects of one condition obscure or negate the symptoms of another, complicating the diagnostic process. For example, if Ashok has an infection that typically causes fever but also has a condition that suppresses fever, his actual body temperature

might not reflect the underlying infection. Such cancellations require the doctor to consider less obvious connections between symptoms and potential diagnoses.

Given these complexities, shifting the diagnostic perspective from simple classification to abduction is beneficial. Abduction involves formulating hypotheses that can comprehensively explain the observed data, accommodating the nuances of complex interactions and cancellation effects among symptoms.

This nuanced approach acknowledges that diagnosis is more than labeling symptoms; it's about understanding the underlying interactions and ensuring that the hypotheses not only address the symptoms but also cohere with all known data. This comprehensive approach aims to ensure that diagnostic conclusions are as accurate and effective as possible, reflecting the true complexity of the task at hand.

EXPLORING ABDUCTION WITHIN DEDUCTIVE AND INDUCTIVE REASONING FRAMEWORKS

In the domain of logical reasoning, understanding the distinct yet interrelated processes of deduction, induction, and abduction is crucial for both everyday decision-making and complex diagnostic evaluations, particularly in the field of artificial intelligence (AI).^{223,224}

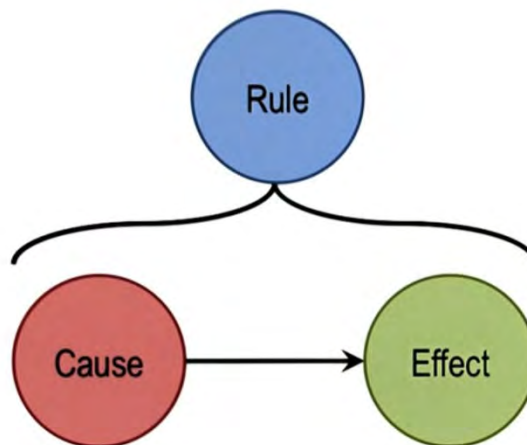


Figure 21-4 Logic relationship diagram: Deduction uses a rule and cause to determine an effect. Induction uses a cause and effect to produce a potential rule. Abduction uses a rule and effect to identify a potential cause.

Deduction: The Certainty of Logical Consequence

Deduction is a method of reasoning where conclusions are necessarily derived from premises through established rules. For example, consider the straightforward rule: “If it is cloudy, it rains.” Here, “it is cloudy” acts as the cause, and “it rains” as the effect. Given the cloudy condition, we can confidently deduce that it will rain, provided the rule holds true. This deductive reasoning applies to various scenarios, such as the predictable behavior of Bob towards Joe based on their personal dynamics, or medically, predicting fever in a patient diagnosed with the flu. This process follows the principle of *modus ponens*, ensuring that if the premises are accurate, the conclusion is also guaranteed.

Induction: Creating General Rules from Observations

Induction involves formulating general rules based on observed instances and patterns, without the absolute certainty provided by deduction. For instance, noticing a frequent correlation between cloudy skies and rainfall might lead to the generalized rule that cloudy conditions often result in rain. Similarly, observing Bob's consistent departure when Joe arrives might suggest a general dislike. In a medical context, a doctor might induce that flu typically causes fever after seeing multiple cases exhibit this pattern. However, induction does not confirm the truth of these rules; it only suggests a likely connection based on the evidence at hand.

Abduction: Reasoning from Effect to Possible Cause

Abduction involves hypothesizing the most probable cause based on the observed effects, assuming a known rule is in effect. It plays a crucial role in diagnosis across various fields. For instance, if rain is observed and the rule is “If it is cloudy, then it rains,” one might abduce that it is likely cloudy. This reasoning is extended in medical diagnosis, where symptoms like fever may lead to hypothesizing flu under the rule “If flu, then fever.”

However, abduction, like induction, does not guarantee the truth. Observing rain does not conclusively prove it's cloudy, as other factors could produce rain. Similarly, a fever in Ashok does not definitively indicate flu since multiple conditions can cause fever.

Diagnostic Implications of Logical Reasoning

In practical scenarios, these reasoning forms are often used in conjunction to navigate complex decision-making processes.²²⁵ This integration is particularly evident in scientific and medical diagnostics, where hypotheses about causes (abduction) are formed based on effects, rules are induced from patterns, and deductions are made to predict and verify further outcomes. This cyclical interaction of abduction, induction, and deduction potentially mirrors significant aspects

of cognitive function, suggesting it as a foundational process in human reasoning and AI diagnostics.

Understanding these forms of inference highlights the challenges in AI diagnostics, which go beyond simple classification to involve nuanced reasoning about data and hypotheses. This comprehension is crucial as it underscores the intricate nature of diagnosis, requiring a sophisticated blend of logical processes to effectively manage and interpret the vast arrays of data encountered in various domains.

APPLYING ABDUCTION IN DIAGNOSTIC HYPOTHESIS SELECTION

With our refined understanding of abduction and its role in diagnosis, it's pertinent to explore how this form of reasoning aids in selecting the most suitable hypotheses for explaining discrepancies in data observations. The concept of abduction illuminates the path to diagnosis, emphasizing the selection of hypotheses that most comprehensively explain the observed data.

The Principle of Explanatory Coverage

One of the primary criteria in hypothesis selection is the extent of explanatory coverage. This principle dictates that the chosen hypothesis should account for as much of the observed data as possible. Explanatory coverage ensures that the hypothesis not only addresses the direct symptoms or data points but also encapsulates a broader range of associated observations, thereby reducing the likelihood of overlooking significant connections.

1. Hypotheses must cover as much of the data as possible.

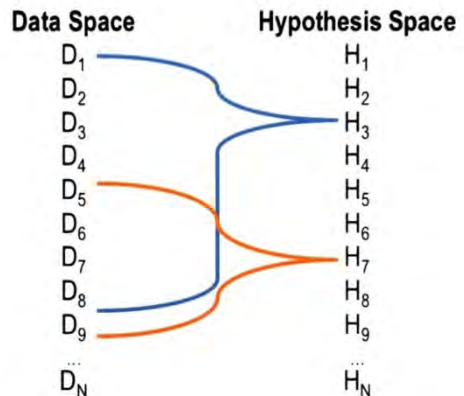


Figure 21-5 Choosing Hypotheses - The Hypothesis must cover as much of the data as possible.

For instance, consider a diagnostic scenario where we analyze the relationships between data points and hypotheses, as illustrated in the accompanying diagram. Hypothesis H3 is shown to explain a broad range of data items, specifically from D1 through D8. In contrast, Hypothesis H7 provides an explanation for a narrower band of data, from D5 to D9. If all data elements from D1 to D9 hold equal significance or salience in the diagnostic evaluation, Hypothesis H3 would be preferable over H7 due to its broader explanatory scope. This preference is grounded in the principle of explanatory coverage: the hypothesis that accounts for more data is often more desirable, assuming it does not introduce extraneous complexities or contradictions.

The Principle of Parsimony in Hypothesis Selection

In diagnostic reasoning, following the initial principle of explanatory coverage, we often employ the principle of Parsimony. This principle, akin to Occam's Razor in scientific methodology, emphasizes simplicity in explanations. It suggests that when faced with multiple competing hypotheses, the simplest one that sufficiently explains the data should be chosen.

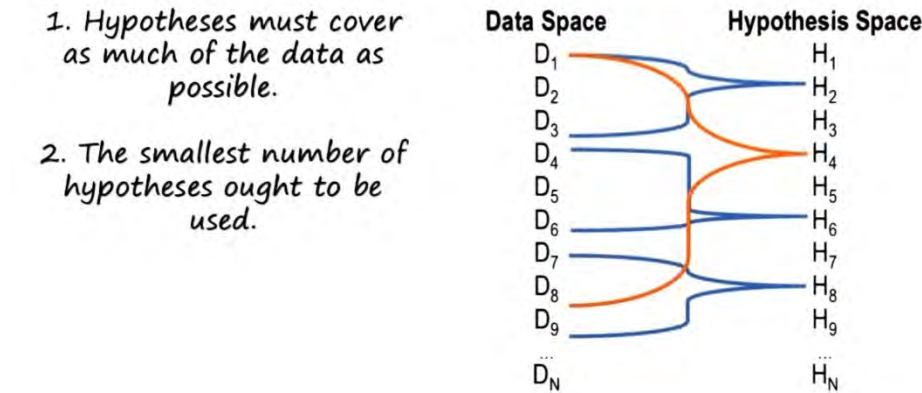


Figure 21-6 Choosing Hypothesis - The smallest number of hypotheses out to be used.

Consider a scenario where we are tasked with explaining a set of data elements based on various hypotheses. Here's how these hypotheses correlate with the data:

- Hypothesis H2 explains data elements D1 to D3.
- Hypothesis H4 covers a broader range, explaining data elements D1 through D8.
- Hypothesis H6 addresses data elements D4 to D6.
- Hypothesis H8 pertains to data elements D7 to D9.

Using the principle of explanatory coverage, one might be tempted to combine H2, H6, and H8 to cover all data points from D1 to D9. While this combination comprehensively addresses

all the data points, it introduces unnecessary complexity by involving multiple hypotheses where fewer could suffice.

In contrast, the principle of Parsimony would guide us to prefer Hypothesis H4. Despite the broad scope of its explanatory range, H4 alone explains nearly all the data points, thereby offering a more streamlined and efficient approach to diagnosis. This choice simplifies the diagnostic process, reducing the potential for error and confusion that could arise from managing multiple overlapping hypotheses.

Balancing Explanatory Coverage and Parsimony in Diagnosis

In diagnostic reasoning, achieving a balance between explanatory coverage and parsimony presents a critical challenge. This balance is essential for developing efficient and comprehensible hypotheses that adequately address all observed data without unnecessary complexity.

Consider the scenario involving hypotheses H2, H4, H6, and H8, where each hypothesis explains different segments of the data. Specifically, H4 covers data elements D1 through D8, and H8 covers D7 to D9. This pairing, H4 combined with H8, ensures complete coverage of all data points, thus fully adhering to the principle of explanatory coverage.

Choosing H4 and H8 together not only ensures comprehensive data coverage but also maintains simplicity by minimizing the number of hypotheses involved:

- **Maximized Coverage:** The combination of H4 and H8 spans the entirety of the observed data, ensuring that no aspect of the dataset is overlooked.
- **Parsimony:** Opting for two hypotheses rather than three (as with the combination of H2, H6, and H8) streamlines the explanation, making it simpler and more manageable. This approach aligns with the parsimony principle, which favors the simplest possible explanation that covers all data.

This methodical selection illustrates how effectively balancing these principles can lead to a more streamlined diagnostic process. By integrating both coverage and simplicity, the diagnostic strategy becomes not only thorough but also elegantly concise, facilitating easier understanding and application in varied diagnostic tasks.

The Principle of Confidence in Hypothesis Selection

When navigating the complex task of selecting between competing hypotheses, the third critical criterion is the Principle of Confidence. This principle underscores the importance of prioritizing hypotheses that are not only consistent with the available data but are also regarded as more credible based on past evidence or reliability.

1. Hypotheses must cover as much of the data as possible.
2. The smallest number of hypotheses ought to be used.
3. Some hypotheses may be more likely than others.

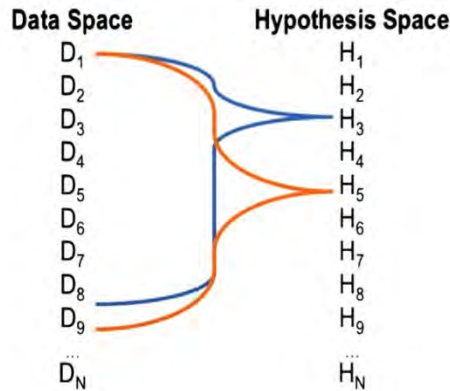


Figure 21-7 Choosing Hypothesis - Some hypotheses may be more likely than others.

This criterion encourages the selection of hypotheses that are deemed more reliable or likely to be accurate. For instance, consider a scenario where hypothesis H_3 accounts for data items D_1 through D_8 , and hypothesis H_5 explains data items D_1 through D_9 . The choice between H_3 and H_5 may extend beyond their explanatory reach. If H_3 has consistently proven accurate in similar contexts or is supported by a stronger theoretical foundation, it might be preferred even though H_5 covers more data points.

In practical diagnostic settings, the principle of confidence is crucial when multiple hypotheses could plausibly account for the observations. The decision often leans towards hypotheses with a higher degree of reliability or those that have been validated through previous applications. For example, if empirical evidence or prior validations lend more support to H_3 compared to H_5 , H_3 becomes the favored choice despite the additional data point explained by H_5 .

It's vital to recognize that these criteria for selecting hypotheses extend beyond traditional diagnostics.²²⁶ They are equally relevant in fields like intelligence analysis, where analysts are tasked with choosing among various explanations for the same observations. Here, the criteria of explanatory coverage, parsimony, and confidence are indispensable in guiding analysts towards the most logical and reliable conclusions.

This methodology emphasizes the broad applicability of diagnostic principles, illustrating their importance across diverse fields where decisions must be made based on incomplete or uncertain information. By applying these principles consistently, professionals can improve the precision and dependability of their determinations, whether in medical diagnostics, mechanical troubleshooting, or intelligence gathering.

Exercise on Diagnostic Hypothesis Selection

Let's engage in an exercise to look deeper into diagnostic reasoning. This exercise involves a slightly more complex dataset than previous ones. On the right-hand side, we have a list of diseases. Based on the available data, let's discuss which disease or combination of diseases best explains the symptoms observed.

Patient:	Illnesses:
A: Normal	☐ Alphaitis: Elevated A, Reduced C, Elevated F
B: High	☐ Betatosis: Elevated B, Reduced C, Elevated E, Reduced H
C: Low	☐ Gammanoma: Elevated D, Elevated E, Elevated F
D: Normal	☐ Deltacol: Elevated B, Reduced C
E: Normal	☐ Epsicusus: Reduced H
F: Low	☐ Zetad: Elevated B, Reduced C, Reduced E, Reduced F
G: Normal	☐ Etaemia: Elevated A, Reduced D, Reduced H
H: Low	☐ Thetadesis: Elevated B, Reduced C, Reduced H
	☐ Iotalgia: Elevated A, Reduced E, Elevated F, Elevated G
	☐ Kappacide: Reduced A, Reduced F, Reduced G
	☐ Lambdacrite: Reduced A, Reduced E, Reduced F, Reduced G
	☐ Mutension: Elevated A, Elevated G

What illness (or set of illnesses) would you use to diagnose this patient?

Figure 21-8 Exercise illustrating the Diagnosis process.

Ashok David, could you share your response to this exercise?

David Certainly, Ashok. After analyzing the data, I concluded that the best explanation is a combination of Betatosis and Zetad. This pairing covers all the symptoms we've considered. Both Betatosis and Zetad lead to High B and Low C. The patient's E level remains normal; interestingly, the effects of Betatosis and Zetad counteract their influence on E, maintaining its normal level. Zetad explains our Reduced F, and Betatosis accounts for Reduced H. Although this choice heavily leans on maximizing coverage, it does sacrifice parsimony, opting to explain all symptoms comprehensively. Alternatively, Thetadesis alone would explain B, C, and H but not F, offering a simpler but less complete explanation. Enhancing Thetadesis with Kappacide and Mutension could cover all symptoms, yet this combination is less parsimonious than just Betatosis and Zetad. I might have favored this broader combination if these three diseases were more prevalent, which would increase my confidence in their likelihood.

Ashok That's an excellent answer, David. How did you develop this answer?

David I started by examining which hypotheses matched the data without conflicts. Betatosis initially seemed suitable as it explained High B, Low C, and Low H but didn't account for Low F and erroneously suggested Elevated E. Thetadesis initially seemed a good fit as it explained three of the four symptoms but missed one. This discrepancy led me to explore other options to complete the coverage. Reevaluating Betatosis, which had a close match except for two mismatches, prompted me to find a hypothesis that would fill these gaps. Zetad complemented Betatosis perfectly, addressing the exact deficiencies in Betatosis's explanation. This approach, by combining Betatosis and Zetad, proved to be the most straightforward and effective in covering all symptoms with minimal complexity.

Ashok That's thorough, David. Your method showcases the delicate balance required in diagnostic reasoning, where you effectively weighed coverage, parsimony, and empirical evidence to arrive at a robust conclusion.

In the exercise described, the approach to solving the diagnostic challenge highlights the versatility and nuance required in selecting the most fitting hypothesis. Alternative methodologies, such as case-based reasoning, provide further insight into how diverse strategies can yield different outcomes. For instance, recalling a previous scenario where a similar set of symptoms was effectively addressed could guide the current diagnostic process. If a prior case identified Thetadesis as the solution for high B and low C, and now an additional symptom of low F is presented, the method would adapt this existing solution to encompass the new symptom. This adaptation might involve augmenting Thetadesis with additional hypotheses like Kappacide and Mutension to create a comprehensive coverage of all symptoms.

This flexibility in hypothesis selection underscores a critical aspect of diagnostic reasoning – the capability to modify and extend existing solutions based on new information or varying contexts. The choice of diagnostic method, whether case-based, rule-based, or another form of reasoning, can significantly influence the outcome.²²⁷ Each method brings its strengths and might lead to different conclusions under the same set of data points. This diversity in reasoning approaches not only enriches the diagnostic process but also introduces complexity in deciding which method to employ. The discussion on how an intelligent agent selects the most appropriate method based on the available data and previous cases will be explored further in the context of meta-reasoning, highlighting the layered decision-making processes in advanced diagnostic systems.

MAPPING DIAGNOSIS TO TREATMENT: COMPLETING THE DIAGNOSTIC JOURNEY

Diagnosis fundamentally involves mapping data from observations of a malfunctioning system to hypotheses about the potential causes of observed discrepancies. This process, crucial in both medicine and mechanics, transitions abstract data into actionable hypotheses, which then guide the formulation of treatment plans.

In medical contexts, once a diagnostic hypothesis regarding a patient's condition is established – be it an illness or a combination of symptoms – it leads to corresponding treatment plans. These plans may involve a variety of therapies or medications tailored to address the diagnosed condition. Similarly, in automotive repair, identifying an issue like a malfunctioning carburetor progresses to concrete actions such as replacing or repairing defective parts.

This transition from hypothesis to treatment involves a nuanced mapping where each hypothesis directly influences specific interventions. For example, if the diagnosis points to an issue with the ignition system of a car, the treatment might involve checking or replacing components like the battery, starter motor, or ignition switch. This approach ensures that interventions are both appropriate and efficient, maximizing the chances of resolving the underlying issue.

Monitoring the outcomes of these interventions is critical. The effectiveness of a treatment provides essential feedback, which may necessitate further data collection and refinement of hypotheses. If a car continues to malfunction despite supposed repairs, this new data could indicate an incorrect initial diagnosis, prompting further investigation. Similarly, if a medical treatment does not alleviate symptoms as expected, it might suggest an alternative diagnosis or a more complex underlying condition than initially thought.

This iterative process – diagnosis, treatment, and evaluation – demonstrates the dynamic nature of problem-solving across various fields. Whether in healthcare, auto repair, or any other area requiring diagnosis, the process is cyclical and adaptive, demanding continuous assessment and adjustments based on outcomes. This not only helps in correcting errors but also in proactively achieving greater accuracy and efficacy in both diagnosis and treatment, ensuring each step is informed by the most current data and results.

In essence, the final stage of diagnosis is akin to a configuration task, where the alignment of hypotheses and treatments is fine-tuned based on ongoing feedback. This phase goes beyond merely correcting errors; it involves a proactive approach to iteratively enhance accuracy and effectiveness in diagnosis and subsequent interventions, leveraging the latest data and outcomes to refine the process continuously.

DIAGNOSIS AND COGNITION

Diagnosis is an intrinsic cognitive task triggered when our expectations clash with reality. It is a reflective process that starts as soon as we detect a discrepancy between what was expected and

what actually occurs. This process is universal, not limited to specialized fields such as medicine or engineering; it pervades our daily experiences, illustrating our deep-seated need to understand and rectify inconsistencies.

The essence of diagnosis lies in probing the reasons behind unexpected outcomes. Whether dealing with complex systems like the human body or everyday occurrences such as traffic patterns, the fundamental question is always, “Why did this happen?” In the medical world, diagnosis involves identifying the underlying diseases causing observed symptoms. In technology, software debugging involves diagnosing the reasons behind a program’s malfunction, using errors and unexpected outcomes as clues.

Various methods are employed to tackle diagnostic challenges, with case-based reasoning being one notable approach. This method analyzes previous analogous situations to inform understanding and resolution of current issues, utilizing past experiences to hypothesize about present anomalies.

Diagnosis is a routine part of everyday life. It's present when we try to figure out the cause of sudden traffic – a possible accident or roadwork might be disrupting the flow. It also surfaces in personal interactions; unexpected responses from colleagues might prompt an internal quest to understand the dynamics at play.

These instances highlight diagnosis as a fundamental cognitive activity. It is not merely about problem-solving but about aligning our perception with reality, constantly adjusting our understanding and responses based on new insights. This ongoing adjustment is crucial not only for addressing immediate issues but also for enhancing our ability to interact with and navigate the complex systems that surround us. Thus, diagnosis is more than a mechanism for fixing issues – it is a vital aspect of cognitive functionality that helps us interpret and adapt to the world more effectively.

INTEGRATING KBAI: DIAGNOSIS IN THE MODERN AI ECOSYSTEM

Diagnosis, the process of identifying the causes of malfunctions or deviations from expected behavior, is a fundamental KBAI task with profound implications for modern AI. While contemporary AI, especially machine learning, excels at pattern recognition and prediction, the structured reasoning inherent in diagnostic processes offers complementary strengths crucial for building robust, reliable, and understandable AI systems.

- **Enhancing Interpretability and Explainability (XAI):** Diagnostic reasoning, particularly model-based or abductive approaches, provides causal explanations for system behavior. When a machine learning model makes a prediction or an autonomous system takes an action, diagnostic KBAI techniques can help explain *why* by identifying the contributing factors or violated assumptions. This is vital for debugging complex AI systems and building user trust.

- **Improving Robustness and Safety:** Modern AI systems, especially those deployed in critical applications (e.g., autonomous vehicles, medical AI), must be robust to unexpected inputs or internal faults. Diagnostic capabilities allow these systems to detect anomalies, identify the root cause of failures (e.g., sensor malfunction, adversarial attack, distributional shift), and potentially initiate corrective actions or safe fallback modes.
- **Facilitating Learning from Failure and Sparse Data:** When an AI system fails, a diagnostic analysis can pinpoint the reasons for the failure. This understanding can then be used to refine the model, update the knowledge base, or collect more targeted data, leading to more efficient and effective learning, especially in scenarios with limited data.
- **Synergy with Machine Learning Models:**
 - **Model Debugging:** Diagnostic principles can be applied to debug machine learning models themselves, helping to understand why a model is making certain errors or exhibiting biases.
 - **Hybrid Systems:** KBAI diagnostic engines can work in tandem with machine learning components. For instance, a neural network might detect an anomaly (the “symptom”), and a symbolic diagnostic reasoner could then use a model of the system to hypothesize and test potential causes.
 - **Causal Inference:** Diagnosis is inherently linked to causality. Integrating diagnostic reasoning can help machine learning models move beyond mere correlations to a deeper understanding of causal relationships in data.
- **Knowledge Representation and Reasoning:** Effective diagnosis often relies on rich knowledge representations (e.g., causal models, fault taxonomies, system ontologies). Developing and leveraging such knowledge is a core KBAI strength that can make AI systems more knowledgeable and capable of deeper reasoning.
- **Applications Across AI Domains:**
 - **Troubleshooting Complex Systems:** From IT networks and manufacturing lines to software debugging and scientific discovery, AI-powered diagnostic tools can significantly speed up problem resolution.
 - **Healthcare:** AI-driven medical diagnosis continues to be a major area of research, aiming to assist clinicians by identifying diseases based on symptoms, medical imaging, and patient history.
 - **Personalized Interventions:** By diagnosing the root causes of issues (e.g., in education, why a student is struggling; in personalized medicine, why a treatment is ineffective), AI can help tailor more effective interventions.

While modern machine learning has made incredible strides, the principles of diagnostic reasoning from KBAI provide essential tools for building AI systems that are not only predictive but also understandable, robust, and capable of reasoning about cause and effect. The future likely lies in hybrid approaches that combine the data-driven power of ML with the knowledge-rich reasoning of KBAI for more comprehensive and reliable AI.

CHAPTER SUMMARY

This chapter explored the concept of diagnosis, extending its application from traditional medical contexts to the broader field of knowledge-based AI. Diagnosis was defined as the process of identifying the root causes of malfunctions or deviations from expected behavior in various systems, including mechanical, software, and biological entities. We established that diagnosis fundamentally involves mapping observable data from a data space (symptoms, errors, observations) to a hypothesis space (potential causes or faults).

We began by illustrating the diagnostic process through a medical exercise, highlighting principles like coverage (explaining all data) and parsimony (preferring simpler explanations), and noting the complexities arising from interacting hypotheses. The chapter emphasized that diagnosis is not limited to one domain but is a general problem-solving approach applicable to automotive repair, software debugging, and more, utilizing methods like rule-based, case-based, and model-based reasoning.

A key focus was the shift from viewing diagnosis merely as classification to understanding it as a form of abduction. While classification categorizes faults, abduction involves inferring the most likely causes from observed effects, providing a more nuanced approach to handling complex scenarios with multiple interacting or canceling hypotheses. We situated abduction within the broader framework of logical reasoning, alongside deduction and induction, and discussed criteria for selecting diagnostic hypotheses, including explanatory coverage, parsimony, and confidence.

The chapter also touched upon the cognitive aspects of diagnosis, recognizing it as an intrinsic human reasoning process triggered by discrepancies between expectations and reality. It underscored that diagnosis is a routine part of everyday problem-solving.

Finally, we examined the integration of diagnostic KBAI principles with modern AI. Diagnosis enhances interpretability (XAI), improves system robustness and safety, facilitates learning from failures, and can be synergistically combined with machine learning models in hybrid architectures. By leveraging explicit knowledge representation and causal reasoning, diagnostic capabilities are crucial for advancing AI in complex domains like healthcare, troubleshooting, and autonomous systems. This highlights diagnosis as a vital skill for AI agents, enabling them to understand, adapt, and effectively interact with their environments.

ADDITIONAL READINGS & REFERENCES

- 223 Koitz-Hristov, R. & Wotawa, F. Applying algorithm selection to abductive diagnostic reasoning. *Applied Intelligence* **48**, 3976-3994 (2018).
- 224 Zhang, S., Song, L., Zhang, W., Hu, Z. & Yang, Y. Optimal sequential diagnostic strategy generation considering test placement cost for multimode systems. *Sensors* **15**, 25592-25606 (2015).
- 225 Richens, J. G., Lee, C. M. & Johri, S. Improving the accuracy of medical diagnosis with causal machine learning. *Nature communications* **11**, 3923 (2020).

- 226 Bai, J. *et al.* Advancing abductive reasoning in knowledge graphs through complex logical hypothesis generation. *arXiv preprint arXiv:2312.15643* (2023).
- 227 Mustafa, E. M., Saad, M. M. & Rizkallah, L. W. Building an enhanced case-based reasoning and rule-based systems for medical diagnosis. *Journal of Engineering and Applied Science* **70**, 139 (2023).

Chapter 22

Learning by Correcting Mistakes

INTRODUCTION TO LEARNING BY CORRECTING MISTAKES

In this chapter, we explore a critical method of learning that underpins much of intelligent behavior – Learning by Correcting Mistakes. This approach focuses on how agents can refine their decision-making processes by identifying, analyzing, and rectifying errors in their previous actions. It's a method that not only enhances an agent's operational accuracy but also deepens its understanding of the tasks at hand.

Consider a common scenario: you're driving and decide to change lanes. Suddenly, you hear cars honking. Instantly, you realize your mistake. This moment of recognition – realizing a mistake and reflecting on the misjudgment – mirrors the process through which AI agents can enhance their reasoning capabilities. It begs the questions: Why did I make that mistake? What led to the erroneous decision? More importantly, how can I adjust my understanding to avoid repeating this error?

This process of self-correction is what we term learning by correcting mistakes, serving as an introductory lesson into the broader KBAI area of meta-reasoning. Through this lens, agents not only rectify errors but begin to understand the deeper layers of their knowledge and reasoning processes that led to these errors.

We begin our journey into this method with a revisit to explanation-based learning – an essential precursor that sets the stage for understanding how agents isolate and correct mistakes. This approach will be paralleled with the process of medical or technical diagnosis, where the focus is traditionally on identifying and fixing errors. However, in the context of KBAI, we use these diagnostic strategies to enhance the agent's ability to self-correct.

By dissecting the mechanisms through which agents learn from their mistakes, we aim to illuminate why explanations are so vital in knowledge-based AI. These explanations not only identify what went wrong but also pave the way for integrating new, correct strategies into the agent's repertoire.

As we progress, we will examine how these corrected understandings form the foundation for advanced meta-reasoning discussions, illustrating the profound impact of learning by correcting mistakes on the development of intelligent systems.²²⁸ This chapter sets out to show that the ability to self-correct is not just a feature of advanced AI but a fundamental characteristic of all learning processes, closely mirroring how humans reflect on and learn from their mistakes.²²⁹

LEARNING BY CORRECTING MISTAKES THROUGH PRACTICAL APPLICATION: THE ROBOT AND THE CUP

Revising an earlier discussion on explanation-based learning, an engaging example emerges from a seemingly routine task, highlighting the process of learning by correcting mistakes.²³⁰ Let's revisit an earlier scenario involving a household robot purchased from the Acme hardware store. One morning, you instruct this robot with a simple request: "Go get me a cup of coffee."

A Cup

A cup is an object that is stable and enables drinking.

An Object

This object is light and made of porcelain. It has a decoration, a concavity, and a handle. The bottom is flat.

Can we prove this object is a cup?

Figure 22-1 An example of Explanation Based Learning knowledge

The robot, pre-equipped with specific knowledge definitions about household items, understands that a cup is an object that is stable and facilitates drinking. It ventures into your kitchen to fulfill your request, only to find that there are no clean cups available. Faced with this challenge, the robot's creativity is put to the test. It begins to survey the kitchen for alternatives that might serve as a suitable replacement.

Among various objects, it spots one particularly interesting item. This object is described as light and made of porcelain, adorned with decorations, possessing a concave shape, equipped with a handle, and importantly, having a flat bottom. Based on these characteristics, the robot considers whether this object could function as a cup.

Driven by its capability to engage in meta-reasoning – a form of reasoning about its own reasoning – the robot constructs an explanation to justify its choice. It reasons that the object's flat bottom provides stability, the handle aids in handling, its concavity suggests it can hold liquid, and its light weight makes it easy to lift. All these factors align with its pre-loaded definition of a cup.

This scenario exemplifies the core of learning by correcting mistakes. The robot uses its programmed knowledge to make an immediate decision but is also equipped to learn from this experience, whether it succeeds in satisfactorily replacing a cup or not. This process not only demonstrates the robot's ability to adapt and learn from novel situations but also underscores the importance of explanation in knowledge-based AI, setting the stage for deeper discussions on meta-reasoning and the refinement of decision-making processes.

Interactive Exercise: Exploring Object Identification and Definitions

To further explore the concept of learning by correcting mistakes, we present an interactive exercise that underscores the intricacies of object classification based on predefined criteria. Displayed here are six distinct objects, each with unique features and uses. This exercise will challenge your understanding of the definitions we have discussed and your ability to apply these definitions practically.

Which of these objects are cups?
Check the box in the top left for each cup.

Which of these objects meet our definition of a cup?
Check the box in the top right for each object that meets our definition.



Figure 22-2 Identification and definitions of “cups”

Exercise Instructions

- **Object Recognition:** Review the images of the six objects presented. Decide which of these you instinctively recognize as a cup. For each object you identify as a cup, mark the checkbox located at the top left of its image.
- **Definition Alignment:** Recall the specific definition of a cup as provided earlier: an object that is stable and enables drinking. Examine each object again and determine whether it meets this formal definition. For each object that aligns with this definition, mark the checkbox on the top right of its image.

Let’s join David and Ashok as they explore this question.

Ashok David, what are your answers?

David According to my intuitive definition of a cup, three of these objects are cups. The object on the top right is not a cup because it's a potato masher. The object in the top left isn't a cup either because it's a little pail. And the object in the bottom middle is not a cup because it's actually a candlestick holder. However, based on the definition of a cup that the robot was bootstrapped with, only the potato masher doesn't meet the definition. All five other objects have flat bottoms, they all exhibit concavity, they are all light, and they are all liftable. We might not think the candlestick holder is liftable because it lacks a handle, but we might also

consider it liftable because it's light enough. Either way, we'll work with that. But it's clear that the pail meets all our criteria for a cup but is not actually a cup.

Ashok Thanks, David.

Let's consider a scenario where the robot, after being instructed to fetch a cup of coffee, ventures into the kitchen. On this particular morning, the robot encounters a pail instead of a clean cup. Based on its predefined criteria – a flat bottom, a handle, light weight, and concavity – the robot determines that the pail meets the definition of a cup. Acting on this conclusion, the robot fills the pail with water and presents it to you as if it were a cup.

Upon seeing the pail filled with water, your immediate reaction is corrective: “No, robot, this is not a cup.” This moment is crucial, as it presents a learning opportunity for the robot. Cognitive agents, including robots, greatly benefit from learning through such failures. Mistakes provide them with invaluable lessons.

In this instance, the robot's error stems from its rigid adherence to the initial criteria it was programmed to follow. It correctly identified the pail's features that matched the 'cup' criteria but failed to capture the essence of what typically constitutes a cup in everyday use.

Expanding on this, let's explore a hypothetical adjustment to the robot's definition of a cup, adding that a cup must have a handle. If, under this new definition, the robot fails to recognize a handle-equipped object as a cup, and you later point out that the object was, in fact, suitable as a cup due to its liftable nature, the robot is then faced with a contradiction. It must reconcile why it failed to classify a liftable object with a handle as a cup.

Through such experiences, the robot – and, by extension, any intelligent agent – is expected to refine its understanding and judgment. It learns not only from straightforward failures but also from those instances where its reasoning led to an unexpected outcome. The process involves recognizing the shortcomings in its decision-making framework and adjusting its criteria to better align with practical definitions and user expectations. This continuous cycle of action, feedback, and adjustment is essential for developing true cognitive flexibility in artificial agents.

ADDRESSING LEARNING FROM MISTAKES THROUGH CRITICAL QUESTIONS

As we explore the concept of learning by correcting mistakes, it becomes clear that cognitive agents must tackle three essential questions to refine their decision-making abilities. This approach not only reflects the principles of metacognition but also blends seamlessly with diagnostic and explanation-based learning methodologies.²³¹ Here's how these critical questions unfold:

1. **Isolating the Error in the Model:** The first step for an AI agent after making a mistake is to isolate the error within its model of the world. This process is akin to diagnosing a medical condition, where the agent identifies the specific error that led to

the mistake. Understanding precisely what went wrong allows the agent to address the flaw directly, enhancing its ability to adjust and improve.

2. **Explaining the Fault:** After isolating the error, the agent needs to explain to itself how this fault led to the failure. This involves constructing a narrative that links the fault directly to the failure, clarifying the cause-and-effect relationship. This detailed explanation helps the agent fully understand the factors contributing to the mistake, which is crucial for effective correction.
3. **Repairing the Fault:** The final question focuses on the agent's ability to repair the fault to prevent future errors. This step involves modifying the agent's model or altering its decision-making rules to ensure that the same mistake does not recur. The repair process is both reflective and iterative, embodying the agent's continuous improvement through learning from each failure.

Through this framework, the agent not only corrects errors but also enhances its understanding of its operational environment. Each mistake is treated as a unique opportunity to improve, highlighting the agent's ongoing development. This structured approach to error correction showcases how cognitive systems evolve and adapt, using each failure as a steppingstone towards greater accuracy and efficiency in decision-making.

Practical Application of Critical Questions in Error Correction

To effectively understand how agents apply the critical questions of learning by correcting mistakes, let's look at some concrete examples, starting with the scenario involving a pail.

In the example where the agent encounters a pail in the kitchen and mistakes it for a cup, the first step involves identifying the error. The agent needs to recognize that the pail's moveable handle, unlike the fixed handles typically found on cups, fundamentally changes how the object should be classified. This attribute of having a moveable handle does not align with the attributes of a standard cup, primarily because it may compromise the stability required when used for drinking.

Following the identification, the agent must explain the fault logically. It needs to establish why the presence of a moveable handle makes the pail an unsuitable choice for a cup. This involves linking the physical characteristic – the moveable handle – to its functionality, demonstrating how it could lead to potential spills and thus, fails to fulfill the basic function of a cup.

The third critical step is repairing the fault. After understanding why the moveable handle is problematic, the agent adjusts its model of what defines a cup. This adjustment would involve updating the criteria to specifically exclude objects with moveable handles from being recognized as cups in the future, thereby preventing a recurrence of this specific error.

Another illustrative example involves a desktop assistant tasked with retrieving important documents. Suppose you instruct the assistant to fetch “that important file from last Tuesday.”

Based on its predefined criteria, which emphasized the relevance of content without considering timeliness, the assistant fetches an older document.

When the returned file is not what was expected, the agent must isolate the error in its decision-making process. Here, it recognizes that the file's age was an overlooked factor; older documents were not previously categorized as important despite their content. The agent then needs to rationalize why it erroneously considered an old document important. This might involve acknowledging that its model overly emphasized content relevance without weighing the document's recency.

Finally, the agent updates its model to include document recency as a critical criterion for importance. This ensures that older documents are not automatically classified as important unless they meet all the newly adjusted criteria, which includes their relevance and timeliness.

These examples not only demonstrate the agent's ability to refine its understanding and decision-making process but also highlight its adaptability in complex real-world scenarios where definitions and expectations can vary significantly.

Credit and Blame Assignment in AI

In artificial intelligence, identifying why an agent's knowledge led to an error is commonly known as credit assignment. However, considering the context of errors and failures, “blame assignment” might be a more fitting term. When a failure occurs, pinpointing the specific fault or gap in knowledge responsible is essential. This chapter focuses on such knowledge gaps or errors, though it's important to note that errors can also arise from an agent's reasoning or architectural setup. Credit assignment encompasses these various types of errors.

Marvin Minsky, among other AI theorists, has emphasized credit assignment as a central challenge in learning for AI agents. The complexity of this issue is linked to the fact that AI agents operate in dynamic environments. Achieving a perfect, error-free AI agent is unfeasible; even with complete knowledge and flawless reasoning, the ever-changing nature of the world ensures that an AI agent will eventually face situations that lead to failures. As the environment changes, the agent must adapt, necessitating continuous self-correction in its knowledge, reasoning, and architecture.

This constant need for adjustment highlights the role of metacognition in AI. Unlike diagnosing and repairing an external system, such as an electrical circuit or a car, AI agents engage in self-diagnosis and self-repair.⁶² This introspective process enables them to analyze their own cognitive processes, autonomously identify errors, and adapt to maintain or improve functionality in a changing world. This self-regulatory capability is crucial for developing more autonomous, robust, and adaptable AI agents.

VISUALIZING ERROR DETECTION: USING VENN DIAGRAMS

In our discussion on cognitive agents and their learning process, we specifically concentrate on errors in classification knowledge, which is a central theme addressed repeatedly throughout this book. This focus enables us to examine deeply into how agents process and categorize information, and what happens when errors occur in these categorizations.

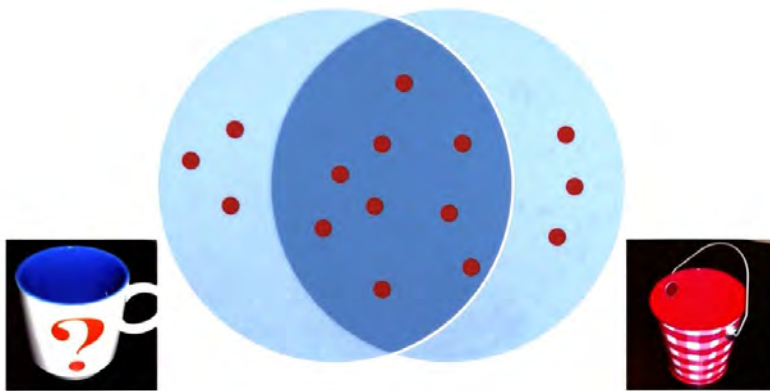


Figure 22-3 Venn Diagram representing positive and negative examples.

To illustrate, let's consider an AI agent faced with two distinct scenarios involving the task of classifying objects identified as cups. In the first scenario, illustrated in the lower left of the accompanying image, the agent successfully identifies an object as a cup and receives affirmative feedback, confirming its classification was correct. This positive experience reinforces the agent's existing model of what constitutes a cup.

In contrast, the second scenario, depicted in the lower right, presents a challenge. Here, the agent again classifies an object as a cup, but unlike the first instance, it receives negative feedback; this object should not have been classified as a cup. This error highlights a flaw in the agent's classification criteria.

To systematically identify what led to the erroneous classification, we employ a Venn diagram. The left circle in the diagram encapsulates all features associated with the positive example, possibly including attributes such as the presence of a handle or a blue interior – features that align with the agent's correct understanding of a cup. The right circle, however, outlines features related to the negative example, such as a movable handle and a red interior with distinctive white markings, which are incorrect or misleading features according to the received feedback.

By examining the Venn diagram, we can delineate features unique to each experience as well as those common to both. This method allows us to pinpoint specific attributes that led to the misclassification. For instance, common features such as concavity and handles might not be definitive enough for classification, whereas unique features like the color of the interior or the type of handle provide critical clues in distinguishing between correct and incorrect classifications.

This detailed analysis of features helps in refining the agent’s model for future classifications, ensuring it learns from its mistakes and adjusts its criteria to prevent recurring errors. Through this rigorous examination of each classification instance, the agent progressively refines its understanding and enhances its decision-making accuracy.²³²

Identifying and Addressing False Suspicious Features

In this example, the concept of “false suspicious features” becomes crucial. We define these features because they only associate with negative experiences, highlighting potential misclassifications by the agent. For instance, consider a feature like a movable handle. This feature is deemed “false suspicious” because it contributed to a negative outcome by leading the agent to mistakenly classify a non-cup object as a cup. It’s termed *false* because the resulting classification was incorrect, and *suspicious* because it isn’t present in positive instances, thus potentially responsible for the error.

However, the identification of false suspicious features introduces a challenge – deciding which of these features to prioritize for correction. This scenario mirrors the challenges faced in incremental concept learning, where we sequence examples so each varies from the existing concept by precisely one feature. This strategic presentation helps the agent pinpoint exactly which feature to adjust its focus on.

The process of isolating which feature to address could be approached by testing each feature individually to see if altering it resolves the misclassification, gaining additional feedback with each test. Alternatively, the agent might encounter numerous examples covering a spectrum of experiences.

For instance, if other positive examples in the dataset demonstrate cups without movable handles but sharing other characteristics, the feature “movable handle” stands out as a false suspicious feature. The agent can then adjust its focus to this specific attribute. Conversely, a feature like “red interior,” initially suspected of contributing to a negative classification, might be dismissed as irrelevant if subsequent positive examples include cups with red interiors, refining the agent’s classification model.

By continuously refining its focus based on accumulated examples, the agent iteratively corrects its knowledge model, enhancing its classification accuracy over time.²³³

Identifying and Addressing True Suspicious Features

Conversely, there's a scenario involving “true suspicious features,” which highlight instances where an agent incorrectly dismisses an object due to restrictive criteria embedded within its operational definitions. Imagine an agent that determines something is not a cup specifically because its interior is blue, contradicting its pre-set rules that a cup cannot have a blue interior. Consequently, the agent reports that no cups are available, even though there is one in the kitchen. When you identify the object as a cup, it signals a crucial moment for the agent's learning process.

In this instance, the crucial features that led to the agent's error include the blue interior and potentially other unique identifiers, such as a distinctive marking – a question mark on the exterior. These features are termed “true suspicious” because they directly influenced the agent's decision to wrongly exclude the object from being categorized as a cup.

These true suspicious features are precisely the attributes that caused the agent to incorrectly filter out an object that fulfills the functional role of a cup. By identifying these limiting factors in its classification logic, the agent can refine its model to prevent such misclassifications in the future. This adjustment enables the agent to accurately recognize cups, regardless of non-functional variations like color or external markings, enhancing its overall effectiveness in classifying objects in dynamic environments.

ALGORITHM FOR ISOLATING MISTAKES: IDENTIFYING TRUE AND FALSE SUSPICIOUS FEATURES

The process of refining an agent's classification knowledge to prevent future misclassifications involves a meticulous analysis of the features associated with both erroneous and correct identifications. This analysis helps the agent learn from its mistakes, thereby enhancing its decision-making accuracy over time.

Algorithm for Isolating Mistakes

```

To find suspicious true-success relations:
  Intersect all true successes ( $\cap T$ )
  Union all false successes ( $\cup F$ )
  Remove assertions in union from
  intersection ( $\cap T - \cup F$ )

To find suspicious false-success relations:
  Intersect all false successes ( $\cap F$ )
  Union all true successes ( $\cup T$ )
  Remove all assertions in union from
  intersection ( $\cap F - \cup T$ )

```

Figure 22-4 Algorithm of Isolating Mistakes (Identifying True and False Suspicious Features)

Consider the example of an agent that mistakenly identifies a non-cup object, such as a pail, as a cup. To address this “false success,” we first aggregate all features that have consistently appeared in these incorrect classifications. These features might typically include attributes like lightness, having a flat bottom, or possessing a handle – qualities that superficially fit our definition of a cup.

We then juxtapose these features with those from true successes – instances where the agent’s classifications were accurate. By extracting features common to all true successes and eliminating them from the set linked to false successes, we isolate misleading attributes. For instance, if a movable handle frequently appears in misclassifications but not in correct identifications, it becomes a key suspect for exclusion in future classifications. The agent can then refine its criteria, potentially by specifying that a valid cup must have a fixed handle, thus improving its evaluative criteria.

Simultaneously, we also consider “true suspicious” features – attributes that caused the agent to incorrectly dismiss actual cups. Here, we collect characteristics found in all false negatives and subtract those present in accurate classifications. This reverse operation helps us identify specific features that erroneously influenced the agent’s decisions, such as an unusual color or design, which are irrelevant to the object’s function as a cup.

This methodical approach, employing algorithms to dissect and reconstruct the agent’s knowledge base, ensures that our AI systems not only recognize objects more reliably but also understand the deeper, functional attributes that define them. This ongoing refinement is critical for developing AI agents capable of adapting and performing effectively in dynamic environments. The algorithms we use, such as the one depicted in the accompanying image, facilitate this intricate process by methodically isolating the elements responsible for

misclassifications, thereby fostering a deeper understanding and correction of the underlying errors in the agent's knowledge.

In concluding our discussion on the algorithm for isolating mistakes, it's important to recognize how the mathematical operations of unions and intersections play a pivotal role. By analyzing the attributes that distinguish between correct and incorrect classifications, this algorithm effectively clarifies the feature sets associated with both positive and negative experiences.

The effectiveness of this approach is contingent upon the number of examples it handles. The complexity of the concept or object under scrutiny dictates the quantity of examples needed – both positive and negative. This requirement is because a broader range of examples ensures a more thorough exploration of the feature space, allowing the algorithm to accurately identify which features reliably indicate a category and which do not.

Thus, the algorithm's success is directly proportional to the diversity and volume of examples processed. With a sufficient number of cases, the algorithm can meticulously parse through the features, distinguishing those that are genuinely indicative of a category from those that lead to misclassifications. This rigorous method enhances the AI's decision-making capabilities, enabling it to refine its classifications and better adapt to dynamic environments. This refinement process is closely tied to the identification of false suspicious and true suspicious features, ensuring that the AI learns from both its successes and failures to enhance its knowledge base continuously.

REFINEMENT THROUGH RULE REVISION: EVOLVING THE CONCEPT OF A “CUP”

In the journey of cognitive learning, particularly through the method of learning by correcting mistakes, we often see a dynamic refinement in the conceptual understanding of objects. Let's look deeper into this process using the transformation of a concept rule for a “cup,” which vividly illustrates how cognitive agents evolve their rules over time to enhance accuracy and reliability.

Old Rule	New Rule
If: Object has bottom Bottom is flat Object has concavity Object is lightweight Object has a handle	If: Object has bottom Bottom is flat Object has concavity Object is lightweight Object has a handle Handle is fixed
Then: Object is a cup	Then: Object is a cup

Figure 22-5 Refining the rules of understanding what defines a cup.

The initial rule defining a cup might state that an object qualifies as a cup if it possesses a flat bottom, concavity, is lightweight, and has a handle. This definition, though adequate in some instances, is adjusted as the agent encounters more complex real-world examples. A critical revision is introduced: the handle of the cup must be fixed. This nuanced change underscores the incremental nature of concept learning, where each new example refines the previous understanding, ensuring that the agent's classifications align more closely with practical realities.

This revision process is similar to what we observe in incremental concept learning, where the definition of a concept continuously evolves with each new example and feedback received. The specific inclusion of “handle is fixed” as a criterion represents a direct response to a misclassification, enhancing the practical usability and stability of the object classified as a cup.

The importance of such revisions lies in their contribution to a deeper understanding within knowledge-based AI. Unlike other AI paradigms, knowledge-based AI emphasizes not only classification based on observable features but also a robust explanation of why certain features are critical. The requirement for a fixed handle isn't merely a trivial addition; it's essential for ensuring that the cup serves its functional purpose effectively without risks associated with a movable handle.

Moreover, this approach emphasizes the importance of explanations in AI learning processes. It prompts the AI not only to apply features mechanically but to understand and articulate the rationale behind feature selection. This explanatory depth enables the AI to justify its classifications transparently, making it a key differentiator from other AI approaches where classification may lack a deep contextual basis.

By engaging in this continuous refinement and seeking to explain the rationale behind each feature, the AI not only corrects its past errors but also prepares itself better for future challenges. This ongoing cycle of adjustment and explanation ensures that the AI remains effective in its

functions and true to the complexities of real-world applications, embodying the essence of learning from failures.

Thus, the section on 'Refinement Through Rule Revision' encapsulates how AI agents don't just learn but adapt and explain, using their errors as stepping stones towards deeper understanding and more reliable performance.²³⁴

Constructing and Revising Explanations in AI Learning

Recall the explanation from our lesson on explanation-based learning, where the agent developed a structured explanation to demonstrate why a specific object qualifies as a cup. This process is depicted in the accompanying diagram, illustrating how attributes such as stability, flatness of the bottom, lightness, and the presence of a handle contribute to classifying an object as a cup.

Consider now the case of a pail. Initially, the agent might apply a similar explanatory framework to classify the pail as a cup, simply substituting “pail” for “object” in the original explanation. However, feedback reveals that the pail does not qualify as a cup, indicating a flaw in the initial explanation. The agent learns that not just any handle will suffice – the handle must be fixed. This discovery necessitates a revision of the explanation to incorporate the new requirement that the handle must be fixed.

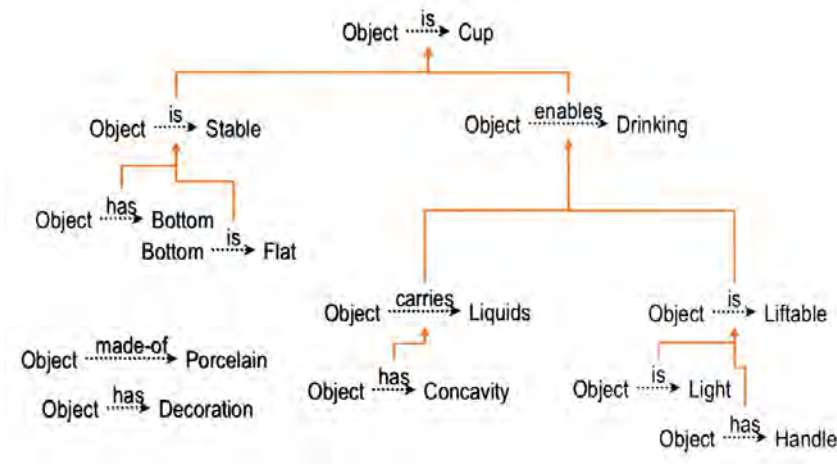


Figure 22-6 Explanation of a cup.

This adjustment leads us to a critical question: How should the explanation be altered to integrate the new criterion of a “fixed handle”? Specifically, we must determine where within the existing framework this new criterion should be inserted. This integration is crucial as it distinguishes objects that merely resemble cups from those that functionally act as cups, highlighting the importance of precise feature definitions within AI's learning algorithms.

This process of revising explanations exemplifies how AI systems adapt their models based on feedback, refining their knowledge base to prevent future misclassifications. It's not merely about adding more details but about enhancing the explanation's accuracy to align with practical experiences. This nuanced examination and adjustment of explanations underscore the depth and adaptability of learning in knowledge-based AI, ensuring that the agent's learning is both comprehensive and contextually appropriate.

As we consider the revisions required to refine the agent's understanding of what constitutes a cup, it's insightful to examine a discussion between Ashok and David as they deliberate on the optimal placement of this new criterion within the agent's explanatory framework.

Ashok David, what do you think? Where should the agent place “Handle is Fixed” in this explanation?

David Well, it seems like the most logical spot would be under the “Object is Lifiable” connection. We need the handle to be fixed to ensure that the object can be lifted safely, which in turn facilitates drinking. Positioning it there ties the handle's functionality directly to its role in enabling the cup's primary purpose.

Fine-Tuning the Explanation Model

David's initial placement of “Handle is Fixed” under the attribute “Object is Lifiable” aimed to underscore the idea that a fixed handle is essential for defining an object as a cup suitable for drinking. He posited that fixed handles inherently enable drinking by ensuring the cup's stability, thus situating the attribute precisely where it seems most functionally relevant. However, this placement, while thoughtful, isn't without its complications and may not represent the optimal configuration for this rule in the explanatory model.

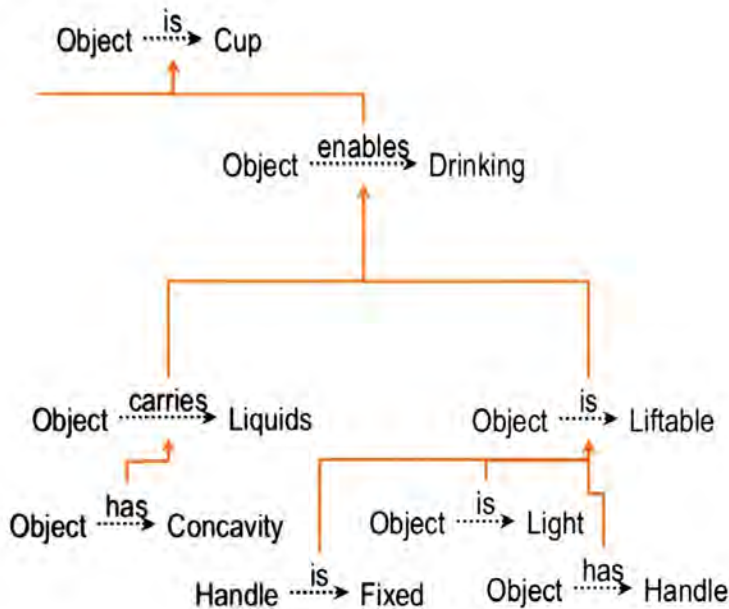


Figure 22-7 Placement of “Handle is Fixed”

The limitation of David's approach is that it too narrowly defines liftability as dependent on having a fixed handle. This inference could inadvertently exclude objects that are perfectly liftable and functional, even without a fixed handle, such as pails or briefcases with movable handles. These examples illustrate that liftability does not strictly require handle fixation, thereby highlighting an oversight in the initial explanatory setup.

To refine the model, it would be more accurate to position the “Handle is Fixed” attribute in a way that acknowledges its contribution to stability during use, particularly for drinking, but not as an absolute requirement for an object's liftability. A more fitting placement might be directly linking “Handle is Fixed” with “Object Enables Drinking.” This arrangement would maintain the relevance of handle stability to the drinking function, but without overly restricting the definition of what can be considered liftable.

Incorporating “Handle is Fixed” above “Object is Liftable” but still beneath “Object Enables Drinking” offers a balanced solution. This configuration aligns with the understanding that while a fixed handle can enhance the experience by providing additional stability when drinking, the ability to lift an object effectively does not solely depend on this feature. Such a placement avoids the potential pitfalls of over-specification in the agent's classification rules, allowing for a more flexible and accurate recognition of various objects as cups based on a broader range of functional attributes.

This adjusted approach not only corrects the narrow scope of the initial rule but also enriches the agent's understanding, ensuring that the classification system remains robust and adaptable to different real-world scenarios. This is vital in environments where the agent must discern and adapt to varied and evolving object functionalities.

Enhanced Explanation in AI Models: Refining Placement

In our ongoing refinement of the AI's explanation model, the placement of “Handle is Fixed” presents an illustrative case of how nuanced understanding is developed. This feature is appropriately positioned below “Object Enables Drinking” within the model, indicating its relevance to how an object is used for drinking but not directly tied to the “Object is Lifiable” feature. This positioning suggests that while the fixed handle enhances the object's usability for its intended function – drinking – it does not solely determine its liftability.

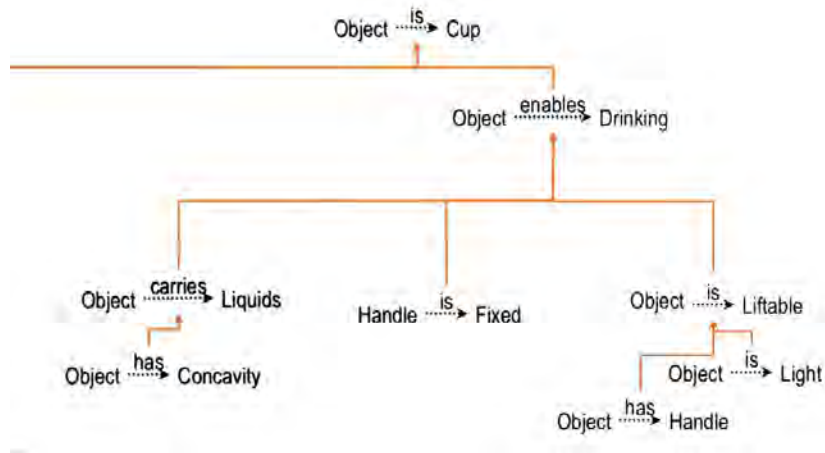


Figure 22-8 Revised placement of “Handle is Fixed”

Should the AI possess background knowledge that emphasizes the significance of a fixed handle, it understands that this feature not only supports stability but also facilitates manipulation, which is essential for drinking. This detailed knowledge allows the agent to place the “Handle is Fixed” feature where it logically supports the drinking function, illustrating that the handle's fixity enhances manipulation.

Moreover, if the agent is informed that the combination of a handle being present and fixed improves the object's orientation, it can develop a richer, more informed explanation. This depth of understanding allows the agent to see beyond simple classifications, appreciating how specific features contribute to the practical use of an object.

This refined approach underscores the importance of explanation beyond basic classification in knowledge-based AI systems. By situating “Handle is Fixed” in this specific part of the model, the agent not only categorizes objects more accurately but also gains insights into how features interrelate and affect functionality.

Such comprehensive explanations enable AI systems to not just recognize objects but also interact with them more effectively, applying a deeper understanding of their functional attributes. This enriched learning process ensures that the AI's interactions are both accurate and contextually appropriate, reflecting a sophisticated grasp of the objects it encounters.

INTEGRATING KNOWLEDGE WITH ACTION: COGNITIVE ARCHITECTURE IN LEARNING FROM MISTAKES

In our earlier discussions on incremental concept learning, we explored how an AI agent acquires knowledge. However, an equally important aspect is how this knowledge is applied, particularly in learning by correcting mistakes. This connection between knowledge acquisition and application lies at the heart of Knowledge-Based AI, where learning is not just an internal process but one that directly informs action and decision-making.

In Knowledge-Based AI, the agent's learning is intertwined with its reasoning and actions. As shown in the cognitive architecture, the AI continuously evaluates how it uses knowledge to make decisions. Learning is not a passive accumulation of data but a process of actively refining how the agent behaves based on feedback from its actions. This approach ensures that the agent doesn't just learn concepts – it learns how and when to apply them in real-world scenarios. The cognitive system shown emphasizes how reasoning, learning, and memory interact, specifically in the context of action selection. Feedback loops from the world help inform this learning, turning failures into opportunities for growth and refinement.

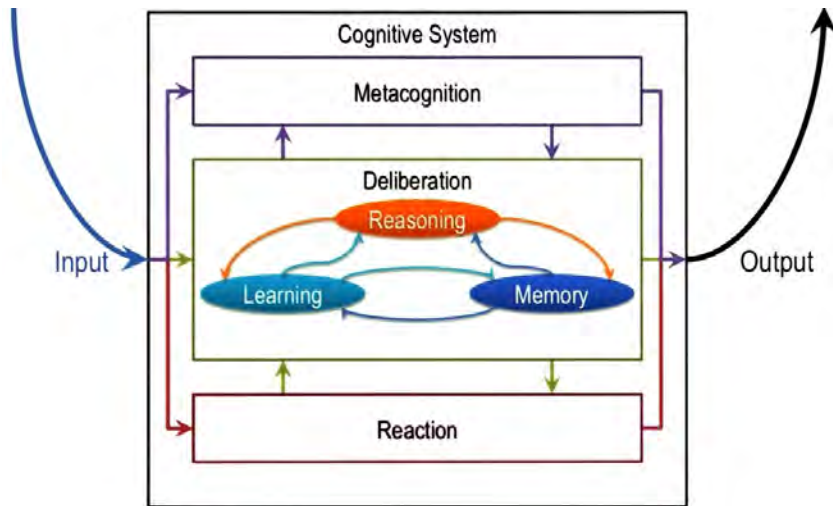


Figure 22-9 KBAI Architecture

A critical aspect of this process is viewing learning as problem-solving. When an agent encounters a failure, it doesn't stop at recognizing the error. It systematically investigates the cause, develops explanations, and corrects its knowledge. These steps, which include diagnosing the mistake, learning from it, and fixing it, show the tight integration between memory, reasoning, and learning. The entire system is designed to ensure that learning informs future actions, making it a continuous, evolving process.

The metacognitive layer within this architecture plays a key role in this. Metacognition involves thinking about thinking. It's the system's way of overseeing its cognitive processes, ensuring that the AI agent learns not just from the world but also from its own thought processes. This layer examines past decisions, identifying errors in reasoning or gaps in knowledge. By reflecting on its own performance, the agent enhances its ability to learn from mistakes, making it more adept at adapting to new situations. In the context of learning from failure, metacognition ensures that the agent improves both its knowledge base and its ability to apply that knowledge correctly.

This integration of learning, reasoning, and metacognitive evaluation underpins the depth of Knowledge-Based AI. It ensures that each learned element is not just memorized but understood and applied effectively in varying contexts. The next chapter will look deeper into the concept of meta-reasoning, where we will explore how intelligent systems develop these self-reflective capabilities to continuously improve.

LEARNING BY CORRECTING MISTAKES AND COGNITION

Learning by correcting mistakes is a fundamental aspect of human learning that mirrors the active, engaged way we typically approach learning and practicing skills.²³⁵ In our everyday lives, we are far from being passive recipients of information; instead, we actively participate in shaping our understanding and knowledge. This dynamic interaction with learning material is particularly crucial in complex environments, where merely listening isn't enough – instead, you are required to actively use your knowledge and reasoning to make sense of the information presented.

As learners, we continuously set expectations based on our prior knowledge and experiences. Sometimes, these expectations are challenged – when the new information conflicts with our predictions, it creates a learning opportunity. This discrepancy isn't merely a hurdle; it catalyzes a process of reflection and adjustment. We begin to generate explanations for why our expectations were unmet, examining the errors in our knowledge or reasoning. This is the essence of learning by correcting mistakes.

This learning process is inherently introspective, pushing us to examine our own thought processes. It's a step towards meta-reasoning, a concept explored in this chapter, which involves reflecting on and analyzing the quality and pathways of our own thinking. By understanding how we arrive at conclusions and identifying where our reasoning may falter, we enhance our ability to learn more effectively and deeply.

In sum, learning by correcting mistakes is not just about fixing errors; it's an active, reflective process that is integral to human cognition. It involves a constant cycle of setting expectations, identifying when those expectations are not met, and adjusting our understanding accordingly. This cycle not only deepens our knowledge but also refines our cognitive processes, contributing to our ongoing intellectual growth and development.

INTEGRATING KBAI: LEARNING BY CORRECTING MISTAKES IN MODERN AI

Learning by Correcting Mistakes (LBCM) is a core KBAI principle that emphasizes an agent's ability to identify, explain, and repair errors in its knowledge or reasoning. This iterative refinement process is highly relevant to modern AI, offering pathways to create more robust, adaptable, and intelligent systems, especially when combined with data-driven machine learning approaches.

- **Enhancing Robustness and Adaptability in Machine Learning:**
 - **Addressing Model Deficiencies:** Machine learning models, particularly complex ones like deep neural networks, can make errors or exhibit unexpected behavior, especially with out-of-distribution data. LBCM principles can provide a framework for diagnosing these failures (e.g., identifying problematic features or training instances) and guiding model retraining or fine-tuning.

- **Lifelong Learning:** For AI systems to operate effectively over long periods in dynamic environments, they must adapt to new information and correct outdated knowledge. LBCM offers a mechanism for such continuous improvement, allowing models to evolve rather than requiring complete retraining from scratch.
- **Improving Explainability and Trust:**
 - **Causal Attribution for Errors:** A key part of LBCM is explaining *why* a mistake occurred. This aligns with the goals of Explainable AI (XAI). By identifying the specific knowledge or reasoning flaw that led to an error, an AI system can provide more meaningful explanations for its failures, fostering user trust and facilitating debugging.
 - **Interactive Learning and Correction:** LBCM naturally supports human-in-the-loop systems where users can provide corrective feedback. The AI can then use this feedback not just to fix the immediate error but to generalize the correction to its underlying knowledge or rules.
- **Synergy with Reinforcement Learning (RL):**
 - **Learning from Negative Rewards:** In RL, agents learn through trial and error, often receiving negative rewards for undesirable actions. LBCM provides a structured way to analyze these “mistakes,” understand the policy or value function deficiencies that led to them, and make targeted updates.
 - **Model-Based RL:** In model-based RL, an agent learns a model of the environment. If the learned model is inaccurate and leads to poor decisions, LBCM techniques can be used to diagnose and repair the model based on observed discrepancies between predicted and actual outcomes.
- **Knowledge Base Refinement:**
 - **Maintaining Consistency and Accuracy:** For KBAI systems that rely on explicit knowledge bases (e.g., ontologies, rule sets), LBCM is crucial for maintaining the integrity of this knowledge. When new information contradicts existing knowledge or when rules lead to incorrect inferences, LBCM processes can identify and resolve these conflicts.
 - **Explanation-Based Refinement:** As seen in the chapter, explanations for why a mistake occurred can directly guide the modification of rules or concepts in the knowledge base, leading to more precise and reliable knowledge.
- **Developing Meta-Cognitive Capabilities:** LBCM is a foundational step towards meta-reasoning – the ability of an agent to reason about its own reasoning processes. By analyzing its own mistakes, an agent begins to understand the limits of its knowledge and the flaws in its reasoning strategies, paving the way for more sophisticated self-improvement.

While modern AI has achieved remarkable successes through data-driven approaches, the ability to learn from errors in a principled, explanatory way is vital for creating truly intelligent and autonomous systems. Integrating LBCM from KBAI can lead to AI that is not only proficient at pattern recognition but also capable of deep understanding, self-correction, and continuous adaptation in complex, evolving worlds.

CHAPTER SUMMARY

In this chapter, we thoroughly examined the process of learning by correcting mistakes, a fundamental aspect of both human learning and artificial intelligence. We revisited critical concepts such as explanation-based learning and incremental-concept learning, which are essential for understanding how individuals and intelligent agents process information incrementally, continually refining their understanding based on new data and feedback.

We explored how agents use feedback to assess the accuracy of their decisions and conclusions, enhancing the learning experience by directly addressing and correcting errors. This approach mirrors problem diagnosis, where the initial task is accurately isolating the mistake by identifying “false suspicious features” (present in errors but not successes) or “true suspicious features” (leading to incorrect rejections). This isolation is critical as it allows the learner to focus on specific inaccuracies in their knowledge or reasoning processes.

After isolating a mistake, the next step involves explaining the mistake. This phase is crucial for building human-like intelligent agents and facilitating effective error correction. Understanding *why* a mistake occurred, often through revising existing explanations (as seen in the “cup” example where the “fixed handle” attribute was integrated), allows for the refinement of the agent's knowledge base, making future decisions more accurate. This process isn't just about identifying what was wrong but also involves understanding the underlying causes, enhancing the agent's ability to avoid repeating the same mistakes.

Correcting these identified and understood mistakes, often through rule revision, is the final phase discussed. Once an error is thoroughly explained, correcting it becomes simpler and more direct. This systematic approach to learning from mistakes ensures that each error contributes to a cumulative improvement in the agent's overall functionality and accuracy, linking closely with the KBAI cognitive architecture where learning, reasoning, and memory interact.

The chapter also touched upon the cognitive parallels of learning by correcting mistakes, emphasizing its active, reflective nature in human learning and its foundational role in developing meta-reasoning capabilities.

Finally, we considered how learning by correcting mistakes, as a KBAI principle, can enhance modern AI. It can improve the robustness and adaptability of machine learning models, contribute to explainability, synergize with reinforcement learning, refine knowledge bases, and foster meta-cognitive abilities in AI systems.

As we wrapped up our exploration of correcting mistakes, we set the stage for the next chapter, which will focus on meta-reasoning. We will expand our discussion to include not just

mistakes in knowledge but also errors in reasoning and architectural setups, as well as addressing gaps in knowledge. This chapter has underscored that learning by correcting mistakes is an iterative, dynamic process that plays a pivotal role in developing robust cognitive systems capable of high levels of autonomy and efficiency.

ADDITIONAL READINGS & REFERENCES

- 62 Laird, J. E., Newell, A. & Rosenbloom, P. S. Soar: An architecture for general intelligence. *Artificial intelligence* **33**, 1-64 (1987).
- 228 Minsky, M. Steps toward artificial intelligence. *Proceedings of the IRE* **49**, 8-30 (2007).
- 229 Ohlsson, S. Learning from performance errors. *Psychological review* **103**, 241 (1996).
- 230 Winston, P. H. Learning structural descriptions from examples. (1970).
- 231 Reiter, R. A theory of diagnosis from first principles. *Artificial intelligence* **32**, 57-95 (1987).
- 232 Hoppe, D. B., Hendriks, P., Ramscar, M. & van Rij, J. An exploration of error-driven learning in simple two-layer networks from a discriminative learning perspective. *Behavior Research Methods* **54**, 2221-2251 (2022).
- 233 Zheng, Y., Liu, X. L., Nishiyama, S., Ranganath, C. & O'Reilly, R. C. Correcting the hebbian mistake: Toward a fully error-driven hippocampus. *PLoS computational biology* **18**, e1010589 (2022).
- 234 Ray, O. in *International Conference on Inductive Logic Programming*. 219-233 (Springer).
- 235 Kotseruba, I. & Tsotsos, J. K. 40 years of cognitive architectures: core cognitive abilities and practical applications. *Artificial Intelligence Review* **53**, 17-94 (2020).

Chapter 23

Meta-Reasoning

INTRODUCTION TO META-REASONING

Welcome to the chapter on Meta-Reasoning, an intricate and profound aspect of knowledge-based artificial intelligence that focuses on introspection and self-analysis. Meta-Reasoning – often encapsulated in the adage “thinking about thinking” – is the process through which an agent reflects not on the external world, but on its own internal cognitive processes.²³⁶ This involves the agent's deliberations over its own knowledge, reasoning capabilities, and learning experiences.

Consider a simple yet illustrative example: if asked for President Obama’s telephone number, most of us would immediately recognize that we do not possess that information. But crucially, how do we know that we don't know? This self-aware recognition of our knowledge limits is a fundamental form of meta-reasoning, reflecting our ability to assess the contents of our own knowledge base.

In previous discussions, particularly when examining how agents learn from correcting mistakes, we briefly touched upon the concept of meta-reasoning. There, our focus was primarily on identifying and rectifying errors within the agent's knowledge base. However, errors can extend beyond mere factual inaccuracies to include flaws in reasoning processes and learning strategies.

This chapter will look deeper into various facets of meta-reasoning, starting with a thorough exploration of how agents can identify and amend mistakes in reasoning and learning, as well as

rectify errors within their knowledge. More than just fixing mistakes, we will explore how agents identify when crucial pieces of knowledge are missing – knowledge gaps that might impede effective decision-making or problem-solving.

Moreover, we'll expand our exploration into a vital meta-cognitive skill that is pivotal for advanced AI applications: strategy selection and integration. Throughout this book, we have introduced numerous methods by which intelligent agents can solve problems. Yet, we have not fully addressed how these agents can strategically combine multiple methods to tackle complex, multi-faceted problems effectively.

The discussion will progress to even more abstract levels of self-reflection such as meta-meta-reasoning and beyond, questioning how deep the rabbit hole of self-reflection can go.²³⁷ Can an intelligent agent reflect on its own process of meta-reasoning? And if so, what are the implications and benefits of such a recursive capability?

Finally, we will illustrate these concepts through practical examples, particularly focusing on what is known as goal-based autonomy. This refers to the capability of AI systems to adapt their strategies and learning processes in response to new or evolving goals, a crucial feature for achieving robustness and flexibility in dynamic environments.

By the end of this chapter, you will gain a comprehensive understanding of how meta-reasoning empowers AI agents not only to monitor and refine their internal processes but also to adaptively integrate various strategies to meet emerging challenges and opportunities. This discussion is not merely academic; it reflects the ongoing evolution of AI into ever more adaptable and insightful systems, capable of mimicking the profound flexibility of human intelligence.

ADDRESSING KNOWLEDGE ERRORS IN META-REASONING

Throughout this book, we have extensively discussed the concept of meta-reasoning – a form of high-level introspection where an agent reflects on its own cognitive processes. Meta-reasoning involves an agent's examination of its own knowledge, reasoning, and learning, assessing and correcting errors within its internal processes to improve decision-making.

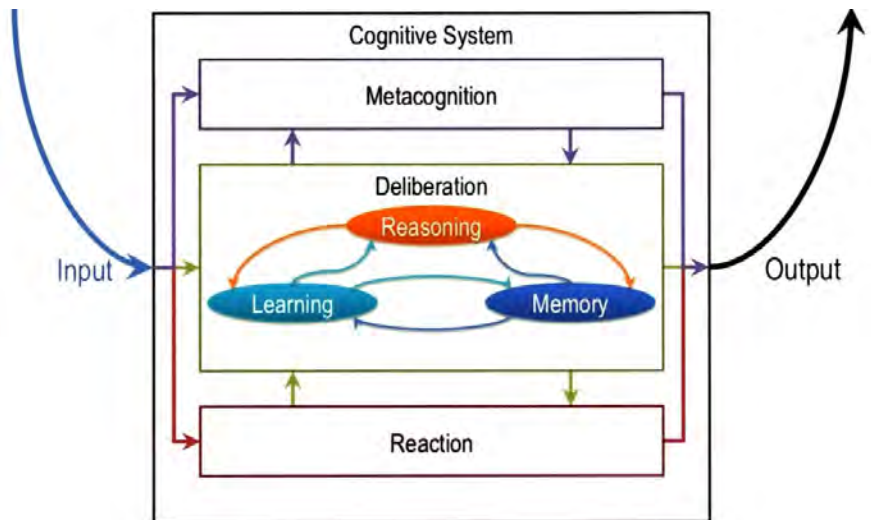


Figure 23-1 Meta-reasoning in the context of the KBAI Architecture

A pertinent instance of meta-reasoning we previously encountered involves an agent tasked with identifying a cup. Initially, the agent determined that a particular object was a cup, but this decision was later deemed incorrect.²³⁸ The agent's subsequent meta-reasoning involved scrutinizing the rationale it had constructed. It reflected on the knowledge it applied and questioned its own reasoning, probing where the fault might lie and what aspects of its explanation led to the incorrect classification. Such reflection might involve distinguishing between knowledge stored in short-term versus long-term memory, and whether the recall or application of this knowledge was flawed.²³⁹

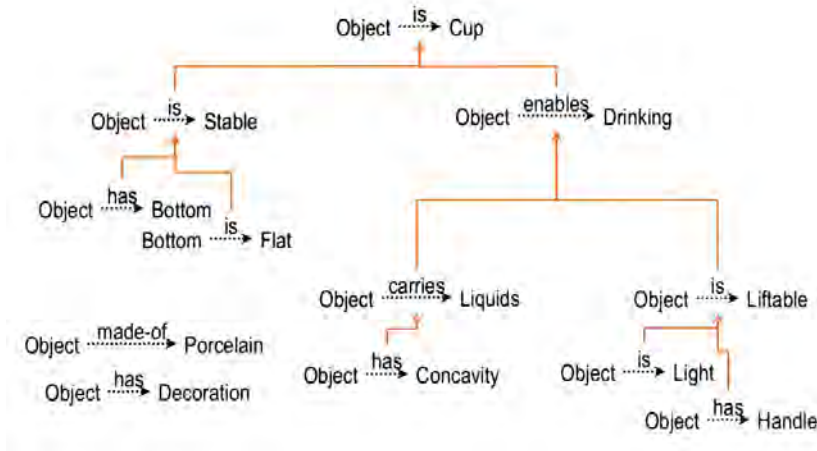


Figure 23-2 Reasoning that an object is not a cup.

This type of introspection is crucial not only for addressing errors in knowledge but also for identifying faults in the agent's reasoning and learning processes. For example, an error in reasoning was illustrated early in this book through the use of mean-end analysis in the blocks microworld scenario. Here, the agent was tasked with organizing blocks from an initial setup to a goal state, with objectives such as placing block D on the table, C on D, and B on C. However, the agent encountered multiple dead ends or 'cul-de-sacs' where proceeding with one goal prevented the achievement of another. This situation highlighted a metacognitive challenge over reasoning, prompting the agent to analyze what went wrong in its approach and how it could navigate out of these reasoning traps.



Figure 23-3 Reasoning with Means-End Analysis

Similarly, we observed metacognition over learning when discussing how the agent addressed its own errors through learning by correcting mistakes. An example was provided where the agent initially failed to include that a handle is fixed, an oversight it corrected upon reviewing its explanation. This adjustment was a result of the agent engaging in meta-level reflection over its learning process, particularly its use of explanation-based learning. The agent critically assessed how it arrived at the flawed explanation and what needed to change in its learning approach to prevent repeating the same error. This process illustrates the agent's capability to introspect on how it acquired knowledge, recognize deficiencies in its learning strategy, and amend its method to enhance future learning.

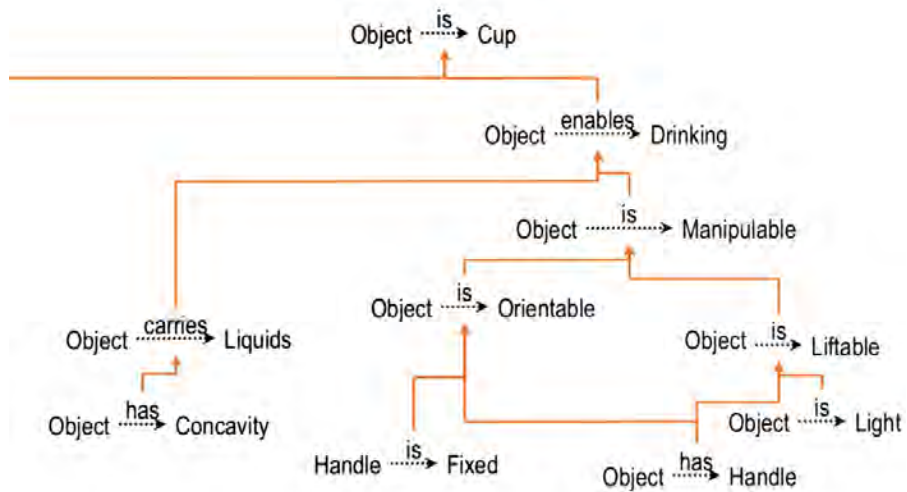


Figure 23-4 Reasoning with Learning by Correcting Mistakes

These examples demonstrate the integral role of meta-reasoning in enabling an agent to not only perform tasks but to understand and refine the cognitive processes underlying its actions. By continuously evaluating and adjusting its knowledge, reasoning, and learning strategies, an agent becomes capable of more sophisticated and accurate decision-making.

ADDRESSING KNOWLEDGE GAPS IN META-REASONING

In the area of meta-reasoning, we often encounter not only errors but also gaps in knowledge, reasoning, or learning.²⁴⁰ These gaps signify not just a lack of correctness but a lack of completeness in the agent's understanding or capabilities. Let's explore the concept of a knowledge gap, a scenario where the knowledge required to connect disparate pieces of an explanation is missing, inhibiting the agent's ability to derive or complete a logical inference.

Consider a specific exercise under explanation-based learning where the agent faces a knowledge gap. The agent has constructed two segments of an explanation about an object's attributes but fails to link these segments due to a missing piece of conceptual understanding. For example, the agent recognizes that an object has “thick sides” and that these sides “limit heat transfer,” yet it lacks the knowledge to infer that this configuration protects against heat, a critical piece of understanding when classifying objects like mugs, which are designed to handle hot liquids.

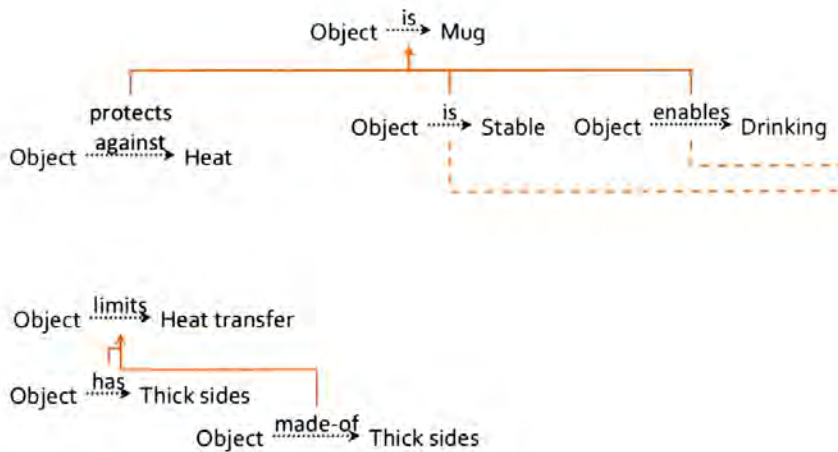


Figure 23-5 Illustration of a knowledge gap.

The knowledge structure, as shown in the accompanying diagram, visually represents this gap. The top part of the figure identifies the object as a “Mug” and attributes its ability to enable “Drinking” while being “Stable.” It also mentions that the object is made of materials with “Thick sides” which inherently “limits heat transfer.” However, the crucial linkage that thick sides can protect against heat, thereby making the mug suitable for hot beverages, is missing.

Once this gap is detected, the agent is prompted to establish a learning goal: to acquire the missing knowledge that connects these two isolated parts of the explanation. This learning goal isn't just about obtaining new information – it's about integrating this information in a way that enhances the agent's overall understanding and reasoning capabilities.

Detecting a knowledge gap thus acts as a catalyst for the agent to spawn a new goal, specifically a learning goal. This process illustrates a dynamic aspect of meta-reasoning where the agent not only identifies deficiencies in its existing knowledge base but also actively seeks to remedy these deficiencies by setting specific objectives for knowledge acquisition. This self-driven goal-setting is crucial for developing autonomous agents capable of adapting and evolving in response to new challenges or incomplete information.

By setting a learning goal to bridge a knowledge gap, the agent moves towards a more integrated and comprehensive understanding of the objects and scenarios it encounters. This process enhances the agent's ability to apply its reasoning across broader contexts, thereby improving its functionality and effectiveness in real-world applications.

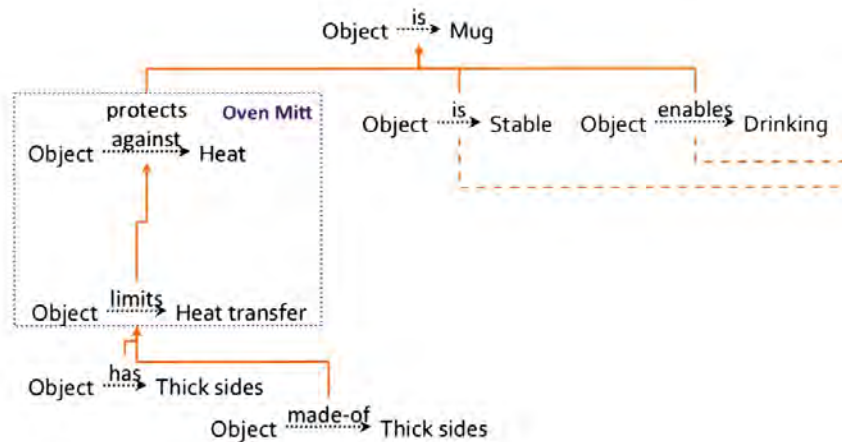


Figure 23-6 Closing the knowledge gap

To see where the lines between metacognition and deliberation are blurry, let us return to this example from explanation-based learning. Recall an instance during an explanation-based learning exercise where the agent successfully identified a knowledge gap. To bridge this gap, the agent revisited its memory, searching for a precedent that could connect disparate parts of its explanation. Upon finding a relevant piece of knowledge, the agent effectively formed a link that completed the explanation. This scenario exemplifies how agents can fulfill learning goals by leveraging existing knowledge stored in memory. Alternatively, the agent might seek knowledge externally, such as consulting a teacher, to satisfy its learning objectives.

This process highlights the agent's capacity to not only recognize when it lacks necessary information but also to actively seek out solutions that enable it to complete its cognitive tasks. Such activities are pivotal in demonstrating how metacognition supports the agent in continuously refining its knowledge base and reasoning capabilities.

Further exploring metacognition's breadth, consider its application in resolving reasoning gaps, particularly evident in complex problem-solving scenarios like the blocks microworld exercise. Here, an agent employing means-end analysis may encounter a cul-de-sac – a dead end in its reasoning process where no further progress seems possible without backtracking or revising previous steps.

To navigate out of this impasse, the agent can formulate a new reasoning goal. It might recall and apply a problem-reduction strategy, which simplifies the overarching goal into smaller, manageable tasks. By segmenting the larger problem, the agent can tackle each segment independently, gradually achieving the overall objective. This method not only resolves the immediate reasoning gap but also illustrates a sophisticated integration of multiple problem-solving strategies within the agent's cognitive framework.

In the architecture of cognitive systems, it's tempting to delineate metacognition and deliberation as distinct processes. However, in practice, there is substantial overlap between these cognitive functions. Many processes traditionally categorized under deliberation can exhibit metacognitive attributes, and vice versa.

For instance, in deliberative tasks, the agent may engage in metacognitive reflection to evaluate its strategies and adjust its approach accordingly. This interplay enhances the flexibility and adaptability of the agent, allowing it to handle complex and dynamic problem spaces more effectively.

By examining these scenarios, we observe how metacognition not only aids in filling knowledge gaps but also plays a crucial role in refining reasoning strategies and learning processes. This holistic application of metacognition ensures that AI agents can adapt to new challenges and continuously improve their performance by learning from past experiences and adjusting their approaches as needed.

Thus, metacognition stands as a cornerstone of advanced cognitive systems, embodying the essence of adaptive and self-aware computing. Through its application, agents are not merely reactive but become proactive learners and problem solvers, capable of navigating the complexities of both their internal cognitive processes and the external environments they interact with.

BLURRING THE LINES BETWEEN METACOGNITION AND DELIBERATION

In exploring the intricate relationship between metacognition and deliberation, it is essential to examine their interaction within the cognitive architecture of an agent, particularly through the lens of explanation-based learning. This approach provides a nuanced understanding of how these cognitive processes often intersect and overlap in practical scenarios.

Consider an agent engaged in explanation-based learning, attempting to understand why a particular object is recognized as a cup. Initially, the agent might struggle to connect disparate parts of its knowledge to form a cohesive explanation. For instance, it knows the object is stable, carries liquids, and has a handle, but how do these attributes collectively define a cup?

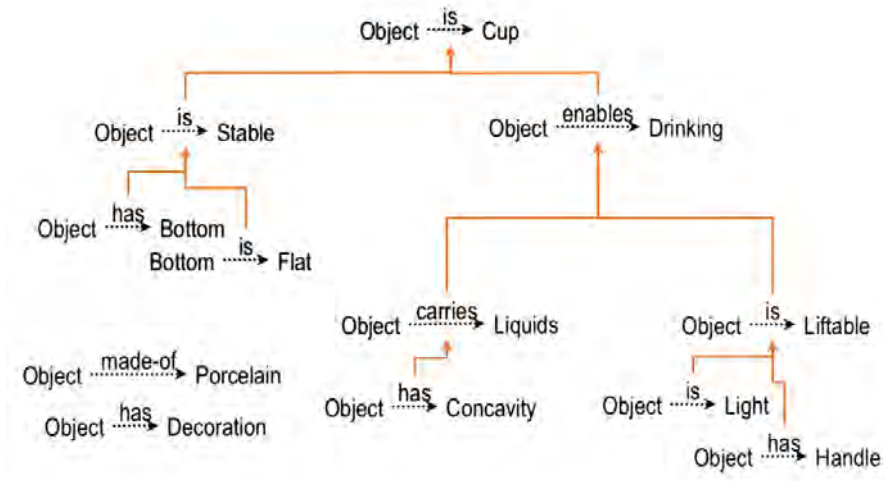


Figure 23-7 Metacognition with Explanation Based Learning

At this juncture, the agent may not explicitly engage in what we traditionally label as 'metacognition,' yet it is inherently evaluating and adjusting its cognitive strategies. The agent might say to itself, "I do not know how to build a connection between this attribute being stable and this attribute enabling the act of drinking." Consequently, it sets up a reasoning goal to bridge this gap, pulling in additional knowledge or previous cases where similar connections were established.

Once we apply the vocabulary of metacognition to this scenario, it becomes evident that the agent's task of forming connections and setting reasoning goals is deeply metacognitive. It reflects on its reasoning processes and adjusts them to overcome cognitive hurdles. However, these activities also fall squarely within the area of deliberation, where the agent assesses and executes specific cognitive tasks.²⁴¹

This overlap suggests that the separation between metacognition and deliberation is not as clear-cut as traditional models might imply. Instead of viewing these as two independent 'boxes' within the cognitive architecture, it is more productive to see them as intersecting domains – where metacognition and deliberation form a continuum rather than discrete entities.

The cognitive architecture of an agent is better conceptualized as comprising overlapping spaces – metacognitive and deliberative – where various processes can coexist and interact. This model encourages a more flexible approach to cognitive modeling, where the focus shifts from compartmentalizing mental activities to understanding the fluid dynamics between different cognitive processes.

In this revised framework, the critical questions are not about which box a process belongs to, but rather about the content of knowledge required and the processes necessary to address a particular cognitive challenge. By focusing on the needs and dynamics of the cognitive tasks at

hand, we can build more adaptive and robust AI systems that reflect the true complexity of human cognition.²⁴²

This perspective not only enriches our understanding of AI cognitive architectures but also aligns more closely with the realities of how cognitive processes operate in both artificial and natural systems. It underscores the importance of an integrated approach to cognition, where metacognitive monitoring and deliberative reasoning are part of a unified system, enhancing the agent's ability to learn from and adapt to new challenges.

STRATEGY SELECTION AND METACOGNITION

Throughout this book, we have explored an array of reasoning methods that artificial intelligence can employ, ranging from case-based reasoning and constraint propagation to more advanced techniques like plan refinement and logic scripts. In programming AI agents, we typically choose a specific method and encode it into the agent. However, one critical question arises: How does an AI agent autonomously recognize and select the most suitable method for addressing a given problem? This challenge is fundamentally a question of strategy selection, a domain where metacognition plays a pivotal role.

Metacognition in Strategy Selection

Metacognition, or the ability of an agent to “think about its own thinking,” is crucial for strategy selection. It allows an AI to assess various reasoning methods and choose the one that best fits the problem at hand based on specific criteria.^{243,244}

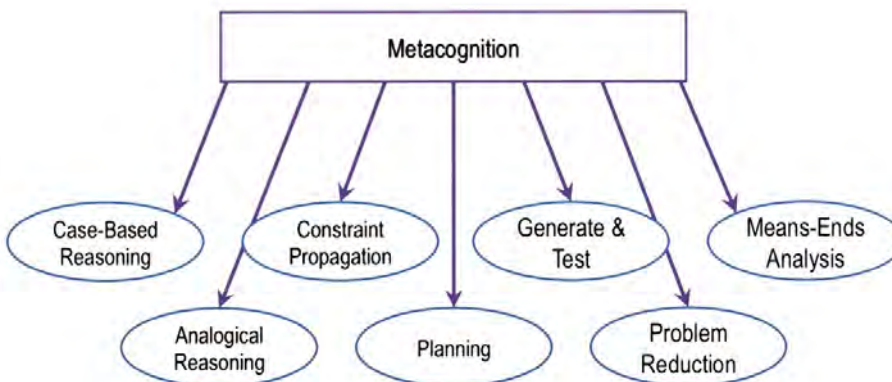


Figure 23-8 Using metacognition to select a KBAI method

Criteria for Selecting Reasoning Methods

Knowledge Availability: Each reasoning method demands certain types of knowledge. For instance:

- Case-based reasoning hinges on the availability of previous similar cases.
- Constraint propagation requires a detailed understanding of the constraints that apply to the problem.

If the necessary knowledge for a particular method is lacking – for example, if there are no available cases for case-based reasoning – the agent must discard that option in favor of another method where prerequisites are met.

Computational Efficiency: Different methods also vary in their computational demands. An agent must evaluate which method provides the most efficient solution to a problem given its computational resources. For instance:

- Case-based reasoning is highly efficient when the problem closely mirrors a previously solved case but may be less efficient if the new problem diverges significantly.
- Generate-and-test methods may work well for simpler problems but can become unwieldy and inefficient for complex scenarios.

Quality of Solutions: Some methods may offer higher reliability or accuracy in their solutions. Logic, for example, provides rigorous proofs that ensure solution correctness, making it ideal for problems where the quality of the outcome is paramount, despite potentially higher computational demands.

Practical Implications of Strategy Selection

The process of strategy selection is dynamic and context-dependent. An AI equipped with metacognitive capabilities evaluates not only what methods are available but also the nature of the problem it faces. This evaluation involves a thorough assessment of the knowledge the AI possesses and the computational cost associated with each method.

For instance, in a scenario where an AI faces a problem with multiple interacting goals, means-ends analysis might initially seem appropriate. However, if this approach leads to repetitive dead-ends or 'cul-de-sacs,' the AI might reassess and switch to a different strategy like problem reduction, which might handle the complexities of the situation more effectively.

In essence, metacognition enables AI to navigate between different strategies fluidly, applying the most appropriate one as the situation demands. This adaptability is crucial for developing sophisticated AI systems that can operate effectively in a wide range of scenarios, enhancing their autonomy and effectiveness in real-world applications.

STRATEGY SELECTION IN LEARNING

Just as artificial intelligence systems must select appropriate reasoning strategies, they must also choose among various learning methods when confronted with new problems. The process of selecting a learning strategy is similar to that of choosing a reasoning method, relying on an understanding of the problem's nature and the applicable methods.

Factors Influencing the Selection of Learning Methods

Applicability to the Problem: Each learning method has specific scenarios where it excels. For instance:

- **Incremental Concept Learning** is most effective when examples are received sequentially over time. This method allows the model to evolve as new data arrives.
- **Decision-Tree Learning**, or its variant identification-tree learning, is suited for situations where a complete dataset is available from the start. This allows the model to analyze the entire dataset in one go, establishing a comprehensive model that covers all variations in the data.

Computational Efficiency and Quality of Solutions: Beyond mere applicability, two additional criteria play crucial roles:

- **Computational Efficiency:** Some learning methods may require more computational resources than others. For example, decision-tree learning might be computationally intensive if the dataset is very large, as it involves analyzing numerous potential splits at each node of the tree.
- **Quality of Solutions:** The robustness of the model produced is also a critical factor. Some methods might provide more accurate or reliable predictions than others, depending on the complexity and nature of the data.

Choosing the right learning method involves a metacognitive assessment of the method's fit for the task at hand, its efficiency, and the quality of the outcomes it can deliver. This decision-making process is vital for ensuring that AI systems can adapt to various learning scenarios effectively, optimizing both their performance and the relevance of their learned models to specific problems.

By integrating metacognitive strategies into the learning process, AI systems can not only adapt and learn from new data but also apply the most effective techniques for data analysis and pattern recognition, thereby enhancing their overall learning efficiency and effectiveness in real-world applications.

STRATEGY INTEGRATION IN METACOGNITION

Having discussed how an agent selects strategies, we now turn to the concept of strategy integration in metacognitive processes. Even after selecting an initial strategy, an intelligent agent is not bound to it rigidly; instead, it remains flexible, adapting and shifting strategies as the problem-solving context evolves.

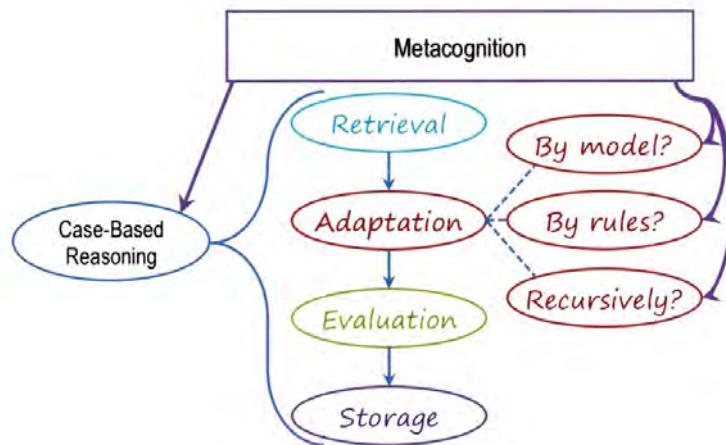


Figure 23-9 Strategy Integration in Metacognition

Case-Based Reasoning and Dynamic Strategy Integration

Let's consider a scenario where an agent uses case-based reasoning to tackle a problem. Case-based reasoning inherently involves several sub-tasks:

- **Retrieval:** The agent retrieves relevant cases from memory that are similar to the current problem.
- **Adaptation:** The agent modifies the solutions of retrieved cases to fit the new problem.
- **Evaluation:** The agent assesses how well the adapted solution solves the new problem.
- **Storage:** The agent stores the new problem and its solution for future reference.

During the adaptation phase, metacognition plays a crucial role in examining and deciding the most effective approach. Here are some possibilities:

- **Recursive Case-Based Reasoning:** The agent might recursively use case-based reasoning within the adaptation sub-task itself, drawing on a deeper set of past cases.
- **Rule-Based Adaptation:** Alternatively, the agent could switch to a rule-based method, applying specific rules to adapt the retrieved case to the new situation.

- **Model-Based Adaptation:** The agent could also employ models that predict outcomes based on certain adaptations, guiding the adjustment of the retrieved case to better fit the problem.

Flexibility in Strategy Application

This flexibility allows the agent to fluidly move between strategies, ensuring optimal problem-solving. For instance, if metacognition chooses rule-based reasoning for adaptation, the next decision becomes choosing the appropriate rule. This decision is again guided by metarules – rules about which rules to apply in given situations – illustrating a meta-level of thinking that characterizes advanced AI systems.

By employing such dynamic strategy integration, the agent not only solves the problem more effectively but also enhances its learning from the experience, continuously refining its problem-solving toolkit. This integration of multiple strategies, assessed and adjusted in real-time by metacognitive processes, is a cornerstone of advanced cognitive systems that can tackle complex, evolving problems with high efficiency.

Practical Application of Strategy Integration: The Blocks Microworld

In practical scenarios, such as the blocks microworld example previously discussed, the application of metacognition in strategy integration is vividly illustrated. Here, we encountered a typical problem-solving challenge where means-ends analysis led to a cul-de-sac – a situation where no further progress could be made without revising previous steps.

In such instances, metacognition facilitates a strategic pivot. Recognizing the impasse, the agent's metacognitive processes initiate a shift by setting up a new reasoning goal, specifically choosing the strategy of problem reduction to navigate out of the cul-de-sac. Problem reduction involves breaking down the overarching problem into smaller, more manageable sub-problems, each capable of being tackled independently.

Once these sub-goals are established, the agent may revert to means-ends analysis for each sub-goal. This methodical oscillation between means-ends analysis and problem reduction exemplifies the seamless integration of strategies. It allows the agent to efficiently manage complex problems by dynamically adjusting its approach based on the specific demands of the situation at hand.

This integration of means-ends analysis with problem reduction demonstrates metacognition's role not just in selecting and applying strategies but also in fluidly transitioning between them as dictated by problem-solving needs. Such adaptability is crucial for developing AI systems that can handle real-world complexity, where challenges often require a flexible, nuanced approach to problem-solving.

By employing metacognitive processes to oversee and adjust strategies in real time, agents can achieve solutions more effectively, showcasing the profound impact of metacognition on advanced cognitive systems' capability to navigate and resolve complex problems. This dynamic adjustment and integration of strategies underscore the sophisticated nature of metacognitive involvement in AI problem-solving architectures.

THE INNER WORKINGS OF META-REASONING: APPLYING STRATEGIES TO ITSELF

Throughout this discourse on meta-reasoning, we have explored its utility in correcting errors, filling gaps, and integrating strategies within cognitive processes. However, an intriguing aspect yet to be discussed concerns the inherent processes and strategies employed by metacognition itself.

Meta-reasoning, intriguingly, utilizes the very strategies over which it presides. For instance, just as an AI might employ case-based reasoning to solve a problem, metacognition uses this approach to reflect on its own decision-making process. Consider a scenario where metacognition faces a dilemma. It may recall previous strategies employed for similar problems, applying case-based reasoning at a meta level. For example, if constraint propagation proved effective in a past situation, metacognition might opt to deploy this strategy once again under similar circumstances.

Further illustrating its versatility, metacognition might encounter a different problem and adopt a planning approach. Here, it views each potential strategy as an operator within a larger plan, methodically structuring the approach to problem-solving. This planning stage is crucial as it sets the framework within which various strategies are evaluated and executed.

In another scenario, metacognition could be confronted with a novel challenge and decide to leverage every strategy available. It systematically generates potential solutions using diverse methods. Subsequently, it employs generate-and-test techniques to evaluate these solutions, determining the most effective approach through heuristic analysis.

These examples highlight how meta-reasoning does not merely execute cognitive strategies but also reflects on and regulates them. By utilizing established reasoning methods such as case-based reasoning, planning, and generate-and-test, metacognition enhances its capability to oversee and optimize decision-making processes across different levels of cognitive function.

Meta-reasoning enriches the cognitive architecture by providing a layer of self-reflection and regulation that ensures more robust and adaptive problem-solving capabilities. This reflective process not only applies known reasoning strategies but also adapts and integrates them based on the specific demands of the situation, showcasing the dynamic and versatile nature of metacognition in advanced cognitive systems.²⁴⁵

Reflecting on the Layers of Meta-Reasoning: Is Infinite Regress Necessary?

In our exploration of meta-reasoning, a provocative question arises: Could we establish a hierarchy within meta-reasoning itself, where layers such as meta-metacognition and even higher levels, such as meta-meta-metacognition, exist to oversee each other? This would imply an infinite regress up the cognitive hierarchy, each layer reasoning over the one below.

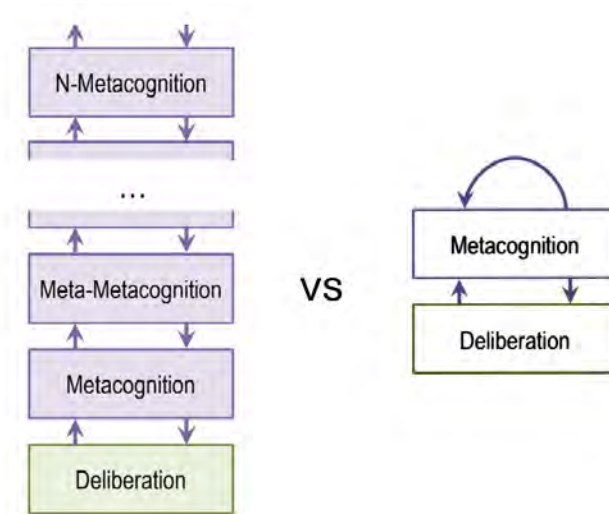


Figure 23-10 Metacognition reasoning over metacognition

Upon deeper reflection, however, introducing such a multi-layered hierarchy within meta-reasoning appears unnecessary and potentially cumbersome. Meta-reasoning inherently possesses the capability to apply the cognitive strategies it oversees directly to itself. For instance, just as meta-reasoning utilizes case-based reasoning, problem reduction, or means-end analysis to solve problems, it can similarly apply these strategies introspectively to manage its own processes.

This recursive ability implies that meta-reasoning can effectively reason about its reasoning, thus negating the need for additional layers. This concept echoes current theories in cognitive science, which typically discuss a two-layer system where meta-reasoning directly interacts with and supervises the process of deliberation. In essence, while the idea of an extensive hierarchy of meta-reasoning layers might initially seem intriguing, it ultimately proves redundant.

Meta-reasoning's power lies in its simplicity and efficiency – able to apply familiar reasoning strategies both externally and internally.²⁴⁶ This capability allows cognitive agents to remain agile

and effective without the complexity of managing an ever-expanding hierarchy of meta-cognitive layers. Such an approach ensures that meta-reasoning remains a practical and powerful tool in enhancing cognitive processes, reinforcing its role as a central component in advanced artificial intelligence systems.

AN EXAMPLE OF META-REASONING IN ACTION: THE ROBOT AND THE CAMERA

In discussing the intricacies of meta-reasoning, we often encounter its inherently circular logic, which can obscure the specific operations we aim to examine. To clarify this concept, consider a compelling scenario involving a robot tasked with assembling cameras – a routine it performs diligently every day. However, the story takes an interesting turn when the robot's routine is suddenly altered.

One day, the robot's manager instructs it to switch tasks from assembling to disassembling a camera. This new request presents a unique challenge, as the robot is not programmed for disassembly. Faced with this task, the robot activates its meta-reasoning capabilities to decide on a strategy. Drawing from its database, it recognizes that its experience is limited to assembling cameras. Here, the robot's meta-reasoning guides it to leverage what it knows best – albeit in reverse.

Determined, the robot selects a method from its memory that suggests reversing the assembly process to achieve disassembly. This approach exemplifies not only strategy selection but also integration, as the robot repurposes its assembly knowledge for disassembly tasks. With meticulous care, it follows the steps in reverse and successfully disassembles the camera.

However, the task completion time does not meet the manager's expectations. Critiqued for its pace, the robot is challenged to disassemble another camera, but faster. This feedback propels the robot into deeper meta-cognitive reflection. It reassesses its previous method selection, recognizing the need for an alternative strategy beyond mere case-based reasoning, which had proven suboptimal under the new criteria for speed.

In this reflective state, the robot exemplifies what might be called meta-metacognition: it thinks about how it thought about its methods. Determined to improve, it decides to forgo its previous strategy in favor of a more efficient one, though unspecified, in future tasks. This decision highlights the robot's ability to use meta-reasoning not just to solve problems but to learn from its experiences and optimize its approach based on outcomes and feedback.

This narrative encapsulates the essence of meta-reasoning within artificial intelligence. The robot's journey from following orders to self-assessment and strategy refinement illustrates how advanced AI systems can employ meta-reasoning to not only adhere to instructions but also evolve and adapt autonomously. Through this process, the robot enhances its operational efficacy and aligns closer with the dynamic requirements of real-world tasks, marking a significant leap in cognitive computing.

Goal-Based Autonomy: Adapting to New Objectives

The narrative of the camera-assembling robot that learned to disassemble offers a poignant illustration of goal-based autonomy. This scenario extends beyond mere troubleshooting following errors or failures; it underscores a crucial aspect of adaptive intelligence where the agent is tasked with entirely new objectives.

In such instances, it's not the detection of an error that triggers cognitive recalibration but the introduction of a novel goal. This shift demands that agents not crumble under the unforeseen challenge – exemplifying the kind of resilience we strive to avoid creating brittle systems. Instead, we aspire to cultivate agents capable of adjusting their cognitive and operational frameworks proactively. Such adaptability allows them to pursue and realize goals they were not initially designed to achieve.

Reflecting on human cognition, which is markedly robust and versatile, we confront and surmount countless challenges and achieve a myriad array of goals. The vision for artificial intelligence is to imbue machines with a comparable level of adaptability. This adaptability becomes particularly vital as tasks and environments evolve, often unpredictably.

Metacognition stands out as a cornerstone in this endeavor, facilitating not only the robustness and flexibility needed but also enhancing the AI's ability to dynamically navigate among an array of potential strategies. It enables the integration of multiple strategies, refining problem-solving processes as they unfold. By leveraging the strategies and knowledge it has been programmed with, an AI can extend its capabilities, adapting to new goals with the same efficacy as tasks it was originally designed to handle.

This capacity for dynamic adaptation not only mirrors but also extends human problem-solving abilities, positioning AI to not just replicate but potentially exceed our own cognitive flexibility in specific contexts.

CONNECTIONS: METACOGNITION ACROSS KBAI TOPICS

Throughout this book, even without specifically naming it each time, we've engaged in discussions that pivot around the concept of metacognition. Initially, these discussions centered on agents reflecting on their knowledge and taking corrective actions when mistakes were uncovered. This reflection isn't merely about recognition but extends to deliberation over the processes that initially led to the erroneous knowledge, enabling the agents to amend the underlying learning processes themselves.

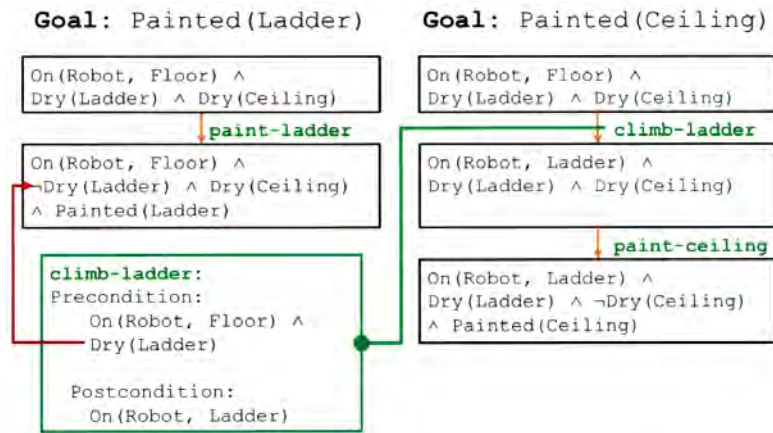


Figure 23-11 Metacognition with Partial Order Planning

One compelling instance of this metacognitive process is evident in the technique of partial order planning, which we've explored in detail. In this strategy, agents are tasked with managing multiple plans simultaneously while also tasked to resolve any arising conflicts between these plans. Here, metacognition manifests as the agent contemplates its strategies – planning out one goal while simultaneously planning another, all the while remaining vigilant about potential conflicts between these plans. When such a conflict is detected, the agent doesn't merely recognize it but engages in a meta-level reasoning to devise a new strategy that resolves or circumvents the conflict. This is a profound demonstration of metacognition, as the agent is essentially reasoning about its reasoning process, ensuring that its operations remain coherent and aligned towards the end goals. This ability to dynamically adjust and recalibrate its plans based on self-assessment and reflection typifies the nuanced application of metacognition in knowledge-based artificial intelligence.

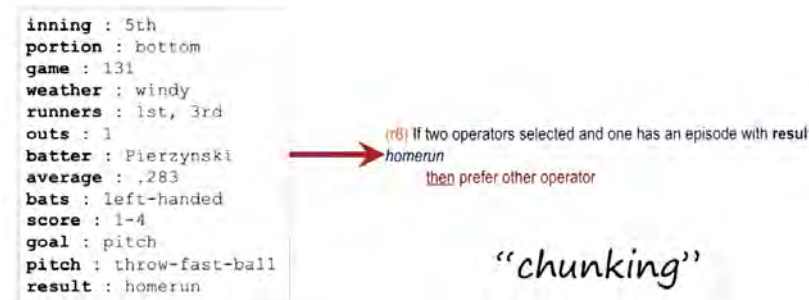


Figure 23-12 Metacognition with Production Systems

In production systems, metacognition is crucial for handling complex decision-making scenarios effectively. Consider an example where an agent encountered a dilemma between two different pitching strategies during a simulated baseball game. Unable to decide between the two options, the agent faced an impasse. This situation triggered a new learning goal: to discover a rule that could clearly determine the better pitching strategy. To address this, the agent used the “chunking” strategy, diving into its memory to retrieve relevant cases that could guide its decision. Through chunking, the agent synthesized a rule from previous experiences, which effectively resolved the deadlock. This process highlights how metacognition enables an agent to not only recognize and formulate a learning goal but also to strategically navigate its knowledge base to achieve that goal. It showcases the dynamic integration of learned knowledge and strategic thinking to overcome challenges in complex environments.

In exploring version spaces, we observe metacognition playing a pivotal role. An agent leverages metacognitive strategies to manage its understanding through specific and general models. These models represent different levels of abstraction – the specific model details precise attributes while the general model encompasses broader characteristics. The agent's metacognitive task is to reconcile these models, aiming for a point of convergence where both models align perfectly, forming a comprehensive and accurate representation of the concept.

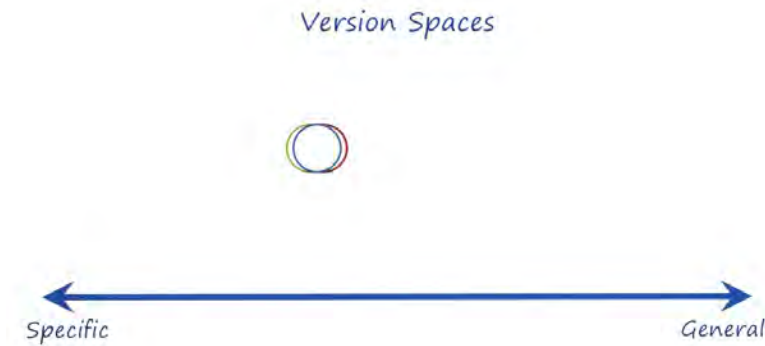


Figure 23-13 Metacognition and Version Spaces

This process sees the agent constantly evaluating its models, striving to refine the general model to avoid excessive generalization and to enhance the specific model to prevent overly restrictive interpretations. The ultimate objective is to achieve convergence, harmonizing the specific and general into a unified model that accurately encapsulates the essence of the concept. This reflects the agent's capacity to self-regulate its cognitive processes, adapting its knowledge structures to better match the reality of the concept it learns.

In our exploration of diagnostic processes, metacognition plays a crucial role, especially when treatments based on initial diagnoses fail to yield expected results. These outcomes aren't just endpoints; they serve as new data that feed back into our diagnostic framework, enhancing the depth and accuracy of our analyses.

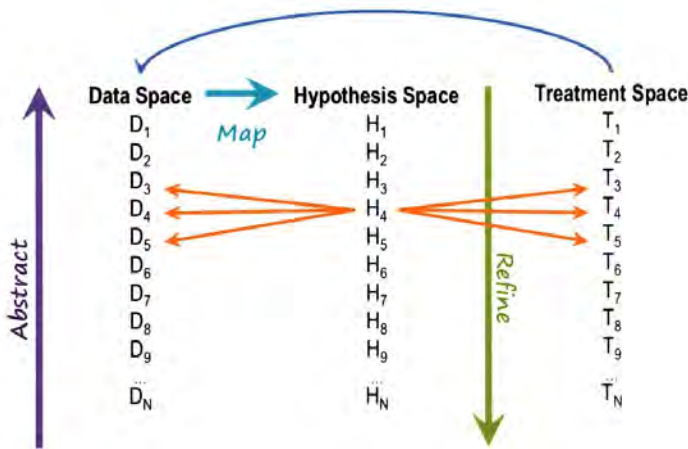


Figure 23-14 Metacognition and Diagnosis

This iterative process exemplifies metacognition in action: we not only aim to rectify the current malfunction but also strive to understand the underlying reasons for the inaccuracies in our initial diagnoses. By doing so, we engage in a meta-diagnostic process, where we scrutinize and refine our diagnostic methods themselves.²⁴⁷ This reflective approach ensures that our problem-solving methods are dynamic and adaptive, continually evolving to improve our responses to similar challenges in the future. Thus, each outcome, successful or not, becomes a valuable input for refining our cognitive processes, highlighting the self-corrective nature of systems enhanced by metacognitive insights.

THE METACOGNITION EXPERIENCE IN THIS BOOK

In an effort to embed as much metacognitive reflection as possible into our learning experience, this book has been crafted with a unique pedagogical approach. Each chapter begins by presenting a practical problem that invites direct engagement. For instance, in the section on incremental concept learning, we introduce various items categorized as 'foos' or 'not foos,' prompting you to determine the classification of new examples. Similarly, in our exploration of production systems, we detail a scenario from a baseball game, challenging you to strategize the next move for the pitcher. In the study of learning by recording cases, we present a world filled with rectangles of different colors, asking which hue a newly introduced rectangle might bear. In our discussions on classification, we confront you with an array of images, urging you to identify which among them depict birds. And in our coverage of planning, we employ a blocks microworld, where you are tasked with devising a strategy to transition from an initial state to a goal state.

These initial challenges serve not merely as problems to solve but as catalysts for designing artificial agents capable of tackling similar tasks. This methodical approach ensures that our discourse is not just about how we can solve these problems, but how we might construct an agent to do so effectively.²⁴⁸

Each lesson circles back to its introductory example, applying the reasoning methods developed within the chapter to revisit and resolve the initial problem. This not only demonstrates the practical application of the theoretical content but also deepens your understanding of the underlying cognitive processes.

This process of reflection – considering how you initially approached the problem versus how an agent would handle it – provides profound insights into both artificial and human cognitive processes. It's through this exploration that the book aims to enhance your metacognitive skills, helping you to not only learn about intelligent systems but also to better understand the complexity of your own thinking processes.

METACOGNITION AND HUMAN COGNITION

Metacognition, often referred to as “thinking about thinking,” is widely regarded as a cornerstone of human cognition. Researchers have increasingly pointed to the development of metacognitive skills at an early age as a critical predictor of success later in life, underscoring its significance beyond mere academic performance.

Metacognition is not just about acquiring new information; it involves a deeper engagement with how one learns. This process involves mastering new reasoning strategies and seamlessly integrating new information into existing cognitive frameworks, allowing individuals to manage their learning processes effectively. It enables the identification of gaps in understanding, the recognition of errors, and the strategic adjustment of approaches to overcome challenges.

Furthermore, metacognition is intimately tied to creativity. It involves the active monitoring of one's reasoning processes, the dynamic setting and revising of goals, and adept navigation through complex decision-making scenarios. Creativity often demands the ability to conceive novel solutions or reconsider existing ones from new perspectives. Metacognition facilitates this by empowering individuals to critically assess their thought processes, experiment with different thinking modalities, and innovate. This aspect of metacognition is not just about generating new ideas but also about developing novel processes that lead to intriguing and effective solutions.

By bolstering metacognitive abilities, individuals become better equipped to handle complex problems and more proficient in learning from each experience. This ongoing enhancement of understanding and capability highlights the profound impact of metacognition on broader cognitive functions. It enriches how people assimilate knowledge, plan actions, evaluate outcomes, and refine strategies, embodying a core component of lifelong learning and adaptability.

INTEGRATING KBAI: META-REASONING IN THE MODERN AI LANDSCAPE

Meta-reasoning, the capacity of an AI system to monitor, reflect upon, and control its own reasoning processes, is a crucial KBAI concept that offers substantial benefits to modern AI, especially as systems become more complex and autonomous. Integrating meta-reasoning can lead to more robust, adaptable, explainable, and efficient AI.

- **Enhancing Learning and Adaptation:**
 - **Self-Correction and Improvement:** Meta-reasoning allows an AI to detect errors or inefficiencies in its own learning or decision-making processes (as seen in learning by correcting mistakes). It can then trigger mechanisms to repair its knowledge, adjust its learning strategies, or select different problem-solving methods, leading to continuous improvement.
 - **Dealing with Novelty and Uncertainty:** When faced with unfamiliar situations or incomplete information (knowledge gaps), meta-reasoning can help an AI recognize the limits of its current knowledge and initiate strategies to acquire new information or adapt existing knowledge. This is vital for lifelong learning systems.
- **Improving Problem Solving and Decision Making:**
 - **Dynamic Strategy Selection and Integration:** As discussed, meta-reasoning enables an AI to choose the most appropriate reasoning or learning strategy for a given task based on criteria like knowledge availability, computational cost, and desired solution quality. It can also dynamically switch or combine strategies as a problem unfolds.
 - **Resource Management:** Meta-reasoning can involve monitoring computational resources (time, memory) and adjusting reasoning strategies to meet deadlines or operate within constraints. For example, it might switch from an optimal but slow algorithm to a faster, heuristic one if time is limited.
- **Increasing Transparency and Explainability (XAI):**
 - **Explaining Reasoning Traces:** By reflecting on its own reasoning steps, a meta-reasoning system can generate more coherent and understandable explanations for its decisions or conclusions. It can articulate not just *what* it decided, but *how* and *why* it chose a particular line of reasoning.
 - **Confidence Assessment:** Meta-reasoning can involve assessing the confidence in its own conclusions or the reliability of its knowledge sources, providing valuable metadata for human users or other AI components.
- **Facilitating Goal-Based Autonomy:**
 - **Adaptive Goal Management:** Meta-reasoning is key to goal-based autonomy, where an AI can formulate, prioritize, and adapt its goals in

response to changing environments or higher-level objectives. It can reason about how to achieve new goals even if they weren't explicitly programmed.

- **Self-Monitoring of Progress:** An agent with meta-reasoning can monitor its progress towards goals, detect impasses (like cul-de-sacs in planning), and initiate alternative approaches or seek help.
- **Synergies with Modern Machine Learning:**
 - **Debugging and Improving ML Models:** Meta-reasoning principles can be applied to analyze the behavior of machine learning models, understand their failure modes, and guide their refinement.
 - **Automated Machine Learning (AutoML):** The selection of appropriate ML models, architectures, and hyperparameters can be framed as a meta-reasoning task, where the AI reasons about the learning process itself.
 - **Hybrid AI Architectures:** Meta-reasoning can act as a control layer in hybrid systems, orchestrating the interaction between symbolic reasoning components and sub-symbolic machine learning modules.

While building sophisticated meta-reasoning capabilities remains a significant research challenge, its principles are fundamental to achieving more human-like intelligence in AI. By endowing AI systems with the ability to “think about their thinking,” we move towards agents that are not just tools for specific tasks but are more general, adaptive, and self-aware problem solvers.

CHAPTER SUMMARY

This chapter has deeply explored meta-reasoning, which stands as a pivotal concept integrating the diverse cognitive processes we've examined throughout this book. Meta-reasoning, essentially “thinking about thinking,” acts as a foundational framework for understanding and enhancing the decision-making and problem-solving capabilities of both human and artificial intelligences.

Our journey began with a review of how learning from mistakes, by addressing errors in knowledge, reasoning, and learning processes, fundamentally informs improvements in AI agents. We then explored how meta-reasoning helps identify and address gaps in knowledge, prompting learning goals to achieve more complete understanding. The chapter also clarified the often-blurry lines between deliberation and metacognition, suggesting they form an interactive continuum rather than discrete processes.

We then examined two key metacognitive techniques – strategy selection and strategy integration. These methods illustrate meta-reasoning's role in dynamically choosing the most suitable reasoning or learning methods based on criteria like knowledge availability and computational efficiency, and then flexibly combining or switching between these strategies to tackle specific challenges more effectively. This enhances an agent's adaptability and efficiency.

The discussion on meta-meta-reasoning addressed whether there's a need for considering infinite layers of cognitive processing. We concluded that such layered distinctions are generally unnecessary, as meta-reasoning is inherently capable of self-reflection and can adapt its strategies internally, showcasing a recursive ability to optimize itself.

A standout example of meta-reasoning in practice is goal-based autonomy, which exemplifies how agents can autonomously adjust their goals and methods in response to new or changing conditions, highlighting an advanced level of self-directed operational adaptability. We also revisited several KBAI topics like partial order planning, production systems, version spaces, and diagnosis, to illustrate how metacognitive principles are implicitly or explicitly at play in these methods. The pedagogical approach of the book itself was presented as an exercise in fostering metacognitive reflection in the reader.

The chapter also touched upon the significance of metacognition in human cognition, linking it to learning, problem-solving, and creativity.

Finally, we considered how meta-reasoning, as a KBAI principle, integrates with and enhances modern AI. It can improve learning and adaptation, facilitate better problem-solving through dynamic strategy management, increase transparency and explainability (XAI), enable goal-based autonomy, and create synergies with machine learning in areas like AutoML and hybrid architectures.

In conclusion, meta-reasoning is not merely a cognitive tool but the linchpin of our discussions in this book. It ties together various threads of reasoning, learning, and adaptation, providing a comprehensive approach to employing these concepts effectively. As we move forward to advanced topics, we continue to leverage the robust, integrative framework of meta-reasoning to look deeper into the complex aspects of knowledge-based artificial intelligence.

ADDITIONAL READINGS & REFERENCES

- 236 Russell, S. & Wefald, E. Principles of metareasoning. *Artificial intelligence* **49**, 361-395 (1991).
- 237 Caro, M. F., Cox, M. T. & Toscano-Miranda, R. E. A validated ontology for metareasoning in intelligent systems. *Journal of Intelligence* **10**, 113 (2022).
- 238 Cox, M. T. (2011).
- 239 Schmill, M. D. *et al.* 12 the metacognitive loop and reasoning about anomalies. *Metareasoning: Thinking about thinking*, 183 (2011).
- 240 Mohan, S. *et al.* A domain-independent agent architecture for adaptive operation in evolving open worlds. *Artificial Intelligence* **334**, 104161 (2024).
- 241 Johnson, B., Floyd, M. W., Coman, A., Wilson, M. A. & Aha, D. W. Goal reasoning and trusted autonomy. *Foundations of trusted autonomy*, 47-66 (2018).
- 242 Addison, U. Human-inspired goal reasoning implementations: A survey. *Cognitive Systems Research* **83**, 101181 (2024).
- 243 Parashar, P., Goel, A. K., Sheneman, B. & Christensen, H. I. Towards life-long adaptive agents: using metareasoning for combining knowledge-based planning with situated learning. *The Knowledge Engineering Review* **33**, e24 (2018).

- 244 Sung, Y., Kaelbling, L. P. & Lozano-Pérez, T. in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 4692-4699 (IEEE).
- 245 Ball, L. J. & Richardson, B. H. Vol. 13 5 (MDPI, 2025).
- 246 Cox, M. *et al.* Computational metacognition. *arXiv preprint arXiv:2201.12885* (2022).
- 247 Richardson, B. H. & Ball, L. J. Progressing the development of a collaborative metareasoning framework: Prospects and challenges. *Journal of Intelligence* **12**, 28 (2024).
- 248 Basich, C. *et al.* Competence-aware systems. *Artificial Intelligence* **316**, 103844 (2023).

Glossary

Abstraction: The process of simplifying complex systems or concepts by focusing on essential characteristics while ignoring irrelevant details. In KBAI, abstraction is crucial for creating manageable knowledge representations and for generalizing from specific examples to broader concepts (e.g., in Incremental Concept Learning, Explanation-Based Learning). It allows an agent to reason at different levels of detail. *Example from text:* In Explanation-Based Learning, generalizing a specific explanation for why an object is a “cup” to a broader rule about what makes any object a cup involves abstracting away instance-specific details.

Action Frame: A structured knowledge representation, often a type of frame, used in commonsense reasoning and natural language understanding to represent the typical components and sequence of an action or event. It includes slots for thematic roles (like agent, object, instrument, destination) associated with a verb or primitive action. *Example from text:* The action frame for the primitive action “PROPEL” (e.g., “John pushed the cart”) would have slots for the agent (John), the object (cart), and potentially source or destination.

Adaptation (in Case-Based Reasoning): The process of modifying a retrieved solution from a past case to fit the specific requirements and nuances of a new, current problem. This is a core step in Case-Based Reasoning (CBR) that allows an agent to solve problems that are similar, but not identical, to past experiences. *Example from text:* Adapting a known navigation route (e.g.,

from office to doctor) to a new route (e.g., from office to a nearby restaurant) by using a model of the environment or heuristics.

Agent (Cognitive Agent): An entity, whether human or artificial, that perceives its environment through sensors and acts upon that environment through actuators. In KBAI, a cognitive agent is often defined as a function that maps a history of perceptual experiences to an action, typically driven by goals and utilizing knowledge. *Example from text:* The robot in the “cup” example, which perceives the kitchen, reasons about objects, and acts to fetch coffee.

Agentic AI: A paradigm in AI focusing on the development of autonomous agents that can perceive their environment, make decisions, and take actions to achieve goals independently. KBAI principles are fundamental to building sophisticated agentic systems. *Example from text:* Discussed in the “Integrating KBAI” sections as an area where KBAI concepts like planning, commonsense reasoning, and meta-reasoning are highly relevant.

Algorithm / Symbol Level (Marr/Newell's Levels): One of the intermediate levels of abstraction in Marr's or Newell's frameworks for analyzing information processing systems. This level describes the algorithms used to perform computations and the symbolic representations of information that these algorithms manipulate. It bridges the gap between the high-level task/knowledge description and the low-level hardware implementation. *Example from text:* Means-Ends Analysis as an algorithm, or Semantic Networks as a symbolic representation, operate at this level.

Analogical Mapping: A key stage in analogical reasoning where correspondences are identified between the elements, attributes, and relations of a source domain (a familiar case) and a target domain (a new problem). This process establishes the structural alignment that enables knowledge transfer. *Example from text:* Mapping the sun in the solar system to the nucleus in an atom, and planets to electrons, based on their relational structure (revolving around a central body).

Analogical Reasoning: A cognitive process of drawing inferences or solving problems in a new situation (target domain) by relating it to a known, similar situation (source domain). It involves identifying similarities, mapping correspondences, and transferring knowledge or solutions. *Example from text:* Using the solution to Duncker's radiation problem (multiple weak beams converging) to solve the military problem of attacking a fortress (multiple small troop contingents converging).

Analogical Transfer: The stage in analogical reasoning where knowledge, solutions, or principles from the source domain are applied or adapted to the target domain, based on the established mapping. *Example from text:* Transferring the strategy of “dividing a force to

overcome a central obstacle without triggering defenses” from the military scenario to the medical radiation problem.

Antecedent (in Logic): The “if” part of a conditional statement (implication). In an expression $P \Rightarrow Q$, P is the antecedent. Its truth value is a condition for the truth value of the consequent. *Example from text:* In “If an animal has feathers, then it is a bird,” the antecedent is “an animal has feathers.”

Architecture (Cognitive Architecture): A blueprint or underlying structure that defines the organization and interaction of the components of an intelligent system, such as memory, reasoning mechanisms, and learning processes. KBAI often explores architectures inspired by human cognition. The equation “architecture + content = behavior” is often cited. *Example from text:* Soar is presented as an exemplar cognitive architecture, with components like working memory and long-term memory (procedural, semantic, episodic).

Associative Property (in Logic): A property of some binary logical operations (like conjunction and disjunction) stating that the order in which operations are performed does not affect the outcome when the operator is the same. For example, $(A \wedge B) \wedge C$ is equivalent to $A \wedge (B \wedge C)$. *Example from text:* Illustrated with truth tables showing $(A \vee B) \vee C$ is equivalent to $A \vee (B \vee C)$.

Atom (in Logic/Knowledge Representation): A basic, indivisible unit of representation. In propositional logic, an atom is a propositional variable. In KBAI, it can refer to a simple predicate or concept before it's combined into more complex structures like frames or semantic networks. *Example from text:* Used metaphorically to compare production rules (atoms) with frames (molecules).

Axiomatic Concepts: Concepts that are defined by a precise set of necessary and sufficient conditions, often found in formal systems like mathematics and logic. These concepts have clear, unambiguous boundaries. *Example from text:* A “right triangle” is an axiomatic concept defined by having three sides and one 90-degree angle.

Background Knowledge: Pre-existing information, concepts, rules, and experiences that an agent possesses and uses to interpret new information, solve problems, and learn. It plays a crucial role in many KBAI methods, such as Incremental Concept Learning and Explanation-Based Learning. *Example from text:* Knowing that bricks and wedges are types of “blocks” allows an AI agent to generalize the concept of an arch more effectively in Incremental Concept Learning.

Blade (in Line Labeling): In the context of visual perception and line labeling for 3D object recognition (e.g., Waltz's algorithm), a blade refers to an edge of an object that represents an exterior boundary or silhouette, where one of the adjacent surfaces is the background. *Example from text:* L-junctions are typically formed by two blades.

Blocks World: A classic AI problem domain used to illustrate and test planning and problem-solving algorithms. It involves a set of blocks on a table that an agent (e.g., a robot arm) must rearrange from an initial state to a goal state, subject to constraints like moving only one block at a time and only if it's clear. *Example from text:* Used to demonstrate Means-Ends Analysis, Problem Reduction, and Partial-Order Planning.

Bottom-Up Processing: A type of information processing that starts with raw sensory data or specific details and works upwards to form higher-level concepts, interpretations, or understanding. It is data-driven. *Example from text:* In language understanding, lexical analysis (identifying words) is a bottom-up process that feeds into syntactic and semantic analysis. In visual perception, detecting edges from pixels is a bottom-up step.

Case (in Case-Based Reasoning): A stored representation of a past problem-solving experience, typically including the problem description, the solution applied, and the outcome. Cases form the knowledge base in CBR systems. *Example from text:* A stored route from an office to a doctor, including the steps taken, used in the navigation example for CBR.

Case-Based Reasoning (CBR): A problem-solving paradigm where solutions to new problems are derived by retrieving and adapting solutions from similar past problems (cases) stored in a case library. The core cycle involves Retrieve, Adapt, Evaluate, and Store. *Example from text:* Diagnosing a patient's illness by recalling similar past patient cases and adapting their treatments.

Causality / Causal Explanation: The relationship between cause and effect. In KBAI, understanding and representing causal relationships is crucial for explanation, diagnosis, and planning. Causal explanations identify why an event occurred or why a property holds. *Example from text:* In Explanation-Based Learning, the AI explains why an object is a “cup” by citing causal links like “flat bottom CAUSES stability.”

Chunking (in Soar / Production Systems): A learning mechanism in cognitive architectures like Soar where sequences of rule firings that lead to successful subgoal achievement (often resolving an impasse) are compiled into a new, more complex production rule (a “chunk”). This speeds up future problem-solving in similar situations. *Example from text:* In the baseball pitcher scenario, learning a new rule to prefer a curveball over a fastball against a specific batter after resolving an impasse.

Classification: The cognitive and computational task of assigning objects, percepts, or instances to predefined categories or equivalence classes based on their features or properties. It is a fundamental process for organizing knowledge and guiding action. *Example from text:* Classifying animals as “birds” based on features like having feathers, wings, and laying eggs.

Clause (in Logic): In propositional logic, particularly in Conjunctive Normal Form (CNF), a clause is a disjunction (OR) of one or more literals (atomic propositions or their negations). *Example from text:* $(\text{Can_Move}(\text{widget}) \vee \neg \text{Liftable}(\text{widget}))$ is a clause derived during conversion to CNF for Resolution Theorem Proving.

Climb-Tree Heuristic (in Incremental Concept Learning): A generalization heuristic where a concept definition is broadened by replacing a specific feature or category with a more general one from a known conceptual hierarchy (background knowledge). *Example from text:* If a concept includes “brick OR wedge” and background knowledge states “brick IS-A block” and “wedge IS-A block,” the climb-tree heuristic generalizes this to “block.”

Close-Interval Heuristic (in Incremental Concept Learning): A heuristic used for generalizing or specializing concepts based on continuous-valued attributes. If positive examples span a certain range for an attribute, the concept's definition for that attribute is adjusted to cover that interval. *Example from text:* If initial examples of “dogs” are small, but a larger dog is presented as a positive example, the “size” attribute interval for “dog” is expanded.

Cognitive Architecture: (See Architecture, Cognitive)

Cognitive System: A system, natural or artificial, that exhibits human-like intelligence through processes such as perception, learning, reasoning, memory, and problem-solving. KBAI aims to build and understand such systems. *Example from text:* Defined as a system that maps a history of percepts to actions, often involving layers like reaction, deliberation, and metacognition.

Commonsense Reasoning: The ability to make presumptions and inferences about ordinary situations and events in the world, based on a vast body of implicit, everyday knowledge. Replicating commonsense reasoning is a major challenge in AI. *Example from text:* Inferring that if “Ashok ate a frog,” the frog is likely dead and inside Ashok, even though this is not explicitly stated.

Commutative Property (in Logic): A property of some binary logical operations (like conjunction and disjunction) stating that the order of the operands does not affect the result. For example, $A \wedge B$ is equivalent to $B \wedge A$. *Example from text:* Illustrated with truth tables showing $A \vee B$ is equivalent to $B \vee A$.

Computational Psychometrics: An interdisciplinary field that involves designing computational agents (AI) to take tests originally designed for humans (e.g., intelligence tests like Raven's Progressive Matrices) and then analyzing the AI's performance, including its errors, to gain insights into human cognitive processes. *Example from text:* A core concept in the KBAI course, where students build an AI to solve Raven's problems and compare its reasoning to human reasoning.

Concept (Conceptual Hierarchy): A mental representation of a category of objects, events, or ideas that share common properties. Concepts can be organized into conceptual hierarchies, where more general concepts (superclasses) subsume more specific ones (subclasses). *Example from text:* The hierarchy "Animal -> Vertebrate -> Bird -> Eagle" is a conceptual hierarchy.

Conceptual Combination: An advanced aspect of analogical reasoning or creative cognition involving the merging of elements or features from two or more distinct concepts to create a new, hybrid concept. *Example from text:* Mentioned as an open issue in analogy research, e.g., creating a new understanding of atomic structures by combining aspects of the solar system model with existing atomic theories.

Conclude (Primitive Action): One of the primitive actions in Schank's conceptual dependency theory (or similar ontologies), representing the mental act of forming a decision or reaching a conclusion. *Example from text:* "John *decided* to go to the store" might map to the primitive action CONCLUDE.

Configuration: A type of design task where all components of an artifact are known, and the goal is to assign values to the variables of these components and arrange them according to specific constraints to meet functional requirements. It's a form of routine design. *Example from text:* Designing a chair by specifying materials, dimensions, and number of legs, subject to cost and weight constraints. Designing a basement layout.

Conjunction (Logical AND): A logical operator, typically denoted by $\wedge$ (or &&), that results in true if and only if all of its operands are true. *Example from text:* $\text{Lays_Eggs}(x) \wedge \text{Flies}(x) \Rightarrow \text{Bird}(x)$ means if an animal lays eggs AND flies, then it is a bird.

Conjunctive Normal Form (CNF): A standard form for logical sentences in propositional logic, where a sentence is expressed as a conjunction (AND) of one or more clauses, each clause being a disjunction (OR) of literals. It is a required input format for Resolution Theorem Proving. *Example from text:* $(\neg \text{Has_Wings}(x) \vee \text{Has_Fur}(x) \vee \text{Bird}(x))$ is a clause in CNF.

Consequent (in Logic): The “then” part of a conditional statement (implication). In an expression $P \Rightarrow Q$, Q is the consequent. Its truth value is asserted if the antecedent is true. *Example from text:* In “If an animal has feathers, then it is a bird,” the consequent is “it is a bird.”

Constraint Propagation: A method of inference where an agent assigns values to variables characterizing a problem in such a way that global (and local) constraints are satisfied. Information about variable assignments is propagated through the constraint network to prune inconsistent values for other variables. *Example from text:* Line labeling in visual perception (e.g., identifying edges as folds or blades based on junction types like Y, L, W, T junctions) or parsing a sentence according to grammatical rules.

Constraint Satisfaction Problem (CSP): A problem defined by a set of variables, a domain of possible values for each variable, and a set of constraints that specify allowable combinations of values for subsets of variables. The goal is to find an assignment of values to all variables such that all constraints are satisfied. *Example from text:* The basement design problem can be framed as a CSP where room dimensions are variables and layout rules are constraints.

Convergence (in Version Spaces): The process in the Version Spaces algorithm where the set of most specific hypotheses (S-set) and the set of most general hypotheses (G-set) gradually approach each other as more training examples are processed, ideally meeting to define a single, correct concept description. *Example from text:* In the allergic reaction example, the S-set and G-set converge on the rule “allergic to cheap meals at Sam's” after five examples.

Correspondence Problem (in Analogical Mapping): The challenge in analogical reasoning of identifying which elements, attributes, and relations in the source domain correspond to which elements, attributes, and relations in the target domain. Solving this is key to successful mapping. *Example from text:* Determining that the “rebel army” in the military problem corresponds to the “radiation beam” in the medical problem.

Credit Assignment (or Blame Assignment): The problem in AI and machine learning of determining which specific components, rules, decisions, or pieces of KBAI Glossary knowledge in a complex system are responsible for a particular success (credit) or failure (blame). This is crucial for learning and refinement. *Example from text:* In learning by correcting mistakes, identifying the specific feature (e.g., “movable handle”) that caused a pail to be misclassified as a cup.

Cross-Domain Analogy: An analogy where the source domain and the target domain are from different fields or contexts, requiring the reasoner to identify abstract structural or relational similarities rather than superficial feature matches. *Example from text:* The analogy between Duncker's radiation problem (medical domain) and the military problem of attacking a fortress.

Cul-de-Sac (in Problem Solving/Planning): A state in a search space or a point in a plan from which no further progress towards the goal can be made using the current strategy, often requiring backtracking or a change in strategy. *Example from text:* Encountered in Means-Ends Analysis for the Blocks World problem when a chosen operator leads to a state that blocks subsequent necessary actions.

Data Space (in Diagnosis): The set of all observable data, symptoms, malfunctions, or deviations from expected behavior in a system that is being diagnosed. Diagnostic reasoning involves mapping from this data space to a hypothesis space. *Example from text:* In medical diagnosis, the patient's reported symptoms (e.g., fever of 104°F, fatigue) and test results constitute the data space.

Decision Tree Learning: A supervised machine learning method used for classification and regression that creates a model by learning simple decision rules inferred from the data features. The tree is built by selecting features that best split the data into homogeneous groups with respect to the target variable. Unlike incremental discrimination tree learning, it typically requires all examples upfront to build an optimal tree. *Example from text:* Used to classify allergic reactions based on features like restaurant, meal, and cost, or to predict sunburn based on hair color and lotion use.

Deduction (Deductive Reasoning): A form of logical reasoning where a conclusion is necessarily derived from a set of premises or general rules. If the premises are true and the deduction is valid, the conclusion must be true. *Example from text:* Given the rule “If it is cloudy, it rains” and the premise “It is cloudy,” one can deduce “It rains.” Modus Ponens is a common deductive inference rule.

Deep Similarity: In analogical reasoning, similarity based on underlying relational structures, causal patterns, or abstract principles, rather than on surface-level features of objects. Deep similarity is crucial for cross-domain analogies. *Example from text:* The similarity between the solar system and atomic structure based on the relationship of smaller objects revolving around a larger central object due to attractive forces, despite the objects themselves (planets vs. electrons, sun vs. nucleus) being semantically different.

Default Values (in Frames/Scripts): Pre-assigned or typical values for slots in a frame or script that are assumed to be true in the absence of specific information to the contrary. They represent common or expected attributes or events in a stereotypical situation, allowing for efficient reasoning and expectation generation. *Example from text:* In an “eating” frame, a default value for “object-alive” might be “false,” and for “subject-mood” might be “happy.”

Deliberation: The cognitive process of thoughtful consideration, reasoning, and decision-making, often involving weighing alternatives and potential outcomes. In KBAI architectures, deliberation is a core component, often interacting with memory and learning, and can be contrasted with more immediate reactive processes. *Example from text:* The pitcher in the baseball scenario deliberates whether to pitch or walk the batter, considering the game state and goals.

De Morgan's Law: A pair of transformation rules in Boolean algebra and propositional logic that describe how to distribute a negation operator over conjunctions and disjunctions: $\neg(A \wedge B)$ is equivalent to $(\neg A \vee \neg B)$, and $\neg(A \vee B)$ is equivalent to $(\neg A \wedge \neg B)$. *Example from text:* Used in converting logical sentences to Conjunctive Normal Form (CNF) for Resolution Theorem Proving.

Design: A cognitive and practical activity aimed at creating specifications for an artifact (which can be a physical product, process, program, or policy) to satisfy specific needs, goals, or functions. Design is often characterized by the co-evolution of the problem understanding and the solution. *Example from text:* Designing a robot to walk on water, a search engine, or economic policies.

Diagnosis: The process of identifying the nature and cause(s) of a fault, malfunction, disease, or problem by analyzing symptoms, data, and observed behaviors. It involves mapping from a data space (observations) to a hypothesis space (potential causes). *Example from text:* Medical diagnosis of diseases based on patient symptoms; diagnosing why a car won't start.

Difference (in Means-Ends Analysis): The discrepancy or mismatch between the current state and the goal state in a problem-solving process. Means-Ends Analysis aims to select operators that reduce these differences. *Example from text:* In the Blocks World problem, if block A is on the table in the current state but should be on block B in the goal state, this is a difference.

Discrimination Tree: A tree-like knowledge structure used for organizing and retrieving cases or concepts efficiently. Internal nodes represent questions or tests on features, and branches represent the outcomes, leading to leaf nodes that store the specific cases or concept instances. It is often built incrementally. *Example from text:* Used in Case-Based Reasoning to store and retrieve navigation routes based on features like origin coordinates.

Disjunction (Logical OR): A logical operator, typically denoted by $\vee$ (or $|$), that results in true if at least one of its operands is true. *Example from text:* $\text{Has_Feathers}(x) \vee \text{Has_Talons}(x) \Rightarrow \text{Bird}(x)$ means if an animal has feathers OR talons, then it is a bird.

Distributive Property (in Logic): A property that governs how two binary operations interact. For logical operations, it typically refers to how conjunction distributes over disjunction, $A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$.

$\vee C) \equiv (A \wedge B) \vee (A \wedge C)$, and how disjunction distributes over conjunction, $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$. *Example from text:* Illustrated with truth tables.

Drop-Link Heuristic (in Incremental Concept Learning): A generalization heuristic where a condition or relational link is removed from a concept definition to make the concept broader and capable of encompassing more examples. *Example from text:* If an arch concept initially requires a top brick to be supported by another brick, dropping this “supports” link makes the concept more general.

Empty Clause (in Resolution Theorem Proving): Represented as $\square$ or False, the empty clause is derived when two clauses containing complementary literals (e.g., P and $\neg P$) are resolved, and there are no other literals remaining. Deriving the empty clause in a proof by refutation signifies a contradiction in the augmented knowledge base, thus proving the original goal. *Example from text:* Derived in the robot and widget example, proving the widget was not liftable.

Enlarge-Set Heuristic (in Incremental Concept Learning): A generalization heuristic where the set of allowable values or types for a feature in a concept definition is expanded to include new instances. *Example from text:* If an arch concept initially only allows “brick” for the top element, and a new positive example shows a “wedge” on top, the “enlarge-set” heuristic would modify the concept to allow “brick OR wedge” for the top element.

Entry Conditions (in Scripts): The set of prerequisites or conditions that must be true for a script to be initiated or activated. They define the context in which the script is applicable. *Example from text:* For a “Restaurant Script,” entry conditions might include “customer is hungry” and “customer has money.”

Episodic Memory: A component of long-term memory that stores specific personal experiences, events, and episodes, tagged with temporal and contextual information. It contrasts with semantic memory (general knowledge) and procedural memory (skills). *Example from text:* In the Soar architecture, episodic memory might store details of a past baseball game, which can be used for learning via chunking.

Equivalence Classes: In classification, sets of objects, percepts, or instances that are grouped together because they are considered equivalent with respect to certain features, properties, or criteria. Classification involves mapping diverse inputs into these equivalence classes. *Example from text:* Grouping eagle, bluebird, and penguin into the equivalence class “bird” based on shared avian characteristics.

Evaluation (in Case-Based Reasoning / Analogical Reasoning): The stage where an adapted solution (derived from a past case or analogy) is assessed for its effectiveness in solving

the current target problem. This can involve simulation, real-world testing, or logical analysis. *Example from text:* Simulating a proposed navigation route to see if it successfully reaches the destination without issues.

Exemplar Concepts: Concepts that are understood or defined primarily through specific examples or instances rather than by a set of necessary and sufficient conditions or typical properties. These concepts are often abstract, subjective, or experiential. *Example from text:* “Beauty” or “freedom” are exemplar concepts, understood through examples like a Van Gogh painting or a specific act of liberation.

Existential Quantifier ($\exists$): A symbol in predicate calculus (first-order logic) used to assert that there exists at least one individual or object in the domain for which a given predicate or statement is true. It is read as “there exists.” *Example from text:* $\exists x (\text{Flies}(x) \wedge \neg \text{Bird}(x))$ means “there exists an x such that x flies and x is not a bird” (e.g., a bat).

Expectation Generation / Violation (in Scripts): Scripts create expectations about the sequence of events in a familiar situation. Expectation generation is the process of anticipating these events. An expectation violation occurs when an observed event deviates from the script's predicted sequence, often leading to surprise or a need for re-interpretation. *Example from text:* In a restaurant script, expecting the waiter to bring a menu; if the waiter brings a bill instead, it's an expectation violation.

Explanation (Causal Explanation): (See Causality / Causal Explanation)

Explanation-Based Learning (EBL): A machine learning paradigm where an agent learns from a single (or few) training example(s) by constructing an explanation of why the example is an instance of a concept, using its existing domain knowledge (background theory). This explanation is then generalized to form a new, more operational rule or schema for recognizing future instances of the concept. It is a form of “speed-up learning.” *Example from text:* Learning the concept of a “cup” by explaining why a specific object (e.g., light, concave, flat bottom, handle) meets the operational criteria for a cup, then generalizing this explanation.

False Suspicious Features (in Learning by Correcting Mistakes): Features that are present in negative examples (misclassifications where the agent incorrectly identified an instance as belonging to a concept) but absent in positive examples (correct classifications). These features are “suspicious” because they might be responsible for the error, and “false” because they led to a false positive classification. *Example from text:* A “movable handle” on a pail might be a false suspicious feature if the pail was misclassified as a cup, and actual cups in positive examples have fixed handles.

Filler (in Frames): The specific value or instance that occupies a slot in a frame. Fillers provide the concrete data for the attributes or relations defined by the slots. *Example from text:* In a frame for “Ashok ate a frog,” “Ashok” is the filler for the “subject” slot, and “a frog” is the filler for the “object” slot.

Fitness Function (in Genetic Algorithms): A function that evaluates the quality or suitability of a candidate solution (an individual in the population) in a genetic algorithm. It guides the selection process, favoring individuals with higher fitness scores for reproduction. *Example from text:* Mentioned as the “tester” component in the generate-and-test analogy for genetic algorithms.

Fold (in Line Labeling): In visual perception and line labeling for 3D object recognition, a fold refers to an edge where two visible surfaces of an object meet. Folds can be convex (bulging out) or concave (indenting in). *Example from text:* Y-junctions are typically formed by three folds.

Forbid-Link Heuristic (in Incremental Concept Learning): A specialization heuristic where a concept definition is refined by adding a condition that prohibits a specific relationship or link between elements. This helps exclude negative examples that possess the forbidden link. *Example from text:* If an arch concept incorrectly includes an example where two supporting bricks touch, the “forbid-link” heuristic adds a “must-not-touch” constraint between them.

Frame: A structured knowledge representation that organizes information about a stereotypical object, event, or concept. Frames consist of slots (attributes or relations) and fillers (values for those slots). They support default reasoning, inheritance, and expectation generation. *Example from text:* The “eating” frame with slots for subject, object, location, time, etc. Frames for “cup,” “brick,” “briefcase.”

Function (in Design): The intended purpose or role that an artifact is designed to fulfill. Understanding the function is crucial in design tasks, including configuration and diagnosis. *Example from text:* The function of a cup is to enable drinking and be stable; the function of a robot might be to assemble cameras.

Generate and Test: A fundamental problem-solving method in AI where potential solutions (or states) are generated one by one (or in batches) and then tested against a set of criteria or constraints to see if they are valid or optimal. If a solution fails the test, a new one is generated. *Example from text:* Used in the Guards and Prisoners problem to generate possible moves and test their legality and productivity. Also discussed in the context of Raven's problems.

General Model (in Version Spaces): Also known as the G-set, this is the set of the most general hypotheses that are consistent with all observed training examples so far. It represents the broadest possible definition of the concept being learned. It is specialized when a negative

example is encountered that it incorrectly covers. *Example from text:* In the allergic reaction example, initially, the general model might be “allergic to everything,” which is then specialized based on negative instances.

Generalization: The process of deriving a broader concept, rule, or principle from specific examples or instances. It involves identifying common features or patterns and abstracting away details that are specific to individual examples. This is a core process in learning. *Example from text:* Generalizing from “Buddy is a dog” and “Scout is a dog” to “four-legged, furry animals are dogs.” Heuristics like “drop-link,” “enlarge-set,” and “climb-tree” are used for generalization in Incremental Concept Learning.

Goal / Goal State: In problem-solving and planning, the desired state or outcome that an agent aims to achieve. Goals drive an agent's behavior and are used to select actions and evaluate plans. *Example from text:* In the Blocks World problem, the goal state is a specific arrangement of blocks (e.g., A on B, B on C, C on table). In the painting robot example, the goal is Painted(Ceiling) $\wedge$ Painted(Ladder).

Goal-Based Autonomy: The capability of an AI agent to formulate, pursue, and adapt its goals independently, often in response to new information or changing environmental conditions, without explicit step-by-step programming for every situation. Meta-reasoning is crucial for goal-based autonomy. *Example from text:* The robot that was initially programmed to assemble cameras but then adapted to the new goal of disassembling a camera, and further refined its strategy for speed.

Goal Clobbering (Goal Interference): A situation in planning where the actions taken to achieve one goal inadvertently undo or prevent the achievement of another (previously achieved or concurrently pursued) goal. *Example from text:* In the painting robot scenario, painting the ladder first (achieving Painted(Ladder)) makes the ladder wet ($\neg$ Dry(Ladder)), which clobbers the precondition Dry(Ladder) needed for the climb-ladder action required to paint the ceiling.

Grammar (Spatial Grammar / Syntactic Grammar): A set of rules that define the valid structures or arrangements of elements within a system. Spatial grammar refers to rules governing the arrangement of visual elements (e.g., lines and junctions in 3D object recognition). Syntactic grammar refers to rules governing the structure of sentences in a language. *Example from text:* The rules for Y, L, W, T junctions form a spatial grammar for interpreting line drawings. The rules $S \rightarrow NP VP$, $NP \rightarrow (ADJ^*) N$, $VP \rightarrow V (ADV)$ form a simple syntactic grammar.

Heuristic: A problem-solving approach, rule of thumb, or guideline that is not guaranteed to be optimal or perfect but is sufficient for reaching an immediate goal or making a practical decision, especially when exhaustive search is infeasible. Heuristics often trade optimality for

speed or simplicity. *Example from text:* In configuration design, a heuristic might be to “allocate costs evenly among components” or to “address the most constrained variable first.” The “drop-link” or “climb-tree” rules in Incremental Concept Learning are heuristics for generalization.

Heuristic Classification: A diagnostic reasoning strategy that integrates both bottom-up (data-driven abstraction) and top-down (hypothesis-driven refinement) classification processes to efficiently map observed data to plausible hypotheses. *Example from text:* A doctor abstracting a patient's specific temperature of 104°F to “high fever” (bottom-up), then using “high fever” to hypothesize “flu,” and then seeking specific flu symptoms to confirm (top-down).

Hierarchical Planning / Hierarchical Task Network (HTN) Planning: A planning paradigm where problems are solved by decomposing high-level tasks into smaller, more manageable subtasks, often organized in a hierarchy. HTN planning uses a library of methods that specify how to break down abstract tasks into sequences of simpler tasks or primitive actions. *Example from text:* Planning to build a tower in Blocks World by using macro-operators like “unstack-all-blocks-onto-table” and “stack-blocks-ascending,” which are themselves composed of primitive “move” actions.

Horn Clause: A clause in propositional or first-order logic that contains at most one positive literal. Horn clauses are a subset of CNF and are important because reasoning with them (e.g., using forward or backward chaining) can be done much more efficiently (often in polynomial time) than general resolution. Prolog is based on Horn clauses. *Example from text:* $\neg A \vee \neg B \vee C$ (equivalent to $(A \wedge B) \Rightarrow C$) is a Horn clause.

Hypothesis Space (in Diagnosis / Learning): The set of all possible explanations, causes, or concept definitions that an agent considers when trying to understand data or learn a concept. In diagnosis, it's the set of potential faults or diseases. In concept learning, it's the set of all possible concept descriptions. *Example from text:* In medical diagnosis, the hypothesis space includes all known diseases that could explain the patient's symptoms. In Version Spaces, the hypothesis space is implicitly defined by the language used to describe concepts.

Implication (Logical Implication): A logical connective, typically denoted by $\Rightarrow$ (or $\rightarrow$), that expresses a conditional relationship: “if P, then Q.” The statement $P \Rightarrow Q$ is false only when P is true and Q is false; otherwise, it is true. *Example from text:* $\text{Has_Feathers}(x) \Rightarrow \text{Bird}(x)$ means “if x has feathers, then x is a bird.”

Implication Elimination: A logical equivalence rule used to rewrite an implication as a disjunction: $P \Rightarrow Q$ is equivalent to $\neg P \vee Q$. This transformation is often used in converting sentences to Conjunctive Normal Form (CNF). *Example from text:* Transforming $\neg \text{Can_Move}(\text{widget}) \Rightarrow \neg \text{Liftable}(\text{widget})$ into $\text{Can_Move}(\text{widget}) \vee \neg \text{Liftable}(\text{widget})$.

Implied Action (in Commonsense Reasoning): An action that is not explicitly stated in a sentence or narrative but can be inferred through context, background knowledge, and commonsense understanding. *Example from text:* The sentence “John fertilized the field” implies the action “John put fertilizer on the field,” which maps to the primitive action MOVE-OBJECT.

Impasse (in Soar / Production Systems): A situation in a production system like Soar where the agent cannot proceed with its decision-making cycle because no production rules match the current state of working memory, or multiple conflicting rules match, leading to an inability to select a single operator. Impasses often trigger learning mechanisms like chunking. *Example from text:* The baseball pitcher agent reaching an impasse when unable to decide between throwing a curveball or a fastball.

Incremental Concept Learning: A learning method where an agent refines its definition of a concept gradually, one example at a time. It typically involves generalizing the concept definition when a new positive example is encountered that is not covered, and specializing it when a negative example is encountered that is incorrectly covered. This process often relies on background knowledge and assumes examples may be presented in a somewhat structured order. *Example from text:* Learning the concept of an “arch” or “foo” by processing a sequence of positive and negative examples and applying heuristics like “drop-link,” “forbid-link,” etc.

Indexing / Indexing Schemes (in Case-Based Reasoning): The process of assigning labels or features (indices) to cases in a case library to facilitate efficient retrieval of relevant cases when a new problem is encountered. Indexing schemes determine how cases are organized and accessed. *Example from text:* Indexing navigation routes by their origin and/or destination coordinates, or by features like “scenic” or “fast.”

Induction (Inductive Reasoning): A form of logical reasoning where general rules or principles are inferred from specific observations or examples. The conclusions of inductive reasoning are probable but not guaranteed to be true, as they go beyond the information explicitly present in the premises. *Example from text:* Observing that several swans are white and inducing the general rule “All swans are white.”

Inference (Logical Inference): The process of deriving new logical conclusions from a set of known premises or axioms using rules of inference. *Example from text:* Using Modus Ponens to infer Bird(Harry) from Has_Feathers(x) $\Rightarrow$ Bird(x) and Has_Feathers(Harry).

Inheritance (in Frames / Semantic Networks): A mechanism in knowledge representation where more specific concepts (subclasses or instances) automatically acquire or inherit properties

(slots and fillers, or attributes and relations) from more general concepts (superclasses) to which they are linked. This promotes knowledge reuse and organization. *Example from text:* A “canary” frame inheriting the “has_wings” property from a “bird” frame.

Initial State: In problem-solving and planning, the state of the world or problem at the beginning of the process, before any actions are taken by the agent. *Example from text:* In the Blocks World, the initial arrangement of blocks on the table. In the painting robot example, the initial state is $\text{On}(\text{Robot}, \text{Floor}) \wedge \text{Dry}(\text{Ladder}) \wedge \text{Dry}(\text{Ceiling})$.

Instance-Based Learning: (See Learning by Recording Cases)

Integrating KBAI: A recurring section in each chapter that discusses how the KBAI concepts covered in that chapter connect with, complement, or enhance modern AI paradigms such as Machine Learning, Deep Learning, Reinforcement Learning, NLP, Generative AI, and Agentic AI.

Intelligence Test (e.g., Raven's Progressive Matrices): Standardized tests designed to measure cognitive abilities, such as abstract reasoning, problem-solving, and general intelligence. In KBAI, AI agents can be designed to take these tests as part of Computational Psychometrics. *Example from text:* Raven's Progressive Matrices (RPM) are used as a key project in the KBAI course.

Junction (in Line Labeling): In visual perception, a point in a 2D line drawing where two or more lines meet or intersect. The type of junction (e.g., L, Y, W, T) provides constraints for interpreting the 3D structure of the depicted object. *Example from text:* The corners of a cube in a line drawing form various types of junctions.

K-Nearest Neighbors (KNN): An instance-based learning algorithm used for classification and regression. For a new data point, KNN finds the K closest training examples (neighbors) in the feature space and assigns the new point a class label based on the majority class among its neighbors (for classification) or a value based on the average of its neighbors (for regression). *Example from text:* Discussed as an extension of the nearest neighbor method for retrieving similar cases.

Knowledge Base: A centralized repository of information, facts, rules, and procedures used by an AI system to reason, solve problems, and make decisions. In KBAI, the knowledge base contains explicitly represented knowledge about a domain. *Example from text:* The set of logical sentences defining properties of animals (e.g., $\text{Has_Feathers}(x) \Rightarrow \text{Bird}(x)$) forms a knowledge base.

Knowledge Gap (in Meta-Reasoning): A situation where an agent recognizes that it lacks the necessary knowledge or a specific piece of information required to solve a problem, complete an explanation, or achieve a goal. Identifying a knowledge gap often triggers a learning goal. *Example from text:* An agent trying to prove an object is a “mug” realizes it lacks the rule connecting “limits heat transfer” to “protects against heat.”

Knowledge Representation: The field of AI concerned with how knowledge can be formally represented in a way that allows a computer system to use it for reasoning and problem-solving. Common KBAI representations include semantic networks, frames, production rules, and logic. *Example from text:* Representing the concept of an “arch” using nodes and links in a semantic network or slots and fillers in a frame.

Learning by Correcting Mistakes: A learning process where an agent refines its knowledge or reasoning strategies by identifying errors in its past performance, explaining the causes of these errors, and then repairing the underlying faults in its knowledge base or decision-making rules. *Example from text:* A robot misclassifying a pail as a cup, then revising its definition of “cup” to exclude objects with movable handles after receiving corrective feedback.

Learning by Recording Cases: A simple form of instance-based learning where an agent stores specific past experiences (cases) in memory. When a new problem arises, the agent retrieves the most similar recorded case and reuses its solution, often without significant adaptation. *Example from text:* Predicting the color of a new block based on the color of the most similar block previously seen. Tying shoelaces by recalling past instances of tying shoelaces.

Lexical Analysis (Lexicon): In natural language processing, the process of analyzing text to identify and categorize individual words or tokens based on their lexical properties (e.g., part of speech, meaning). The lexicon is the vocabulary of a language, including words and their associated information. *Example from text:* Identifying “Ashok” as a noun and “made” as a verb in the sentence “Ashok made pancakes.”

Line Labeling: A technique in computer vision, pioneered by Huffman and Clowes and extended by Waltz, for interpreting 2D line drawings of 3D polyhedral objects. It involves assigning labels (e.g., convex, concave, occluding) to lines based on the types of junctions they form, subject to geometric constraints, to infer the 3D structure. *Example from text:* Labeling edges of a cube as “folds” or “blades” based on Y, L, W, and T junction constraints.

Literal (in Logic): An atomic proposition (a simple declarative statement) or its negation. Literals are the basic building blocks of clauses in forms like Conjunctive Normal Form (CNF). *Example from text:* Has_Wings(x) is a positive literal; $\neg \text{Bird}(x)$ is a negative literal.

Logic (Formal Logic): A formal system of reasoning that uses precise rules and notation to represent knowledge, make inferences, and determine the validity of arguments. Propositional logic and predicate calculus (first-order logic) are common types used in AI. *Example from text:* Using predicates like `Lays_Eggs(x)` and connectives like $\wedge$, $\vee$, $\Rightarrow$, $\neg$ to represent statements and rules.

Long-Term Memory: A component of cognitive architectures (like Soar) that serves as a permanent store for an agent's knowledge, including procedural knowledge (how to do things, often as production rules), semantic knowledge (facts and concepts about the world), and episodic memory (specific past experiences). *Example from text:* In Soar, production rules for a baseball pitcher are stored in procedural long-term memory.

L-Junction (in Line Labeling): A type of junction in a 2D line drawing where two lines meet, typically forming an “L” shape. In the context of interpreting trihedral objects, L-junctions often indicate exterior boundaries of an object, and the lines forming them are usually labeled as blades. *Example from text:* The outer corners of a depicted cube often form L-junctions.

Macro Operator (in Hierarchical Planning): An abstract operator that encapsulates a sequence of lower-level primitive actions or other macro operators. Macro operators allow an agent to reason and plan at a higher level of abstraction, simplifying complex problems. *Example from text:* In the Blocks World, “unstack-all-blocks-onto-table” could be a macro operator composed of several individual “move” actions.

Marr's Framework (for Visual Perception): A highly influential theory proposed by David Marr for understanding visual perception, which breaks down the process into distinct stages or levels of representation: from a primal sketch (detecting edges, bars, blobs from pixel intensities), to a 2.5D sketch (representing surfaces, orientation, and depth relative to the viewer), to a full 3D model representation (object-centered description of shape and spatial organization). *Example from text:* Used to illustrate how constraint propagation can be seen as operating at different stages of visual processing, from line detection to surface grouping to 3D object recognition.

Means-Ends Analysis (MEA): A problem-solving heuristic where an agent iteratively works to reduce the difference between the current state and the goal state. It involves identifying the most significant difference, selecting an operator relevant to reducing that difference, and applying it. If the operator's preconditions are not met, achieving those preconditions becomes a new subgoal. *Example from text:* Used to solve the Blocks World problem by identifying differences (e.g., block A is on table but should be on B) and selecting “move” operators to reduce them.

Memory (Working Memory / Long-Term Memory): A fundamental component of cognitive architectures responsible for storing and retrieving information. Working memory (or short-term memory) holds information currently being processed. Long-term memory stores knowledge more permanently, often categorized into procedural, semantic, and episodic memory. *Example from text:* Soar's architecture includes both working memory and different types of long-term memory. Case-Based Reasoning relies heavily on retrieving cases from memory.

Meta-Reasoning (Metacognition): The cognitive process of “thinking about thinking,” where an agent reflects on, monitors, and controls its own internal reasoning, learning, and knowledge states. This includes identifying errors or gaps in knowledge, selecting appropriate problem-solving strategies, and adapting learning processes. *Example from text:* An agent realizing its current strategy (e.g., Means-Ends Analysis) is stuck in a cul-de-sac and deciding to switch to another strategy (e.g., Problem Reduction).

Model-Based Adaptation (in Case-Based Reasoning): A strategy for adapting a retrieved case to a new problem by using an explicit model of the domain. This model provides causal or structural knowledge that guides how the old solution should be modified. *Example from text:* Adapting a known navigation route from an office to a doctor to a new route from the office to a nearby restaurant by using a map (the model) to find a path from the doctor's location to the restaurant.

Modus Ponens: A fundamental rule of inference in classical logic. It states that if an implication $P \Rightarrow Q$ is true, and its antecedent P is true, then its consequent Q can be inferred to be true. *Example from text:* Given “If an animal has feathers, then it is a bird” ($P \Rightarrow Q$) and “Harry has feathers” (P), we can infer “Harry is a bird” (Q).

Modus Tollens: A fundamental rule of inference in classical logic. It states that if an implication $P \Rightarrow Q$ is true, and its consequent Q is false (i.e., $\neg Q$ is true), then its antecedent P can be inferred to be false (i.e., $\neg P$ is true). *Example from text:* Given “If an animal has feathers, then it is a bird” ($P \Rightarrow Q$) and “Buzz is not a bird” ($\neg Q$), we can infer “Buzz does not have feathers” ($\neg P$).

Molecule (of Knowledge Representation): A metaphor used to describe more complex, structured knowledge representations like frames, which are composed of multiple interconnected elements (slots and fillers), in contrast to simpler, “atomic” representations like individual production rules. *Example from text:* Frames are likened to molecules because they encapsulate a rich set of related information about a concept or stereotype.

Natural Language Processing (NLP): A subfield of artificial intelligence concerned with the interaction between computers and humans through natural language. It involves enabling computers to process, understand, interpret, and generate human language. *Example from text:*

Discussed in relation to understanding sentences, resolving ambiguity, and the role of KBAI concepts in enhancing NLP capabilities.

Natural Language Understanding (NLU): A component of NLP focused specifically on machine reading comprehension, i.e., enabling AI systems to understand the meaning of human language input. KBAI methods like frames and semantic roles are crucial for NLU. *Example from text:* The process of an AI agent interpreting sentences like “Ashok ate a frog” or “David went to the meeting with Ashok by car” involves NLU.

Nearest Neighbor Method: An instance-based learning technique where a new problem or data point is classified or assigned a value based on the single most similar case (its nearest neighbor) in a stored dataset. Similarity is often measured using a distance metric like Euclidean distance. *Example from text:* Used in Learning by Recording Cases to predict the color of a new block based on the color of the most similar block in terms of width and height.

Negation (Logical NOT): A unary logical operator, typically denoted by $\neg$, $!$, or $\sim$, that reverses the truth value of its operand. If a proposition P is true, then $\neg P$ is false, and vice versa. *Example from text:* In $\text{Flies}(x) \wedge \neg \text{Bird}(x) \Rightarrow \text{Bat}(x)$, $\neg \text{Bird}(x)$ means “ x is not a bird.”

Node (in Semantic Networks / Trees): A fundamental unit in graph-based knowledge representations like semantic networks or tree structures like discrimination trees. Nodes typically represent concepts, objects, entities, or decision points, and are connected by links or edges that represent relationships or transitions. *Example from text:* In a semantic network for a Raven's problem, objects like “circle” or “diamond” are represented as nodes. In a discrimination tree, internal nodes are questions, and leaf nodes are cases.

Ontology: In AI, an explicit formal specification of the concepts, categories, properties, and relations among them in a particular domain of knowledge. Ontologies provide a shared vocabulary and a structured framework for representing knowledge, enabling more consistent and reusable knowledge bases. *Example from text:* The ontology of primitive actions (e.g., INGEST, PROPEL, MOVE-OBJECT) used in commonsense reasoning. An ontology classifying “people” as “agents” and “trains” as “conveyances” to help disambiguate prepositional phrases.

Operator (in Planning / Problem Solving): A definition of an action that an agent can perform to transition from one state to another. Operators are typically defined by a set of preconditions (conditions that must be true before the operator can be applied) and a set of postconditions or effects (changes to the state that occur after the operator is applied). *Example from text:* In the Blocks World, $\text{Move}(\text{BlockX}, \text{BlockY})$ is an operator. In the painting robot scenario, climb-ladder and paint-ceiling are operators.

Operational Criteria (in Explanation-Based Learning): A clear, testable definition of a concept that specifies the conditions under which an example is considered an instance of that concept. EBL uses these criteria to guide the construction of explanations. *Example from text:* The operational criteria for a “cup” might be $\text{Is_Stable}(X) \wedge \text{Enables_Drinking}(X)$.

Overgeneralization: A learning error where a concept or rule is applied too broadly, encompassing instances that should not be included. *Example from text:* A child concluding that all four-legged, furry animals are “dogs,” thereby incorrectly including cats.

Partial-Order Planning (POP) / Nonlinear Planning: A planning approach that does not commit to a strict linear sequence of actions from the start. Instead, it builds a plan by adding actions and ordering constraints as needed to achieve goals and resolve conflicts (like goal clobbering) between different parts of the plan. This allows for more flexibility in handling interacting subgoals. *Example from text:* Used to solve the painting robot problem (paint ceiling and ladder) by identifying and resolving the conflict where painting the ladder first makes it unusable for painting the ceiling.

Parsimony (Principle of Parsimony / Occam's Razor): A principle often used in science and hypothesis selection, stating that among competing hypotheses that explain the data equally well, the simplest one (i.e., the one with the fewest assumptions or entities) should be preferred. *Example from text:* In diagnosis, preferring a single disease hypothesis that explains all symptoms over a combination of multiple diseases, if both explanations have similar coverage.

Percept / Perceptual History (P*): A percept is a piece of sensory input or observation that an agent receives from its environment at a given time. The perceptual history (P*) is the sequence of all percepts an agent has experienced up to a certain point. Cognitive agents are often defined as mapping P* to actions. *Example from text:* For a robot, percepts could be camera images or sensor readings. For the baseball pitcher, percepts include the inning, score, runners on base, etc.

Planning: The AI task of finding a sequence of actions (a plan) that an agent can execute to achieve a specific goal or set of goals, starting from a given initial state. Planning involves reasoning about actions, their preconditions, and their effects. *Example from text:* The painting robot planning the sequence of actions: climb ladder, paint ceiling, descend ladder, paint ladder.

Postcondition (Effect): In the definition of an operator for planning, the postconditions describe the changes to the state of the world that result from executing the operator. They specify which facts become true or false after the action. *Example from text:* For the paint-ceiling

operator, postconditions include Painted(Ceiling) (becomes true) and $\neg$ Dry(Ceiling) (becomes true, meaning Dry(Ceiling) becomes false).

Pragmatic Similarity: In analogical reasoning, similarity based on the goals, purposes, or intended functions within the source and target domains, rather than on the features of the objects or the structure of the relations themselves. *Example from text:* The pragmatic similarity between Duncker's radiation problem (goal: destroy tumor without harming tissue) and the military fortress problem (goal: capture fort without triggering mines) lies in the shared high-level goal of achieving an objective while overcoming a critical obstacle.

Precondition: In the definition of an operator for planning, the preconditions are the set of conditions that must be true in the current state of the world for the operator to be applicable or executable. *Example from text:* For the climb-ladder operator, preconditions are $\text{On}(\text{Robot}, \text{Floor}) \wedge \text{Dry}(\text{Ladder})$.

Predicate (in Logic): A function or relation symbol in logic that describes a property of an object or a relationship among objects. When applied to arguments, a predicate forms an atomic proposition that can be true or false. *Example from text:* Has_Feathers(x), Bird(x), On(A, B).

Predicate Calculus (First-Order Logic): An extension of propositional logic that includes predicates, variables, and quantifiers (universal $\forall$ and existential $\exists$). It allows for more expressive statements about objects, their properties, and relations, and for generalizations over collections of objects. *Example from text:* $\forall x (\text{Lays_Eggs}(x) \wedge \text{Flies}(x) \Rightarrow \text{Bird}(x))$.

Primitive Action: In commonsense reasoning and natural language understanding (e.g., Schank's Conceptual Dependency theory), a basic, elemental action that is part of a predefined ontology of actions. Complex events or verbs are often decomposed into or mapped onto these primitive actions. *Example from text:* The list of 14 primitive actions including INGEST, PROPEL, MOVE-OBJECT, SPEAK, CONCLUDE. "John ate ice cream" maps to INGEST.

Problem Reduction: A problem-solving strategy where a complex problem is broken down into a set of simpler, more manageable sub-problems. Solving these sub-problems individually or sequentially contributes to solving the original problem. *Example from text:* In the Blocks World, the goal of achieving A on B $\wedge$ B on C $\wedge$ C on D can be reduced to subgoals like "achieve C on D," then "achieve B on C," etc. Used in conjunction with Means-Ends Analysis to escape cul-de-sacs.

Problem Space: The set of all possible states reachable from an initial state by applying a sequence of valid operators in a problem-solving context. Finding a solution often involves searching for a path through this space from the initial state to a goal state. *Example from text:*

The set of all possible arrangements of guards and prisoners on either side of the river in the Guards and Prisoners problem.

Procedural Memory / Knowledge: A type of long-term memory that stores knowledge about how to perform actions, skills, and procedures. In production systems like Soar, procedural knowledge is often represented as a set of IF-THEN production rules. *Example from text:* In Soar, the rules for deciding whether a baseball pitcher should pitch or walk a batter are part of procedural memory.

Production Rule: A condition-action rule, typically in an IF-THEN format, used in production systems. If the conditions (the IF part, or antecedent) match the current state of working memory, then the actions (the THEN part, or consequent) are executed, potentially modifying working memory or triggering external actions. *Example from text:* IF goal is to escape inning AND 2 outs AND runner on 2nd AND no runner on 1st THEN suggest intentional_walk_operator.

Production System: A cognitive architecture or AI model that uses a set of production rules (stored in production memory or long-term memory) to operate on a working memory (short-term memory containing the current state). The system repeatedly matches rules against working memory and executes the actions of matched rules in a recognize-act cycle. *Example from text:* Soar is an example of a production system architecture.

Propositional Logic (Zeroth-Order Logic): A system of formal logic that deals with propositions (simple declarative statements that can be true or false) and logical connectives (like AND, OR, NOT, IMPLIES) to form more complex sentences. It does not involve variables or quantifiers. *Example from text:* HasFeathers $\Rightarrow$ IsBird where HasFeathers and IsBird are propositions about a specific, implicit entity.

Props (in Scripts): One of the components of a script, referring to the typical objects, artifacts, or resources that are involved in the stereotypical sequence of events described by the script. *Example from text:* In a “Restaurant Script,” props include tables, menu, food, check, money.

Prototypical Concepts: Concepts that are defined by a set of typical or characteristic features rather than by strict necessary and sufficient conditions. Membership in a prototypical category is often graded, with some instances being “better” examples than others. These concepts allow for exceptions and variations. *Example from text:* “Chair” is a prototypical concept; a typical chair has four legs, a seat, and a back, but a stool (no back) or a beanbag can still be considered types of chairs.

Qualia: Subjective, qualitative properties of experience; the “what it is like” aspect of conscious mental states. Examples include the redness of red, the painfulness of pain, or the saltiness of salt. These are often considered challenging to capture or represent in AI. *Example from text:* “Saltiness” is mentioned as an example of an exemplar concept, bordering on qualia, as it's understood through direct sensory experience.

Quantifier (Universal / Existential): In predicate calculus (first-order logic), symbols used to specify the extent to which a predicate or statement applies to a collection of objects. The universal quantifier ($\forall$, “for all”) asserts that a property holds for every object in the domain. The existential quantifier ($\exists$, “there exists”) asserts that there is at least one object in the domain for which the property holds. *Example from text:* $\forall x (\text{Bird}(x) \Rightarrow \text{Has_Wings}(x))$ (For all x , if x is a bird, then x has wings). $\exists x (\text{Bat}(x) \wedge \text{Flies}(x))$ (There exists an x such that x is a bat and x flies).

Raven's Progressive Matrices (RPM): A non-verbal intelligence test used to measure abstract reasoning abilities. Participants are shown a matrix of geometric patterns with one entry missing and must select the correct entry from a set of options to complete the pattern. *Example from text:* A central example used throughout the book to illustrate various KBAI techniques like semantic networks, generate and test, and analogical reasoning. The KBAI course includes a project to build an AI agent to solve RPM problems.

Reactive System / Layer: A type of AI system or a component within a cognitive architecture that responds directly to environmental stimuli with pre-programmed or learned actions, without extensive deliberation or planning. It focuses on immediate stimulus-response behaviors. *Example from text:* In the three-layered cognitive systems architecture, the reactive layer handles immediate responses, like a driver braking when seeing brake lights ahead.

Reasoning: The cognitive process of thinking, understanding, and forming judgments or conclusions logically. In KBAI, reasoning involves manipulating knowledge representations to make inferences, solve problems, plan actions, and learn. It is a core component of deliberation in cognitive architectures. *Example from text:* The AI agent reasoning about whether an object is a “cup” based on its features and a definition of a cup.

Recognize-Act Cycle: The fundamental operational cycle of a production system. In each cycle, the system: 1. Recognizes which production rules have their conditions satisfied by the current contents of working memory. 2. Selects one rule from the matched set (conflict resolution). 3. Acts by executing the actions specified in the chosen rule's THEN part, which typically modifies working memory. *Example from text:* The Soar architecture operates on a recognize-act cycle when the baseball pitcher agent decides on the next pitch.

Recursive Case-Based Reasoning: An adaptation strategy in Case-Based Reasoning where a sub-problem encountered during the adaptation of a retrieved case is itself solved by initiating a new CBR cycle (i.e., retrieving, adapting, evaluating, and storing another case specifically for that sub-problem). *Example from text:* To find a route from home to a restaurant, an agent might retrieve a case for “home to office,” then treat “office to restaurant” as a new sub-problem to be solved by retrieving another case.

Refutation (Proof by Refutation): A proof technique, commonly used in Resolution Theorem Proving, where to prove a statement (goal), one assumes the negation of the statement and adds it to a set of known true axioms. If a contradiction (e.g., the empty clause) can be derived from this augmented set, then the original assumption (the negated goal) must be false, and thus the goal itself is proven true. *Example from text:* Used in Resolution Theorem Proving to prove a widget is not liftable by assuming it is liftable and deriving a contradiction.

Require-Link Heuristic (in Incremental Concept Learning): A specialization heuristic where a concept definition is refined by adding a mandatory relationship or link between elements. This helps to exclude negative examples that lack this required link, making the concept more specific. *Example from text:* If an arch concept incorrectly includes an example where the top brick is not supported, the “require-link” heuristic adds a “must-support” constraint.

Resolution Theorem Proving: A sound and refutation-complete inference rule and algorithm for propositional and first-order logic, particularly suited for automated theorem proving. It works by converting all sentences to Conjunctive Normal Form (CNF) and repeatedly applying the resolution rule to pairs of clauses containing complementary literals to derive new clauses, aiming to derive the empty clause (contradiction) when proving a goal by refutation. *Example from text:* Used to prove that a widget is not liftable given rules like $\neg \text{Can_Move}(w) \Rightarrow \neg \text{Liftable}(w)$ and the observation $\neg \text{Can_Move}(w)$.

Retrieval (in Case-Based Reasoning / Analogical Reasoning): The initial stage in CBR and analogical reasoning where the agent searches its memory (case library or knowledge base) to find past cases or source analogs that are similar or relevant to the current target problem. *Example from text:* Retrieving the “office to doctor” navigation case when faced with the new “office to restaurant” problem.

Roles (in Scripts / Frames): Slots in a script or frame that represent the typical participants or entities involved in the event or concept being described. *Example from text:* In a “Restaurant Script,” roles include Customer (S), Waiter (W), Cook (C), etc. In an “eating” frame, roles include agent, object, beneficiary.

Rule-Based Adaptation (in Case-Based Reasoning): An adaptation strategy where explicit rules or heuristics are used to modify a retrieved case's solution to fit the new problem. *Example from text:* Using a heuristic like “to go from restaurant to home, reverse the directions of the 'home to restaurant' route.”

Rule-Based Expert System: An AI system that uses a knowledge base of facts and IF-THEN rules (often elicited from human experts) and an inference engine to solve problems or provide advice in a specific domain, mimicking the decision-making of a human expert. *Example from text:* Mentioned in the context of production systems being foundational to classical expert systems.

Scene (in Scripts): A component of a script representing a standard subsequence of events that occur within the larger stereotypical situation described by the script. Scripts are often composed of multiple scenes. *Example from text:* In a “Restaurant Script,” scenes might include “Entering,” “Ordering,” “Eating,” and “Exiting/Paying.”

Schema (plural: Schemata or Schemas): A mental framework or structured representation of knowledge about a concept, object, event, or sequence of events. It organizes information and provides expectations. Frames and scripts are types of schemas. *Example from text:* EBL generalizes an explanation into a new rule or schema for recognizing future instances of a concept.

Script: A structured knowledge representation that describes a stereotypical sequence of causally coherent events that occur in a common, familiar situation. Scripts include components like entry conditions, results, props, roles, tracks (variations), and scenes. They help in understanding narratives and predicting events. *Example from text:* The “Restaurant Script” detailing the typical sequence of events when dining out, with tracks for different types of restaurants like coffeehouses or formal dining.

Search (Problem Solving as Search): A fundamental AI paradigm where problem-solving is viewed as searching for a solution within a state space. This involves exploring possible states and transitions (actions/operators) to find a path from an initial state to a goal state. Various search algorithms (e.g., breadth-first, depth-first, A*) exist. *Example from text:* Generate and Test can be seen as a form of search. Planning often involves searching through a space of possible plans or states.

Semantic Analysis (Semantics): In natural language processing and knowledge representation, the process of determining the meaning of words, phrases, sentences, or larger units of text. Semantics is concerned with the interpretation of symbols and structures. *Example from text:* Interpreting the sentence “Ashok made pancakes for David with a griddle” by identifying Ashok as the agent, pancakes as the object, etc. Frames and thematic roles are tools for semantic analysis.

Semantic Memory: A type of long-term memory that stores general factual knowledge about the world, concepts, meanings, and language, independent of personal experience. It contrasts with episodic memory (specific events) and procedural memory (skills). *Example from text:* In Soar, semantic memory might store general concepts like “what a human is” or principles of aerodynamics.

Semantic Network: A knowledge representation formalism that uses a graph structure consisting of nodes (representing concepts, objects, or events) and labeled, directed links (representing relationships between them). *Example from text:* Used to represent Raven's Progressive Matrices problems by creating nodes for shapes and links for relationships like “inside,” “above,” or transformations like “expanded,” “deleted.”

Semantic Roles (Thematic Roles): The underlying semantic relationship that a participant (typically a noun phrase) has with the main verb in a clause. Common semantic roles include agent, patient/theme, instrument, beneficiary, location, source, goal/destination. They help define “who did what to whom, with what, where, when, and why.” *Example from text:* In “David went to the meeting with Ashok by car,” David is the agent, meeting is the destination, Ashok is the co-agent, and car is the conveyance.

Slot (in Frames / Scripts): A named component or attribute within a frame or script that can hold a specific value or refer to another frame (filler). Slots define the structure of the knowledge representation. *Example from text:* An “eating” frame has slots like “subject,” “object,” “location,” “time.” A “cup” frame might have slots for “material,” “shape,” “handle.”

Soar: A cognitive architecture developed by Allen Newell, John Laird, and Paul Rosenbloom, designed to model general intelligence. It is based on a production system and incorporates concepts like working memory, long-term memory (procedural, semantic, episodic), impasses, and learning through chunking. *Example from text:* Used as an exemplar cognitive architecture to illustrate production systems and learning, particularly with the baseball pitcher scenario.

Source Case / Domain (in Analogical Reasoning / CBR): The familiar, well-understood problem, situation, or domain from which knowledge or a solution is retrieved to be applied to a new, less familiar target problem or domain. *Example from text:* In Duncker's radiation problem analogy, the military strategy for attacking a fortress is the source domain.

Spatial Grammar: (See Grammar)

Specialization: The process of refining a concept, rule, or hypothesis to make it more specific or narrower in scope, typically by adding conditions or constraints. This is often done in response to negative examples that are incorrectly covered by a more general definition. *Example from text:*

In Incremental Concept Learning, if a concept of “arch” incorrectly includes an example where supporting bricks touch, the concept is specialized by adding a “must-not-touch” constraint. In Version Spaces, the general model (G-set) is specialized.

Specific Model (in Version Spaces): Also known as the S-set, this is the set of the most specific hypotheses that are consistent with all observed training examples so far, particularly covering all positive examples. It represents the narrowest possible definition of the concept being learned. It is generalized when a new positive example is encountered that it does not cover. *Example from text:* In the allergic reaction example, the specific model might initially be “allergic to breakfast at Sam's on Friday if cheap and not vegan,” which is then generalized based on further positive instances.

Specification Space (in Configuration): The initial phase in the configuration design process where the requirements, constraints, and goals for the artifact to be configured are defined. *Example from text:* For designing a basement, specifications include total area, room types, minimum dimensions, and overall basement dimensions.

Speed-Up Learning: A type of learning where the goal is not necessarily to acquire entirely new knowledge or capabilities, but to learn to perform existing tasks more efficiently or faster. Explanation-Based Learning is often characterized as a form of speed-up learning because it transforms declarative knowledge into more operational (efficient) rules. *Example from text:* EBL creates generalized rules that allow an agent to solve similar problems more quickly in the future.

State (in Problem Solving / Planning): A description of the world or a problem at a particular point in time, typically represented by a set of variables and their current values, or a set of true propositions. Problem-solving and planning involve transitioning between states via operators or actions. *Example from text:* In the Blocks World, a state is a specific arrangement of blocks. In the painting robot example, $\text{On}(\text{Robot}, \text{Floor}) \wedge \text{Dry}(\text{Ladder}) \wedge \text{Dry}(\text{Ceiling})$ is a state.

State-Change Frame (in Commonsense Reasoning): An action frame or a component of an action frame that explicitly represents the changes in the state of an object or agent resulting from an action. *Example from text:* For “Ashok enjoyed eating the frog,” a state-change frame might capture the result $\text{Mood}(\text{Ashok}, \text{Happy})$.

State Space: (See Problem Space)

Stereotype (in Frames / Scripts): A generalized, typical representation of a concept, object, event, or situation, captured by the default structure and slot values in a frame or script. Stereotypes allow for efficient reasoning and expectation generation but can also lead to biases if not applied flexibly. *Example from text:* A frame for “chair” represents a stereotypical chair with

common attributes like legs and a seat. A “Restaurant Script” represents a stereotypical sequence of events when dining out.

Storage (in Case-Based Reasoning / Analogical Reasoning): The final stage in the CBR and analogical reasoning cycles where a newly solved problem, along with its solution and the reasoning process (if successful and novel), is incorporated into the agent's memory (case library or knowledge base) for future use. This is a form of learning. *Example from text:* Storing the successfully adapted navigation route as a new case in the CBR system.

Strategy Integration (in Meta-Reasoning): A metacognitive process where an agent combines or dynamically switches between different problem-solving or learning strategies to tackle a complex problem more effectively. *Example from text:* An agent using case-based reasoning might use rule-based adaptation for one part of a problem and model-based adaptation for another, or switch from Means-Ends Analysis to Problem Reduction when an impasse is reached.

Strategy Selection (in Meta-Reasoning): A metacognitive process where an agent chooses the most appropriate problem-solving or learning strategy from a set of available methods, based on criteria such as the nature of the problem, knowledge availability, computational efficiency, and desired solution quality. *Example from text:* Deciding whether to use Case-Based Reasoning or Incremental Concept Learning based on the characteristics of the input examples and available background knowledge.

STRIPS (Stanford Research Institute Problem Solver): An early and influential AI planning system. The STRIPS representation for operators defines actions in terms of preconditions (a set of positive literals that must be true), an add list (literals that become true after the action), and a delete list (literals that become false after the action). *Example from text:* The convention of using only positive literals in operator preconditions is attributed to STRIPS.

Structural Similarity: In analogical reasoning, similarity based on the pattern of relationships and the organization of components within the source and target domains, rather than on the intrinsic properties of the components themselves. This is often key to deep analogies. *Example from text:* The structural similarity between the solar system (planets revolving around a sun due to gravity) and the atomic model (electrons revolving around a nucleus due to electrostatic force) is based on the shared relational graph structure.

Sub-Action (in Commonsense Reasoning): A more granular action that is part of a larger, more abstract primitive action or event. Understanding sub-actions allows for a deeper, more detailed comprehension of how complex actions are performed. *Example from text:* The primitive action MOVE-OBJECT (e.g., “Ashok put the wedge on the block”) can be decomposed into sub-actions like “grasp wedge,” “move hand,” “release wedge.”

Subgoal: In problem reduction and hierarchical planning, a smaller, intermediate goal that must be achieved as part of solving a larger, more complex problem or achieving a higher-level goal.

Example from text: In Means-Ends Analysis, if an operator's precondition is not met, satisfying that precondition becomes a subgoal. In Problem Reduction, a main goal is broken into several subgoals.

Superficial Similarity: In analogical reasoning, similarity based on surface-level features, attributes, or appearances of objects or entities in the source and target domains, rather than on deeper relational or structural commonalities. *Example from text:* Two problems might be superficially similar if they involve objects of the same color or shape, even if the underlying problem structure is different.

Supervised Learning: A type of machine learning where the agent learns from labeled training data, meaning each example is paired with a correct output or target label (e.g., class, value). The goal is to learn a function that maps inputs to outputs. *Example from text:* Incremental Concept Learning is a form of supervised learning as it uses positive and negative (labeled) examples provided by a “teacher.”

Syntactic Analysis (Syntax): In natural language processing, the process of analyzing the grammatical structure of sentences to determine how words are combined to form phrases, clauses, and sentences according to the rules of a grammar. Syntax is concerned with form rather than meaning. *Example from text:* Identifying “Colorless green ideas” as a noun phrase and “sleep furiously” as a verb phrase in the sentence “Colorless green ideas sleep furiously.”

Target Problem / Domain (in Analogical Reasoning / CBR): The new, unfamiliar, or unsolved problem or situation to which knowledge or a solution from a source case/domain is being applied. *Example from text:* In Duncker's radiation problem analogy, the medical task of destroying a tumor is the target problem.

Task / Knowledge Level (Marr/Newell's Levels): The highest level of abstraction in Marr's or Newell's frameworks for analyzing information processing systems. It describes the goals of the computation, why these goals are appropriate, and the logic of the strategy by which they can be carried out. It focuses on *what* the system does and *why*, and the knowledge it uses. *Example from text:* At the task/knowledge level, an AI playing baseball is concerned with “selecting a pitch” or “playing baseball,” using knowledge about game rules and strategies.

Thematic Roles (Semantic Roles): (See Semantic Roles)

Top-Down Processing: A type of information processing that starts with higher-level knowledge, concepts, expectations, or goals and works downwards to interpret or guide the

processing of lower-level sensory data or specific details. It is knowledge-driven or hypothesis-driven. *Example from text:* When a verb in a sentence activates a specific action frame from memory, the frame's slots create expectations that guide the interpretation of the rest of the sentence (top-down). In classification, the “establish-refine” method is top-down.

Track (in Scripts): A specific variation or instantiation of a more general script, tailored to a particular context or subtype of the general situation. *Example from text:* A “Restaurant Script” might have tracks for “Fast Food,” “Formal Dining,” or “Coffeehouse,” each with slightly different event sequences or roles.

Trihedral Object: A polyhedral object where exactly three surfaces meet at each vertex (corner). Cubes are common examples of trihedral objects. Understanding the geometry of trihedral objects provides constraints for line labeling in visual perception. *Example from text:* A cube is used to illustrate Y, L, W, and T junctions in line labeling.

True Suspicious Features (in Learning by Correcting Mistakes): Features present in positive examples that were incorrectly rejected by an agent (false negatives), which caused the agent to misclassify them. These features are “suspicious” because they might have contributed to the erroneous rejection, and “true” because the example was actually a positive instance of the concept. *Example from text:* If an agent wrongly rejects a cup because its interior is blue (based on a faulty rule), “blue interior” would be a true suspicious feature.

Truth Table: A table that lists all possible combinations of truth values (True or False) for a set of propositional variables and shows the resulting truth value of a complex logical sentence formed using those variables and logical connectives (like AND, OR, NOT, IMPLIES). *Example from text:* Used to define and illustrate the behavior of logical operators like disjunction ($A \vee B$) and implication ($A \Rightarrow B$), and properties like commutativity.

T-Junction (in Line Labeling): A type of junction in a 2D line drawing where one line segment terminates by meeting another line segment, forming a “T” shape. In interpreting 3D scenes, T-junctions often indicate occlusion, where one object or surface is partially hidden behind another. The stem of the T is typically an occluded edge, while the bar of the T is an occluding edge (blade). *Example from text:* Discussed as one of the four main junction types for interpreting trihedral objects.

Undergeneralization: A learning error where a concept or rule is too specific and fails to apply to all appropriate instances. The learned concept is too narrow. *Example from text:* A child concluding that a “dog” must be a black, four-legged animal named Buddy, thus failing to recognize other dogs that don't fit this exact description.

Understanding (in KBAI): A broad cognitive capability involving the interpretation of events, stories, and diverse data from the world by leveraging world constraints, structured knowledge representations (like frames and scripts), and the dynamic interplay of bottom-up and top-down processing. It encompasses making sense of complex information and drawing appropriate inferences. *Example from text:* Understanding the Slabovian earthquake stories by using frames and background knowledge to disambiguate the meaning of “killed.”

Understanding (Natural Language Understanding - NLU): (See Natural Language Understanding)

Universal Quantifier ($\forall$): A symbol in predicate calculus (first-order logic) used to assert that a given predicate or statement is true for all individuals or objects in the domain. It is read as “for all” or “for every.” *Example from text:* $\forall x (\text{Bird}(x) \Rightarrow \text{Has_Wings}(x))$ means “For all x, if x is a bird, then x has wings.”

Variabilization (in Incremental Concept Learning): The process of transforming specific instances or constants in an example into variables to create a more generalized concept definition. This allows the concept to apply to a wider range of similar instances. *Example from text:* When learning the concept of an “arch” from an example with specific bricks labeled A, B, C, variabilization involves replacing these specific labels with variables like “Brick1,” “Brick2,” “Brick3,” representing any bricks that satisfy the structural relationships.

Version Spaces: A concept learning algorithm that incrementally builds a description of a concept by maintaining two sets of hypotheses: a set of the most specific hypotheses (S-set) consistent with the training examples, and a set of the most general hypotheses (G-set) consistent with the training examples. The true concept is expected to lie between these two sets. As more examples are processed, the S-set is generalized and the G-set is specialized, ideally converging to a single concept description. *Example from text:* Used to learn the conditions under which an allergic reaction occurs by processing positive and negative examples of restaurant visits.

Visuospatial Reasoning: The cognitive ability to perceive, analyze, manipulate, and reason about visual patterns, spatial relationships, and the geometric properties of objects and scenes. *Example from text:* Interpreting 2D line drawings as 3D objects (like a cube) using line labeling and junction constraints. Solving Raven's Progressive Matrices involves visuospatial reasoning.

Weak Methods (in AI Problem Solving): General-purpose problem-solving methods that rely on limited domain-specific knowledge and can be applied across a wide range of problems. They are often heuristic in nature and do not guarantee optimal solutions but provide a starting point when strong, domain-specific methods are unavailable. *Example from text:* Generate and Test,

Means-Ends Analysis, and Problem Reduction are described as weak methods because their effectiveness is not tightly coupled with deep knowledge of a specific domain.

W-Junction (in Line Labeling): A type of junction in a 2D line drawing where three lines meet, with one of the angles between adjacent lines being greater than 180 degrees, forming a shape resembling a “W” or an arrowhead. In interpreting trihedral objects, W-junctions typically have a “blade-fold-blade” labeling pattern. *Example from text:* One of the four main junction types used for interpreting line drawings of cubes.

Working Memory (Short-Term Memory): A component of cognitive architectures (like Soar) that holds and manipulates information currently being used for active processing, reasoning, and decision-making. It has limited capacity and duration compared to long-term memory. *Example from text:* In Soar, the current state of the baseball game (inning, score, batter, etc.) is held in working memory and matched against production rules.

Y-Junction (in Line Labeling): A type of junction in a 2D line drawing where three lines meet, and all angles between adjacent lines are less than 180 degrees, forming a “Y” shape. In interpreting trihedral objects, Y-junctions typically indicate a corner where three surfaces meet, and the lines are usually labeled as folds. *Example from text:* The central junction on the front face of a depicted cube often forms a Y-Junction.

Recommended Readings

CLASSICAL ARTIFICIAL INTELLIGENCE

Artificial Intelligence (Patrick Winston)

Patrick Winston's "Artificial Intelligence" is a seminal work in the field of classical AI, providing a comprehensive overview of the foundational techniques and theories that have shaped the development of artificial intelligence. While Winston's book is renowned for its thorough exploration of classical AI techniques and foundational theories, this KBAI book looks into how AI systems can be engineered to mimic and understand human cognition. This nuanced approach sets it apart, highlighting the symbiotic relationship between AI and cognitive science. The KBAI book not only covers the traditional algorithms and computational methods foundational to AI but also emphasizes the importance of integrating these with insights from human thought processes, offering a more holistic view of what artificial intelligence can achieve by learning from human intelligence.

Artificial Intelligence (Elaine Rich and Kevin Knight)

"Artificial Intelligence" by Elaine Rich and Kevin Knight is a seminal text in the area of Classical AI, offering a comprehensive exploration of the fundamental concepts and techniques that

define the field. Similar to Patrick Winston's work, Rich and Knight's book examines the core aspects of artificial intelligence, such as problem-solving, knowledge representation, planning, and learning, providing a solid foundation in what is now known as Classical AI. Unlike this KBAI book, which intricately examines the intersection of AI with human cognitive processes, Rich and Knight's text remains focused on the traditional computational and logical methodologies that have historically driven the development of AI. While this KBAI book seeks to blend AI principles with insights from cognitive science to understand and replicate human intelligence, "Artificial Intelligence" by Rich and Knight stands as a thorough guide to the classical theories and techniques, offering a detailed and systematic approach to understanding the mechanisms of artificial intelligence in its traditional form.

Artificial intelligence: A Modern Approach (Stuart Russell and Peter Norvig)

"Artificial Intelligence: A Modern Approach" by Stuart Russell and Peter Norvig is a comprehensive survey of the entire field of AI, widely recognized for its extensive coverage that makes it an invaluable reference for students and professionals alike. The book meticulously explores the full spectrum of artificial intelligence, from foundational principles to cutting-edge research, making it a quintessential guide to understanding the breadth and depth of the discipline. In contrast to this KBAI book's focused examination of the nexus between classical AI and human cognition, Russell and Norvig's work offers a broader perspective, encapsulating a wide array of topics including machine learning, robotics, and problem-solving techniques. While this KBAI book examines how AI systems can mimic and understand human cognitive processes, "Artificial Intelligence: A Modern Approach" provides a panoramic view of the field, presenting a diverse range of AI methodologies and their applications in a structured and accessible manner.

Introduction to Knowledge Systems (Mark Stefik)

"Introduction to Knowledge Systems" by Mark Stefik is a focused examination of knowledge systems, particularly expert systems, and the engineering principles behind their development. Stefik's work examines the intricacies of designing and implementing knowledge-based systems, offering detailed insights into the architectures, reasoning mechanisms, and practical applications of these systems. Unlike the broader approach of the KBAI book, which explores the intersection of classical AI and human cognition, Stefik's book zeroes in on the specific domain of knowledge systems, providing a deep dive into the methodologies and techniques used to create intelligent systems that emulate expert human decision-making. While the KBAI book aims to bridge AI and cognitive science, presenting a broader perspective on how AI can mimic and understand human intelligence, Stefik's "Introduction to Knowledge Systems" is more

narrowly tailored, focusing on the practical aspects of building systems that capture and utilize expert knowledge. This distinction makes Stefik's work an essential guide for those specifically interested in the development and application of knowledge-based systems, while the KBAI book offers a more integrated view of AI, blending technical approaches with cognitive insights to foster a deeper understanding of intelligent systems.

COGNITIVE SCIENCE

A Thousand Brains: A New Theory of Intelligence (Jeff Hawkins)

Jeff Hawkins' *A Thousand Brains* proposes a novel theory of how the brain, particularly the neocortex, creates intelligence. It focuses on the concept of "reference frames" and how the cortex learns a model of the world through sensory input and movement. While this KBAI book explores how AI systems can be built using explicit knowledge representations and reasoning mechanisms to achieve intelligent behavior, Hawkins' work offers a neuroscientifically grounded perspective on how intelligence might emerge from the brain's architecture. *A Thousand Brains* provides a compelling model of biological intelligence that can inspire new approaches in AI, complementing KBAI's focus on cognitive functions with a theory of their potential neural implementation.

On Intelligence (Jeff Hawkins with Sandra Blakeslee)

In *On Intelligence*, Jeff Hawkins presents his earlier "memory-prediction framework" theory of how the brain works, arguing that the neocortex constantly makes predictions about future sensory inputs based on stored patterns (memories). This predictive capability is posited as the core of intelligence. This KBAI book focuses on building AI systems that reason and learn using structured knowledge. Hawkins' work, in contrast, offers a theory of how the brain itself might achieve understanding and prediction through a fundamentally different, pattern-based mechanism. For readers of KBAI, *On Intelligence* provides a thought-provoking perspective on the nature of biological intelligence and how its principles might inform or contrast with knowledge-based approaches to AI.

The Deep Learning Revolution (Terrence J. Sejnowski)

Terrence J. Sejnowski's *The Deep Learning Revolution* chronicles the history and explains the principles behind deep learning, one of the most impactful areas of modern AI. It details how neural networks, inspired by the brain's structure, learn from vast amounts of data to perform

tasks like image recognition and natural language processing. While this KBAI book focuses on explicit knowledge representation and symbolic reasoning, Sejnowski's work explores the power of connectionist approaches where knowledge is often implicit in the learned weights of a network. *The Deep Learning Revolution* offers a crucial understanding of data-driven AI, which stands in contrast to, but is increasingly being integrated with, the knowledge-based methods discussed in KBAI to create more robust and versatile AI systems.

The Computational Brain (Patricia S. Churchland and Terrence J. Sejnowski)

The Computational Brain by Churchland and Sejnowski is a foundational work in computational neuroscience that explores how neural networks and computational models can explain various brain functions and cognitive phenomena. It bridges the gap between neuroscience and computation, examining how the brain's physical structures give rise to complex behaviors and mental processes. While this KBAI book approaches intelligence through the lens of knowledge representation and symbolic reasoning, *The Computational Brain* offers a complementary perspective by investigating the neural-level mechanisms that might implement cognitive functions. It provides readers with an understanding of how the brain computes, which is essential for those interested in biologically plausible AI and the neural basis of cognition.

NEUROSCIENCE

Neuroscience (Edited by Dale Purves et al.)

Neuroscience (Edited by Dale Purves et al.) *Neuroscience*, often referred to simply as "Purves," is a widely respected and comprehensive textbook that provides a detailed overview of the field of neuroscience. It covers a vast range of topics, from molecular and cellular neuroscience to sensory and motor systems, and higher cognitive functions. While this KBAI book focuses on the computational and representational aspects of intelligence from a cognitive and AI perspective (the "mind"), Purves delves into the biological and physiological underpinnings of these processes (the "brain"). Understanding the neural hardware and its organization, as detailed in Purves, can provide a valuable context for appreciating the constraints and inspirations for cognitive architectures and mechanisms discussed in KBAI. It offers a complementary perspective, grounding abstract models of thought in the physical realities of neural systems.

Brain and Behavior: An Introduction to Behavioral Neuroscience (David Eagleman and Jonathan Downar)

Eagleman and Downar's *Brain and Behavior* offers an engaging introduction to how the brain gives rise to thought, emotion, and behavior. The book is known for its accessible style and its focus on connecting neural mechanisms to observable human actions and experiences. While this KBAI book explores how we can model cognitive functions like reasoning, learning, and problem-solving using computational approaches, *Brain and Behavior* examines these same functions from the perspective of their neural substrates. It provides insights into the biological machinery that enables cognition, which can inform and constrain the design of AI systems aiming for human-like intelligence. For readers of KBAI interested in the "wetware" that supports the "software" of the mind, this book offers a compelling bridge between neuroscience and the study of behavior and cognition.

Appendix I – Raven’s Progressive Matrices

This appendix provides a thorough guide to the Raven’s Progressive Matrices (RPM) project, a cornerstone of Georgia Tech’s Knowledge-Based Artificial Intelligence (KBAI) course.³⁷ The project tasks students with developing an AI agent capable of solving RPM problems, which are widely used to measure human intelligence. The project blends AI reasoning techniques with cognitive psychology, offering students the opportunity to reflect on the parallels between human and AI problem-solving strategies.²⁴⁹

This guide will cover the project's objectives, detailed instructions for each milestone, and key concepts such as knowledge representation, reasoning, and computational psychometrics. By following these instructions, you will be able to develop an AI agent that mirrors human cognitive processes while tackling challenging visual analogy tasks.

INTRODUCTION TO RAVEN’S PROGRESSIVE MATRICES (RPM)

Raven’s Progressive Matrices (RPM) are a form of non-verbal intelligence test designed to measure abstract reasoning. The matrices consist of a grid of images, typically in a 2x2 or 3x3 layout, with one image missing. Participants must infer the rules governing the progression of the matrix and select the missing image from a set of options.

2x2 Basic Problem 12

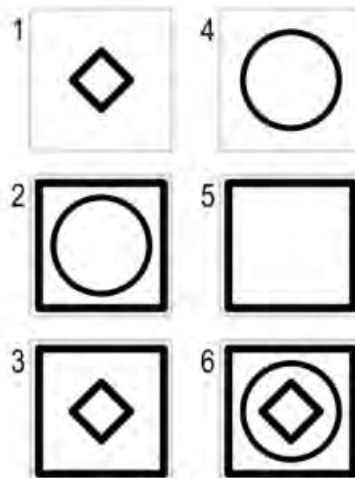
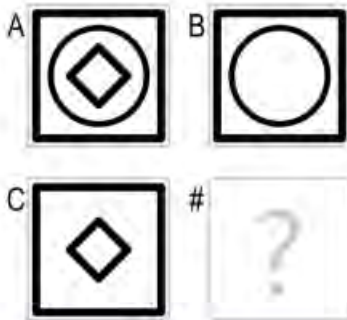


Figure 0-1 Example of a Raven's Progressive Matrices 2x2 problem

3x3 Basic Problem 22

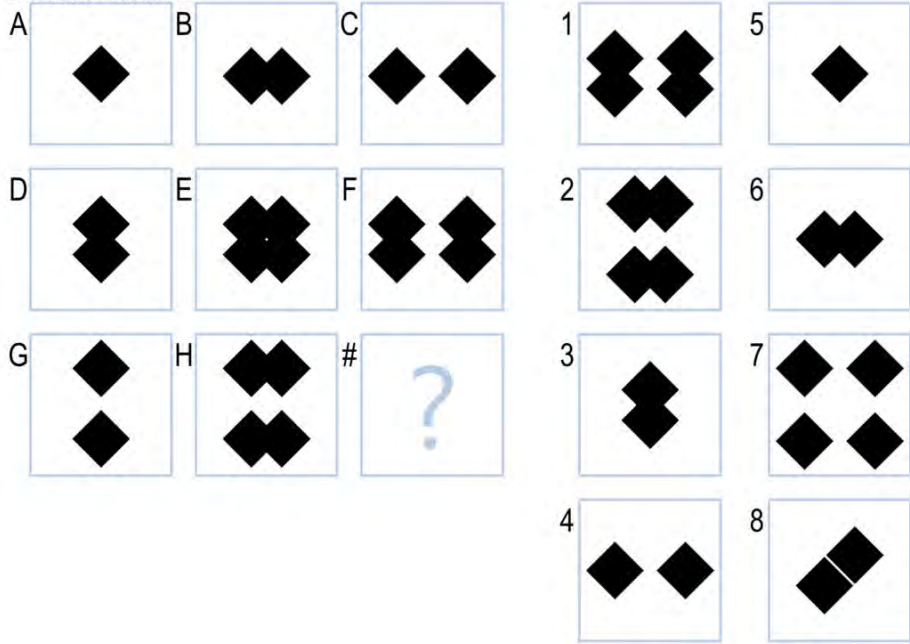


Figure 0-2 Example of a Raven's Progressive Matrices 3x3 problem

Your task is to create an AI agent capable of solving these problems by recognizing the patterns and relationships that exist between the images. In doing so, you will explore various AI techniques and reflect on their connection to human cognition.

Key Project Concepts

- **Knowledge Representation:** AI agents require structured data representations to reason about the relationships between elements in the matrices. This involves capturing attributes such as shape, color, orientation, and movement in a way that the agent can process.
- **Reasoning Techniques:** To solve RPM problems, AI agents employ various reasoning strategies, including rule-based reasoning, case-based reasoning, and meta-reasoning. These techniques allow agents to infer the rules governing the relationships between matrix elements.

- **Computational Psychometrics:** This field examines how AI agents can take tests typically administered to humans and use the results to generate insights into human cognition. By comparing AI performance with human performance, you can hypothesize about cognitive processes.
- **Problem Space and Hypothesis Space:** The problem space refers to the set of all possible matrix configurations. The hypothesis space consists of potential solutions or transformations the agent can use to solve the matrix (e.g., rotation, reflection, scaling).

Learning Outcomes

By completing this project, you will:

- Design and implement an AI agent that uses knowledge-based AI techniques to solve RPM problems.
- Understand key AI reasoning methods and their applicability to practical tasks.
- Use AI agents to reflect on human cognition, comparing AI reasoning strategies with human problem-solving processes.
- Analyze errors made by AI agents and humans to generate hypotheses about cognitive behavior.

MILESTONE 1: DEFINING THE PROBLEM REPRESENTATION

The first milestone is to define how your AI agent will represent the elements of an RPM problem. This includes deciding how to structure the information about the images, relationships between them, and the transformations that govern the matrix.

Key Tasks

Structured Data Representation

- Define attributes for each image (e.g., shape, color, size, position, rotation).
- Ensure that your representation can capture both horizontal and vertical relationships between images, as many RPM problems require reasoning along both axes.
- Consider including additional attributes for advanced matrices, such as the number of transformations an object undergoes.

Set up the Hypothesis Space

- The hypothesis space contains potential transformations (e.g., rotation, reflection, scaling) that your agent will use to infer the missing image. Make sure your hypothesis space is comprehensive enough to solve various RPM types.

Deliverables

- A detailed description of your chosen data representation scheme.
- An explanation of the hypothesis space and how it will be explored to identify the correct solution.

Additional Insights

Humans can intuitively recognize patterns in RPM problems. However, teaching an AI to recognize and apply these patterns requires clear and structured representations. Ensure that your agent is capable of processing both low-level features (e.g., shape, size) and higher-order transformations (e.g., reflection or scaling across an entire row or column).

MILESTONE 2: IMPLEMENTING REASONING STRATEGIES

The next step is to implement the reasoning strategies that your agent will use to generate and evaluate potential solutions for RPM problems.

Key Tasks

Reasoning Techniques

- **Rule-Based Reasoning:** Develop a set of rules that can be applied to the matrix (e.g., “If shape A is rotated by 90 degrees, then shape B should also be rotated by 90 degrees”).
- **Case-Based Reasoning:** Implement a method where your agent recalls and adapts solutions from past problems.
- **Meta-Reasoning:** Equip your agent with the ability to reflect on its reasoning process. If a strategy repeatedly fails, the agent should be able to switch to an alternative approach.

Generate-and-Test Strategy

- The generate-and-test strategy allows the agent to propose possible solutions and evaluate them against the known relationships in the matrix. The agent should systematically test each hypothesis and refine its approach based on failures.

Handling Ambiguity

- Some RPM problems may have multiple valid answers. Your agent should be able to handle ambiguity by evaluating the most probable or “best fit” solution, much like human test-takers do.

Deliverables

- A detailed description of each reasoning technique used by the agent.
- A clear explanation of how the generate-and-test strategy is implemented.
- An analysis of how the agent handles ambiguous cases, where multiple solutions are plausible.

Additional Insights

Humans often rely on intuition and visual reasoning to solve RPM problems, making connections between images in a way that feels natural. For your AI agent, generating this “intuitive” behavior may require complex rule sets and reasoning strategies. Keep this in mind when implementing the generate-and-test process.

MILESTONE 3: META-REASONING AND ERROR ANALYSIS

Meta-reasoning allows your agent to reflect on its own problem-solving process, improving performance over time. Additionally, you will analyze the errors made by the AI agent and compare them to human test-takers.

Key Tasks

Implement Meta-Reasoning

- Develop a system that allows your agent to assess its own performance and adjust its strategy when necessary. For example, if the agent notices that it consistently fails on rotational problems, it should adjust its hypothesis space to better handle such cases.

Compare AI and Human Errors

- As part of computational psychometrics, you will compare the types of errors made by your AI agent with errors made by human test-takers. This analysis can offer insights into the similarities and differences between human and machine cognition.
- Investigate whether the AI’s reasoning is parallel to human reasoning (e.g., does it tend to make the same mistakes on complex visual transformations?).

Deliverables

- A report detailing how the agent uses meta-reasoning to refine its problem-solving strategy.
- A comparative analysis of AI vs. human errors on RPM problems.

Additional Insights

Meta-reasoning is critical for improving AI performance on complex tasks. Humans, when faced with difficulty, often rethink their strategies and adapt. This milestone challenges you to replicate that adaptive behavior in your AI agent.

MILESTONE 4: EVALUATION AND FINAL REFLECTION

The final milestone involves evaluating the overall performance of your AI agent and reflecting on how its reasoning compares to human cognition.

Key Tasks

Evaluate the AI's Performance

- Run your agent through a series of RPM problems of increasing difficulty and evaluate its performance. Use metrics such as accuracy, time to solution, and error rates.
- Compare your AI's performance with benchmarks from human test-takers, analyzing areas where the AI performs well and areas where it struggles.

Reflect on AI and Human Cognition

- Reflect on the extent to which your AI agent mimics human cognition. Are there patterns in human reasoning that the AI struggles to replicate? What insights can be gained from these differences?
- Consider the implications of your findings for the broader field of AI and cognitive science.

Deliverables

- A comprehensive evaluation of the AI's performance on the RPM problems, with comparisons to human performance.
- A reflection on how the project informs our understanding of human cognition and what this means for the future of AI.

Additional Insights

This milestone provides an opportunity to look back on the entire project and consider the broader implications of your work. By understanding where the AI excels or falters in comparison to humans, you can draw meaningful conclusions about the nature of cognition itself.

CONCLUSION

The Raven's Progressive Matrices project allows students to explore the intricate relationship between AI reasoning and human cognition. By developing an AI agent capable of solving visual analogy problems, you will gain deeper insights into both AI techniques and cognitive processes.

The key to success in this project is not only developing a capable agent but also reflecting on the broader implications of its reasoning patterns. As you work through each milestone,

consider how the AI’s strategies mirror or diverge from human cognition, and what this reveals about both.

Final Deliverable:

- A comprehensive final report including all deliverables from each milestone, a fully functioning AI agent, and a reflection on the project’s cognitive implications.

RECOMMENDED READING

For deeper insights into the topics covered in this project, the following readings are recommended:

- Artificial Intelligence by Patrick Winston⁹⁶
- Knowledge Systems by Mark Stefik²⁵⁰
- Artificial Intelligence: A Modern Approach by Stuart Russell and Peter Norvig¹⁰³

With this thorough guide, you are now well-prepared to tackle the Raven’s Progressive Matrices project. This project not only enhances your understanding of KBAI but also opens the door to fascinating explorations in human cognition and computational psychometrics.

ADDITIONAL READINGS & REFERENCES

- 37 Kunda, M., McGregor, K. & Goel, A. K. in *AAAI Fall Symposium: Multi-Representational Architectures for Human-Level Intelligence*.
- 96 Winston, P. *Artificial Intelligence*. (Pearson, 1992).
- 103 Russell, S. & Norvig, P. *Artificial Intelligence: A Modern Approach*. (Pearson, 2015).
- 249 Leysen, J. Towards the planning of actions in complex environments. *European journal of operational research* **37**, 83-91 (1988).
- 250 Stefik, M. *Introduction to knowledge systems*. (Elsevier, 2014).

Index

- abstraction, 3, 107, 199, 328, 385, 407, 417, 463, 567, 635
- action frame, 367, 374, 379, 635
- adaptation, 163, 166, 168, 169, 171, 172, 190, 619, 620, 630, 635, 653, 660
- agent. *See* cognitive agent
- agentic AI, 23, 24, 53, 190, 228, 229, 251, 252, 253, 293, 294, 295, 331, 332, 636, 650
- analogical mapping, 449, 636, 641
- analogical reasoning, xx, 21, 23, 431, 432, 442, 462, 463, 464, 636, 644, 659, 661, 663, 664
- analogical transfer, 452, 453, 454, 636
- antecedent, 637
- architecture. *See* cognitive architecture
- associative property, 269, 272, 273, 274, 637
- atom, 637
- axiomatic concepts, 243, 244, 637
- background knowledge, 196, 211, 223, 471, 490, 637
- blade, 638
- Blame Assignment. *See* credit assignment
- blocks world, 74, 76, 78, 80, 81, 83, 87, 90, 99, 164, 165, 298, 300, 319, 332, 638, 642, 643, 647, 648, 650, 652, 654, 656, 662
- bottom-up processing, 638
- case-based reasoning, 21, 157, 160, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 174, 187, 188, 189, 190, 191, 199, 401, 402, 431, 432, 470, 535, 550, 552, 555, 557, 558, 619, 635, 638, 643, 644, 649, 653, 659, 660, 663, 679
- causal explanation, 638, 645
- causality, 413, 414, 415, 416, 638, 645

- CBR. *See* case-based reasoning
- chunking, 116, 117, 118, 638
- classification, 20, 147, 159, 231, 232, 233, 236, 248, 249, 250, 251, 257, 401, 499, 535, 550, 551, 558, 567, 568, 639, 644, 648
- clause, 285, 639, 644
- climb-tree, 217, 639
- close-interval, 217, 229, 639
- CNF. *See* conjunctive normal form
- cognitive agent, 636
- cognitive architecture, 104, 107, 108, 554, 600, 637, 639
- cognitive system, 15, 639
- commonsense reasoning, 331, 359, 371, 373, 376, 378, 383, 400, 639, 649, 662, 663
- commutative property, 269, 270, 639
- computational psychometrics, xix, 640, 650, 678
- Concept. *See* conceptual hierarchy
- conceptual combination, 463, 640
- conceptual hierarchy, 640
- configuration, 22, 535, 536, 537, 539, 540, 541, 542, 543, 544, 546, 547, 550, 551, 552, 553, 554, 555, 556, 557, 558, 640, 662
- conjunction, 285, 640
- conjunctive normal form, 284, 285, 288, 295, 639, 640, 643, 648, 651, 659
- consequent, 641
- constraint propagation, 21, 509, 510, 514, 517, 526, 529, 530, 531, 641
- constraint satisfaction problem, 557, 641
- credit assignment, 641
- cross-domain analogy, 641
- CSP. *See* constraint satisfaction problem
- cul-de-sac, 642
- data space, 565, 642
- De Morgan's Law, 274, 643
- decision tree, 499, 500, 501, 502, 503, 504, 642
- deduction, 569, 570, 642
- deductive reasoning, 642
- deep similarity, 444, 642
- default values, 132, 642
- deliberation, 9, 10, 15, 614, 643
- diagnosis, 22, 561, 562, 564, 565, 567, 568, 573, 575, 577, 578, 579, 580, 628, 642, 643, 648
- discrimination tree, 181, 183, 184, 186, 187, 470, 643
- disjunction, 266, 285, 643
- distributive property, 269, 271, 272, 643
- drop-link, 205, 217, 644
- EBL. *See* explanation-based learning
- enlarge-set, 217, 644
- episodic memory, 644
- equivalence classes, 236, 644
- existential quantifier, 283, 645
- expectation generation, 396, 397, 645
- expert system, 660
- explanation-based learning, 21, 407, 421, 426, 427, 431, 635, 637, 638, 645, 655, 662
- false suspicious features, 591, 592, 593, 645
- filler, 646
- first-order logic, 656
- fitness function, 646
- fold, 646
- forbid-link, 216, 646
- formal logic, 263, 289, 652
- frame, 138, 339, 341, 357, 646, 662
- frames, 125, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 142, 143, 144, 245, 338, 350, 352, 355, 366, 367, 368, 370, 371, 372, 381, 396, 400, 642, 646, 649, 653, 659, 660, 661, 662

- general model, 474, 478, 481, 485, 486, 646
- generalization, 201, 202, 209, 211, 215, 217, 222, 228, 417, 469, 472, 494, 506, 507, 647
- generate and test, 17, 55, 56, 58, 63, 67, 69, 70, 71, 402, 646, 660, 666
- genetic algorithm, 65, 69, 71, 646
- goal. *See* goal state
- goal clobbering, 309, 647
- goal interference, 647
- goal state, 74, 301, 647
- goal-based autonomy, 624, 630, 647
- heuristic, 308, 568, 639, 644, 646, 647, 648, 659
- hierarchical planning, 648, 652
- hierarchical task network, 325, 329, 330, 648
- horn clause, 648
- HTN. *See* hierarchical task network
- hypothesis space, 565, 648, 678, 679
- impasse, 649
- implication elimination, 278, 648
- implied action, 649
- incremental concept learning, 20, 147, 193, 194, 196, 199, 200, 223, 225, 226, 228, 400, 470, 471, 472, 473, 478, 479, 480, 618, 635, 637, 639, 644, 646, 647, 648, 649, 659, 662, 663, 664, 666
- indexing scheme, 434, 649
- induction, 569, 570, 649
- inductive reasoning, 292, 569, 649
- inference. *See* logical inference
- inheritance, 133, 649
- initial state, 301, 650
- instance-based learning. *See* learning by recording cases
- integrating KBAI, 23, 24, 52, 69, 98, 120, 142, 158, 189, 226, 251, 293, 330, 355, 384, 403, 427, 464, 506, 531, 556, 578, 602, 630, 636, 650
- intelligence test, 67, 95, 650
- junction, 520, 524, 650, 652, 665, 667
- K-Nearest Neighbors, 156, 157, 158, 160, 190, 650
- KNN. *See* K-Nearest Neighbors
- knowledge base, 286, 287, 603, 650
- knowledge gap, 651
- knowledge representation, 28, 579, 637, 651, 653, 677
- learning by correcting mistakes, 23, 583, 584, 602, 611, 645, 651, 665
- learning by recording cases, 147, 149, 158, 160, 163, 164, 400, 650, 651, 654
- lexical analysis, 337, 370, 651
- lexicon, 34, 370, 651
- line labeling, 520, 638, 646, 650, 651, 652, 665, 667
- literal, 285, 288, 651
- logic. *See* formal logic
- logical inference, 281, 649
- long-term memory, 652, 653
- macro operator, 652
- Marr, 104, 517, 518, 520, 533, 636, 652, 664
- MEA. *See* means-ends analysis
- means-ends analysis, 73, 74, 75, 76, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 95, 96, 97, 98, 99, 104, 299, 307, 308, 309, 310, 311, 315, 401, 636, 638, 642, 643, 652, 653, 656, 663, 664, 667
- metacognition, xx, 10, 15, 23, 601, 614, 615, 616, 619, 622, 624, 625, 626, 627, 628, 629, 653
- meta-reasoning, 23, 607, 608, 611, 621, 622, 623, 630, 651, 653, 663, 679, 680, 681
- Modus Ponens, 279, 280, 281, 282, 284, 292, 294, 642, 649, 653
- Modus Tollens, 279, 280, 281, 282, 284, 294, 653

- molecule, 653
- natural language processing, 23, 24, 121, 143, 190, 227, 251, 253, 293, 355, 384, 403, 532, 653
- natural language understanding, 144, 354, 654, 666
- nearest neighbor method, 151, 654
- Newell, 104, 108, 636, 661, 664
- NLP. *See* natural language processing
- NLU. *See* natural language understanding
- node, 654
- nonlinear planning. *See* partial-order planning
- ontology, 53, 344, 345, 363, 654
- operational criteria, 419, 655
- operator, 321, 654
- overgeneralization, 472, 655
- parsimony. *See* principle of parsimony
- partial-order planning, 298, 300, 311, 312, 314, 315, 317, 318, 330, 332, 638, 655
- percept, 655
- perceptual history, 655
- POP. *See* partial-order planning
- postcondition, 303, 655
- pragmatic similarity, 656
- precondition, 303, 656
- predicate, 283, 284, 294, 656
- predicate calculus. *See* first-order logic
- primitive action, 372, 373, 640, 656
- principle of parsimony, 572, 655
- problem reduction, 17, 73, 74, 88, 89, 90, 95, 96, 97, 98, 99, 100, 315, 401, 420, 638, 653, 656, 663, 664, 667
- problem space, 656, 662, 678
- procedural memory, 657
- production rule, 657
- production system, 657
- production systems, 17, 101, 108, 109, 112, 116, 120, 122, 139, 400, 626, 638, 649
- proof by refutation, 285, 659
- propositional logic, 256, 300, 657
- prototypical concept, 244, 657
- qualia, 246, 658
- quantifier, 283, 658, 666
- Raven's Progressive Matrices. *See* intelligence test
- reactive system, 658
- recognize-act cycle, 113, 658
- refutation. *See* proof by refutation
- require-link, 216, 659
- resolution theorem proving, 255, 284, 285, 289, 290, 295, 639, 640, 643, 644, 659
- retrieval, 150, 151, 154, 159, 160, 163, 166, 185, 186, 187, 189, 434, 442, 462, 619, 659
- roles, 112, 338, 342, 343, 366, 367, 368, 369, 393, 394, 659, 661, 664
- scene, 395, 396, 660
- schema, 418, 660
- script, 392, 393, 394, 395, 396, 398, 399, 400, 401, 644, 657, 659, 660, 663, 665
- scripts, 389, 390, 391, 392, 393, 396, 398, 399, 400, 401, 402, 403, 404, 405, 642, 644, 645, 657, 659, 660, 661, 662, 665
- semantic analysis, 338, 346, 353, 660
- semantic memory, 661
- semantic network, 30, 31, 32, 33, 38, 46, 68, 661
- Soar, 108, 109, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 144, 637, 638, 644, 649, 652, 653, 657, 658, 661, 667
- source case, 661
- spatial grammar. *See* syntactic grammar
- specialization, 201, 205, 207, 215, 217, 220, 229, 496, 661
- specific model, 474, 478, 481, 485, 486, 662

- specification space, 541, 662
- speed-up learning, 662
- Stanford Research Institute Problem Solver, 304, 663
- stereotype, 662
- storage, 164, 166, 174, 175, 176, 177, 443, 456, 619, 663
- strategy integration, 619, 620, 663
- strategy selection, 616, 617, 618, 630, 663
- STRIPS. *See* Stanford Research Institute Problem Solver
- structural similarity, 455, 663
- sub-action, 663
- subgoal, 420, 664
- superficial similarity, 664
- supervised learning, 226, 664
- syntactic analysis, 337, 664
- syntactic grammar, 647
- target problem, 664
- top-down processing, 664
- track, 394, 665
- triangular object, 665
- true suspicious feature, 592, 665
- truth table, 266, 267, 270, 273, 275, 277, 665
- undergeneralization, 472, 665
- variabilization, 193, 201, 202, 228, 666
- version spaces, 20, 147, 469, 470, 473, 477, 478, 481, 482, 483, 484, 485, 486, 487, 488, 489, 491, 493, 494, 495, 496, 497, 498, 504, 505, 506, 507, 627, 641, 646, 648, 662, 666
- visual perception, 520, 652
- visuospatial reasoning, xx, 21, 22, 666
- weak methods, 97, 666
- working memory, 111, 120, 653, 667
- zeroth-order logic. *See* propositional logic